REALTHREAD / TECHNICAL DOCUMENTATION

Titan-Board SDK

Technical Documentation

The Titan Board, developed by RT-Thread, is built on Renesas' R7KA8P1 chip featuring a dual-core architecture of Cortex-M85 and Cortex-M33. It offers engineers a flexible and comprehensive development platform, empowering developers to gain deeper expertise in the embedded IoT field.

Document version **1.0.0**

Last updated 2026-10-30

Copyright (c) 2006-2025, RT-Thread Development Team

Contents

Getting Started

1. Titan Board BSP Description . . . 1
   - 1.1 Introduction . . . 1
   - 1.2 Development Board Introduction . . . 1
   - 1.3 Peripheral Support . . . 2
   - 1.4 User Guide . . . 3
   - 1.5 Contact Information . . . 10
   - 1.6 Contribute Code . . . 10

Basics

1. RGB Usage Instructions . . . 11
   - 1.1 Introduction . . . 11
   - 1.2 GPIO Introduction . . . 11
   - 1.3 Hardware Description . . . 12
   - 1.4 Software Description . . . 13
   - 1.5 Compilation & Download . . . 15
   - 1.6 Run Effect . . . 15
2. Buzzer Usage Instructions . . . 16
   - 2.1 Introduction . . . 16
   - 2.2 Introduction to PWM / GPT . . . 16
   - 2.3 Hardware Description . . . 17
   - 2.4 FSP Configuration . . . 18
   - 2.5 RT-Thread Settings Configuration . . . 19
   - 2.6 Example Project Instructions . . . 19
   - 2.7 Compilation & Download . . . 21
   - 2.8 Running Results . . . 22
3. Button Interrupt Usage Instructions . . . 23
   - 3.1 Introduction . . . 23
   - 3.2 RA8 Series MCU Interrupt System Overview . . . 23
   - 3.3 Hardware Description . . . 24
   - 3.4 FSP Configuration . . . 25
   - 3.5 Example Code Description . . . 27
   - 3.6 Compilation & Download . . . 28
   - 3.7 Run Effect . . . 29

Drivers

1. ADC Application Example Usage Instructions . . . 30
   - 1.1 Introduction . . . 30
   - 1.2 RA8P1 ADC Features . . . 30
   - 1.3 RT-Thread ADC Driver Framework . . . 32
   - 1.4 Hardware Description . . . 34
   - 1.5 FSP Configuration . . . 34
   - 1.6 RT-Thread Settings Configuration . . . 36
   - 1.7 Example Code Description . . . 37
   - 1.8 Compilation & Download . . . 38
   - 1.9 Run Effect . . . 38
2. CANFD Usage Instructions . . . 39

2.1 Introduction . . . 39
2.2 CAN and CAN FD Protocol Overview . . . 39
2.3 RA8 Series CANFD Features . . . 42
2.4 RT-Thread CAN Device Interface . . . 44
2.5 Hardware Description . . . 46
2.6 FSP Configuration . . . 46
2.7 RT-Thread Settings Configuration . . . 49
2.8 Example Code Description . . . 49
2.9 Compilation & Download . . . 53
2.10 Run Effect . . . 53
3. Ethernet Usage Guide . . . 54
3.1 Introduction . . . 54
3.2 Ethernet Introduction . . . 54
3.3 RA8 Series Ethernet Features . . . 56
3.4 Hardware Description . . . 59
3.5 FSP Configuration . . . 60
3.6 RT-Thread Settings Configuration . . . 63
3.7 Software Overview . . . 63
3.8 Build & Download . . . 64
3.9 Running Result . . . 65
4. GPT Usage Guide . . . 67
4.1 Introduction . . . 67
4.2 PWM Introduction . . . 67
4.3 Timer Introduction . . . 68
4.4 RA8 Series GPT Module Overview . . . 70
4.5 RT-Thread PWM Framework Introduction . . . 71
4.6 RT-Thread Hardware Timer Framework Introduction . . . 72
4.7 Hardware Description . . . 74
4.8 FSP Configuration . . . 74
4.9 RT-Thread Settings Configuration . . . 76
4.10 Example Project Instructions . . . 76
4.11 Compilation & Download . . . 79
4.12 Running Results . . . 80
5. HyperRAM Usage Guide . . . 81
5.1 Introduction . . . 81
5.2 HyperRAM Overview . . . 81
5.3 RA8 Series OSPI (Octa-SPI) Features . . . 83
5.4 Hardware Description . . . 84
5.5 FSP Configuration . . . 85
5.6 RT-Thread Settings Configuration . . . 89
5.7 Software Overview . . . 90
5.8 Build & Download . . . 94
5.9 Running Result . . . 94
6. IST8310 Example Usage Instructions . . . 96
6.1 Introduction . . . 96
6.2 IST8310 Magnetometer Introduction . . . 96
6.3 RA8 Series I2C Master Features . . . 97
6.4 RT-Thread I2C Framework Introduction . . . 100
6.5 Hardware Description . . . 101
6.6 FSP Configuration . . . 102
6.7 RT-Thread Settings Configuration . . . 103
6.8 Example Project Description . . . 103
6.9 Compilation & Download . . . 105

6.10 Run Effect . . . 105
7. MIPI LCD Usage Instructions . . . 106
7.1 Introduction . . . 106
7.2 RA8 Series GLCDC Module . . . 106
7.3 Hardware Description . . . 108
7.4 FSP Configuration . . . 111
7.5 RT-Thread Settings Configuration . . . 118
7.6 Example Code Description . . . 118
7.7 Compilation & Download . . . 119
7.8 Run Effect . . . 120
8. RGB LCD Usage Instructions . . . 121
8.1 Introduction . . . 121
8.2 RA8 Series GLCDC Module . . . 121
8.3 Hardware Description . . . 124
8.4 FSP Configuration . . . 125
8.5 RT-Thread Settings Configuration . . . 132
8.6 Example Code Description . . . 133
8.7 Compilation & Download . . . 134
8.8 Run Effect . . . 134
9. RS485 Driver Usage Instructions . . . 136
9.1 Introduction . . . 136
9.2 RS-485 Overview . . . 136
9.3 RA8 Series UART Module Overview . . . 138
9.4 RT-Thread UART v2 Driver Framework . . . 139
9.5 Hardware Description . . . 141
9.6 FSP Configuration . . . 141
9.7 RT-Thread Settings Configuration . . . 143
9.8 Example Code Description . . . 143
9.9 Build & Download . . . 146
9.10 Running Result . . . 146
10. RTC and Alarm Example Usage Instructions . . . 148
10.1 Introduction . . . 148
10.2 RTC (Real-Time Clock) Overview . . . 148
10.3 RA8 Series RTC Module Overview . . . 150
10.4 RT-Thread RTC Driver Framework . . . 151
10.5 Hardware Description . . . 154
10.6 FSP Configuration . . . 154
10.7 RT-Thread Settings Configuration . . . 155
10.8 Example Code Description . . . 156
10.9 Compilation & Download . . . 158
10.10 Run Effect . . . 158
10.11 Extension: RTC Operation During Power Loss . . . 158
11. SD Card File System Usage Instructions . . . 161
11.1 Introduction . . . 161
11.2 Hardware Description . . . 165
11.3 FSP Configuration . . . 166
11.4 RT-Thread Settings Configuration . . . 167
11.5 Example Project Description . . . 167
11.6 Compilation & Download . . . 170
11.7 Run Effect . . . 170
12. BMI088 Gyroscope Usage Instructions . . . 172
12.1 Introduction . . . 172
12.2 Overview of BMI088 . . . 172

12.3 RA8 Series SPI Module (r_spi_b) Instructions . . . 174
12.4 RT-Thread SPI Driver Framework . . . 176
12.5 Hardware Description . . . 178
12.6 FSP Configuration . . . 178
12.7 RT-Thread Settings Configuration . . . 179
12.8 Software Description . . . 180
12.9 Compilation & Download . . . 184
12.10 Run Effect . . . 184
13. USB-HHID Usage Instructions . . . 186
13.1 Introduction . . . 186
13.2 USB HHID Overview . . . 186
13.3 RA8 USB HHID Module Overview . . . 187
13.4 Hardware Description . . . 188
13.5 FSP Configuration . . . 189
13.6 RT-Thread Settings Configuration . . . 190
13.7 Compilation & Download . . . 192
13.8 Running Results . . . 192
14. USB-PCDC Usage Instructions . . . 193
14.1 Introduction . . . 193
14.2 USB PCDC Instructions . . . 193
14.3 RA8 Series USB PCDC Module (r_usb_pcdc) Overview . . . 194
14.4 Hardware Description . . . 196
14.5 FSP Configuration Configuration . . . 197
14.6 RT-Thread Settings Configuration . . . 199
14.7 Compilation & Download . . . 200
14.8 Running Effect . . . 200
15. WDT Usage Instructions . . . 202
15.1 Introduction . . . 202
15.2 WDT (Watchdog Timer) Instruction . . . 202
15.3 RA8 Series WDT Module (r_wdt) Instruction . . . 203
15.4 RT-Thread WDT Framework Instruction . . . 204
15.5 Hardware Description . . . 206
15.6 FSP Configuration . . . 206
15.7 RT-Thread Settings Configuration . . . 207
15.8 Example project description . . . 207
15.9 Compilation & Download . . . 209
15.10 Run Effects . . . 209
16. Wi-Fi Usage Instructions . . . 211
16.1 Introduction . . . 211
16.2 WiFi Introduction . . . 211
16.3 RA8 Series SDHI Module Introduction . . . 213
16.4 Hardware Description . . . 215
16.5 FSP Configuration . . . 215
16.6 RT-Thread Settings Configuration . . . 217
16.7 Compilation & Download . . . 218
16.8 Run Effect . . . 219

Components

1. Flash File System Usage Instructions . . . 224
   - 1.1 Introduction . . . 224
   - 1.2 RA8 Series OSPI Features . . . 224
   - 1.3 Abstraction Layer . . . 225
   - 1.4 Hardware Description . . . 226
   - 1.5 FSP Configuration . . . 226
   - 1.6 RT-Thread Settings Configuration . . . 230
   - 1.7 Software Overview . . . 230
   - 1.8 Build & Download . . . 233
   - 1.9 Running Result . . . 233
2. Netutils Usage Instructions . . . 234
   - 2.1 Instruction . . . 234
   - 2.2 Hardware Description . . . 234
   - 2.3 FSP Configuration . . . 234
   - 2.4 RT-Thread Settings Configuration . . . 236
   - 2.5 Software Description . . . 236
   - 2.6 Compilation & Download . . . 238
   - 2.7 Run Effect . . . 238

Multimedia Display

1. CEU Camera Usage Instructions . . . 245
   - 1.1 Introduction . . . 245
   - 1.2 RA8 Series CEU (Camera Engine Unit) Features . . . 245
   - 1.3 RA8 Series MCU GLCDC (Graphics LCD Controller) Features . . . 247
   - 1.4 Hardware Description . . . 250
   - 1.5 FSP Configuration . . . 252
   - 1.6 RT-Thread Settings Configuration . . . 262
   - 1.7 Compilation & Download . . . 263
   - 1.8 Run Effect . . . 263
2. MIPI CSI Camera Usage Instructions . . . 264
   - 2.1 Introduction . . . 264
   - 2.2 RA8 Series MIPI CSI Features . . . 264
   - 2.3 RA8 Series MCU GLCDC (Graphics LCD Controller) Features . . . 266
   - 2.4 Hardware Description . . . 268
   - 2.5 FSP Configuration . . . 270
   - 2.6 RT-Thread Settings Configuration . . . 283
   - 2.7 Compilation & Download . . . 283
   - 2.8 Run Effect . . . 284
3. CEU Camera Usage Instructions(MIPI LCD) . . . 285
   - 3.1 Introduction . . . 285
   - 3.2 RA8 Series CEU (Camera Engine Unit) Features . . . 285
   - 3.3 RA8 Series GLCDC Module . . . 287
   - 3.4 Hardware Description . . . 290
   - 3.5 FSP Configuration . . . 295
   - 3.6 RT-Thread Settings Configuration . . . 303
   - 3.7 Compilation & Download . . . 304
   - 3.8 Run Effect . . . 304
4. MIPI DSI LVGL Usage Instructions . . . 306
   - 4.1 Instruction . . . 306
   - 4.2 LVGL Instruction . . . 306
   - 4.3 Hardware Description . . . 308
   - 4.4 FSP Configuration . . . 311

4.5 RT-Thread Settings Configuration . . . 318
4.6 Software Description . . . 318
4.7 Compilation & Download . . . 320
4.8 Run Effect . . . 320
5. RGB565 LVGL Usage Instructions . . . 322
5.1 Instruction . . . 322
5.2 LVGL Instruction . . . 322
5.3 Hardware Description . . . 324
5.4 FSP Configuration . . . 326
5.5 RT-Thread Settings Configuration . . . 336
5.6 Software Description . . . 337
5.7 Compilation & Download . . . 342
5.8 Run Effect . . . 342

Multicore Communication

1. Dual-core Project Instructions (Core0) . . . 343
1.1 Introduction . . . 343
1.2 Hardware Description . . . 343
1.3 FSP Configuration . . . 344
1.4 RT-Thread Settings Configuration . . . 346
1.5 Example Code Description . . . 346
1.6 Compilation & Download . . . 347
1.7 Run Effect . . . 350
1.8 Notes on Using Dual-Core Projects . . . 351
2. Dual-core Project Instructions (Core1) . . . 352
2.1 Introduction . . . 352
2.2 Hardware Description . . . 352
2.3 FSP Configuration . . . 353
2.4 RT-Thread Settings Configuration . . . 354
2.5 Example Code Description . . . 354
2.6 Compilation & Download . . . 355
2.7 Run Effect . . . 358
2.8 Notes on Using Dual-Core Projects . . . 359
3. RPMsg-Lite Usage Instructions（Core0 Master） . . . 360
3.1 Introduction . . . 360
3.2 RPMsg-Lite Instructions . . . 360
3.3 Hardware Description . . . 362
3.4 FSP Configuration . . . 362
3.5 RT-Thread Settings Configuration . . . 363
3.6 Example Code Description . . . 363
3.7 Compilation & Download . . . 363
3.8 Run Effect . . . 366
3.9 Notes on Using Dual-Core Projects . . . 368
4. RPMsg-Lite Usage Instructions（Core0 Master） . . . 370
4.1 Introduction . . . 370
4.2 RPMsg-Lite Instructions . . . 370
4.3 Hardware Description . . . 372
4.4 FSP Configuration . . . 372
4.5 RT-Thread Settings Configuration . . . 373
4.6 Example Code Description . . . 373
4.7 Compilation & Download . . . 373
4.8 Run Effect . . . 376
4.9 Notes on Using Dual-Core Projects . . . 378

NPU Intelligent Vision

1. Titan Board OpenMV . . . 380
   1.1 Hardware . . . 380
   1.2 Build and download . . . 380
   1.3 Run an example . . . 381
   1.4 Examples and dependencies . . . 381
2. NPU Accelerated Face Detection Instructions . . . 382
   2.1 Instructions . . . 382
   2.2 Overall system architecture . . . 382
   2.3 Arm® Ethos™-U55 NPU Features . . . 382
   2.4 YOLO-Fastest Model Introduction . . . 383
   2.5 Hardware Description . . . 383
   2.6 FSP Configuration . . . 385
   2.7 RT-Thread Settings Configuration . . . 394
   2.8 Build & Download . . . 394
   2.9 Run Effect . . . 394

Factory Tests

1. Production Test Program User Guide . . . 396
   1.1 Test Preparation . . . 396
   1.2 Hardware Connections . . . 396
   1.3 LED Indicators . . . 401
   1.4 Program Execution Flow . . . 402
   1.5 Test Result Description . . . 407

FAQ

1. Frequently Asked Questions (FAQ) . . . 409
   1.1 Firmware Programming . . . 409
   1.2 Camera and Display Connection . . . 420
   1.3 Runtime Issues . . . 423

Getting Started

1. Titan Board BSP Description

English | Chinese

1.1 Introduction

This document provides the BSP (Board Support Package) description for the RT-Thread **Titan Board** development board. By reading the *Quick Start* section, developers can quickly get started with this BSP and run RT-Thread on the development board.

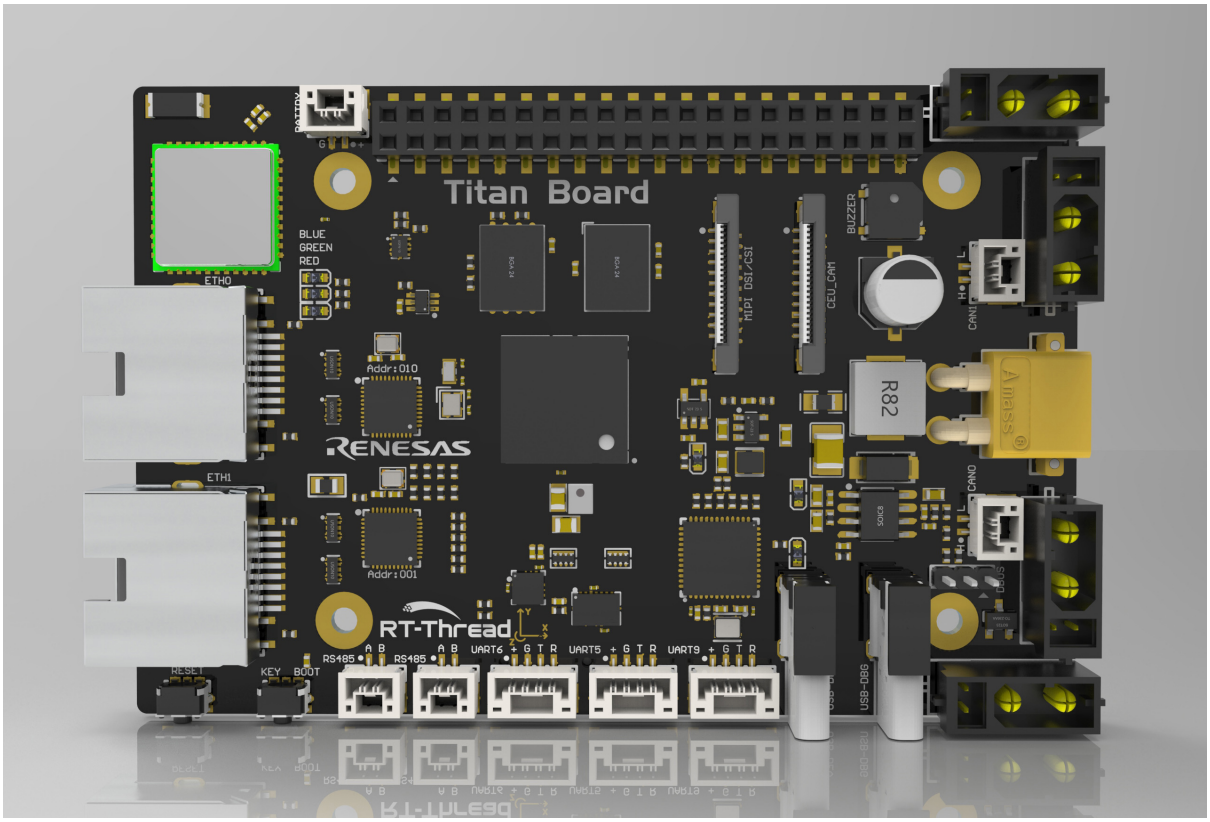
The main contents are as follows:

- Introduction to the development board
- BSP Quick Start Guide

1.2 Development Board Introduction

The **Titan Board** development board is based on the Renesas RA8P1 and is designed for embedded system application development through flexible configuration of software packages and IDEs.

The front view of the development board is shown below:



The commonly used **on-board resources** of this development board are as follows:

- **MCU**: R7KA8P1KFLCAC, high-performance dual-core with 1GHz Arm® Cortex®-M85 and 250MHz Arm® Cortex®-M33, 1MB code MRAM, and 2MB ECC-enabled SRAM.
- **Debug interface**: On-board DAP-Link interface
- **Board interfaces**: 40-pin Raspberry Pi interface, MIPI DSI/CSI, CEU Camera, RGB LCD interface, 2 Ethernet interfaces, 4 CAN motor interfaces, 2 RS485 interfaces, 3 UARTs, 1 DBUS interface, 1 USBHS/USBFS interface, 1 RTC battery interface.

1.3 Peripheral Support

The current peripheral support status in this BSP is as follows:

On-chip Peripheral	**Support Status**	**Component**	**Support Status**
UART	Supported	LWIP	Supported
GPIO	Supported	TCP/UDP	Supported
HWTIMER	Supported	MQTT	Supported
I2C	Supported	TFTP	Supported
WDT	Supported	Telnet	Supported
RTC	Supported	**Multicore Communication**	**Support Status**
ADC	Supported	RPMsg-Lite	Supported
DAC	Supported	**Extended peripheral**	**Support Status**
SPI	Supported	MIPI CSI Camera	Supported
RS485	Supported	CEU Camera	Supported
CANFD	Supported	RGB LCD	Supported
SDHI	Supported	CYW43438 WIFI	Supported
USB	Supported		
HyperRAM	Supported		
HyperFlash	Supported		

1.4 User Guide

The user guide is divided into the following two sections:

- **Quick Start**

  This section is intended for beginners who are just getting started with RT-Thread. By following simple steps, you can run the RT-Thread operating system on this development board and observe the experimental results.

- **Advanced Usage**

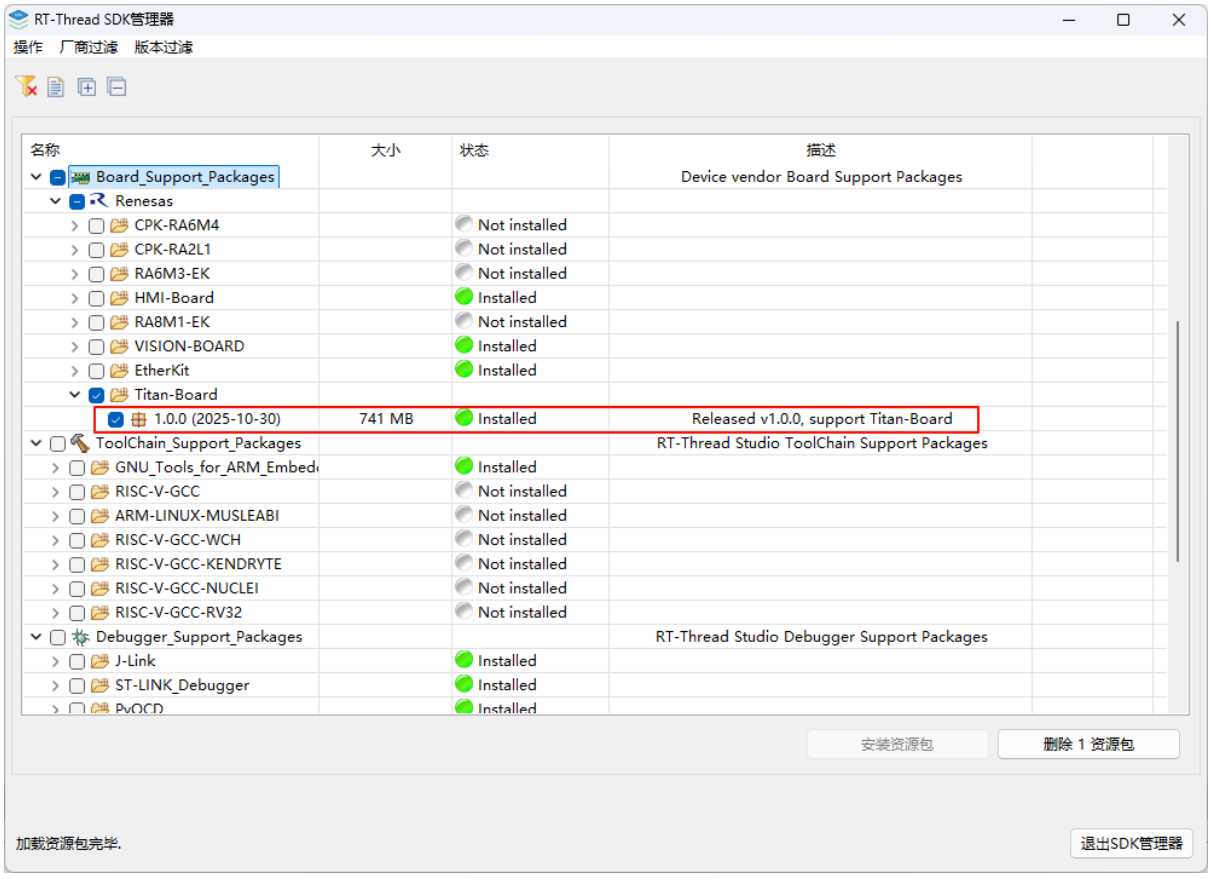
  This section is intended for developers who need to use more board resources on the RT-Thread operating system. By using the FSP and RT-Thread Settings tools to configure the project, more on-board resources can be enabled to achieve advanced functionality.

Quick Start

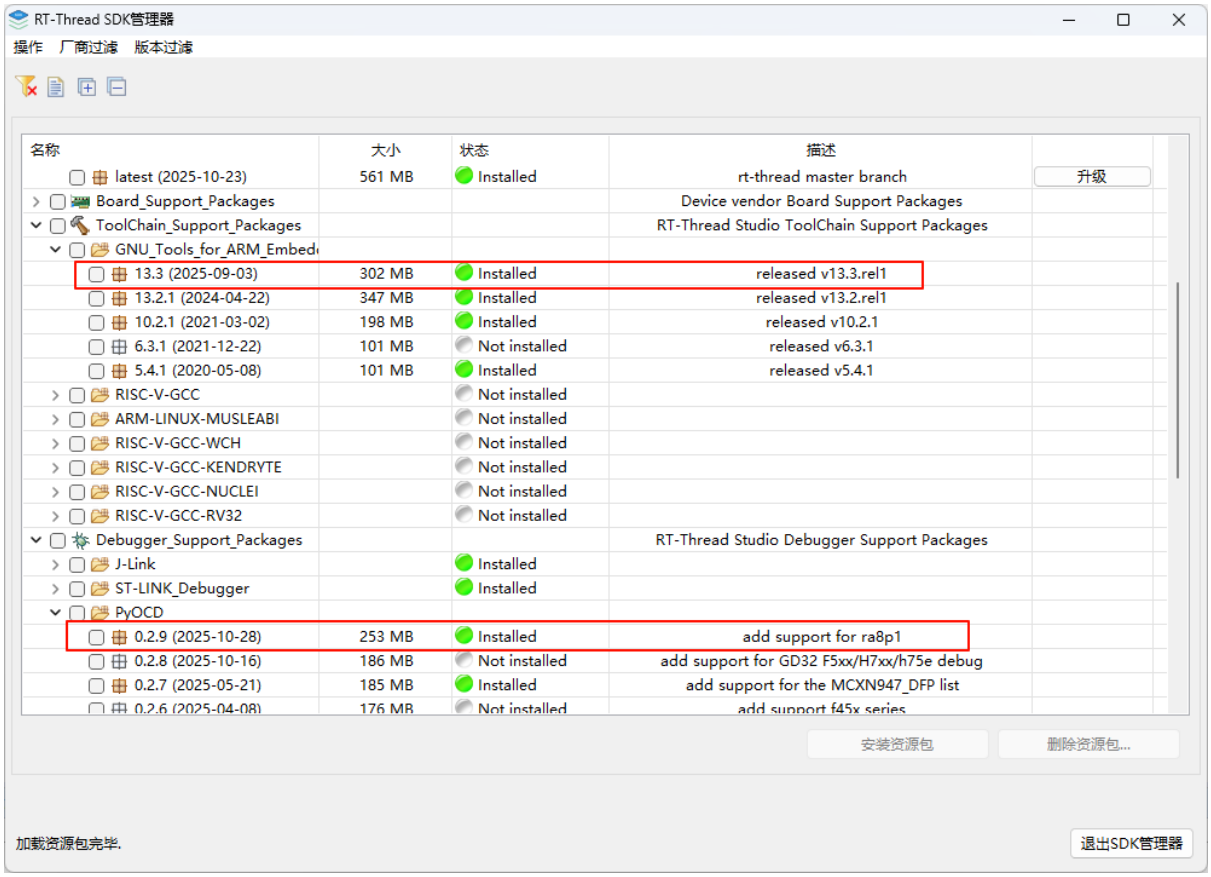
This BSP currently provides an RT-Thread Studio project. Below, we use the RT-Thread Studio development environment as an example to illustrate how to get the system running.

Create a Project

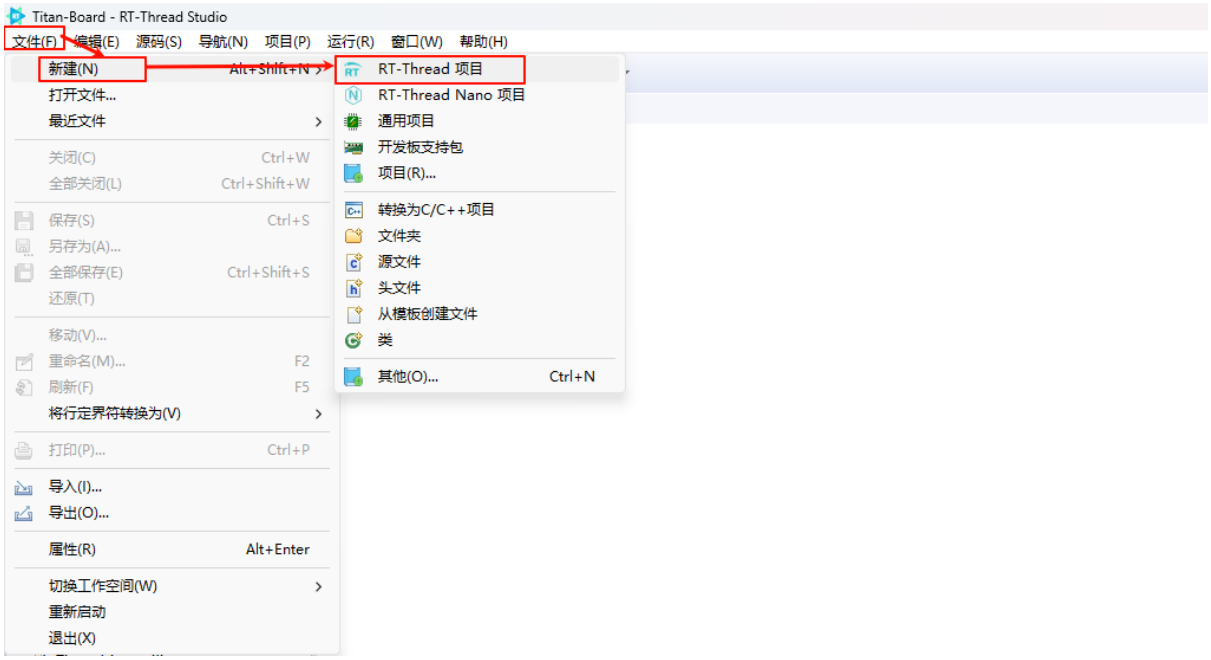
- Download the **Titan Board SDK** resource package.

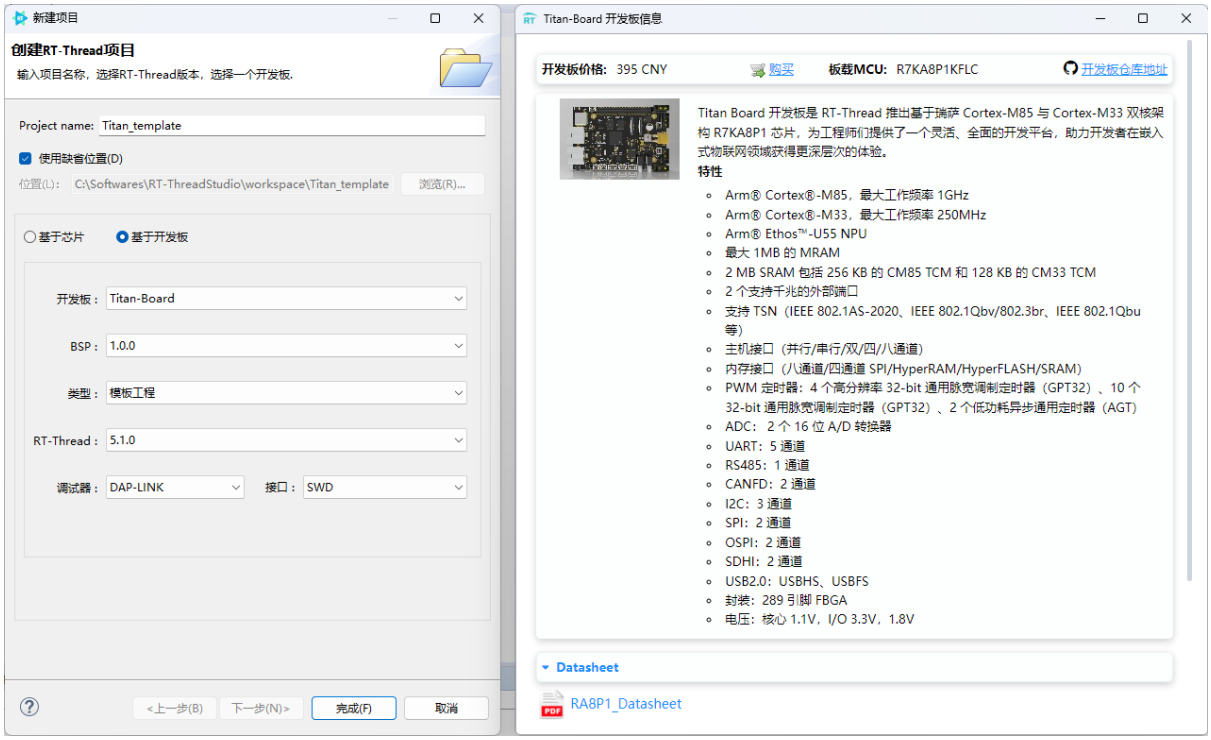


- Download the compilation toolchain and debugging tools.



- Create a new RT-Thread project.

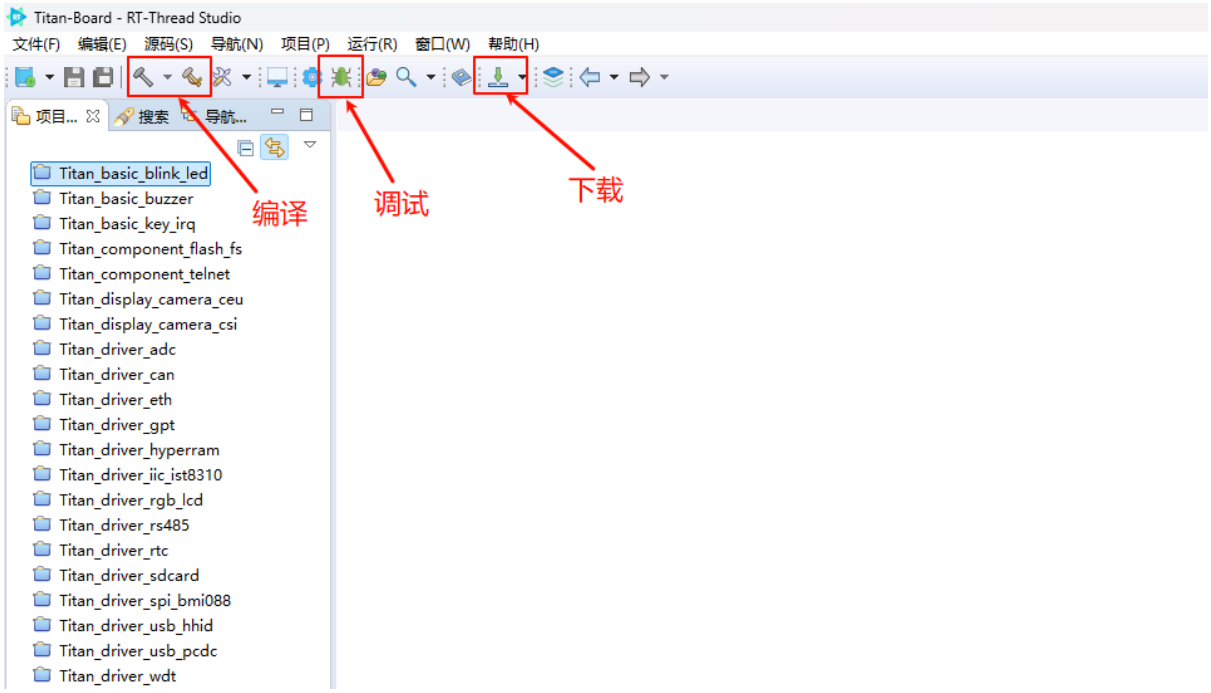




Hardware Connection

Use a USB cable to connect the development board to the PC, and use the DAP-Link interface to download and debug the program.

Build and Download



View Running Results

After the program is successfully downloaded, the system will automatically run and print system information.

Connect the development board’ s corresponding serial port to the PC, open the corresponding serial port in a terminal tool (115200-8-1-N), and reset the device. You will then see the RT-Thread output information. Enter the help command to view the commands supported in the system.

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Aug 5 2025 17:24:30
 2006 - 2024 Copyright by RT-Thread team

================================================
Hello, Titan Board!
================================================
msh >ps
thread       pri  status      sp     stack size max used left tick   error  tcb
  addr
------------ ---  ------- ---------- ----------  ------  ---------- ------- -----
  -----
tshell        20  running 0x000000cc 0x00001000    16%   0x0000000a OK
  0x22022568
sys workq     23  suspend 0x000000c4 0x00000800    11%   0x0000000a OK
  0x22021a60
tidle0        31  ready   0x000000a4 0x00000100    67%   0x00000011 OK
  0x220203c8
timer          4  suspend 0x000000a4 0x00000200    38%   0x00000009 EINTRPT
  0x22020818
main          10  suspend 0x000000dc 0x00000800    18%   0x00000007 EINTRPT
  0x22021128
msh >
```

Application Entry Function

The entry function of the application layer is located in **src\hal_entry.c** within void hal_entry(void). User-created source files can be placed directly in the **src** directory.

```
void hal_entry(void)
{
    rt_kprintf("\n================================================\n");
    rt_kprintf("Hello, Titan Board!\n");
    rt_kprintf("================================================\n");

    while (1)
    {
```

(continues on next page)

(continued from previous page)

```
        rt_pin_write(LED_PIN_0, PIN_HIGH);
        rt_thread_mdelay(1000);
        rt_pin_write(LED_PIN_0, PIN_LOW);
        rt_thread_mdelay(1000);
    }
}
```

The FSP of the Titan_template project has already pre-configured the following peripherals. Users can refer to the FSP configuration instructions below to generate their own setup.

- Uart8、Uart5、Uart9
- HyperFlash
- HyperRAM
- timer0、timer1
- SD Card
- SDIO WiFi
- I2C0、I2C2
- SPI0
- MIPI CSI
- RGB565 LCD
- ETH1
- pwm2、pwm7、pwm10、pwm12
- CAN0、CAN1
- RTC
- ADC0

Advanced Usage

Resources and Documentation

- Development Board Official Homepage
- Development Board Datasheet
- Development Board Hardware Manual
- Dualcore Development Guide
- Renesas RA8P1 Group

FSP Configuration

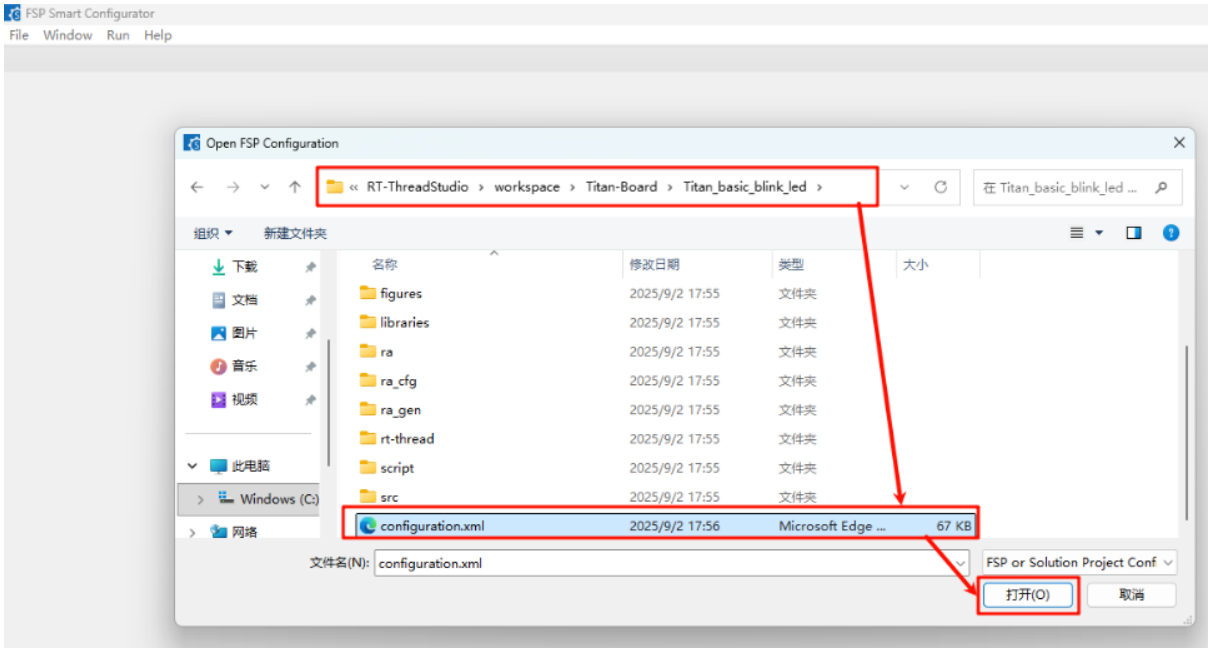
If you need to modify the Renesas BSP peripheral configuration or add new peripheral interfaces, you will need to use the Renesas Flexible Software Package (FSP) configuration tool. Please make sure to follow the steps below for configuration. If you encounter any issues, you may ask questions in the RT-Thread Community Forum.

1. Download Flexible Software Package (FSP) | Renesas, please use **FSP version 6.0.0**
2. Refer to the documentation: Configuring Peripheral Drivers Using FSP for the RA Series.

- **Method 1: Configure development by importing FSP:**

  Users can locate the configuration.xml file in the project and import it into FSP to start configuration:

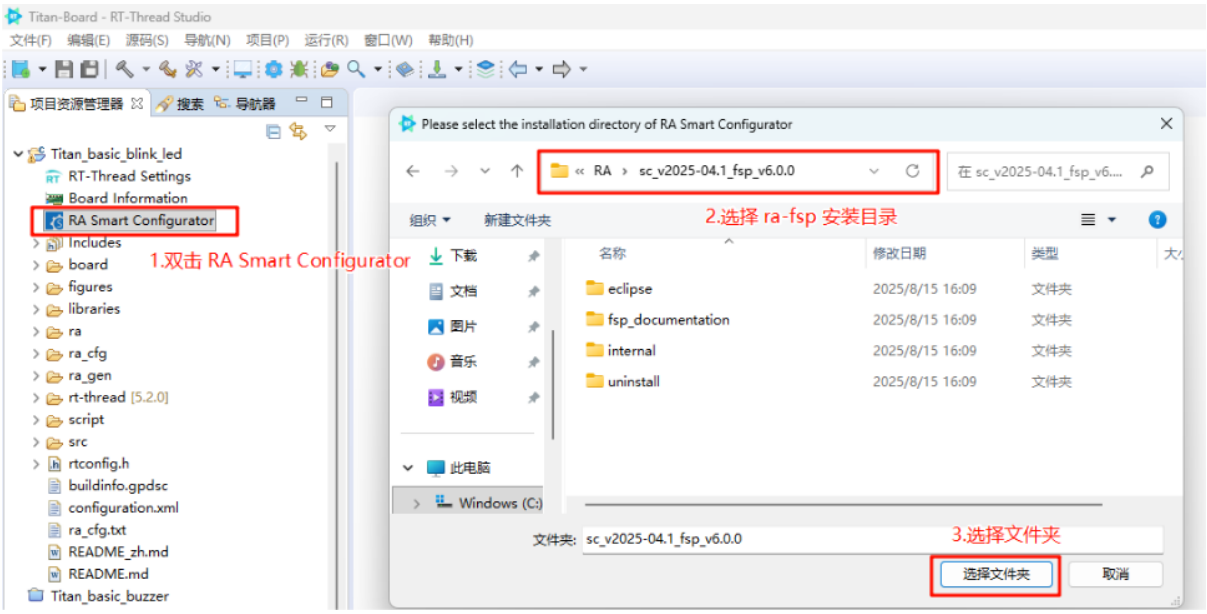
  Select **File** → **Open** at the top-left corner to open the configuration file.



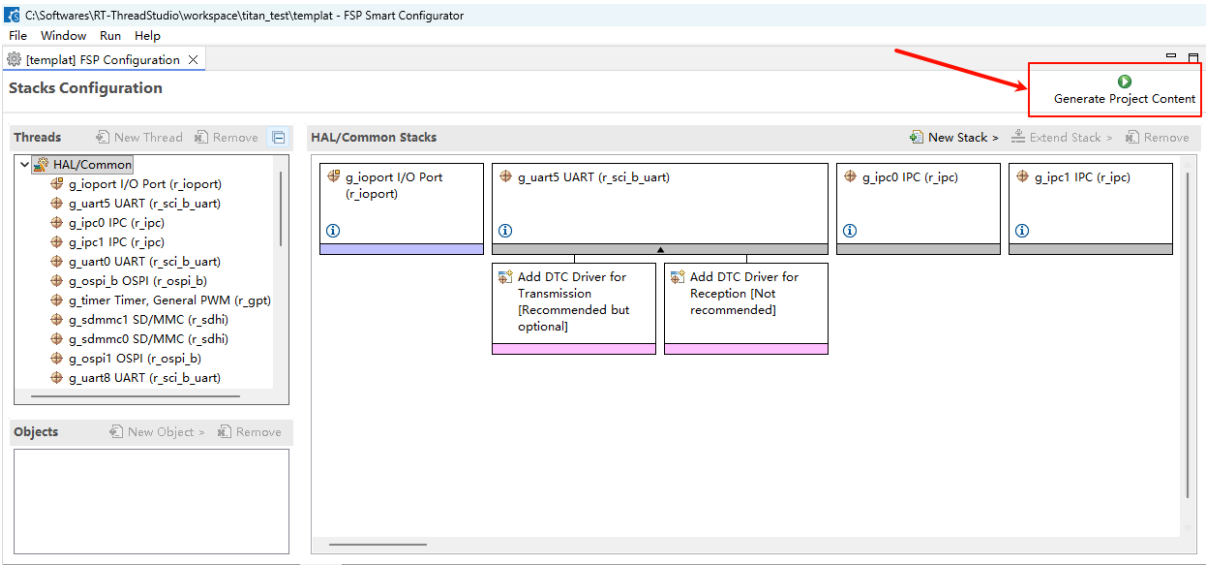
- **Method 2: Configure FSP using built-in commands in RT-Thread Studio:**

After installing RA-FSP, first create a **Titan Board** project in Studio. Under the project, you will see the built-in Renesas FSP configuration option: **RA Smart Configurator**.

Double-click this option, then locate the root directory where RA-FSP was installed, and set this directory as the FSP installation path.

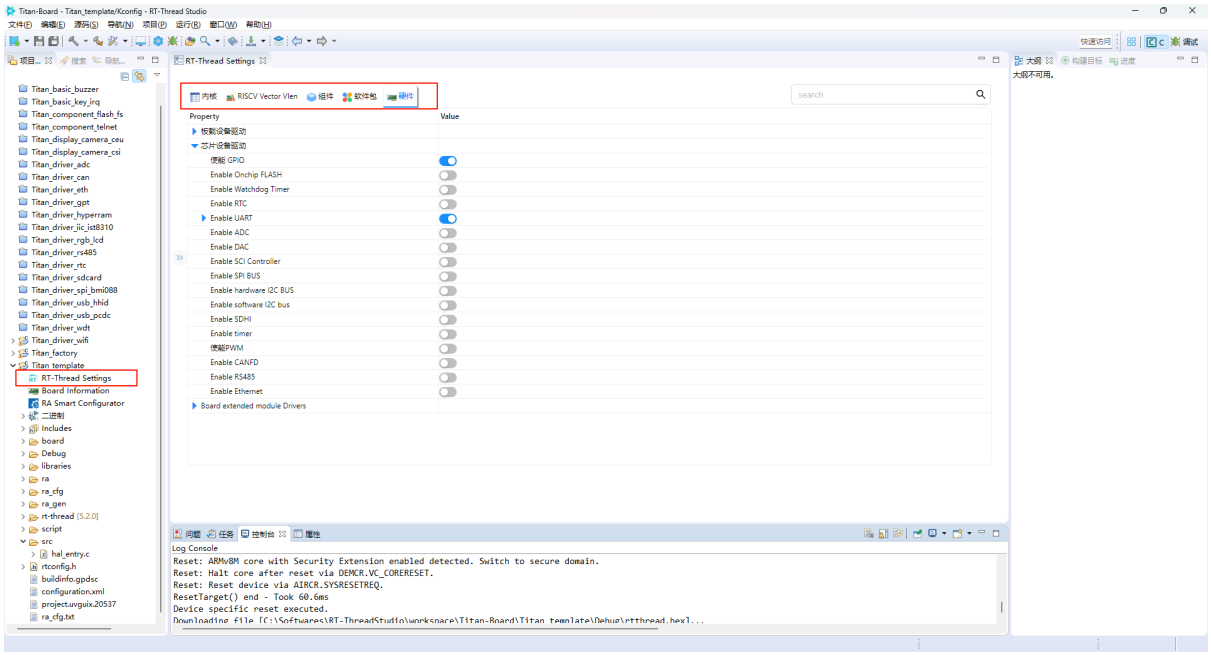


- **Generate FSP Code:**



RT-Thread Settings

In **RT-Thread Settings**, you can configure the RT-Thread kernel, components, software packages, and Titan Board device drivers.



1.5 Contact Information

If you have any thoughts or suggestions during usage, please feel free to contact us via the RT-Thread Community Forum.

1.6 Contribute Code

If you're interested in Titan Board and have some exciting projects you'd like to share, we welcome code contributions. Please refer to How to Contribute to RT-Thread Code.

Basics

1. RGB Usage Instructions

English|Chinese

1.1 Introduction

This example is the first and simplest example in the SDK, similar to the "Hello World" program that programmers encounter as their first program. The main function of this example is to make the onboard RGB-LED blink periodically.

1.2 GPIO Introduction

GPIO (General Purpose Input/Output) is one of the most commonly used peripheral interfaces of an MCU. It can be configured under software control as either **input mode** or **output mode**:

- **Input mode**: Used to read external voltage levels, such as button inputs.
- **Output mode**: Used to control peripheral voltage levels, such as lighting up an LED or driving a buzzer.

Features of RA8 Series GPIO

- **Rich pin resources**: Each port can be independently configured as input/output/multiplexed function.
- **Multi-function multiplexing**: In addition to GPIO functionality, the same pin can also serve as a signal line for peripherals such as UART, SPI, or I2C.
- **Level control**: In output mode, pins can drive either high or low levels; some pins support open-drain mode.
- **Direction control**: Input/output direction can be dynamically switched.

RT-Thread Abstraction for GPIO

RT-Thread provides a **PIN device driver framework**, which offers a unified interface to shield hardware differences:

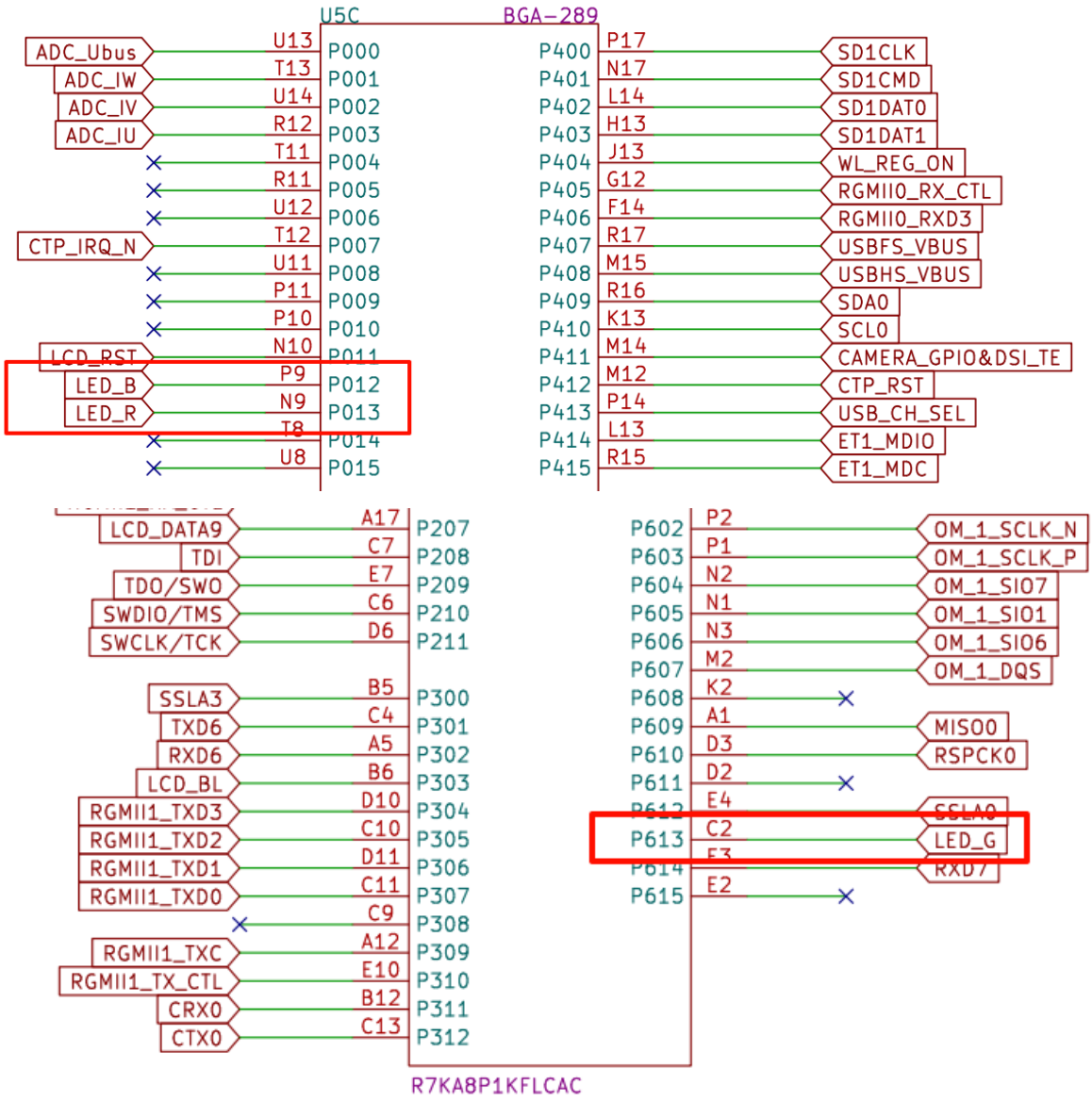
- rt_pin_mode(pin, mode) : Set the working mode of a pin (input/output/pull-up/pull-down, etc.)
- rt_pin_write(pin, value) : Output a voltage level (high/low)
- rt_pin_read(pin) : Read an input voltage level

This way, developers don't need to directly manipulate registers; GPIO control can be achieved through RT-Thread's API.

In this example, the LED pin is configured as **output mode**, and the software continuously toggles between high and low levels to make the LED blink.

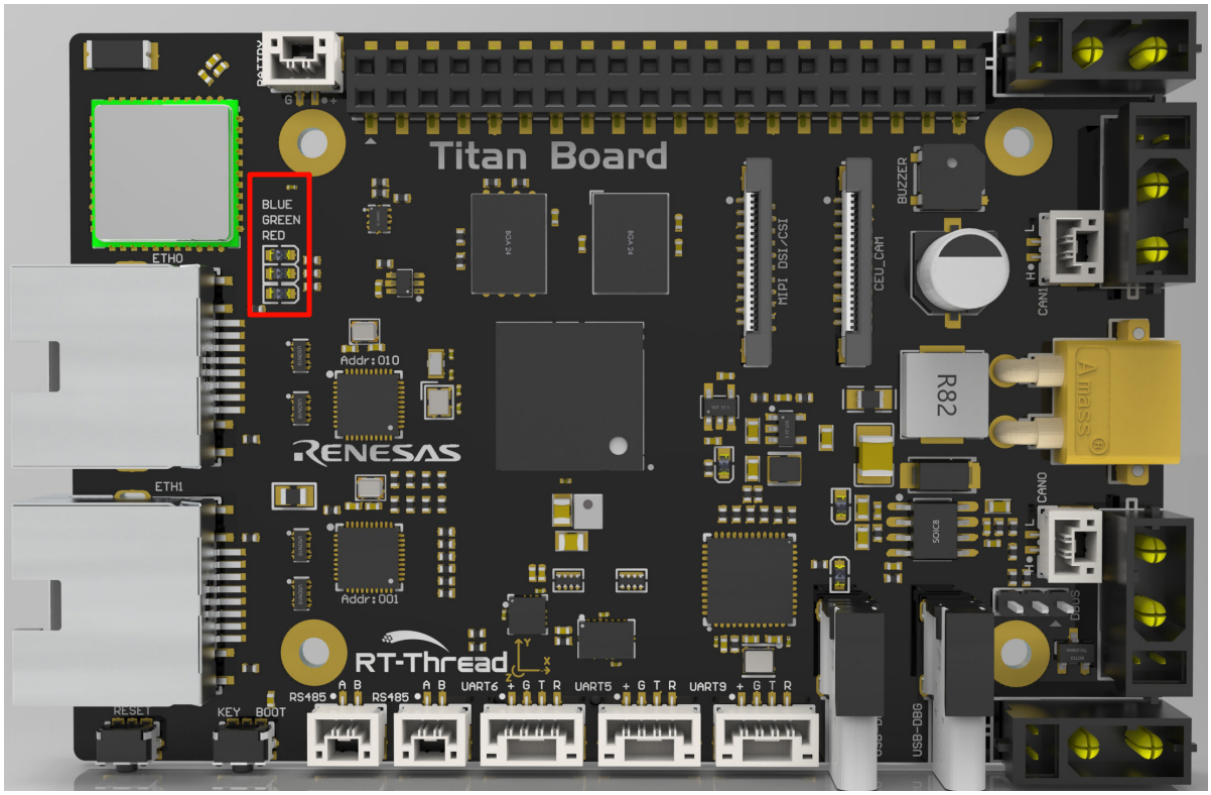
Reference: RT-Thread PIN Device

1.3 Hardware Description



As shown in the diagram above, the Titan Board provides three user LEDs: LED_R (RED), LED_B (BLUE), and LED_G (GREEN). The BLUE LED corresponds to pin P012 on the MCU. To turn on the LED, a low level signal is output from the MCU pin, and to turn it off, a high level signal is output.

The positions of the LEDs on the development board are shown in the diagram below:



1.4 Software Description

The source code for this example is located in /project/Titan_basic_blink_led. The MCU pin definitions for the RGB LEDs and the RGB control logic can be found insrc/hal_entry.c.

```
/* Configure LED pins */
#define LED_PIN_R   BSP_IO_PORT_00_PIN_13
#define LED_PIN_B   BSP_IO_PORT_00_PIN_12
#define LED_PIN_G   BSP_IO_PORT_06_PIN_13

/* Define the LED on/off level */
#define LED_ON  (0)
#define LED_OFF (1)

/* Define eight groups of LED flash tables whose order is R B G */
static const rt_uint8_t _blink_tab[][3] =
{
    {LED_OFF, LED_OFF, LED_OFF},
    {LED_ON,  LED_OFF, LED_OFF},
    {LED_OFF, LED_ON,  LED_OFF},
    {LED_OFF, LED_OFF, LED_ON},
    {LED_ON,  LED_OFF, LED_ON},
    {LED_ON,  LED_ON,  LED_OFF},
    {LED_OFF, LED_ON,  LED_ON},
```

(continues on next page)

(continued from previous page)

```
    {LED_ON,  LED_ON,  LED_ON},
};

void hal_entry(void)
{
    rt_kprintf("\nHello RT-Thread!\n");
    rt_kprintf("====================================================\n");
    rt_kprintf("This example project is a RGB flicker routine!\n\n");
    rt_kprintf("====================================================\n");

    unsigned int count = 0;
    unsigned int group_num = sizeof(_blink_tab)/sizeof(_blink_tab[0]);
    unsigned int group_current;

    /* Set the RGB lamp pin to output mode */
    rt_pin_mode(LED_PIN_R, PIN_MODE_OUTPUT);
    rt_pin_mode(LED_PIN_G, PIN_MODE_OUTPUT);
    rt_pin_mode(LED_PIN_B, PIN_MODE_OUTPUT);
    rt_pin_write(LED_PIN_R, LED_OFF);
    rt_pin_write(LED_PIN_G, LED_OFF);
    rt_pin_write(LED_PIN_B, LED_OFF);

    do
    {
        /* Get the group number */
        group_current = count % group_num;

        /* Control RGB LEDs */
        rt_pin_write(LED_PIN_R, _blink_tab[group_current][0]);
        rt_pin_write(LED_PIN_B, _blink_tab[group_current][1]);
        rt_pin_write(LED_PIN_G, _blink_tab[group_current][2]);

        /* Output LOG information */
        LOG_D("group: %d | red led [%-3.3s] | | blue led [%-3.3s] | | green led [
  %-3.3s]",
            group_current,
            _blink_tab[group_current][0] == LED_ON ? "ON" : "OFF",
            _blink_tab[group_current][1] == LED_ON ? "ON" : "OFF",
            _blink_tab[group_current][2] == LED_ON ? "ON" : "OFF");

        count++;

        /* Delay for a short time */
        rt_thread_mdelay(500);
    }while(count > 0);
}
```

1.5 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio's package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board's USB-DBG interface to the PC and download the firmware to the development board.

1.6 Run Effect

After pressing the reset button to restart the development board, observe the actual effect of the RGB-LED on the development board. When the program is running normally, the RGB LEDs will change periodically.

At this point, you can also open the default serial port of the development board using a terminal tool on the PC, with the baud rate set to 115200N. The board's runtime log information will be output in real-time.

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Jul 30 2025 13:29:07
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
================================================
This example project is an RGB flicker routine!
================================================
[D/main] group: 0 | red led [OFF] | | blue led [OFF] | | green led [OFF]
[D/main] group: 1 | red led [ON ] | | blue led [OFF] | | green led [OFF]
[D/main] group: 2 | red led [OFF] | | blue led [ON ] | | green led [OFF]
[D/main] group: 3 | red led [OFF] | | blue led [OFF] | | green led [ON ]
[D/main] group: 4 | red led [ON ] | | blue led [OFF] | | green led [ON ]
[D/main] group: 5 | red led [ON ] | | blue led [ON ] | | green led [OFF]
[D/main] group: 6 | red led [OFF] | | blue led [ON ] | | green led [ON ]
[D/main] group: 7 | red led [ON ] | | blue led [ON ] | | green led [ON ]
```

2. Buzzer Usage Instructions

English | Chinese

2.1 Introduction

In practical applications, the use of **timers** is almost always indispensable. This example demonstrates how to use the **GPT (General Purpose Timer)** on the Titan Board to generate PWM signals and drive a passive buzzer to play a melody.

The selected melody in this example is the classic children's song "**Two Tigers**". By dynamically adjusting the PWM frequency through the timer, the buzzer produces different musical notes.

Through this example, users will learn:

- How to configure the RA8 GPT module and generate PWM output
- How to use the PWM device driver framework under RT-Thread
- How to synthesize different tones using PWM duty cycle and frequency, and thereby implement simple music playback

2.2 Introduction to PWM / GPT

Passive Buzzer and PWM

- A **passive buzzer** is essentially a small piezoelectric or electromagnetic transducer that requires an **external AC signal (square wave) at a certain frequency** to produce sound.
- The **pitch** is determined by the **PWM frequency**, while the **loudness/timbre** is mainly influenced by the **duty cycle** and driving capability. A **50% duty cycle** is commonly used to produce a pure tone with the strongest fundamental frequency and minimal harmonics.
- If the buzzer has a high rated current (or the onboard GPIO cannot drive it directly), it is recommended to use an **NPN/MOSFET low-side switch + diode** (for electromagnetic types) for protection. Piezoelectric buzzers generally do not require a freewheeling diode but adding a **100–1kΩ resistor** in series is still recommended to limit current and reduce noise (depending on the specific device and board).

Basic Concepts of PWM

- **Period (T):** The total time of one high-low cycle.
- **Frequency (f):** f = 1 / T (in Hz).
- **Duty Cycle (D):** The ratio of high-level duration to the total period, D = t_high / T.
- **Voltage Level:** Usually VCC (e.g., 3.3V).

When using PWM to play music on a buzzer:

- Changing **frequency f** → changes the pitch.
- Changing **duration** → changes the beat and rhythm.

- Fine-tuning **duty cycle D** (typically 40–60%) → affects loudness and timbre.

Overview of RA8 GPT (for PWM)

- **GPT (General PWM Timer):** A general-purpose timer in the RA8 series that supports **PWM output**, typically with A/B channels (as mapped by FSP/BSP pins).
- Typical features: configurable **count source/ divider**, **period/ compare value**, **edge-aligned/ center-aligned modes**, and support for complementary output with dead time (often used for motors/drivers).
- Under **RT-Thread**, GPT can be controlled via the **PWM device driver**: simply set the **period** and **pulse width** to generate PWM.

Setting Frequency and Duty Cycle in RT-Thread

rt_pwm_set(device, channel, period_ns, pulse_ns) uses nanoseconds:

- Period (ns): T_ns = 10^9 / f
- High-level duration (ns): t_high = T_ns * D

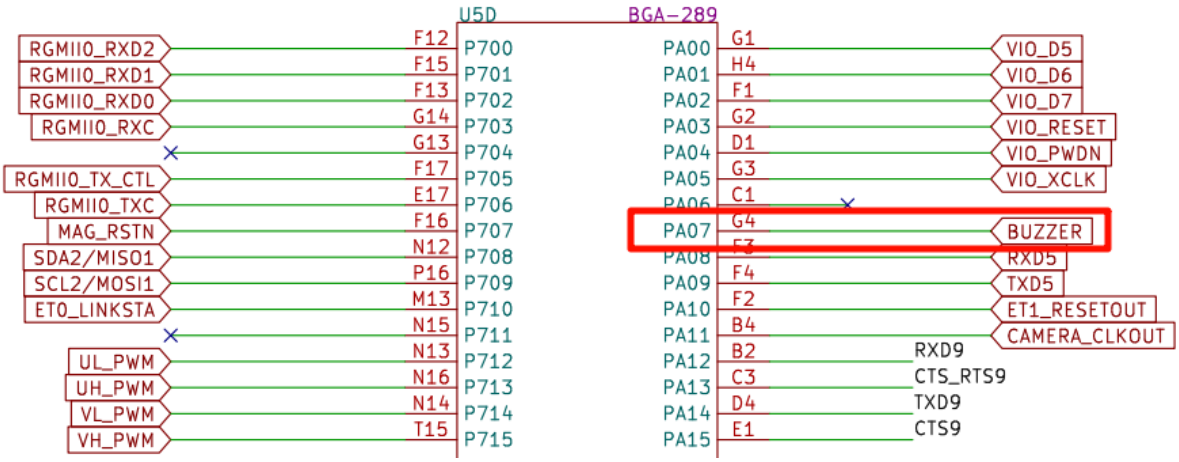
For example, with D = 50%, t_high = T_ns / 2.

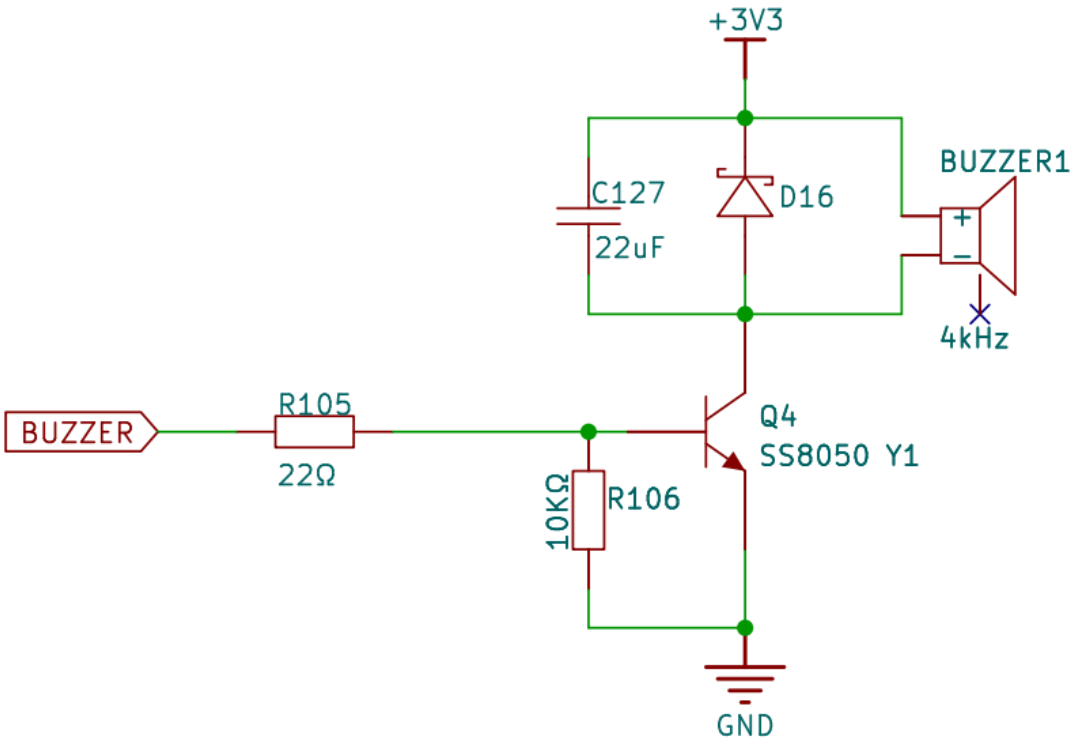
Note: period_ns and pulse_ns are usually 32-bit values. Avoid extremely low frequencies that result in very large periods; for very low frequencies, consider software timers or segmented playback.

Reference: RT-Thread PWM Device

2.3 Hardware Description

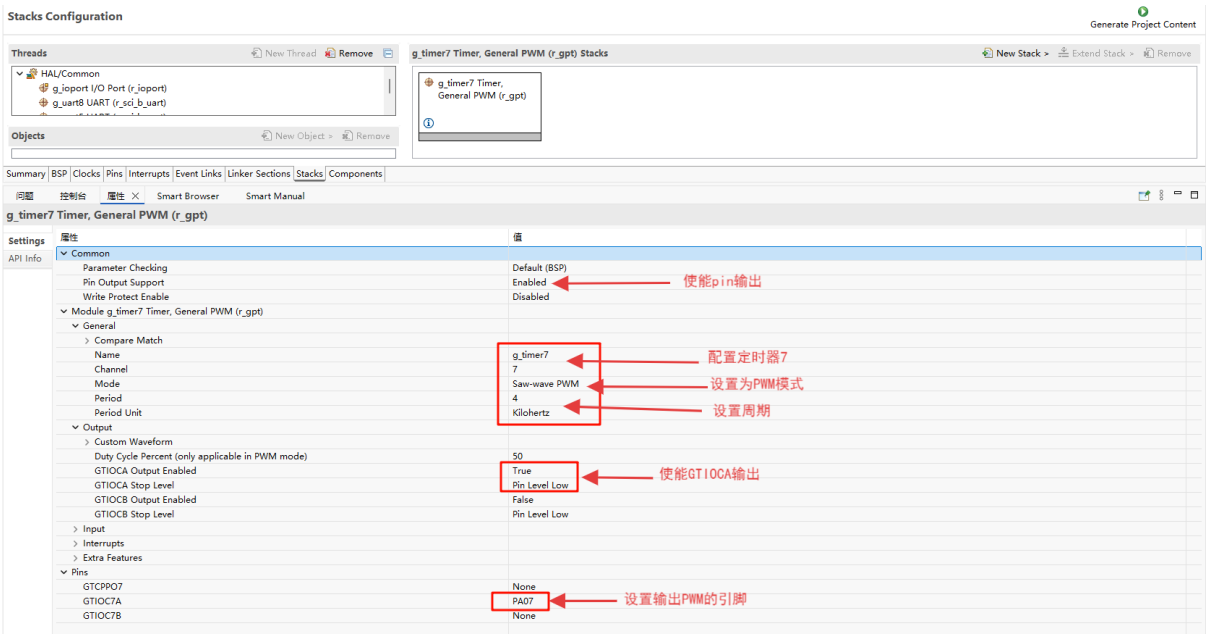
The pin of the buzzer is PA07.



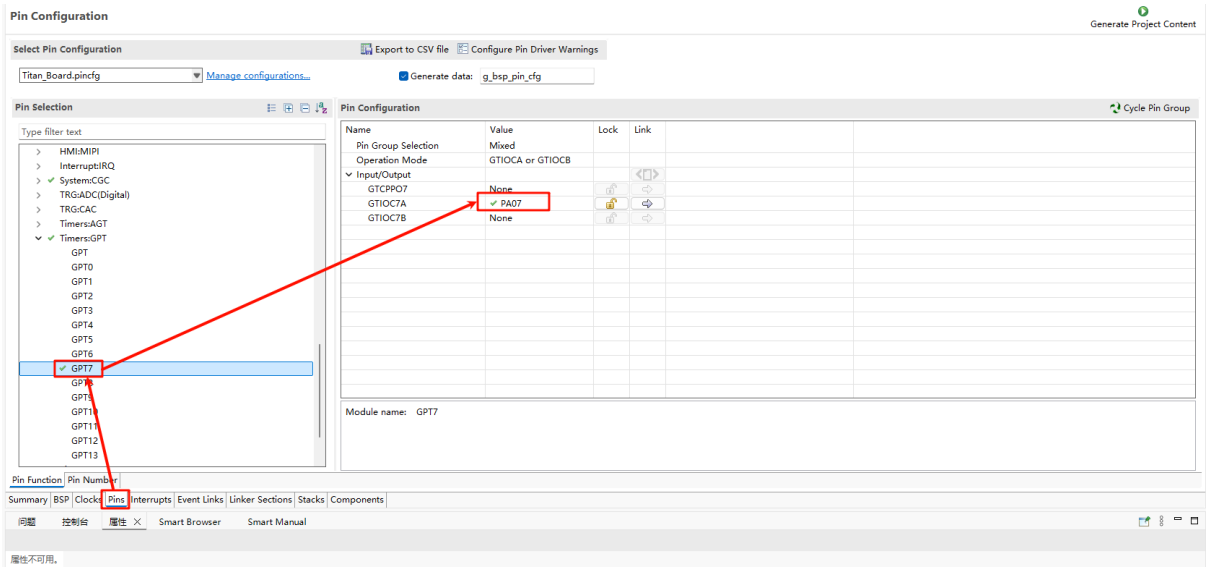


2.4 FSP Configuration

- The FSP is configured to enable GPT7 as a PWM mode:

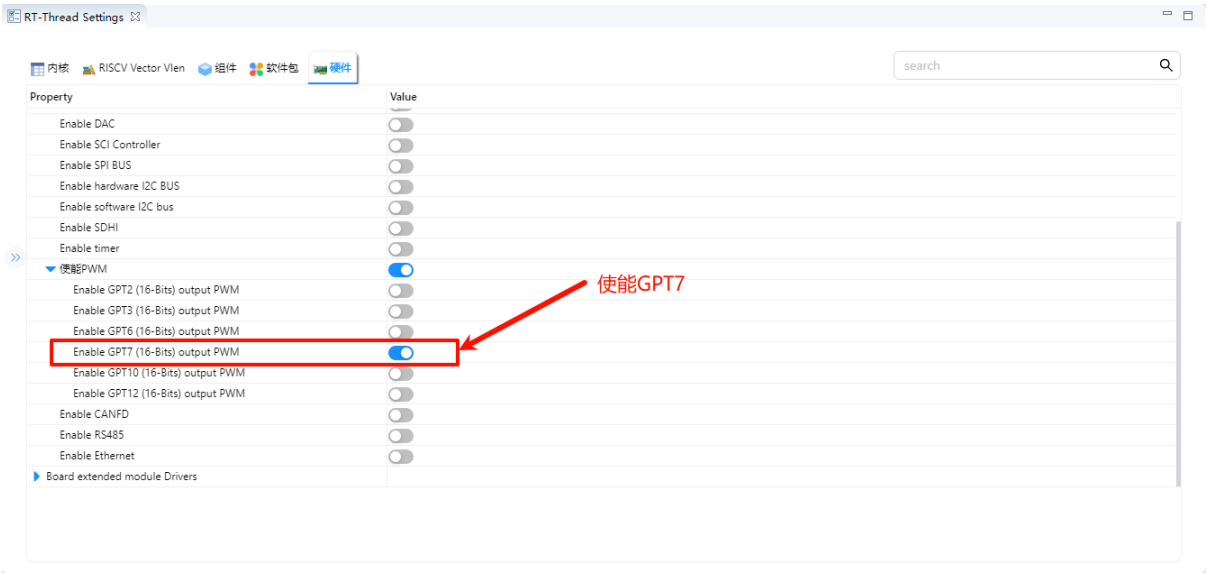


- Additionally, the pins for GPT7 are enabled:



2.5 RT-Thread Settings Configuration

- In the configuration, enable PWM7:



2.6 Example Project Instructions

To play "**Two Tigers**", we need to define the frequencies corresponding to the numbered musical notation (unit: Hz):

Note	Frequency (Hz)
C4	262
D4	294
E4	330
F4	349
G4	392
A4	440
B4	494
C5	523

The actual output frequency may be affected by **GPT clock/divider/count step quantization**. The driver will select the nearest achievable value within the range. For more accurate pitch, it's recommended to choose frequencies with **smaller quantization errors** first (or adjust the system clock/divider).

The source code for this example is located in /project/Titan_basic_buzzer/src/buzzer.c:

```
#include <rtthread.h>
#include <rtdevice.h>

#define PWM_DEV_NAME        "pwm7"  /* PWM Device Name */
#define PWM_DEV_CHANNEL     0       /* PWM Channel */

struct rt_device_pwm *pwm_dev;

typedef struct
{
    uint16_t freq;   // Frequency (Hz)
    uint16_t duration; // Duration (ms)
} note_t;

note_t song[] =
{
    {262,400}, {294,400}, {330,400}, {262,400},  // 1 2 3 1
    {262,400}, {294,400}, {330,400}, {262,400},  // 1 2 3 1
    {330,400}, {349,400}, {392,800},             // 3 4 5
    {330,400}, {349,400}, {392,800},             // 3 4 5
    {392,200}, {440,200}, {392,200}, {349,200}, {330,400}, {262,400}, // 5 6 5 4
  3 1
    {392,200}, {440,200}, {392,200}, {349,200}, {330,400}, {262,400}, // 5 6 5 4
  3 1
    {262,400}, {196,400}, {262,400}, {0,400},    // 1(低) 7(低) 1 高 休止
    {262,400}, {196,400}, {262,400}, {0,400},    // 1 7 1 休止
};
```

(continues on next page)

(continued from previous page)

```
static int buzzer_test(void)
{
    pwm_dev = (struct rt_device_pwm *)rt_device_find(PWM_DEV_NAME);
    if (!pwm_dev)
    {
        rt_kprintf("Cannot find PWM device %s\n", PWM_DEV_NAME);
        return -1;
    }

    for (size_t i = 0; i < sizeof(song)/sizeof(song[0]); i++)
    {
        if (song[i].freq == 0)
        {
            rt_pwm_disable(pwm_dev, PWM_DEV_CHANNEL);
        }
        else
        {
            uint32_t period_ns = 1000000000 / song[i].freq;  // ns
            uint32_t pulse_ns  = period_ns / 2;              // 50%

            rt_pwm_set(pwm_dev, PWM_DEV_CHANNEL, period_ns, pulse_ns);
            rt_pwm_enable(pwm_dev, PWM_DEV_CHANNEL);
        }

        rt_thread_mdelay(song[i].duration);
    }

    rt_pwm_disable(pwm_dev, PWM_DEV_CHANNEL);
    return 0;
}
MSH_CMD_EXPORT(buzzer_test, Play song on buzzer);
```

2.7 Compilation & Download

- **RT-Thread Studio**: Download the Titan Board resource package in the RT-Thread Studio package manager, then create a new project and compile it.

After compilation, connect the development board’s USB-DBG interface to the PC, and download the firmware to the development board.

2.8 Running Results

Type buzzer_sample in the serial terminal to see the effect. The buzzer will play "*Two Tigers*".

3. Button Interrupt Usage Instructions

English | Chinese

3.1 Introduction

The main function of this routine is to realize the external interrupt through the KEY on the board. When USER_KEY is pressed, the state of LED_G(Green) is changed.

3.2 RA8 Series MCU Interrupt System Overview

The RA8 series MCU has the following interrupt features:

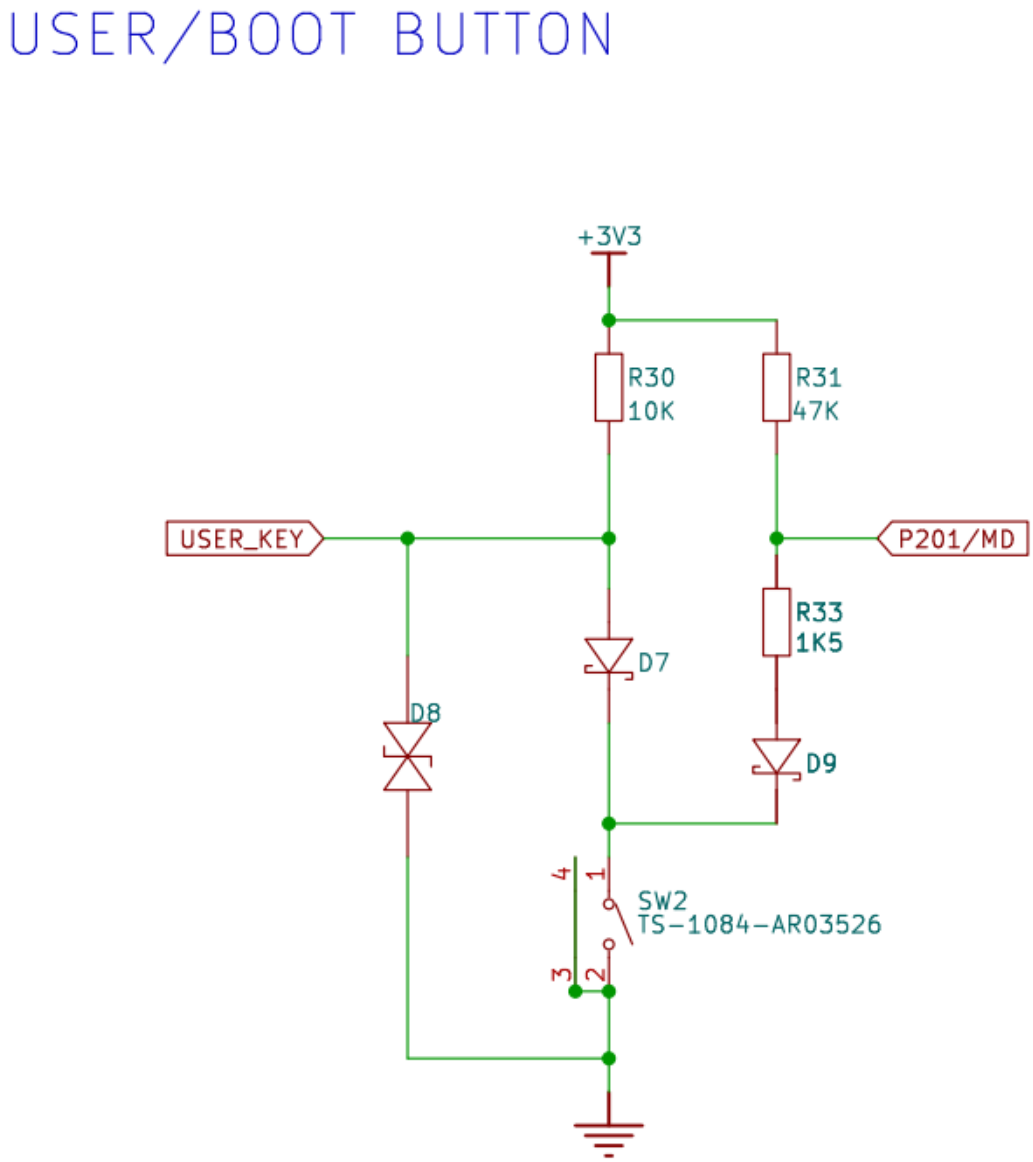
1. **NVIC (Nested Vectored Interrupt Controller)**
   - Supports **nested interrupts**: higher-priority interrupts can preempt lower-priority ones.
   - Supports **priority grouping** (Preemption Priority and Subpriority).
   - IRQn numbers are mapped to the interrupt vector table, and the NVIC calls the corresponding ISR.
2. **GPIO External Interrupt (EXTI)**
   - Each GPIO pin can be configured as an external interrupt input.
   - Supports **rising edge, falling edge, or both-edge triggers**.
   - Interrupt channels are managed by the **ICU (Interrupt Controller Unit)**.
   - Can be configured via Renesas FSP or directly through low-level registers.
   - Provides low-latency response, suitable for buttons, sensors, and other event-driven applications.
3. **RT-Thread PIN Driver Model**
   - GPIOs are encapsulated as **PIN devices** in RT-Thread.
   - Provides a unified interface:
     - rt_pin_mode(pin, mode): configure pin as input or output.
     - rt_pin_read(pin) / rt_pin_write(pin, value): read or write GPIO.
     - rt_pin_attach_irq(pin, mode, callback, args): register an interrupt callback.
     - rt_pin_irq_enable(pin, enable): enable or disable interrupts.

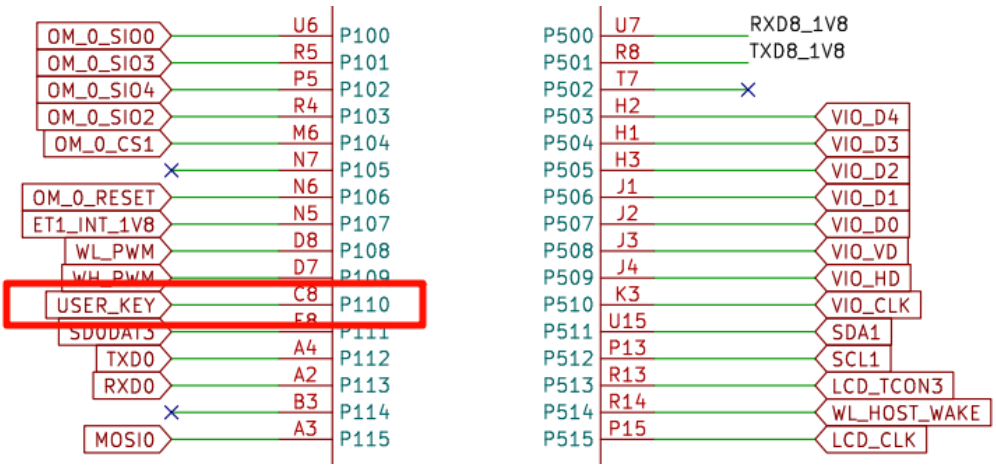
   Reference: RT-Thread PIN Device

   Using the RT-Thread PIN driver allows interrupt handling without directly accessing the NVIC or MCU registers.

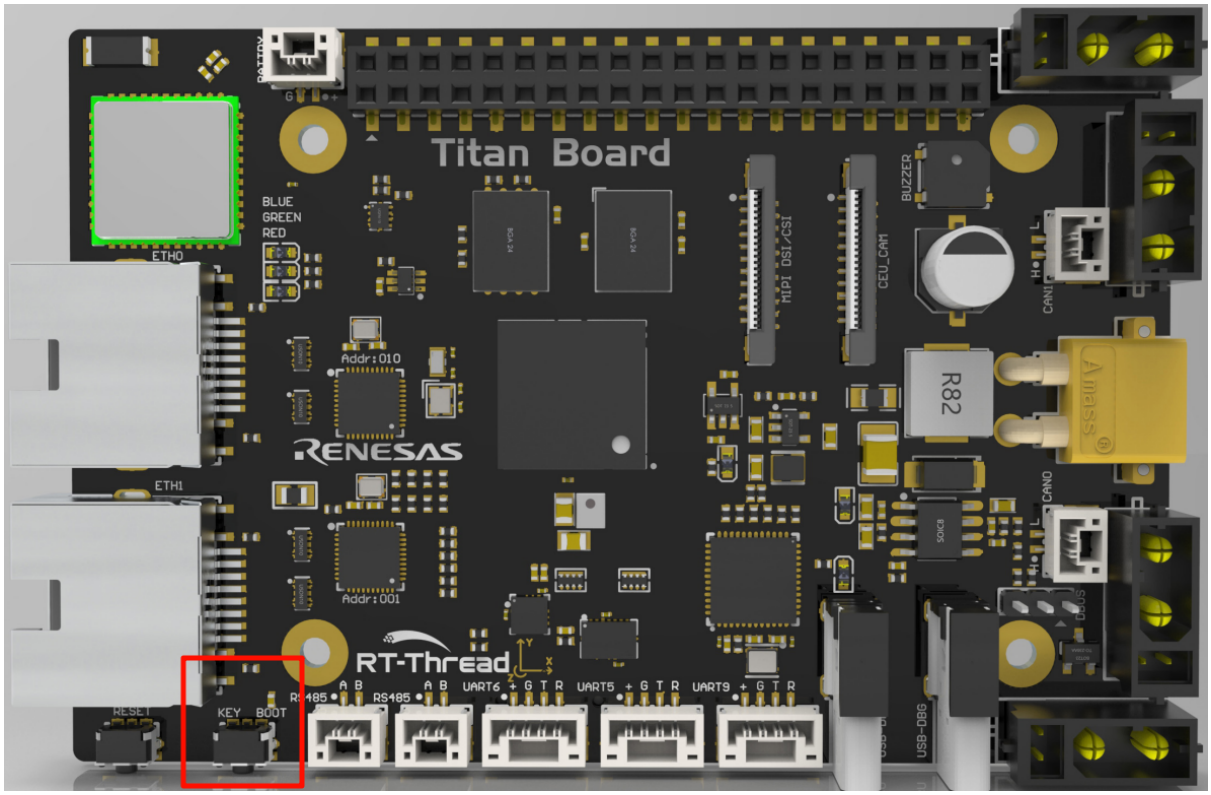
3.3 Hardware Description

As shown in the figure below, the USER_KEY pin is connected to the P110 pin of the microcontroller, and the KEY button is pressed for low level and released for high level.





The position of the KEY in the development board is shown as follows:

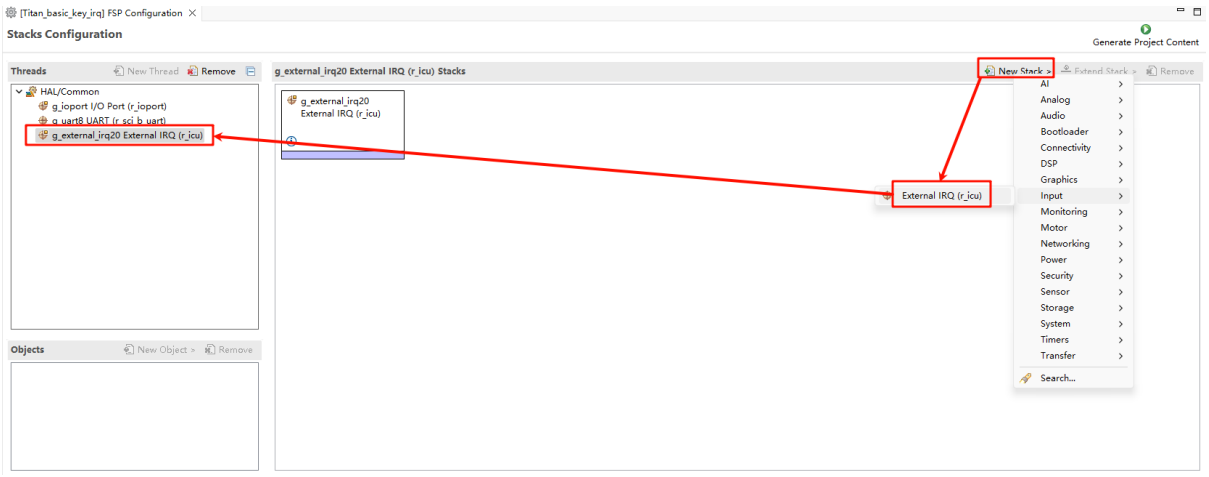


3.4 FSP Configuration

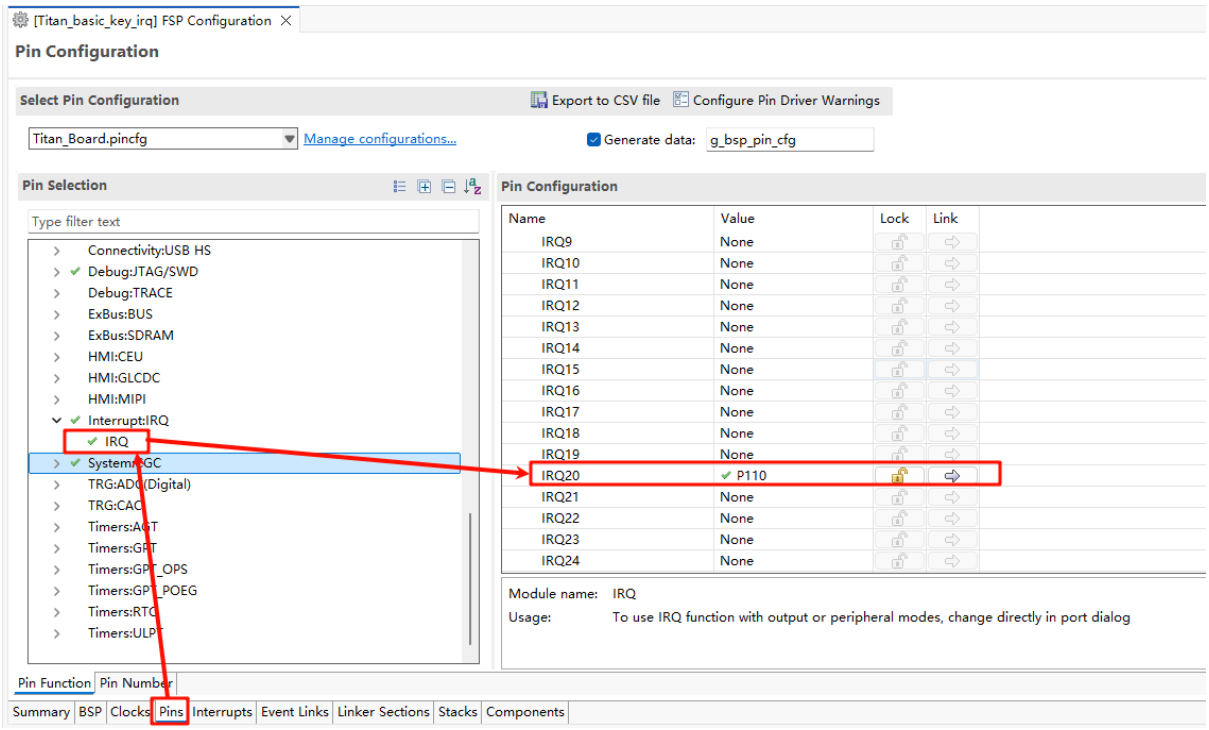
- First, download the official FSP code generation tool:

  https://github.com/renesas/fsp/releases/download/v6.0.0/setup_fsp_v6_0_0_rasc_v2025-04.1.exe

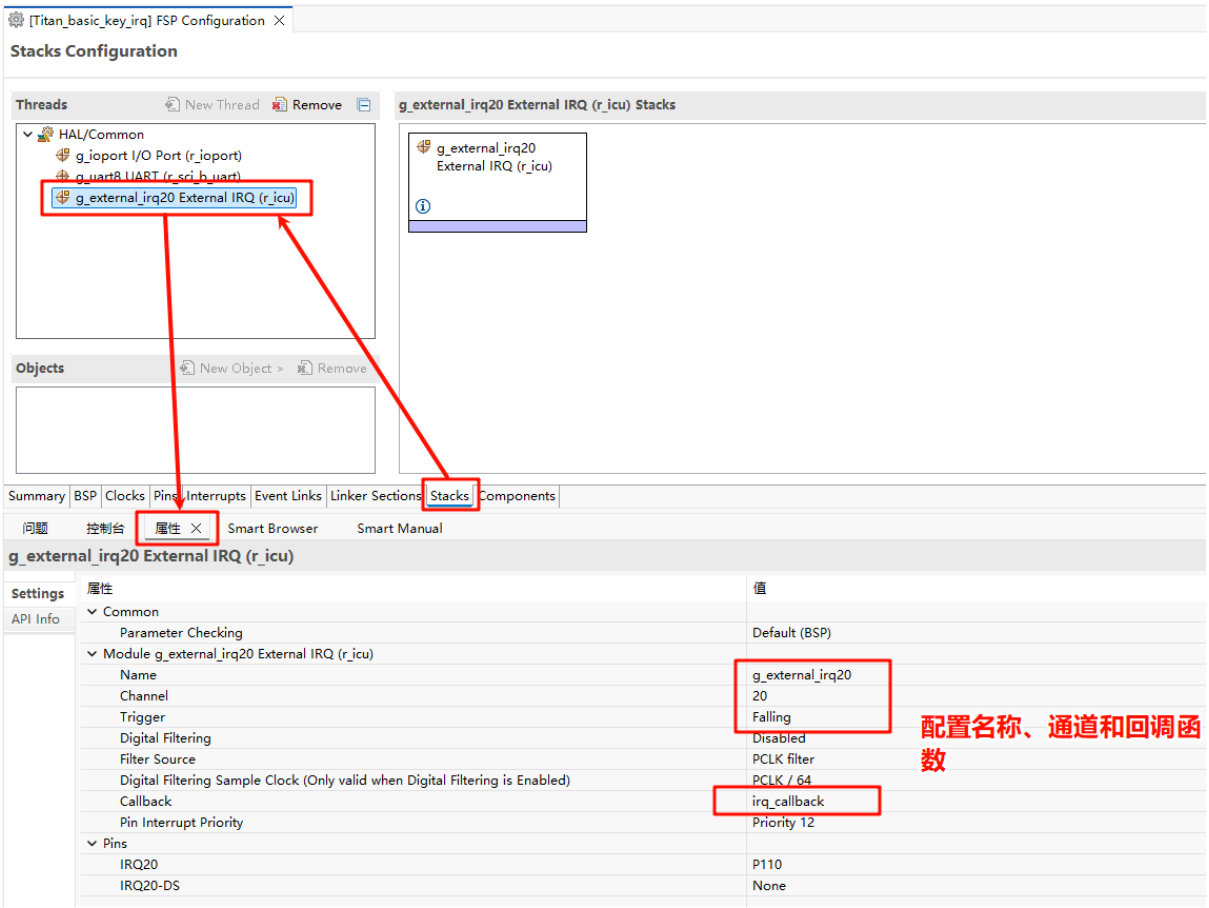
- Once installed, double-click on the rasc.exe in eclipse and open the configuration file configuration.xml in the Titan_basic_key_irq project directory:
- Let’ s add two stacks: New Stack->Input->External IRQ(r_icu)：



- Next we need to enable the IRQ feature in the pin config. Select the interrupt pin we want to enable as shown in the following screenshot: USER_KEY(IRQ20) :



- Back in the Stacks interface, set IRQ20, and configure the interrupt name, channel number, and interrupt callback function:



3.5 Example Code Description

The source code for this routine is located at /project/Titan_basic_key_irq.

The microcontroller pin corresponding to USER_KEY is defined as follows:

```
/* Configure key IRQ pins */
#define KEY_PIN     BSP_IO_PORT_01_PIN_10 /* Onboard KEY pins */
```

LED 灯的单片机引脚定义如下:

```
/* Configure LED pins */
#define LED_PIN_B   BSP_IO_PORT_00_PIN_12 /* Onboard LED pins */
#define LED_PIN_G   BSP_IO_PORT_06_PIN_13 /* Onboard LED pins */
```

The source code for the keypress interrupt is located at /project/Titan_basic_key_irq/src/hal_entry.c. When the USER_KEY key is pressed, the green LED is turned on and off.

```
#define LED_PIN_B   BSP_IO_PORT_00_PIN_12 /* Onboard LED pins */
#define LED_PIN_G   BSP_IO_PORT_06_PIN_13 /* Onboard LED pins */
```

(continues on next page)

(continued from previous page)

```
#define KEY_PIN     BSP_IO_PORT_01_PIN_10 /* Onboard KEY pins */

volatile rt_bool_t flag = 0;

void key_callback(void)
{
    if (flag)
        rt_pin_write(LED_PIN_G, PIN_HIGH);
    else
        rt_pin_write(LED_PIN_G, PIN_LOW);

    flag = flag ? RT_FALSE : RT_TRUE;
}

void hal_entry(void)
{
    rt_kprintf("\nHello RT-Thread!\n");
    rt_kprintf("================================================\n");
    rt_kprintf("This example project is a key routine!\n");
    rt_kprintf("================================================\n");

    LOG_I("Tips:");
    LOG_I("You can press the button to run the test case for button interruption.
  ");
    LOG_I("Use the buttons to control the on/off state of the green LED.");

    rt_pin_mode(KEY_PIN, PIN_MODE_INPUT_PULLUP);
    rt_pin_attach_irq(KEY_PIN, PIN_IRQ_MODE_FALLING, key_callback, RT_NULL);
    rt_pin_irq_enable(KEY_PIN, PIN_IRQ_ENABLE);

    while (1)
    {
        rt_pin_write(LED_PIN_B, PIN_HIGH);
        rt_thread_mdelay(1000);
        rt_pin_write(LED_PIN_B, PIN_LOW);
        rt_thread_mdelay(1000);
    }
}
```

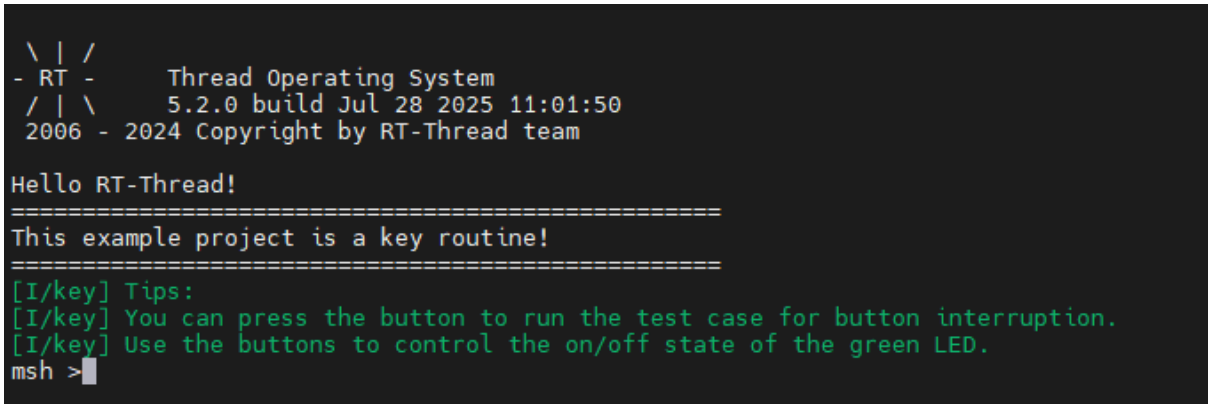
3.6 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio’s package manager, download the Titan Board resource package, create a new project, and compile it.

After compiling, connect the development board’s USB-DBG interface to the PC and download the firmware to the development board.

3.7 Run Effect

Press the reset button to restart the development board. LED_G in the initial state is in the light off state. When the USER_KEY is pressed, the light off state of LED_G(Green) will change.



Drivers

1. ADC Application Example Usage Instructions

English | Chinese

1.1 Introduction

This example demonstrates how to use the **RA8 series MCU ADC (Analog-to-Digital Converter)** on the **Titan Board**, combined with the **RT-Thread ADC driver framework** for analog signal acquisition and processing.

Main functionalities include:

- Initializing the ADC hardware module
- Configuring ADC channels, sampling time, and trigger mode
- Reading analog signals through the RT-Thread ADC driver interface
- Supporting single-shot sampling, continuous sampling, and hardware-triggered sampling

1.2 RA8P1 ADC Features

ADC Overview

ADC (Analog-to-Digital Converter) is a device or module that converts continuous analog signals into discrete digital signals. It is a core component in modern digital control systems, signal processing, and measurement systems.

- **Function**: Converts continuous signals like voltage and current into digital values for processing by microcontrollers (MCU), DSPs, or FPGAs.
- **Key Parameters**:
  - **Resolution**: The number of digital bits output by the ADC, representing the number of distinguishable levels. The RA8P1 has **16-bit** resolution, i.e., 2^16 = 65,536 distinct levels.
  - **Sampling Rate**: The number of ADC samples per second, affecting the frequency range of signals that can be captured.
  - **Input Range**: The range of analog voltages the ADC can process.
  - **Accuracy**: Indicates how close the ADC output is to the actual input signal, affected by noise, nonlinearity, and offset errors.

ADC Working Principle

The ADC operation can be divided into several stages:

1. **Sample & Hold (S/H)**
   - Captures and holds the analog signal at the sampling instant to ensure stability during conversion.
   - RA8 series ADCs support **adjustable sample-and-hold time**, optimizing sampling for high-impedance signals.
2. **Quantization**
   - Divides the analog signal into discrete levels, each corresponding to a digital code.
   - A 16-bit ADC divides the input voltage range into 65,536 levels, with quantization precision:ΔV = VREF / 65536.
3. **Encoding**

- Converts the quantized level into a binary code output.
- RA8P1 ADC supports **right-aligned / left-aligned data output** for flexible data handling.

ADC Type and RA8P1 Features

This MCU contains two units of **Noise-Shaping SAR-type A/D converters (ADC16H)** that is the hybrid architecture with

features of SAR-type and Delta-Sigma modulation-type. The A/D converter unit 0 (ADC0) can select up to 15 channels of

analog inputs. The A/D converter unit 1 (ADC1) can select up to 15 channels of analog inputs. The temperature sensor,

internal reference voltage, VBATT 1/6 voltage monitor output, and D/A converters can be A/D-converted by A/D converter

unit 0 or unit 1. A/D conversion data can be selected from 16-bit, 14-bit, 12-bit, and 10-bit length data formats.

The ADC16H has the following features:

Resolution: Up to 16-bit

Fast conversion: Up to 6.25 Msps (0.16 µs per channel) (when A/D conversion clock ADCLK = 50 MHz)

Input channels: Up to 23 analog input channels

Support single-ended input or differential inputs

Self-calibration function

Built-in channel-dedicated sample-and-hold circuit (S&H)

SAR ADC Working Flow:

1. Sample and hold the input voltage
2. Stepwise comparison with DAC output
3. Adjust the binary code according to the comparison result
4. Output the digital value

Key ADC Parameters

1. **Resolution**

- RA8P1 16-bit ADC: theoretical minimum detectable voltage ΔV = VREF / 65536
- Example: VREF = 3.3V, $\Delta V \approx 0.00005$ V ≈ 50 μV

2. **Sampling Time**

- Determines input voltage stability and ADC error.
- High-impedance signals require longer sampling time to avoid errors.

3. **Linearity**

- **INL (Integral Nonlinearity)**: Cumulative error, deviation from ideal straight line
- **DNL (Differential Nonlinearity)**: Step size error between adjacent codes

4. **Noise and Accuracy**

- System noise affects low-level resolution. The effective resolution in 16-bit mode is usually slightly below 16 bits (e.g., 15 effective bits).

5. **Input Impedance**

- High-impedance signals can be sampled directly; low-impedance or high-speed signals may require buffering.

Typical RA8P1 ADC Applications

- **Industrial Measurement**: Temperature, pressure, and flow sensor acquisition
- **Motor Control**: Current and voltage sampling for closed-loop control
- **Signal Processing**: Audio acquisition, vibration monitoring
- **Data Logging**: Multi-channel high-speed sampling for storage or transmission to a host

1.3 RT-Thread ADC Driver Framework

The RT-Thread ADC (Analog to Digital Converter) framework is a unified interface provided by the RT-Thread device driver layer to manage ADC hardware modules across various MCUs. This

framework abstracts the underlying ADC hardware into a standardized device interface, allowing applications to obtain digital values from analog signals through a uniform API and achieve cross-platform ADC functionality.

Device Model

In RT-Thread, ADCs are managed as **device objects** (subclass of struct rt_device, type RT_Device_Class_ADC). Developers do not need to manipulate hardware registers directly. All channel enabling, sampling, and disabling operations can be done via standard RT-Thread device interfaces.

Operation Interfaces

Applications access ADC devices through RT-Thread's I/O device management interfaces. Key APIs include:

- Find ADC device

```
rt_device_t rt_device_find(const char* name);
```

- Enable ADC channel

```
rt_err_t rt_adc_enable(rt_adc_device_t dev, rt_uint32_t channel);
```

- Read ADC channel value

```
rt_uint32_t rt_adc_read(rt_adc_device_t dev, rt_uint32_t channel);
```

- Disable ADC channel

```
rt_err_t rt_adc_disable(rt_adc_device_t dev, rt_uint32_t channel);
```

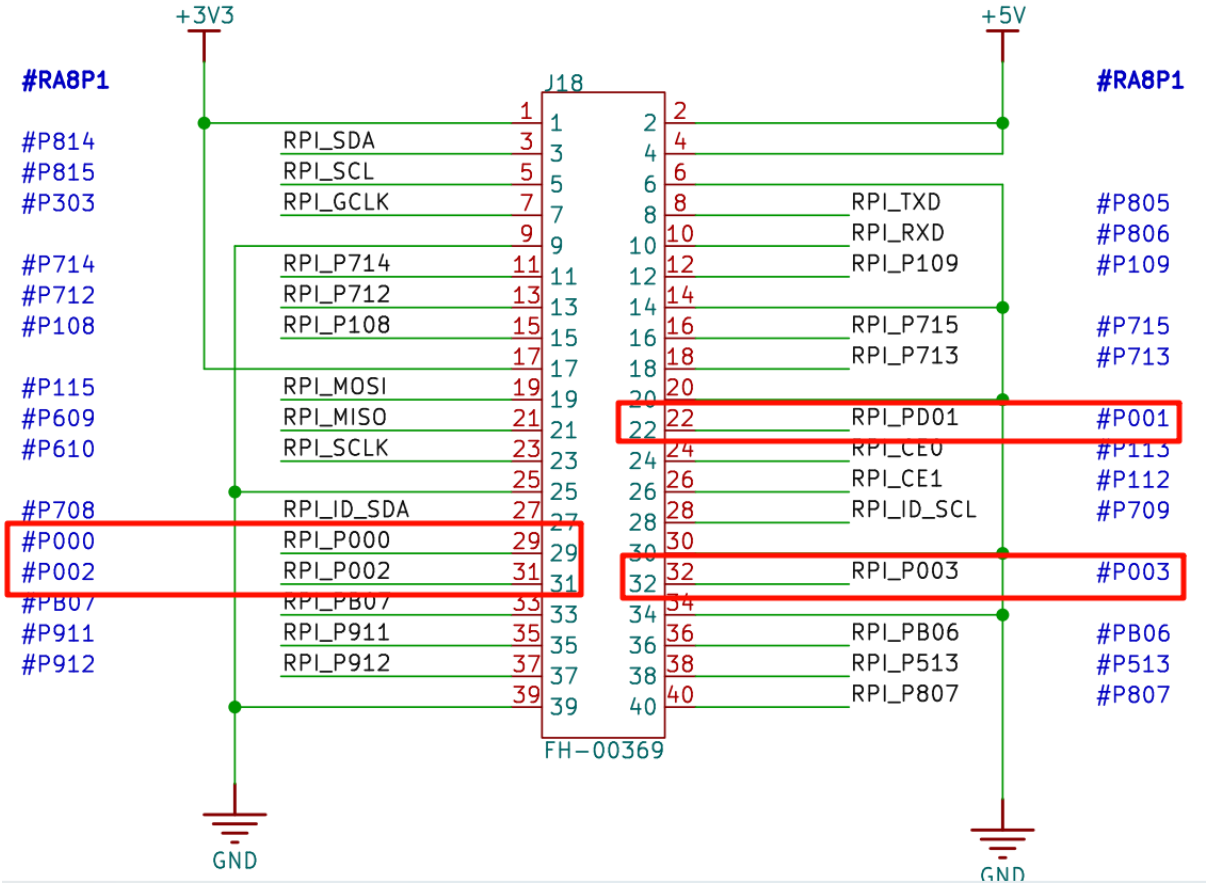
Framework Features

- **Unified Interface**: All ADC hardware modules are accessed through the same interface, simplifying application development.
- **Cross-Platform Support**: Applications can be ported across different MCU platforms without modifying ADC code.
- **Flexible Channel Control**: Supports independent enable/disable for multiple channels.
- **High Extensibility**: Can be combined with DMA, timers, and other modules for high-speed data acquisition.
- **Accurate Sampling**: Supports multiple resolutions and different reference voltage configurations.

Reference: RT-Thread ADC Device

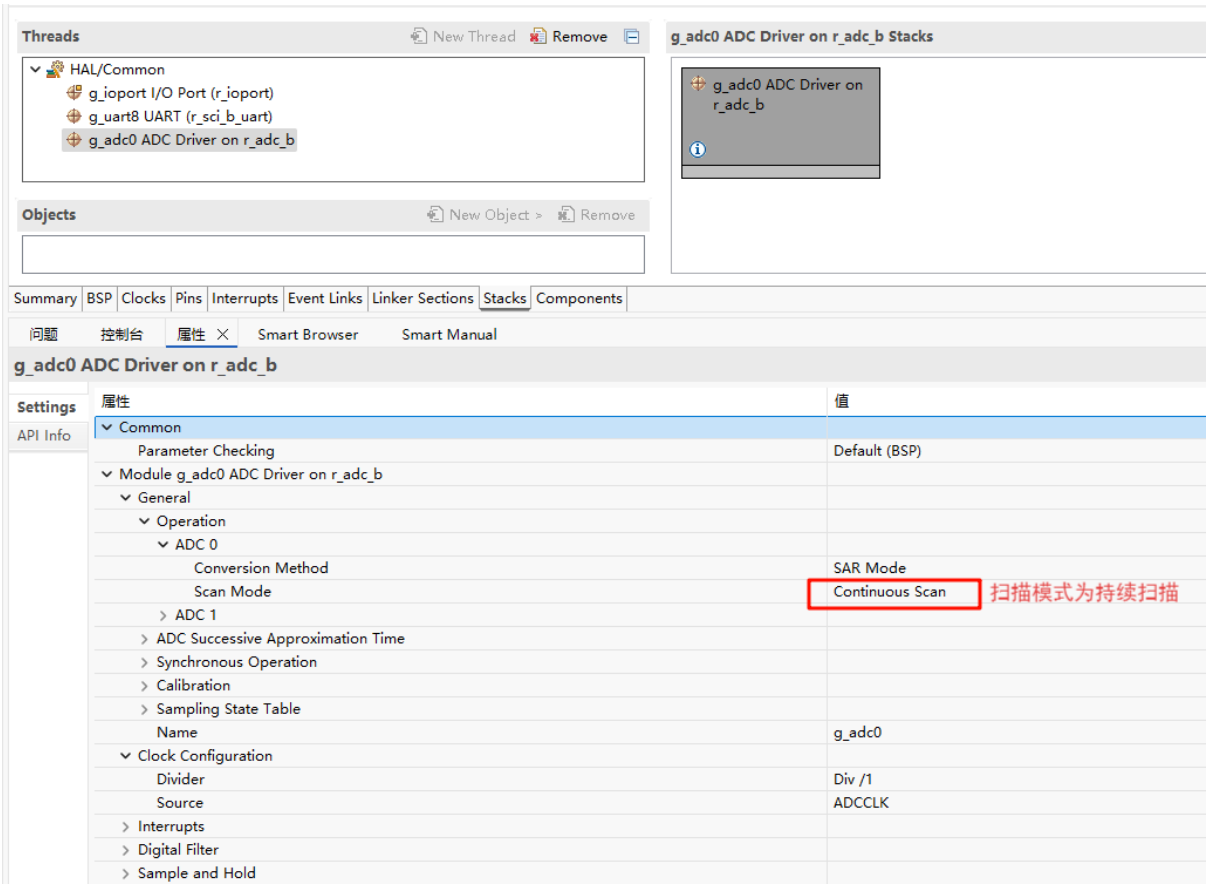
1.4 Hardware Description

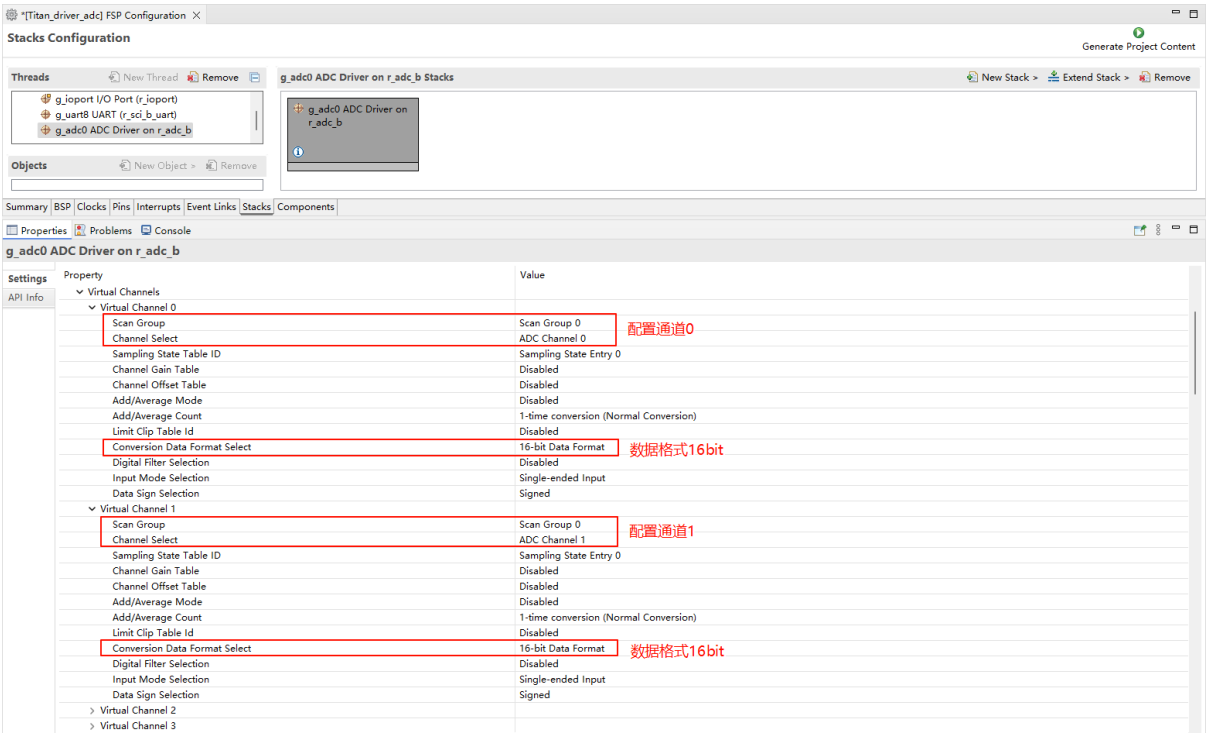
As shown in the following schematic diagram, there are 4 ADC channel interfaces on the Titan Board, namely channels 0, 1, 2 and 3 of adc0.

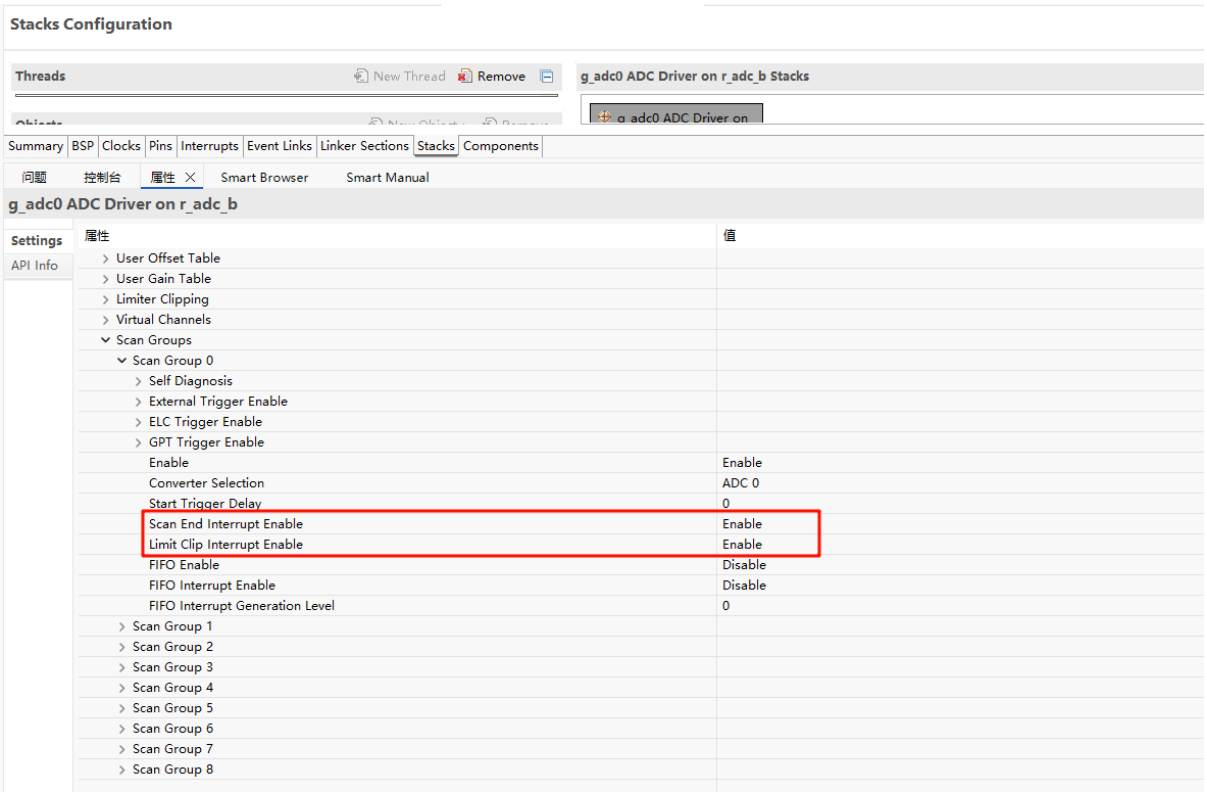


1.5 FSP Configuration

- Step 1: Open FSP and import the XML configuration file (or directly click the FSP link file in RT-Thread Studio).
- Step 2: Create a new r_adc stack, configure the ADC device, and select the channels to be used.


Threads
New Thread
Remove
HAL/Common
g_ioport I/O Port (r_ioport)
g_uart8 UART (r_sci_b_uart)
g_adc0 ADC Driver on r_adc_b
Objects
New Object »
Remove
g_adc0 ADC Driver on r_adc_b Stacks
g_adc0 ADC Driver on r_adc_b
Summary
BSP
Clocks
Pins
Interrupts
Event Links
Linker Sections
Stacks
Components
问题
控制台
属性
Smart Browser
Smart Manual
g_adc0 ADC Driver on r_adc_b
Settings
API Info
属性
值
Common
Parameter Checking
Default (BSP)
Module g_adc0 ADC Driver on r_adc_b
General
Operation
ADC 0
Conversion Method
SAR Mode
Scan Mode
Continuous Scan
扫描模式为持续扫描
ADC 1
ADC Successive Approximation Time
Synchronous Operation
Calibration
Sampling State Table
Name
g_adc0
Clock Configuration
Divider
Div /1
Source
ADCCLK
Interrupts
Digital Filter
Sample and Hold

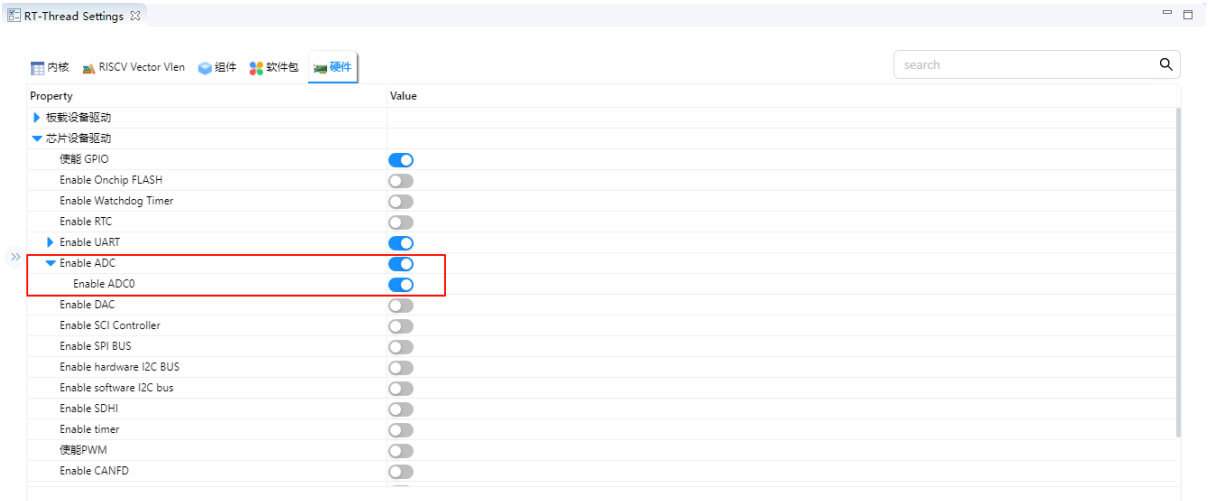

*[Titan_driver_adc] FSP Configuration
Stacks Configuration
Generate Project Content
Threads
New Thread
Remove
g_ioport I/O Port (r_ioport)
g_uart8 UART (r_sci_b_uart)
g_adc0 ADC Driver on r_adc_b
Objects
New Object »
Remove
g_adc0 ADC Driver on r_adc_b Stacks
New Stack »
Extend Stack »
Remove
g_adc0 ADC Driver on r_adc_b
Summary
BSP
Clocks
Pins
Interrupts
Event Links
Stacks
Components
Properties
Problems
Console
g_adc0 ADC Driver on r_adc_b
Settings
API Info
Property
Value
Virtual Channels
Virtual Channel 0
Scan Group
Scan Group 0
Channel Select
ADC Channel 0
配置通道0
Sampling State Table ID
Sampling State Entry 0
Channel Gain Table
Disabled
Channel Offset Table
Disabled
Add/Average Mode
Disabled
Add/Average Count
1-time conversion (Normal Conversion)
Limit Clip Table Id
Disabled
Conversion Data Format Select
16-bit Data Format
数据格式16bit
Digital Filter Selection
Disabled
Input Mode Selection
Single-ended Input
Data Sign Selection
Signed
Virtual Channel 1
Scan Group
Scan Group 0
Channel Select
ADC Channel 1
配置通道1
Sampling State Table ID
Sampling State Entry 0
Channel Gain Table
Disabled
Channel Offset Table
Disabled
Add/Average Mode
Disabled
Add/Average Count
1-time conversion (Normal Conversion)
Limit Clip Table Id
Disabled
Conversion Data Format Select
16-bit Data Format
数据格式16bit
Digital Filter Selection
Disabled
Input Mode Selection
Single-ended Input
Data Sign Selection
Signed
Virtual Channel 2
Virtual Channel 3



- Step 3: Save and click “Generate Project”. The generated code will be saved in hal_data.c.

1.6 RT-Thread Settings Configuration

- Enable ADC0:



1.7 Example Code Description

The ADC source code is located in /project/Titan_driver_adc/src/hal_entry.c. The macros used are defined as follows:

```
15
16          #define DBG_TAG             "adc"
17          #define DBG_LVL             DBG_LOG
18          #include <rtdbg.h>
19          #define ADC_DEV_NAME        "adc0"
20          #define ADC_DEV_CHANNEL     1
21          #define REFER_VOLTAGE       330
22          #define CONVERT_BITS        (1 << 16)
23
24          #define LED_PIN             BSP_IO_PORT_00_PIN_12 /* Onboard LED pins */
25
```

The specific function is to sample the analog voltage from ADC0 channel 1 every 1000ms and perform a conversion. The code is as follows:

```
static int adc_vol_sample()
{
    rt_adc_device_t adc_dev;
    rt_uint32_t value, vol;
    rt_err_t ret = RT_EOK;

    adc_dev = (rt_adc_device_t)rt_device_find(ADC_DEV_NAME);
    if (adc_dev == RT_NULL)
    {
        rt_kprintf("adc sample run failed! can't find %s device!\n", ADC_DEV_
  NAME);
        return RT_ERROR;
    }

    ret = rt_adc_enable(adc_dev, ADC_DEV_CHANNEL);

    while(1)
    {
        value = rt_adc_read(adc_dev, ADC_DEV_CHANNEL);
        rt_kprintf("the value is :%d \n", value);

        vol = value * REFER_VOLTAGE / CONVERT_BITS;
        rt_kprintf("the voltage is :%d.%02d \n", vol / 100, vol % 100);

        rt_thread_mdelay(1000);
    }

    ret = rt_adc_disable(adc_dev, ADC_DEV_CHANNEL);

    return ret;
}
```

In the example, the while loop calls adc_vol_sample every 1000ms.

1.8 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio’s package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board’s USB-DBG interface to the PC and download the firmware to the development board.

1.9 Run Effect

The effect when using ADC0 channel 1 to sample a 1.8V voltage is shown below:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Jul 25 2025 14:33:42
 2006 - 2024 Copyright by RT-Thread team
[D/drv.adc] adc0 init success

Hello RT-Thread!
==================================================
This example project is an driver adc routine!
==================================================
[I/adc] Tips:
[I/adc] You can run adc testcase by executing the instruction: 'adc_sample'
[I/adc] And apply a voltage ranging from 0 to 3.3V to the P001 pin.
msh >adc_sample
the value is :0
the voltage is :0.00
msh >the value is :22544
the voltage is :1.13
the value is :22452
the voltage is :1.13
the value is :12320
the voltage is :0.62
the value is :12068
the voltage is :0.60
the value is :35516
the voltage is :1.78
the value is :35504
the voltage is :1.78
the value is :35516
the voltage is :1.78
the value is :35504
the voltage is :1.78
the value is :35480
the voltage is :1.78
the value is :35480
the voltage is :1.78
```

2. CANFD Usage Instructions

English | Chinese

2.1 Introduction

This example demonstrates how to use the **CANFD (CAN with Flexible Data Rate) interface** on the **Titan Board**, combined with the **RT-Thread CAN driver framework**, to implement high-speed automotive or industrial communication.

Main features include:

- Initialization of the CANFD hardware interface
- Configuration of baud rate, data frames, and timing parameters
- Transmission and reception of standard and extended frames
- Unified device management via RT-Thread CAN API
- Support for high-speed data transfer using interrupts, FIFO, and DMA

2.2 CAN and CAN FD Protocol Overview

CAN Protocol Overview

CAN (Controller Area Network) is a multi-master serial communication protocol developed by Bosch in the 1980s for automotive electronics. It is characterized by high reliability and strong real-time capability, suitable for automotive, industrial automation, and smart devices.

- **Data Transmission**: Bus communication based on differential signals using CAN_H and CAN_L lines
- **Communication Mode**: Multi-master with **non-destructive bus arbitration**; higher-priority messages are transmitted first
- **Physical Layer Standards**: ISO 11898-2 (High-Speed CAN), ISO 11898-3 (Low-Speed Fault-Tolerant CAN)

CAN Frame Formats

CAN transmits data in **frames**, which are categorized as:

1. **Data Frame**: Transmits data
2. **Remote Frame**: Requests data
3. **Error Frame**: Reports bus errors
4. **Overload Frame**: Provides inter-frame spacing control

Standard Frame (CAN 2.0A)

- **Identifier (ID)**: 11 bits
- **Data Length (DLC)**: 0–8 bytes

Field	Length	Description
SOF	1 bit	Start of frame
ID	11 bits	Message identifier
RTR	1 bit	Data/remote frame flag
IDE	1 bit	Standard/extended frame
r0	1 bit	Reserved
DLC	4 bits	Data length code
Data	0–8 bytes	Data field
CRC	15 bits + 1	CRC check
ACK	2 bits	Acknowledgment
EOF	7 bits	End of frame

Extended Frame (CAN 2.0B)

- **Identifier (ID)**: 29 bits
- Other fields are similar to the standard frame; only the ID is extended by 18 bits

CAN Protocol Features

1. **Non-Destructive Bus Arbitration**: Messages with lower ID values have higher priority. Lower-priority messages automatically stop transmitting in case of conflict without affecting higher-priority messages.
2. **Differential Signal Transmission**: CAN_H - CAN_L differential signals provide strong interference resistance.
3. **Speed and Bus Length**: High-speed CAN up to 1 Mbps; maximum bus length depends on speed:
   - 1 Mbps $\rightarrow$ max 40 m
   - 500 kbps $\rightarrow$ max 100 m
   - 125 kbps $\rightarrow$ max 500 m
4. **Error Detection and Fault Tolerance**: CRC check, bit stuffing, ACK check, error counters; nodes can go offline automatically in case of repeated errors.

CAN FD Protocol Overview

CAN FD (Flexible Data-rate), introduced by Bosch in 2012 (ISO 11898-1:2015), extends the CAN standard to address bandwidth limitations and data length restrictions.

- **Key Improvements**:
  1. Extended data length: from 8 bytes to 64 bytes
  2. Variable data rate: arbitration phase remains at standard speed, while the data phase can operate 2–8 times faster

CAN FD Frame Format

CAN FD Data Frame Structure

Field	Description
SOF	Start of frame
ID	11/29-bit identifier
RTR	Data frame flag (remote frame unused)
IDE	Extended frame flag
FDF	CAN FD frame flag
BRS	Bit Rate Switch
ESI	Error State Indicator
DLC	Data length code (0–64 bytes)
Data	Data field (0–64 bytes)
CRC	CRC check (17/21 bits)
ACK	Acknowledgment
EOF	End of frame

- **BRS = 1**: Data phase transmitted at higher speed
- **ESI = 1**: Node is in error passive state

CAN FD Features

1. Larger data payload: up to 64 bytes per frame
2. Higher data rate: data phase can be 2–8 times faster than classic CAN
3. Compatibility: CAN FD nodes can coexist with classic CAN nodes; FD functionality requires hardware support
4. Enhanced error handling: maintains CAN error mechanisms while increasing CRC length for longer data

CAN vs CAN FD Comparison

Feature	CAN	CAN FD
Max Data Length	8 bytes	64 bytes
Data Rate	Up to 1 Mbps	Arbitration phase 1 Mbps, data phase 2–8 Mbps
Frame Format	Standard/Extended	Standard/Extended + FDF/BRS/ESI
Compatibility	All nodes supported	Backward compatible with classic CAN; FD nodes required
CRC Check	15-bit	17/21-bit
Applications	Vehicle ECUs, industrial control	High-speed data acquisition, cameras, ADAS systems

2.3 RA8 Series CANFD Features

RA8 series MCUs integrate a high-performance **CANFD controller**, supporting ISO CAN 2.0A/B and CANFD protocols with high-speed communication, flexible frame length, and hardware FIFO capabilities.

Basic Parameters

1. **Protocol Support**
   - ISO CAN 2.0A/B (Classic CAN)
   - CAN FD (Flexible Data Rate)
   - Standard frame (11-bit ID) and extended frame (29-bit ID)
2. **Data Frame Length**
   - CAN: 0–8 bytes
   - CAN FD: 0–64 bytes
3. **Communication Speed**
   - CAN: up to 1 Mbps
   - CAN FD: data phase up to 8 Mbps (depends on MCU clock and PHY)
4. **Hardware FIFO**
   - Independent RX/TX FIFOs
   - Multi-frame buffering for improved throughput
   - FIFO-triggered interrupts can be handled via RT-Thread ISR

Hardware Features

1. **Controller Architecture**
   - Independent RX/TX FIFOs
   - Hardware filters (standard ID / extended ID / combined)
   - Auto retransmission, error detection, and bus status monitoring
2. **Timing and Baud Rate**
   - Configurable synchronization segment, time segments 1/2, and sampling point
   - CAN FD allows asynchronous baud rate for data segment
   - Provides auto bit-rate calculation tool
3. **Interrupt Mechanism**
   - Transmission complete (TX)
   - Frame reception (RX FIFO)
   - Error warning/passive
   - Bus-off/recovery
4. **DMA Support**
   - Works with MCU DMAC for high-speed transfers
   - TX/RX FIFO automatically read/written to memory
   - Reduces CPU load
5. **Error Handling and Bus Monitoring**
   - Error counters (TEC/REC)
   - Bus states: Error Active / Error Passive / Bus Off
   - Automatic recovery

Frame Format

1. **Classic CAN Frames**
   - Standard frame: 11-bit ID, 0–8 bytes
   - Extended frame: 29-bit ID, 0–8 bytes
2. **CAN FD Frames**
   - Supports 0–64 bytes
   - High-speed data phase
   - Includes **BRS (Bit Rate Switch)** for data acceleration
3. **Frame Control Bits**
   - RTR (Remote Transmission Request)

- IDE (Identifier Extension)
- FDF (CAN FD Frame)
- ESI (Error State Indicator)

Applications

- Vehicle ECU communication
- Industrial control networks (e.g., CANopen, J1939)
- Real-time sensor data acquisition
- High-speed data logging and distributed control

2.4 RT-Thread CAN Device Interface

The RT-Thread CAN (Controller Area Network) framework is a unified interface provided by the RT-Thread device driver layer for managing CAN controller hardware modules on various MCUs. This framework abstracts the underlying CAN controller into a standard device interface, allowing applications to send, receive, and control CAN devices through a uniform API, supporting cross-platform CAN communication development.

Device Model

In RT-Thread, CAN devices are managed as **device objects** (subclass of struct rt_device, type RT_Device_Class_CAN). Developers do not need to manipulate hardware registers directly; all operations including initialization, transmission, reception, and device closure can be completed through the standard RT-Thread interface.

Operation Interfaces

Applications access CAN hardware through RT-Thread's I/O device management interfaces. Key APIs include:

- Find CAN device

```
rt_device_t rt_device_find(const char* name);
```

- Open CAN device

```
rt_err_t rt_device_open(rt_device_t dev, rt_uint16_t oflags);
```

- Control CAN device

```
rt_err_t rt_device_control(rt_device_t dev, rt_uint8_t cmd, void* arg);
```

- Send data

```
rt_size_t rt_device_write(rt_device_t dev, rt_off_t pos, const void* buffer, rt_
  size_t size);
```

- Set receive callback

```
rt_err_t rt_device_set_rx_indicate(rt_device_t dev, rt_err_t (*rx_ind)(rt_device_
  t dev, rt_size_t size));
```

- Receive data

```
rt_size_t rt_device_read(rt_device_t dev, rt_off_t pos, void* buffer, rt_size_t
  size);
```

- Close CAN device

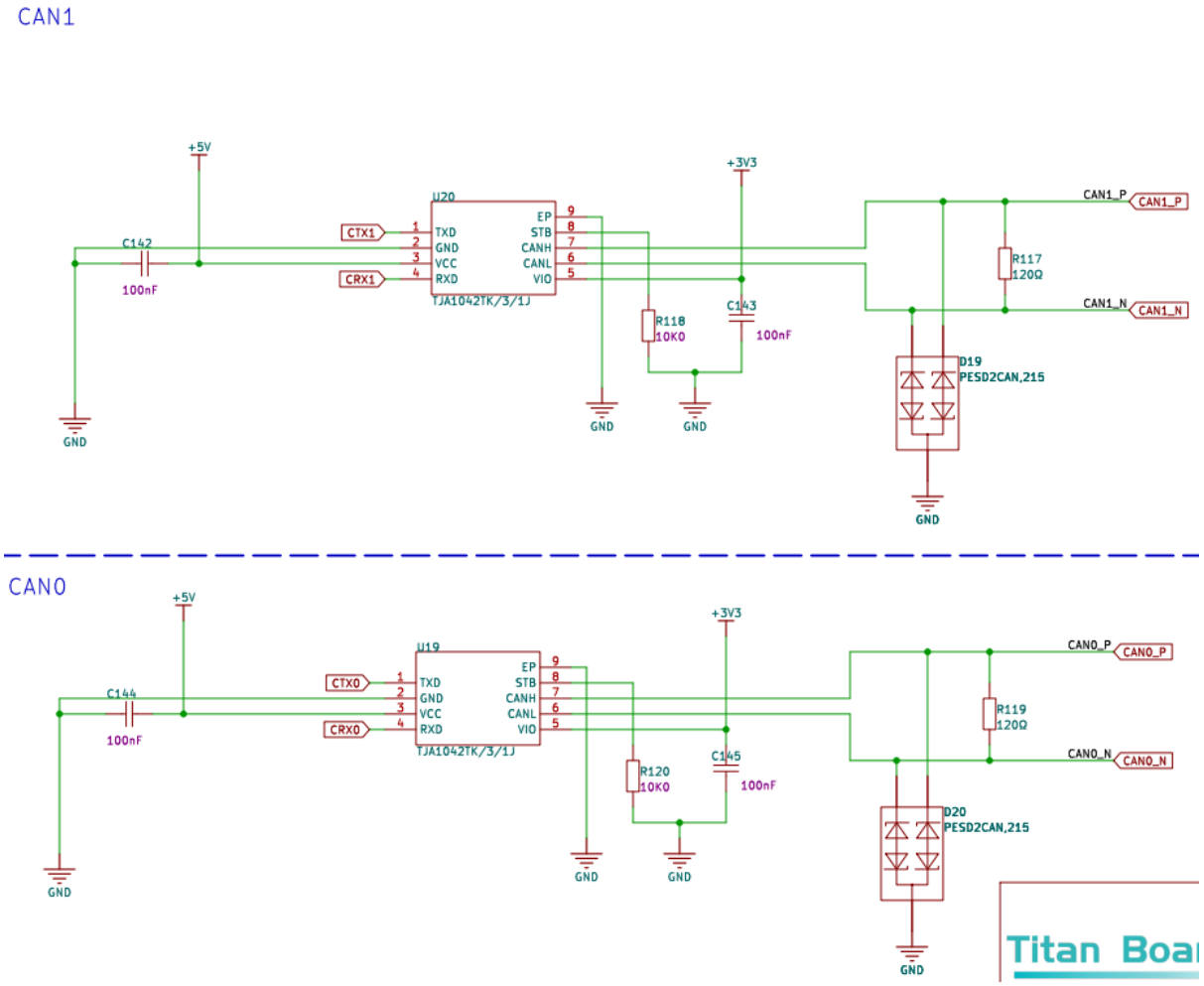
```
rt_err_t rt_device_close(rt_device_t dev);
```

Framework Features

- **Unified Device Abstraction**: All hardware CAN controllers are accessed through the same interface.
- **Cross-Platform Support**: Applications can be ported across different MCU platforms without modifying CAN code.
- **Efficient Communication**: Supports transmission, reception, and interrupt callback mechanisms for real-time data handling.
- **Flexible Expansion**: Can be combined with multiple channels, filters, and message queues for complex communication scenarios.
- **Safe and Reliable**: Hardware and software mechanisms ensure data integrity and reliable transmission.

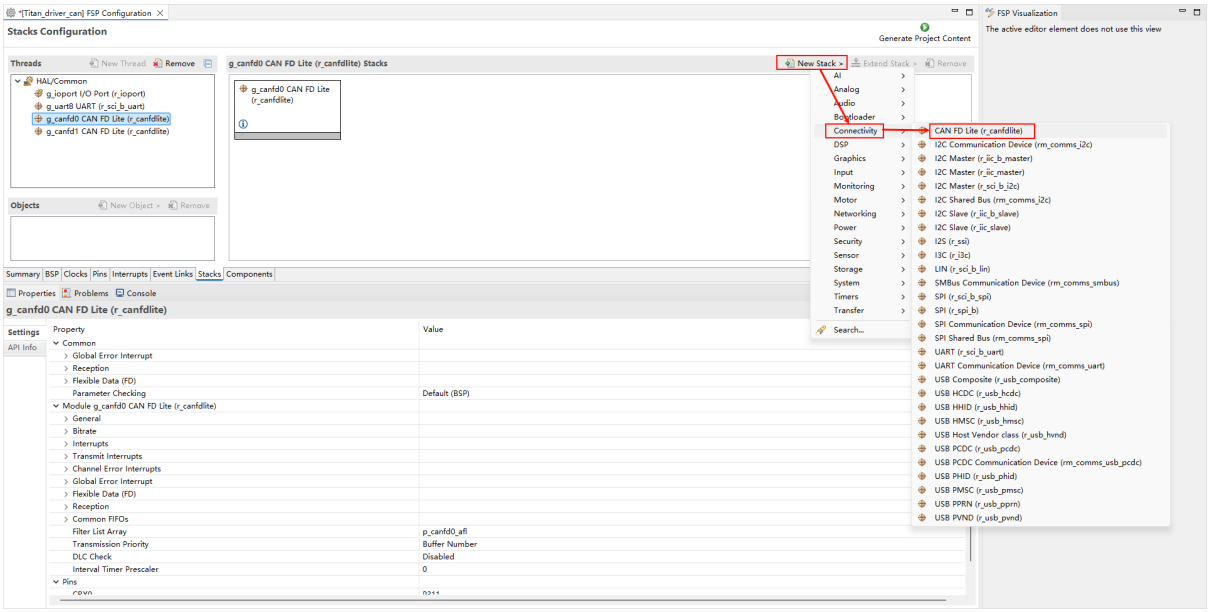
Reference: RT-Thread CAN Device

2.5 Hardware Description

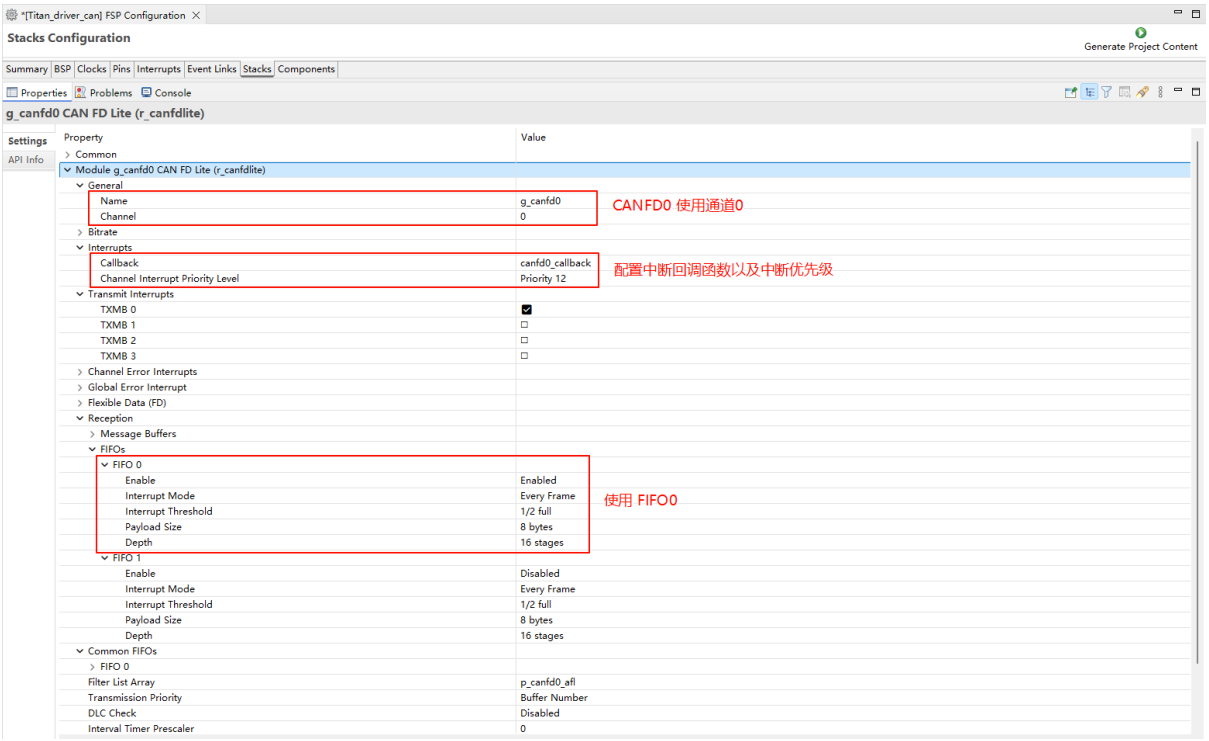


2.6 FSP Configuration

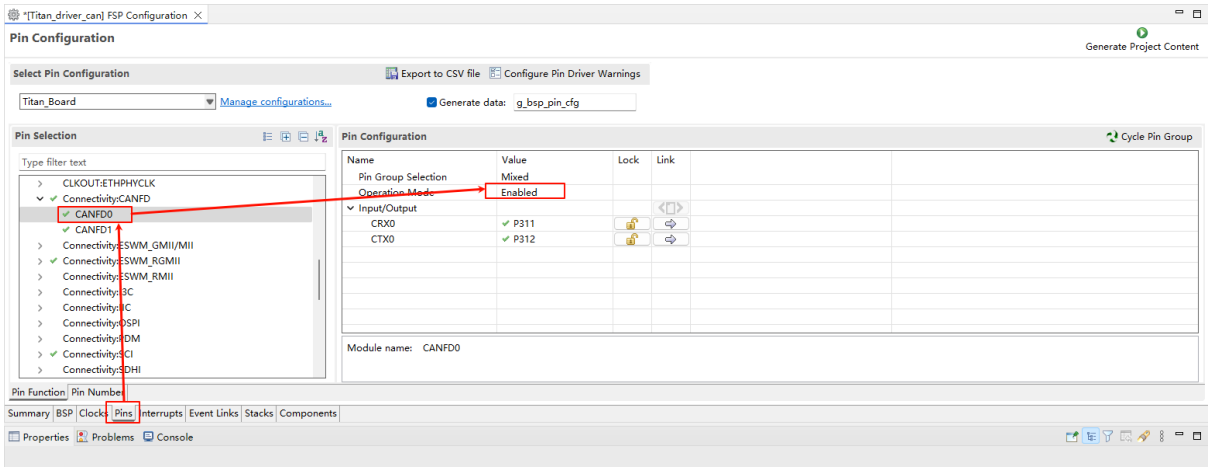
- Create two r_canfdlite stacks:



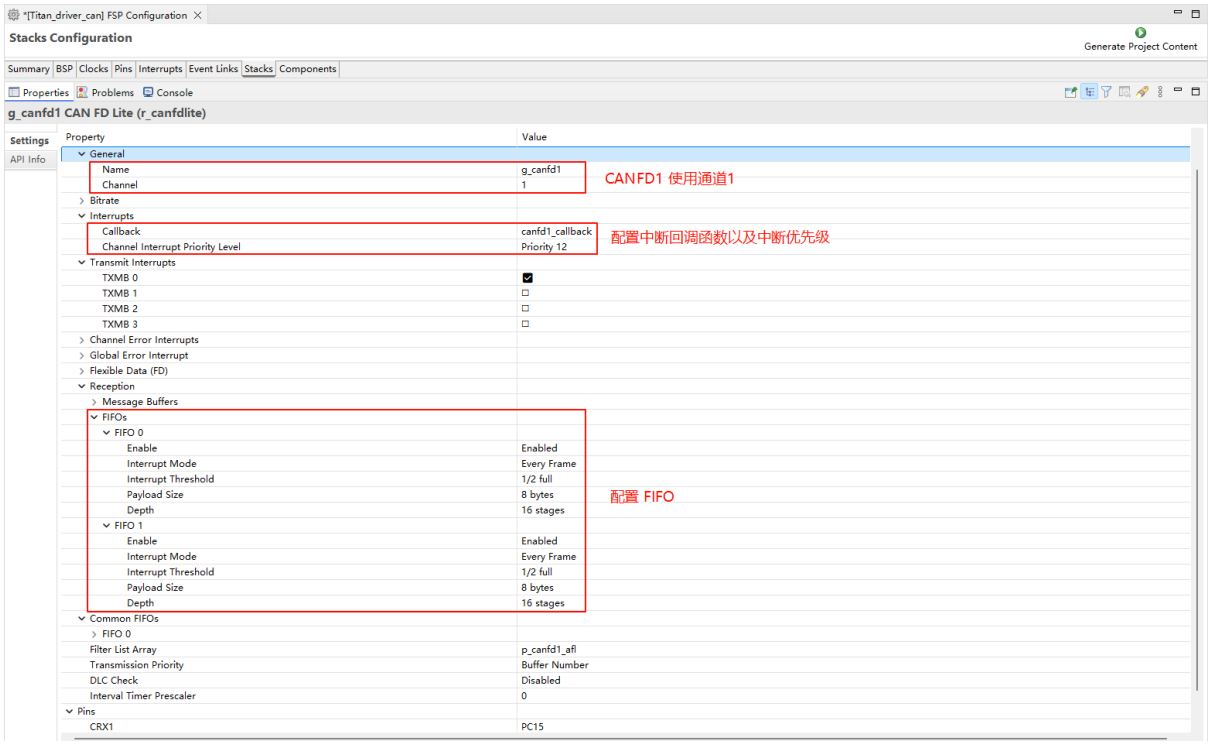
- Configure the CANFD0 stack:



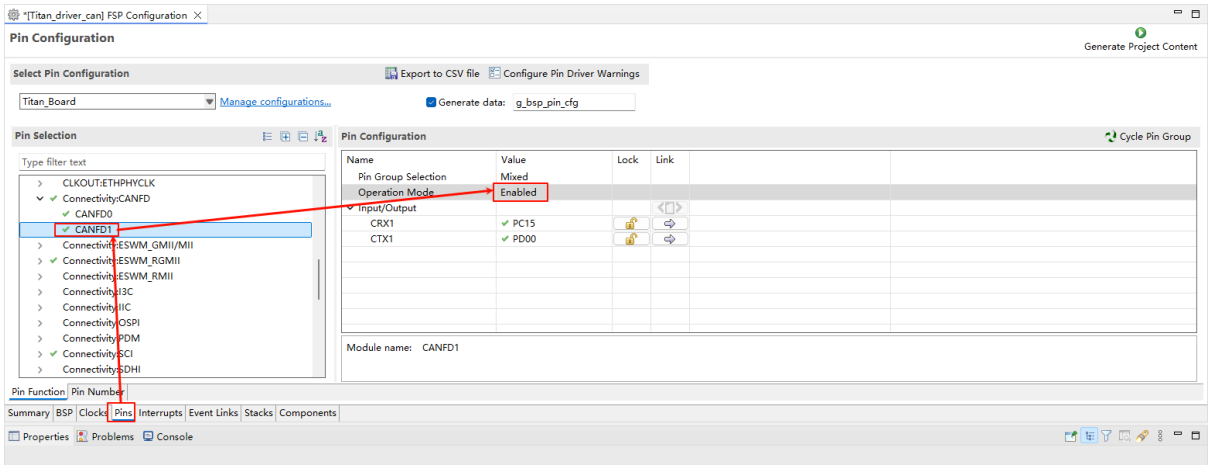
- Configure CANFD0 pin:



- Configure the CANFD1 stack:

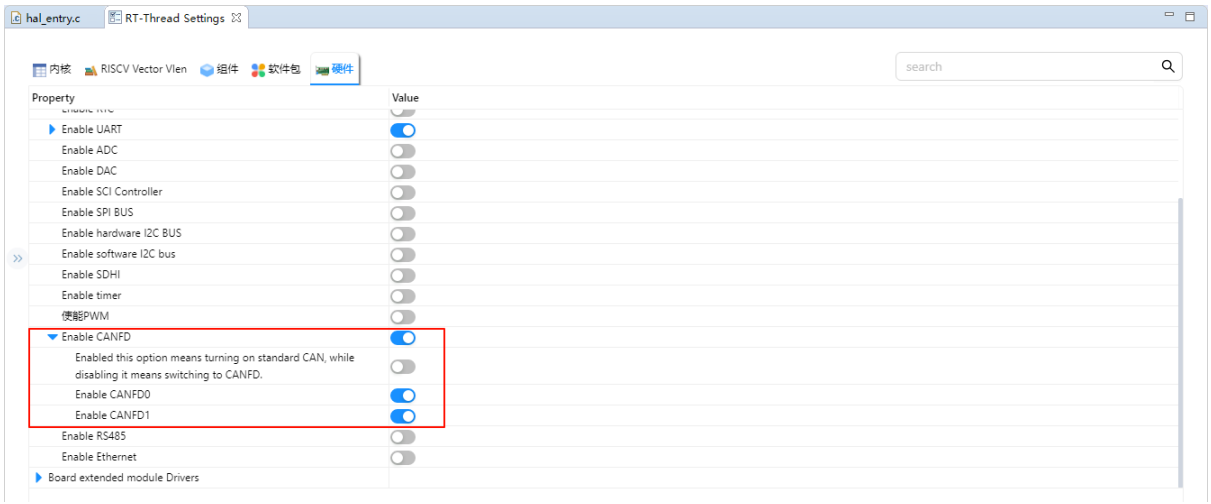


- Configure CANFD1 pin:



2.7 RT-Thread Settings Configuration

- Enable CANFD0 and CANFD1:



2.8 Example Code Description

The project sends the message through canfd0 and canfd1 receives the message and prints it using the serial port.

Send code example:

```
static void can_tx_thread(void *parameter)
{
    struct rt_can_msg msg = {0};
    rt_size_t  size;
    msg.id = 0x78;              /* ID is 0x78 */
    msg.ide = RT_CAN_STDID;     /* Standard format */
    msg.rtr = RT_CAN_DTR;       /* Data frame */
    msg.len = 8;                /* The data length is 8 */
```

(continues on next page)

(continued from previous page)

```
    /* The 8-byte data to be sent */
    rt_memset(msg.data,0,sizeof(msg.data));
    while(1)
    {
        /* Send a CAN data frame */
        for(int i = 0; i < 8; i++)
        {
            msg.data[i] = i;
        }
        size = rt_device_write(can0_dev, 0, &msg, sizeof(msg));
        if (size == 0)
        {
            rt_kprintf("can dev write data failed!\n");
        }
        rt_thread_delay(1000);
    }
}

int can_send(int argc, char *argv[])
{
    rt_err_t res;
    rt_thread_t thread;
    char can_name[RT_NAME_MAX];

    if (argc == 2)
    {
        rt_strncpy(can_name, argv[1], RT_NAME_MAX);
    }
    else
    {
        rt_strncpy(can_name, CAN0_DEV_NAME, RT_NAME_MAX);
    }
    /* Find CAN device */
    can0_dev = rt_device_find(can_name);
    if (!can0_dev)
    {
        rt_kprintf("find %s failed!\n", can_name);
        return RT_ERROR;
    }

    /* Open CAN device in interrupt receive and send mode */
    res = rt_device_open(can0_dev, RT_DEVICE_FLAG_INT_TX | RT_DEVICE_FLAG_INT_
  RX);
    RT_ASSERT(res == RT_EOK);
    /* Create data receiving thread */
    thread = rt_thread_create("can0_tx", can_tx_thread, RT_NULL, 1024, 25, 10);
```

(continues on next page)

(continued from previous page)

```
    if (thread != RT_NULL)
    {
        rt_thread_startup(thread);
    }
    else
    {
        rt_kprintf("create can_rx thread failed!\n");
    }

    return res;
}
MSH_CMD_EXPORT(can_send, can device sample);
```

Receive code example:

```
/* Receiving Data Callback Function */
static rt_err_t can_rx_call(rt_device_t dev, rt_size_t size)
{
    /* The CAN generates an interrupt after receiving data, invokes this callback
  function, and then sends a receive semaphore. */
    rt_sem_release(&rx_sem);

    return RT_EOK;
}

static void can_rx_thread(void *parameter)
{
    struct rt_can_msg rxmsg = {0};

    /* Set up the receiving callback function */
    rt_device_set_rx_indicate(can1_dev, can_rx_call);

    while (1)
    {
        /* hdr value is -1, indicating to read data directly from the uselist
  linked list */
        rxmsg.hdr_index = -1;
        /* Block waiting to receive semaphore */
        rt_sem_take(&rx_sem, RT_WAITING_FOREVER);
        /* Read a frame of data from CAN */
        rt_device_read(can1_dev, 0, &rxmsg, sizeof(rxmsg));
        /* Print Data ID and Content */
        rt_kprintf("ID:%x\tmessege:", rxmsg.id);
        for(int i = 0; i < 8; i++)
        {
            rt_kprintf("%d",  rxmsg.data[i]);
```

(continues on next page)

(continued from previous page)

```
        }
        rt_kprintf("\n");
    }
}

int can_receive(int argc, char *argv[])
{
    rt_err_t res;
    rt_thread_t thread;
    char can_name[RT_NAME_MAX];

    if (argc == 2)
    {
        rt_strncpy(can_name, argv[1], RT_NAME_MAX);
    }
    else
    {
        rt_strncpy(can_name, CAN1_DEV_NAME, RT_NAME_MAX);
    }
    /* Find CAN device */
    can1_dev = rt_device_find(can_name);
    if (!can1_dev)
    {
        rt_kprintf("find %s failed!\n", can_name);
        return RT_ERROR;
    }

    /* Initialize CAN receive semaphore */
    rt_sem_init(&rx_sem, "rx_sem", 0, RT_IPC_FLAG_FIFO);

    /* Open CAN device in interrupt receive and send mode */
    res = rt_device_open(can1_dev, RT_DEVICE_FLAG_INT_TX | RT_DEVICE_FLAG_INT_
  RX);
    RT_ASSERT(res == RT_EOK);
    /* Create data receiving thread */
    thread = rt_thread_create("can1_rx", can_rx_thread, RT_NULL, 1024, 25, 10);
    if (thread != RT_NULL)
    {
        rt_thread_startup(thread);
    }
    else
    {
        rt_kprintf("create can_rx thread failed!\n");
    }
    return res;
}
MSH_CMD_EXPORT(can_receive, can device sample);
```

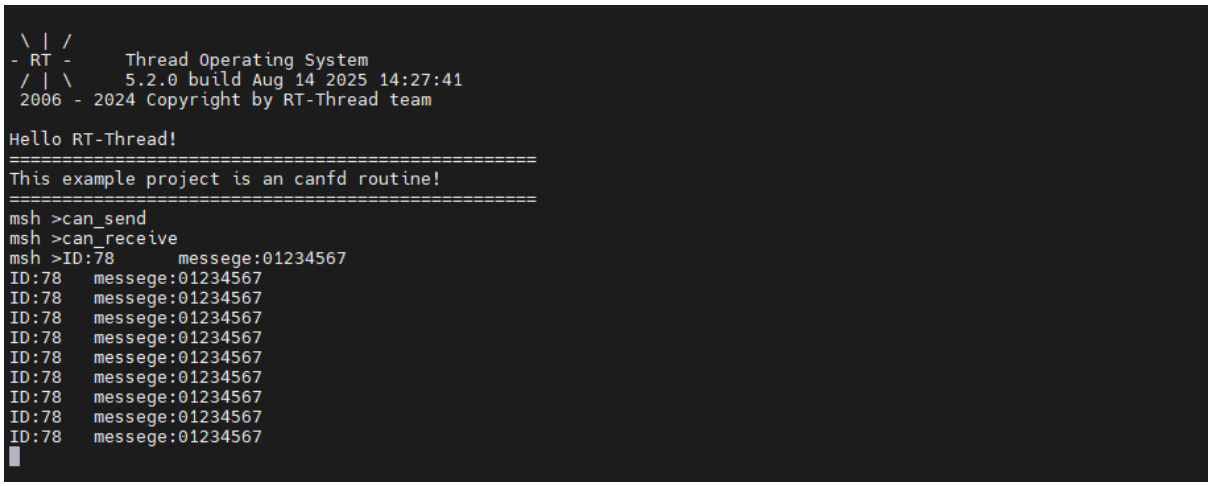
2.9 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio’s package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board’s USB-DBG interface to the PC and download the firmware to the development board.

2.10 Run Effect

Docking CAN0 and CAN1, input ‘can_send’ and can_receive commands respectively for loop back test.



3. Ethernet Usage Guide

English | Chinese

3.1 Introduction

This example demonstrates how to use the **Ethernet interface** on the **Titan Board** with the **RT-Thread Ethernet driver framework** to implement network connectivity.

Main features include:

- Initialization of RA8 Ethernet hardware
- Configuration of IP address, subnet mask, and gateway
- Sending and receiving Ethernet frames
- Integration with RT-Thread’ s netdev framework for unified network device management
- Support for high-speed data transfer using DMA and interrupts

3.2 Ethernet Introduction

Overview

Ethernet is the most widely used Local Area Network (LAN) technology. It was first proposed by Xerox PARC in the 1970s and later standardized by IEEE 802.3. Ethernet has the following characteristics:

- **Data Transmission Method**: Packet-based communication using **frames**, physically transmitted over twisted pair cables, fiber optics, or wireless media.
- **Topology**: Traditional Ethernet used bus or star topologies; modern Ethernet primarily adopts star and tree topologies.
- **Protocol Layer**: Operates at the data link layer (Layer 2) and physical layer (Layer 1) of the OSI model.

Ethernet Frame Structure

Ethernet uses **frames** as the unit of data transmission. An Ethernet frame consists of the following fields:

Field	Length	Description
Preamble	7 bytes	Used for frame synchronization
Start Frame Delimiter (SFD)	1 byte	Frame start indicator (10101011)
Destination MAC Address	6 bytes	Receiver hardware address
Source MAC Address	6 bytes	Sender hardware address
Type/Length	2 bytes	Upper layer protocol type or frame length
Payload	46–1500 bytes	Upper layer data (e.g., IP packet)
CRC	4 bytes	Cyclic Redundancy Check for integrity

Minimum Frame Length: 64 bytes **Maximum Frame Length**: 1518 bytes (without VLAN tag)

Ethernet Physical Layer and Speed

Ethernet supports multiple speeds and physical media standards:

Standard	Speed	Medium	Feature
10BASE-T	10 Mbps	Twisted pair	Original Ethernet standard
100BASE-TX	100 Mbps	Twisted pair	Fast Ethernet
1000BASE-T	1 Gbps	Twisted pair	Gigabit Ethernet
10GBASE-T	10 Gbps	Twisted pair	10G Ethernet
10GBASE-SR	10 Gbps	Fiber	Short-range fiber transmission
40GBASE / 100GBASE	40/100 Gbps	Fiber	High-speed data center Ethernet

- **BASE-T**: Twisted pair transmission
- **BASE-SX / LX / SR**: Fiber optic transmission

Ethernet Access Control

Ethernet traditionally uses **CSMA/CD (Carrier Sense Multiple Access with Collision Detection)**:

1. **Carrier Sense**: Listen to the channel before transmitting.
2. **Multiple Access**: All nodes share the same channel.
3. **Collision Detection**: If a collision occurs, nodes stop transmitting and retry after a random delay.

Note: Modern switch-based Ethernet typically operates in **full-duplex**, so CSMA/CD is no longer used.

VLAN and Ethernet Extensions

- **VLAN (Virtual LAN)**: Defined by IEEE 802.1Q to logically segment networks, improving security and manageability.
- **VLAN Tagged Frame Structure**:

Field	Length	Description
TPID	2 bytes	Ethernet type identifier 0x8100
TCI	2 bytes	VLAN ID and priority
Payload	46–1500 bytes	Upper layer data
CRC	4 bytes	Frame check

- **QoS Support**: The PCP field in VLAN tags can indicate priority levels.

Ethernet Frame Types

1. **Unicast**: Frame destined for a single MAC address.
2. **Broadcast**: Frame destined for all nodes (MAC = FF:FF:FF:FF:FF:FF).
3. **Multicast**: Frame destined for a group of nodes.

Ethernet Trends

1. **Higher Speeds**: From 10 Mbps → 100 Mbps → 1 Gbps → 10/40/100 Gbps.
2. **Industrial Ethernet**: Supports real-time control protocols like EtherCAT, Profinet, TSN.
3. **Power over Ethernet (PoE)**: Transmits data and power over the same cable.
4. **Full-Duplex Switch Networks**: Eliminates collisions and improves bandwidth utilization.
5. **Time-Sensitive Networking (TSN)**: Provides deterministic latency for industrial automation and automotive networks.

3.3 RA8 Series Ethernet Features

The RA8 series MCUs (such as RA8P1) integrate a high-performance **Ethernet MAC** that supports RGMII, RMII, and MII interfaces, enabling robust and high-speed network communication. The MAC can work with external PHYs and is compatible with the LwIP TCP/IP stack.

Network Interface Features

1. **Interface Types**
    - **RMII (Reduced Media Independent Interface)**: saves pins, supports 10/100 Mbps
    - **MII (Media Independent Interface)**: standard interface, supports 10/100 Mbps

- **RGMII (Reduced Gigabit MII)**: supports 10/100/1000 Mbps, high-speed interface suitable for Gigabit Ethernet

2. **PHY Connection**
   - External PHY is connected via MDC/MDIO interface
   - Supports auto-negotiation of speed and duplex
   - Provides access to PHY registers for configuration and status monitoring

MAC Features

1. **Duplex and Speed Support**
   - Full/half duplex
   - Supports 10/100/1000 Mbps (RGMII)
   - Auto-negotiation or forced configuration
2. **Frame Handling**
   - VLAN tag support (IEEE 802.1Q, optional)
   - Multicast and broadcast filtering
   - Hardware CRC generation and checking
3. **MAC Address Management**
   - Supports single or multiple MAC addresses
   - Configurable via FSP or software

DMA and Buffering

1. **Independent TX/RX DMA Engines**
   - Simultaneous transmission and reception
   - Reduces CPU overhead and increases throughput
2. **Descriptor Queues**
   - Configurable TX/RX buffer count
   - Supports chained DMA for efficient large data transfer
3. **Multi-Buffer Management**
   - Ring buffer support for continuous streaming
   - Reduces packet loss
4. **Hardware Acceleration**
   - Frame filtering, length checking, and CRC verification

Interrupt Mechanism

1. **Interrupt Types**
    - Receive complete (RX)
    - Transmit complete (TX)
    - Error interrupts (CRC error, buffer overflow)
2. **Configuration**
    - Priority configurable via FSP
    - Compatible with RT-Thread ISR integration
3. **Optimization**
    - RX/TX interrupts can be combined with DMA
    - Selective interrupt enable for performance

PHY Management

1. **MDIO Interface**
    - Read/write PHY registers for configuration, reset, and status monitoring
2. **Auto-Negotiation**
    - Automatic negotiation of speed (10/100/1000 Mbps) and duplex mode
3. **Link Monitoring**
    - Detects link up/down
    - Detects CRC errors and collisions

Protocol and Stack Support

1. **TCP/IP Stack Integration**
    - Compatible with LwIP
    - Supports TCP/UDP/ICMP, DHCP client/server, ARP
2. **Application Layer**
    - Supports Telnet, HTTP, MQTT, and other protocols
    - Multi-thread safe for concurrent access

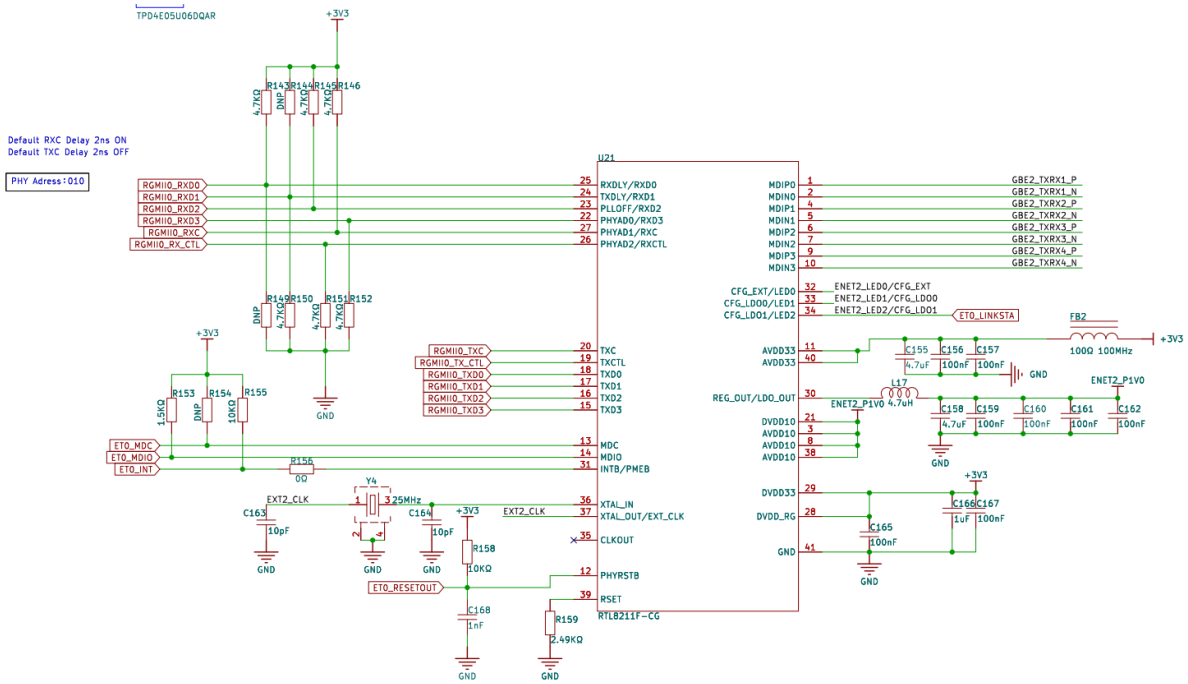
Performance and Reliability

1. **Throughput Optimization**
    - DMA + interrupts reduce CPU load
    - Configurable TX/RX buffer sizes
2. **Reliability Features**
    - Hardware CRC checking

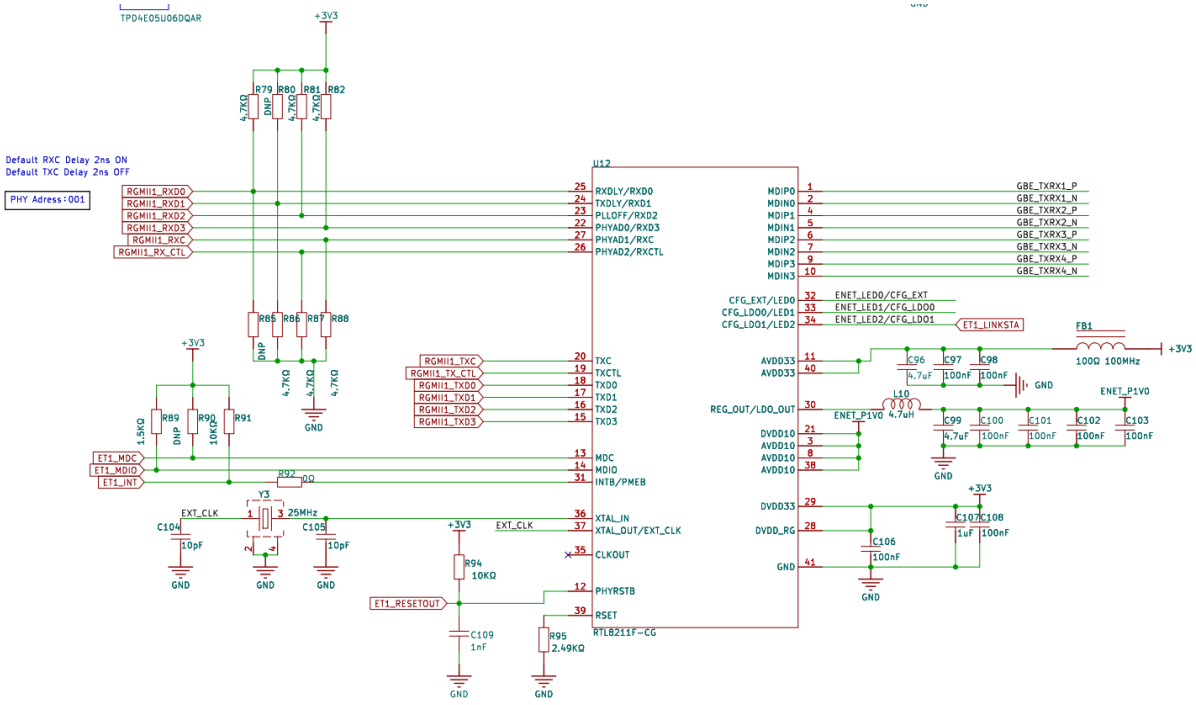
- VLAN and multicast filtering reduce interference
- Link detection and automatic reconnection

3.4 Hardware Description

ETH0:



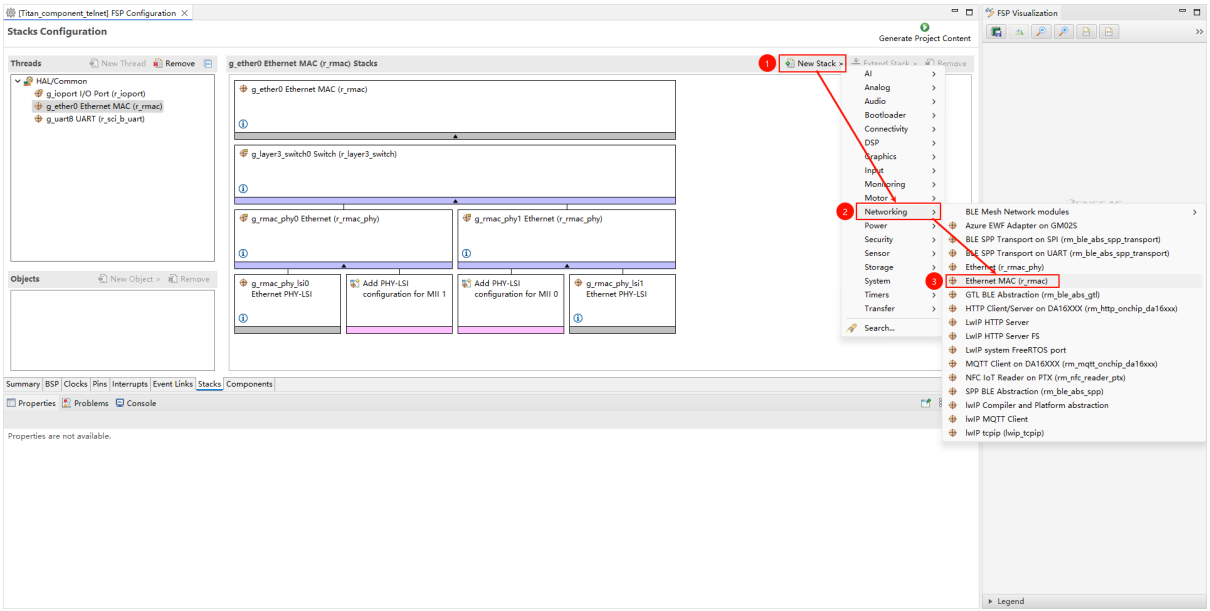
ETH1:



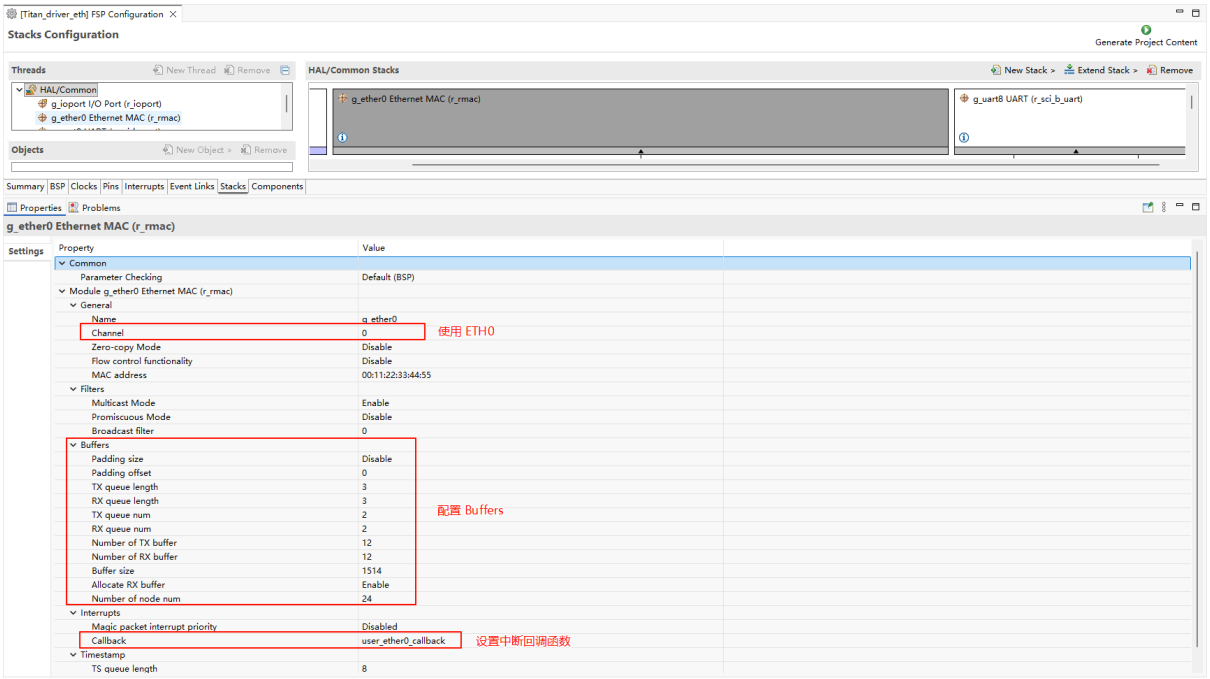
3.5 FSP Configuration

Note: The FSP version used in this project is 6.2.0, please use FSP 6.2.0 when using the FSP configuration feature.

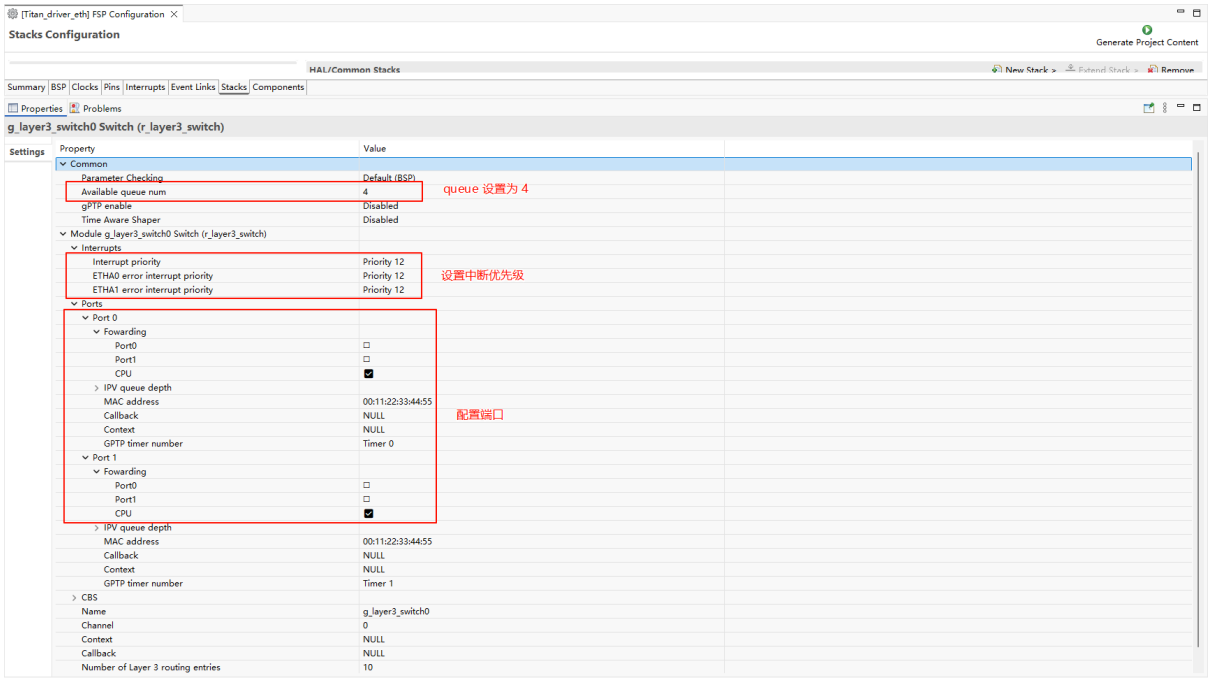
- Create a r_mac stack:



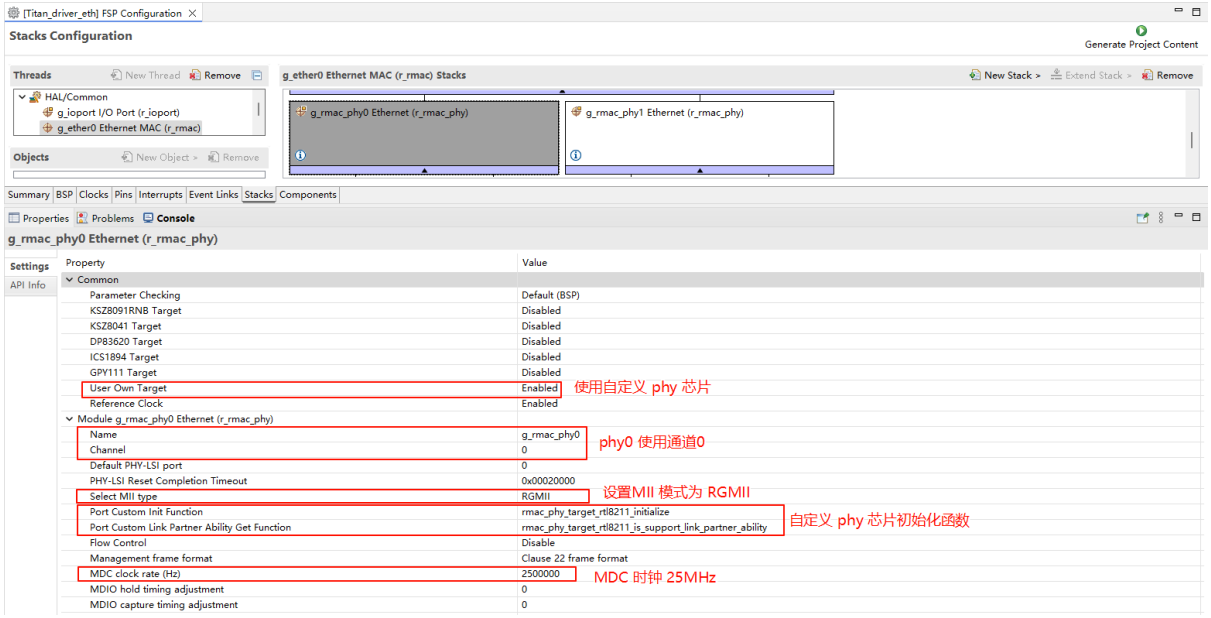
- Configure the r_mac stack:



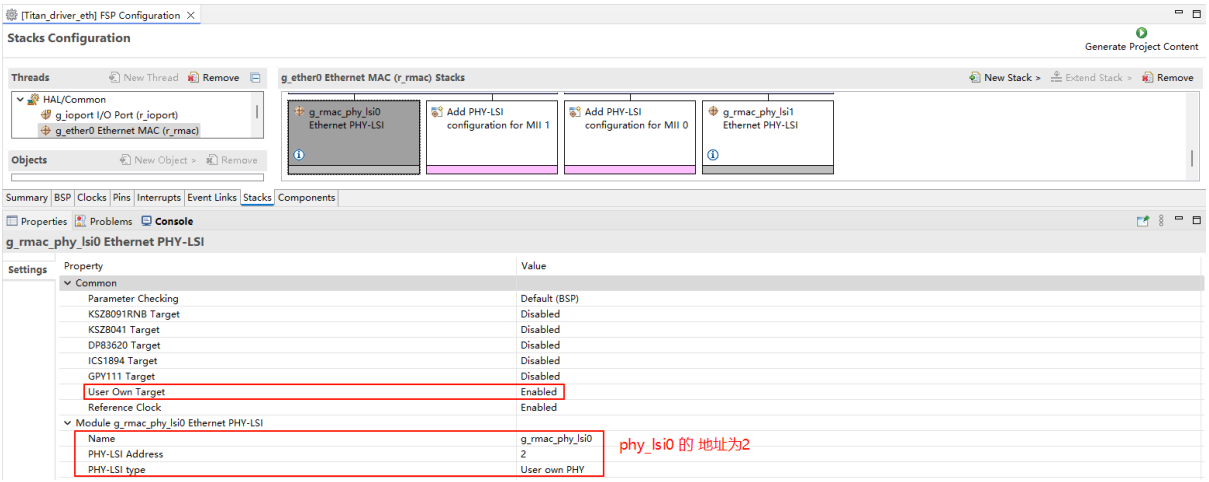
- Configure r_layer3_switch：



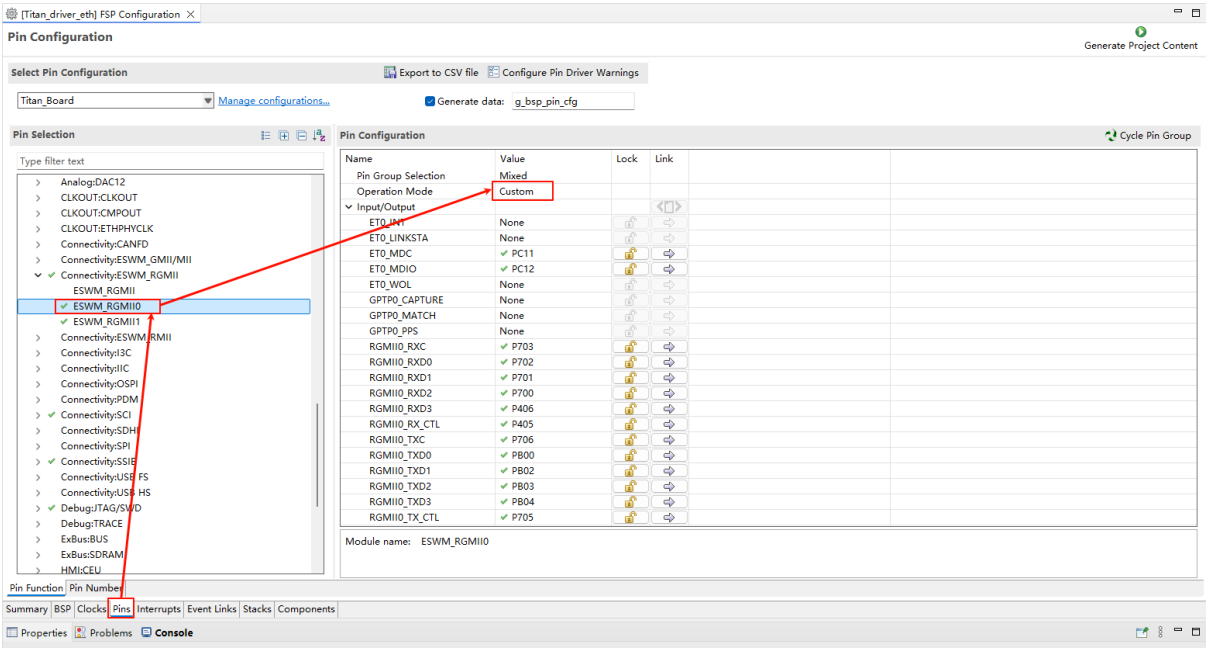
- Configure r_rmac_phy：



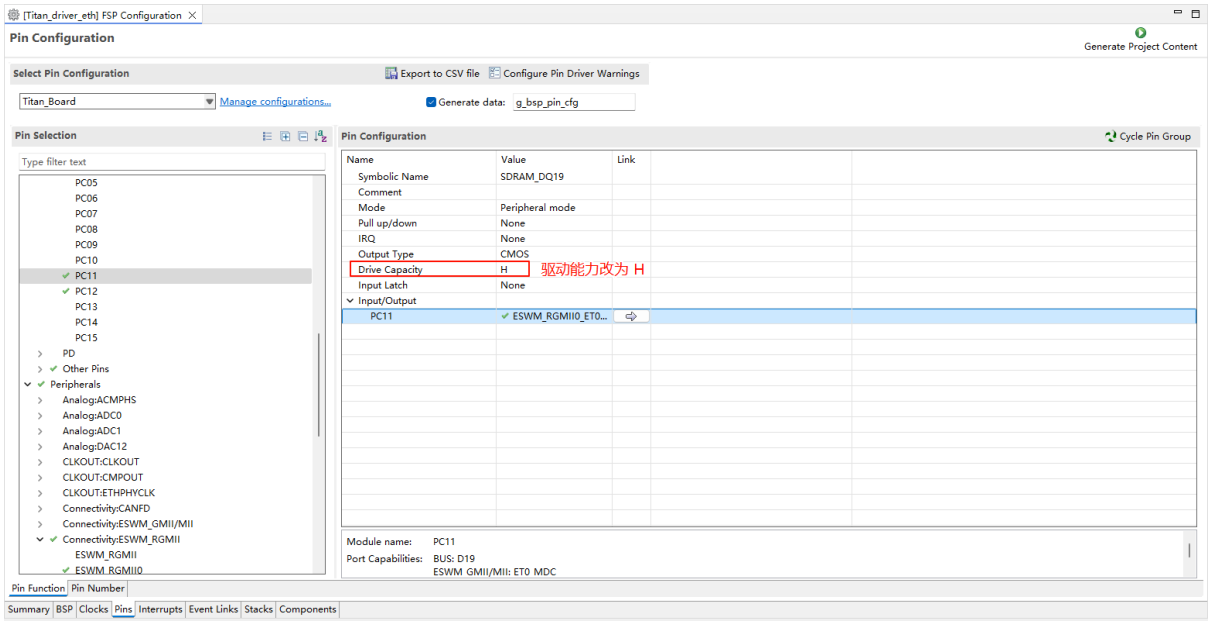
- Configure g_rmac_phy_lsi0：



- ETH0 pin configuration:

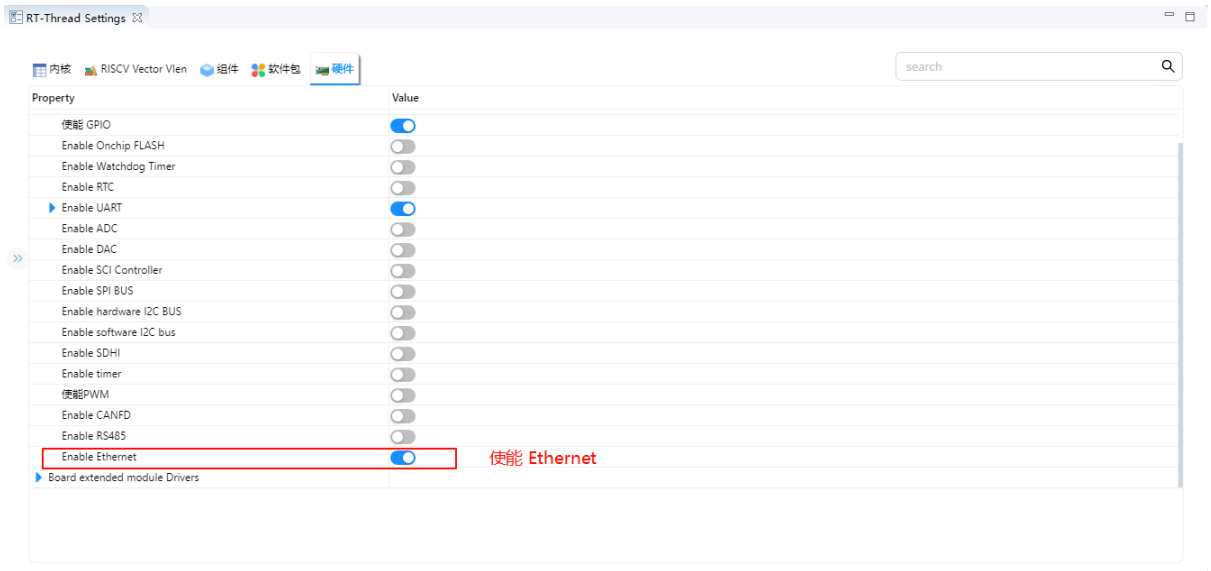


- **Note:** All pins related to ETH need to change the drive capability to H.



3.6 RT-Thread Settings Configuration

- Enable Ethernet in RT-Thread Settings.



3.7 Software Overview

The Ethernet phy chip initialization function is in ./board/ports/drv_rtl8211.c:

```
void rmac_phy_target_rtl8211_initialize (rmac_phy_instance_ctrl_t * phydev)
{
#define RTL_8211F_PAGE_SELECT 0x1F
#define RTL_8211F_EEELCR_ADDR 0x11
```

(continues on next page)

(continued from previous page)

```
#define RTL_8211F_LED_PAGE 0xD04
#define RTL_8211F_LCR_ADDR 0x10

    uint32_t val1, val2 = 0;

    /* switch to led page */
    R_RMAC_PHY_Write(phydev, RTL_8211F_PAGE_SELECT, RTL_8211F_LED_PAGE);

    /* set led1(green) Link 10/100/1000M, and set led2(yellow) Link 10/100/
  1000M+Active */
    R_RMAC_PHY_Read(phydev, RTL_8211F_LCR_ADDR, &val1);
    val1 |= (1 << 5);
    val1 |= (1 << 8);
    val1 &= (~(1 << 9));
    val1 |= (1 << 10);
    val1 |= (1 << 11);
    R_RMAC_PHY_Write(phydev, RTL_8211F_LCR_ADDR, val1);

    /* set led1(green) EEE LED function disabled so it can keep on when linked */
    R_RMAC_PHY_Read(phydev, RTL_8211F_EEELCR_ADDR, &val2);
    val2 &= (~(1 << 2));
    R_RMAC_PHY_Write(phydev, RTL_8211F_EEELCR_ADDR, val2);

    /* switch back to page0 */
    R_RMAC_PHY_Write(phydev, RTL_8211F_PAGE_SELECT, 0xa42);
}

bool rmac_phy_target_rtl8211_is_support_link_partner_ability (rmac_phy_instance_
  ctrl_t * p_instance_ctrl,
                                                              uint32_t
        line_speed_duplex)
{
    FSP_PARAMETER_NOT_USED(p_instance_ctrl);
    FSP_PARAMETER_NOT_USED(line_speed_duplex);

    /* This PHY-LSI supports half and full duplex mode. */
    return true;
}
```

3.8 Build & Download

- **RT-Thread Studio:** Download the Titan Board resource pack from the RT-Thread Studio package manager, then create a new project and compile it.

Once compiled, connect the development board’ s USB-DBG interface to the PC, and download the firmware to the development board.

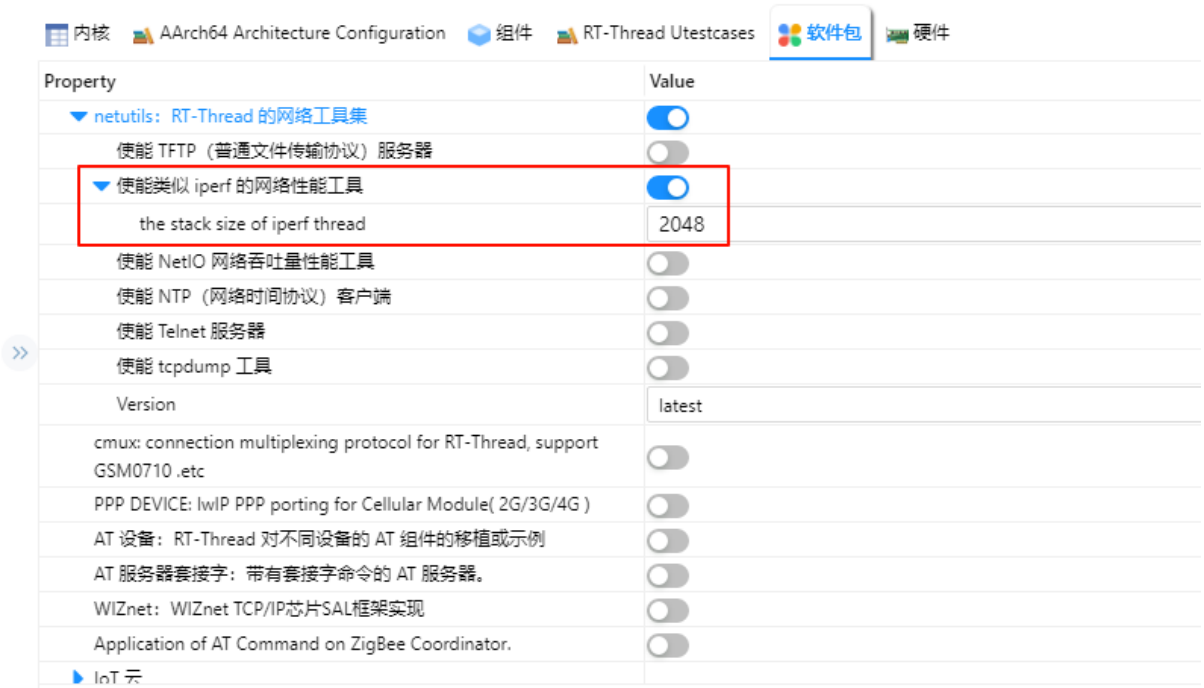
3.9 Running Result

Insert the network cable into the ETH0 network port, press the reset button to restart the development board, wait for PHY0 link up, then enter ifconfig to view the IP address obtained by the development board, and then enter ping baidu.com command to test the connectivity.

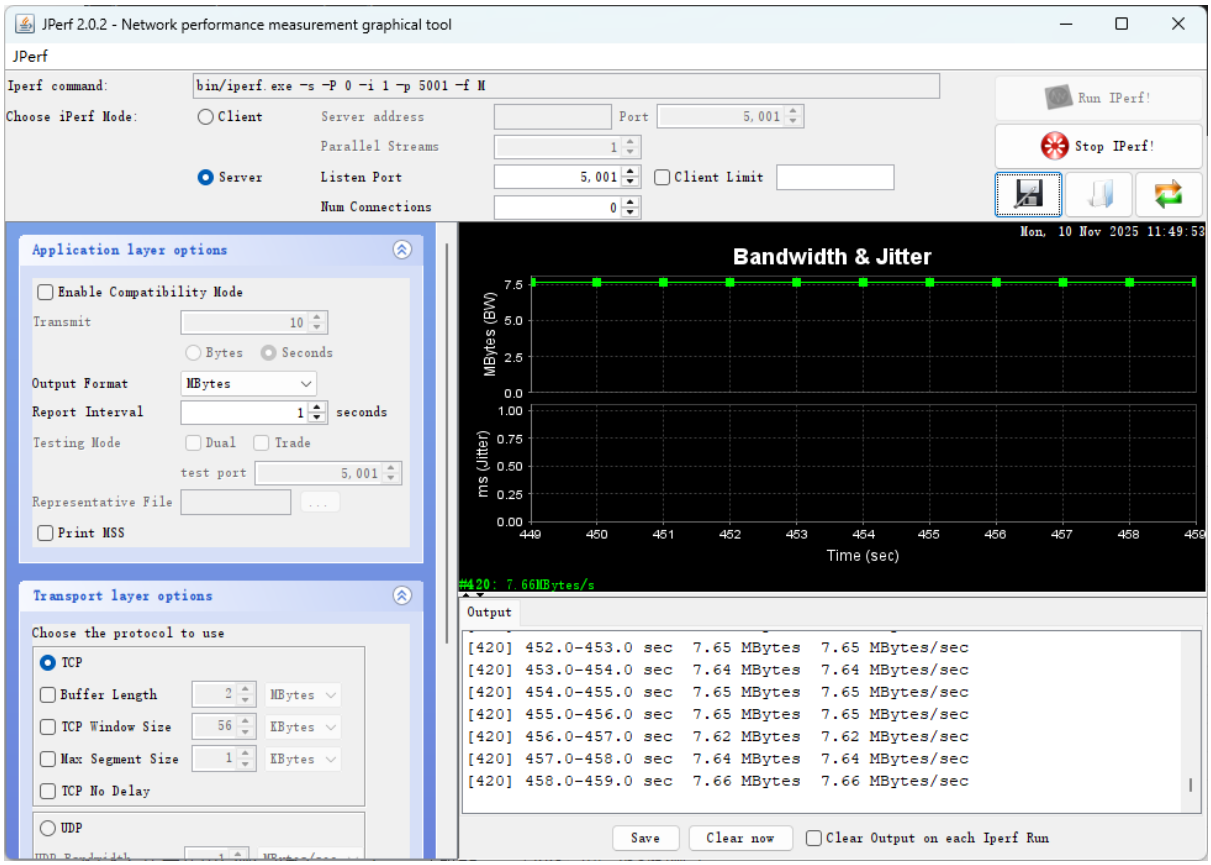
```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Aug 14 2025 15:15:46
 2006 - 2024 Copyright by RT-Thread team
lwIP-2.0.3 initialized!
[I/DBG] Initializing 2 PHY ports...
[I/DBG] PHY0 initialization:
[I/DBG]   Basic Status: 0x7989
[I/DBG]   Control Reg:  0x1000
[I/DBG]   Link Status:  DOWN
[I/DBG]   Auto-Neg:     In Progress
[I/DBG] PHY1 initialization:
[I/DBG]   Basic Status: 0x0000
[I/DBG]   Control Reg:  0x0000
[I/DBG]   Link Status:  DOWN
[I/DBG]   Auto-Neg:     In Progress
[I/DBG] PHY Link Status Summary:
[I/DBG]   PHY0: DOWN
[I/DBG]   PHY1: DOWN
[W/DBG] R_ETHER_Write failed!, res = 4001
[I/sal.skt] Socket Abstraction Layer initialize success.

Hello RT-Thread!
============================================
This example project is an ethernet routine!
============================================
msh />[W/DBG] R_ETHER_Write failed!, res = 4001
[I/DBG] Interrupt: Link ON detected, status: 0x01
[I/DBG] PHY0 Manual Link status changed: UP
[I/DBG] PHY0 link up

msh />if
ifconfig
msh />ifconfig
network interface device: e0 (Default)
MTU: 1500
MAC: 00 11 22 33 44 55
FLAGS: UP LINK_UP INTERNET_DOWN DHCP_ENABLE ETHARP BROADCAST IGMP
ip address: 10.23.8.224
gw address: 10.23.8.254
net mask  : 255.255.255.0
dns server #0: 10.23.8.11
dns server #1: 0.0.0.0
msh />ping baidu.com
ping: not found specified netif, using default netdev e0.
60 bytes from 182.61.201.211 icmp_seq=1 ttl=51 time=29 ms
60 bytes from 182.61.201.211 icmp_seq=2 ttl=51 time=28 ms
60 bytes from 182.61.201.211 icmp_seq=3 ttl=51 time=28 ms
60 bytes from 182.61.201.211 icmp_seq=4 ttl=51 time=28 ms

--- 182.61.201.211 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
minimum = 28ms, maximum = 29ms, average = 28ms
msh />
```

iperf test

Open RT-Thread Settings, add the netutils package and enable the iperf tool.



After compiling and downloading, input iperf -c the ip address of your computer -p 5001 in the serial terminal to conduct the iperf test.



4. GPT Usage Guide

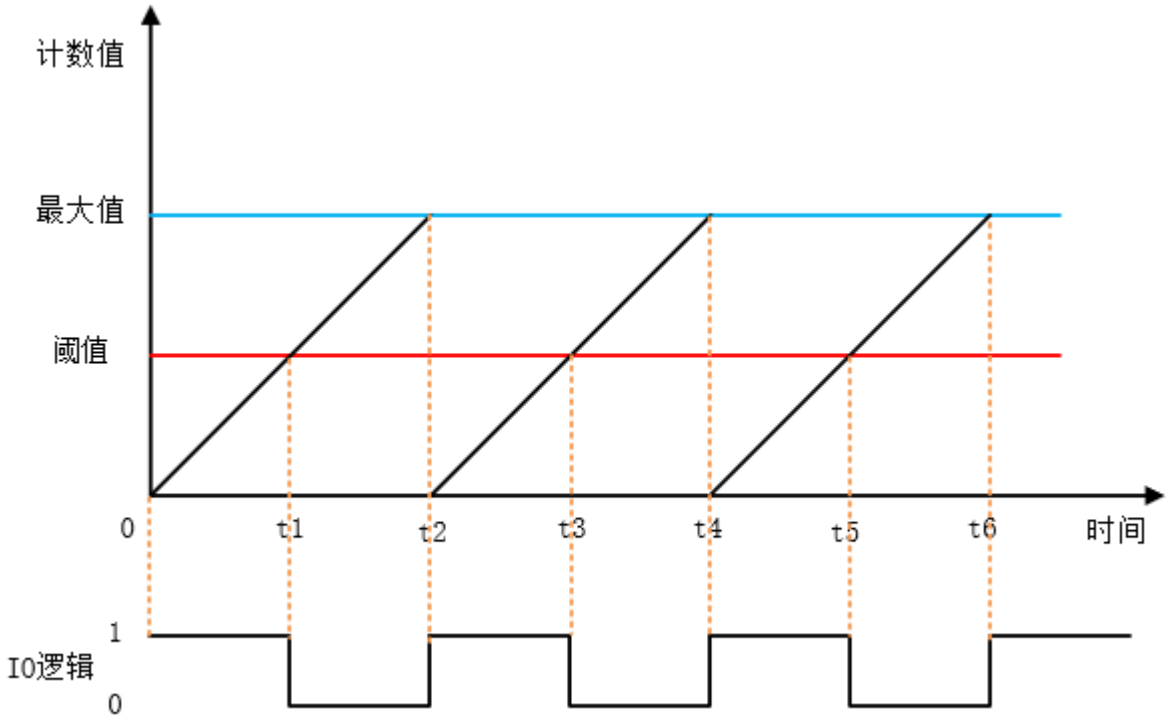
English | Chinese

4.1 Introduction

In our specific application scenarios, the use of timers is often indispensable. This example mainly introduces how to use the GPT device on the Titan Board, including the basic timer usage and PWM usage.

4.2 PWM Introduction

PWM (Pulse Width Modulation) is a method of digitally encoding the level of an analog signal. By generating pulses at different frequencies and varying the duty cycle of a square wave, PWM encodes the level of a specific analog signal. The output produces a series of pulses with equal amplitude, and these pulses can be used by devices to approximate the desired waveform.



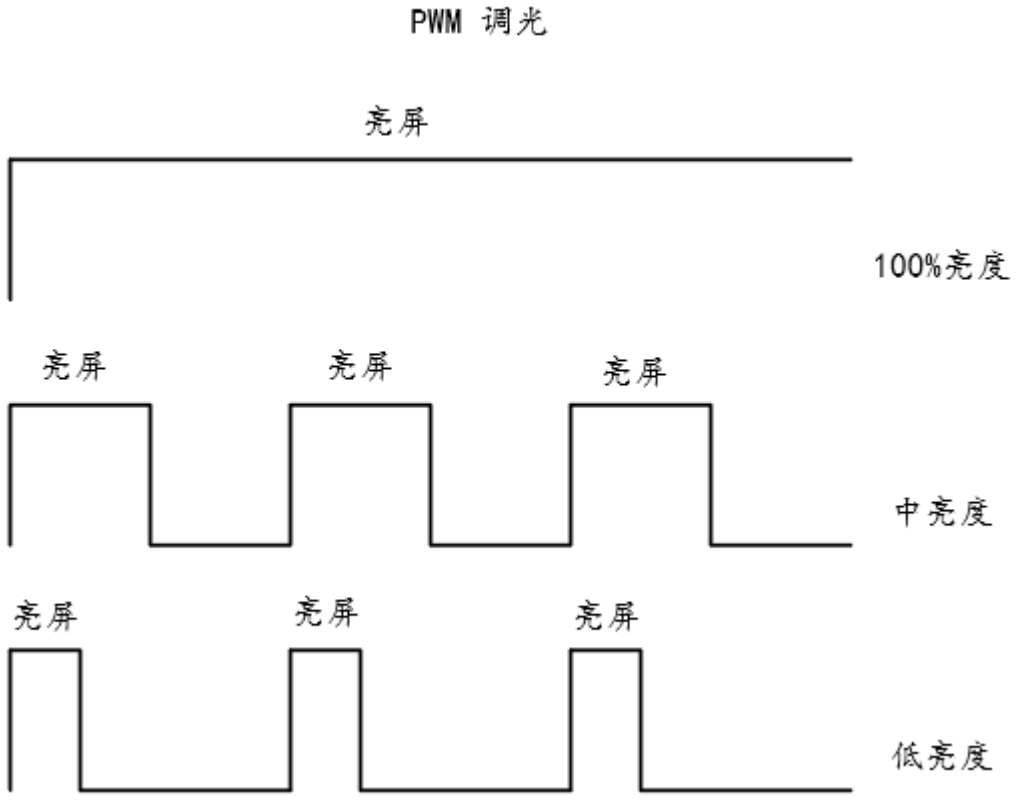
The figure above illustrates a simple principle of PWM. Assume the timer is operating in **up-counting mode**:

- When the counter value is **less than the threshold**, the output is in one logic state (e.g., high level).
- When the counter value **exceeds the threshold**, the output switches to the opposite logic state (e.g., low level).

- Once the counter reaches its maximum value, it resets to 0 and begins counting again, returning to the initial logic state.

The ratio of the **high-level duration (pulse width)** to the **total period** is called the **duty cycle**, which ranges from 0% to 100%. In the example shown above, the high level lasts exactly half of the period, so the duty cycle is 50%.

A common use case for PWM is **controlling the brightness of lights or displays**. By adjusting the duty cycle, the brightness can be controlled. PWM does not keep the light continuously on; instead, it repeatedly turns the screen **on and off** at high speed. When the on/off switching is fast enough, the human eye perceives it as a constant light. During this cycle, if the **off duration is longer**, the screen appears dimmer. Conversely, if the **on duration is longer**, the screen appears brighter.



4.3 Timer Introduction

Overview

A **Timer** is a hardware module in microcontrollers (MCUs) or embedded systems that is used to generate precise time intervals or count events. A timer typically can:

- Generate interrupts at fixed intervals
- Measure time duration or event intervals

- Drive PWM signal generation
- Count external pulses

Timers are core modules for MCU real-time control, PWM output, event counting, periodic sampling, and more.

Types of Timers

Depending on their functionality and applications, timers can be categorized into the following types:

1. **Basic Timer**
   - A simple counter, mainly used for generating fixed interval interrupts
   - Usually supports only up-counting
2. **General Purpose Timer (GPT)**
   - Supports multiple modes: one-shot counting, periodic counting, PWM output, input capture
   - Configurable prescaler, counting mode, and auto-reload value
3. **Advanced Timer**
   - Provides more complex features such as dead-time insertion, synchronization triggers, and complementary PWM output
   - Commonly used in motor control and power electronics applications
4. **Watchdog Timer**
   - Used for system reliability monitoring
   - If the MCU fails to reset the timer within a given time, a system reset or interrupt will be triggered
5. **Real-Time Clock (RTC) Timer**
   - Provides calendar/clock functions
   - Low-power, continues timing in standby mode

Working Principle of Timers

A timer usually consists of the following components:

1. **Counter**
   - Used to accumulate or decrement count values
   - Counting unit is determined by the **Timer Clock Frequency**
2. **Prescaler**
   - Divides the system clock to slow down the counter frequency
   - Formula: Timer Tick Frequency = Timer Clock / (Prescaler + 1)

3. **Auto-Reload Register (ARR / Period Register)**
    - When the counter reaches this value, it triggers an interrupt or event
    - Enables cyclic counting
4. **Interrupt/Event Generation**
    - When the counter matches ARR, it can trigger interrupts, update PWM, or generate external events

Timer Modes

1. **One-Shot Mode**
    - Counter stops after reaching the target value
    - Used for single delays or event triggering
2. **Continuous / Auto-Reload Mode**
    - Counter automatically reloads after reaching the target value and continues counting
    - Used for periodic timers and PWM output
3. **PWM Mode**
    - Uses a compare register to generate PWM signals with adjustable duty cycles
    - Widely used in motor control, LED dimming
4. **Input Capture Mode**
    - Captures timestamps of external signal edges
    - Used to measure pulse width, frequency, or event intervals
5. **Output Compare Mode**
    - Changes output state when the counter matches the compare value
    - Used for timed event triggering or waveform generation

4.4 RA8 Series GPT Module Overview

The Renesas RA8 series microcontrollers integrate a high-performance **General Purpose Timer (GPT)** module that supports multiple timing and control features suitable for various application scenarios.

GPT Module Features

- **Supported Modes**: Periodic mode, One-shot mode, and PWM mode
- **Counting Sources**: Supports PCLK, external trigger pins (GTETRG), GTIOC pins, or ELC events
- **PWM Output**: Supports output of periodic or PWM signals through GTIOC pins
- **Reconfigurable**: Supports runtime reconfiguration of period, duty cycle, and compare values
- **Counting Direction**: Supports up-counting, down-counting, or up/down counting

- **High Resolution**: Supports high-resolution PWM waveform generation, suitable for precision control applications

4.5 RT-Thread PWM Framework Introduction

The RT-Thread PWM (Pulse Width Modulation) framework is a unified interface provided by the RT-Thread device driver layer to manage PWM hardware modules across various MCUs. This framework abstracts the underlying PWM functionality into a standardized device interface, allowing applications to configure period and pulse width through a uniform API and achieve cross-platform PWM control.

Device Model

In RT-Thread, PWM is managed as a **device object** (subclass of struct rt_device, type RT_Device_Class_PWM). Developers do not need to manipulate hardware registers directly. PWM channel configuration, enabling, and disabling can all be performed via the standard RT-Thread device interface.

Operation Interfaces

Applications access PWM devices through RT-Thread's I/O device management interfaces. Key APIs include:

- Find PWM device

```
rt_device_t rt_device_find(const char* name);
```

- Set PWM period and pulse width

```
rt_err_t rt_pwm_set(struct rt_device_pwm *device, int channel, rt_uint32_t
  period, rt_uint32_t pulse);
```

- Enable PWM channel

```
rt_err_t rt_pwm_enable(struct rt_device_pwm *device, int channel);
```

- Disable PWM channel

```
rt_err_t rt_pwm_disable(struct rt_device_pwm *device, int channel);
```

Framework Features

- **Unified Interface**: All PWM hardware modules are accessed through the same interface, simplifying application development.
- **Cross-Platform Support**: Applications can be ported across different MCU platforms without modifying PWM code.

- **Flexible Channel Control**: Supports independent configuration, enabling, and disabling of multiple channels.
- **Accurate Control**: Configurable period and pulse width for high-precision PWM output.
- **High Extensibility**: Can be combined with timers, DMA, and other modules for complex control scenarios.

Runtime PWM Parameter Adjustment

The RA8 GPT module supports runtime adjustment of PWM period and duty cycle.

Example: Set PWM period to 500000 and duty cycle to 70%:

```
#define PWM_DEV_NAME    "pwm12"
#define PWM_DEV_CHANNEL 0

struct rt_device_pwm *pwm_dev = RT_NULL;
rt_uint32_t period = 500000;
rt_uint32_t pulse  = 350000;

pwm_dev = (struct rt_device_pwm *)rt_device_find(PWM_DEV_NAME);
rt_pwm_set(pwm_dev, PWM_DEV_CHANNEL, period, pulse);
rt_pwm_enable(pwm_dev, PWM_DEV_CHANNEL);
```

Reference: RT-Thread PWM Device

4.6 RT-Thread Hardware Timer Framework Introduction

The RT-Thread Hardware Timer (hwtimer) framework provides a unified driver interface for MCU hardware timers. It enables high-precision timing control, making it suitable for periodic task scheduling, event timing, and PWM triggering. Through the hwtimer framework, developers can control timer start/stop, mode configuration, callback setup, and value reading via standardized APIs.

Device Model

In RT-Thread, hardware timers are managed as **device objects** (a subclass of struct rt_device, type RT_Device_Class_Timer). Developers don't need to access hardware registers directly—timer control is done via RT-Thread's unified device interface.

Operation Interfaces

Applications access hwtimer devices via the RT-Thread device management API. Common interfaces include:

- Find timer device

```
rt_device_t rt_device_find(const char* name);
```

- Open timer device (read/write mode)

```
rt_err_t rt_device_open(rt_device_t dev, rt_uint16_t oflags);
```

- Set timeout callback

```
rt_err_t rt_device_set_rx_indicate(rt_device_t dev, rt_err_t (*rx_ind)(rt_device_
  t dev, rt_size_t size));
```

- Control timer (mode/frequency/start/stop)

```
rt_err_t rt_device_control(rt_device_t dev, rt_uint8_t cmd, void* arg);
```

Common command macros:

```
#define HWTIMER_CTRL_FREQ_SET     (0x10)  /* Set timer frequency */
#define HWTIMER_CTRL_MODE_SET     (0x11)  /* Set timer mode */
#define HWTIMER_CTRL_START        (0x12)  /* Start timer */
#define HWTIMER_CTRL_STOP         (0x13)  /* Stop timer */
```

- Set timeout value and start timer

```
rt_size_t rt_device_write(rt_device_t dev, rt_off_t pos, const void* buffer, rt_
  size_t size);
```

- Get current timer value

```
rt_size_t rt_device_read(rt_device_t dev, rt_off_t pos, void* buffer, rt_size_t
  size);
```

- Close timer device

```
rt_err_t rt_device_close(rt_device_t dev);
```

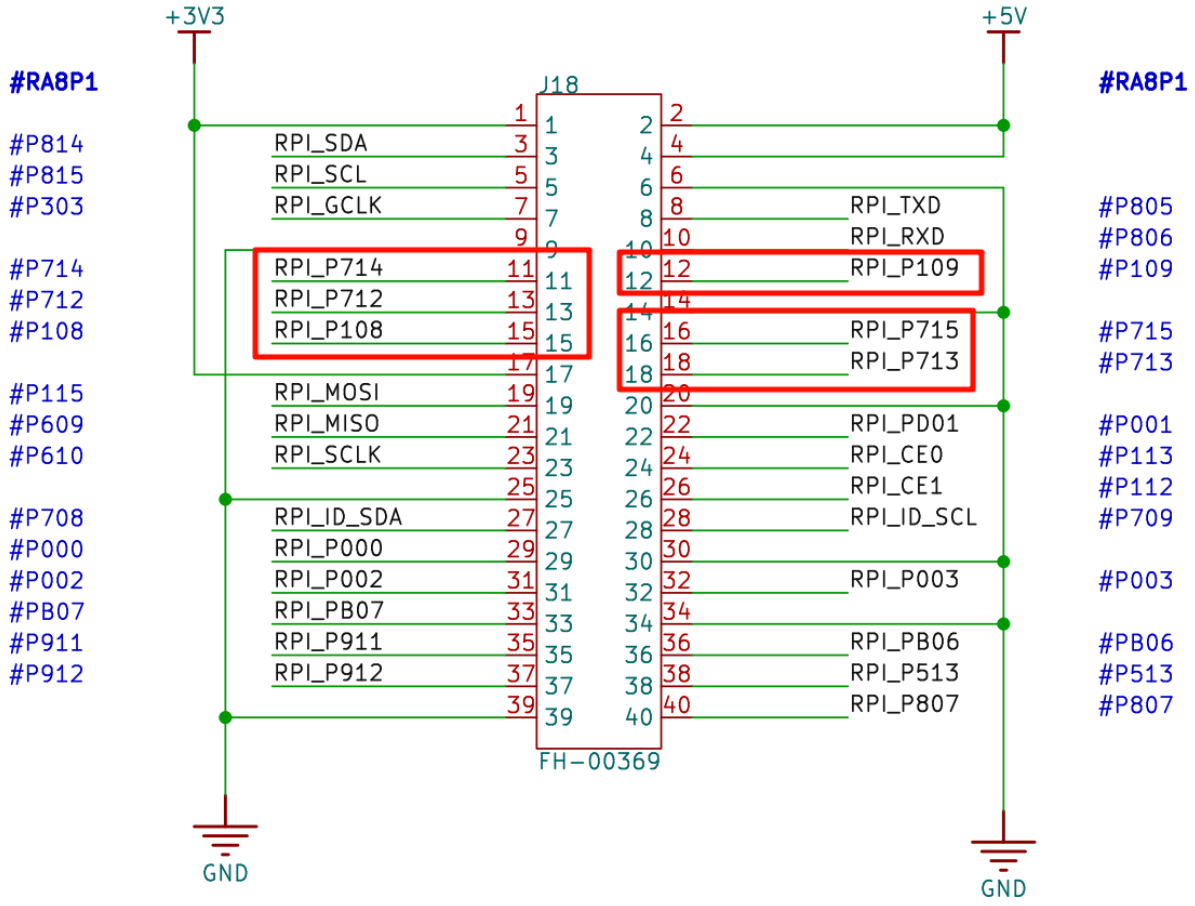
Framework Features

- **Unified Interface**: All hardware timers share the same API for control and configuration.
- **High Precision**: Supports microsecond-level timing for real-time applications.
- **Multiple Modes**: Supports both one-shot and periodic timer modes.
- **Callback Mechanism**: Timeout events can trigger user-defined callback functions.
- **Cross-Platform Compatibility**: Applications can be easily ported across MCU platforms.

Reference: RT-Thread HWTIMER Device

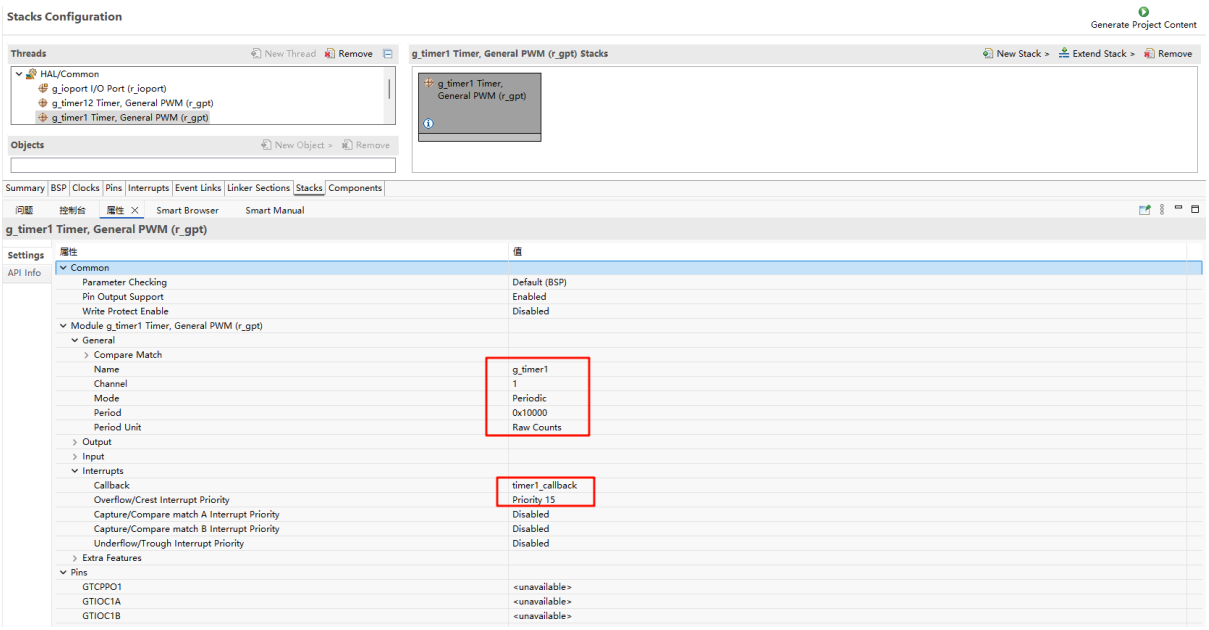
4.7 Hardware Description

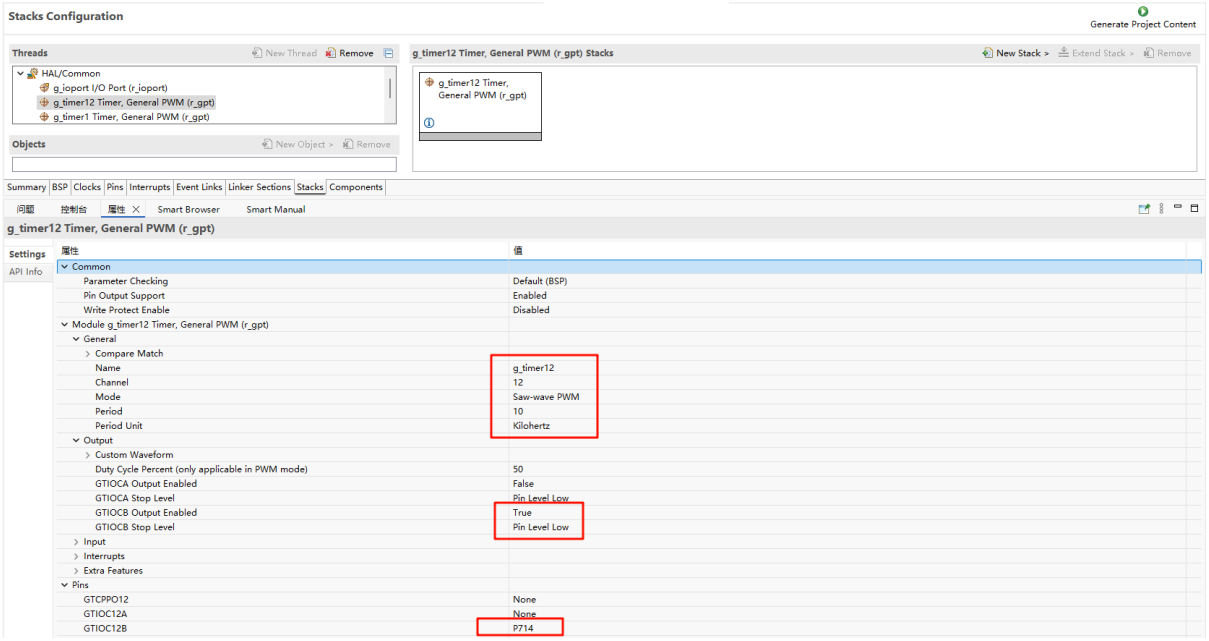
The Raspberry Pi interface of the Titan Board has 6 PWM interfaces. In this example, P714 is used to output PWM waves.



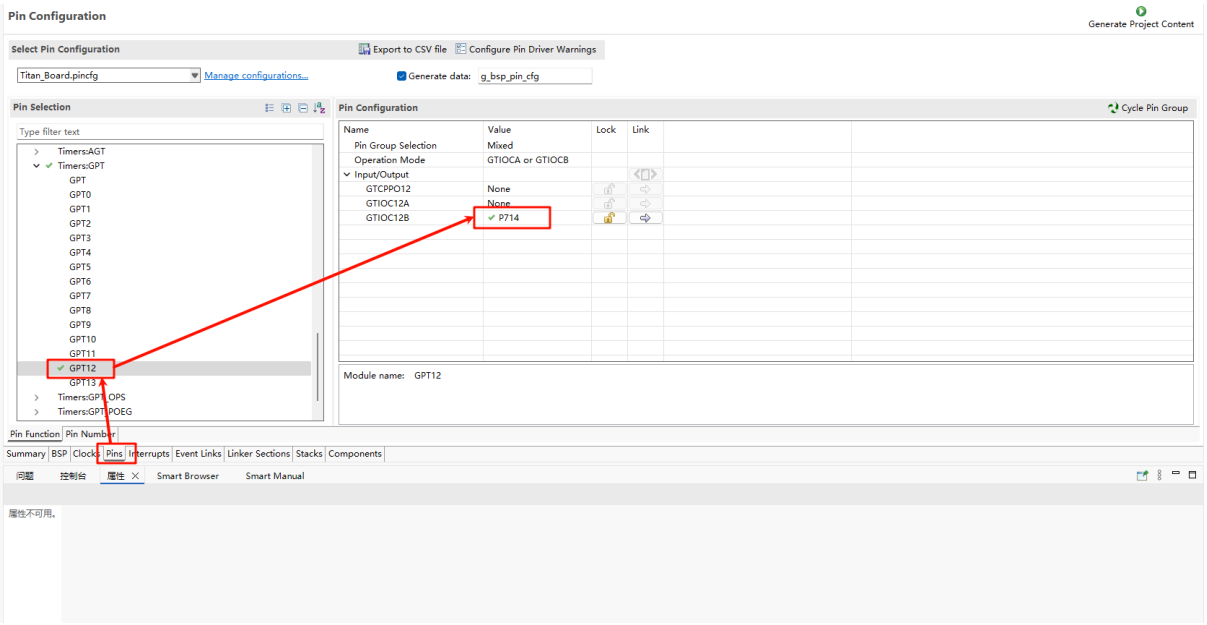
4.8 FSP Configuration

- The FSP is configured to enable GPT1 as a basic timer mode and GPT12 as a PWM mode:



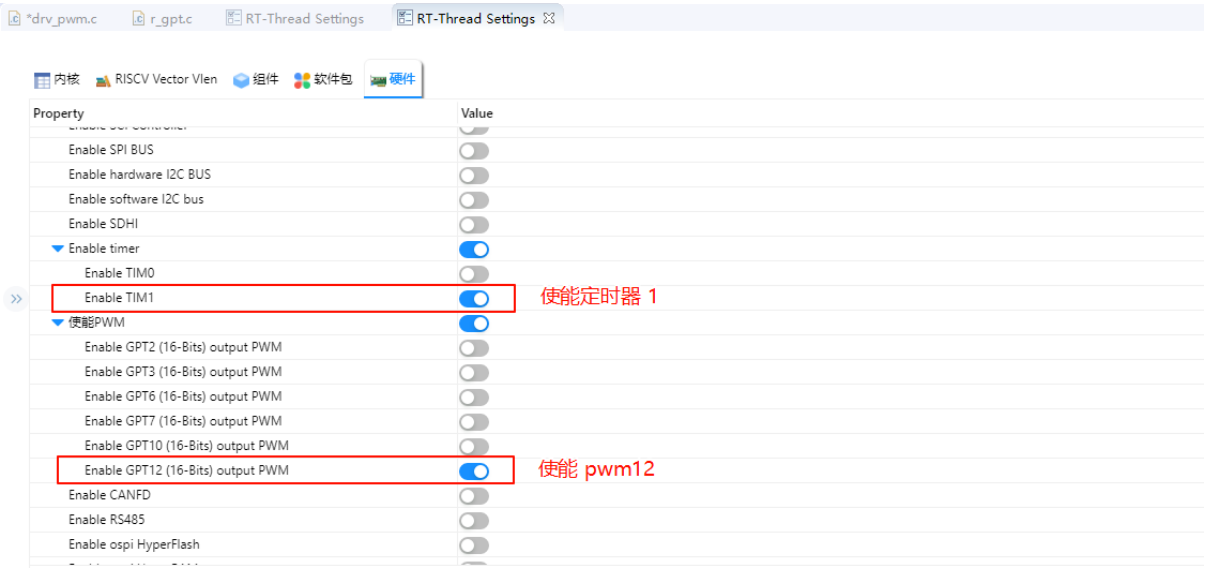


- Additionally, the pins for GPT12 are enabled:



4.9 RT-Thread Settings Configuration

- In the configuration, enable timer1 and PWM12:



4.10 Example Project Instructions

The source code for this example is located in /project/Titan_driver_gpt:

```
/* This is a hwtimer example */
#define HWTIMER_DEV_NAME "timer1" /* device name */
static rt_err_t timeout_cb(rt_device_t dev, rt_size_t size)
{
```

(continues on next page)

(continued from previous page)

```
    rt_kprintf("this is hwtimer timeout callback fucntion!\n");
    rt_kprintf("tick is :%d !\n", rt_tick_get());
    return RT_EOK;
}

int hwtimer_sample(void)
{
    rt_err_t ret = RT_EOK;
    rt_hwtimerval_t timeout_s;
    rt_device_t hw_dev = RT_NULL;
    rt_hwtimer_mode_t mode;

    rt_uint32_t freq = R_FSP_SystemClockHzGet(FSP_PRIV_CLOCK_PCLKD) >> g_timer1_
  cfg.source_div;
    rt_kprintf("GPT Timer clock freq is: %d hz\n", freq);

    hw_dev = rt_device_find(HWTIMER_DEV_NAME);
    if (hw_dev == RT_NULL)
    {
        rt_kprintf("hwtimer sample run failed! can't find %s device!\n", HWTIMER_
  DEV_NAME);
        return -RT_ERROR;
    }

    ret = rt_device_open(hw_dev, RT_DEVICE_OFLAG_RDWR);
    if (ret != RT_EOK)
    {
        rt_kprintf("open %s device failed!\n", HWTIMER_DEV_NAME);
        return ret;
    }

    rt_device_set_rx_indicate(hw_dev, timeout_cb);

    rt_device_control(hw_dev, HWTIMER_CTRL_FREQ_SET, &freq);

    mode = HWTIMER_MODE_PERIOD;
    ret = rt_device_control(hw_dev, HWTIMER_CTRL_MODE_SET, &mode);
    if (ret != RT_EOK)
    {
        rt_kprintf("set mode failed! ret is :%d\n", ret);
        return ret;
    }

    /* Example Set the timeout period of the timer */
    timeout_s.sec = 1;  /* secend */
    timeout_s.usec = 0; /* microsecend */
```

(continues on next page)

(continued from previous page)

```
    if (rt_device_write(hw_dev, 0, &timeout_s, sizeof(timeout_s)) !=
  sizeof(timeout_s))
    {
        rt_kprintf("set timeout value failed\n");
        return -RT_ERROR;
    }

    /* read hwtimer value */
    rt_device_read(hw_dev, 0, &timeout_s, sizeof(timeout_s));
    rt_kprintf("Read: Sec = %d, Usec = %d\n", timeout_s.sec, timeout_s.usec);

    return ret;
}
MSH_CMD_EXPORT(hwtimer_sample, hwtimer sample);
```

The interrupt callback function is triggered every 1 second, printing output. Below is the PWM configuration and enablement:

PWM-related macros:

The current version of the PWM driver treats each channel as a separate PWM device, with each device having only one channel (channel 0). Using the PWM12 device, note that channel 0 is selected here:

```
#define PWM_DEV_NAME        "pwm12" /* PWM device name */
#define PWM_DEV_CHANNEL      0      /* PWM channel */
struct rt_device_pwm *pwm_dev;      /* PWM device handle */
```

Configure the PWM period and duty cycle:

```
static int pwm_sample(int argc, char *argv[])
{
    rt_uint32_t period, pulse;

    if (argc != 3)
    {
        LOG_I("Usage: pwm_sample <period> <pulse>");
        LOG_I("Example: pwm_sample 500000 250000");
        return -RT_ERROR;
    }

    period = (rt_uint32_t)atoi(argv[1]);
    pulse  = (rt_uint32_t)atoi(argv[2]);

    if (period == 0 || pulse > period)
```

(continues on next page)

(continued from previous page)

```
    {
        LOG_E("Error: Invalid parameters. Ensure period > 0 and pulse <= period.
  ");
        return -RT_ERROR;
    }

    pwm_dev = (struct rt_device_pwm *)rt_device_find(PWM_DEV_NAME);
    if (pwm_dev == RT_NULL)
    {
        LOG_E("Error: Cannot find PWM device named '%s'\n", PWM_DEV_NAME);
        return -RT_ERROR;
    }

    if (rt_pwm_set(pwm_dev, PWM_DEV_CHANNEL, period, pulse) != RT_EOK)
    {
        LOG_E("Error: Failed to set PWM configuration.");
        return -RT_ERROR;
    }

    if (rt_pwm_enable(pwm_dev, PWM_DEV_CHANNEL) != RT_EOK)
    {
        LOG_E("Error: Failed to enable PWM output.");
        return -RT_ERROR;
    }

    LOG_I("PWM started on device: %s, channel: %d", PWM_DEV_NAME, PWM_DEV_
  CHANNEL);
    LOG_I("Period: %u ns, Pulse: %u ns", period, pulse);
    LOG_I("Please connect the \'P714\' to a logic analyzer or oscilloscope for
  waveform observation.");

    return RT_EOK;
}
MSH_CMD_EXPORT(pwm_sample, Configure and start PWM output: pwm_sample <period>
  <pulse>);
```

4.11 Compilation & Download

- **RT-Thread Studio**: Download the Titan Board resource package in the RT-Thread Studio package manager, then create a new project and compile it.

After compilation, connect the development board’s USB-DBG interface to the PC, and download the firmware to the development board.

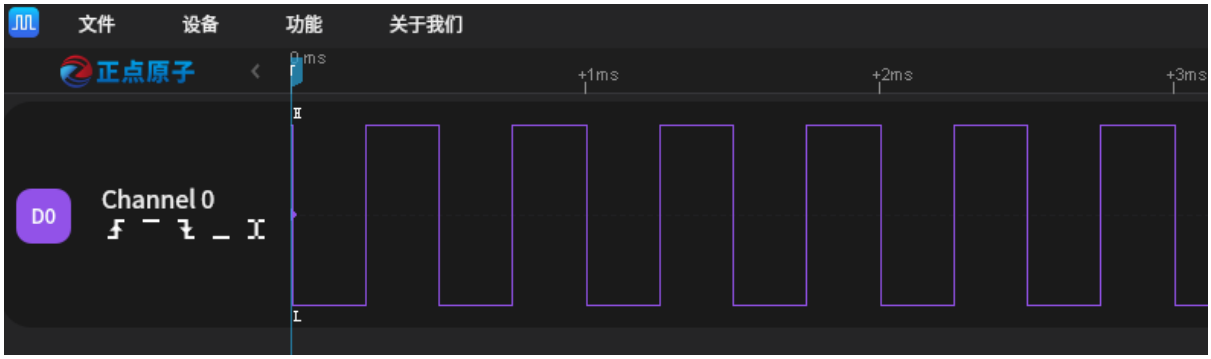
4.12 Running Results

In the serial terminal, input pwm_sample and hwtimer_sample to see the specific results.

The callback function is triggered every 1 second, and the output is printed:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Jul 25 2025 15:18:01
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
================================================
This example project is an pwm(GPT) routine!
================================================
[I/pwm] Tips
[I/pwm] Please enter one of the above commands to run the corresponding sample.
[I/pwm]   • hwtimer_sample : Demonstration of Hardware Timer usage.
[I/pwm]   • pwm_sample     : Demonstration of PWM (Pulse Width Modulation) usage.
msh >pwm_sample 500000 250000
[I/pwm] PWM started on device: pwm12, channel: 0
[I/pwm] Period: 500000 ns, Pulse: 250000 ns
[I/pwm] Please connect the 'P714' to a logic analyzer or oscilloscope for waveform observation.
msh >hwtimer_sample
GPT Timer clock freq is: 250000000 hz
Read: Sec = 1, Usec = 0
msh >this is hwtimer timeout callback fucntion!
tick is :42566 !
this is hwtimer timeout callback fucntion!
tick is :43561 !
this is hwtimer timeout callback fucntion!
tick is :44556 !
this is hwtimer timeout callback fucntion!
tick is :45551 !
this is hwtimer timeout callback fucntion!
tick is :46546 !
this is hwtimer timeout callback fucntion!
tick is :47541 !
this is hwtimer timeout callback fucntion!
tick is :48536 !
```

Using a logic analyzer, the PWM output waveform is measured as follows:



5. HyperRAM Usage Guide

English | Chinese

5.1 Introduction

This example demonstrates how to use the **Octa-SPI (OSPI) interface** on the **Titan Board** to connect to an external **HyperRAM** memory device and perform read/write tests based on the RT-Thread driver framework. Through this example, users can become familiar with the configuration of the RA8 OSPI controller, the operating modes of HyperRAM, and the application flow of accessing external memory under RT-Thread.

5.2 HyperRAM Overview

General Description

HyperRAM is a type of **high-performance, low-pin-count (LPC) DRAM** first introduced by **Cypress (now Infineon)**.

It is based on the **HyperBus interface** and is mainly designed for **embedded systems, display controllers, IoT devices, and automotive electronics** that require **high bandwidth, low power consumption, and a simple interface**.

HyperRAM belongs to the **pSRAM (Pseudo-SRAM)** category. Externally, it behaves like SRAM (simple read/write, no refresh required by the user), but internally it is implemented with low-power DRAM (self-refresh).

Architecture and Interface

HyperRAM uses the **HyperBus interface**, which has the following features:

- **Low pin count**: typically requires only **13 signal pins** (8-bit data bus + control/clock), significantly reducing PCB complexity compared to traditional SDRAM (dozens of pins).
- **Double Data Rate (DDR) transfer**: data is transmitted on both clock edges to increase bandwidth.
- **Serialized control protocol**: memory is accessed through a command–address–data sequence, simplifying design.

Interface signals include:

- **DQ[7:0]**: 8-bit bidirectional data bus
- **RWDS (Read-Write Data Strobe)**: data strobe for synchronization
- **CLK**: clock input
- **CS#**: chip select
- **RESET#**: reset
- **CKE**: clock enable

Operating Principle

HyperRAM access consists of **command + address + data** phases:

1. **Command phase**
   - The host sends a read/write command and target address.
2. **Latency phase**
   - HyperRAM prepares its internal memory array (latency is configurable).
3. **Data transfer phase**
   - Data is transferred on **DQ[7:0]** in DDR mode, with RWDS providing synchronization.

Internally, it is DRAM with **self-refresh**, but externally it appears as SRAM —no refresh management is needed by the user.

Performance Characteristics

- **Data bus width**: 8 bits
- **Operating voltage**: 1.8 V or 3.0 V (low power design)
- **Data rate**: up to **400 MB/s (200 MHz DDR × 8-bit)**
- **Capacity range**: 32 Mb ~ 512 Mb (4 MB ~ 64 MB)
- **Low power**: supports deep power-down mode, standby current < 10 μA
- **Simple interface**: only 13 pins required for high-speed access

Advantages of HyperRAM

1. **Low pin count**
   - Far fewer pins compared to SDRAM/PSRAM (30+ pins), saving PCB routing.
2. **High bandwidth**
   - DDR interface, bandwidth up to 400 MB/s, suitable for **frame buffer and display refresh**.
3. **Low power consumption**
   - Ideal for battery-powered devices such as IoT and wearables.
4. **Ease of use**
   - Externally behaves like SRAM —simple to use, no refresh required.

Comparison with Other Memories

Feature	HyperRAM	SDRAM / DDR	SRAM	NOR Flash
Interface	HyperBus (13-pin)	Parallel 16–32-bit	Parallel/Serial	SPI/QSPI
Capacity	32 Mb ~ 512 Mb	64 Mb ~ 1 Gb+	Small (Kb~Mb)	4 Mb ~ 2 Gb
Bandwidth	~400 MB/s	~800 MB/s+	~50 MB/s	~100 MB/s
Power	Low	Higher	Low	Low
Use cases	Cache/Frame buffer	System memory	High-speed small memory	Program storage

5.3 RA8 Series OSPI (Octa-SPI) Features

The **OSPI (Octa-SPI)** is a key peripheral module in the RA8 MCU family, designed to connect high-speed external memories such as **Octa Flash, HyperFlash, and HyperRAM**, fulfilling the need for **large-capacity storage, high-speed caching, and XIP execution** in embedded applications.

Protocol Support

RA8 OSPI supports multiple interface protocols, making it highly adaptable:

- **Standard SPI mode**: 1-1-1 (single-line instruction, address, data)
- **Dual-SPI**: 2-2-2 mode
- **Quad-SPI**: 4-4-4 mode
- **Octa-SPI**: 8-8-8 mode (DDR/SDR supported)
- **HyperBus protocol**:
  - Supports **HyperRAM / HyperFlash**
  - DDR data sampling on both edges
  - Throughput up to **333 MB/s**

Data Transfer Capability

- **Maximum frequency**: up to **166 MHz clock**
- **DDR mode**: data transferred on both rising and falling edges (double effective bandwidth)
- **Effective bandwidth**:
  - Octa-SPI DDR mode, 8-bit bus, 166 MHz $\rightarrow$ **333 MB/s**
  - HyperBus mode also achieves **333 MB/s**
- **Burst access**: configurable burst lengths (16/32/64/128 bytes) for higher efficiency

Memory Mapping and Execution

RA8 OSPI allows external memories to be **mapped into the MCU address space**, enabling direct access similar to on-chip SRAM:

- **XIP (Execute-In-Place)**: run code directly from external Flash
- **Mapped memory cache**:
  - Built-in prefetch buffer improves instruction fetch speed
  - Suitable for frequent read access
- **Configurable access modes**:
  - Direct access (command-driven)
  - Memory-mapped access (address-space based)

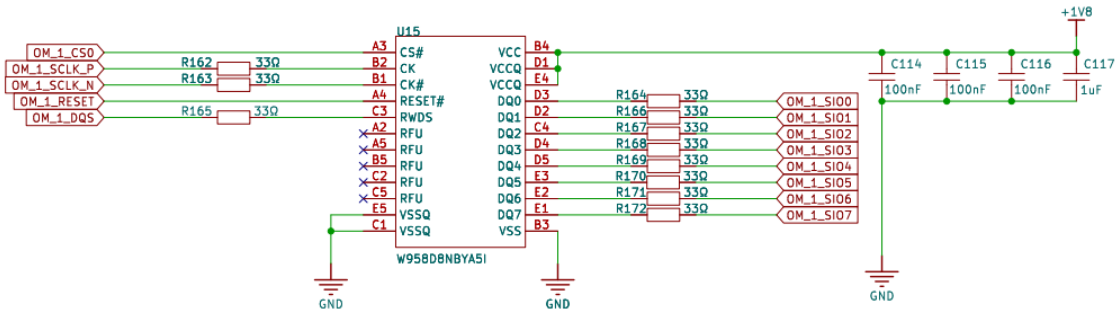
Command and Timing Support

- **Programmable command set**: adaptable for different Flash/HyperRAM vendors
- **DQS (Data Strobe) support**: improves data integrity in DDR mode
- **Chip select control**: multiple CS pins supported for multiple devices
- **Latency adjustment**: configurable clock delay and sampling point to match high-speed HyperRAM/HyperFlash

Low Power and Sleep Management

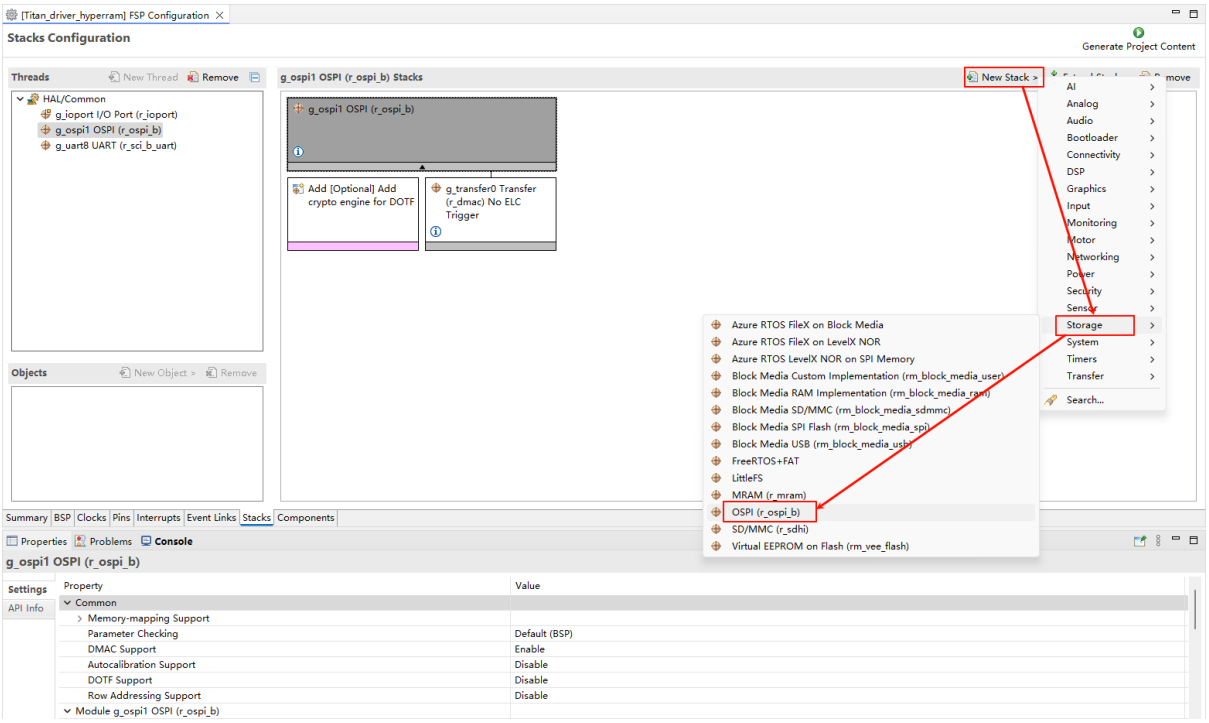
- **Supports peripheral low-power modes**:
  - HyperRAM deep power-down mode
  - HyperFlash standby mode
- **Fast wake-up**: quick recovery of memory access via chip select
- **Battery-powered applications ready**

5.4 Hardware Description

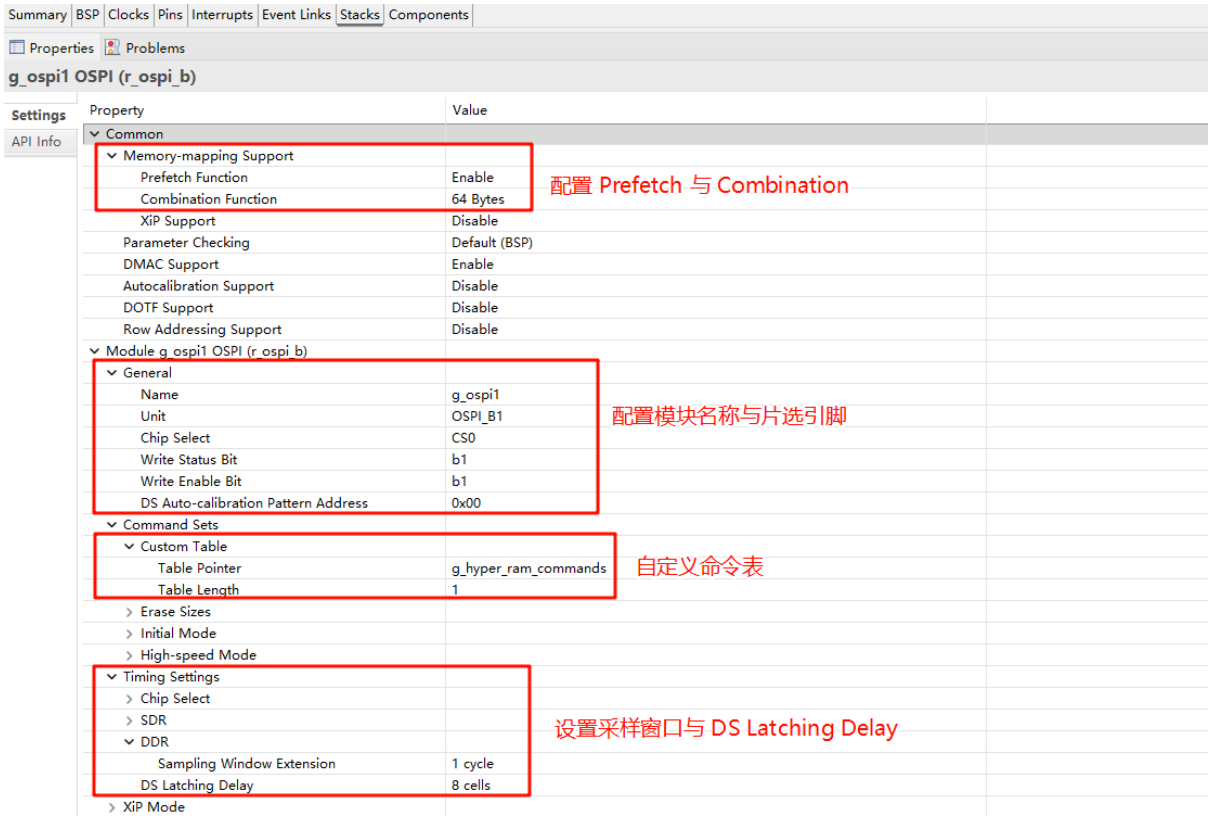


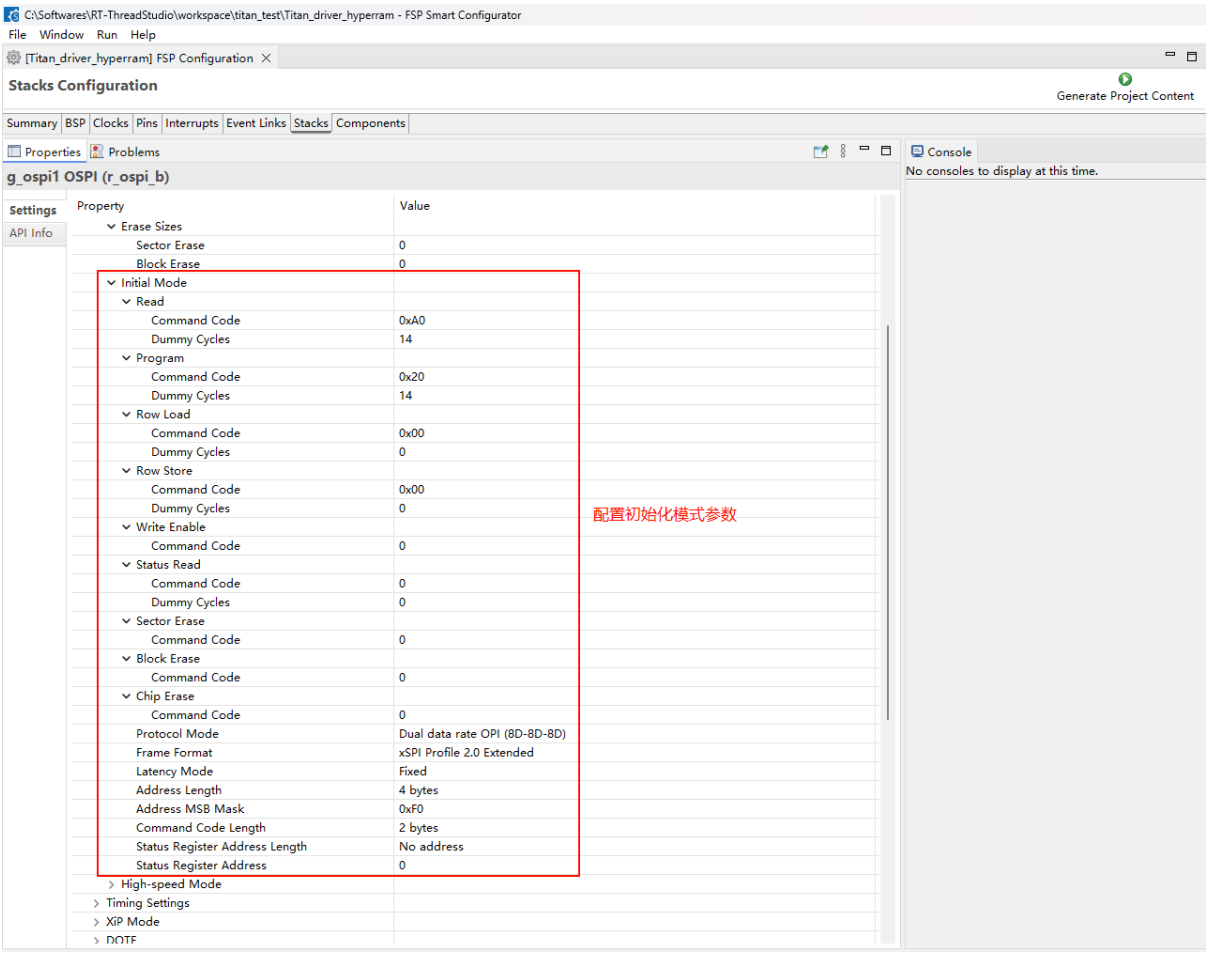
5.5 FSP Configuration

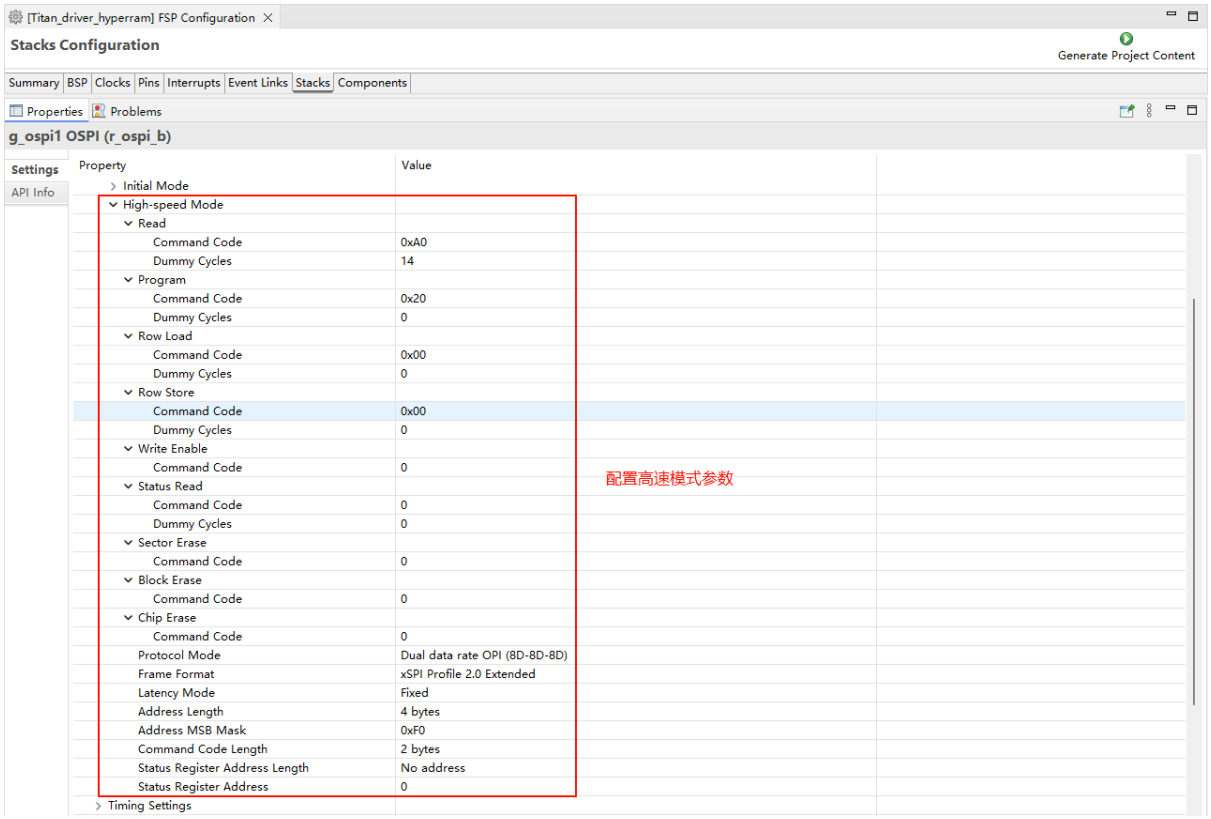
- Create a r_ospi_b stack:



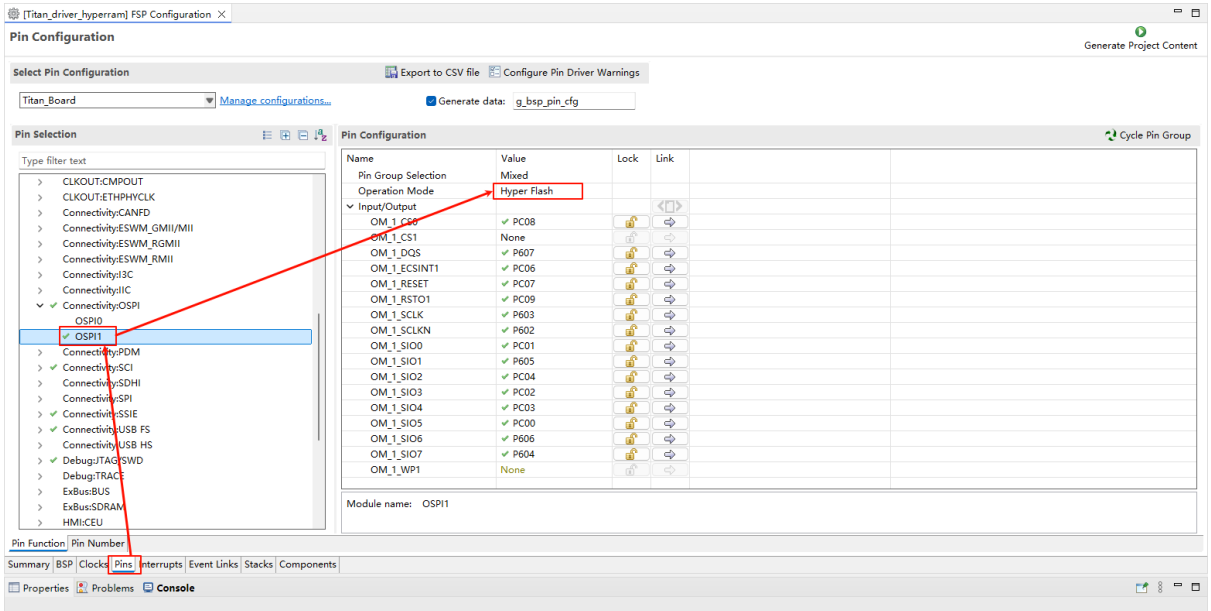
- Configure the r_ospi_b stack:


Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems
g_ospi1 OSPI (r_ospi_b)
Settings
API Info
Property
Value
Common
Memory-mapping Support
Prefetch Function
Enable
Combination Function
64 Bytes
XiP Support
Disable
Parameter Checking
Default (BSP)
DMAC Support
Enable
Autocalibration Support
Disable
DOTF Support
Disable
Row Addressing Support
Disable
Module g_ospi1 OSPI (r_ospi_b)
General
Name
g_ospi1
Unit
OSPI_B1
Chip Select
CS0
Write Status Bit
b1
Write Enable Bit
b1
DS Auto-calibration Pattern Address
0x00
Command Sets
Custom Table
Table Pointer
g_hyper_ram_commands
Table Length
1
Erase Sizes
Initial Mode
High-speed Mode
Timing Settings
Chip Select
SDR
DDR
Sampling Window Extension
1 cycle
DS Latching Delay
8 cells
XiP Mode
配置 Prefetch 与 Combination
配置模块名称与片选引脚
自定义命令表
设置采样窗口与 DS Latching Delay

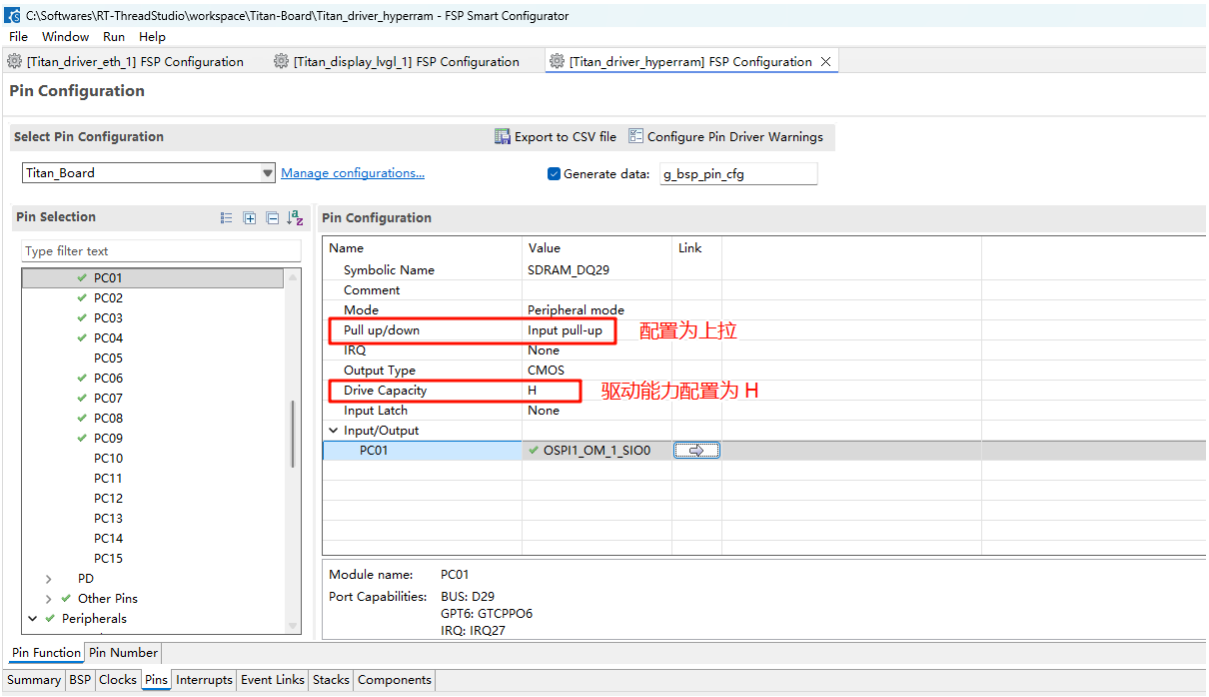

C:\Softwares\RT-ThreadStudio\workspace\titan_test\Titan_driver_hyperram - FSP Smart Configurator
File Window Run Help
[Titan_driver_hyperram] FSP Configuration
Stacks Configuration
Generate Project Content
Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems
g_ospi1 OSPI (r_ospi_b)
Settings
API Info
Property
Value
Erase Sizes
Sector Erase 0
Block Erase 0
Initial Mode
Read
Command Code 0xA0
Dummy Cycles 14
Program
Command Code 0x20
Dummy Cycles 14
Row Load
Command Code 0x00
Dummy Cycles 0
Row Store
Command Code 0x00
Dummy Cycles 0
Write Enable
Command Code 0
Status Read
Command Code 0
Dummy Cycles 0
Sector Erase
Command Code 0
Block Erase
Command Code 0
Chip Erase
Command Code 0
Protocol Mode Dual data rate OPI (8D-8D-8D)
Frame Format xSPI Profile 2.0 Extended
Latency Mode Fixed
Address Length 4 bytes
Address MSB Mask 0xF0
Command Code Length 2 bytes
Status Register Address Length No address
Status Register Address 0
High-speed Mode
Timing Settings
XiP Mode
DOTF
Console
No consoles to display at this time.
配置初始化模式参数



- HyperRAM pin configuration:

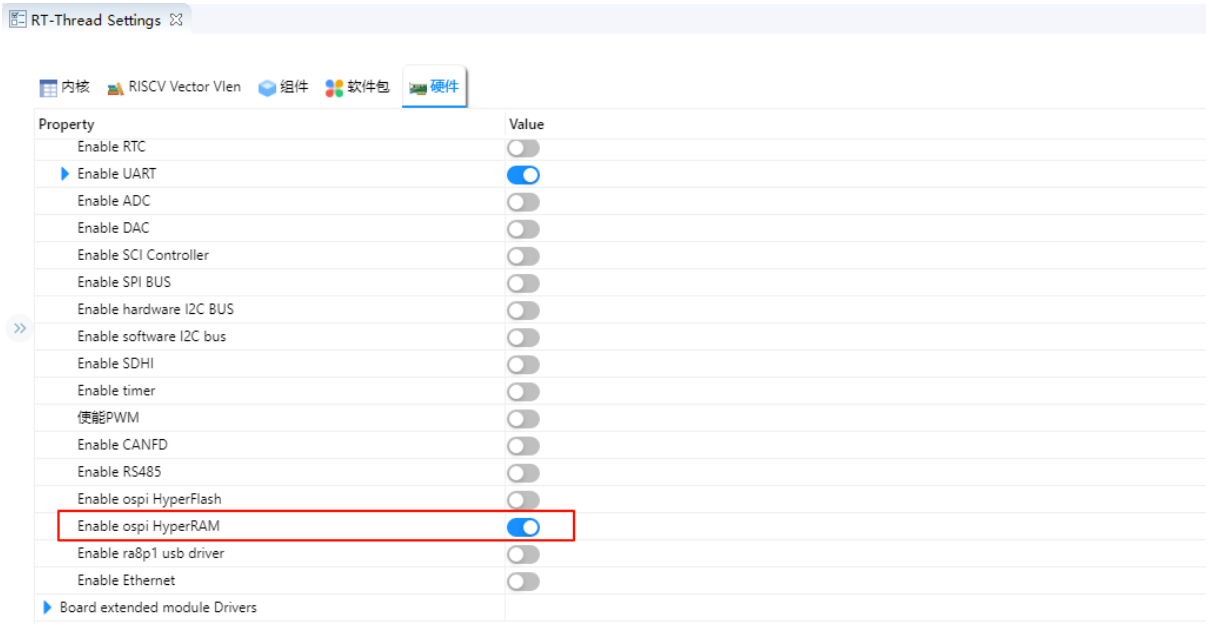


- The drive capability of all pins related to HyperRAM should be configured as H, and OM_1_SIO0 to OM_1_SIO7 need to be configured as Input pull-up.



5.6 RT-Thread Settings Configuration

- Enable HyperRAMin RT-Thread Settings.



5.7 Software Overview

The HyperRAM test source for this routine is located in ./board/ports/hyperram:

```
#include <string.h>
#include <inttypes.h>
#include "hal_data.h"
#include "hyper_ram_test.h"
#include <rtthread.h>

#define DBG_TAG "hyperram"
#define DBG_LVL DBG_INFO
#include <rtdbg.h>

#define ram_cfg     g_ospi1_cfg
#define ram_ctrl    g_ospi1_ctrl

#define OSPI_OM_RESET               BSP_IO_PORT_12_PIN_07
#define HYPER_RAM_RESET_DELAY()     R_BSP_SoftwareDelay(10UL, BSP_DELAY_UNITS_
  MICROSECONDS)

#define HYPER_RAM_CFG_REG_0_ADDRESS (0x01000000)
#define HYPER_RAM_CFG_REG_1_ADDRESS (0x01000001)

ospi_b_xspi_command_set_t g_hyper_ram_commands[] =
{
    {
        .protocol = SPI_FLASH_PROTOCOL_8D_8D_8D,
        .frame_format = OSPI_B_FRAME_FORMAT_XSPI_PROFILE_2_EXTENDED,
        .latency_mode = OSPI_B_LATENCY_MODE_FIXED,
        .command_bytes = OSPI_B_COMMAND_BYTES_1,
        .address_bytes = SPI_FLASH_ADDRESS_BYTES_4,

        .read_command = 0xA0,
        .read_dummy_cycles = 11,
        .program_command = 0x20,
        .program_dummy_cycles = 11,

        .address_msb_mask = 0xF0,
        .status_needs_address = false,

        .p_erase_commands = NULL,
    }
};

uint16_t swap16(uint16_t value)
{
    uint16_t ret;
```

(continues on next page)

(continued from previous page)

```
    ret  = value << 8;
    ret |= value >> 8;
    return ret;
}

static fsp_err_t hyper_ram_config_get(uint32_t address, uint16_t * const p_value_
  out)
{
    spi_flash_direct_transfer_t xfer = {
            .address = address,
            .address_length = 4,
            .command_length = 2,
            .command = 0xE000,
            .data_length = 2,
            .dummy_cycles = 11,
    };

    fsp_err_t err = R_OSPI_B_DirectTransfer(&ram_ctrl, &xfer, SPI_FLASH_DIRECT_
  TRANSFER_DIR_READ);
    if (err != FSP_SUCCESS)
    {
        LOG_E("HyperRAM config get failed!");
        return err;
    }

    *p_value_out = (uint16_t) xfer.data;

    return FSP_SUCCESS;
}

static fsp_err_t hyper_ram_config_set(uint32_t address, uint16_t value)
{
    spi_flash_direct_transfer_t xfer = {
            .address = address,
            .address_length = 4,
            .command = 0x6000,
            .command_length = 2,
            .data = (uint16_t) value,
            .data_length = 2,
            .dummy_cycles = 0,
    };

    fsp_err_t err = R_OSPI_B_DirectTransfer(&ram_ctrl, &xfer, SPI_FLASH_DIRECT_
  TRANSFER_DIR_WRITE);
    if (err != FSP_SUCCESS)
    {
```

(continues on next page)

(continued from previous page)

```
        LOG_E("HyperRAM config set failed!");
        return err;
    }

    return FSP_SUCCESS;
}


/* Define the static array at address 0x70000000 using section attribute */
#define TEST_SIZE 8388608 // 32MB (8M x 4 bytes)
static uint32_t test_array[TEST_SIZE] __attribute__((section(".ospi1_cs0_noinit
  ")));

void hyper_ram_test(void)
{
    const uint32_t test_bytes = TEST_SIZE * 4; // Total bytes: 32MB
    uint32_t errors = 0;
    rt_tick_t start_time, end_time;
    uint32_t write_speed_kbs, read_speed_kbs;
    uint32_t write_time_ms, read_time_ms;

    /* Write pattern to RAM and measure time */
    start_time = rt_tick_get();
    for (uint32_t i = 0; i < TEST_SIZE; i++)
    {
        test_array[i] = i ^ 0xA5A5A5A5; // Use XOR pattern for better error
  detection
    }
    end_time = rt_tick_get();

    /* Calculate write speed in KB/s and time in ms */
    write_time_ms = (end_time - start_time) * 1000 / RT_TICK_PER_SECOND;
    if (write_time_ms > 0)
    {
        write_speed_kbs = (test_bytes / 1024) / write_time_ms * 1000; // KB/s
    }
    else
    {
        write_speed_kbs = 0; // Avoid division by zero
    }

    /* Verify written pattern while measuring read time */
    start_time = rt_tick_get();
    for (uint32_t i = 0; i < TEST_SIZE; i++)
    {
        uint32_t expected = i ^ 0xA5A5A5A5;
```

(continues on next page)

(continued from previous page)

```
        uint32_t actual = test_array[i];
        if (actual != expected)
        {
            errors++;
//            rt_kprintf("errors:%d actual:%d\n", errors, actual);
        }
    }
    end_time = rt_tick_get();

    /* Calculate read speed in KB/s and time in ms */
    read_time_ms = (end_time - start_time) * 1000 / RT_TICK_PER_SECOND;
    if (read_time_ms > 0)
    {
        read_speed_kbs = (test_bytes / 1024) / read_time_ms * 1000; // KB/s
    }
    else
    {
        read_speed_kbs = 0; // Avoid division by zero
    }

    /* Print test results */
    if (errors == 0)
    {
        LOG_I("Hyper RAM test passed successfully!");
    }
    else
    {
        LOG_E("Hyper RAM test failed with %u errors", errors);
    }

    /* Print read and write speeds as integers */
    LOG_I("Write speed: %u KB/s (%u ms)", write_speed_kbs, write_time_ms);
    LOG_I("Read speed: %u KB/s (%u ms)", read_speed_kbs, read_time_ms);
}
MSH_CMD_EXPORT(hyper_ram_test, hyper_ram_test);

int hyper_ram_init(void)
{
    /* Change OM_RESET back to normal IO mode. */
    R_IOPORT_PinCfg(&g_ioport_ctrl,
                    OSPI_OM_RESET,
                    IOPORT_CFG_PORT_DIRECTION_OUTPUT
                        | IOPORT_CFG_DRIVE_HIGH
                        | IOPORT_CFG_PORT_DIRECTION_OUTPUT
                        | IOPORT_CFG_PORT_OUTPUT_HIGH);
```

(continues on next page)

(continued from previous page)

```
    /* Pin reset the OctaFlash */
    R_BSP_PinWrite(OSPI_OM_RESET, BSP_IO_LEVEL_LOW);
    HYPER_RAM_RESET_DELAY();
    R_BSP_PinWrite(OSPI_OM_RESET, BSP_IO_LEVEL_HIGH);
    HYPER_RAM_RESET_DELAY();

    /* Open the interface and immediately transition to 8D-8D-8D mode */
    R_OSPI_B_Open((spi_flash_ctrl_t *)&ram_ctrl, &ram_cfg);
    R_OSPI_B_SpiProtocolSet(&ram_ctrl, SPI_FLASH_PROTOCOL_8D_8D_8D);

    uint16_t cfg_reg0 = 0;
    hyper_ram_config_get(HYPER_RAM_CFG_REG_0_ADDRESS, &cfg_reg0);
    LOG_D("Read CR0 value: 0x%x", swap16(cfg_reg0));

    uint16_t value0 = 0x8f1d;
    hyper_ram_config_set(HYPER_RAM_CFG_REG_0_ADDRESS, swap16(value0));
    LOG_D("Set CR0 to 0x%x", value0);

    cfg_reg0 = 0;
    hyper_ram_config_get(HYPER_RAM_CFG_REG_0_ADDRESS, &cfg_reg0);
    LOG_D("Read CR0 value: 0x%x", swap16(cfg_reg0));

    return RT_EOK;
}
INIT_BOARD_EXPORT(hyper_ram_init);
```

5.8 Build & Download

- **RT-Thread Studio:** Download the Titan Board resource pack from the RT-Thread Studio package manager, then create a new project and compile it.

Once compiled, connect the development board’ s USB-DBG interface to the PC, and download the firmware to the development board.

5.9 Running Result

Press the reset button to restart the Titan Board and enter the hyper_ram_test command to start the HyperRAM test.

```
[D/hyperram] Set CR0 to 0x8f1d
[D/hyperram] Read CR0 value: 0x8f1d

 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Sep 25 2025 10:04:12
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
================================================
This example project is an hyperram routine!
================================================
msh >hyper_ram_test
[I/hyperram] Hyper RAM test passed successfully!
[I/hyperram] Write speed: 183000 KB/s (179 ms)
[I/hyperram] Read speed: 98000 KB/s (334 ms)
msh >
```

6. IST8310 Example Usage Instructions

English | Chinese

6.1 Introduction

This example demonstrates how to use the **I2C interface of the RA8 series MCU** on the **Titan Board** to connect to the **IST8310 three-axis magnetometer sensor** and read data via the **I2C driver framework provided by RT-Thread**. Through this example, users can become familiar with the **configuration of the RA8 I2C Master peripheral** and the workflow for accessing sensors under RT-Thread.

6.2 IST8310 Magnetometer Introduction

Overview

The **IST8310**, produced by **iSentek (Amys Semiconductor)**, is a **three-axis geomagnetic sensor (Magnetometer)** primarily used to measure geomagnetic field strength for **electronic compass (E-Compass) applications**.

It is commonly used in **drones, smartphones, wearable devices, and navigation systems** to provide heading information or detect magnetic fields.

Key Features

- **Three-axis measurement**: Measures X, Y, and Z directions simultaneously
- **Communication interface**: I^2C (up to 400 kHz)
- **Measurement range**: $\pm$1600 µT
- **Resolution**: 0.3 µT
- **Output rate**: Configurable, up to **200 Hz**
- **Low power consumption**: Operating current ~85 µA, standby current <1 µA
- **Supply voltage**: 1.8 V ~ 3.6 V (compatible with common MCUs)
- **Package**: 3 × 3 × 1 mm LGA, compact for embedded applications

Working Principle

The IST8310 detects magnetic fields using the **Hall effect** or **magneto-resistive effect**, with the core process as follows:

1. **Magnetic field detection**
   - The sensing unit detects the geomagnetic field (~50 µT) or other magnetic variations.
2. **Signal conditioning**
   - Internal analog circuits convert the sensed signal into a voltage, then amplify and filter it.
3. **A/D conversion**
   - Built-in ADC converts the analog signal to digital values.

4. **Data output**
    - Outputs X, Y, Z magnetic field strength values via the I^2C bus for the MCU to calculate heading or orientation.

Performance Specifications

- **Sensitivity**: 0.3 μT / LSB
- **Offset error**: Typical ±1 μT
- **Zero-point drift**: Temperature compensation reduces drift
- **Sampling rate**: Configurable from 0.5 Hz to 200 Hz
- **Operating temperature**: -40 ℃ ~ +85 ℃

Advantages and Limitations

Advantages

- Small size, low power consumption, suitable for battery-powered systems
- Digital output, easy MCU integration
- Wide operating voltage range, strong adaptability

Limitations

- External **calibration** required (hard iron/soft iron correction) for accuracy
- Sensitive to temperature and environmental magnetic interference
- Alone, prone to noise; often fused with IMU (accelerometer + gyroscope)

6.3 RA8 Series I2C Master Features

The RA8 series MCU has a built-in **I2C controller**, a high-performance, multi-functional, master/slave-compatible module suitable for reliable, high-speed communication with I2C devices such as sensors, EEPROMs, and RTC chips.

Architecture Overview

The RA8 I2C controller consists of:

- **Master/Slave Control**
    - Supports master, slave, and multi-master modes
    - Automatic bus arbitration and conflict handling
    - Supports 7-bit and 10-bit addressing
- **Clock Generator & Prescaler**
    - Built-in programmable prescaler
    - Supports standard, fast, and high-speed modes

  - Allows precise baud rate configuration
- **Transmit/Receive FIFO**
  - Independent TX and RX FIFOs
  - Reduces CPU intervention, increases data throughput
  - Supports batch read/write for high-speed sensor data acquisition
- **Interrupt Controller**
  - Supports multiple events: transfer complete, arbitration lost, bus error, FIFO empty/full
  - Can be used in interrupt or polling mode
- **DMA Support**
  - TX/RX data can be transferred via DMA
  - Reduces CPU load, lowers power consumption
  - Suitable for high-frequency or large-data I2C operations

Supported I2C Modes

Mode	Speed	Features
Standard Mode	100 kbps	Compatible with standard I2C
Fast Mode	400 kbps	Supports high-speed sensors and EEPROM
Fast Mode Plus	1 Mbps	Increased bus bandwidth, reduced latency
High-Speed Mode	3.4 Mbps	Suitable for high-speed data acquisition
Address Mode	7-bit / 10-bit	Supports standard and extended slave addressing

Bus Management & Multi-Master Features

- **Bus Arbitration**
  - Detects conflicts automatically when multiple masters transmit
  - Follows I2C standard arbitration rules to avoid bus contention
- **Start/Stop Condition Generation**
  - Hardware automatically generates START/STOP signals
  - Supports repeated start conditions
- **ACK/NACK Support**
  - Automatically detects slave responses
  - NACK detection for communication error handling

- **Clock Stretching**
  - Supports slave-held clock low for slow devices
  - Ensures reliable data transfer with slower slaves

Timing and Data Transfer

- **Bidirectional lines (SDA/SCL)**
  - Open-drain, requires external pull-up resistors (4.7kΩ~10kΩ)
  - Supports 1.8V / 3.3V I/O
- **High-speed data transfer**
  - Standard, fast, high-speed modes supported
  - Fast-Mode Plus up to 1 Mbps
  - Multi-master, bidirectional synchronous communication supported
- **FIFO & Burst support**
  - 8~32 byte FIFO for continuous read/write
  - Reduces CPU interrupt frequency

DMA and Interrupt Mechanisms

- **DMA Support**
  - TX/RX FIFOs can be linked to DMA controller
  - Auto-triggered DMA transfer reduces CPU load
  - Ideal for high-frequency sensor data acquisition or continuous streams
- **Interrupt Types**
  - Transfer complete (TX empty / RX full)
  - Arbitration lost (multi-master conflict)
  - Bus error (START/STOP/ACK/NACK errors)
  - FIFO threshold (high/low water mark)

Fault Detection & Tolerance

- Arbitration lost detection for multi-master safety
- Bus error detection for illegal conditions
- Slave non-response handling with configurable retries
- SCL timeout protection to prevent long bus hold

Electrical & Power Features

- **I/O voltage**: Configurable 1.8V / 3.3V
- **Low power mode**: Supports idle bus entry
- **SDA/SCL idle power consumption** very low
- **Deep sleep wake-up**: I2C peripheral can wake the MCU

6.4 RT-Thread I2C Framework Introduction

The RT-Thread I2C (Inter-Integrated Circuit) framework is a unified interface provided by the RT-Thread device driver layer for managing MCU I2C bus devices and communication with slave devices. This framework abstracts I2C bus hardware into a standard device interface, allowing applications to perform master-slave data transmission through a unified API and supporting cross-platform development.

Device Model

In RT-Thread, I2C buses are managed as **device objects** (subclass of struct rt_device, type RT_Device_Class_I2C). Slave devices communicate via the I2C device interface. Developers do not need to manipulate hardware registers directly; all data transfers can be done using standard device interfaces.

Operation Interfaces

Applications access I2C buses through RT-Thread's I/O device interfaces. Key APIs include:

- Find I2C bus device

```
rt_device_t rt_device_find(const char* name);
```

- I2C data transfer

```
rt_size_t rt_i2c_transfer(struct rt_i2c_bus_device *bus,
                          struct rt_i2c_msg         msgs[],
                          rt_uint32_t               num);
```

I2C message structure:

```
struct rt_i2c_msg
{
    rt_uint16_t addr;    /* Slave address */
    rt_uint16_t flags;   /* Read/Write flags */
    rt_uint16_t len;     /* Data length in bytes */
    rt_uint8_t  *buf;    /* Data buffer pointer */
};
```

For simplicity, RT-Thread provides wrapper functions to read/write data to/from I2C slave devices:

- Send data to I2C slave

```
rt_size_t rt_i2c_master_send(struct rt_i2c_bus_device *bus,
                             rt_uint16_t               addr,
                             rt_uint16_t               flags,
                             const rt_uint8_t         *buf,
                             rt_uint32_t               count);
```

- Receive data from I2C slave

```
rt_size_t rt_i2c_master_recv(struct rt_i2c_bus_device *bus,
                             rt_uint16_t               addr,
                             rt_uint16_t               flags,
                             rt_uint8_t               *buf,
                             rt_uint32_t               count);
```

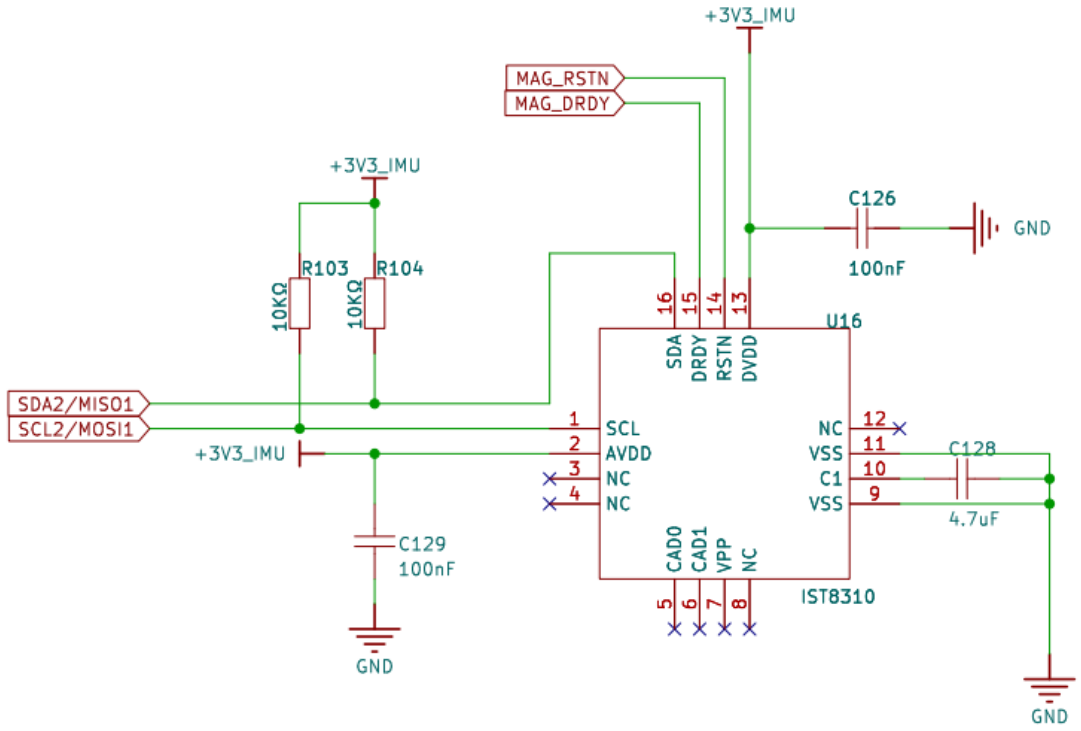
Framework Features

- **Unified Interface**: All I2C bus hardware is exposed to the application through the same interface.
- **Cross-Platform Support**: Applications can be ported across different MCU platforms without modifying I2C code.
- **Flexible Data Transfer**: Supports single or multiple message transfers, including repeated start conditions.
- **Easy-to-Use Wrapper Functions**: rt_i2c_master_send and rt_i2c_master_recv simplify read/write operations.
- **Supports Multiple Transfer Modes**: Can handle standard I2C read/write and complex communication scenarios.

Reference: RT-Thread I2C Bus Device

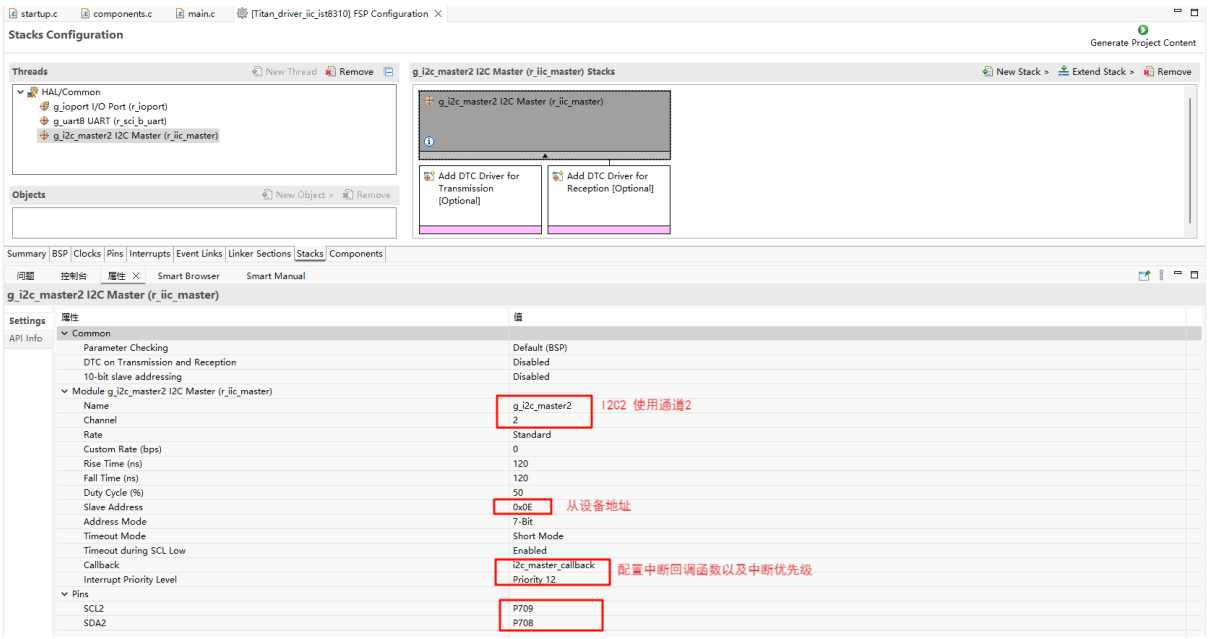
6.5 Hardware Description

Titan Board uses IIC2 to communicate with IST8310.



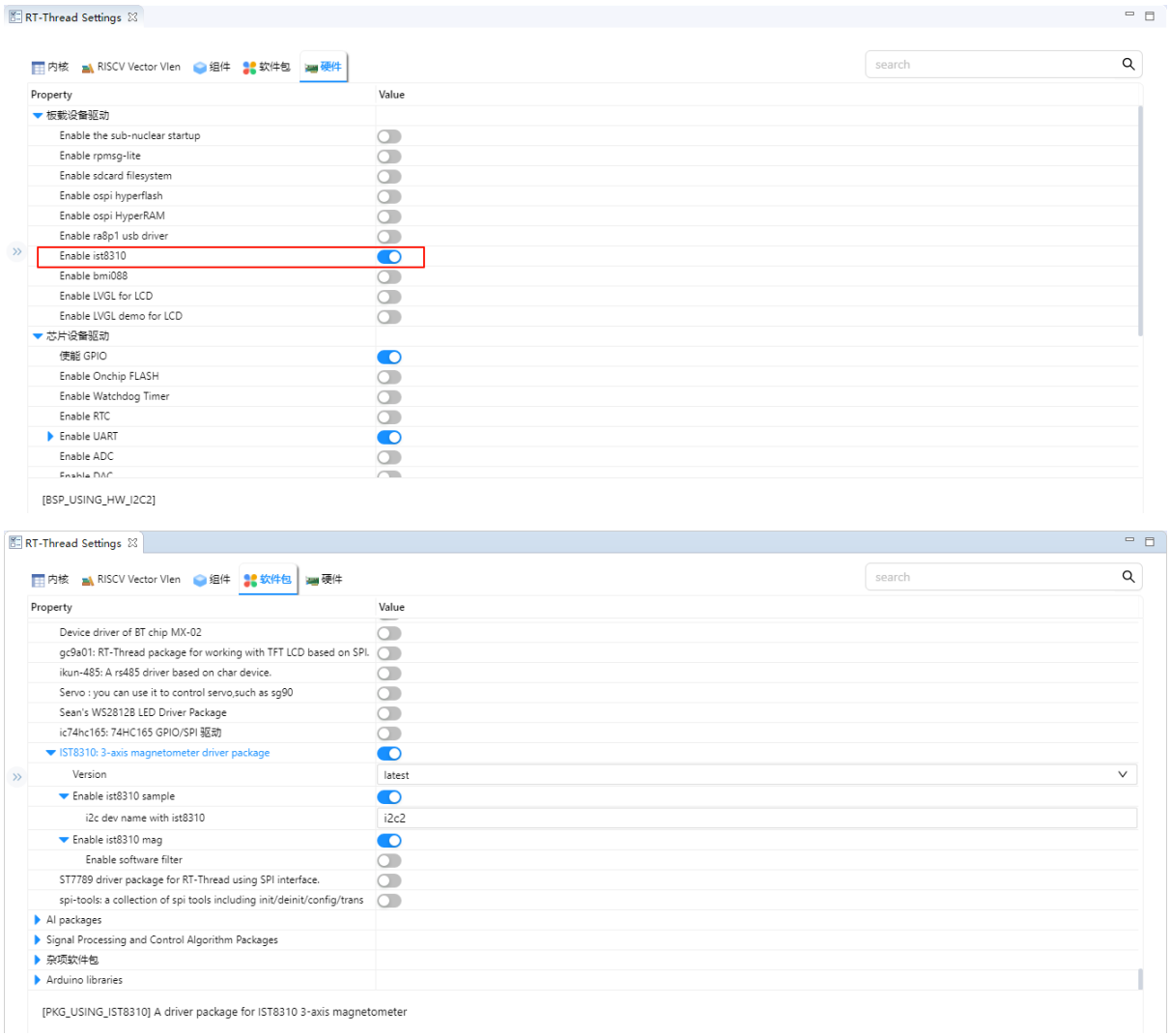
6.6 FSP Configuration

- Create a new stack and select r_iic_master. Then, configure the I2C2 settings as shown below:



6.7 RT-Thread Settings Configuration

- Enable the RT-Thread I2C driver framework and the IST8310 driver software package in the configuration.



6.8 Example Project Description

The magnetometer data communication is realized based on the driver software package IST8310.

```
/*
* Copyright (c) 2006-2025, RT-Thread Development Team
*
* SPDX-License-Identifier: Apache-2.0
*
* Change Logs:
* Date           Author        Notes
```

(continues on next page)

(continued from previous page)

```
* 2025-06-13     kurisaW       first version
*/

#include <rtthread.h>
#include "ist8310.h"

static void ist8310_entry()
{
    ist8310_device_t dev = ist8310_init(IST8310_SAMPLE_I2C_DEV_NAME);
    if (dev == RT_NULL) {
        rt_kprintf("IST8310 init failed\n");
        return;
    }

    /* Set the magnetic declination Angle (set according to the actual position)
  */
    ist8310_set_declination(dev, 0.15f);  /* For example: 0.15 radians */

    while (1)
    {
        ist8310_data_t data;
        if (ist8310_read_magnetometer(dev, &data) == RT_EOK)
        {
            rt_kprintf("Magnetic: X=%.2f μT, Y=%.2f μT, Z=%.2f μT\n", data.x,
  data.y, data.z);
        }

        float heading = ist8310_read_heading(dev);
        rt_kprintf("Heading: %.2f°\n", heading);

        rt_thread_mdelay(1000);
    }
}

void ist8310_app()
{
    rt_thread_t ist8310 = rt_thread_create("ist8310", ist8310_entry, RT_NULL,
  2048, 20, 10);
    if(ist8310 != RT_NULL)
    {
        rt_thread_startup(ist8310);
    }

    return;
}
MSH_CMD_EXPORT(ist8310_app, IST8310 app);
```

6.9 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio's package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board's USB-DBG interface to the PC and download the firmware to the development board.

6.10 Run Effect

Input ist8310_app command in serial port terminal:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Jul 28 2025 09:17:31
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
==================================================
This example project is an sensor routine for IST8310!
==================================================
[I/iic] Tips:
[I/iic] You can run iic testcase by executing the instruction: 'ist8310_app'
msh >ist8310_app
msh >Magnetic: X=0.00 μT, Y=0.00 μT, Z=0.00 μT
Heading: 193.36°
Magnetic: X=1792.00 μT, Y=-21760.00 μT, Z=16128.00 μT
Heading: 193.30°
Magnetic: X=1792.00 μT, Y=-21760.00 μT, Z=16128.00 μT
Heading: 193.36°
Magnetic: X=1792.00 μT, Y=-21760.00 μT, Z=16128.00 μT
Heading: 193.36°
Magnetic: X=1792.00 μT, Y=-21760.00 μT, Z=15872.00 μT
Heading: 193.30°
Magnetic: X=1792.00 μT, Y=-21248.00 μT, Z=16128.00 μT
Heading: 193.30°
Magnetic: X=1792.00 μT, Y=-21760.00 μT, Z=16384.00 μT
Heading: 193.30°
Magnetic: X=1792.00 μT, Y=-21504.00 μT, Z=15872.00 μT
Heading: 193.36°
```

7. MIPI LCD Usage Instructions

English | Chinese

7.1 Introduction

This example demonstrates how to use the **GLCDC module of the RA8 series MCU** together with **RT-Thread' s LCD driver framework** on the **Titan Board** to drive an MIPI LCD screen, enabling image display and interface updates. This document provides a detailed overview of **RA8 GLCDC peripheral features** and the **RT-Thread LCD driver framework**, along with example configurations and usage methods.

The MIPI screen model used for this routine is TL043WVV02CT.

7.2 RA8 Series GLCDC Module

Overview

The **GLCDC (Graphics LCD Controller)** is a high-performance graphics controller integrated in RA8 series MCUs, specifically designed to drive TFT/RGB LCD screens. It supports various resolutions, color formats, and graphics processing functions. Combined with RT-Thread' s **LCD driver framework**, it provides a unified interface for screen initialization, refresh, graphics rendering, and DMA-accelerated operations.

The RA8 **GLCDC** enables image output from either internal MCU memory or external frame buffers to RGB/LCD displays. Key features include:

- **Frame buffer control**: Supports multiple frame buffers for page switching or double-buffered display
- **Color format support**: RGB565, RGB888, ARGB8888, etc.
- **Graphics processing**: Background layers, text/graphics composition, alpha blending, palette mapping
- **Synchronization signal generation**: HSYNC, VSYNC, DE (Data Enable)
- **DMA support**: High-speed data transfer, reducing CPU load
- **Interrupt functionality**: Frame-end and line-end interrupts

Module Architecture

The RA8 GLCDC module mainly consists of the following submodules:

1. **Layer Composition Unit**
    - Supports multiple layer stacking
    - Provides alpha blending, transparency control, and color keying
    - Allows rotation and flipping of layers
2. **Frame Buffer Interface**

- Supports access to MCU internal SRAM or external memory
- Provides single or double buffering to ensure continuous display
- Works with DMA to automatically read image data

3. **DMA Controller**
   - Automatically transfers pixel data to the RGB output port
   - Configurable burst length to improve bandwidth utilization
   - Supports circular transfer, suitable for video or animation scenarios
4. **Timing Generator**
   - Automatically generates HSYNC, VSYNC, and DE signals
   - Supports TTL interface RGB timing
   - Polarity, sync width, and front/back porch timings are configurable
5. **Interrupt and Event Controller**
   - Provides frame-end and line-end interrupts
   - Can be used for page switching, dynamic drawing, or scrolling display
   - Supports interrupts triggered by DMA transfer completion

GLCDC Working Principle

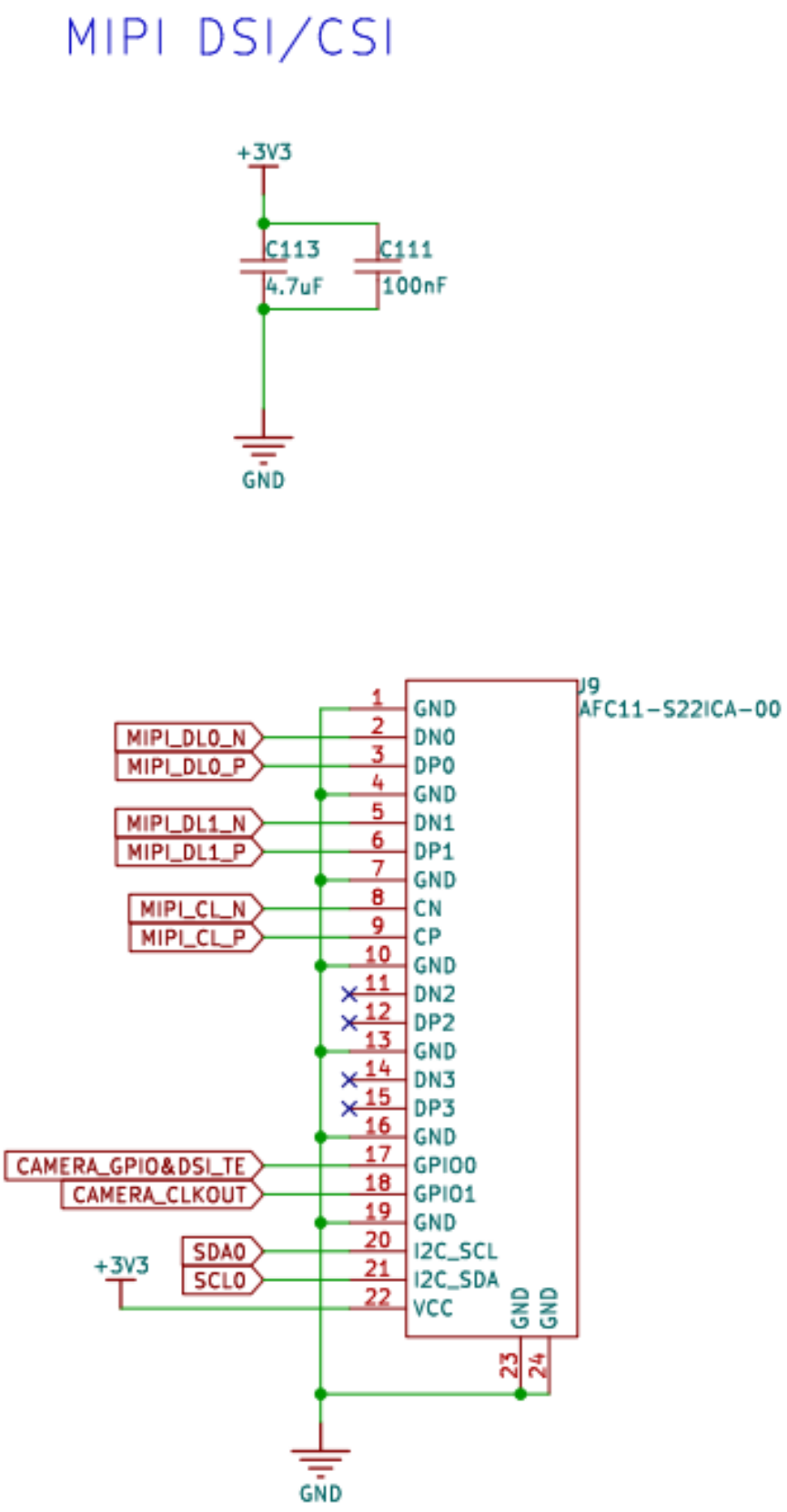
1. **Frame buffer read**
   - GLCDC uses DMA to fetch image data from memory, supporting single or double buffering for continuous display.
2. **Layer composition**
   - Supports multiple layers such as background + foreground + icons/text
   - Provides alpha blending and palette mapping
3. **Pixel timing output**
   - Generates HSYNC/VSYNC/DE signals according to LCD interface requirements
   - Supports RGB parallel interface, TTL interface, or LVDS (depending on board implementation)
4. **Interrupts and events**
   - **Frame-end interrupt (VBlank)**: Can be used to update the next frame
   - **Line-end interrupt**: Useful for scrolling displays or dynamic rendering

GLCDC Supported Features

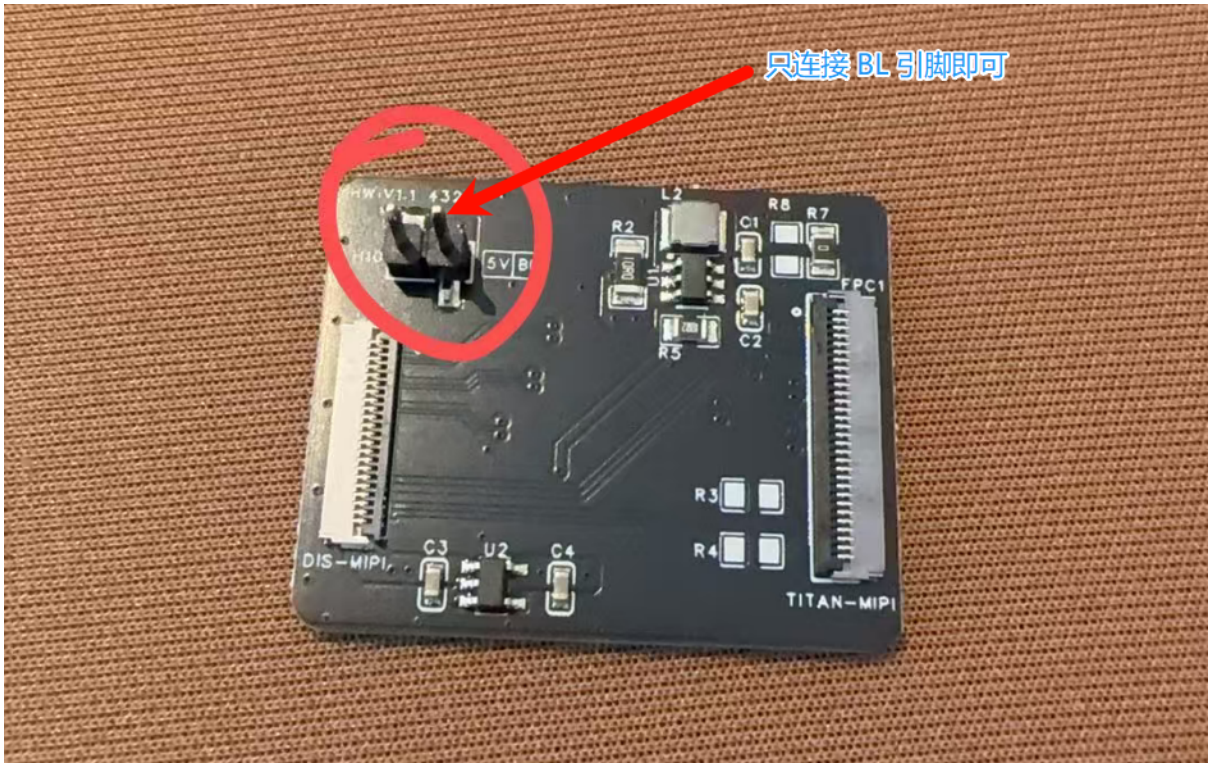
Feature Category	Description
Resolution	Up to 1280x800
Color formats	RGB565, RGB888, ARGB8888, etc.
Multi-layer	Background + foreground + icon layers, supports blending
Frame buffer	Single/double buffer mode, DMA improves performance
Palette	8/16-bit palette mapping for color conversion
Sync signals	HSYNC, VSYNC, DE, configurable polarity and timing
DMA support	Automatic memory transfer, CPU-free
Interrupts	Frame-end, line-end interrupts for synchronized refresh
Rotation/Flip	Supports 90°/180°/270° rotation and X/Y flipping

7.3 Hardware Description

- The following figure shows the MIPI DSI/CSI interface on the Titan Board. To connect the MIPI DSI screen, another adapter board is needed.


MIPI DSI/CSI
+3V3
C113
4.7uF
C111
100nF
GND
U9
AFC11-S22ICA-00
1 GND
2 DN0
3 DP0
4 GND
5 DN1
6 DP1
7 GND
8 CN
9 CP
10 GND
11 DN2
12 DP2
13 GND
14 DN3
15 DP3
16 GND
17 GPIO0
18 GPIO1
19 GND
20 I2C_SCL
21 I2C_SDA
22 VCC
23 GND
24 GND
MIPI_DL0_N
MIPI_DL0_P
MIPI_DL1_N
MIPI_DL1_P
MIPI_CL_N
MIPI_CL_P
CAMERA_GPIO&DSI_TE
CAMERA_CLKOUT
SDA0
SCL0
+3V3
GND

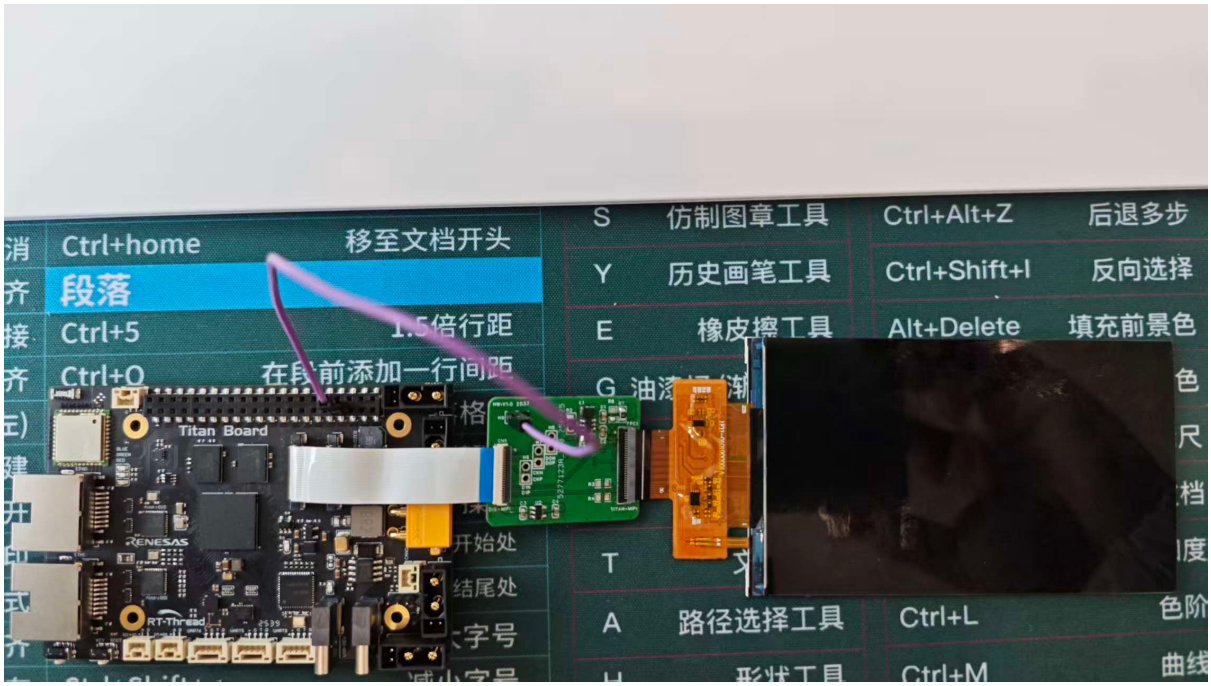
- In this example, a Dupont wire is required to connect the **BL** pin on the display adapter board to the **PB07** pin (the GPIO pin used to provide backlight can be changed as needed).



- The MIPI LCD connection method is as follows:

Display model: **Guanxian TL043WVV02CT**

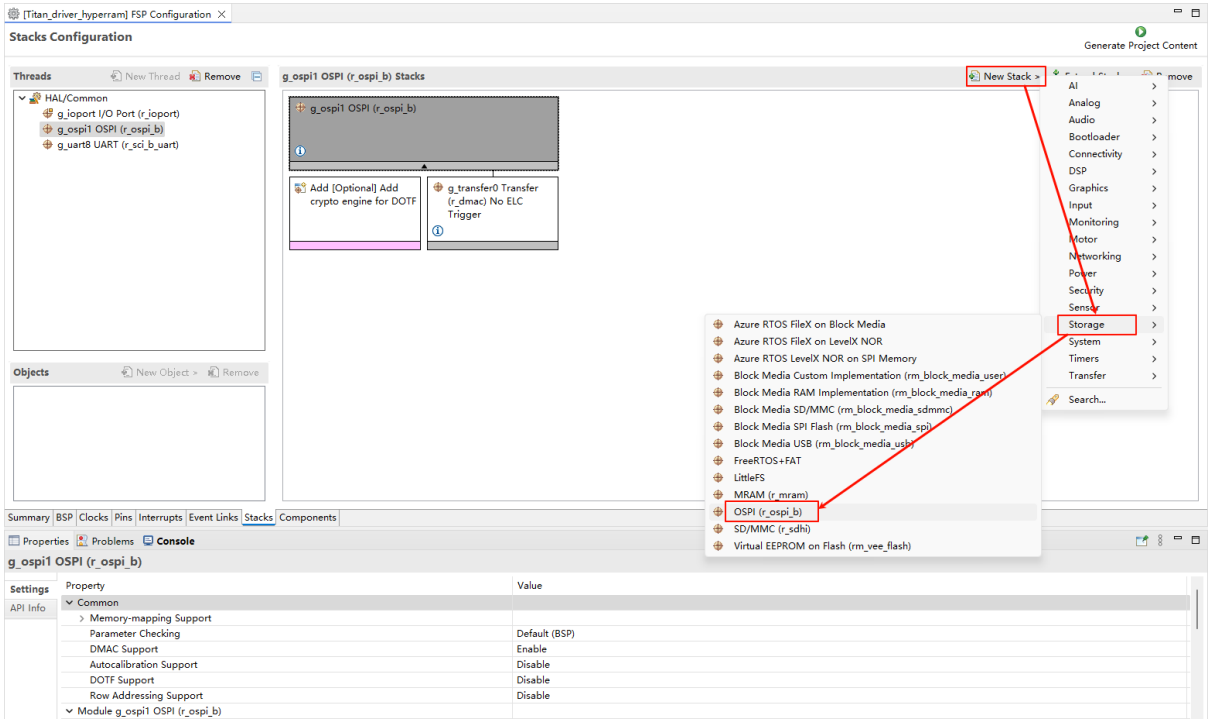
Use a **22-pin reverse FFC cable** to connect the development board's **MIPI DSI/CSI** connector to the display adapter board's **DIS-MIPI** connector. Then connect the **MIPI display** to the adapter board's **TITAN-MIPI** connector.



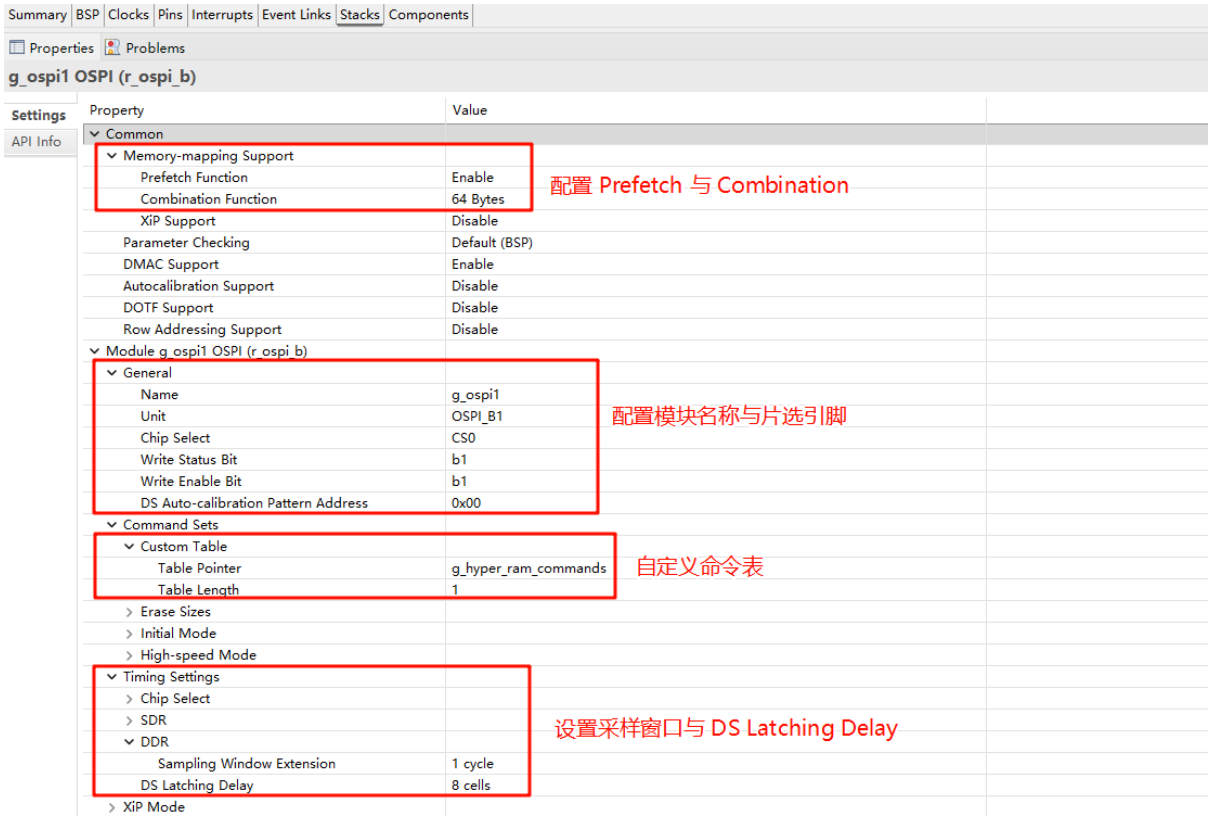
7.4 FSP Configuration

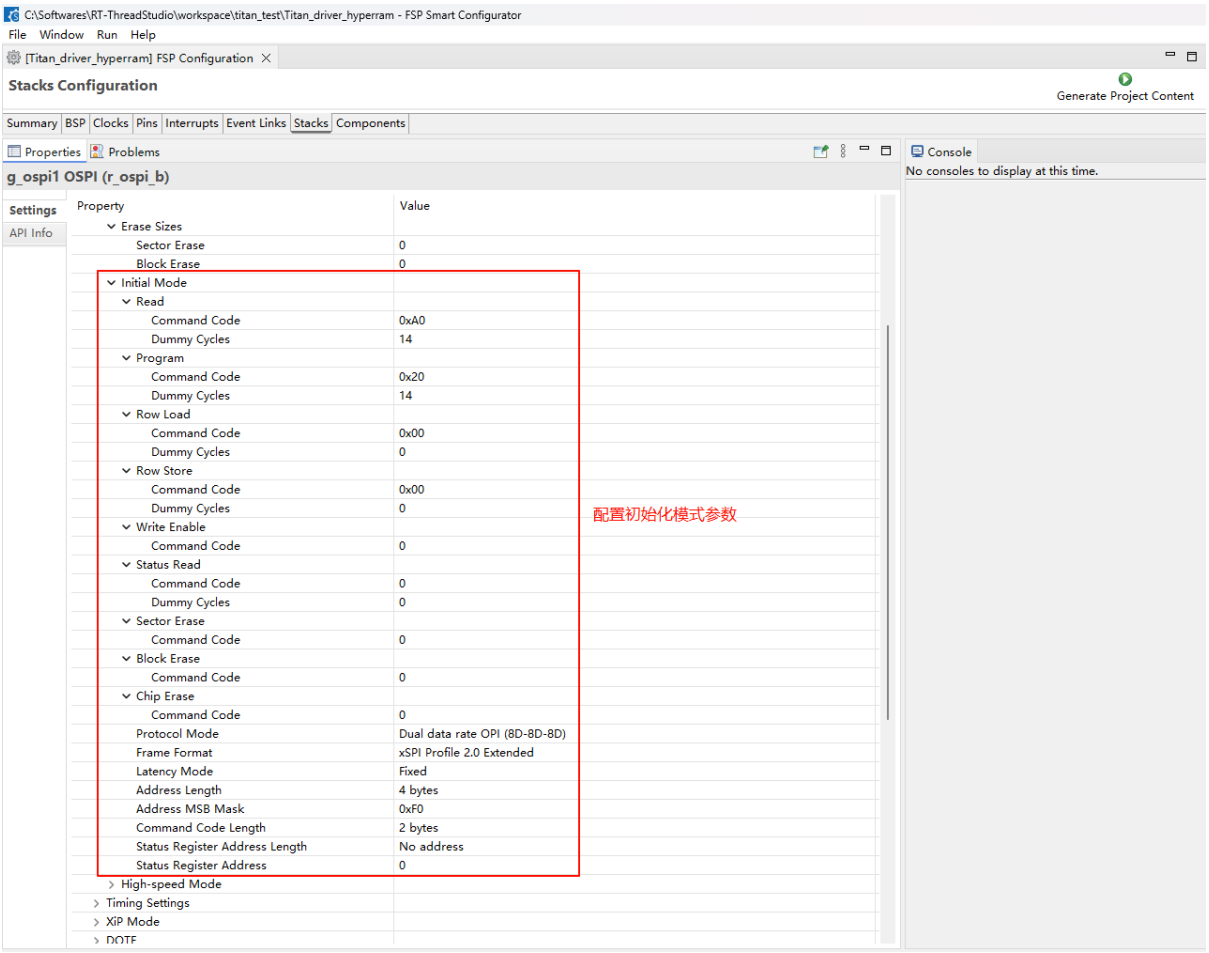
HyperRAM Configuration

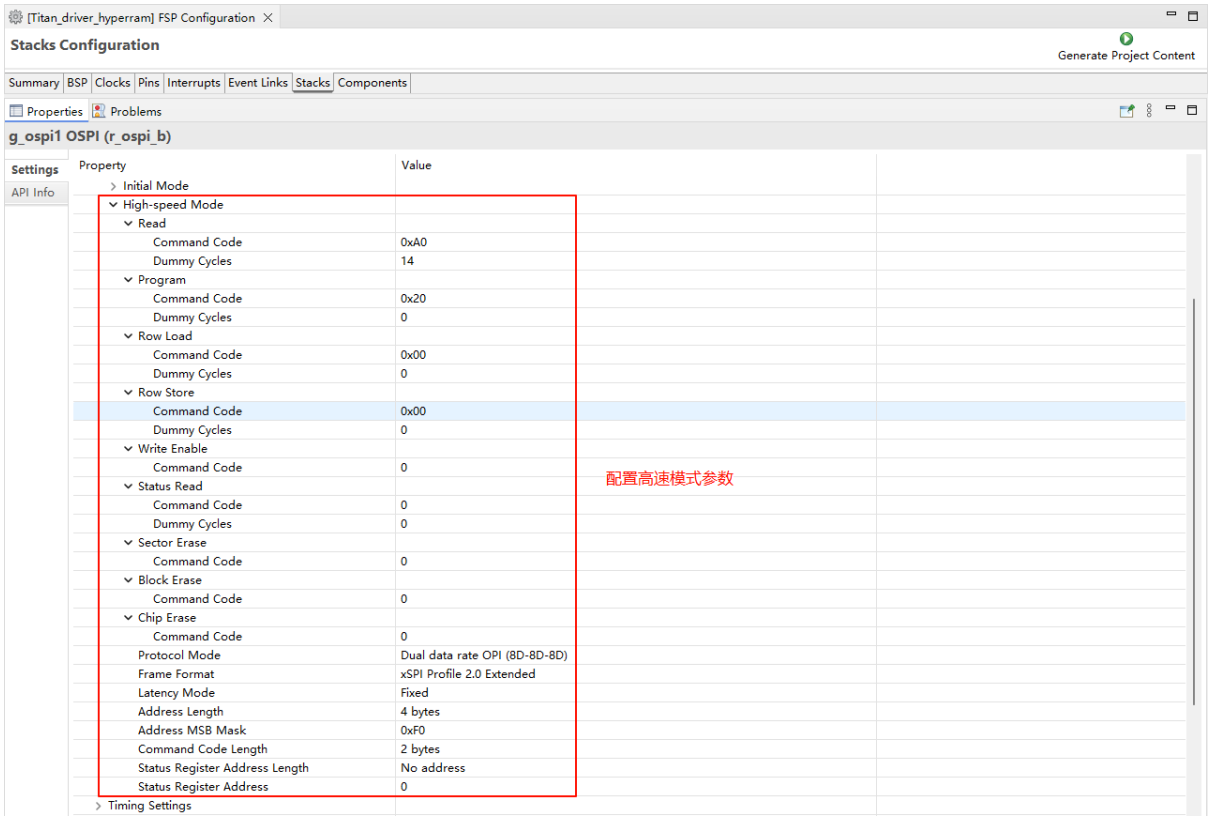
- Create a r_ospi_b stack：



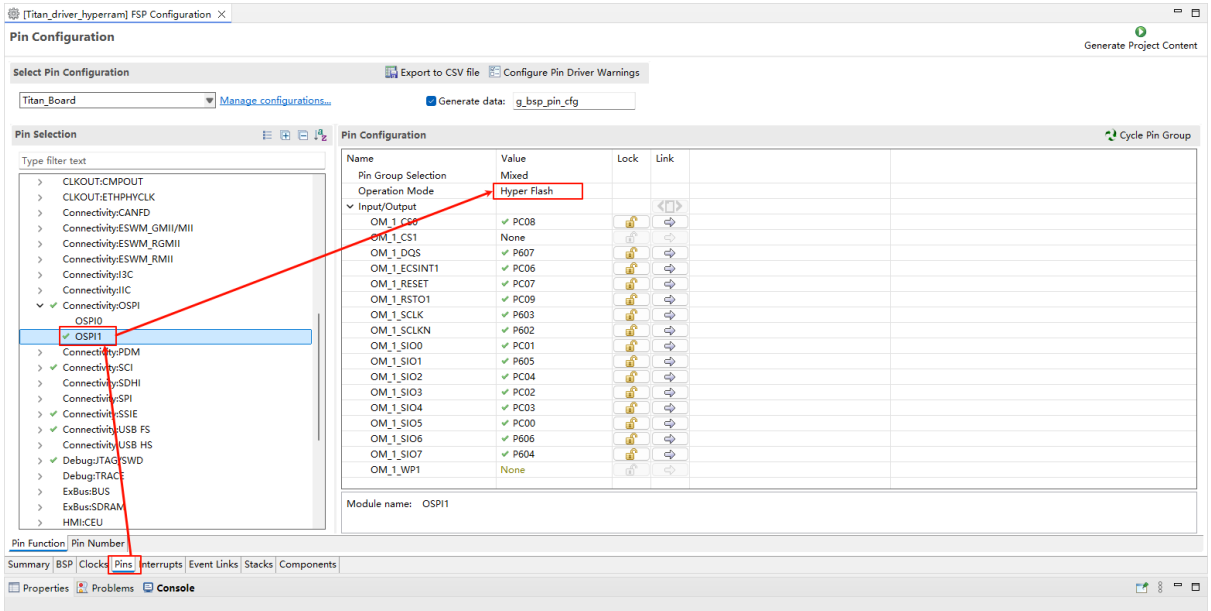
- Configuration r_ospi_b stack：


Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems
g_ospi1 OSPI (r_ospi_b)
Settings
API Info
Property
Value
Common
Memory-mapping Support
Prefetch Function
Enable
Combination Function
64 Bytes
XiP Support
Disable
Parameter Checking
Default (BSP)
DMAC Support
Enable
Autocalibration Support
Disable
DOTF Support
Disable
Row Addressing Support
Disable
Module g_ospi1 OSPI (r_ospi_b)
General
Name
g_ospi1
Unit
OSPI_B1
Chip Select
CS0
Write Status Bit
b1
Write Enable Bit
b1
DS Auto-calibration Pattern Address
0x00
Command Sets
Custom Table
Table Pointer
g_hyper_ram_commands
Table Length
1
Erase Sizes
Initial Mode
High-speed Mode
Timing Settings
Chip Select
SDR
DDR
Sampling Window Extension
1 cycle
DS Latching Delay
8 cells
XiP Mode
配置 Prefetch 与 Combination
配置模块名称与片选引脚
自定义命令表
设置采样窗口与 DS Latching Delay

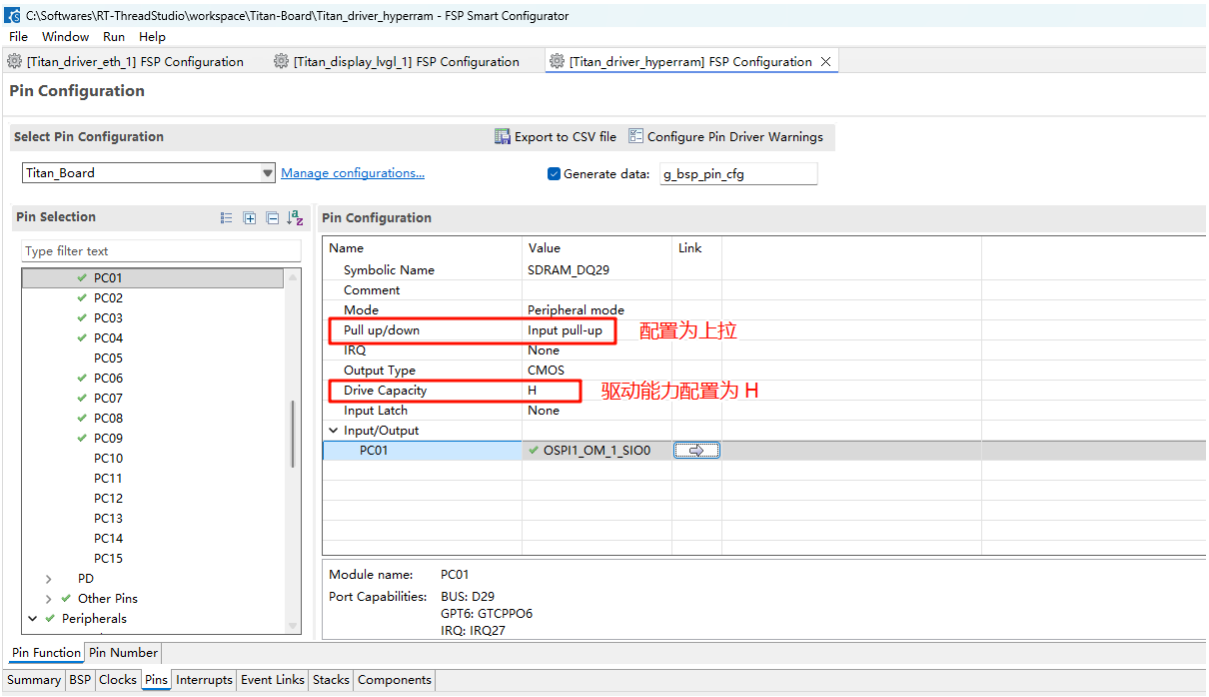

C:\Softwares\RT-ThreadStudio\workspace\titan_test\Titan_driver_hyperram - FSP Smart Configurator
File Window Run Help
[Titan_driver_hyperram] FSP Configuration
Stacks Configuration
Generate Project Content
Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems
g_ospi1 OSPI (r_ospi_b)
Settings
API Info
Property
Value
Erase Sizes
Sector Erase 0
Block Erase 0
Initial Mode
Read
Command Code 0xA0
Dummy Cycles 14
Program
Command Code 0x20
Dummy Cycles 14
Row Load
Command Code 0x00
Dummy Cycles 0
Row Store
Command Code 0x00
Dummy Cycles 0
Write Enable
Command Code 0
Status Read
Command Code 0
Dummy Cycles 0
Sector Erase
Command Code 0
Block Erase
Command Code 0
Chip Erase
Command Code 0
Protocol Mode Dual data rate OPI (8D-8D-8D)
Frame Format xSPI Profile 2.0 Extended
Latency Mode Fixed
Address Length 4 bytes
Address MSB Mask 0xF0
Command Code Length 2 bytes
Status Register Address Length No address
Status Register Address 0
High-speed Mode
Timing Settings
XiP Mode
DOTF
Console
No consoles to display at this time.
配置初始化模式参数



- Configuration HyperRAM pins：

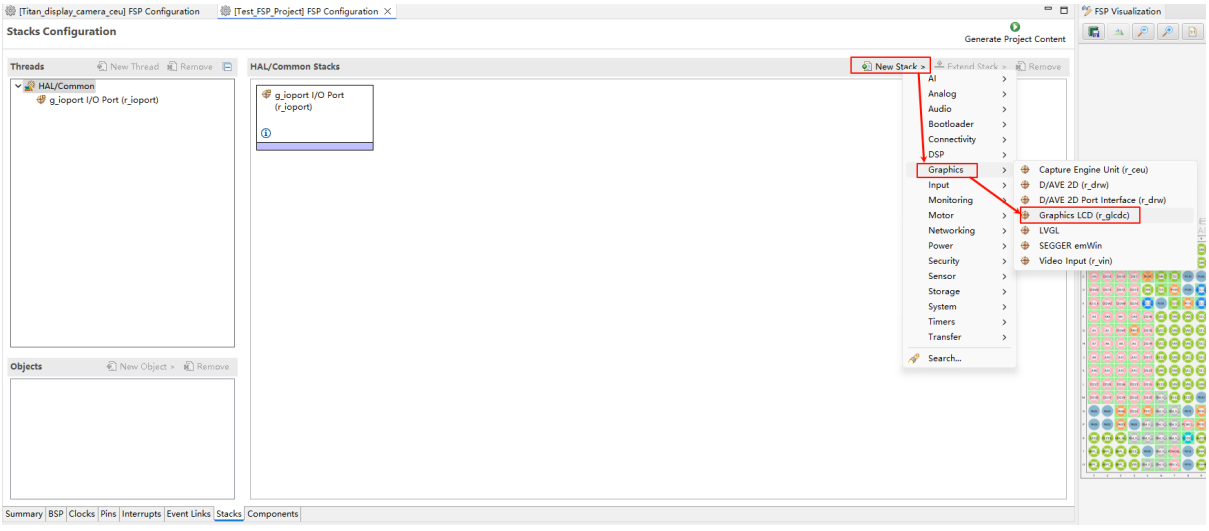


- The drive capability of all pins related to HyperRAM should be configured as H, and OM_1_SIO0 to OM_1_SIO7 need to be configured as Input pull-up.

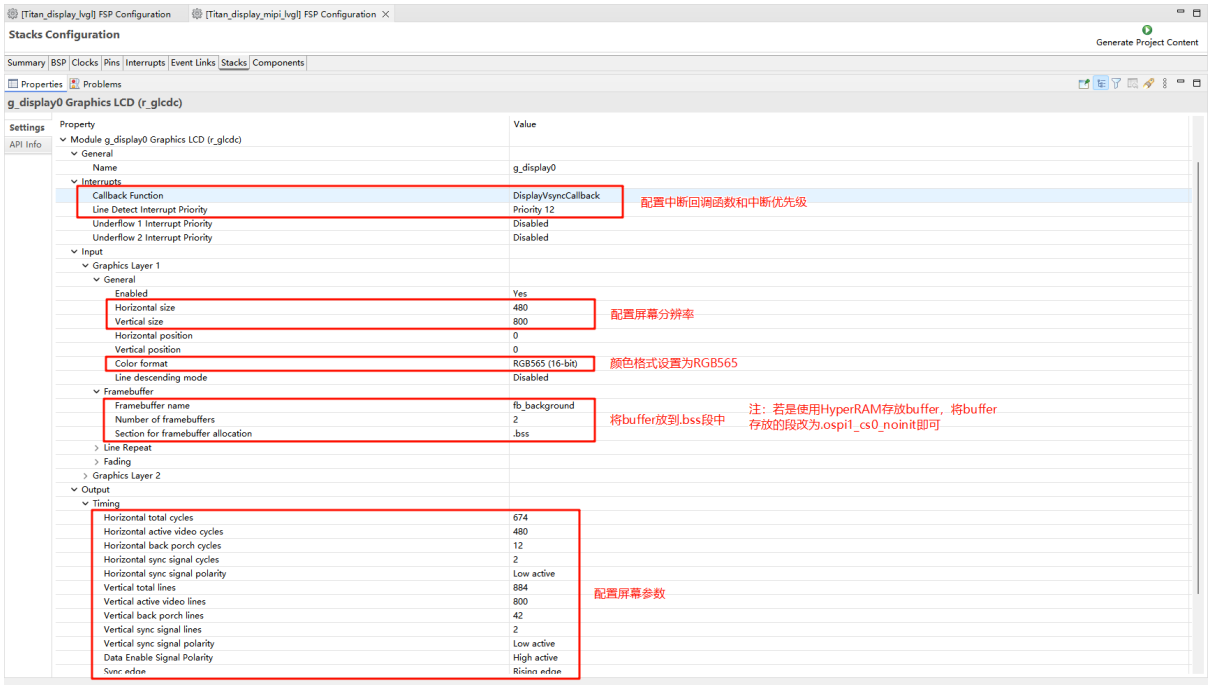


LCD Configuration

- Create a r_glcdc stack:



- Configure interrupt callback and graph layer 1:

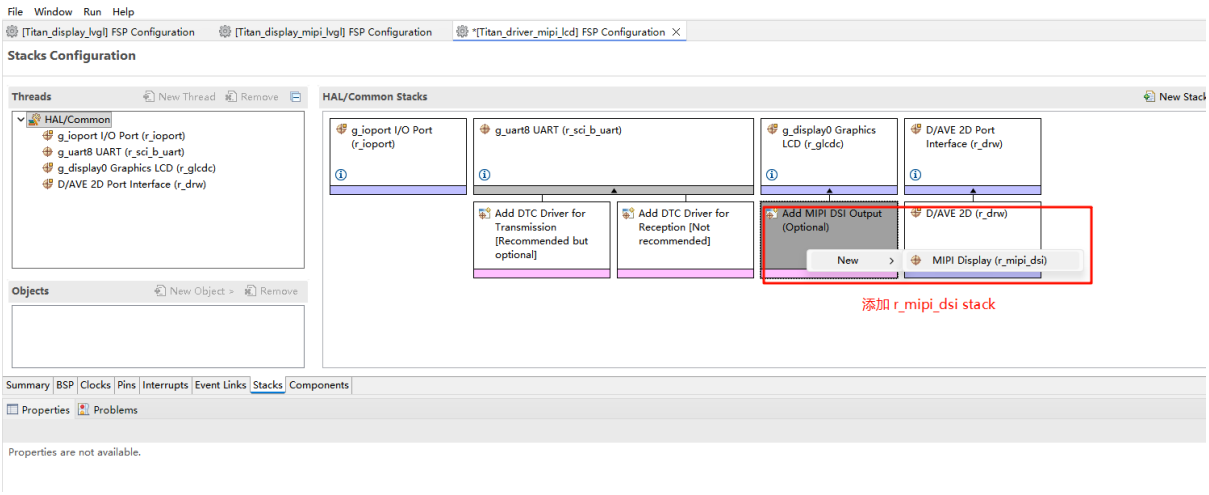


- Configure Output、CLUT、TCON and Dithering:

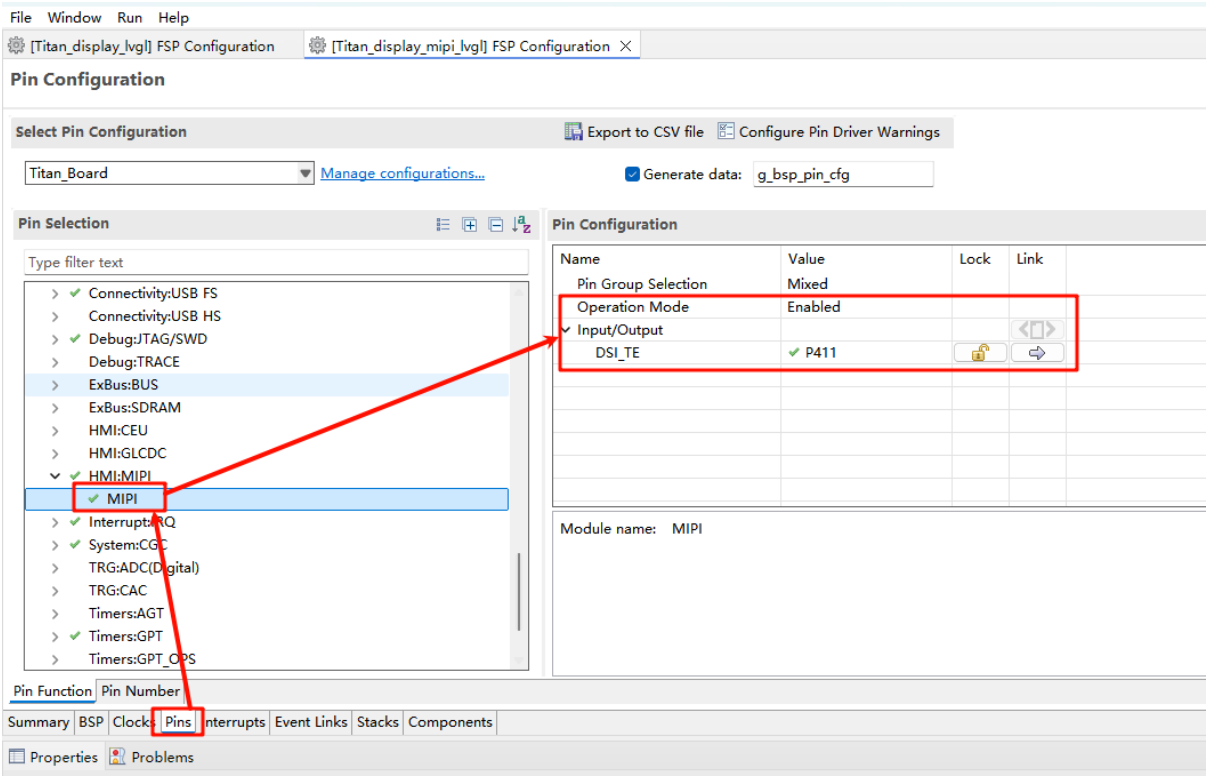
Background	
CLUT	
Enabled	No
Size	256
TCON	
Hsync pin select	LCD_TCON1
Vsync pin select	LCD_TCON0
Data enable (DE) pin select	LCD_TCON2
Panel clock source	Internal clock (GLCDCLK)
Panel clock division ratio	1/8
Dithering	
Enabled	No
Mode	Truncate
Pattern A	Pattern 11
Pattern B	Pattern 11
Pattern C	Pattern 11
Pattern D	Pattern 11

MIPI DSI Configuration

- Create a r_mipi_dsi stack.



- Configure MIPI DSI_TE pins.

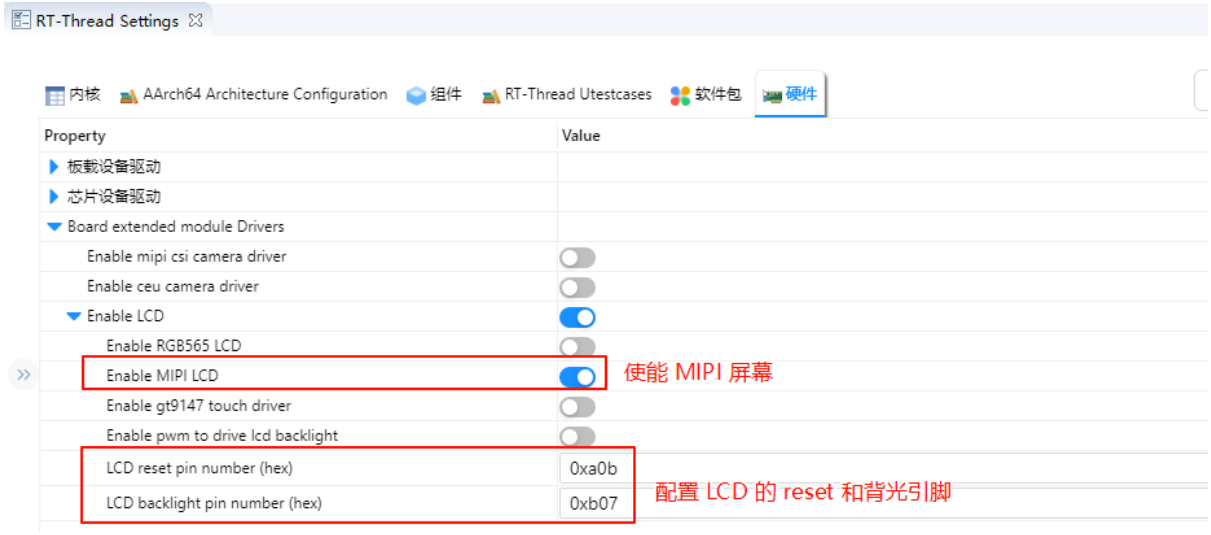


D/AVE 2D Configuration

- Create a r_drw stack:

7.5 RT-Thread Settings Configuration

- Enable MIPI LCD in RT-Thread Settings.



7.6 Example Code Description

```
int lcd_test(void)
{
    struct drv_lcd_device *lcd;
    struct rt_device_rect_info rect_info;
    rect_info.x = 0;
    rect_info.y = 0;
    rect_info.width = LCD_WIDTH;
    rect_info.height = LCD_HEIGHT;

    lcd = (struct drv_lcd_device *)rt_device_find("lcd");

    for (int i = 0; i < 2; i++)
    {
```

(continues on next page)

(continued from previous page)

```
        /* red */
        for (int i = 0; i < LCD_BUF_SIZE / 2; i++)
        {
            lcd->lcd_info.framebuffer[2 * i] = 0x00;
            lcd->lcd_info.framebuffer[2 * i + 1] = 0xF8;
        }
        LOG_D("red buffer...");
        rt_device_control(&lcd->parent, RTGRAPHIC_CTRL_RECT_UPDATE, &rect_info);
        rt_thread_mdelay(1000);
        /* green */
        for (int i = 0; i < LCD_BUF_SIZE / 2; i++)
        {
            lcd->lcd_info.framebuffer[2 * i] = 0xE0;
            lcd->lcd_info.framebuffer[2 * i + 1] = 0x07;
        }
        LOG_D("green buffer...");
        rt_device_control(&lcd->parent, RTGRAPHIC_CTRL_RECT_UPDATE, &rect_info);
        rt_thread_mdelay(1000);
        /* blue */
        for (int i = 0; i < LCD_BUF_SIZE / 2; i++)
        {
            lcd->lcd_info.framebuffer[2 * i] = 0x1F;
            lcd->lcd_info.framebuffer[2 * i + 1] = 0x00;
        }
        LOG_D("blue buffer...");
        rt_device_control(&lcd->parent, RTGRAPHIC_CTRL_RECT_UPDATE, &rect_info);
        rt_thread_mdelay(1000);
    }
    return RT_EOK;
}
MSH_CMD_EXPORT(lcd_test, lcd test cmd);
```

7.7 Compilation & Download

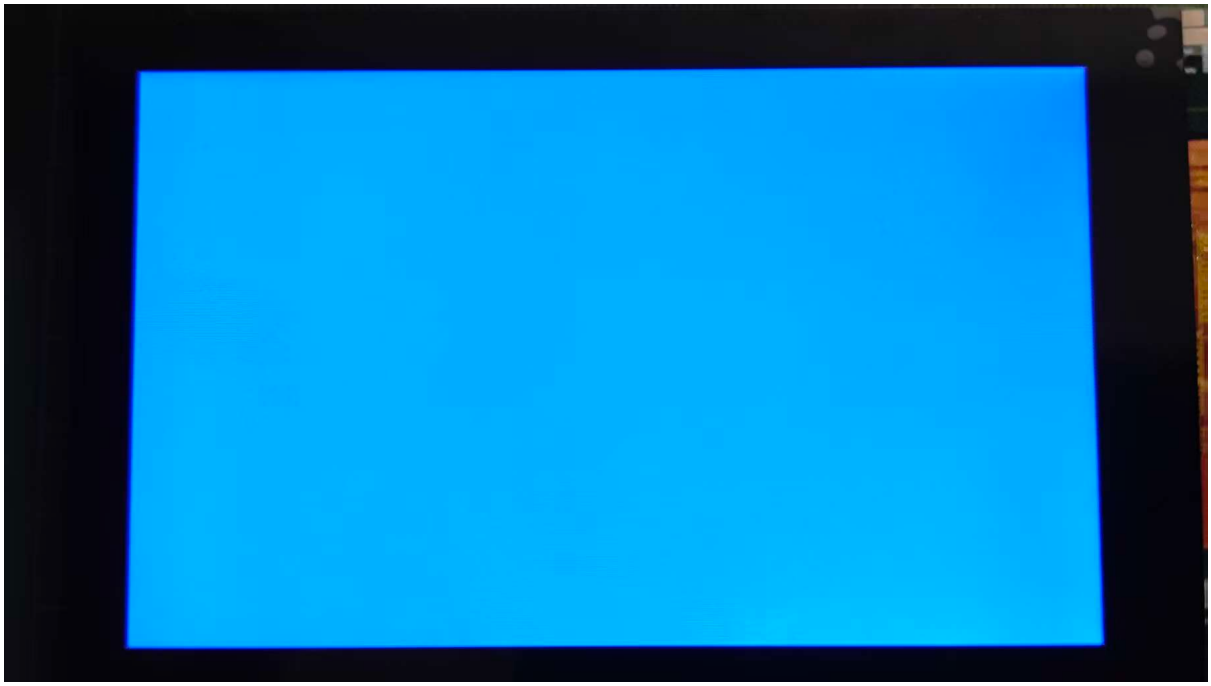
- **RT-Thread Studio**: In RT-Thread Studio’s package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board’s USB-DBG interface to the PC and download the firmware to the development board.

7.8 Run Effect

After reset the development board, type lcd_test command in the terminal to run the brush program.

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Aug 14 2025 16:31:18
 2006 - 2024 Copyright by RT-Thread team
[D/drv_lcd]
lcd framebuffer address:0x70000000

Hello RT-Thread!
================================================
This example project is an rgb lcd routine!
================================================
msh >lcd_test
[D/drv_lcd] red buffer...
[D/drv_lcd] green buffer...
[D/drv_lcd] blue buffer...
[D/drv_lcd] red buffer...
[D/drv_lcd] green buffer...
[D/drv_lcd] blue buffer...
msh >
```



8. RGB LCD Usage Instructions

English | Chinese

8.1 Introduction

This example demonstrates how to use the **GLCDC module of the RA8 series MCU** together with **RT-Thread' s LCD driver framework** on the **Titan Board** to drive an RGB LCD screen, enabling image display and interface updates. This document provides a detailed overview of **RA8 GLCDC peripheral features** and the **RT-Thread LCD driver framework**, along with example configurations and usage methods.

8.2 RA8 Series GLCDC Module

Overview

The **GLCDC (Graphics LCD Controller)** is a high-performance graphics controller integrated in RA8 series MCUs, specifically designed to drive TFT/RGB LCD screens. It supports various resolutions, color formats, and graphics processing functions. Combined with RT-Thread' s **LCD driver framework**, it provides a unified interface for screen initialization, refresh, graphics rendering, and DMA-accelerated operations.

The RA8 **GLCDC** enables image output from either internal MCU memory or external frame buffers to RGB/LCD displays. Key features include:

- **Frame buffer control**: Supports multiple frame buffers for page switching or double-buffered display
- **Color format support**: RGB565, RGB888, ARGB8888, etc.
- **Graphics processing**: Background layers, text/graphics composition, alpha blending, palette mapping
- **Synchronization signal generation**: HSYNC, VSYNC, DE (Data Enable)
- **DMA support**: High-speed data transfer, reducing CPU load
- **Interrupt functionality**: Frame-end and line-end interrupts

Module Architecture

The RA8 GLCDC module mainly consists of the following submodules:

1. **Layer Composition Unit**
    - Supports multiple layer stacking
    - Provides alpha blending, transparency control, and color keying
    - Allows rotation and flipping of layers
2. **Frame Buffer Interface**
    - Supports access to MCU internal SRAM or external memory

- Provides single or double buffering to ensure continuous display
- Works with DMA to automatically read image data

3. **DMA Controller**
   - Automatically transfers pixel data to the RGB output port
   - Configurable burst length to improve bandwidth utilization
   - Supports circular transfer, suitable for video or animation scenarios
4. **Timing Generator**
   - Automatically generates HSYNC, VSYNC, and DE signals
   - Supports TTL interface RGB timing
   - Polarity, sync width, and front/back porch timings are configurable
5. **Interrupt and Event Controller**
   - Provides frame-end and line-end interrupts
   - Can be used for page switching, dynamic drawing, or scrolling display
   - Supports interrupts triggered by DMA transfer completion

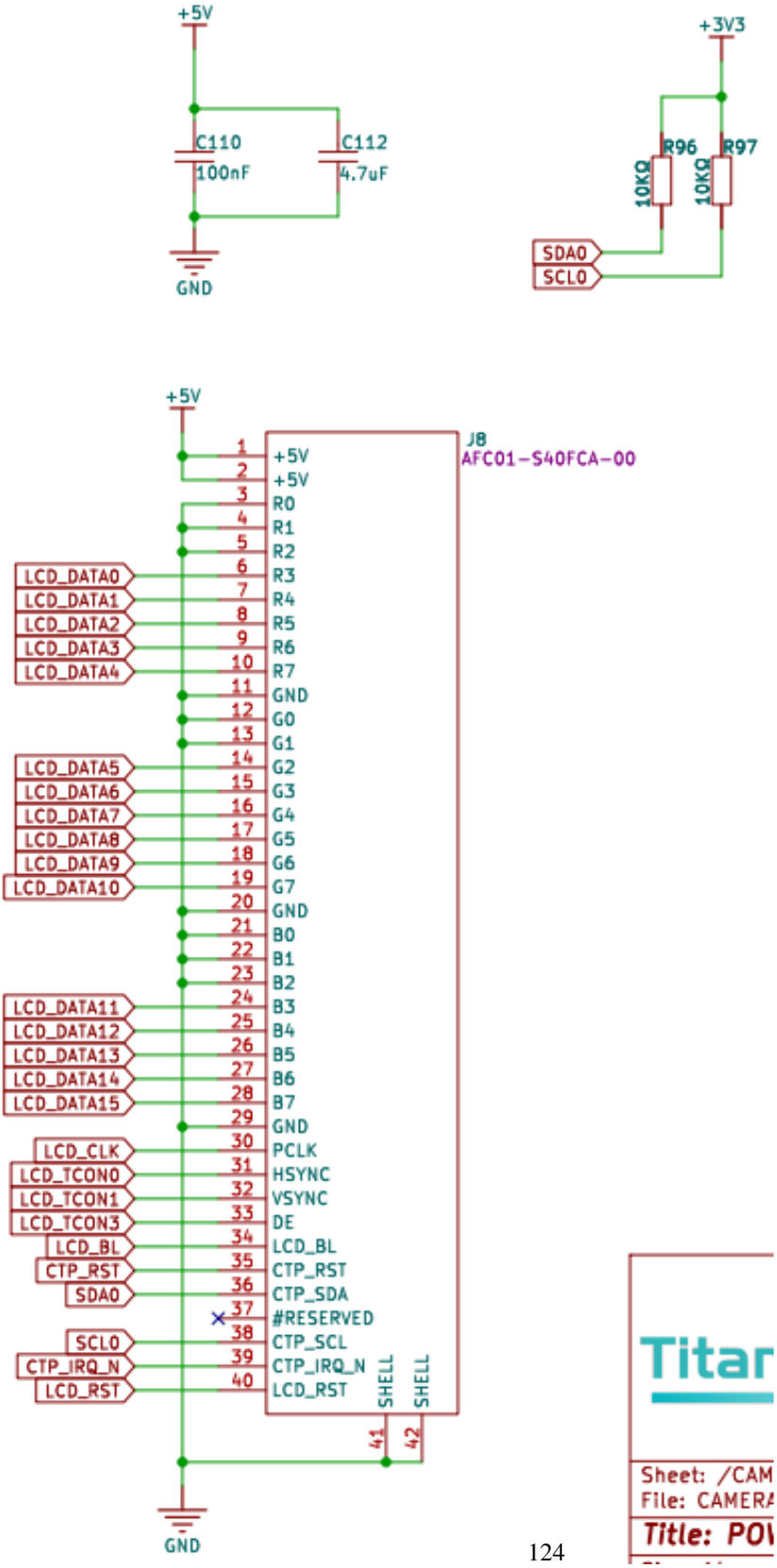
GLCDC Working Principle

1. **Frame buffer read**
   - GLCDC uses DMA to fetch image data from memory, supporting single or double buffering for continuous display.
2. **Layer composition**
   - Supports multiple layers such as background + foreground + icons/text
   - Provides alpha blending and palette mapping
3. **Pixel timing output**
   - Generates HSYNC/VSYNC/DE signals according to LCD interface requirements
   - Supports RGB parallel interface, TTL interface, or LVDS (depending on board implementation)
4. **Interrupts and events**
   - **Frame-end interrupt (VBlank)**: Can be used to update the next frame
   - **Line-end interrupt**: Useful for scrolling displays or dynamic rendering

GLCDC Supported Features

Feature Category	Description
Resolution	Up to 1280x800
Color formats	RGB565, RGB888, ARGB8888, etc.
Multi-layer	Background + foreground + icon layers, supports blending
Frame buffer	Single/double buffer mode, DMA improves performance
Palette	8/16-bit palette mapping for color conversion
Sync signals	HSYNC, VSYNC, DE, configurable polarity and timing
DMA support	Automatic memory transfer, CPU-free
Interrupts	Frame-end, line-end interrupts for synchronized refresh
Rotation/Flip	Supports 90°/180°/270° rotation and X/Y flipping

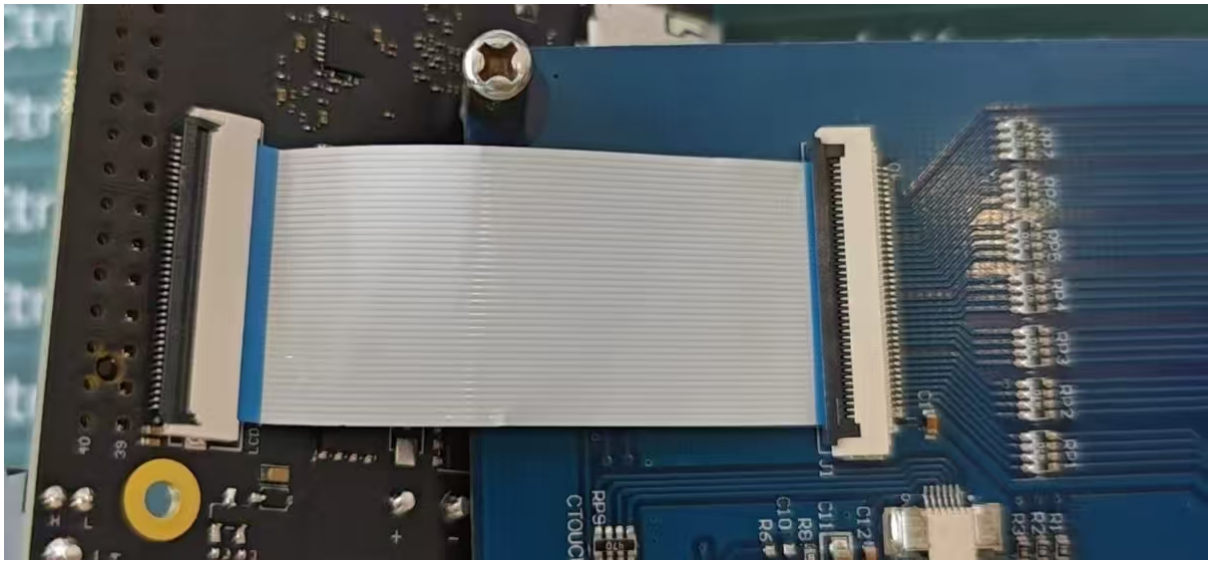
8.3 Hardware Description



- The connection method of RGB LCD is as follows:

Display model: **ALIENTEK 4.3" RGB LCD 800×480**

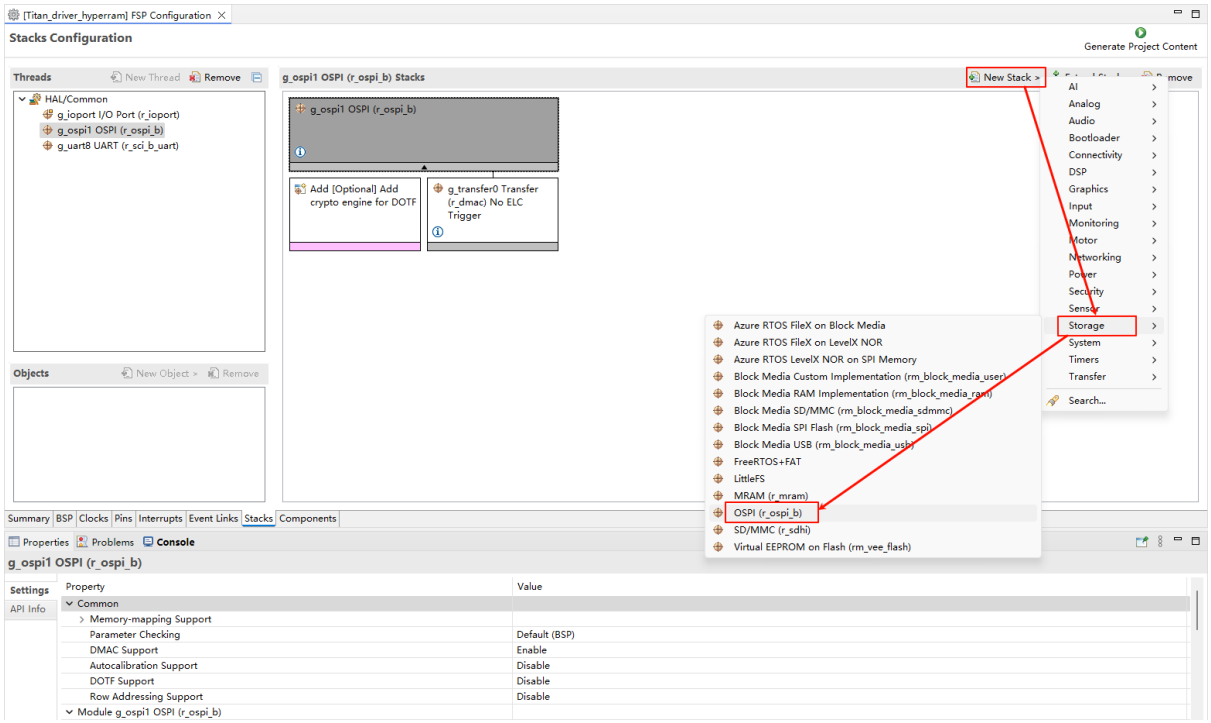
Use a **40-pin same-direction FFC cable** to connect the development board's **LCD_RGB** connector to the display.



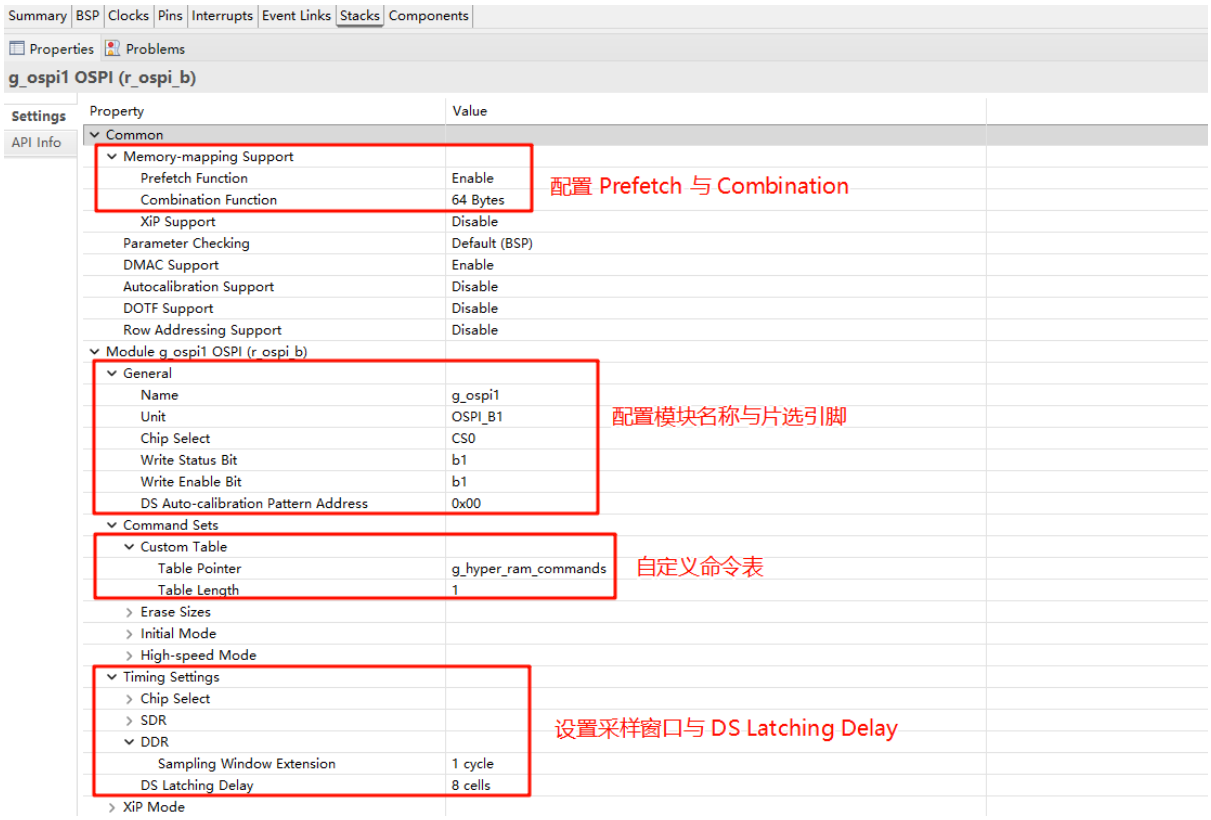
8.4 FSP Configuration

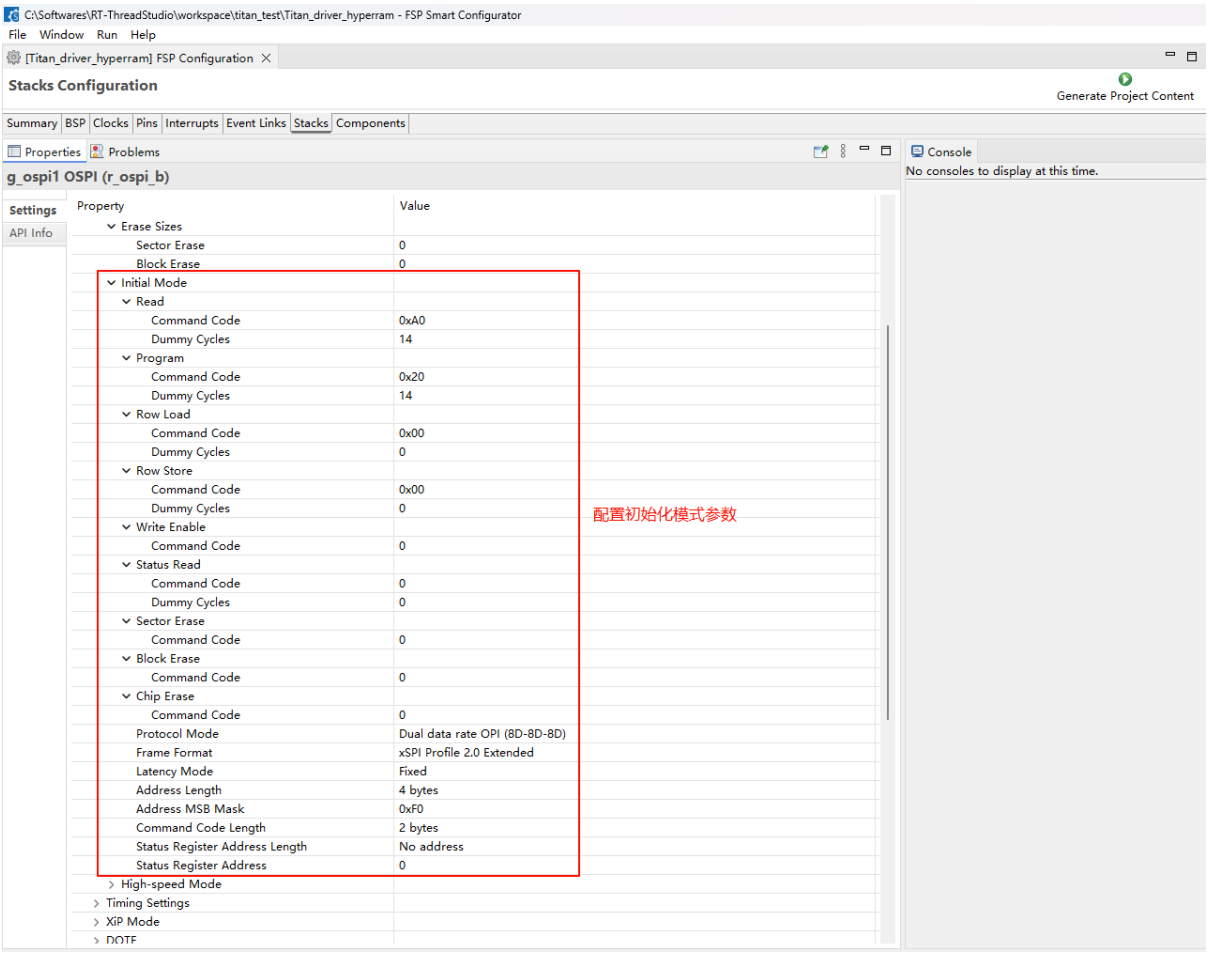
HyperRAM Configuration

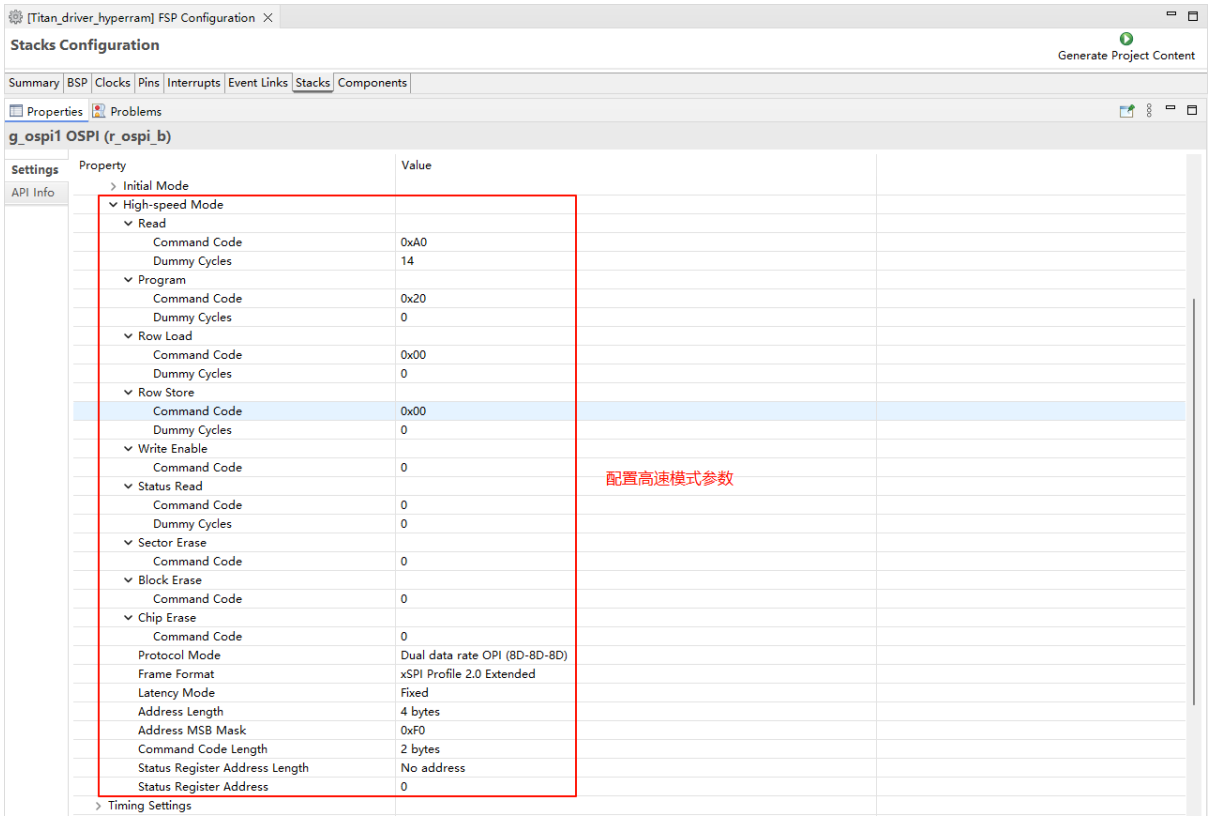
- Create a r_ospi_b stack:



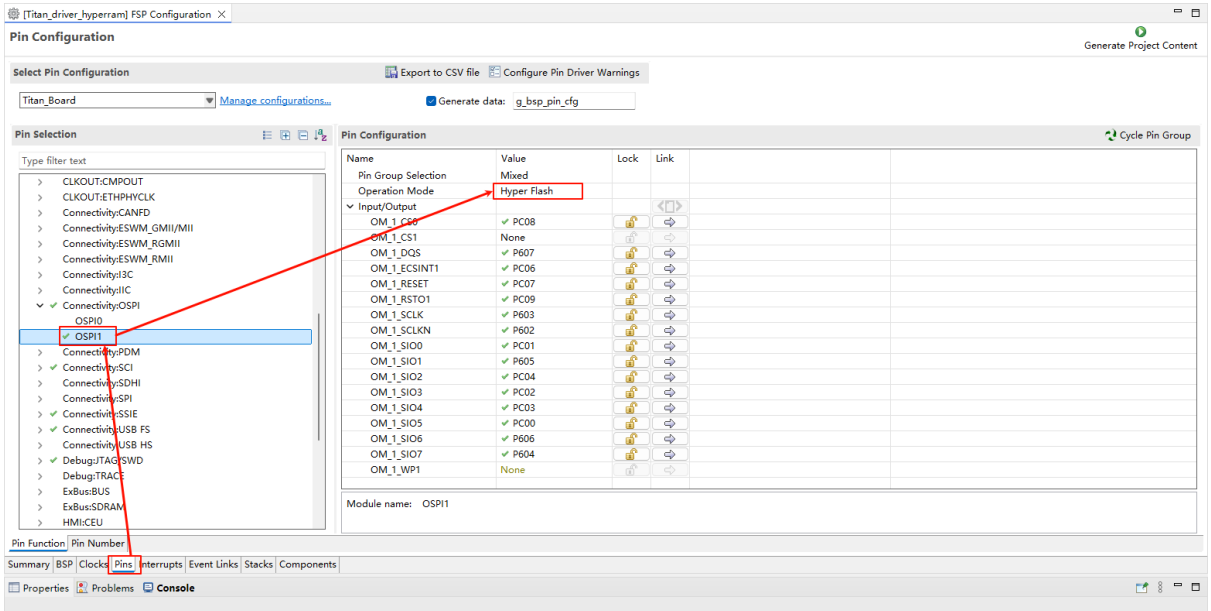
- Configuration r_ospi_b stack：



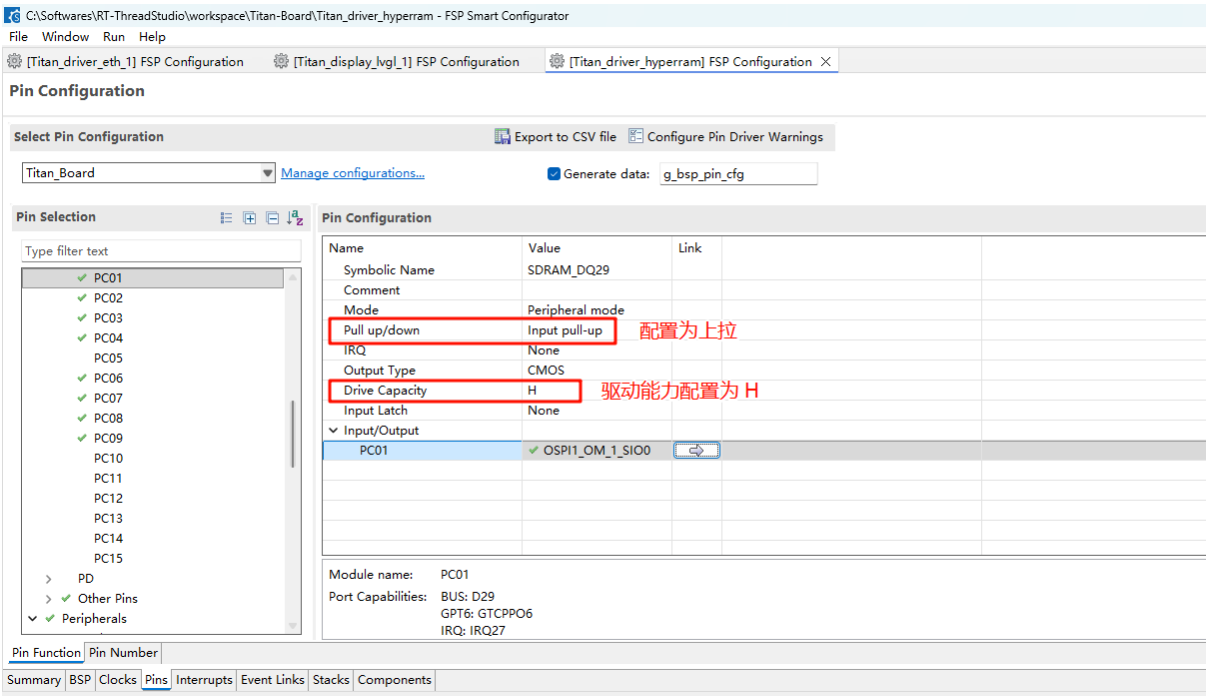

C:\Softwares\RT-ThreadStudio\workspace\titan_test\Titan_driver_hyperram - FSP Smart Configurator
File Window Run Help
[Titan_driver_hyperram] FSP Configuration
Stacks Configuration
Generate Project Content
Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems
g_ospi1 OSPI (r_ospi_b)
Settings
API Info
Property
Value
Erase Sizes
Sector Erase 0
Block Erase 0
Initial Mode
Read
Command Code 0xA0
Dummy Cycles 14
Program
Command Code 0x20
Dummy Cycles 14
Row Load
Command Code 0x00
Dummy Cycles 0
Row Store
Command Code 0x00
Dummy Cycles 0
Write Enable
Command Code 0
Status Read
Command Code 0
Dummy Cycles 0
Sector Erase
Command Code 0
Block Erase
Command Code 0
Chip Erase
Command Code 0
Protocol Mode Dual data rate OPI (8D-8D-8D)
Frame Format xSPI Profile 2.0 Extended
Latency Mode Fixed
Address Length 4 bytes
Address MSB Mask 0xF0
Command Code Length 2 bytes
Status Register Address Length No address
Status Register Address 0
High-speed Mode
Timing Settings
XiP Mode
DOTF
Console
No consoles to display at this time.
配置初始化模式参数



- Configure HyperRAM pins：

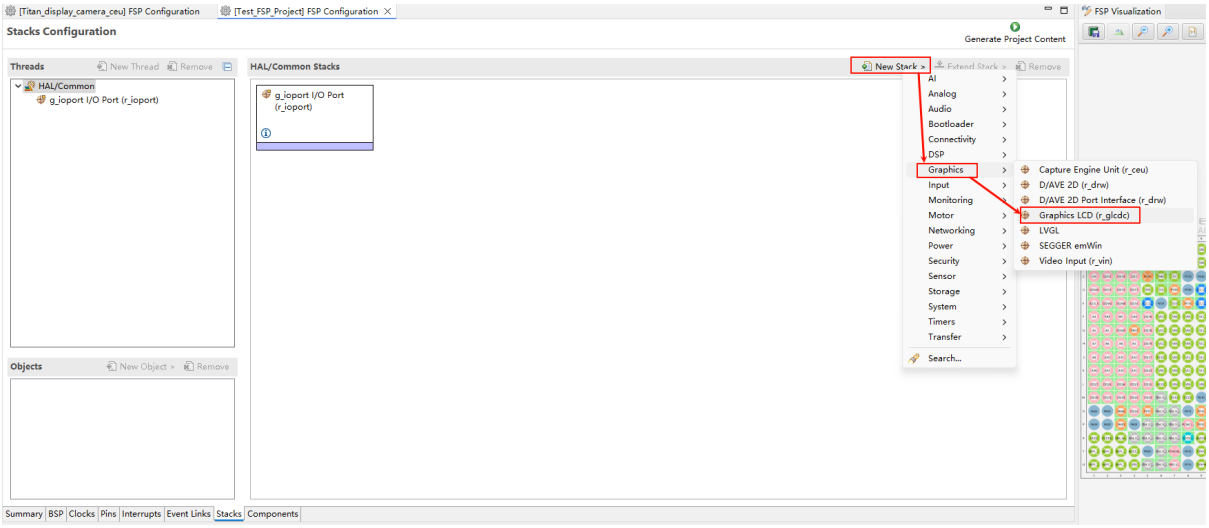


- The drive capability of all pins related to HyperRAM should be configured as H, and OM_1_SIO0 to OM_1_SIO7 need to be configured as Input pull-up.

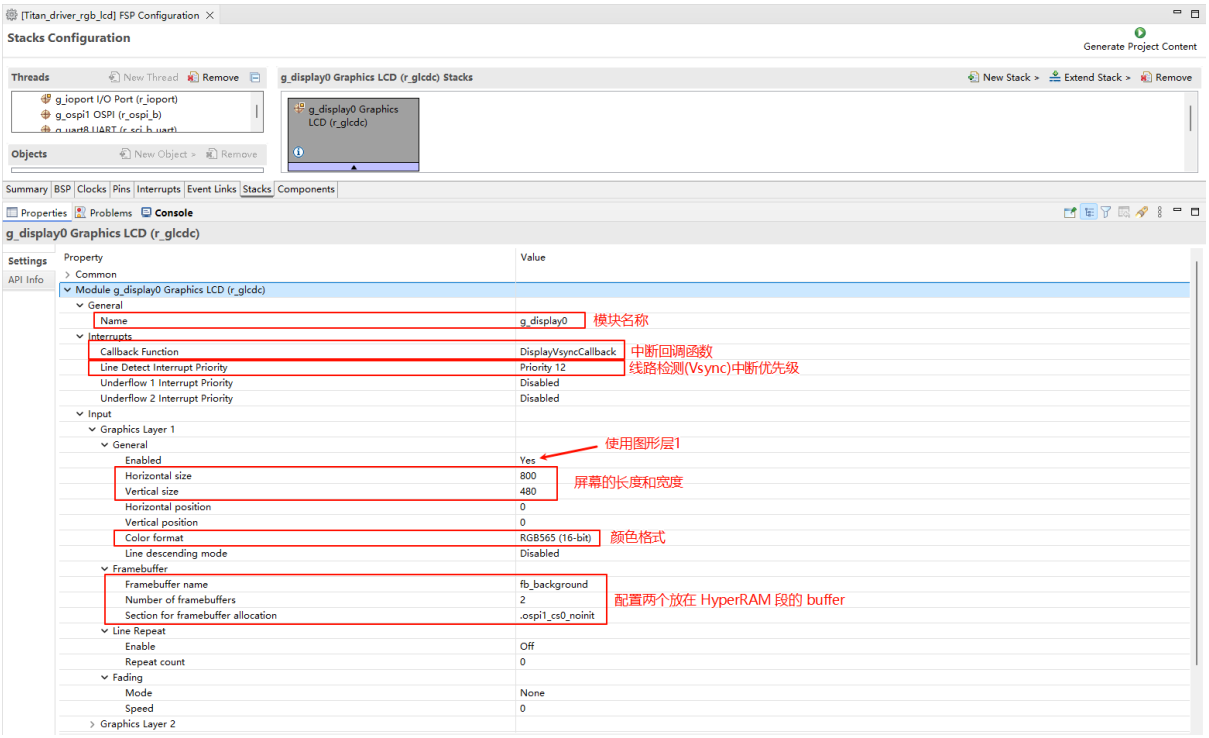


RGB LCD Configuration

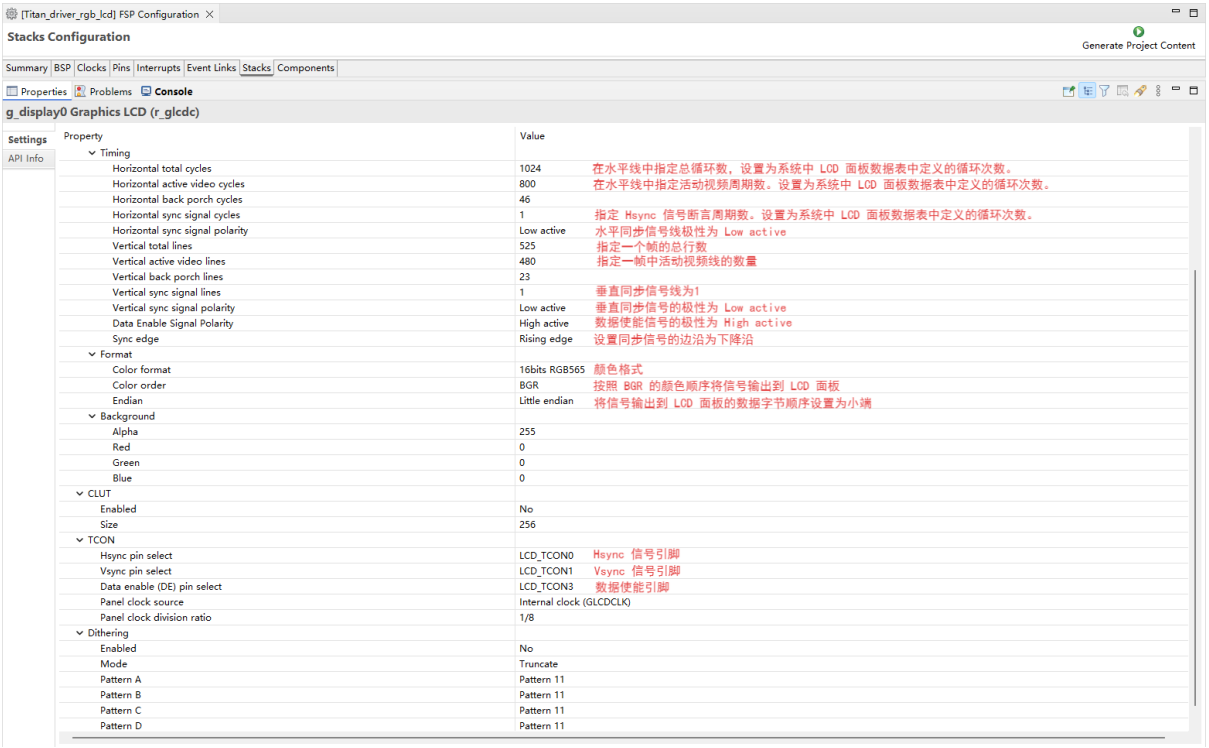
- Create a r_glcdc stack:



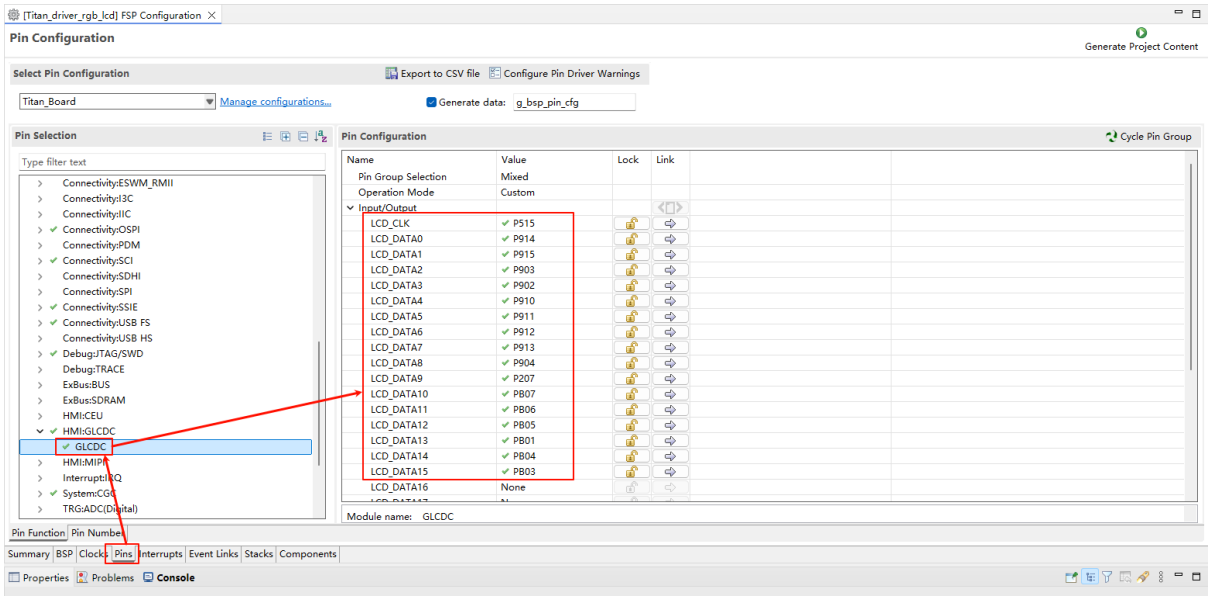
- Configure interrupt callback and graph layer 1:

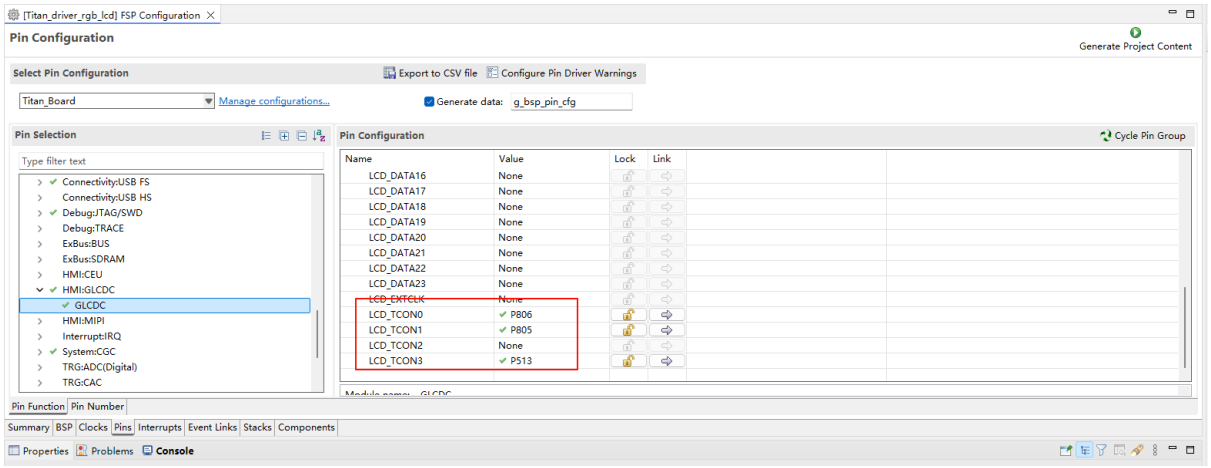


- Configure Output、CLUT、TCON and Dithering:



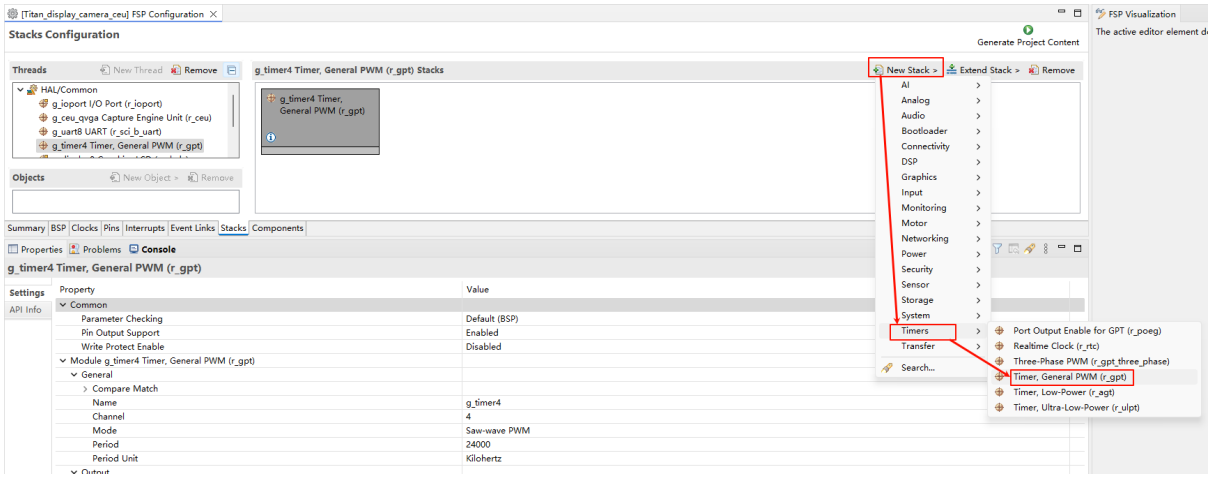
- Configure GLCDC pins:



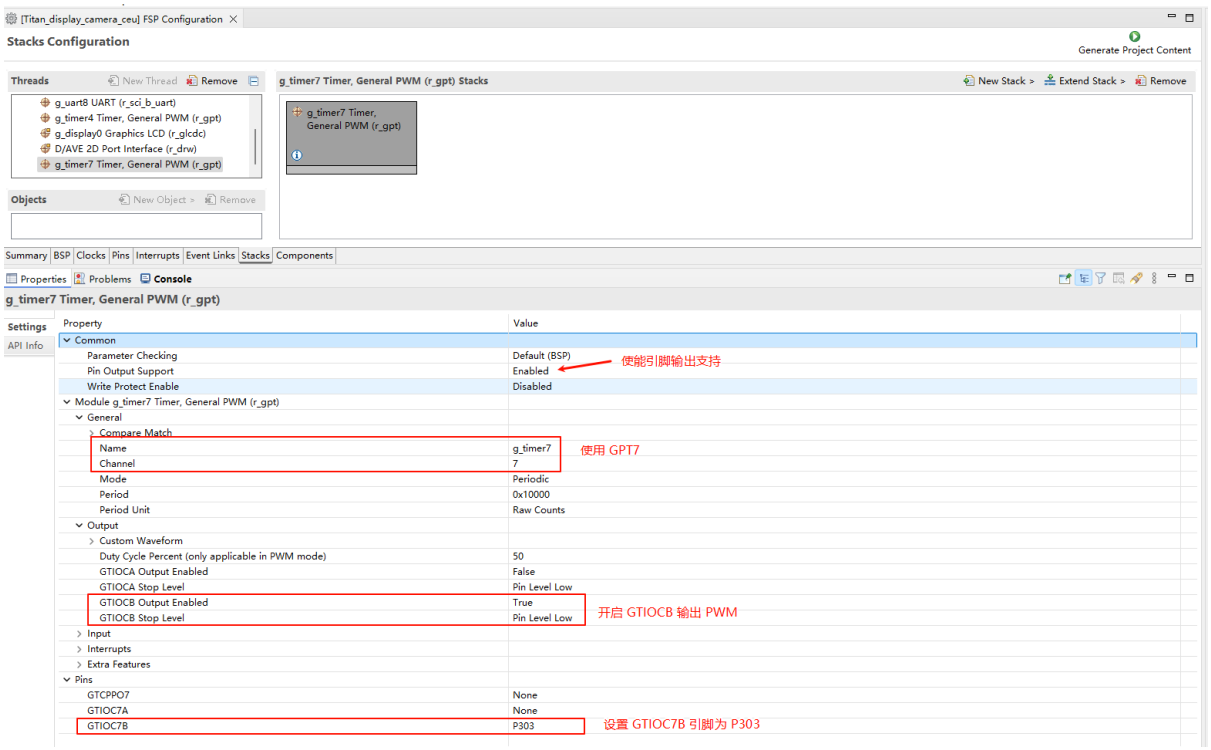


LCD Backlight Configuration

- Create a r_gpt stack:



- Configure back light PWM output:

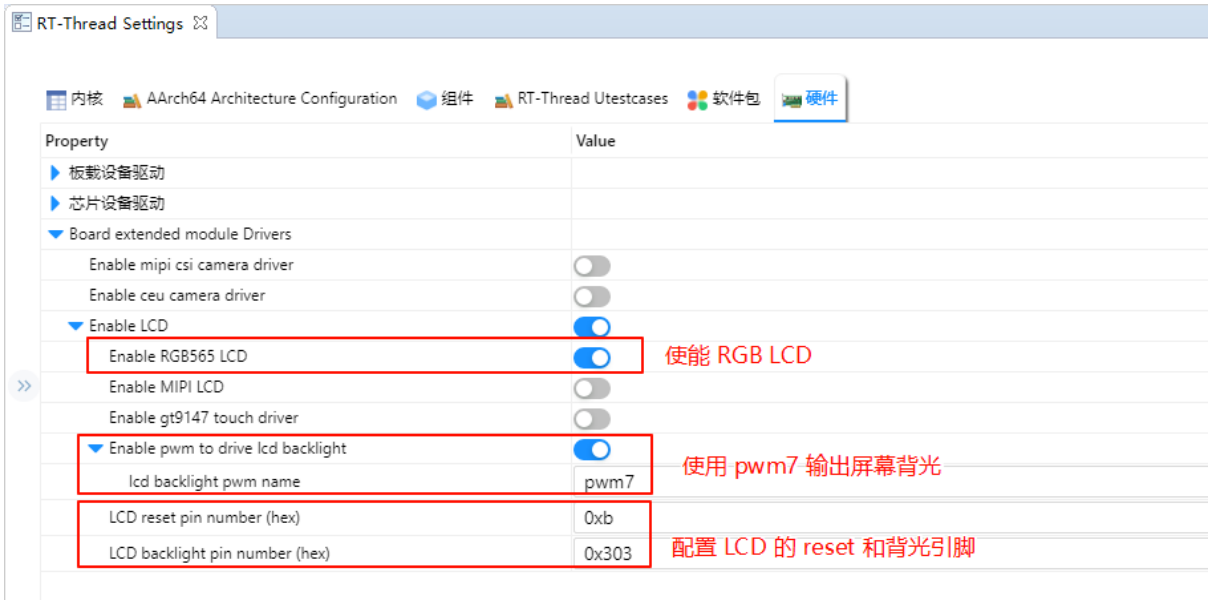


D/AVE 2D Configuration

- Create a r_drw stack:

8.5 RT-Thread Settings Configuration

- Enable RGB565 LCD in RT-Thread Settings, using pwm7 output screen backlight.



8.6 Example Code Description

```
int lcd_test(void)
{
    struct drv_lcd_device *lcd;
    struct rt_device_rect_info rect_info;
    rect_info.x = 0;
    rect_info.y = 0;
    rect_info.width = LCD_WIDTH;
    rect_info.height = LCD_HEIGHT;

    lcd = (struct drv_lcd_device *)rt_device_find("lcd");

    for (int i = 0; i < 2; i++)
    {
        /* red */
        for (int i = 0; i < LCD_BUF_SIZE / 2; i++)
        {
            lcd->lcd_info.framebuffer[2 * i] = 0x00;
            lcd->lcd_info.framebuffer[2 * i + 1] = 0xF8;
        }
        LOG_D("red buffer...");
        rt_device_control(&lcd->parent, RTGRAPHIC_CTRL_RECT_UPDATE, &rect_info);
        rt_thread_mdelay(1000);
        /* green */
        for (int i = 0; i < LCD_BUF_SIZE / 2; i++)
        {
            lcd->lcd_info.framebuffer[2 * i] = 0xE0;
            lcd->lcd_info.framebuffer[2 * i + 1] = 0x07;
```

(continues on next page)

(continued from previous page)

```
        }
        LOG_D("green buffer...");
        rt_device_control(&lcd->parent, RTGRAPHIC_CTRL_RECT_UPDATE, &rect_info);
        rt_thread_mdelay(1000);
        /* blue */
        for (int i = 0; i < LCD_BUF_SIZE / 2; i++)
        {
            lcd->lcd_info.framebuffer[2 * i] = 0x1F;
            lcd->lcd_info.framebuffer[2 * i + 1] = 0x00;
        }
        LOG_D("blue buffer...");
        rt_device_control(&lcd->parent, RTGRAPHIC_CTRL_RECT_UPDATE, &rect_info);
        rt_thread_mdelay(1000);
    }
    return RT_EOK;
}
MSH_CMD_EXPORT(lcd_test, lcd test cmd);
```

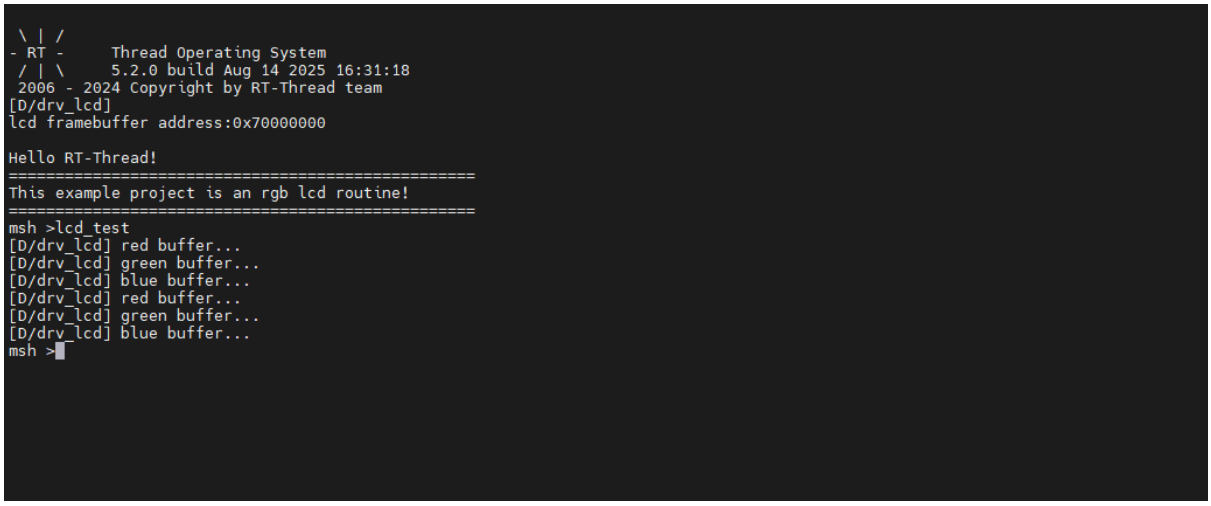
8.7 Compilation & Download

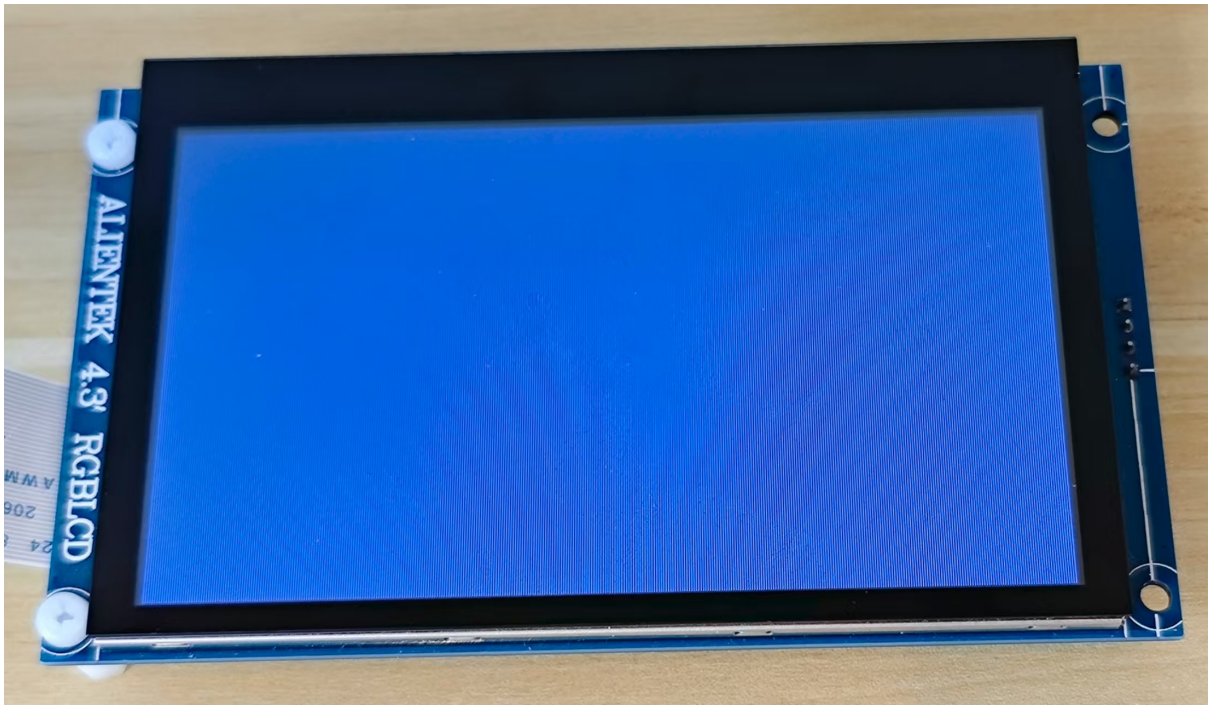
- **RT-Thread Studio**: In RT-Thread Studio's package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board's USB-DBG interface to the PC and download the firmware to the development board.

8.8 Run Effect

After reset the development board, type lcd_test command in the terminal to run the brush program.




ALIENTEK 4.3' RGBLCD

9. RS485 Driver Usage Instructions

English | Chinese

9.1 Introduction

This example demonstrates how to implement **RS485 half-duplex communication** on the **Titan Board** using the **RA8 series MCU UART module**, leveraging the **RT-Thread serial driver framework** for data transmission and reception. Through this example, users can become familiar with the **configuration of RA8 UART peripherals**, setting up RS485 communication modes, and application workflows under RT-Thread.

9.2 RS-485 Overview

General Description

RS-485 (also known as **TIA/EIA-485**) is a **differential serial communication standard** widely used in industrial control, building automation, and long-distance data transmission. It is an enhanced version of RS-232, supporting longer distances and multi-node communication.

Key features:

- **Differential signaling**: Uses two lines (A/B) to transmit signals, improving noise immunity.
- **Multi-point communication**: A single bus can support up to **32 drivers and 32 receivers** (more with extended devices).
- **Long-distance transmission**: Standard up to **1200 meters**, with speed inversely related to distance.
- **Half-duplex or full-duplex**: Flexible configuration; industrial protocols like Modbus RTU typically use half-duplex.

Physical Layer Characteristics

Feature	Description
Signal type	Differential (A/B lines)
Driver voltage	$\pm$1.5 V to $\pm$5 V
Receiver threshold	$\geq \pm$200 mV
Maximum nodes	32 drivers + 32 receivers (standard)
Bus length	Up to 1200 m (longer at lower speeds)
Termination resistor	120 Ω, matches bus impedance, reduces reflections

Differential signaling principle:

- Lines A and B carry opposite currents.

- The receiver measures **V_AB = V_A - V_B**.
- Common-mode noise is rejected, maintaining correct logic levels.

Communication Modes

1. **Half-Duplex**
   - Single-direction communication on the bus.
   - Send/receive enable control switches direction.
   - Common in Modbus RTU and industrial protocols.
2. **Full-Duplex**
   - Uses two differential pairs (A/B + A’/B’) to transmit and receive simultaneously.
   - Increases communication efficiency.
   - Requires more complex PCB routing.
3. **Point-to-Point & Multi-Point**
   - Point-to-point: Simple and reliable two-node communication.
   - Multi-point: Requires bus arbitration to avoid conflicts.

Signal Characteristics

- **Logic levels**:
  - Logic “1” (mark): A < B
  - Logic “0” (space): A > B
- **Noise immunity**:
  - Differential signaling rejects common-mode interference.
  - Suitable for long-distance industrial transmission.
- **Baud rate vs. distance**:

Baud Rate	Maximum Distance
9600 bps	1200 m
115200 bps	100 m

- Higher baud rates reduce maximum transmission distance.

Advantages and Limitations

Advantages:

- Strong noise immunity
- Supports long distances and multiple nodes
- Low bus cost and simple wiring

Limitations:

- Half-duplex requires software or hardware to manage conflicts
- High-speed communication is distance-limited
- Requires termination resistors for proper operation

9.3 RA8 Series UART Module Overview

The RA8 series MCU features a high-performance UART peripheral, supporting multiple communication modes and baud rates, making it suitable for RS485 half-duplex communication.

UART General Features

- **Communication mode**: Standard asynchronous serial
- **Data length**: 5–9 bits
- **Stop bits**: 1, 1.5, 2
- **Parity**: None, even, or odd
- **Baud rate**: 300 bps – 12 Mbps (higher on some models)
- **FIFO support**: TX/RX buffers to reduce CPU load
- **DMA support**: TX/RX can leverage DMA for high throughput
- **Interrupt events**: Transmission complete, FIFO threshold, errors (frame, overflow, parity)

RS485-Specific Features

- **Driver Enable (DE) auto-control**:
  - DE is automatically raised during transmission
  - Lowered after transmission completes
- **Half-duplex mode**:
  - Single-wire transceivers use direction control to switch between send and receive
- **Optional address detection**:
  - Supports address matching for multi-node communication

UART Architecture and Operation

1. **TX/RX FIFO**
   - Independent buffers for transmit and receive
   - FIFO threshold interrupts improve continuous data handling
2. **Baud rate generator**
   - Generates desired baud rate from PCLK and divisors
   - Supports standard and non-standard rates
3. **Interrupt and event handling**

- TX empty interrupt: transmit buffer is empty
- RX full interrupt: receive buffer is full
- Error interrupts: frame, overflow, parity
- Transmission complete interrupt: can trigger RS485 DE auto-switch

9.4 RT-Thread UART v2 Driver Framework

The RT-Thread UART v2 (Universal Asynchronous Receiver/Transmitter) framework is a unified interface provided by the RT-Thread device driver layer to manage serial communication peripherals across various MCUs. Compared to the legacy UART framework, UART v2 introduces standardized APIs, enhanced interrupt handling, and flexible callback mechanisms, enabling unified and portable serial communication for applications.

Device Model

In RT-Thread, UARTs are managed as **device objects** (subclass of struct rt_device, type RT_Device_Class_Char). Developers do not need to operate hardware registers directly. Instead, UART initialization, configuration, transmission, reception, and callback registration are achieved through standardized device APIs.

Operation Interfaces

Applications interact with UART hardware through RT-Thread’s I/O device management APIs as shown below:

- Find UART device

```
rt_device_t rt_device_find(const char* name);
```

- Open UART device

```
rt_err_t rt_device_open(rt_device_t dev, rt_uint16_t oflags);
```

- Control UART device

Through the control interface, applications can configure UART parameters such as baud rate, data bits, parity, stop bits, and buffer size:

```
rt_err_t rt_device_control(rt_device_t dev, rt_uint8_t cmd, void* arg);
```

- Send data

```
rt_size_t rt_device_write(rt_device_t dev, rt_off_t pos, const void* buffer, rt_
  size_t size);
```

- Set TX complete callback

```
rt_err_t rt_device_set_tx_complete(rt_device_t dev, rt_err_t (*tx_done)(rt_
  device_t dev, void* buffer));
```

- Set RX callback

```
rt_err_t rt_device_set_rx_indicate(rt_device_t dev, rt_err_t (*rx_ind)(rt_device_
  t dev, rt_size_t size));
```

- Read received data

```
rt_size_t rt_device_read(rt_device_t dev, rt_off_t pos, void* buffer, rt_size_t
  size);
```

- Close UART device

```
rt_err_t rt_device_close(rt_device_t dev);
```

Common control commands used with rt_device_control() include:

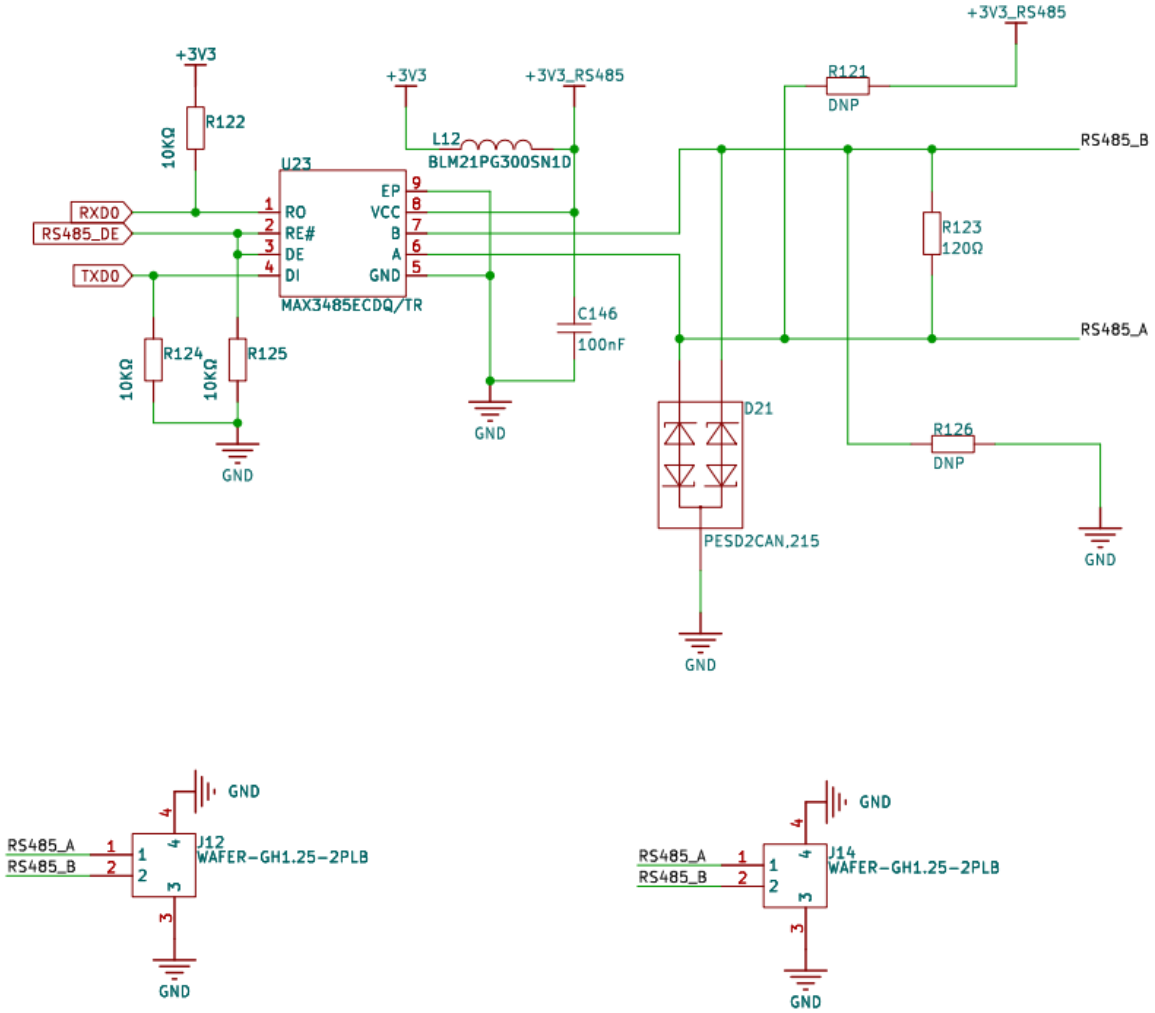
```
#define RT_DEVICE_CTRL_CONFIG        (0x10)   /* Configure UART parameters */
#define RT_DEVICE_CTRL_SET_INT       (0x11)   /* Enable interrupt */
#define RT_DEVICE_CTRL_CLR_INT       (0x12)   /* Disable interrupt */
#define RT_DEVICE_CTRL_CUSTOM_CMD    (0x13)   /* Custom control command */
```

Framework Features

- **Unified API design**: Standardized read, write, and control interfaces for all UART devices.
- **Event callback mechanism**: Supports both RX and TX callbacks for asynchronous communication.
- **Rich configuration options**: Flexible settings for baud rate, parity, data bits, and stop bits.
- **Interrupt & DMA support**: Drivers can implement interrupt-driven or DMA-based data transfer.
- **Cross-platform portability**: The same UART application code runs on multiple MCU platforms.

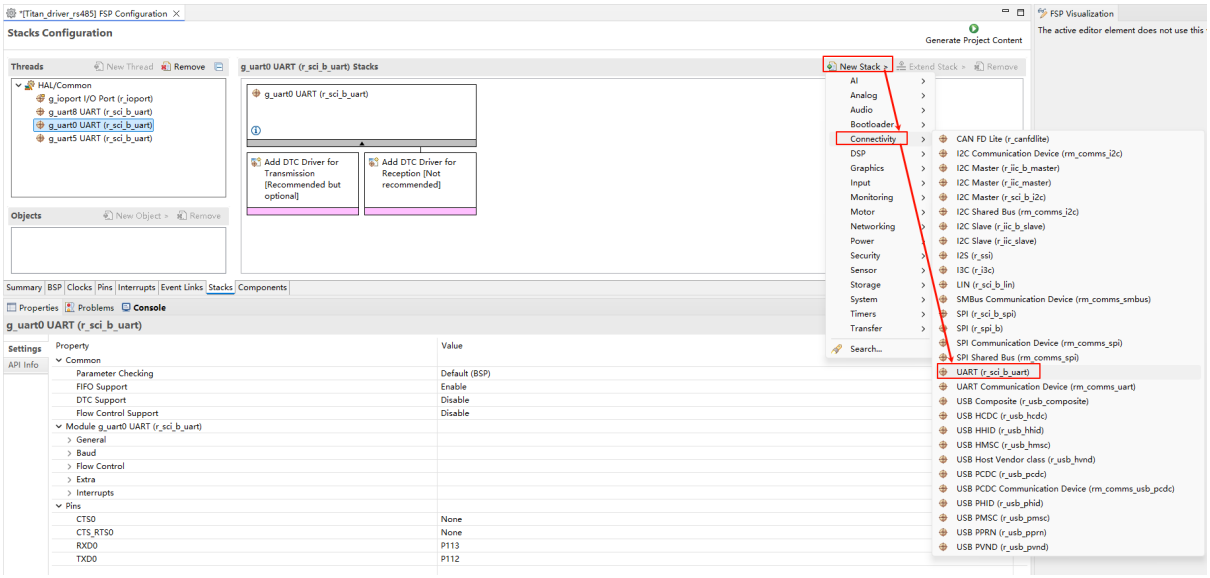
Reference: RT-Thread UART Device V2

9.5 Hardware Description

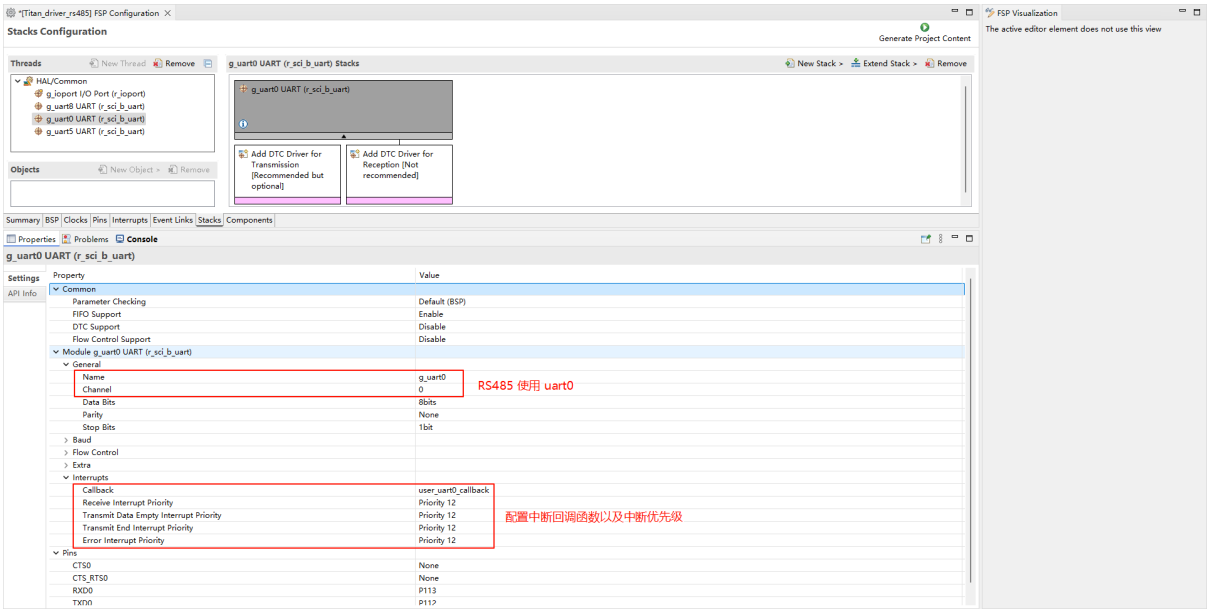


9.6 FSP Configuration

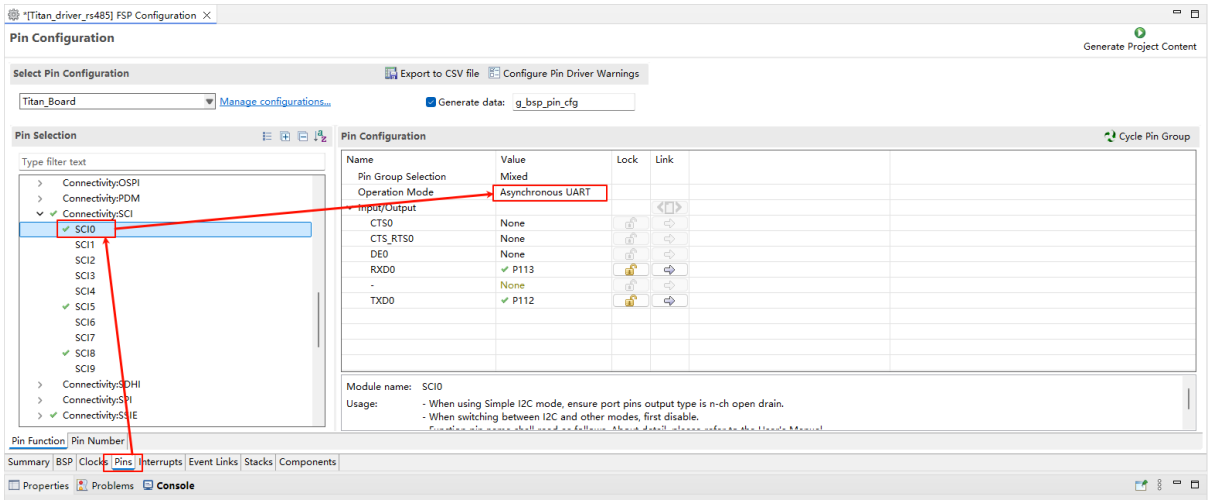
- Open the FSP tool and create a r_sci_b_uart stack:



- Configure r_sci_b_uart stack：

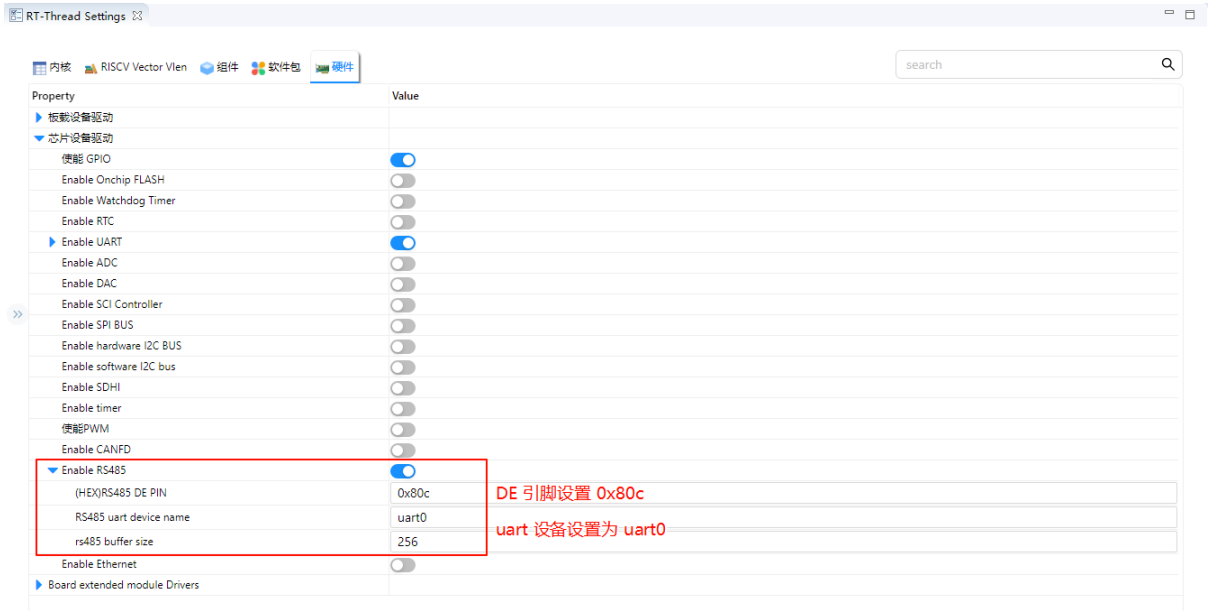


- Configure r_sci_b_uart 引脚：



9.7 RT-Thread Settings Configuration

- Enable and configure RS485.



9.8 Example Code Description

```
#define RS485_OUT       rt_pin_write((rt_base_t)RS485_DE_PIN, PIN_HIGH)
#define RS485_IN        rt_pin_write((rt_base_t)RS485_DE_PIN, PIN_LOW)

static rt_device_t rs485_serial = RT_NULL;
static struct rt_semaphore rs485_rx_sem;
static struct rt_ringbuffer rs485_rx_rb;
static rt_uint8_t rs485_rx_buffer[RS485_RX_BUFFER_SIZE];
```

(continues on next page)

(continued from previous page)

```
/* uart receive data callback function */
static rt_err_t rs485_input(rt_device_t dev, rt_size_t size)
{
    if (size > 0)
    {
        rt_uint8_t ch;
        while (rt_device_read(dev, 0, &ch, 1) == 1)
        {
            rt_ringbuffer_put_force(&rs485_rx_rb, &ch, 1);
        }
        rt_sem_release(&rs485_rx_sem);
    }
    return RT_EOK;
}

/* send data */
int rs485_send_data(const void *tbuf, rt_uint16_t t_len)
{
    RT_ASSERT(tbuf != RT_NULL);

    /* change rs485 mode to transmit */
    RS485_OUT;

    /* send data */
    rt_size_t sent = rt_device_write(rs485_serial, 0, tbuf, t_len);

    if (sent != t_len)
    {
        /* Transmission failed, switch back to receive mode */
        RS485_IN;
        return -RT_ERROR;
    }

    /* Note: We don't switch back to receive mode here -
       that will be done in the tx_complete callback (rs485_output) */

    LOG_I("send==>>");
    for (int i = 0; i < t_len; i++)
    {
        LOG_I("   %d:   %c ", i, ((rt_uint8_t *)tbuf)[i]);
    }
    RS485_IN;

    return RT_EOK;
}
```

(continues on next page)

(continued from previous page)

```
static void rs485_thread_entry(void *parameter)
{
    rt_uint8_t ch;
    rt_size_t length;

    while (1)
    {
        /* Wait for data */
        rt_sem_take(&rs485_rx_sem, RT_WAITING_FOREVER);

        /* Process all available data in the ring buffer */
        while (length = rt_ringbuffer_get(&rs485_rx_rb, &ch, 1))
        {
            if (length == 1)
            {
                LOG_I("recv data:%c", ch);
            }
        }
    }
}

int rs485_init(void)
{
    /* Initialize ring buffer */
    rt_ringbuffer_init(&rs485_rx_rb, rs485_rx_buffer, RS485_RX_BUFFER_SIZE);

    /* find uart device */
    rs485_serial = rt_device_find(RS485_UART_DEVICE_NAME);
    if (!rs485_serial)
    {
        LOG_E("find %s failed!", RS485_UART_DEVICE_NAME);
        return -RT_ERROR;
    }

    /* Open device in interrupt mode with DMA support if available */
    rt_device_open(rs485_serial, RT_DEVICE_FLAG_INT_RX | RT_DEVICE_FLAG_DMA_RX);

    /* set receive data callback function */
    rt_device_set_rx_indicate(rs485_serial, rs485_input);

    /* Initialize RTS pin */
    rt_pin_mode((rt_base_t)RS485_DE_PIN, PIN_MODE_OUTPUT);
    RS485_IN;

    /* Initialize semaphore */
    rt_sem_init(&rs485_rx_sem, "rs485_rx_sem", 0, RT_IPC_FLAG_FIFO);
```

(continues on next page)

(continued from previous page)

```
    rt_thread_t thread = rt_thread_create("rs485", rs485_thread_entry, RT_NULL,
                                          1024, 25, 10);

    if (thread != RT_NULL)
    {
        rt_thread_startup(thread);
    }
    else
    {
        return -RT_ERROR;
    }

    return RT_EOK;
}
INIT_DEVICE_EXPORT(rs485_init);
```

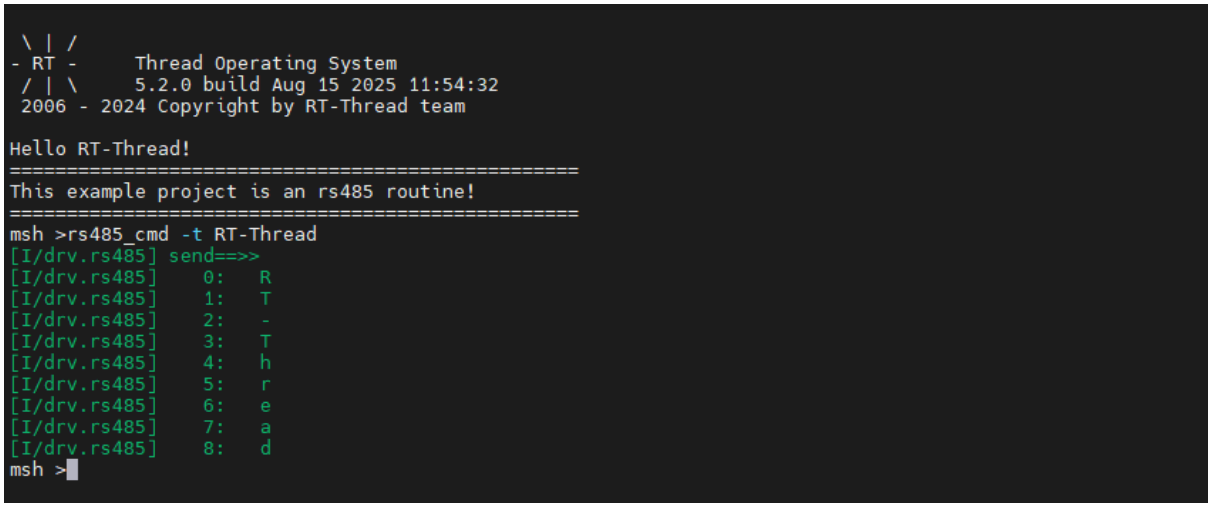
9.9 Build & Download

- **RT-Thread Studio:** Download the Titan Board resource pack from the RT-Thread Studio package manager, then create a new project and compile it.

Once compiled, connect the development board’ s USB-DBG interface to the PC, and download the firmware to the development board.

9.10 Running Result

Type rs485_cmd -t RT-Thread command in terminal to send “RT-Thread” string.



The RS485 interface of the Titan Board is connected to the RS485 interface of another development Board, and the data is continuously sent using the other development board, and the terminal will output the data received by the Titan Board.

```
================================================
This example project is an rs485 routine!
================================================
msh >[I/drv.rs485] recv data:R
[I/drv.rs485] recv data:T
[I/drv.rs485] recv data:-
[I/drv.rs485] recv data:T
[I/drv.rs485] recv data:h
[I/drv.rs485] recv data:r
[I/drv.rs485] recv data:e
[I/drv.rs485] recv data:a
[I/drv.rs485] recv data:d
[I/drv.rs485] recv data:R
[I/drv.rs485] recv data:T
[I/drv.rs485] recv data:-
[I/drv.rs485] recv data:T
[I/drv.rs485] recv data:h
[I/drv.rs485] recv data:r
[I/drv.rs485] recv data:e
[I/drv.rs485] recv data:a
[I/drv.rs485] recv data:d
```

10. RTC and Alarm Example Usage Instructions

English|Chinese

10.1 Introduction

This example demonstrates how to use the **RTC module of RA8 series MCU** on the **Titan Board**, combined with the **RT-Thread RTC driver framework**, to implement basic real-time clock functions and alarm features. The RTC provides accurate real-time information, including year, month, day, hour, minute, and second. Most RTC chips use high-precision crystal oscillators as clock sources. Some chips are equipped with a battery to maintain time even when the main power is off.

The RTC device in RT-Thread provides fundamental time services for the operating system. In increasingly common IoT scenarios, RTC has become standard, and in applications such as SSL secure transmission, RTC is an essential component.

10.2 RTC (Real-Time Clock) Overview

Overview

RTC (Real-Time Clock) is an electronic module or chip used to **measure and track actual time** (year, month, day, hour, minute, second).

It is usually built into a **microcontroller (MCU)** or exists as a standalone chip, providing **system time**, **timed wake-ups**, and **scheduled event triggers**.

The core feature of an RTC is **low power consumption and long-term stable operation**. Even if the main power is off, the RTC can continue running via a backup power source (battery or supercapacitor).

Working Principle

An RTC is essentially a **low-power oscillator + counter** system:

1. **Clock Source**
   - Typically uses a **32.768 kHz quartz crystal** (1 Hz = 2^15 cycles)
   - Provides a stable time base
2. **Frequency Division & Counting**
   - Divides the crystal frequency to generate 1 Hz pulses
   - Accumulates counts to generate minutes, hours, days, months, and years
3. **Register Storage**
   - Internal registers store current time, date, and alarm settings
4. **Power Backup**
   - Powered by an internal battery or supercapacitor to maintain time during main power loss

RTC Types

1. **Internal RTC**
   - Integrated into the MCU
   - Pros: Saves chip count, lower cost
   - Cons: Crystal accuracy affected by PCB layout and temperature
2. **External RTC Chip**
   - Standalone chips, e.g., DS3231, PCF8563
   - Pros: High precision, connects via I^2C/SPI
   - Cons: Increases PCB area and cost

Key Parameters

Parameter	Description
Oscillator Freq	Usually 32.768 kHz, low-power and stable
Time Accuracy	ppm / seconds per day, determines drift
Supply Voltage	1.8~5V, supports backup power
Power Consumption	1~5 μA (low-power mode)
Interface Type	I^2C, SPI, or MCU internal bus
Extended Features	Alarm output, square wave, temperature compensation, wake-up timer

RTC Functions

1. **Real-Time Clock**
   - Provides current time and date
2. **Alarm**
   - Configurable to trigger interrupts or wake the MCU at a specific time
3. **Wake-up Timer**
   - Can wake MCU from low-power mode
4. **Temperature Compensation** (for high-precision chips like DS3231)
   - Auto-calibrates crystal frequency to reduce drift
5. **Square Wave Output**
   - Provides fixed-frequency pulses to drive MCU timers or other modules

RTC Applications

- **Embedded Devices**: System time tracking, event logging
- **Low-Power IoT Devices**: RTC wakes MCU for periodic data collection
- **Watches and Wearables**: Precise timekeeping
- **Data Loggers & Industrial Control**: Timestamping, log recording
- **Automotive Electronics**: Dashcams, in-car entertainment systems

10.3 RA8 Series RTC Module Overview

The RA8 MCU series includes a highly flexible **RTC peripheral**, suitable for low-power, high-precision embedded time management.

General Features

- **Time Management**: Supports seconds, minutes, hours, day, month, year, and weekday
- **Clock Source**:
  - **External low-speed crystal** (usually 32.768 kHz) for high precision
  - **Internal low-speed RC oscillator** for low power, slightly lower precision
- **Alarm**: Supports single and repeating alarms with interrupt generation
- **Timers**: Sub-second timers, periodic wake-up timers
- **Tamper Detection (Optional)**: Detects external tampering via configured pins
- **Low-Power Operation**: RTC continues running in standby or sleep mode to maintain time during main power loss

RTC Module Architecture

RA8 RTC consists of the following functional modules:

1. **Calendar Counter**
   - Maintains current time and date
   - Supports 24-hour or 12-hour format
   - Automatically handles leap years
2. **Alarm & Compare Unit**
   - Compares current time with preset alarm value
   - Triggers interrupts on match
   - Supports repeating alarms (daily, hourly, etc.)
3. **Clock Source & Divider**
   - Divides external 32.768 kHz or internal RC oscillator to 1 Hz time base
   - Allows trade-off between accuracy and power consumption

4. **Interrupt Controller**
    - Supports alarm, periodic, and overflow interrupts
    - Configurable interrupt mask, enable, and status flags
5. **Tamper & Backup Registers (Optional)**
    - Non-volatile registers store data during power loss
    - Tamper pins can trigger interrupts for security applications

Key Features

Feature	Description
Time Resolution	Second-level, optional sub-second counting
Time Range	Year, month, day, hour, minute, second
Alarm	Configurable, triggers interrupts
Clock Source	External 32.768 kHz crystal, internal RC oscillator
Power Consumption	Ultra-low, supports standby/sleep modes
Interrupts	Alarm, periodic, overflow
Backup Registers	Non-volatile, retains data during power loss
Tamper Detection (Optional)	External pin events can trigger interrupts

Working Principle

1. **Select Clock Source**: external crystal or internal RC oscillator
2. **Set Time**: write initial date and time to calendar counter
3. **Configure Alarm**: set alarm time and enable interrupt
4. **Interrupt Handling**: MCU responds to alarm or periodic events via RTC interrupt flag
5. **Power Management**: RTC continues counting in low-power mode, ensuring accurate time

10.4 RT-Thread RTC Driver Framework

The RT-Thread RTC (Real-Time Clock) framework is a unified interface provided by the RT-Thread device driver layer for managing RTC hardware across various MCUs. It abstracts the underlying RTC hardware into a standardized interface, enabling applications to manage date, time, timestamps, and alarms through a consistent API.

Device Model

In RT-Thread, RTC is managed as a **device object** (a subclass of struct rt_device, type RT_Device_Class_RTC). Developers do not need to manipulate hardware registers directly. Time configuration, reading, and alarm control can be achieved using standard interfaces.

Operation Interfaces

Applications access the RTC through RT-Thread's I/O device management interfaces. The main APIs are as follows:

- Find RTC device

```
rt_device_t rt_device_find(const char* name);
```

- Open RTC device

```
rt_err_t rt_device_open(rt_device_t dev, rt_uint16_t oflags);
```

- Set date

```
rt_err_t set_date(rt_uint32_t year, rt_uint32_t month, rt_uint32_t day);
```

- Set time

```
rt_err_t set_time(rt_uint32_t hour, rt_uint32_t minute, rt_uint32_t second);
```

- Get current time

The current timestamp (UTC) can be obtained using the C standard library:

```
time_t time(time_t *t);
```

Alarm Feature

The RTC framework supports an **alarm mechanism**, which can be used for timed wakeups, reminders, or periodic tasks. Applications can operate alarms using the RT-Thread alarm component interfaces:

- Create alarm

```
rt_alarm_t rt_alarm_create(rt_alarm_callback_t callback, struct rt_alarm_setup
  *setup);
```

rt_alarm_setup structure definition:

```
struct rt_alarm_setup
{
    rt_uint32_t flag;               /* alarm flag */
    struct tm wktime;               /* alarm trigger time */
};
```

Alarm modes available:

```
#define RT_ALARM_ONESHOT       0x000   /* Trigger once */
#define RT_ALARM_DAILY         0x100   /* Trigger daily */
#define RT_ALARM_WEEKLY        0x200   /* Trigger weekly */
#define RT_ALARM_MONTHLY       0x400   /* Trigger monthly */
#define RT_ALARM_YEARLY        0x800   /* Trigger yearly */
#define RT_ALARM_HOUR          0x1000  /* Trigger every hour */
#define RT_ALARM_MINUTE        0x2000  /* Trigger every minute */
#define RT_ALARM_SECOND        0x4000  /* Trigger every second */
```

- Start alarm

```
rt_err_t rt_alarm_start(rt_alarm_t alarm);
```

- Stop alarm

```
rt_err_t rt_alarm_stop(rt_alarm_t alarm);
```

- Delete alarm

```
rt_err_t rt_alarm_delete(rt_alarm_t alarm);
```

- Control alarm

```
rt_err_t rt_alarm_control(rt_alarm_t alarm, int cmd, void *arg);
```

Framework Features

- **Unified Interface**: Provides a consistent interface to abstract RTC hardware differences.
- **Cross-Platform Support**: Adaptable to various MCU RTC modules with seamless application portability.
- **Alarm Mechanism**: Supports one-shot and periodic alarm modes.
- **System Time Synchronization**: Compatible with POSIX time APIs for timestamp management.
- **Low Power Support**: Enables wake-up from deep sleep and retains time during power loss.

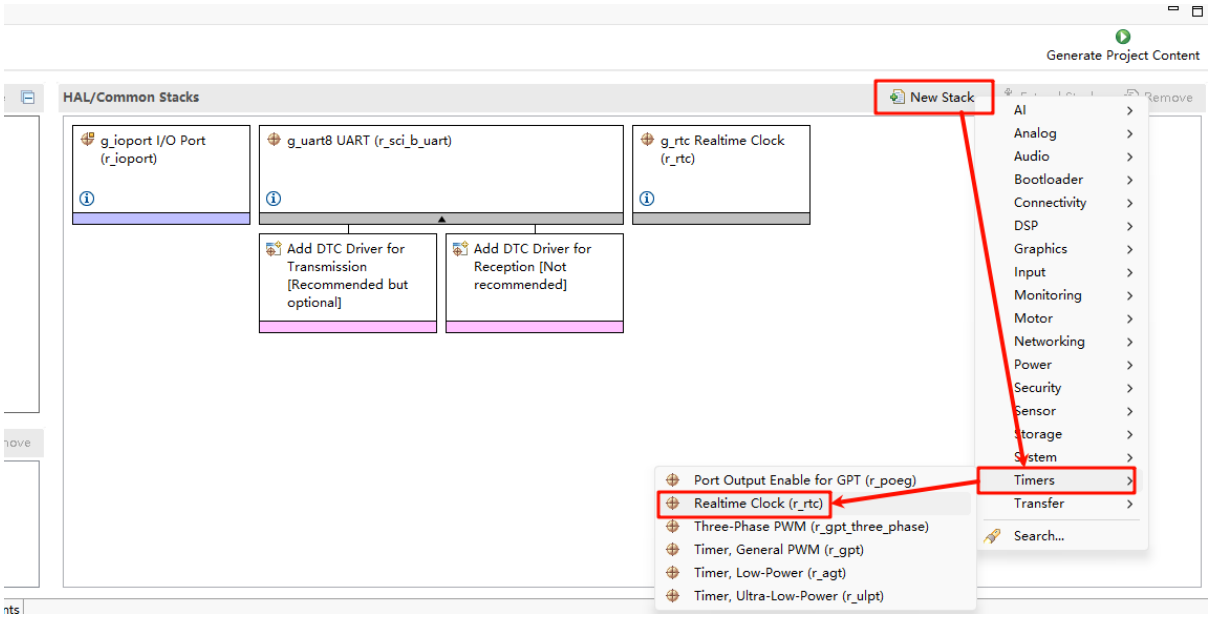
Reference: RT-Thread RTC Device

10.5 Hardware Description

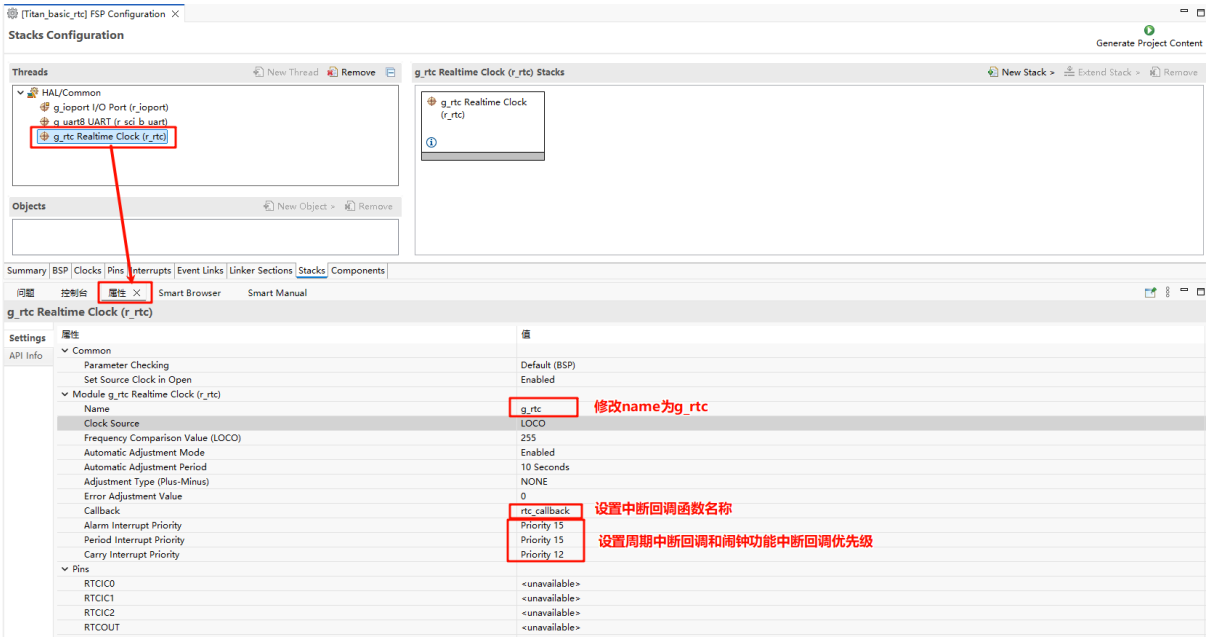
The RTC device used in this example relies on the LOCO (Low speed On Chip Oscillator) clock. No other special connections are required.

10.6 FSP Configuration

- Open the FSP configuration tool and select the configuration.xml file under the corresponding project directory. Add the RTC Stack:

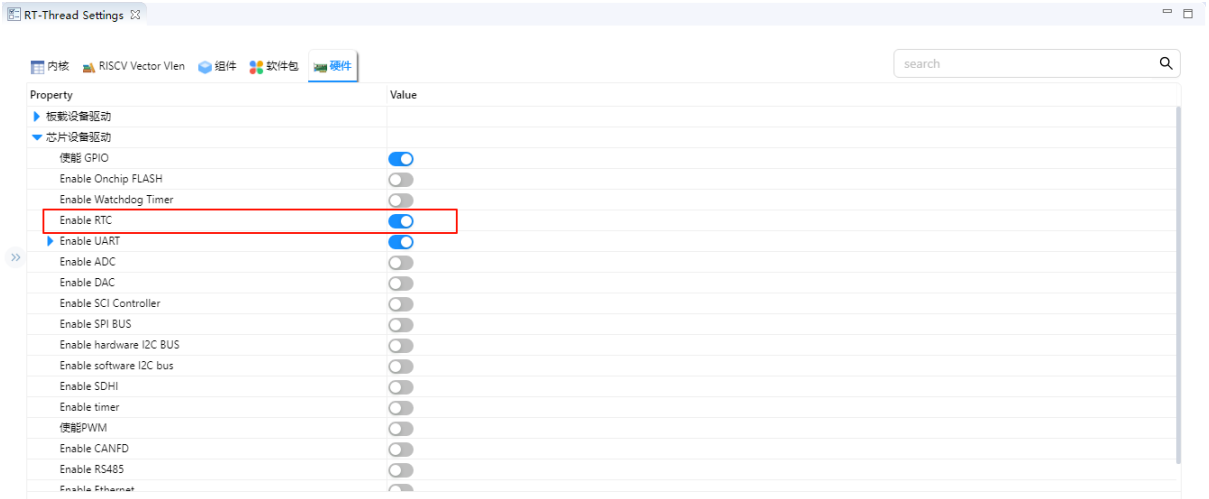


- Next, configure the RTC parameters. Set the RTC stack name to g_rtc, configure the RTC interrupt callback function to rtc_callback, and set the interrupt callback priority:

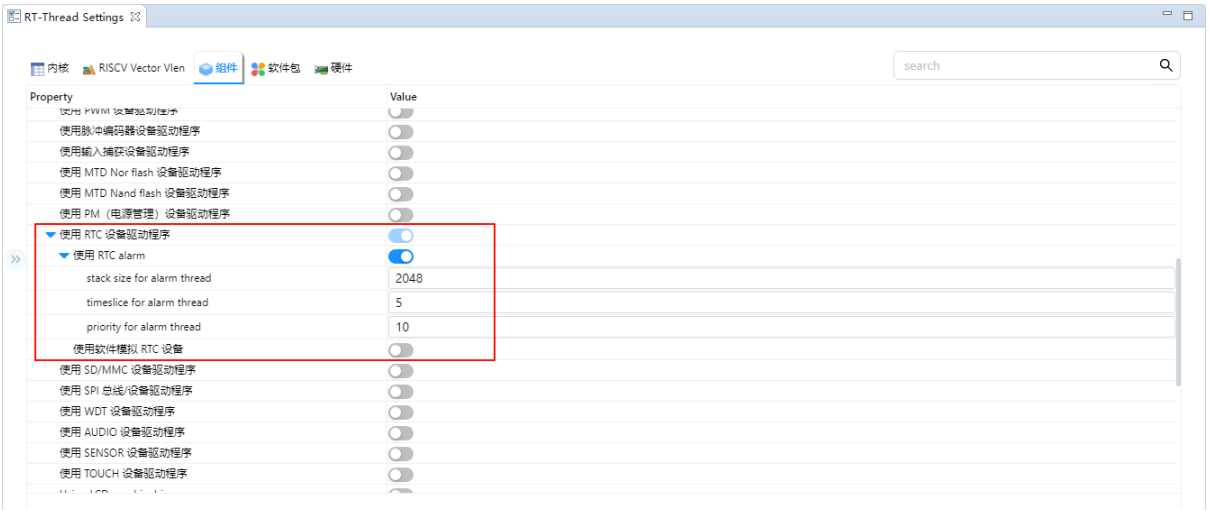


10.7 RT-Thread Settings Configuration

- Open RT-Thread Settings and enable the RTC under the hardware options:



- Next, we configure the RTC. First, we need to enable the RT-Thread RTC device framework and enable the alarm function.



10.8 Example Code Description

The source code for this example is located at /src/hal_entry.c。

```
rt_err_t ret = RT_EOK;
time_t now;
rt_device_t device = RT_NULL;

device = rt_device_find(RTC_NAME);
if (!device)
{
  rt_kprintf("find %s failed!\n", RTC_NAME);
}

if(rt_device_open(device, 0) != RT_EOK)
{
  rt_kprintf("open %s failed!\n", RTC_NAME);
}

/* 设置日期 */
ret = set_date(2025, 8, 1);
rt_kprintf("set RTC date to 2025-8-1\n");
if (ret != RT_EOK)
{
    rt_kprintf("set RTC date failed\n");
}

/* 设置时间 */
ret = set_time(15, 00, 00);
if (ret != RT_EOK)
{
    rt_kprintf("set RTC time failed\n");
```

(continues on next page)

(continued from previous page)

```
    }

    /* 延时 3 秒 */
    rt_thread_mdelay(3000);

    /* 获取时间 */
    get_timestamp(&now);
    rt_kprintf("now: %.*s", 25, ctime(&now));
```

The following code creates an RTC alarm and sets it to trigger in 1 second. Finally, the function is exported to the MSH command line.

```
void user_alarm_callback(rt_alarm_t alarm, time_t timestamp)
{
    LOG_I("user alarm callback function.");
}

void alarm_sample(void)
{
    rt_device_t dev = rt_device_find("rtc");
    struct rt_alarm_setup setup;
    struct rt_alarm * alarm = RT_NULL;
    static time_t now;
    struct tm p_tm;

    if (alarm != RT_NULL)
        return;

    /* Get the current timestamp and set the alarm to trigger in the next second
  */
    now = get_timestamp(NULL) + 1;
    gmtime_r(&now,&p_tm);

    setup.flag = RT_ALARM_SECOND;
    setup.wktime.tm_year = p_tm.tm_year;
    setup.wktime.tm_mon = p_tm.tm_mon;
    setup.wktime.tm_mday = p_tm.tm_mday;
    setup.wktime.tm_wday = p_tm.tm_wday;
    setup.wktime.tm_hour = p_tm.tm_hour;
    setup.wktime.tm_min = p_tm.tm_min;
    setup.wktime.tm_sec = p_tm.tm_sec;

    alarm = rt_alarm_create(user_alarm_callback, &setup);
    if(RT_NULL != alarm)
    {
        rt_alarm_start(alarm);
```

(continues on next page)

(continued from previous page)

```
    }
}
/* export msh cmd */
MSH_CMD_EXPORT(alarm_sample,alarm sample);
```

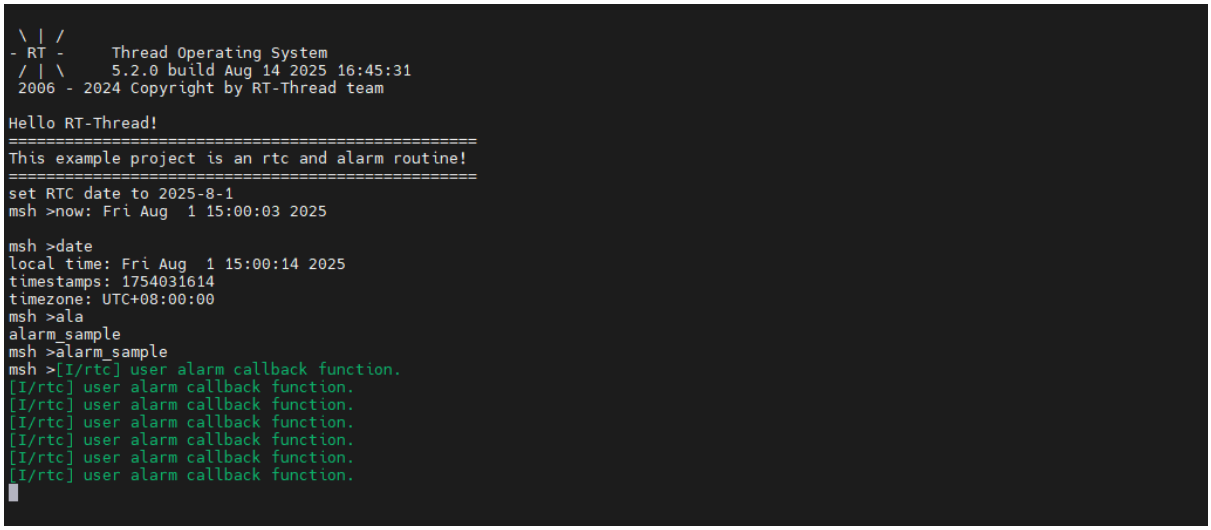
10.9 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio’s package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board’s USB-DBG interface to the PC and download the firmware to the development board.

10.10 Run Effect

Press the reset button to restart the development board, enter the alarm_sample command to run the alarm clock example, you can see the following message printed on the board:



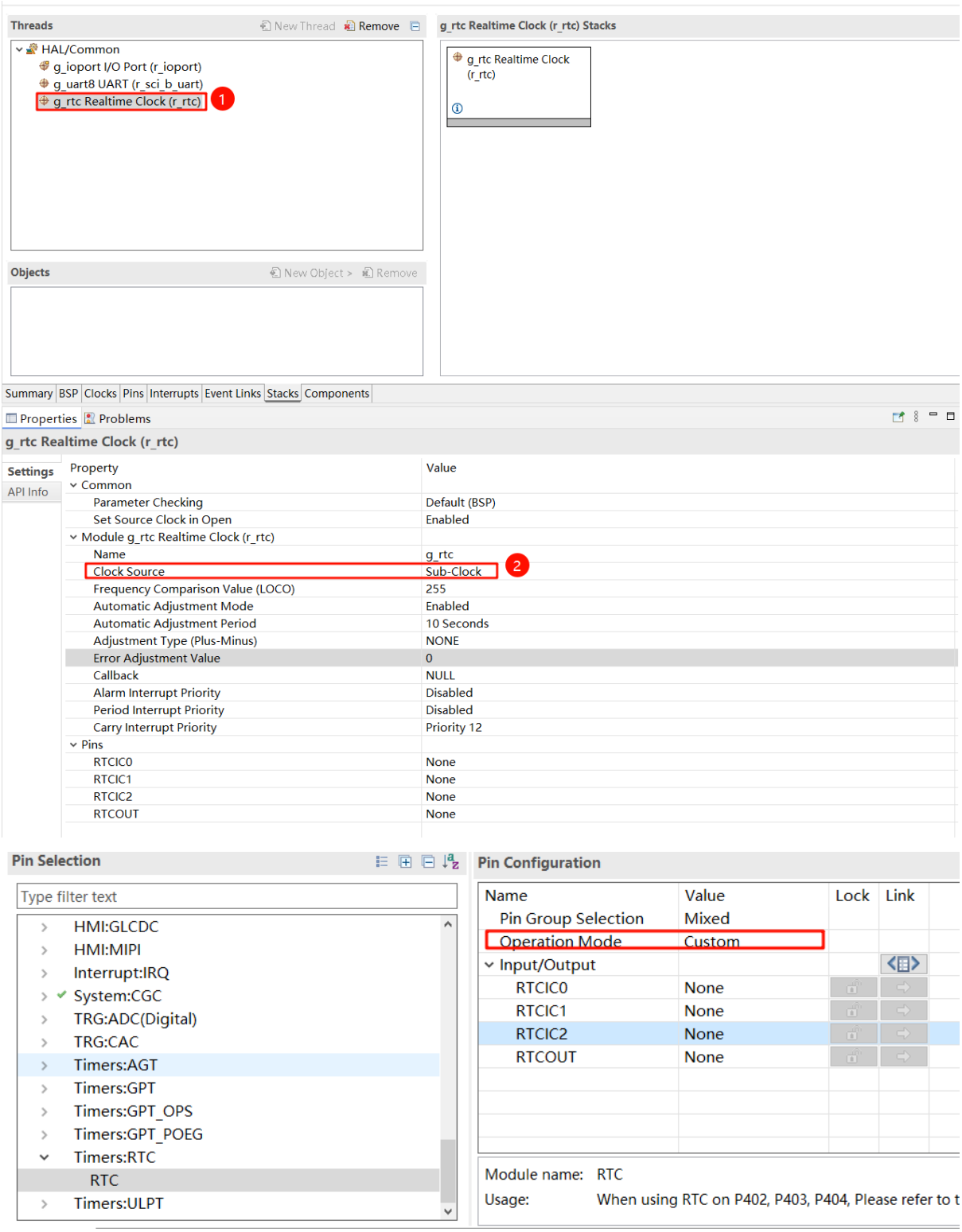
10.11 Extension: RTC Operation During Power Loss

When the main power supply of the board is disconnected, if you want the RTC to keep running during power loss, you need to connect an external power source to the **VBAT** pin. Note that when VBAT supplies power to the RTC, only the **Calendar counter** will continue to run —other RTC functions will not be available. Below is a reference code example:

ps: If the RTC is expected to run during power loss, make sure that rtc_init is only initialized **once**. Afterwards, you can use rtc_get to obtain the current time.

```
#define RTC_YEAR_SET 2025
#define RTC_MON_SET  8
#define RTC_MDAY_SET 5

#define RTC_WDAY_SET (RTC_YEAR_SET-2000 \
                  + ((RTC_YEAR_SET-2000)/4) \
                  - 35 + (26*(RTC_MON_SET+1))/10 \
                  + RTC_MDAY_SET -1 )%7

#define RTC_HOUR_SET 12
#define RTC_SEC_SET 0
#define RTC_MIN_SET 0

void rtc_get()
{
    rtc_time_t get_time;
    R_RTC_CalendarTimeGet(g_rtc.p_ctrl, &get_time);
    rt_kprintf ("%d-%d-%d-%d:%d:%d\r\n", get_time.tm_year + 1900, get_time.tm_mon
  + 1, get_time.tm_mday,
             get_time.tm_hour, get_time.tm_min, get_time.tm_sec);
}
MSH_CMD_EXPORT(rtc_get, rtc_get);

void rtc_init(void)
{
   rtc_time_t set_time =
   {
     .tm_min = RTC_MIN_SET,
     .tm_hour = RTC_HOUR_SET,
     .tm_mday = RTC_MDAY_SET,
     .tm_wday = RTC_WDAY_SET,
     .tm_mon = RTC_MON_SET - 1,
     .tm_year = RTC_YEAR_SET-1900,
   };

   R_RTC_Open (g_rtc.p_ctrl, g_rtc.p_cfg);

   R_RTC_CalendarTimeSet (g_rtc.p_ctrl, &set_time);
}
MSH_CMD_EXPORT(rtc_init, rtc_init);
```

FSP Configuration:


Threads
New Thread
Remove
HAL/Common
g_ioport I/O Port (r_ioport)
g_uart8 UART (r_sci_b_uart)
g_rtc Realtime Clock (r_rtc)
1
g_rtc Realtime Clock (r_rtc) Stacks
g_rtc Realtime Clock (r_rtc)
Objects
New Object >
Remove
Summary
BSP
Clocks
Pins
Interrupts
Event Links
Stacks
Components
Properties
Problems
g_rtc Realtime Clock (r_rtc)
Settings
API Info
Property
Value
Common
Parameter Checking
Default (BSP)
Set Source Clock in Open
Enabled
Module g_rtc Realtime Clock (r_rtc)
Name
g_rtc
Clock Source
Sub-Clock
2
Frequency Comparison Value (LOCO)
255
Automatic Adjustment Mode
Enabled
Automatic Adjustment Period
10 Seconds
Adjustment Type (Plus-Minus)
NONE
Error Adjustment Value
0
Callback
NULL
Alarm Interrupt Priority
Disabled
Period Interrupt Priority
Disabled
Carry Interrupt Priority
Priority 12
Pins
RTCIC0
None
RTCIC1
None
RTCIC2
None
RTCOUT
None
Pin Selection
Type filter text
HMI:GLCDC
HMI:MIPI
Interrupt:IRQ
System:CGC
TRG:ADC(Digital)
TRG:CAC
Timers:AGT
Timers:GPT
Timers:GPT_OPS
Timers:GPT_POEG
Timers:RTC
RTC
Timers:ULPT
Pin Configuration
Name
Value
Lock
Link
Pin Group Selection
Mixed
Operation Mode
Custom
Input/Output
RTCIC0
None
RTCIC1
None
RTCIC2
None
RTCOUT
None
Module name:
RTC
Usage:
When using RTC on P402, P403, P404, Please refer to t

11. SD Card File System Usage Instructions

English|Chinese

11.1 Introduction

This routine uses the SD card in the SD card slot on the development board as the storage device for the file system, shows how to create a file system (format card) on the SD card, and mount the file system into the RT-Thread operating system.

Once the filesystem is mounted, it shows how to use the functionality provided by the filesystem to manipulate directories and files.

SD Card Introduction

Overview

SD Card (Secure Digital Card) is a small, portable, non-volatile storage device widely used in **embedded systems, cameras, mobile phones, and data loggers**.

An SD card is composed of a **controller + NAND Flash memory chip** and communicates with the host through a standard interface.

Key features:

- Compact and lightweight, typically **32 × 24 × 2.1 mm** (standard card)
- Stores data using **non-volatile NAND Flash**
- Supports hot-plugging and power-off protection

SD Card Types

1. **By size**
   - **Standard SD**: 32 × 24 mm
   - **Mini SD**: 21.5 × 20 mm
   - **Micro SD (TF Card)**: 15 × 11 mm, most commonly used
2. **By capacity**

Type	Capacity Range
SDSC	1 MB ~ 2 GB
SDHC	4 GB ~ 32 GB
SDXC	32 GB ~ 2 TB
SDUC	2 TB ~ 128 TB

1. **By speed class**

- **Class 2/4/6/10**: Minimum write speed of 2, 4, 6, 10 MB/s
- **UHS (Ultra High Speed)**: UHS-I/UHS-II/UHS-III, up to **312 MB/s**
- **Video Speed Class (V6/V10/V30/V60/V90)**: Designed for HD/4K video recording

SD Card Interfaces

1. **SPI Mode**
   - Uses SPI bus (MISO, MOSI, SCK, CS)
   - Simple and MCU-friendly
   - Lower data transfer rate
2. **SD Mode (1-bit / 4-bit)**
   - Uses dedicated SD bus
   - Supports 1-bit or 4-bit data lines
   - Faster than SPI mode
3. **UHS Mode**
   - Supports high-speed data transfer
   - Commonly used in cameras and high-performance embedded applications

Working Principle

1. **Command/Data Transfer**
   - Host sends commands (CMD) via SD protocol
   - Card returns responses (R1, R2, etc.)
   - Data read/write in **blocks of 512 bytes**
2. **Controller Management**
   - Internal controller handles **bad block management, ECC, logical-to-physical mapping**
   - Host does not directly manage NAND Flash
3. **Data Storage**
   - Data is stored in NAND Flash
   - Supports multiple erasures with endurance (typically 100k cycles)

SD Card Performance

Parameter	Description
Capacity	1 GB ~ 128 TB
Block Size	512 Bytes (standard)
Interface Speed	SPI / SD 1-bit/4-bit / UHS
Max Speed	25 MB/s (standard), 312 MB/s (UHS-III)
Operating Voltage	3.3 V (some Micro SD support 1.8 V)
Temp Range	-25 ℃ ~ 85 ℃ (industrial grade)
Endurance	10^4 ~ 10^5 erase cycles

SD Card Applications

1. **Consumer Electronics**
   - Phones, tablets, cameras, camcorders
2. **Embedded Systems**
   - MCU/FPGA data storage
   - Logging, config file storage
3. **Industrial Applications**
   - Controllers, data acquisition systems
   - Industrial-grade SD cards for harsh environments
4. **Audio/Video**
   - High-speed video recording (UHS/V Class)
5. **Automotive**
   - Dashcams, navigation systems

RA8 Series SDHI Module Overview

The RA8 series MCU integrates a high-performance **SDHI module** designed for high-speed communication with SD/SDHC/SDXC cards. It supports both **SPI mode** and **SD/SDIO mode**.

Key Features

- **SD Standards Support**
  - SD v1.x / v2.x / SDHC / SDXC
  - SPI mode & SD/MMC mode
- **High-Speed Data Transfer**

  - Up to 50 MHz SDCLK (depending on MCU clock)
  - 1-bit/4-bit data bus support
- **Auto Command & Data Transfer**
  - DMA support to reduce CPU load
  - Auto command sequence generation (CMD0–CMD59)
- **Error Detection**
  - CRC7 for commands, CRC16 for data
  - Timeout & response error detection
- **Interrupt Support**
  - Card insert/remove interrupt
  - Command complete interrupt
  - Data transfer complete interrupt
  - Error interrupt

SDHI Module Architecture

1. **Command Control Unit**
   - Sends SD commands (CMD0–CMD59)
   - Handles responses (R1, R2, R3, R7)
   - Supports timeout detection and CRC check
2. **Data Transfer Unit**
   - Transfers data via FIFO or DMA
   - Block size up to 512 bytes
   - Supports single/multi-block transfer
3. **Clock & Bus Control**
   - Generates/divides SDCLK
   - Switches between 1-bit/4-bit modes
   - Configurable signal timings
4. **Card Detect & Power Control**
   - Detects card insertion/removal
   - Can control card power if supported
5. **Interrupt/Event Controller**
   - Command complete interrupt
   - Data transfer complete interrupt

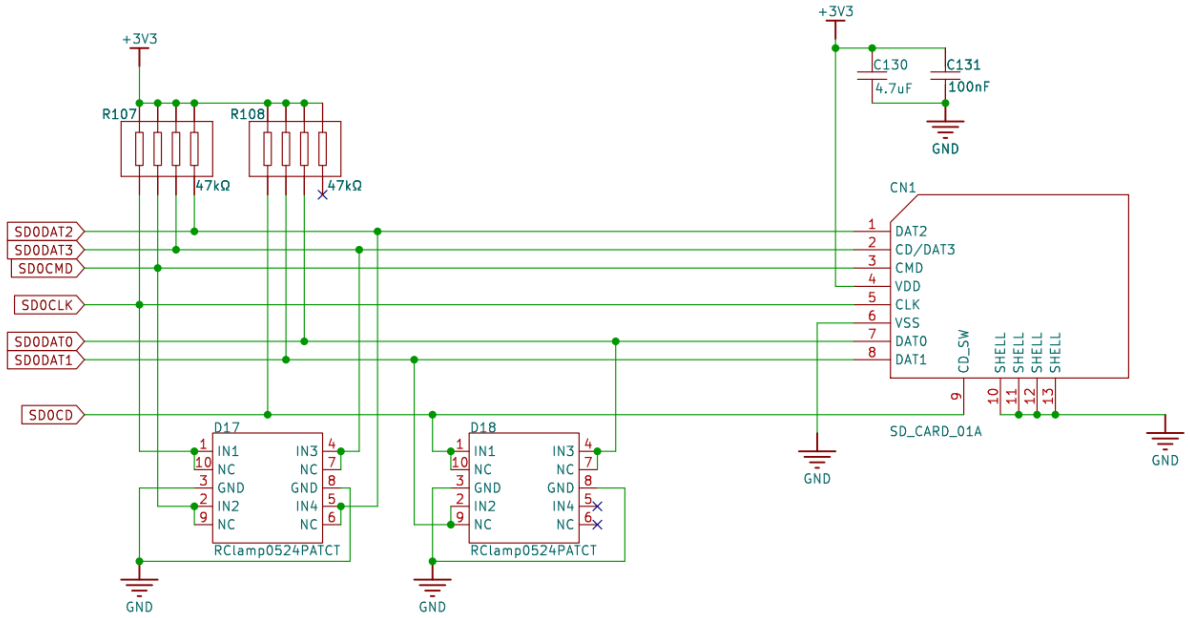
- Error interrupt
- Card detect interrupt

SDHI Working Principle

1. **Initialization**
   - Detect card insertion
   - Send CMD0, CMD8 for initialization
   - Query card capacity & version
2. **Command Transmission**
   - Host sends commands
   - Card responds with CRC check & triggers interrupt
3. **Data Transfer**
   - Reads/writes via FIFO or DMA
   - Supports single/multi-block operations
4. **Error Handling**
   - Timeout, CRC, and response errors
   - SDHI triggers error interrupt, driver retries or recovers

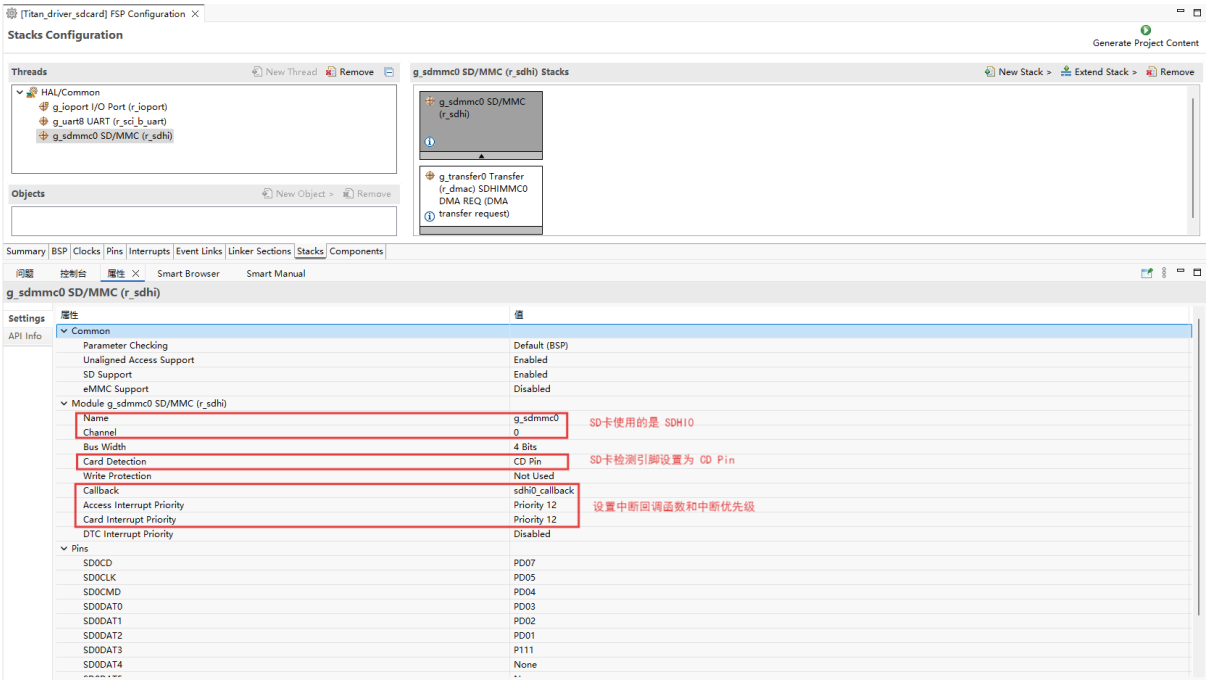
11.2 Hardware Description

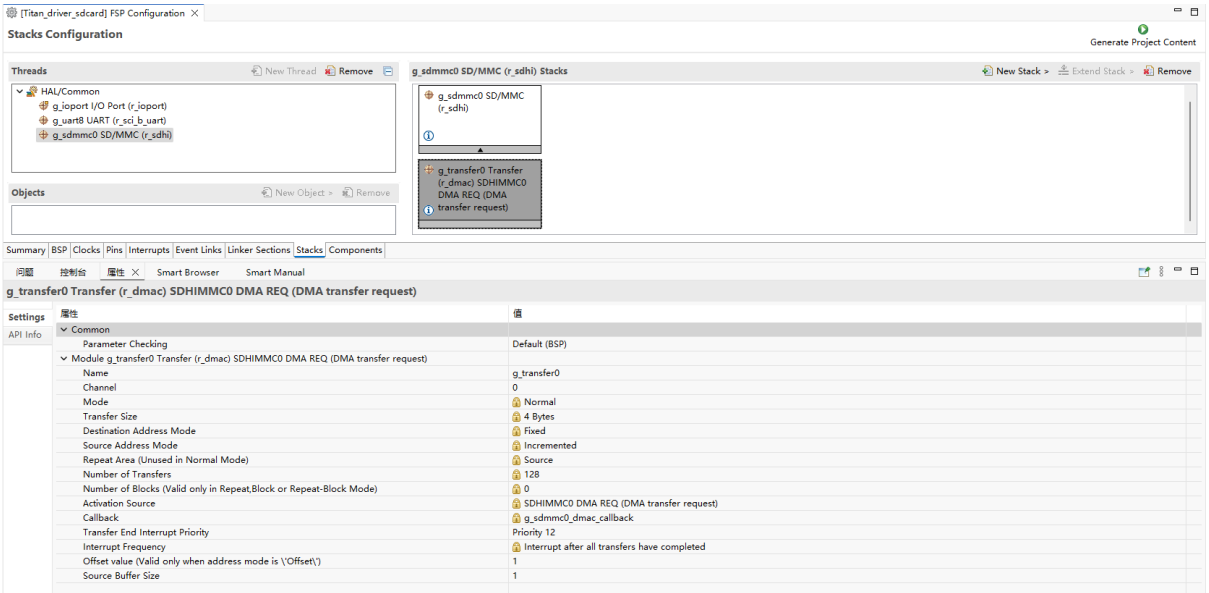
This example is connected to the memory through the SDIO interface, using the SD0 of the hardware, the schematic diagram is as follows:



11.3 FSP Configuration

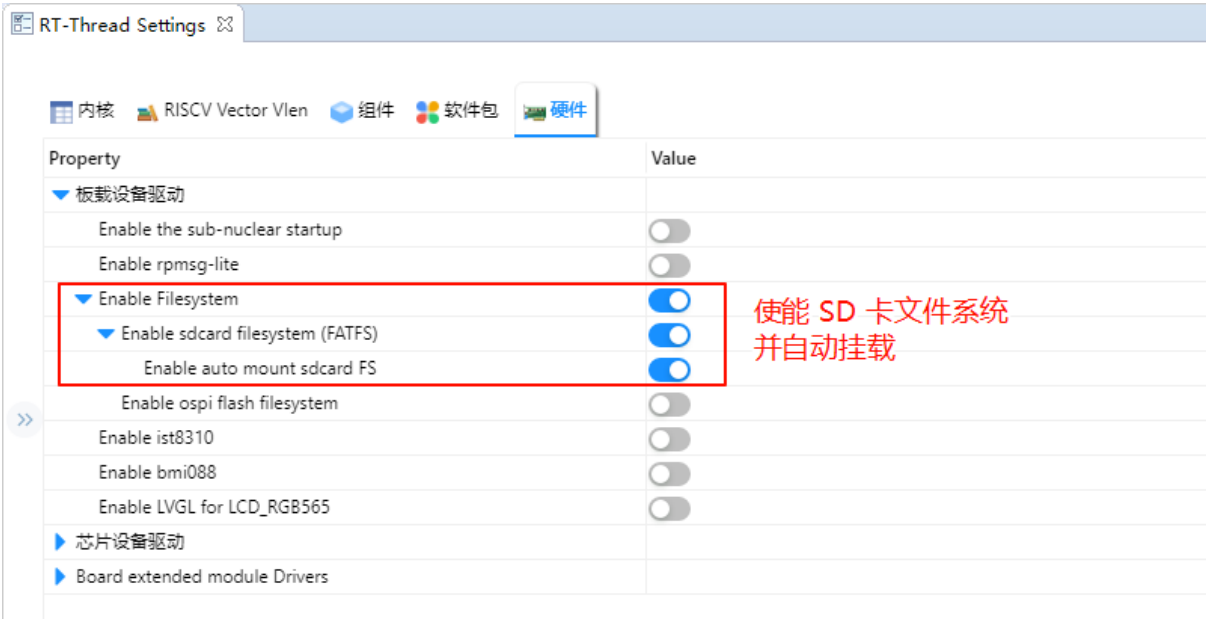
- Create stacks select r_sdhi and configure sdhi0 configuration information as follows:



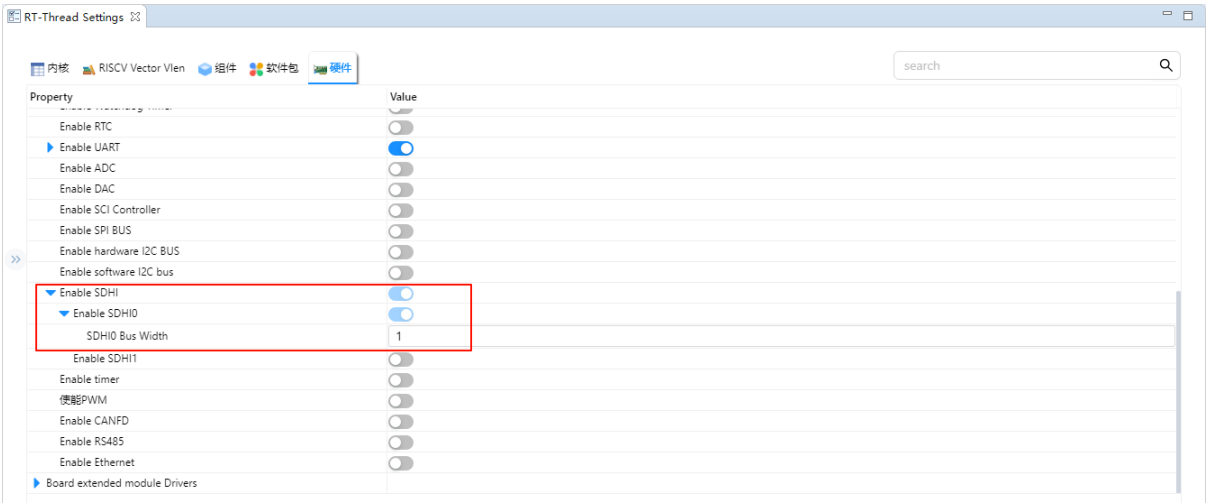


11.4 RT-Thread Settings Configuration

- Enable the SD card file system in the configuration.



- Enable SDHI0 and set the Bus Width of SDHI0 to 1.



11.5 Example Project Description

This routine file system initialization source in ./board/ports/drv_filesystem.c：

```
#include <rtthread.h>

#if defined(BSP_USING_FILESYSTEM)
#include <dfs_romfs.h>
#include <dfs_fs.h>
```

(continues on next page)

(continued from previous page)

```
#include <dfs_file.h>

#if DFS_FILESYSTEMS_MAX < 4
#error "Please define DFS_FILESYSTEMS_MAX more than 4"
#endif
#if DFS_FILESYSTEM_TYPES_MAX < 4
#error "Please define DFS_FILESYSTEM_TYPES_MAX more than 4"
#endif

#define DBG_TAG "app.filesystem"
#define DBG_LVL DBG_INFO
#include <rtdbg.h>

#ifdef BSP_USING_FS_AUTO_MOUNT
#ifdef BSP_USING_SDCARD_FATFS
static int onboard_sdcard_mount(void)
{
    if (dfs_mount("sd", "/sdcard", "elm", 0, 0) == RT_EOK)
    {
        LOG_I("SD card mount to '/sdcard'");
    }
    else
    {
        LOG_E("SD card mount to '/sdcard' failed!");
        rt_pin_write(0x000D, PIN_LOW);
    }

    return RT_EOK;
}
#endif /* BSP_USING_SDCARD_FATFS */
#endif /* BSP_USING_FS_AUTO_MOUNT */

#ifdef BSP_USING_FLASH_FS_AUTO_MOUNT
#ifdef BSP_USING_FLASH_FATFS
#define FS_PARTITION_NAME "filesystem"

static int onboard_fal_mount(void)
{
    /* 初始化 fal 功能 */
    extern int fal_init(void);
    extern struct rt_device* fal_mtd_nor_device_create(const char *parition_
  name);
    fal_init ();
    /* 在 ospi flash 中名为 "filesystem" 的分区上创建一个块设备 */
    struct rt_device *mtd_dev = fal_mtd_nor_device_create (FS_PARTITION_NAME);
    if (mtd_dev == NULL)
```

(continues on next page)

(continued from previous page)

```
    {
        LOG_E("Can't create a mtd device on '%s' partition.", FS_PARTITION_NAME);
        return -RT_ERROR;
    }
    else
    {
        LOG_D("Create a mtd device on the %s partition of flash successful.", FS_
  PARTITION_NAME);
    }

    /* 挂载 ospi flash 中名为 "filesystem" 的分区上的文件系统 */
    if (dfs_mount (FS_PARTITION_NAME, "/fal", "lfs", 0, 0) == 0)
    {
        LOG_I("Filesystem initialized!");
    }
    else
    {
        dfs_mkfs ("lfs", FS_PARTITION_NAME);
        if (dfs_mount ("filesystem", "/fal", "lfs", 0, 0) == 0)
        {
            LOG_I("Filesystem initialized!");
        }
        else
        {
            LOG_E("Failed to initialize filesystem!");
            rt_pin_write(0x000D, PIN_LOW);
        }
    }

    return RT_EOK;
}
#endif /*BSP_USING_FLASH_FATFS*/
#endif /*BSP_USING_FLASH_FS_AUTO_MOUNT*/

const struct romfs_dirent _romfs_root[] =
{
#ifdef BSP_USING_SDCARD_FATFS
    {ROMFS_DIRENT_DIR, "sdcard", RT_NULL, 0},
#endif

#ifdef BSP_USING_FLASH_FATFS
  { ROMFS_DIRENT_DIR, "fal", RT_NULL, 0 },
#endif
        };

const struct romfs_dirent romfs_root =
```

(continues on next page)

(continued from previous page)

```
{
ROMFS_DIRENT_DIR, "/", (rt_uint8_t*) _romfs_root, sizeof(_romfs_root) / sizeof(_
  romfs_root[0])
};

static int filesystem_mount(void)
{

#ifdef RT_USING_DFS_ROMFS
    if (dfs_mount(RT_NULL, "/", "rom", 0, &(romfs_root)) != 0)
    {
        LOG_E("rom mount to '/' failed!");
    }

    /* 确保块设备注册成功之后再挂载文件系统 */
    rt_thread_delay(500);
#endif
#ifdef BSP_USING_FS_AUTO_MOUNT
    onboard_sdcard_mount();
#endif /* BSP_USING_FS_AUTO_MOUNT */

#ifdef BSP_USING_FLASH_FS_AUTO_MOUNT
    onboard_fal_mount ();
#endif

    return RT_EOK;
}
INIT_COMPONENT_EXPORT(filesystem_mount);
#endif /* defined(BSP_USING_FILESYSTEM)*/
```

11.6 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio's package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board's USB-DBG interface to the PC and download the firmware to the development board.

11.7 Run Effect

Press the reset button to restart the development board, wait for the SD to mount, and enter the file system directory of the SD card to view the files on the SD card.

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Jul 28 2025 18:20:14
 2006 - 2024 Copyright by RT-Thread team
msh />[I/SDIO] SD card capacity 3932160 KB.
[I/SDIO] sd: switch to High Speed / SDR25 mode

found part[0], begin: 65536, size: 3.766GB
[I/app.filesystem] SD card mount to '/sdcard'

Hello RT-Thread!
================================================
This example project is the sdio routine!
================================================

msh />cd sdcard
msh /sdcard>ls
Directory /sdcard:
System Volume Inform<DIR>
TEST                <DIR>
C.TXT               5
hello.txt           <DIR>
test.txt            8
Song.wav            44982044
1.wav               0
tt.wav              50732
111.wav             0
2.wav               20268
SeeYouAgain.wav     40488498
msh /sdcard>
```

12. BMI088 Gyroscope Usage Instructions

English | Chinese

12.1 Introduction

This example demonstrates how to use the **SPI module of the RA8 series MCU** on the **Titan Board**, in combination with the **RT-Thread SPI driver framework** and the **BMI088 software package (a 6-axis IMU: 3-axis accelerometer + 3-axis gyroscope)**, to perform sensor initialization and data acquisition.

12.2 Overview of BMI088

General Description

The **BMI088** is a **high-performance 6-axis IMU (Inertial Measurement Unit)** developed by Bosch Sensortec, integrating a **3-axis accelerometer** and a **3-axis gyroscope**.

It is designed for **drones, robotics, industrial automation, and navigation systems**, where high stability and low noise are required.

The BMI088 is well known for its **high vibration robustness** and **low noise performance**. Compared with common consumer-grade IMUs (such as MPU6050, BMI160, etc.), it is more widely used in industrial and UAV applications.

Key Features

(1) Accelerometer

- Measurement range: ±3 g / ±6 g / ±12 g / ±24 g
- Noise density: **approx. 120 µg/√Hz**
- Sampling rate: up to **1.6 kHz**
- Offset: factory calibrated
- Features: high stability, low noise, wide dynamic range

(2) Gyroscope

- Measurement range: ±125 °/s, ±250 °/s, ±500 °/s, ±1000 °/s, ±2000 °/s
- Noise density: **approx. 0.014 °/s/√Hz**
- Sampling rate: up to **2 kHz**
- Built-in **vibration suppression design** to reduce the impact of high-frequency vibration
- Excellent bias stability, suitable for long-term operation

(3) Interface & Power Supply

- **Communication interface**: SPI (up to 10 MHz), I^2C (up to 400 kHz)
- Operating voltage: 1.71V ~ 3.6V
- Package: 3 mm × 4.5 mm × 0.95 mm (LGA-16)

Working Principle

- **Accelerometer**: Detects acceleration in three axes using MEMS structures and outputs corresponding digital signals.
- **Gyroscope**: Measures angular velocity based on the Coriolis effect, providing three-axis rotational motion data.
- **Sensor Fusion**: Combining accelerometer and gyroscope data allows for attitude estimation (Pitch, Roll, Yaw).

Main Advantages

1. **High vibration robustness**
   - The gyroscope features a special anti-vibration structure, making it suitable for drones and other high-vibration environments.
2. **Low-noise performance**
   - Both accelerometer and gyroscope deliver low-noise output, ensuring accurate attitude estimation.
3. **Industrial-grade reliability**
   - More stable and less temperature drift compared to consumer-grade IMUs (e.g., BMI160, MPU6050).
4. **High bandwidth**
   - The gyroscope bandwidth can reach up to 2 kHz, making it ideal for high-speed control scenarios.

Application Scenarios

- **Drones**: flight attitude detection, navigation, and stabilization
- **Robotics**: motion control, SLAM, and balance control
- **Industrial measurement**: platform stabilization, robotic arm motion detection
- **Automotive systems**: inertial navigation, assisted positioning
- **Motion tracking**: VR/AR, wearable devices

12.3 RA8 Series SPI Module (r_spi_b) Instructions

The RA8 series MCU integrates an **enhanced SPI module (r_spi_b)**, which can be used for high-speed serial communication with peripherals such as Flash memory, sensors, displays, and audio codecs. This module supports both master and slave modes, provides full-duplex communication, and integrates rich buffering, clock control, and DMA mechanisms at the hardware level, effectively reducing CPU workload.

Key Features

- **Operating Modes**
  - Supports **Master mode** and **Slave mode**
  - Supports **Full-duplex communication** (simultaneous transmit and receive)
  - Supports **Half-duplex mode** for unidirectional communication devices
- **Clock Features**
  - Supports serial clock (SPCLK) up to **166 MHz** (depending on system clock and device specifications)
  - Configurable **polarity (CPOL)** and **phase (CPHA)**, supporting standard **SPI Modes 0/1/2/3**
- **Data Format**
  - Supports 8-bit, 16-bit, and 32-bit frame formats
  - Supports **MSB-first** and **LSB-first** transmission
- **Chip Select Management**
  - Supports **hardware chip select (SSL0~SSL3)**
  - Also supports **software-controlled GPIO chip select**
- **Buffering & DMA**
  - Built-in **FIFO buffers** to reduce interrupt frequency
  - Supports **DTC/DMA** automatic transfer, ideal for large data transactions
- **Interrupts & Events**
  - Transmit complete interrupt
  - Receive complete interrupt
  - Transfer error interrupt (e.g., overflow, underrun)

SPI Module Architecture

The RA8 series SPI (r_spi_b) consists of the following submodules:

1. **Clock Control Unit**
   - Generates the serial clock (SPCLK)
   - Configurable prescalers for adjusting communication speed

- Provides polarity (CPOL) and phase (CPHA) settings

2. **Transfer Control Unit**
   - Controls data frame length (8/16/32-bit)
   - Configures data order (MSB/LSB)
   - Manages transmit/receive direction
3. **FIFO Buffers**
   - Transmit FIFO
   - Receive FIFO
   - Reduce interrupt frequency, improve performance
4. **Chip Select Control Unit**
   - Provides up to 4 independent chip select signals (SSL0~SSL3)
   - Automatic assertion/deassertion of chip select
   - Supports multiple slave devices
5. **DMA/DTC Support**
   - SPI module can directly cooperate with DMA/DTC
   - Enables **CPU-free streaming data transfer**
   - Especially useful for sensor data acquisition, external memory access, etc.
6. **Interrupt Control**
   - Transmit interrupt (TXI)
   - Receive interrupt (RXI)
   - Transfer end interrupt (TEI)
   - Error interrupt (ERI)

Working Principle

1. **Master Mode Communication**
   - Configure SPCLK, CPOL, and CPHA
   - Configure chip select signal (SSLx)
   - Write data into transmit FIFO
   - SPI hardware automatically shifts out data while receiving incoming data
2. **Slave Mode Communication**
   - Wait for chip select signal from master
   - Receive data from master while optionally transmitting response data
   - Supports DMA for writing received data into memory

3. **FIFO & DMA Coordination**
    - Short transfers: handled via interrupts
    - Large data transfers: automatically handled via DMA/DTC, CPU only needs to configure initial parameters

12.4 RT-Thread SPI Driver Framework

The RT-Thread SPI (Serial Peripheral Interface) framework is a unified interface provided by the RT-Thread device driver layer to manage SPI peripherals across different MCUs. It abstracts low-level hardware register operations, allowing applications to communicate with SPI slave devices via standardized APIs in an efficient, reliable, and portable way.

Device Model

In RT-Thread, the SPI system consists of **buses** and **devices**. SPI devices are managed as **device objects** (subclass of struct rt_device, type RT_Device_Class_SPIBUS or RT_Device_Class_SPIDevice). Developers do not need to access hardware registers directly —all send, receive, and chip-select operations can be performed through standard RT-Thread SPI APIs.

Operation Interfaces

Applications interact with SPI devices through RT-Thread’s device management interfaces. Common APIs include:

- Find SPI device

```
rt_device_t rt_device_find(const char* name);
```

- Custom message transfer

```
struct rt_spi_message *rt_spi_transfer_message(struct rt_spi_device *device,
                                               struct rt_spi_message *message);
```

This function allows transferring a sequence of messages. Users can define multiple message structures to precisely control chip-select behavior and transfer sequence. The struct rt_spi_message is defined as follows:

```
struct rt_spi_message
{
    const void *send_buf;           /* Send buffer pointer */
    void *recv_buf;                 /* Receive buffer pointer */
    rt_size_t length;               /* Number of bytes to transfer */
    struct rt_spi_message *next;    /* Pointer to next message */
    unsigned cs_take    : 1;        /* Chip select active */
```

(continues on next page)

(continued from previous page)

```
    unsigned cs_release : 1;        /* Chip select release */
};
```

- Transfer a single block of data

```
rt_size_t rt_spi_transfer(struct rt_spi_device *device,
                          const void           *send_buf,
                          void                 *recv_buf,
                          rt_size_t             length);
```

- Send-only transfer (ignore received data)

```
rt_size_t rt_spi_send(struct rt_spi_device *device,
                      const void           *send_buf,
                      rt_size_t             length);
```

- Receive-only transfer

```
rt_size_t rt_spi_recv(struct rt_spi_device *device,
                      void                 *recv_buf,
                      rt_size_t             length);
```

- Send two buffers continuously (without releasing chip select)

```
rt_err_t rt_spi_send_then_send(struct rt_spi_device *device,
                               const void           *send_buf1,
                               rt_size_t             send_length1,
                               const void           *send_buf2,
                               rt_size_t             send_length2);
```

- Send then receive data (without releasing chip select)

```
rt_err_t rt_spi_send_then_recv(struct rt_spi_device *device,
                               const void           *send_buf,
                               rt_size_t             send_length,
                               void                 *recv_buf,
                               rt_size_t             recv_length);
```

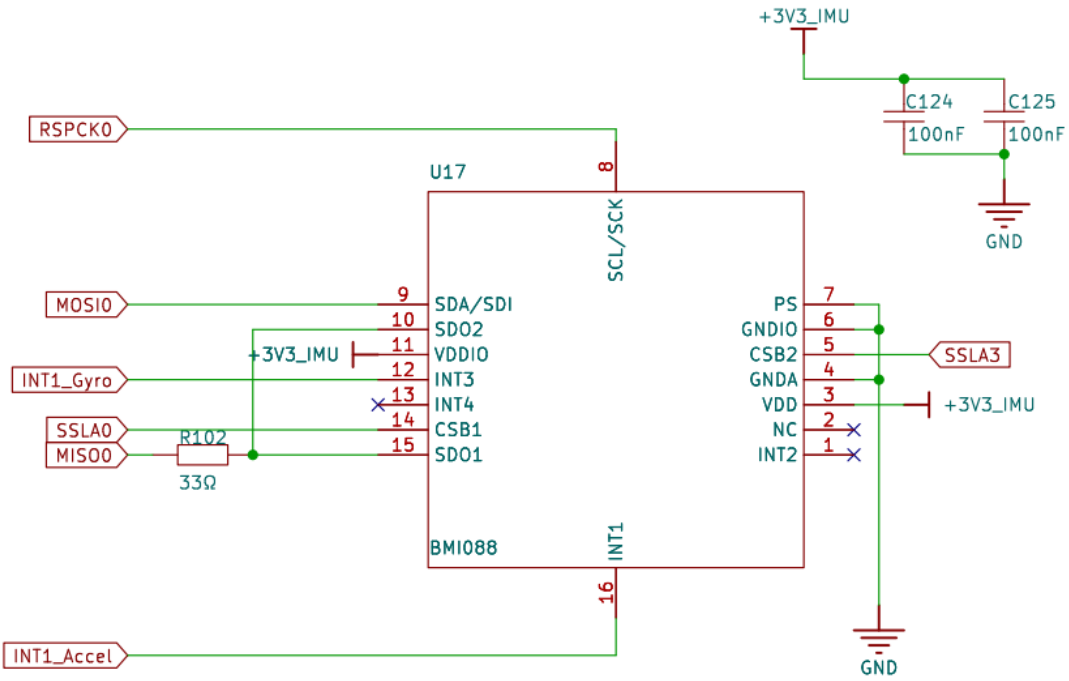
Framework Features

- **Unified abstraction**: Supports both master and slave communication through standard APIs.
- **Cross-platform**: Portable across different MCU platforms with no code changes.
- **Flexible chip-select control**: Supports single, continuous, and combined transfer modes.
- **Message chaining**: Enables complex multi-message transfers via rt_spi_message.
- **High extensibility**: Easily integrates with DMA or RT-Thread IPC mechanisms for better performance.

Reference: RT-Thread SPI Device

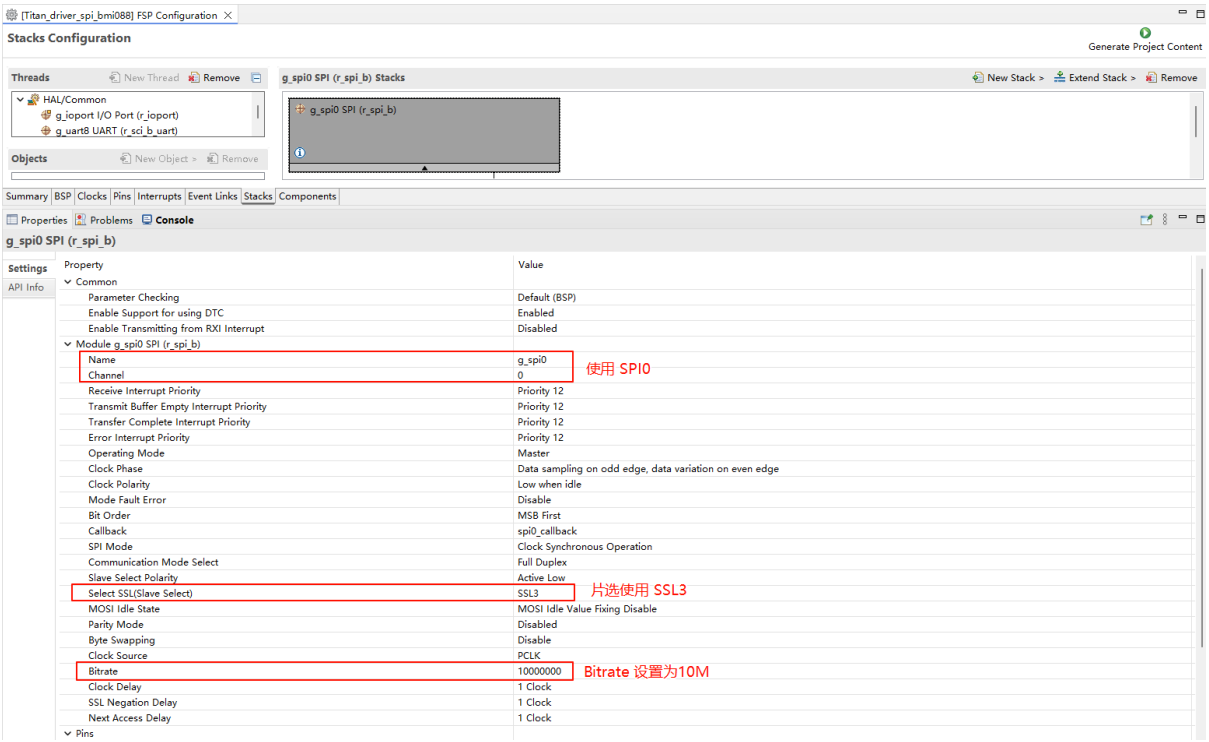
12.5 Hardware Description

The Titan Board uses SPI0 to communicate with the BMI088 gyroscope.

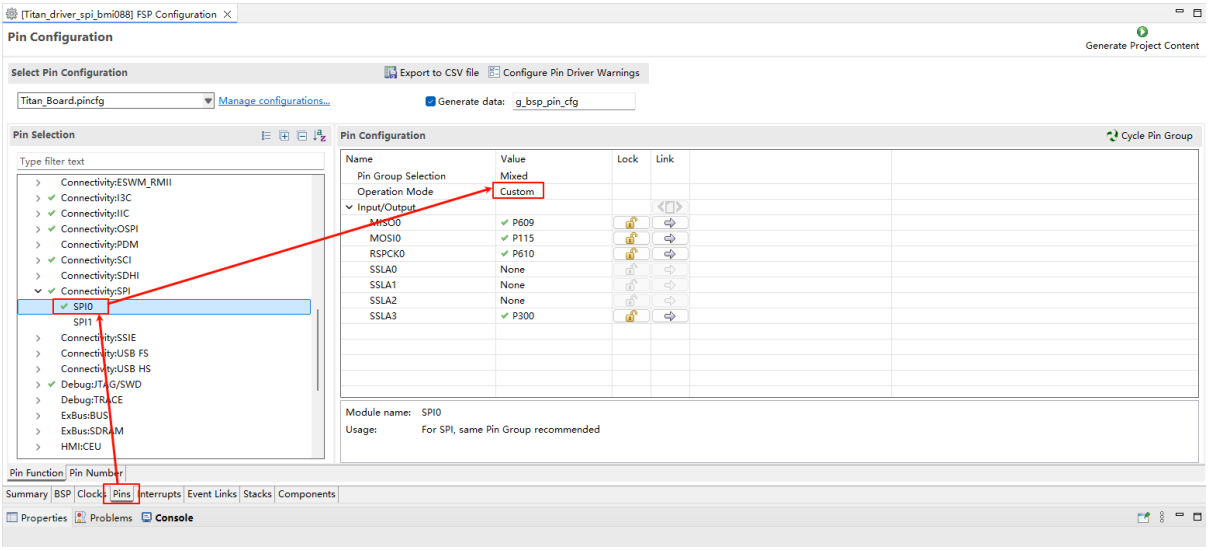


12.6 FSP Configuration

- Open the FSP tool, create a new stack, and select r_spi_b:

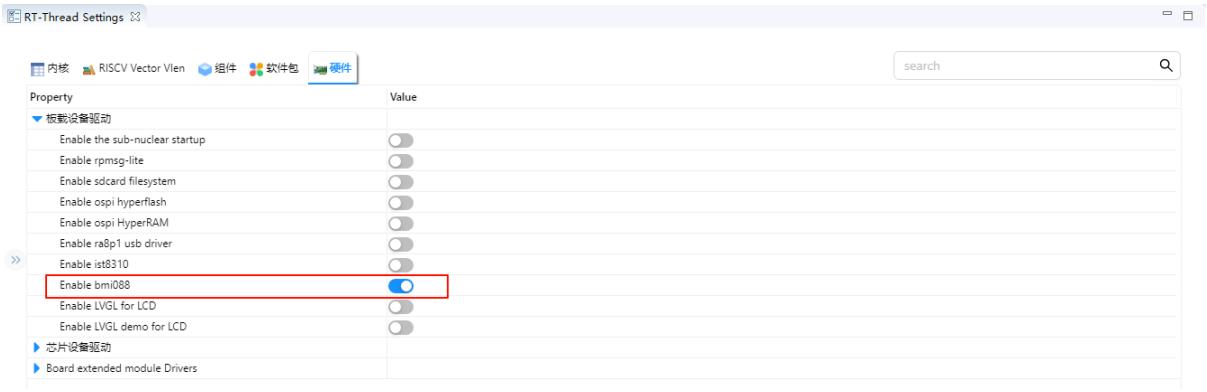


- Configure SPI0 pins:

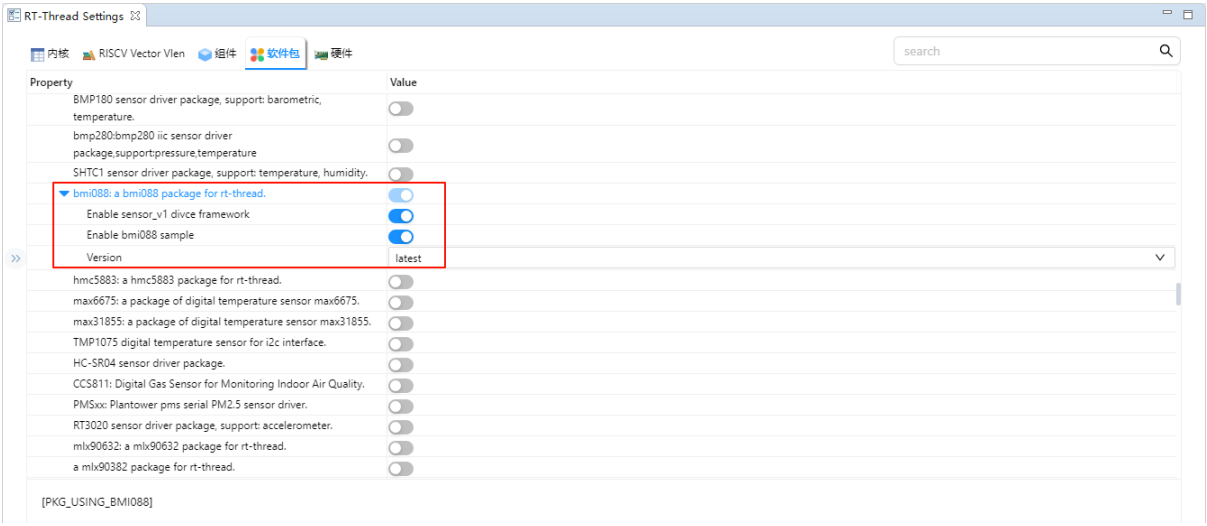


12.7 RT-Thread Settings Configuration

- Enable bmi088:



- Enable sensor_v1 and sample in the bmi088 package configuration:



12.8 Software Description

Using the BMI088 package on the Titan Board requires the following modifications for adaptation:

- Modify ./packages/bmi088-latest/sample/bmi088_sample.c:

Modify the SPI BUS to spi0, and modify the chip selection pin:

RT-Thread Settings | drv_filesystem.c | bmi088_sample.c

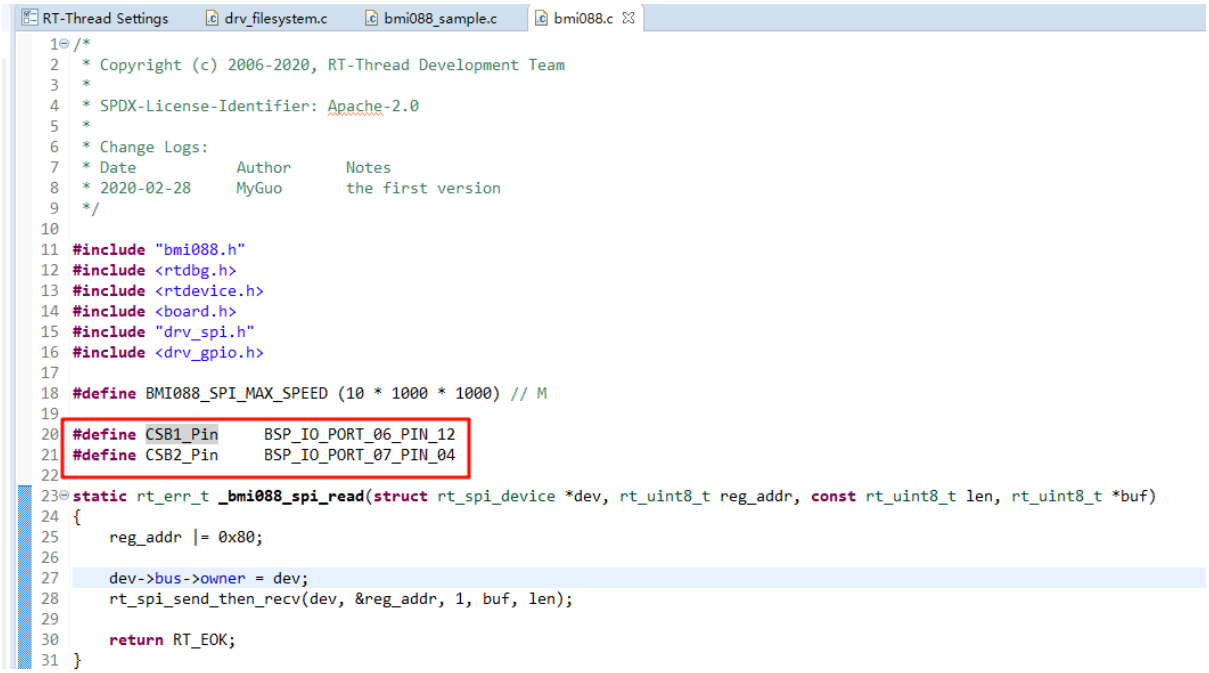
```
 * SPDX-License-Identifier: Apache-2.0
 *
 * Change Logs:
 * Date           Author       Notes
 * 2020-03-10     MyGuo        the first version
 */

#include <rtthread.h>
#include <rtdbg.h>
#include "bmi088.h"
#include "sensor_intf_bmi088.h"
#include "drv_spi.h"
#include "bmi088_sample.h"

#define BMI088_BUS_NAME "spi0"
#define BMI088A_SPI_NAME "spi00"
#define BMI088G_SPI_NAME "spi01"

static rt_device_t acce_device_t;
static rt_device_t gyro_device_t;
struct rt_sensor_data acc_test;
struct rt_sensor_data gyr_test;

void imu_thread_entry(void* parameter)
{
    // 配置spi,配置片选引脚(要在acc、gyr初始化之前配置,因为器件初始化中涉及到引脚操作)
    rt_hw_spi_device_attach(BMI088_BUS_NAME, BMI088A_SPI_NAME, BSP_IO_PORT_06_PIN_12);
    rt_hw_spi_device_attach(BMI088_BUS_NAME, BMI088G_SPI_NAME, BSP_IO_PORT_03_PIN_00);

    // 注册传感器
    struct rt_sensor_config acc_cfg = {0};
    struct rt_sensor_config gyr_cfg = {0};

    acc_cfg.intf.dev_name = BMI088A_SPI_NAME;
    gyr_cfg.intf.dev_name = BMI088G_SPI_NAME;
```

Add imu data printing and create a data ingest thread:

```
    gyro_device_t = rt_device_find("gyro_bmi");
    if (gyro_device_t == RT_NULL)
    {
        LOG_E("Can't find gyro device\r\n");
    }
    else
    {
        rt_device_open(gyro_device_t, RT_DEVICE_OFLAG_RDWR);
    }

    while (1)
    {
        _bmi088_get_data((rt_sensor_t) acce_device_t, &acc_test);
        _bmi088_get_data((rt_sensor_t) gyro_device_t, &gyr_test);

        rt_kprintf("acce value:[X]:%d [Y]:%d [Z]:%d\n", acc_test.data.acce.x, acc_test.data.acce.y, acc_test.data.acce.z);
        rt_kprintf("gyro value:[X]:%d [Y]:%d [Z]:%d\n\n", gyr_test.data.gyro.x, gyr_test.data.gyro.y, gyr_test.data.gyro.z);

        rt_thread_mdelay(500);
    }
}

void bmi088_app(void)
{
    rt_thread_t bmi088 = rt_thread_create("bmi088", imu_thread_entry, RT_NULL, 2048, 25, 10);
    if(bmi088 != RT_NULL)
    {
        rt_thread_startup(bmi088);
    }

    return;
}
MSH_CMD_EXPORT(bmi088_app, bmi088_app);
```

- Modify ./packages/bmi088-latest/src/bmi088.c

  Modify the CS pin adaptation Renesas development board:



The bmi088 driver sample is located at ./packages/bmi088-latest/samples/bmi088_sample:

```
/*
 * Copyright (c) 2006-2020, RT-Thread Development Team
 *
 * SPDX-License-Identifier: Apache-2.0
 *
 * Change Logs:
 * Date           Author       Notes
 * 2020-03-10     MyGuo        the first version
 */

#include <rtthread.h>
#include <rtdbg.h>
#include "bmi088.h"
#include "sensor_intf_bmi088.h"
#include "drv_spi.h"
#include "bmi088_sample.h"

#define BMI088_BUS_NAME "spi0"
#define BMI088A_SPI_NAME "spi00"
#define BMI088G_SPI_NAME "spi01"

static rt_device_t acce_device_t;
static rt_device_t gyro_device_t;
struct rt_sensor_data acc_test;
struct rt_sensor_data gyr_test;
```

(continues on next page)

(continued from previous page)

```
void imu_thread_entry(void* parameter)
{
    // Configure spi, configure chip selection pin (before acc, gyr
  initialization, because device initialization involves pin operation)
    rt_hw_spi_device_attach(BMI088_BUS_NAME, BMI088A_SPI_NAME, BSP_IO_PORT_06_
  PIN_12);
    rt_hw_spi_device_attach(BMI088_BUS_NAME, BMI088G_SPI_NAME, BSP_IO_PORT_07_
  PIN_04);

    // Registering sensors
    struct rt_sensor_config acc_cfg = {0};
    struct rt_sensor_config gyr_cfg = {0};

    acc_cfg.intf.dev_name = BMI088A_SPI_NAME;
    gyr_cfg.intf.dev_name = BMI088G_SPI_NAME;

    rt_hw_bmi088_init("bmi", &acc_cfg, &gyr_cfg);

    acce_device_t = rt_device_find("acce_bmi");
    if (acce_device_t == RT_NULL)
    {
        LOG_E("Can't find acce device\r\n");
    }
    else
    {
        rt_device_open(acce_device_t, RT_DEVICE_OFLAG_RDWR);
    }

    gyro_device_t = rt_device_find("gyro_bmi");
    if (gyro_device_t == RT_NULL)
    {
        LOG_E("Can't find gyro device\r\n");
    }
    else
    {
        rt_device_open(gyro_device_t, RT_DEVICE_OFLAG_RDWR);
    }

    while (1)
    {
        _bmi088_get_data((rt_sensor_t) acce_device_t, &acc_test);
        _bmi088_get_data((rt_sensor_t) gyro_device_t, &gyr_test);

        rt_kprintf("acce value:[X]:%d [Y]:%d [Z]:%d\n", acc_test.data.acce.x,
  acc_test.data.acce.y, acc_test.data.acce.z);
        rt_kprintf("gyro value:[X]:%d [Y]:%d [Z]:%d\n\n", gyr_test.data.gyro.x,
```

(continues on next page)

(continued from previous page)

```
  gyr_test.data.gyro.y, gyr_test.data.gyro.z);

        rt_thread_mdelay(1000);
    }
}

void bmi088_app(void)
{
    rt_thread_t bmi088 = rt_thread_create("bmi088", imu_thread_entry, RT_NULL,
  2048, 25, 10);
    if(bmi088 != RT_NULL)
    {
        rt_thread_startup(bmi088);
    }

    return;
}
MSH_CMD_EXPORT(bmi088_app, bmi088_app);
```

12.9 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio's package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board's USB-DBG interface to the PC and download the firmware to the development board.

12.10 Run Effect

Open the serial port tool and enter the bmi088_app command in the terminal to obtain the gyroscope data:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Jul 28 2025 10:08:14
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
================================================
This example project is an driver spi routine!
================================================
[I/spi] Tips:
[I/spi] You can run spi testcase by executing the instruction: 'bmi088_app'
msh >bmi088_app
msh >[I/sensor] rt_sensor[acce_bmi] init success
[I/sensor] rt_sensor[gyro_bmi] init success
acce value:[X]:21 [Y]:52 [Z]:9625
gyro value:[X]:13988 [Y]:-24480 [Z]:13988

acce value:[X]:-32 [Y]:93 [Z]:9569
gyro value:[X]:-3497 [Y]:24480 [Z]:-3497

acce value:[X]:35 [Y]:78 [Z]:9670
gyro value:[X]:-20982 [Y]:-13988 [Z]:-20982

acce value:[X]:46 [Y]:68 [Z]:9655
gyro value:[X]:55954 [Y]:0 [Z]:55954

acce value:[X]:-32 [Y]:93 [Z]:9585
gyro value:[X]:-13988 [Y]:-10491 [Z]:-13988

acce value:[X]:43 [Y]:80 [Z]:9679
gyro value:[X]:-3497 [Y]:27977 [Z]:-3497

acce value:[X]:21 [Y]:130 [Z]:9605
gyro value:[X]:-20982 [Y]:-17485 [Z]:-20982

acce value:[X]:-17 [Y]:100 [Z]:9603
gyro value:[X]:-20982 [Y]:-48960 [Z]:-20982
```

13. USB-HHID Usage Instructions

English | Chinese

13.1 Introduction

This document introduces a complete example of using the **USB Host HID (HHID) module (r_usb_hhid) of the RA8 MCU series** on the **Titan Board** to implement USB keyboard recognition and key input acquisition.

USB HID stands for *Universal Serial Bus Human Interface Device*. As the name suggests, HID devices are peripherals that directly interact with humans, such as keyboards, mice, and joysticks.

13.2 USB HHID Overview

General Overview

USB HID (Human Interface Device) is a device class defined in the USB specification, designed to enable interaction between humans and computers.

Typical **HID devices** include:

- Keyboards
- Mice
- Game controllers
- Touch screens
- Consumer control devices (e.g., volume buttons, media keys)

USB HHID (Host HID) refers to **Host-side support for HID devices**. In this mode, the MCU or embedded system acts as the USB Host, capable of detecting, managing, and communicating with HID devices.

HID Protocol Characteristics

- **Plug and Play**: No extra drivers are required, as most host OS (Windows, Linux, RTOS, etc.) provide built-in HID support.
- **Report Descriptor Based**: Devices describe their data format through HID Report Descriptors.
- **Low Bandwidth Usage**: Typically uses USB interrupt transfers, with small data size and low latency, suitable for real-time user interactions.
- **Cross-Platform**: Standardized protocol, widely supported across PCs, embedded systems, and mobile devices.

Data Communication Mechanism

HID communication uses **endpoints**:

- **Control Endpoint (Endpoint 0)**: Used for device enumeration and descriptor exchange.
- **Interrupt Endpoint**: Used for HID data transfer, typically via polling.

HID Data Flow:

1. **Device Connection** → Host retrieves device descriptors via control transfer.
2. **Report Descriptor Parsing** → Host learns the data format.
3. **Polling** → Host periodically retrieves input reports from the HID device.
4. **Optional Output Reports** → Host sends control data back to the device (e.g., LED status).

HID Device Categories

- **Input Devices**: Keyboards, mice, touchpads, game controllers.
- **Output Devices**: LEDs, haptic feedback devices.
- **Composite Devices**: Devices with multiple HID functions, e.g., a mouse with extra buttons.

Application Scenarios

- **Embedded System Debugging**: MCU as USB Host to connect keyboards/mice for user input.
- **Industrial Control**: Connection of barcode scanners or control joysticks.
- **Medical Devices**: Standardized connection with input peripherals.
- **Consumer Electronics**: Smart TVs, set-top boxes supporting USB remotes/gamepads.

13.3 RA8 USB HHID Module Overview

Module Role & Functions

- **USB Host Controller Support**: Supports USB Host mode (Full-Speed / High-Speed, depending on MCU hardware). Handles bus-level enumeration, transaction scheduling, endpoint management, and VBUS control.
- **HHID Class Driver Responsibilities**: Implements USB HID Class (0x03) handling, specifically:
  - Enumeration and endpoint setup
  - Parsing HID Report Descriptors or using Boot Protocol
  - Submitting periodic interrupt transfers
  - Decoding keyboard key codes and reporting events
- **Transfer Type**: Interrupt IN transfers (polling-based) for keyboard input. Also supports control transfers (for descriptors, protocol switching, etc.).
- **Event Dispatching**: Key events (make/break, modifier keys) are delivered via callbacks, message queues, or RT-Thread device framework.

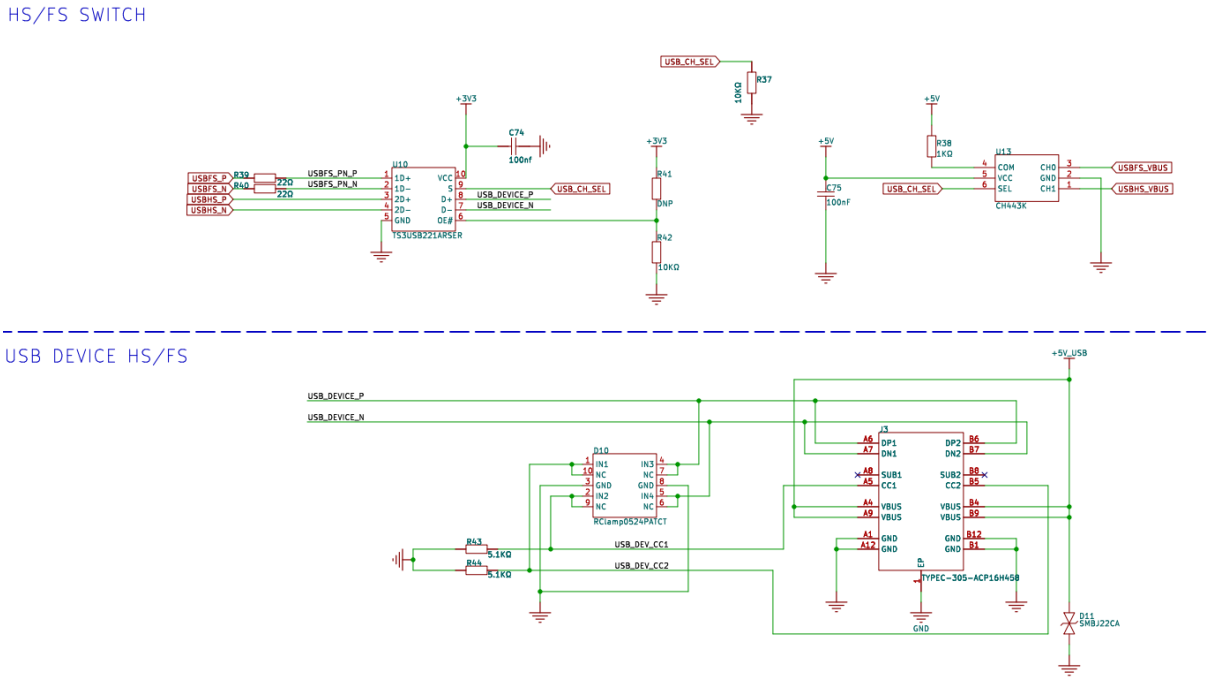
Capabilities

- Full USB enumeration (Get Descriptor, Set Configuration, Set Interface)
- HID Report Descriptor parsing for complex data formats
- Support for **HID Boot Protocol (keyboard)** for simplified parsing
- Poll interval and timeout/retry management
- Hot-plug detection and multi-device support (with USB Hub)
- Error handling (CRC, NAK, STALL, timeouts) with recovery mechanisms
- Optional DMA support (depending on host controller hardware)

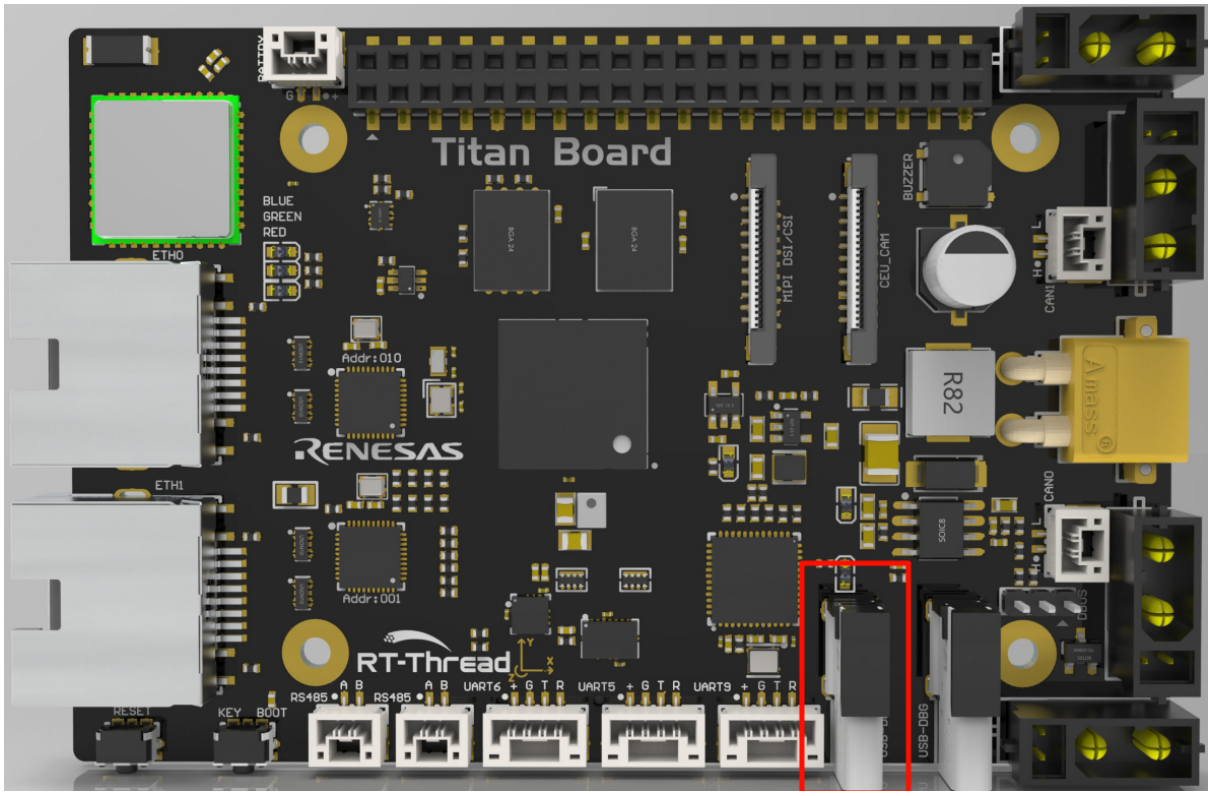
Workflow

1. **Host Initialization**: Initialize USB host controller, enable VBUS, start USB interrupts and SOF.
2. **Device Detection**: Detect device insertion, trigger enumeration.
3. **Enumeration & Class Detection**: Read descriptors to identify HID class (0x03).
4. **Endpoint Setup**: Retrieve HID descriptors, report descriptors, or check Boot Protocol.
5. **Interrupt IN Endpoint Configuration**: Open endpoint and submit periodic interrupt transfers.
6. **Data Reception & Parsing**: Parse HID reports into key events.
7. **Event Reporting**: Forward key events to the application layer (RT-Thread framework).
8. **Device Removal Handling**: Stop polling, release resources, notify removal event.

13.4 Hardware Description

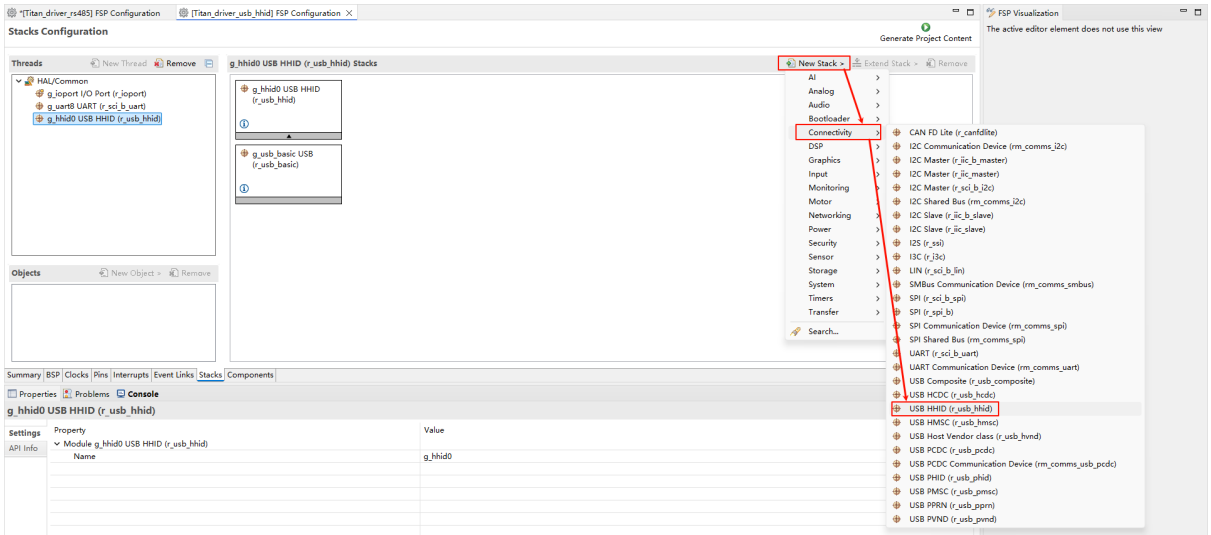


Titan Board provides a USB-Device peripheral, located on the development board as shown below:

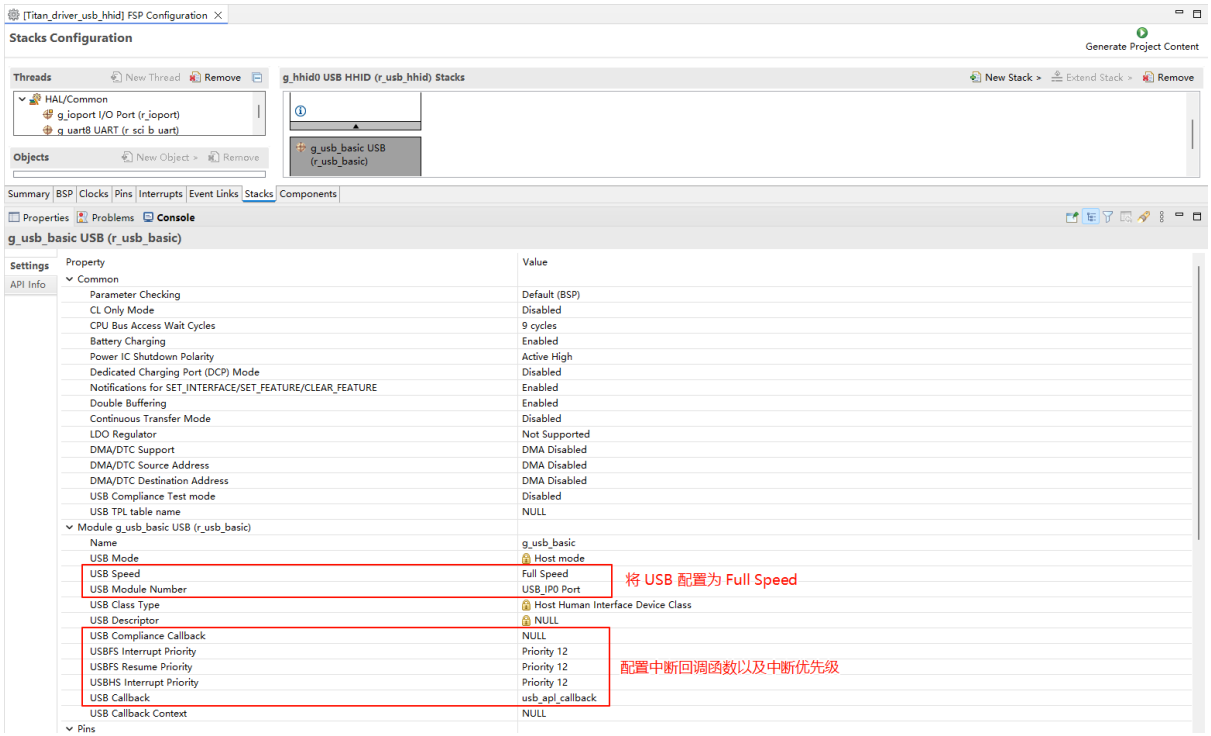


13.5 FSP Configuration

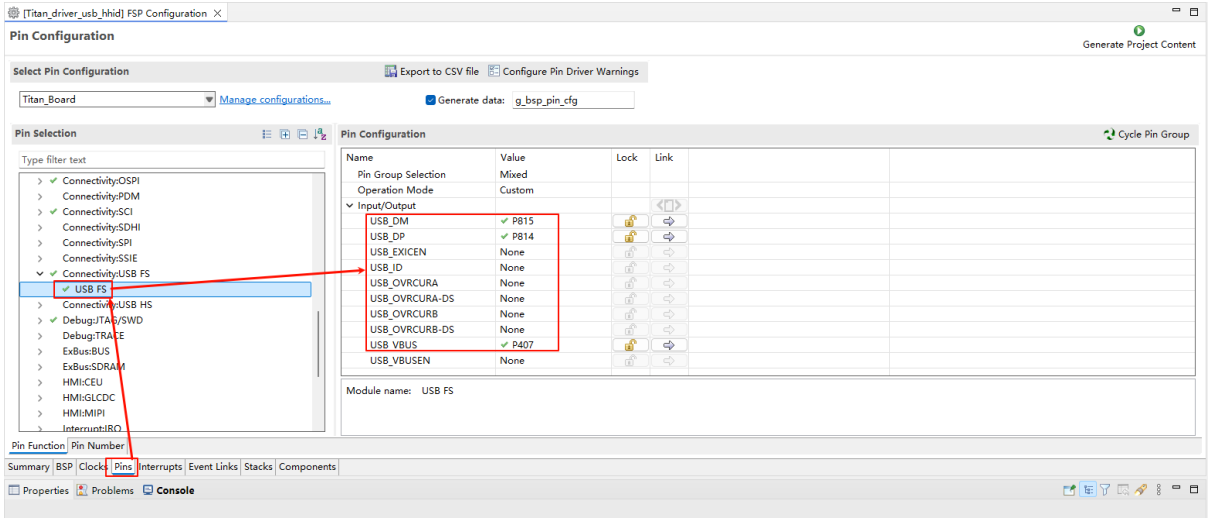
- Create a r_usb_hhid stack.



- Configure r_usb_basic stack.

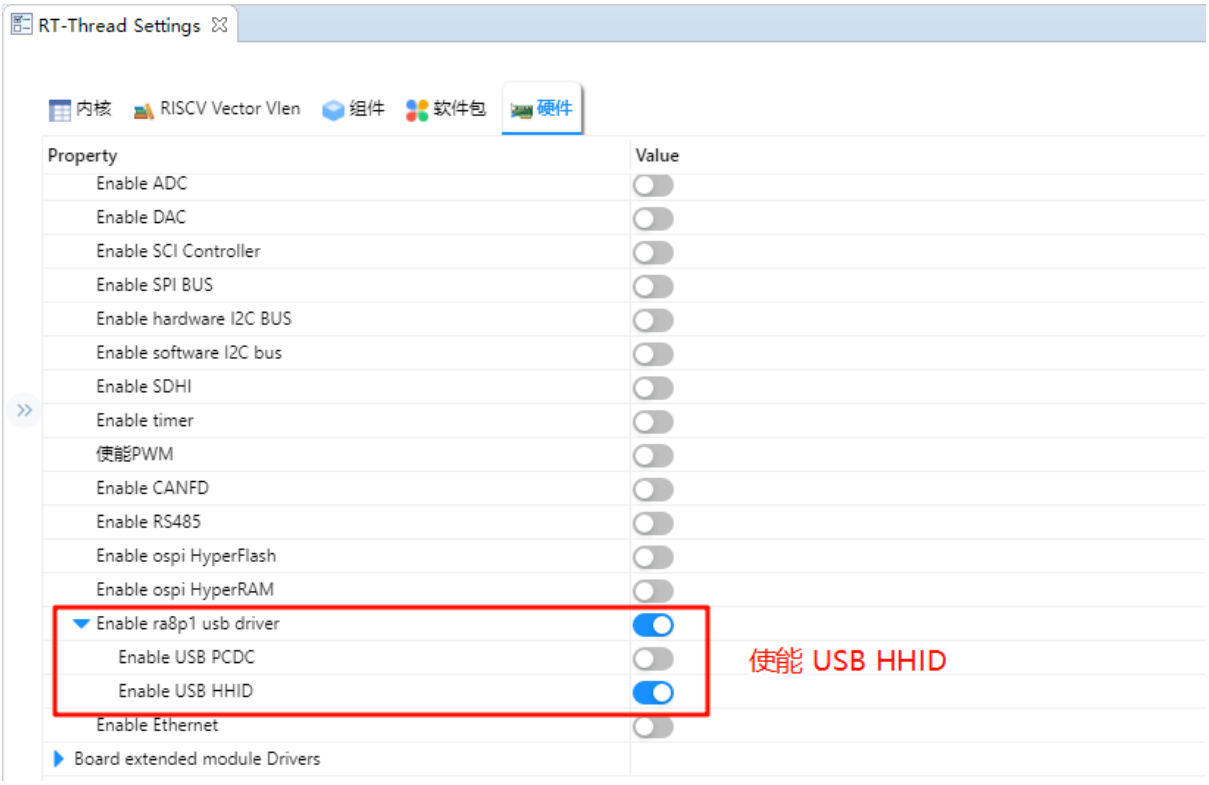


- Configure USB FS pins.

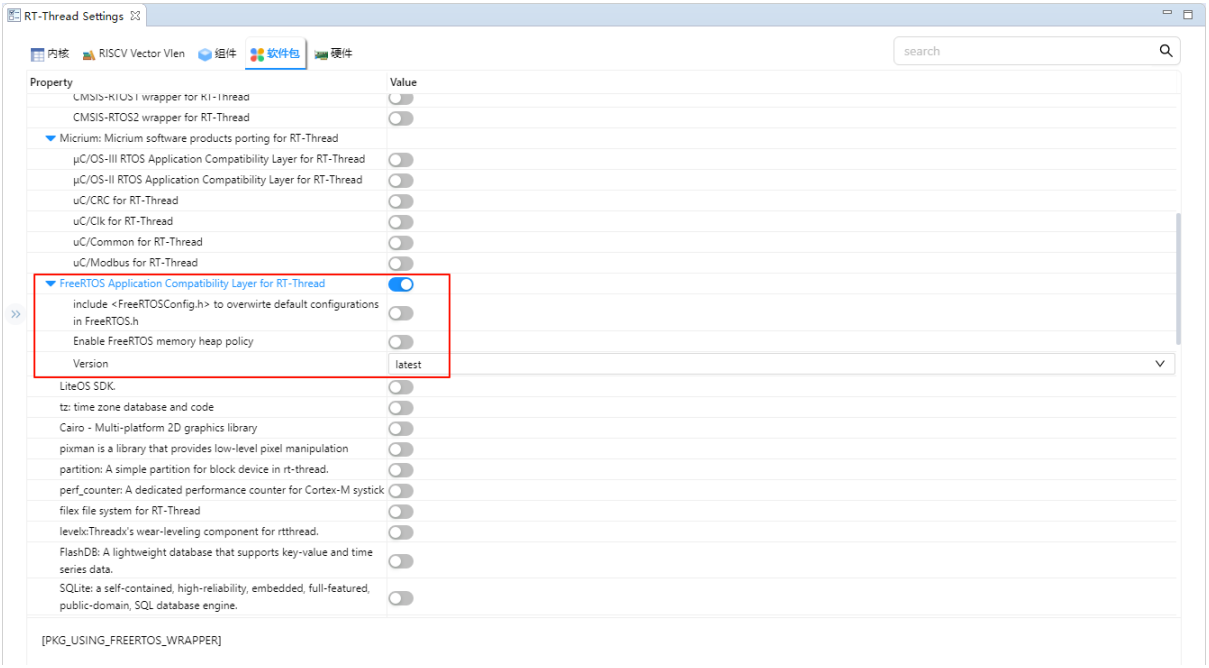


13.6 RT-Thread Settings Configuration

- Enable USB HHID。



- The USB example currently uses the FreeRTOS interface driver, so we also need to enable the FreeRTOS compatibility layer package.



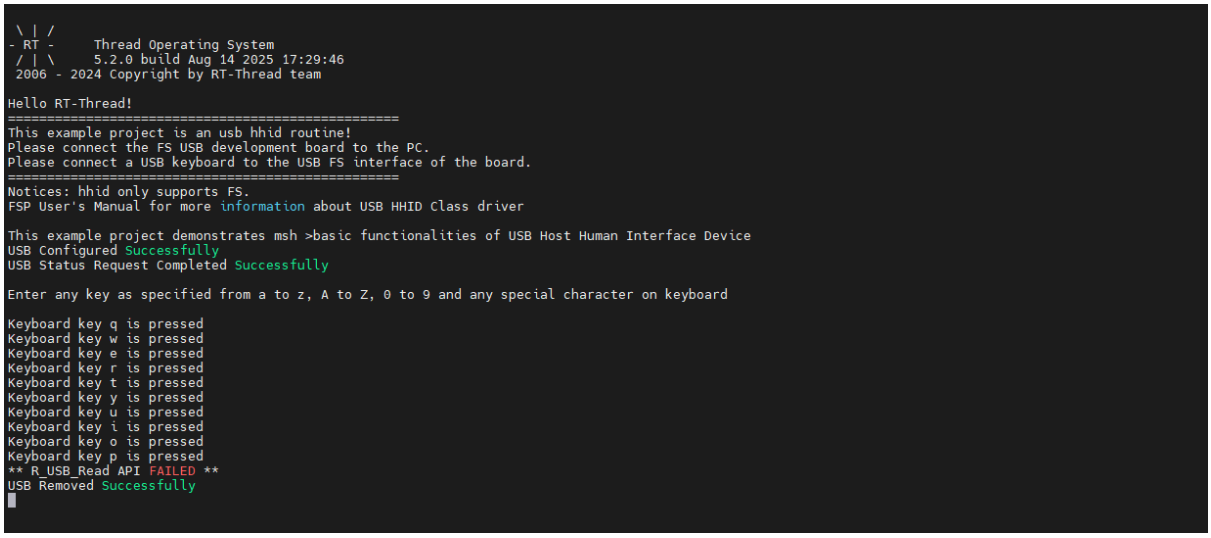
13.7 Compilation & Download

- **RT-Thread Studio**: Download the Titan Board resource package in the RT-Thread Studio package manager, create a new project, and compile it.

Once the compilation is complete, connect the USB-DBG interface of the development board to the PC and download the firmware to the board.

13.8 Running Results

Connect the keyboard to the USB-DEV interface of the Titan Board via a adapter cable. After pressing a key on the keyboard, the terminal will display which key was pressed.



14. USB-PCDC Usage Instructions

English | Chinese

14.1 Introduction

This example demonstrates how to use the **RA8 Series MCU USB PCDC module (r_usb_pcdc)** on the **Titan Board** to enumerate the device as a **USB virtual serial port (CDC)**, allowing serial communication with the board on a PC as a “COM port/ttyACM”.

14.2 USB PCDC Instructions

Overview

USB PCDC (USB Peripheral Communications Device Class) is a specific implementation of the **CDC (Communications Device Class)** defined in the USB protocol.

It primarily allows **embedded devices to appear as a virtual serial port (VCOM) over USB**, enabling the host (PC or embedded host) to interact with the USB device just like a traditional UART serial port.

Common application scenarios:

- MCU as a USB device, recognized by the PC as a virtual serial port (COMx).
- Embedded devices communicating with a host computer for data transfer or debugging.
- Replacing physical RS-232/RS-485 interfaces to reduce hardware cost.

PCDC Protocol Features

1. **CDC Class Protocol**
    - USB-defined communication device class, uses standard class drivers (no extra driver installation needed; automatically recognized on Windows/Linux/macOS).
2. **Data Transfer Methods**
    - **Control Endpoint (Endpoint 0)**: Used for device enumeration and descriptor transfer.
    - **Interrupt Endpoint**: Used for status and control signals (e.g., DTR, RTS).
    - **Bulk Endpoint**: For high-efficiency, reliable serial data transfer.
3. **Virtual Serial Port Features**
    - Appears as a standard serial port (COM port or ttyUSB) on the host.
    - Supports UART configurations such as baud rate, data bits, parity, and stop bits.

Typical Application Architecture

- **USB Device (MCU side)**: Implements the PCDC protocol stack, connecting USB data to the MCU’s internal UART/application layer.
- **USB Host (PC/embedded system)**: Maps a virtual serial port through the OS-provided CDC driver.
- **Application Layer**: Host software (PuTTY, Tera Term, serial monitor tools) can communicate directly.

Advantages

- **No additional driver needed**: Most OSs include built-in CDC drivers.
- **Replaces traditional serial ports**: Enables debugging/communication even without a physical UART.
- **Higher data rates**: Full-Speed (12 Mbps) or High-Speed (480 Mbps) USB exceeds typical UART bandwidth.
- **High compatibility**: Works with various host applications (LabVIEW, Python pySerial, C# serial libraries).

14.3 RA8 Series USB PCDC Module (r_usb_pcdc) Overview

The RA8 Series MCU integrates a **USB PCDC (Peripheral CDC) module**, designed to enumerate the MCU as a **USB virtual serial port (CDC device)**, enabling bidirectional data communication with a PC or other USB hosts. It works together with the RA8 **basic USB module (r_usb_basic)**, and the **r_usb_pcdc class driver** provides full CDC functionality.

Module Functionality

- **Device Mode Support**: MCU acts as a USB device connected to a host (Full-Speed / High-Speed, depending on MCU specification).
- **Virtual Serial Port (CDC) Features**:
  - Handles class requests: SET_LINE_CODING, GET_LINE_CODING, SET_CONTROL_LINE_STATE, SEND_BREAK.
  - Supports standard Bulk IN/OUT endpoints and an Interrupt IN endpoint.
  - Generates serial port status notifications (e.g., DTR/RTS, line state).
- **Data Transfer Management**: Supports high-efficiency transfer with DMA/DTC or FIFO buffers.
- **Event Callback Mechanism**: For connection, disconnection, configuration completion, data transfer completion, class request handling, and error events.
- **Hot-plug and Low Power**: Supports VBUS detection, suspend/resume, and remote wakeup.
- **Application Layer Interface**: Provides API for read/write buffers, send-complete notifications, and event callbacks.

Supported Features

- **Standard USB Enumeration**: Supports Device, Configuration, and Interface Descriptors.
- **Endpoint Management**: Configures Bulk IN/OUT and Interrupt IN endpoints.
- **Class Request Handling**: Parses line coding and control line state according to CDC protocol.
- **Reliable Transfers**: Supports CRC, NAK, STALL, and timeout handling.
- **Multiple Device & Composite Device Support**: Can dynamically manage multiple CDC devices via hubs.
- **DMA Collaboration**: Optimizes data throughput and reduces CPU load.

Module Architecture

```
[ Application Layer Task/Thread ]
        ↑     Callback/Event notifications (connect, configure, transfer
  complete)
[ r_usb_pcdc Class Driver ]
        ↕     Class request parsing, data transfer management, endpoint
  configuration
[ r_usb_basic Basic Driver ]
        ↕     USB device enumeration, endpoint control, SOF interrupt, DMA
  support
[ USB Device Controller/PHY ]
        ↕     USB D+/D- signals (FS) or ULPI/HS PHY (HS)
[ USB Cable ]  <—> Host (PC)
```

Key Submodules:

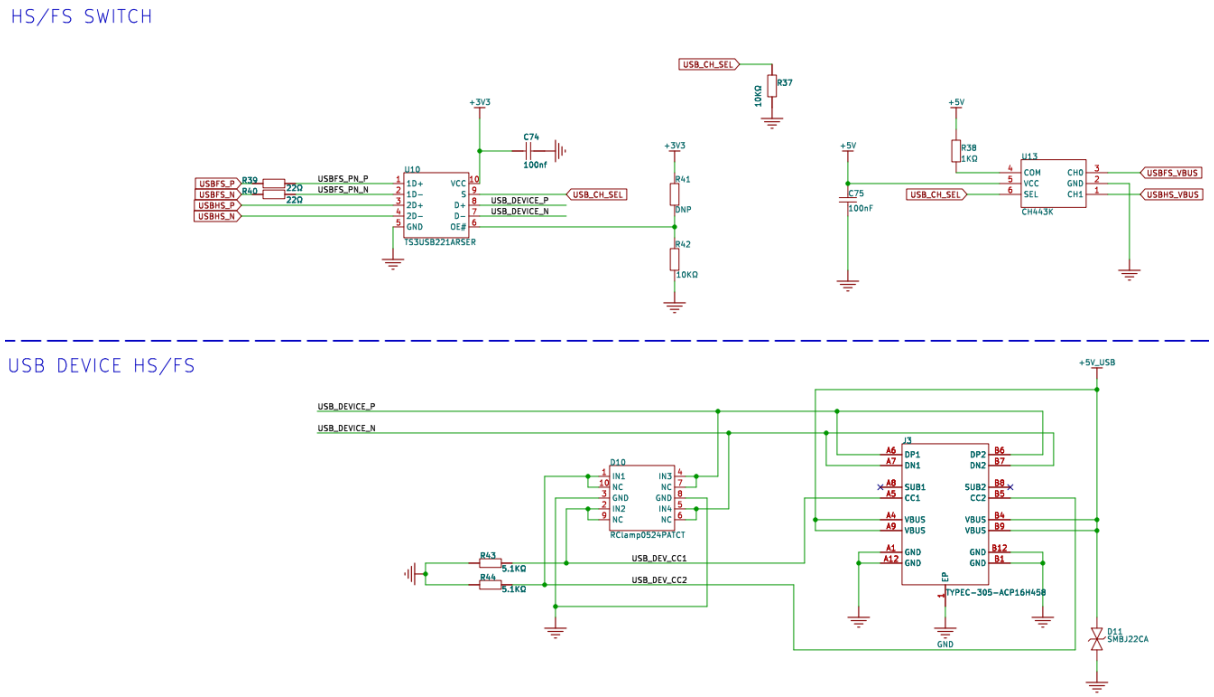
1. **Class Request Handler**: Parses CDC class requests and callbacks the application.
2. **Endpoint Control and Buffers**: Manages Bulk/Interrupt endpoints and supports DMA/FIFO.
3. **Event Management**: Handles connection/disconnection, configuration, suspend/resume, transfer complete, error events.
4. **Power and Clock Control**: USB PHY, VBUS detection, suspend/wake management.

Workflow

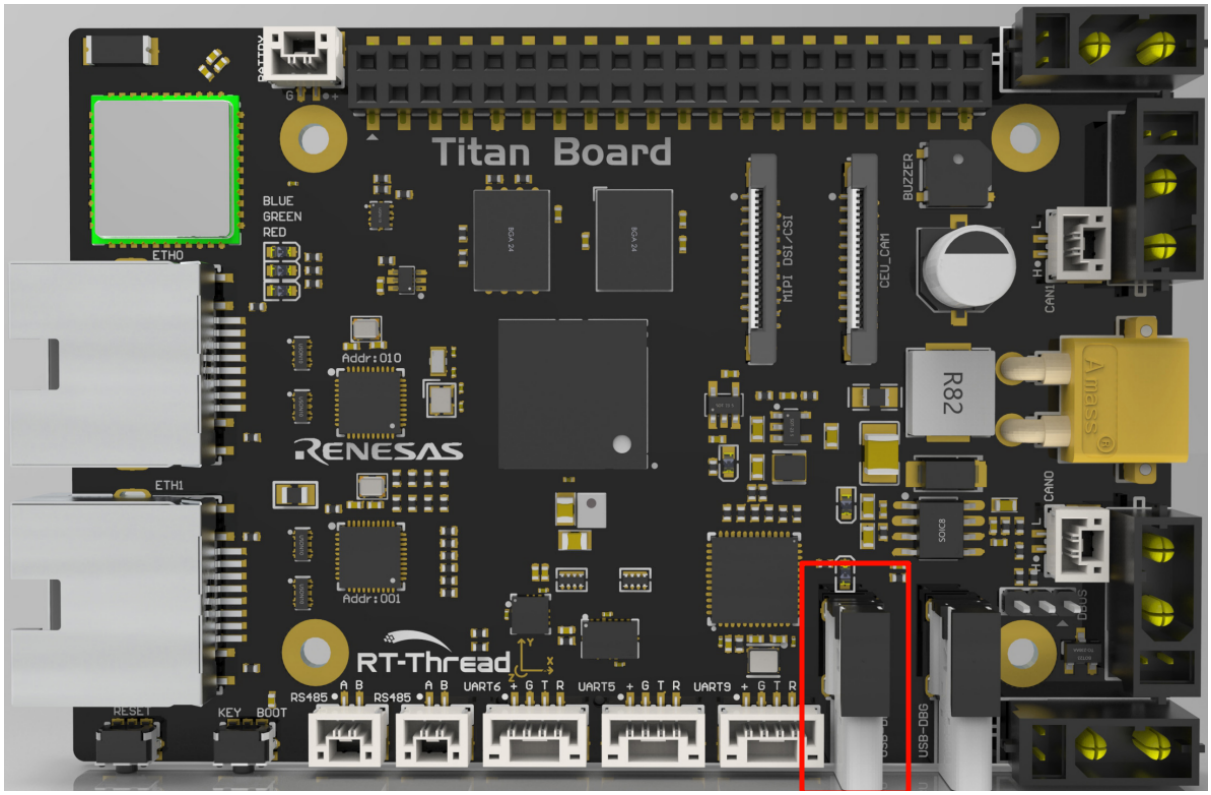
1. **USB Initialization**: Initialize USB controller, PHY, and class driver.
2. **Device Connection Detection**: Detect VBUS/ID signals to trigger enumeration.
3. **Enumeration and Configuration**: Upload descriptors and configure interface/endpoints.
4. **Endpoint Data Transfer**: Application exchanges data with host via Bulk IN/OUT endpoints.
5. **Class Request Handling**: Process CDC control requests from host.
6. **Event Callback Notification**: Notify application of connection, configuration, and transfer completion.

7. **Disconnection Handling**: Stop endpoint transfers, release buffers, and handle device removal events.

14.4 Hardware Description

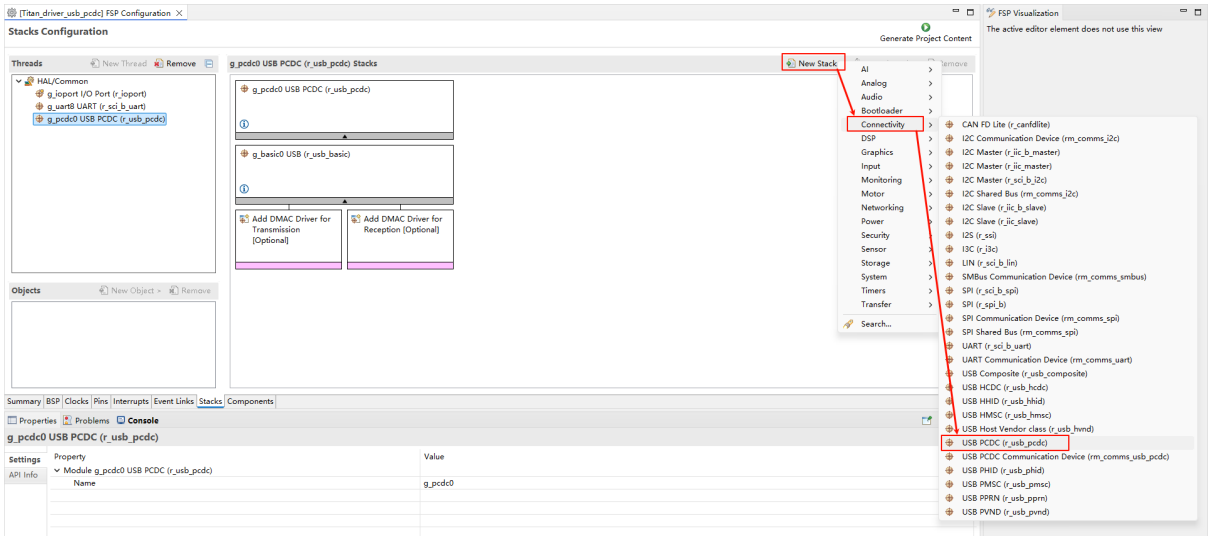


Titan Board provides a USB-Device peripheral, located on the development board as shown below:

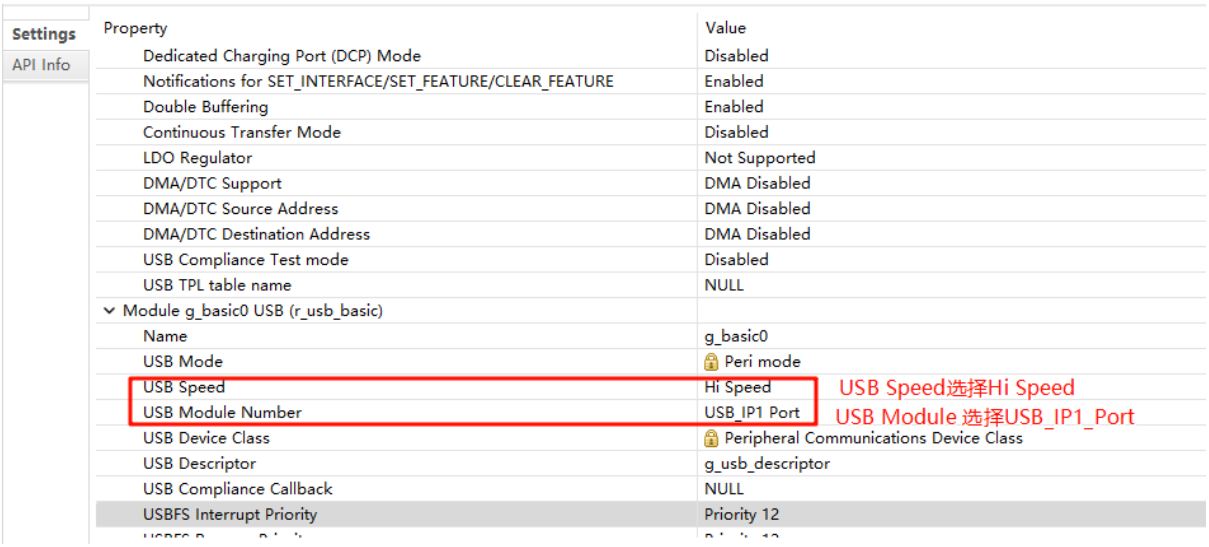


14.5 FSP Configuration Configuration

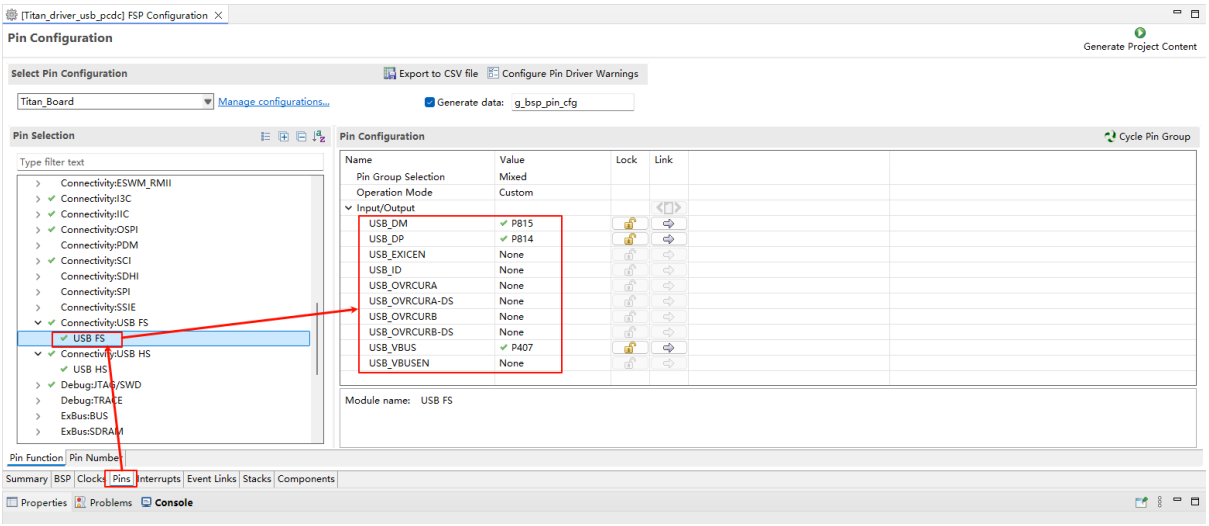
- Create a r_usb_pcdc stack:



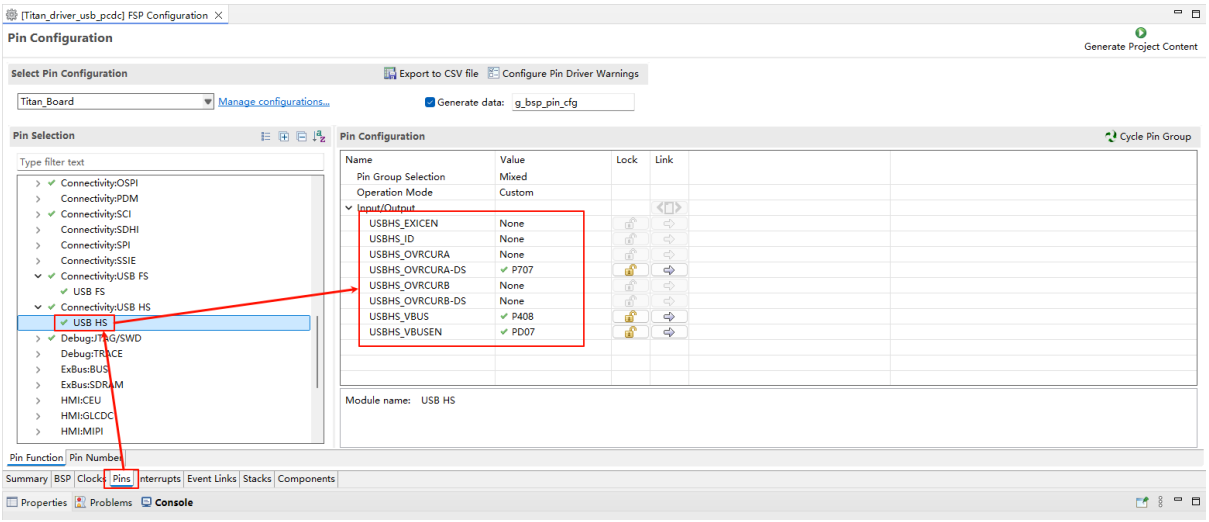
- Configure r_usb_basic:



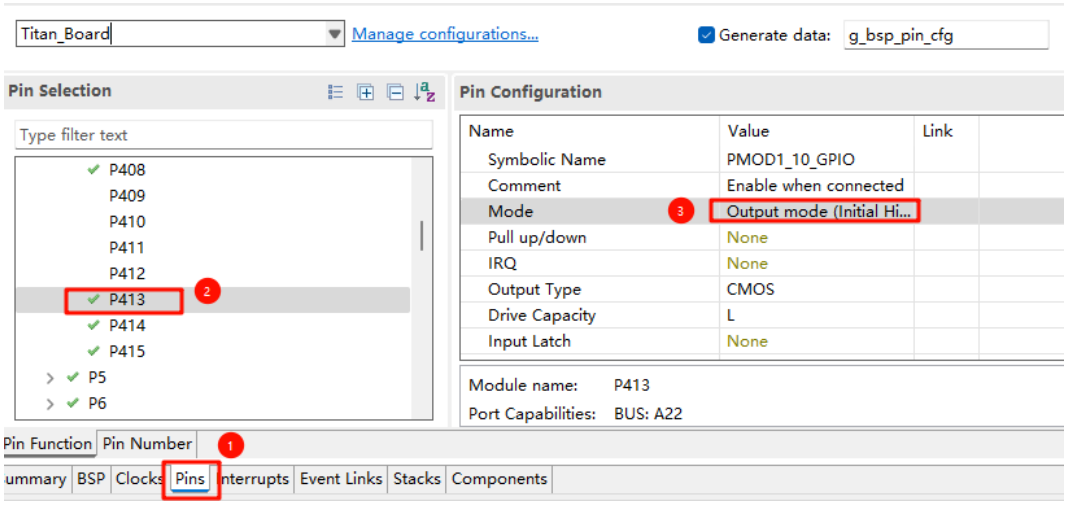
- Configure USB FS pin：



- Configure USB HS pin：

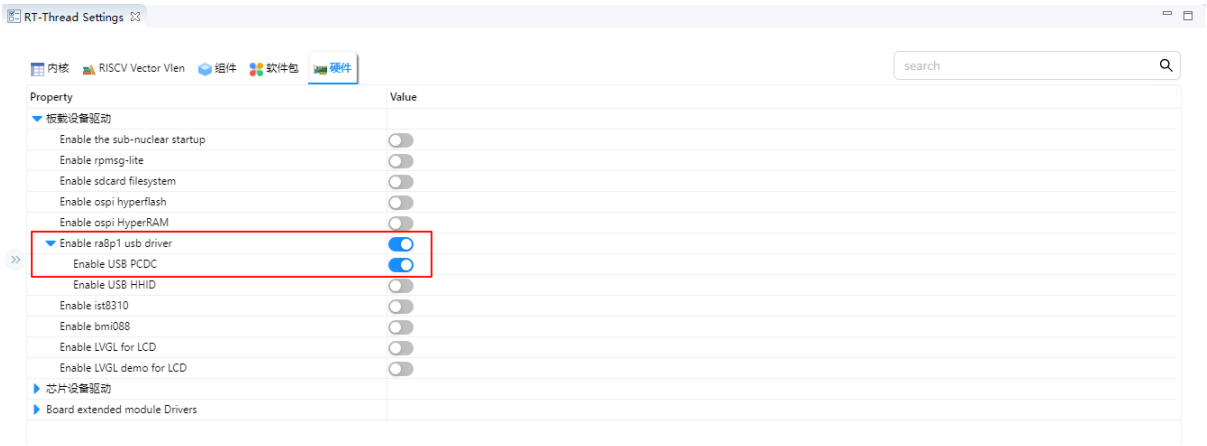


PS: Note that when switching to HS mode, the **P413 pin** needs to be pulled high, as shown in the figure below:



14.6 RT-Thread Settings Configuration

Enable USB PCDC。



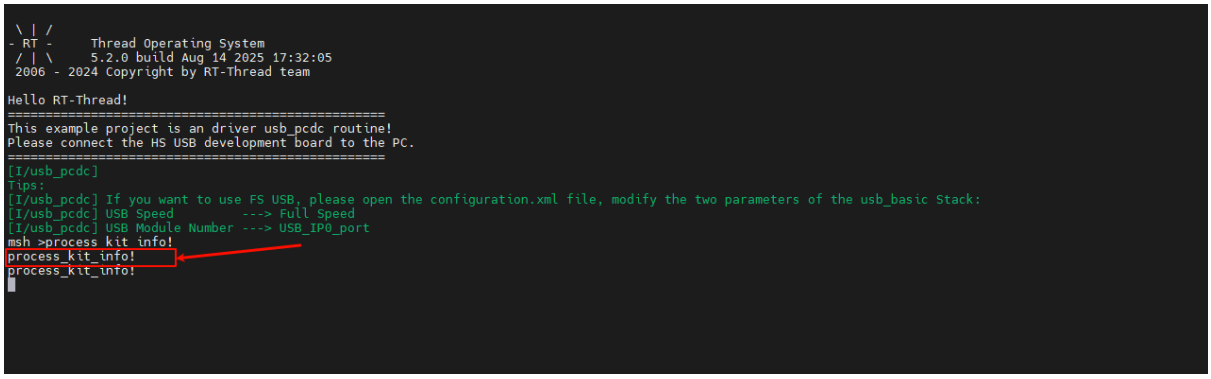
14.7 Compilation & Download

- **RT-Thread Studio**: Download the Titan Board resource package from the RT-Thread Studio package manager, then create a new project and compile.

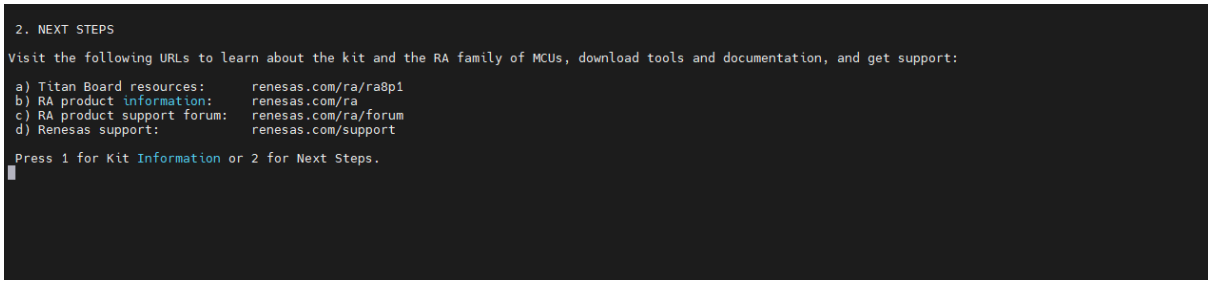
After compiling, connect the USB-DBG interface of the development board to the PC and download the firmware to the development board.

14.8 Running Effect

Use USB cable to connect the USB-DEV interface of Titan Board with the computer, open the serial port terminal with the serial port assistant, input 1 into the serial port terminal, and see the output in the debugging terminal of the development board.



Enter 2 into the USB-PCDC serial terminal, and you will see the following output in the USB-PCDC serial terminal:


2. NEXT STEPS
Visit the following URLs to learn about the kit and the RA family of MCUs, download tools and documentation, and get support:
a) Titan Board resources: renesas.com/ra/ra8p1
b) RA product information: renesas.com/ra
c) RA product support forum: renesas.com/ra/forum
d) Renesas support: renesas.com/support
Press 1 for Kit Information or 2 for Next Steps.

15. WDT Usage Instructions

English | Chinese

15.1 Introduction

The Watchdog Timer (WDT) ensures that our code runs as expected and can effectively prevent the program from "running away" due to uncontrollable factors. This example demonstrates how to use the **WDT module of the RA8 series MCU** on the **Titan Board**, together with the **RT-Thread WDT driver framework**, to implement watchdog functionality, including timer configuration, feeding the watchdog, and handling timeouts.

15.2 WDT (Watchdog Timer) Instruction

General Description

The **WDT (Watchdog Timer)** is a **hardware timer used in embedded systems to monitor system operation and prevent the program from hanging or stalling**.

If the MCU or processor does not reset (Kick/Reload) the WDT within the specified period, the WDT will **automatically reset the system**, ensuring it can resume normal operation.

Typical use cases for WDT:

- Preventing software infinite loops or task stalls
- Improving system reliability and stability
- Critical applications such as industrial control, automotive electronics, drones, and communication devices

Working Principle

1. **Counter Mode**
   - The WDT includes a decrementing counter that starts from an initial value.
   - The MCU must **reset the counter (Reload/Kick)** before it reaches zero.
   - If the counter reaches zero, a **system reset** or **interrupt** is triggered.
2. **Feeding the Watchdog**
   - The MCU periodically writes a specific value to reset the counter.
   - Commonly called "feeding the dog," this ensures that during normal operation, the WDT does not reset the system.
3. **Reset/Interrupt Mode**
   - **Reset Mode**: The WDT expiration directly resets the MCU.
   - **Interrupt Mode**: The WDT expiration generates an interrupt, allowing software to attempt recovery first.

Types of WDT

Type	Description
Independent WDT (IWDT)	Has its own clock source; continues running during reset or suspend
Window WDT (WWDT)	Requires feeding within a specific time window
Super WDT	Offers higher precision or longer configurable periods
Software WDT	Implemented via software timers, maintained by OS or applications

Key Parameters

- **Counter width**: Determines maximum timeout period
- **Timeout**: Time from last feed to system reset trigger
- **Clock source**: Independent low-speed oscillator or system clock
- **Reset method**: Hardware reset or interrupt notification

Example:

- WDT timeout = 500 ms
- MCU feeds the WDT every 100 ms
- If the MCU stalls and fails to feed the WDT for 500 ms → WDT resets the system

Usage Guidelines

1. **Feed periodically**: Ensure WDT is refreshed during normal operation.
2. **Feed in critical tasks**: Typically in the main loop or key tasks.
3. **Interrupt-safe**: Feeding should not be blocked by interrupts.
4. **Reasonable timeout**: Must be longer than task execution time but not excessively long.

15.3 RA8 Series WDT Module (r_wdt) Instruction

The RA8 MCU series integrates a high-performance **WDT module (r_wdt)**, providing reliable system monitoring with support for both reset and interrupt modes.

Key Features

- **Timer mode**: Single-shot or periodic
- **Clock sources**: Low-power oscillator (LPO) or system clock
- **Reset and interrupt outputs**:
  - Timeout can reset the MCU
  - Interrupt mode available for software recovery

- **Counter range**: 8–32 bits, configurable timeout period
- **Safety features**:
  - Protection against unintended writes (unlock sequences required)
  - Multi-level reset modes (WDT reset, NMI reset)
- **Independent operation**: Runs even in low-power or sleep mode

Module Architecture

The RA8 WDT module contains:

1. **Counter Unit** – decrements with clock input, detects overflow
2. **Control Registers** – configure timeout, mode, and feeding sequence
3. **Status Registers** – indicate WDT state, timeout flags, interrupt/reset flags
4. **Reset & Interrupt Unit** – triggers MCU reset or interrupt/NMI upon timeout

Working Flow

1. **Start WDT**: Configure clock source, counter, and timeout period, then enable.
2. **Feed operation**: Software writes the specific sequence to reset the counter.
3. **Timeout event**: If feeding is missed → counter overflows:
   - Triggers MCU reset (reset mode)
   - Triggers interrupt (interrupt mode)
4. **Safety protection**: Registers require unlock before modification.

15.4 RT-Thread WDT Framework Instruction

RT-Thread WDT (Watchdog Timer) Framework is a unified interface provided by the RT-Thread device driver layer for managing hardware watchdog modules across different MCUs. This framework abstracts the hardware WDT functionality into a standardized device interface, allowing applications to use a consistent API across platforms to implement system self-recovery mechanisms.

Device Model

In RT-Thread, the WDT is managed as a **device object** (a subclass of struct rt_device, with type RT_Device_Class_WDT). Developers don't need to deal with low-level hardware register operations —all watchdog functionalities can be implemented through the standard RT-Thread device interfaces.

Operation Interfaces

Applications access the watchdog hardware using RT-Thread's I/O device management interfaces as shown below:

- **Find the Watchdog Device**

```
rt_device_t rt_device_find(const char* name);
```

- **Initialize the Watchdog**

```
rt_err_t rt_device_init(rt_device_t dev);
```

- **Control the Watchdog**

```
rt_err_t rt_device_control(rt_device_t dev, rt_uint8_t cmd, void* arg);
```

Available watchdog control commands are defined as follows:

```
#define RT_DEVICE_CTRL_WDT_GET_TIMEOUT    (1) /* Get the timeout period */
#define RT_DEVICE_CTRL_WDT_SET_TIMEOUT    (2) /* Set the timeout period */
#define RT_DEVICE_CTRL_WDT_GET_TIMELEFT   (3) /* Get the remaining time before
  reset */
#define RT_DEVICE_CTRL_WDT_KEEPALIVE      (4) /* Feed the watchdog */
#define RT_DEVICE_CTRL_WDT_START          (5) /* Start the watchdog */
#define RT_DEVICE_CTRL_WDT_STOP           (6) /* Stop the watchdog */
```

- **Close the Watchdog Device**

```
rt_err_t rt_device_close(rt_device_t dev);
```

Framework Features

- **Unified Device Abstraction**: All hardware WDTs are exposed to upper layers through the same device interface.
- **Cross-Platform Support**: Applications can be ported across MCU platforms without changing WDT-related code.
- **Timeout Protection**: Supports configurable timeout periods; if the system fails to feed the watchdog, it triggers an interrupt or reset.
- **Flexible Operating Modes**: Supports both **interrupt mode** and **reset mode** to meet various application requirements.
- **High Reliability**: Ensures system safety by using hardware-enforced reset mechanisms to recover from system hangs.

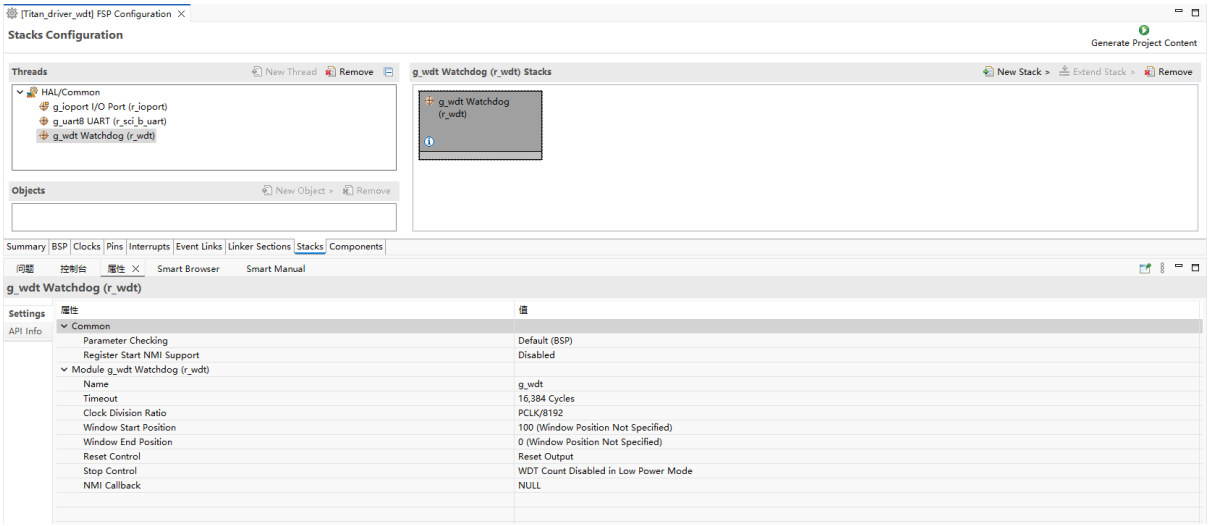
  Reference: RT-Thread WATCHDOG Device

15.5 Hardware Description

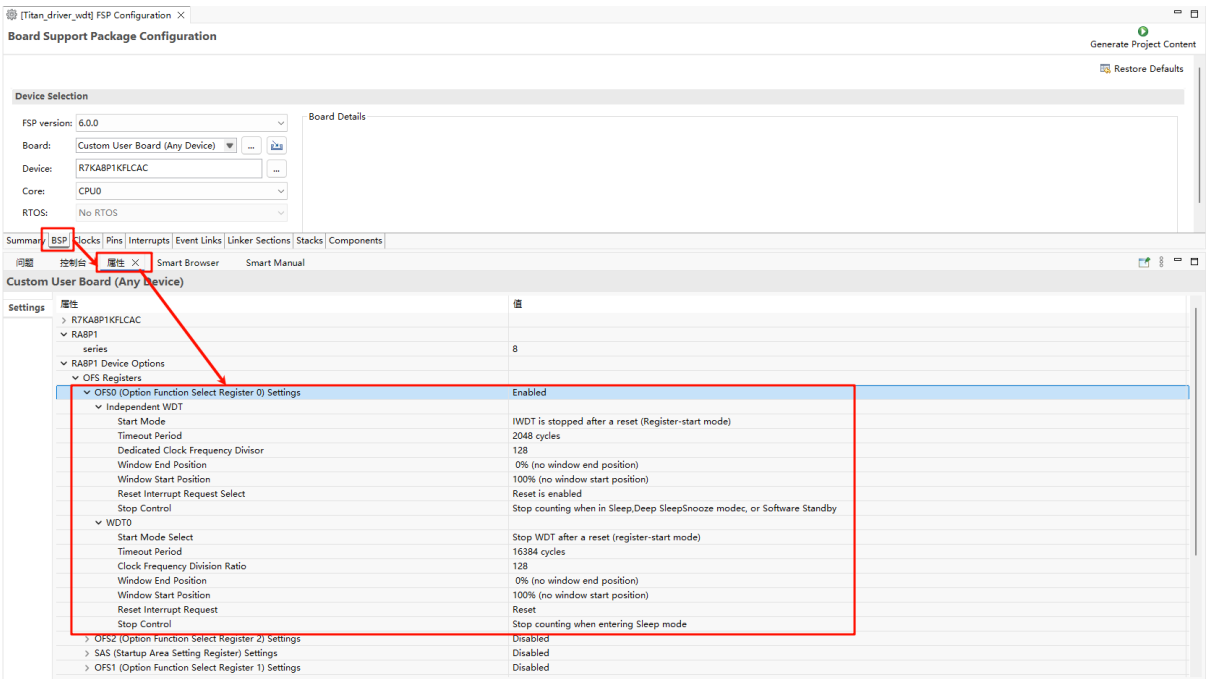
None

15.6 FSP Configuration

- Open the FSP tool and create a new stack, selecting r_wdt.

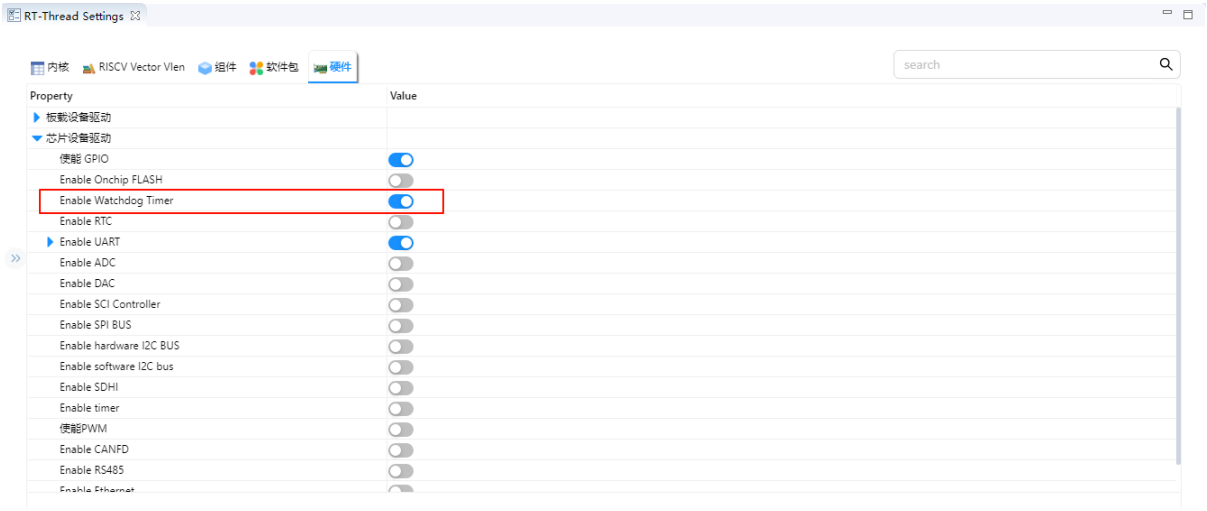


- **Note:** Using WDT requires the OFS0 register configuration to be enabled.



15.7 RT-Thread Settings Configuration

- Enable WDT.



15.8 Example project description

The sample program is located at project/Titan_driver_wdt/src/hal_entry.c.

```
#include <rtthread.h>
#include "hal_data.h"
#include <rtdevice.h>
#include <board.h>

#define DBG_TAG "wdt"
#define DBG_LVL DBG_LOG
#include <rtdbg.h>

#define WDT_DEVICE_NAME "wdt"    // Default watchdog device name, depending on
  the BSP
#define WDT_FEED_INTERVAL 1000   // Feeding interval (unit: ms)
#define WDT_TIMEOUT 3            // Watchdog timeout period (unit: s)

static rt_device_t wdt_dev = RT_NULL;
static rt_thread_t feed_thread = RT_NULL;

#define LED_PIN_0    BSP_IO_PORT_06_PIN_00 /* Onboard LED pins */

void hal_entry(void)
{
    rt_kprintf("\nHello RT-Thread!\n");
    rt_kprintf("====================================================\n");
    rt_kprintf("This example project is an driver wdt routine!\n");
    rt_kprintf("====================================================\n");
```

(continues on next page)

(continued from previous page)

```
    LOG_I("Tips:");
    LOG_I("You can run wdt testcase by executing the instruction: \'wdt_sample\'
  ");

    while (1)
    {
        rt_pin_write(LED_PIN_0, PIN_HIGH);
        rt_thread_mdelay(1000);
        rt_pin_write(LED_PIN_0, PIN_LOW);
        rt_thread_mdelay(1000);
    }
}

static void feed_dog_entry(void *parameter)
{
    int count = 0;

    while (1)
    {
        if (count < 10)
        {
            rt_device_control(wdt_dev, RT_DEVICE_CTRL_WDT_KEEPALIVE, RT_NULL);
            LOG_I("[FeedDog] Feeding watchdog... %d", count);
        }
        else
        {
            LOG_E("[FeedDog] Simulate exception! Stop feeding.");
        }

        count++;
        rt_thread_mdelay(WDT_FEED_INTERVAL);
    }
}

static int wdt_sample(void)
{
    rt_err_t ret;

    wdt_dev = rt_device_find(WDT_DEVICE_NAME);
    if (wdt_dev == RT_NULL)
    {
        LOG_E("Cannot find %s device!", WDT_DEVICE_NAME);
        return -1;
    }
```

(continues on next page)

(continued from previous page)

```
    ret = rt_device_control(wdt_dev, RT_DEVICE_CTRL_WDT_START, RT_NULL);
    if (ret != RT_EOK)
    {
        LOG_E("Start watchdog failed!");
        return -1;
    }

    LOG_I("Watchdog started...", WDT_TIMEOUT);

    feed_thread = rt_thread_create("feed_dog", feed_dog_entry, RT_NULL, 1024, 10,
  10);
    if (feed_thread != RT_NULL)
        rt_thread_startup(feed_thread);

    return 0;
}
MSH_CMD_EXPORT(wdt_sample, wdt_sample);
```

15.9 Compilation & Download

- **RT-Thread Studio**: Download the Titan Board resource package in the RT-Thread Studio package manager, then create a new project and compile it.

After compilation, connect the USB-DBG interface of the development board to the PC, and download the firmware to the development board.

15.10 Run Effects

Enter the wdt_sample command at the terminal to run the WDT test program. After feeding the dog 10 times, stop feeding the dog to simulate an abnormal program situation.

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Jul 28 2025 10:09:53
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
==================================================
This example project is an driver wdt routine!
==================================================
[I/wdt] Tips:
[I/wdt] You can run wdt testcase by executing the instruction: 'wdt_sample'
msh >wdt_sample
[I/wdt] Watchdog started...
[I/wdt] [FeedDog] Feeding watchdog... 0
msh >[I/wdt] [FeedDog] Feeding watchdog... 1
[I/wdt] [FeedDog] Feeding watchdog... 2
[I/wdt] [FeedDog] Feeding watchdog... 3
[I/wdt] [FeedDog] Feeding watchdog... 4
[I/wdt] [FeedDog] Feeding watchdog... 5
[I/wdt] [FeedDog] Feeding watchdog... 6
[I/wdt] [FeedDog] Feeding watchdog... 7
[I/wdt] [FeedDog] Feeding watchdog... 8
[I/wdt] [FeedDog] Feeding watchdog... 9
[E/wdt] [FeedDog] Simulate exception! Stop feeding.
[E/wdt] [FeedDog] Simulate exception! Stop feeding.
```

16. Wi-Fi Usage Instructions

English|Chinese

16.1 Introduction

This example demonstrates how to use the **RA8 series MCU SDHI module (r_sdhi)** on the **Titan Board** to drive the **CYWL6208-GS WiFi module**, enabling wireless network connectivity via the SDIO interface. In this example, the MCU communicates with the CYWL6208-GS module (chip CYW43438) through the **SDIO bus**, and integrates with the RT-Thread network stack (lwIP) to achieve TCP/IP network functionality.

16.2 WiFi Introduction

Overview

WiFi (Wireless Fidelity) is a wireless local area network (WLAN) communication technology based on the **IEEE 802.11 standard**. It transmits data between terminal devices (such as MCUs, smartphones, PCs, or IoT devices) and access points (APs) via radio waves, providing **short-range high-speed wireless communication**.

WiFi is currently one of the most widely used wireless technologies, commonly applied in home networks, office networks, industrial control, smart homes, and IoT devices.

Working Principle

1. **Communication Method**
    - WiFi uses the **ISM bands** (2.4 GHz, 5 GHz, 6 GHz) and modulation schemes such as **OFDM, DSSS, and QAM** for data transmission.
    - WiFi networks consist of **Access Points (APs)** and **Stations (STAs)**.
2. **Data Transmission Flow**
    - The STA scans for available WiFi networks.
    - STA sends authentication and association requests.
    - The AP completes handshake and key negotiation.
    - Once a secure communication channel is established between STA and AP, data transmission occurs.

WiFi Standards

Different WiFi versions correspond to the IEEE 802.11 standards:

Standard	Frequency Band	Max Rate	Features
802.11b	2.4 GHz	11 Mbps	Early version, poor interference resistance
802.11g	2.4 GHz	54 Mbps	Backward compatible with 802.11b
802.11n (WiFi 4)	2.4/5 GHz	600 Mbps	MIMO technology
802.11ac (WiFi 5)	5 GHz	3.5 Gbps	MU-MIMO, 80/160 MHz channels
802.11ax (WiFi 6)	2.4/5/6 GHz	9.6 Gbps	OFDMA, lower latency, higher concurrency
802.11be (WiFi 7)	2.4/5/6 GHz	>30 Gbps	Multi-link operation (MLO), ultra-high throughput

WiFi Modes

- **STA mode (Station)**: Device connects to an AP.
- **AP mode (Access Point)**: Device provides network access as a hotspot.
- **P2P mode (Peer-to-Peer/Ad-Hoc)**: Direct device-to-device communication.
- **Hybrid mode**: Both STA and AP functions coexist.

WiFi Security

- **WEP** (deprecated, insecure)
- **WPA/WPA2**: TKIP/AES-based encryption, mainstream security
- **WPA3**: Enhanced security, supports SAE handshake to prevent brute-force attacks

WiFi Characteristics

Advantages

- High-speed transmission (Gbps level)
- Widely supported (phones, PCs, MCUs, etc.)
- Low cost, mature ecosystem
- Can connect multiple devices simultaneously

Disadvantages

- High power consumption, unsuitable for ultra-low-power applications
- Limited transmission range (typically 10–100 m)
- 2.4 GHz band prone to interference (Bluetooth, microwave ovens, etc.)

16.3 RA8 Series SDHI Module Introduction

The **RA8 series MCU SDHI (SD Host Interface) module** is a host controller compliant with **SD/SDIO/eMMC standards**, capable of communicating with storage devices (SD card, eMMC) or SDIO peripherals (e.g., WiFi, Bluetooth modules) via **1-bit or 4-bit data lines**.

In **storage applications**, it functions as an SD/eMMC controller for high-speed read/write operations. In **peripheral expansion applications**, it acts as an SDIO host to establish data channels with WiFi, Bluetooth, GNSS, and other modules.

In this example, SDHI is used to drive the **CYWL6208-GS WiFi module (SDIO interface)** to enable wireless networking for the MCU.

Key Features

- **SD Standard Support**
  - SD v1.x / v2.x / SDHC / SDXC
  - Supports SPI mode and SD/MMC mode
- **High-Speed Data Transfer**
  - Up to 50 MHz SDCLK (depending on MCU clock configuration)
  - Supports 1-bit / 4-bit bus
- **Automatic Command and Data Transfer**
  - DMA support reduces CPU usage
  - Automatic command sequence generation (CMD0~CMD63)
- **Error Detection**
  - CRC7 for commands, CRC16 for data
  - Timeout detection and response error recognition
- **Interrupt Support**
  - Card insertion/removal detection
  - Command completion
  - Data transfer completion
  - Error interrupt

SDHI Module Architecture

The RA8 SDHI module consists of the following sub-modules:

1. **Command Control Unit**
   - Sends commands (CMD0~CMD63)
   - Handles responses (R1, R2, R3, R7, etc.)
   - Supports command timeout detection and CRC check
2. **Data Transfer Unit**
   - Data send/receive via internal FIFO or DMA
   - Supports block read/write, maximum 512-byte block
   - Single/multi-block transfer modes
3. **Clock and Bus Control**
   - SDCLK generation and division
   - 1-bit / 4-bit bus width switching
   - Configurable high/low-level hold time
4. **Card Detection and Power Control**
   - Detect card insertion/removal
   - Control card power if supported
5. **Interrupt and Event Control Unit**
   - Command completion interrupt
   - Data transfer completion interrupt
   - Error interrupt
   - Card insertion/removal interrupt

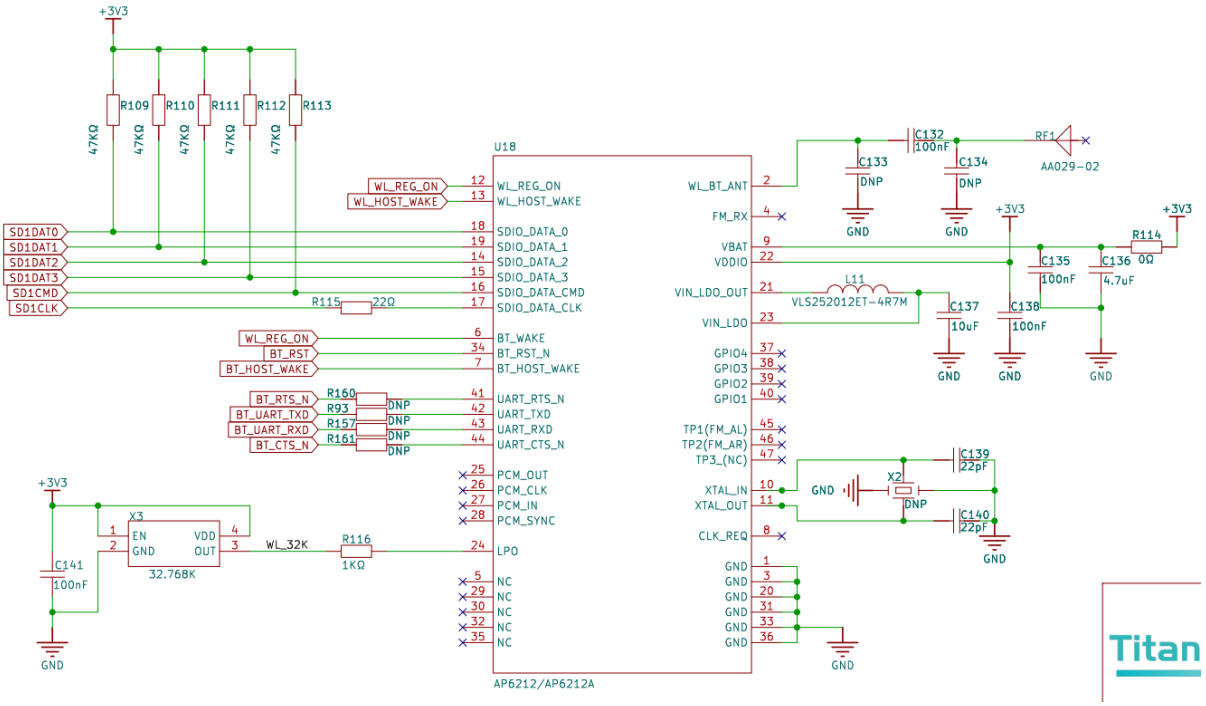
Workflow (Example: SDIO WiFi)

1. **Hardware Initialization**
   - Configure SDHI pins: CMD, CLK, DAT0~3
   - Enable clock, reset SDHI controller
2. **Bus Initialization**
   - Set clock frequency (initial low $\rightarrow$ high speed)
   - Send initialization commands (CMD0, CMD5, CMD3, etc.)
   - Enumerate device and assign RCA
3. **Function Configuration**
   - Identify SDIO WiFi device

- Configure block size, bus width, transfer rate
- Enable interrupts

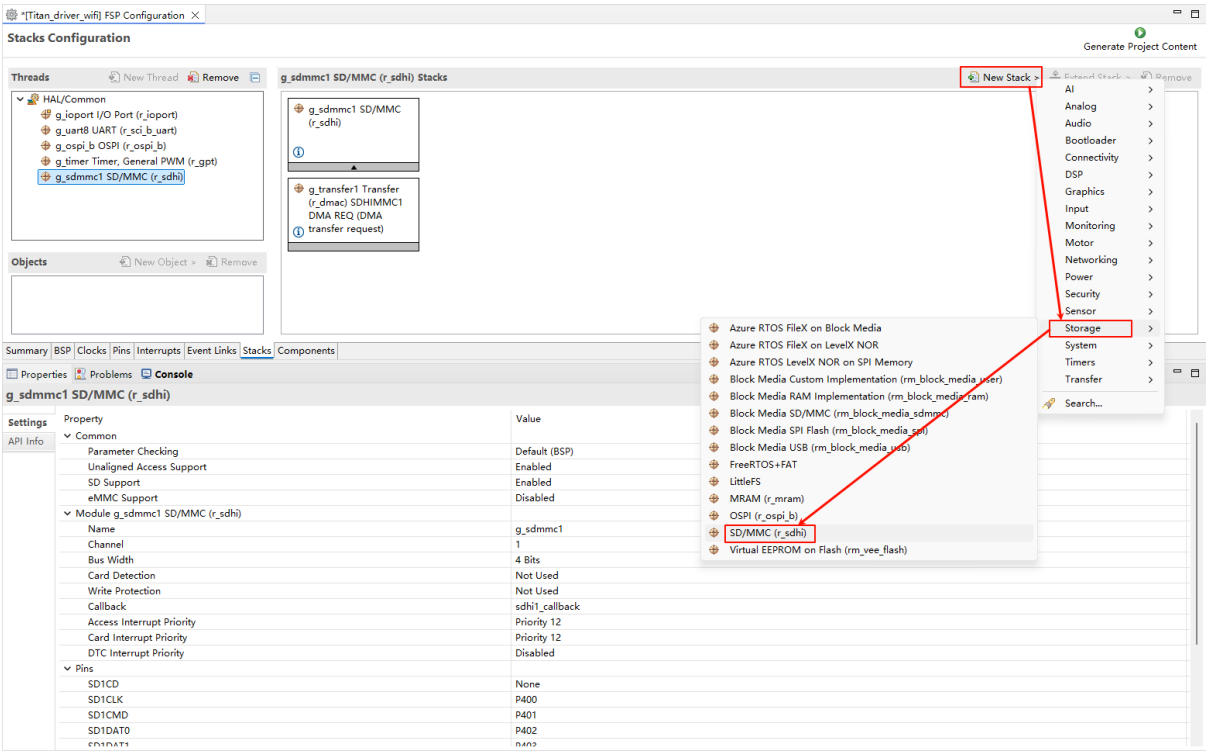
4. **Normal Operation**
   - WiFi driver accesses control registers and data transfer interface
   - MCU exchanges large packets with WiFi chip via **DMA + interrupts**
   - Upper layer protocol stack (lwIP) completes TCP/IP communication

16.4 Hardware Description

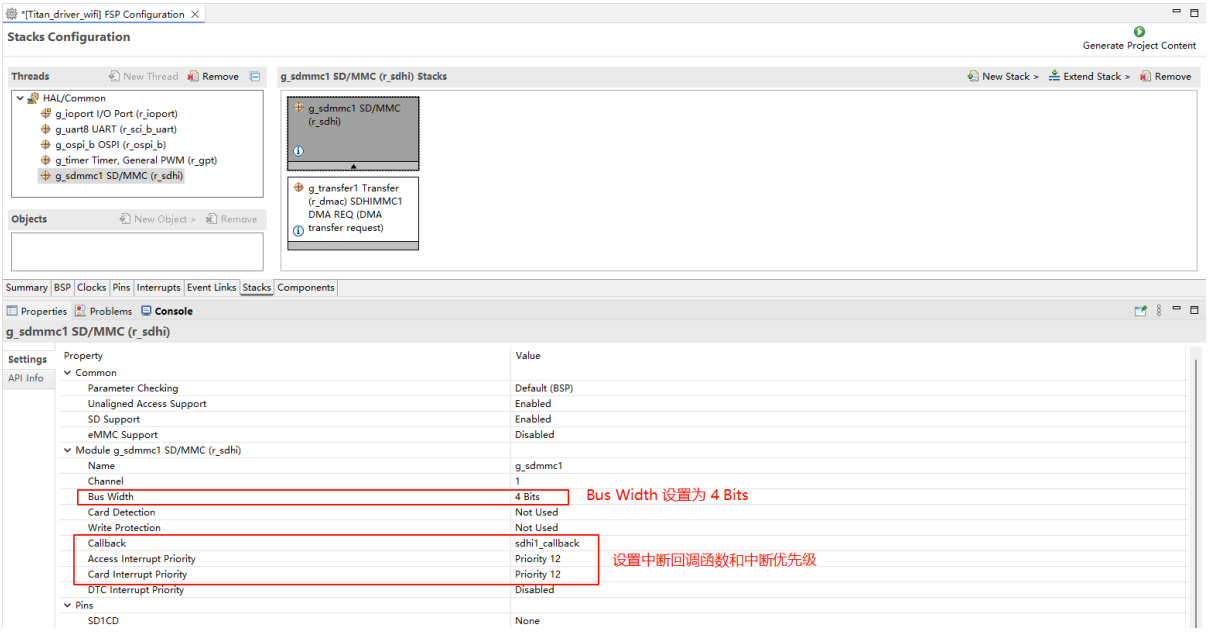


16.5 FSP Configuration

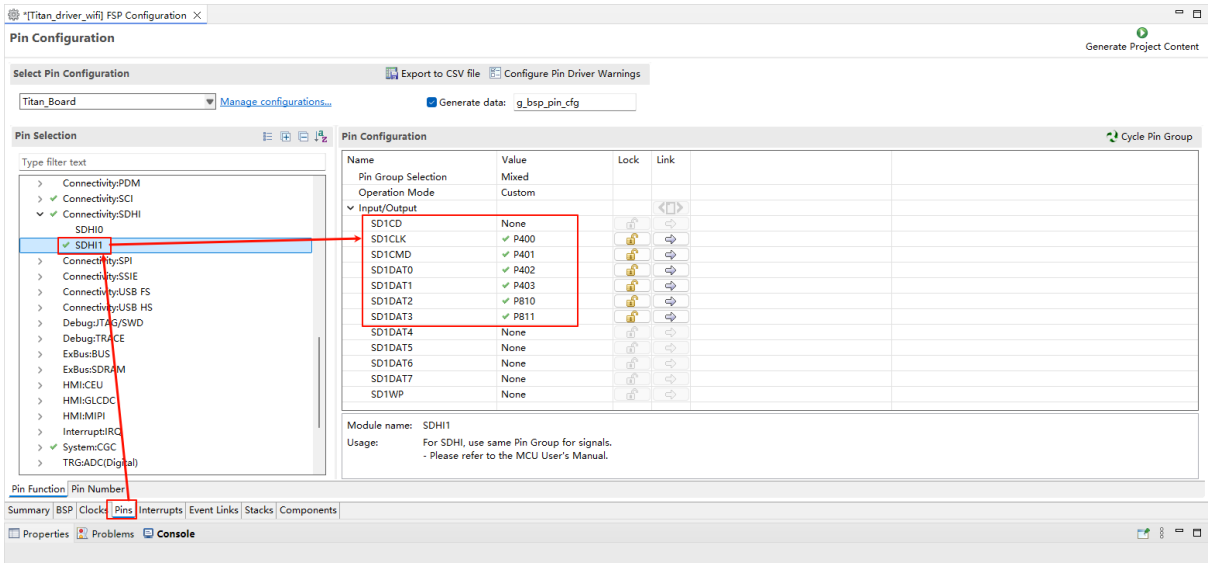
- The first step is to configure Flash. See the README.md in the Titan_component_flash_fs project for the Flash configuration.
- Next, configure SDHI1 and create a new r_sdhi stack:



- Configure r_sdhi stack:

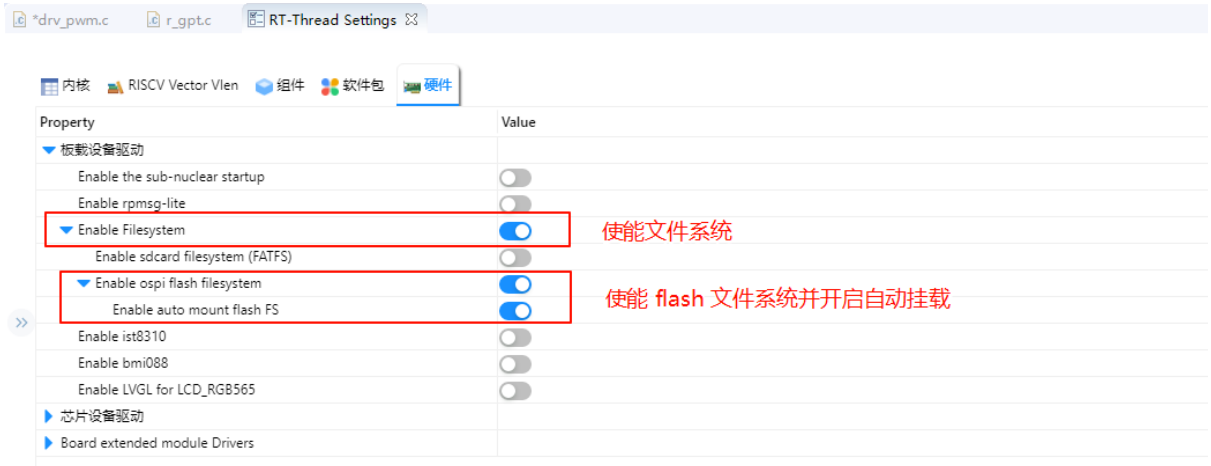


- Configure SDHI1 pins：

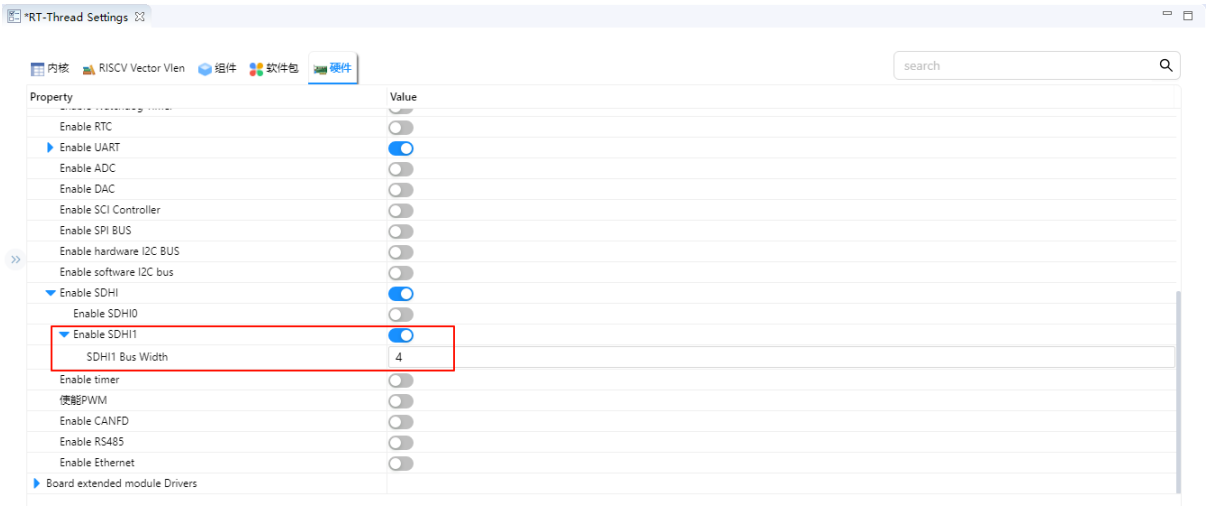


16.6 RT-Thread Settings Configuration

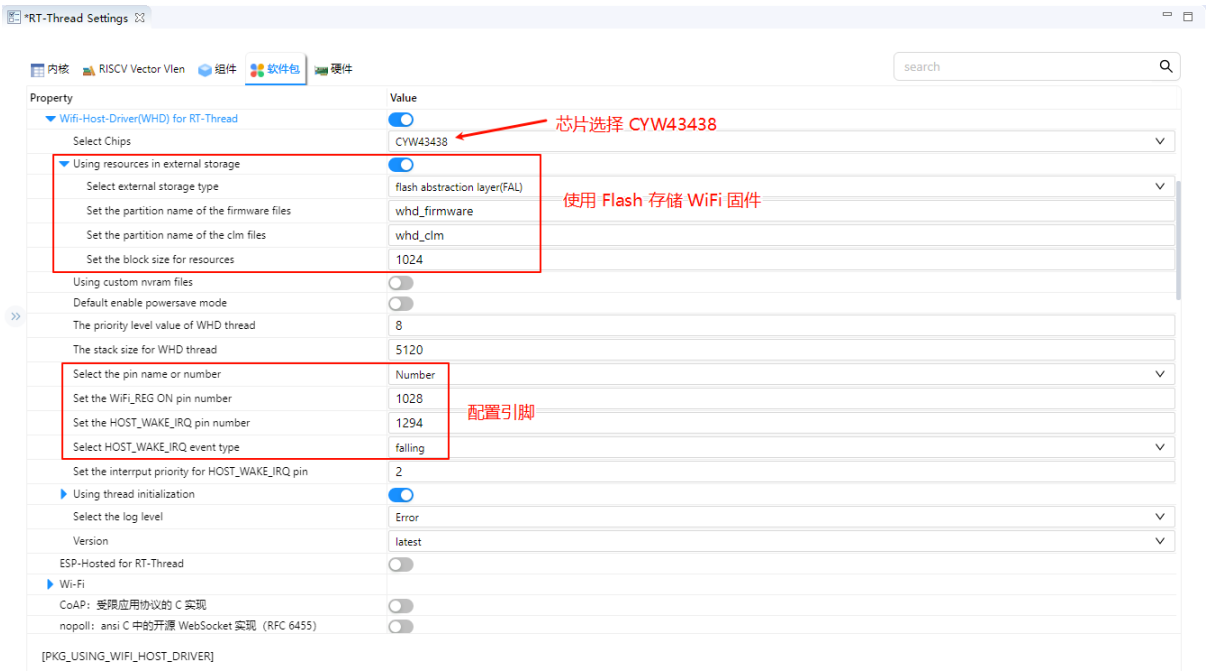
- Enable ospi Flash.



- The Bus Width of SDHI1 is set to 4.



- Configure the WHD package.



16.7 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio’s package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board’s USB-DBG interface to the PC and download the firmware to the development board.

16.8 Run Effect

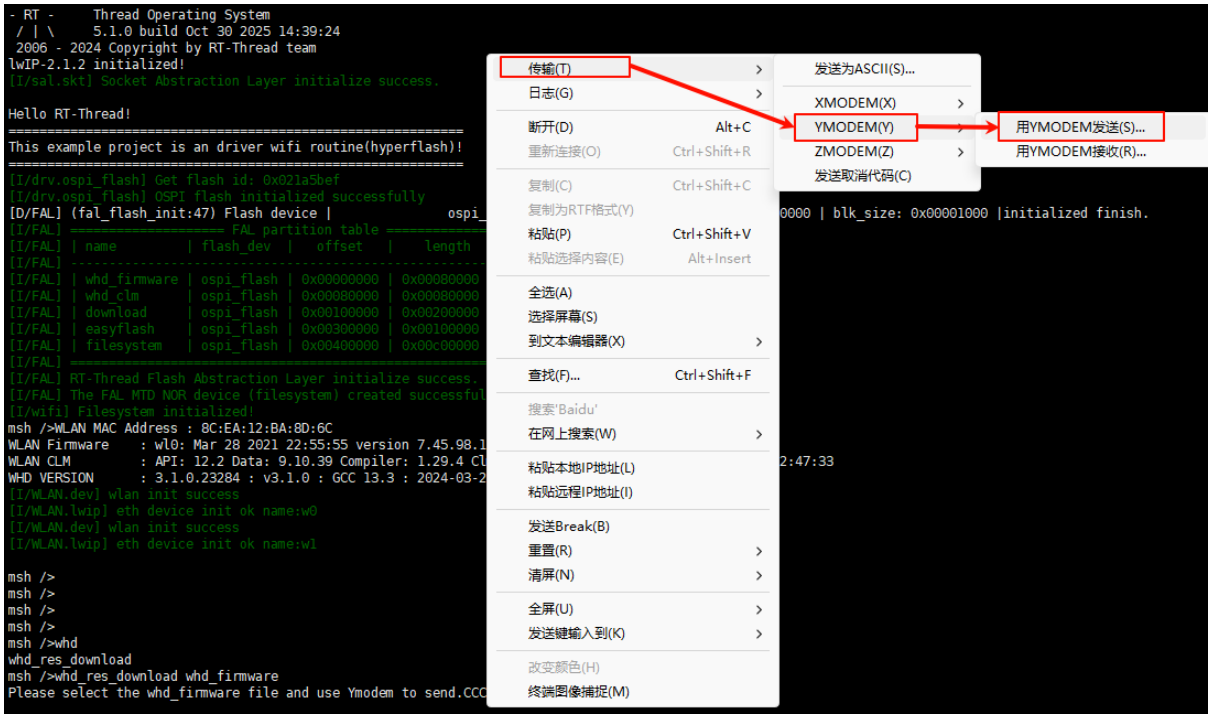
Press the reset button to restart the development board, if the terminal output firmware read error information, then you need to use **ymodem** to download WiFi firmware to Flash. Use the whd_res_download whd_firmware command to download 43438A1.bin into the whd_firmware partition of Flash; Use the whd_res_download whd_clm command to download 43438A1.clm_blob into the whd_clm partition of the Flash.

The WiFi firmware can be found in the /firmware folder of the project root.

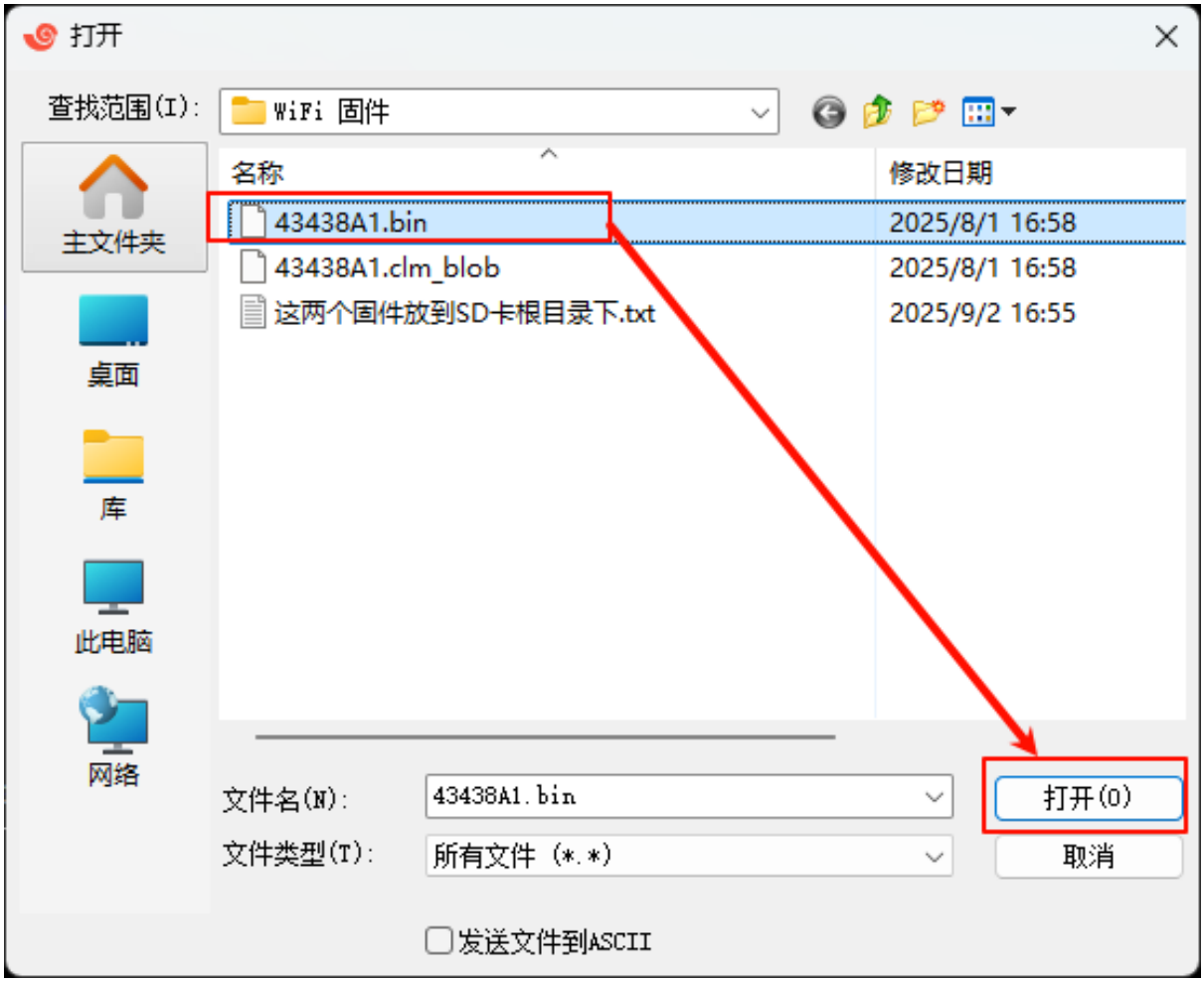
Download WiFi firmware :

Use a serial tool that supports **YMODEM**, such as **Xshell**.

After entering the command whd_res_download whd_firmware, right-click in the window, then select **Transfer** → **YMODEM** → **Send with YMODEM**.

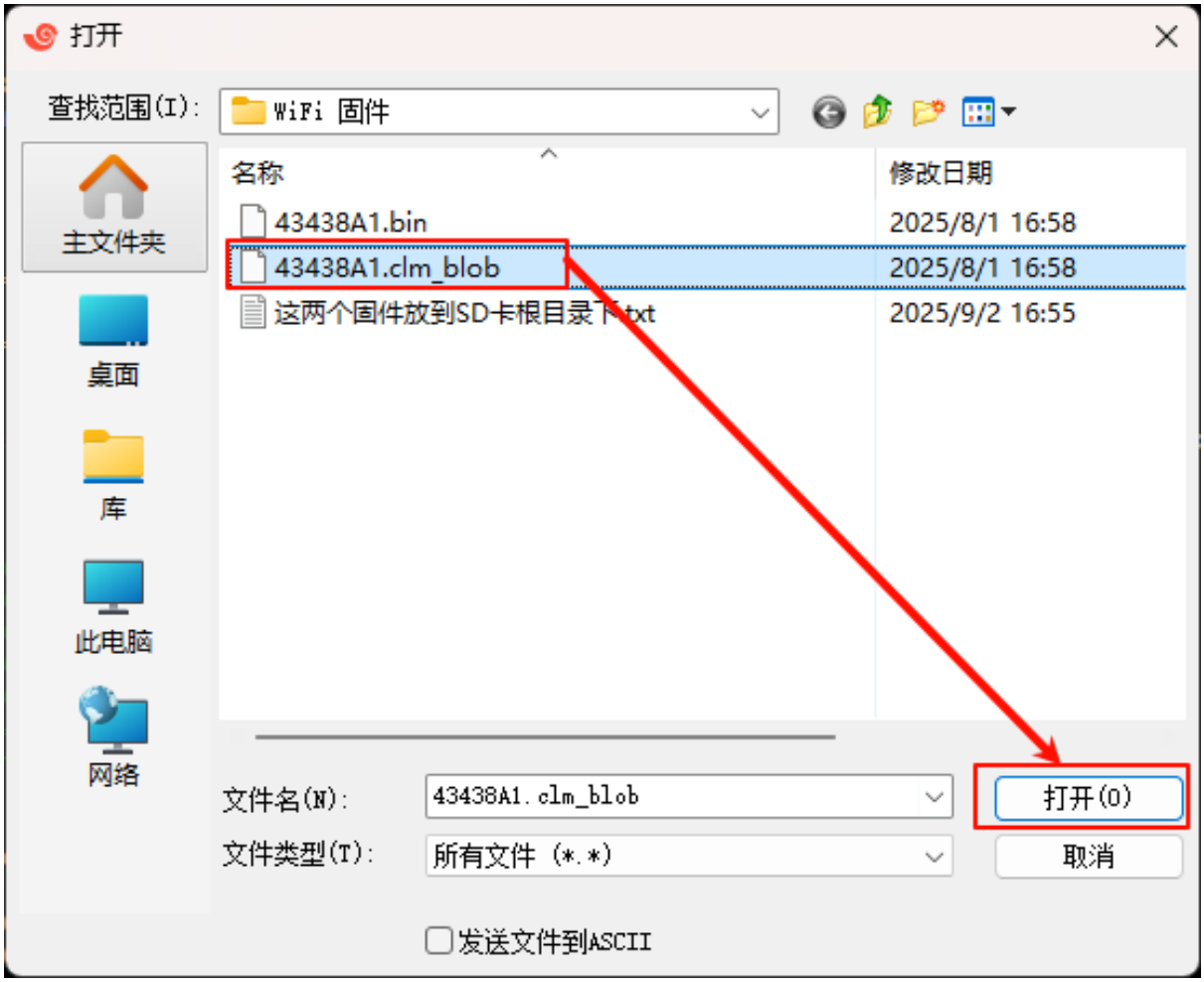


Select the 43438A1.bin file, then click **Open** to start flashing 43438A1.bin.



After entering the command whd_res_download whd_clm, right-click in the window, then select **Transfer** → **YMODEM** → **Send with YMODEM**.

Select the 43438A1.clm_blob file, then click **Open** to start flashing 43438A1.clm_blob.



Test WiFi:

After downloading the WiFi firmware, reset the Titan Board, and you can see that the WiFi firmware is loaded normally and the WiFi module is initialized successfully. Then type wifi join ssid password to connect to WiFi and ping baidu.com to ping test.

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Aug 14 2025 18:08:15
 2006 - 2024 Copyright by RT-Thread team
lwIP-2.0.3 initialized!
[I/sal.skt] Socket Abstraction Layer initialize success.

Hello RT-Thread core0!
[I/drv.ospi_flash] Get flash id: 0x021a5bef
[I/drv.ospi_flash] OSPI flash initialized successfully
[D/FAL] Flash device |              ospi_flash | addr: 0x90000000 | len: 0x04000000 | blk_size: 0x00001000 |initialized finish.
[I/FAL] ==================== FAL partition table ====================
[I/FAL] | name         | flash_dev  |   offset   |    length  |
[I/FAL] -------------------------------------------------------------
[I/FAL] | whd_firmware | ospi_flash | 0x00000000 | 0x00080000 |
[I/FAL] | whd_clm      | ospi_flash | 0x00080000 | 0x00080000 |
[I/FAL] | download     | ospi_flash | 0x00100000 | 0x00200000 |
[I/FAL] | easyflash    | ospi_flash | 0x00300000 | 0x00100000 |
[I/FAL] | filesystem   | ospi_flash | 0x00400000 | 0x00c00000 |
[I/FAL] =============================================================
[I/FAL] RT-Thread Flash Abstraction Layer initialize success.
[I/FAL] The FAL MTD NOR device (filesystem) created successfully
[I/wifi] Filesystem initialized!
msh />WLAN MAC Address : 2C:B0:FD:A2:2D:60
WLAN Firmware    : wl0: Mar 28 2021 22:55:55 version 7.45.98.117 (dc5d9c4 CY) FWID 01-d36e8386
WLAN CLM         : API: 12.2 Data: 9.10.39 Compiler: 1.29.4 ClmImport: 1.36.3 Creation: 2021-03-28 22:47:33
WHD VERSION      : 3.1.0.23284 : v3.1.0 : GCC 13.2 : 2024-03-21 22:57:11 +0800
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w0
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w1

msh />wifi join TitanBoard 12345678
[I/WLAN.mgnt] wifi connect success ssid:TitanBoard
msh />[I/WLAN.lwip] Got IP address : 192.168.173.8

msh />ping baidu.com
ping: not found specified netif, using default netdev w0.
60 bytes from 182.61.201.211 icmp_seq=1 ttl=50 time=60 ms
60 bytes from 182.61.201.211 icmp_seq=2 ttl=50 time=55 ms
60 bytes from 182.61.201.211 icmp_seq=3 ttl=50 time=55 ms
60 bytes from 182.61.201.211 icmp_seq=4 ttl=50 time=245 ms

--- 182.61.201.211 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
minimum = 55ms, maximum = 245ms, average = 103ms
msh />
```

Components

1. Flash File System Usage Instructions

English | Chinese

1.1 Introduction

This example demonstrates how to use **OSPI Flash** on the **Titan Board** to mount a **LittleFS file system**. With this example, you can read and write files on external flash, achieving persistent data storage. The program is based on the **RT-Thread file system component** and the **OSPI driver**.

1.2 RA8 Series OSPI Features

The RA8 series MCU provides an **Octal SPI / Quad SPI interface (OSPI)**, specifically designed for high-speed external flash access. Its main features are as follows:

Multi-line High-Speed Access

- Supports **1–8 data line modes**:
  - 1-line SPI (standard SPI)
  - 4-line QSPI (Quad SPI)
  - 8-line OSPI (Octal SPI)
- Maximum clock frequency up to **100 MHz** (depending on MCU and Flash device), suitable for high-capacity, high-speed read/write operations.
- Supports **continuous read mode**, reducing the overhead of issuing commands for each read.

Flexible Flash Operation Modes

- Supports **Page Program**, typically 256 bytes per page, with block management for cross-page writes.
- Supports **Block Erase**, typical block sizes of 4KB or 64KB, allowing selective erasure to improve write efficiency.
- Supports **random read** and **continuous read** operations.
- Configurable **read/write commands, dummy cycles, and data line width** to match different Flash devices.
- Supports **high-speed modes (DDR / Quad / Octal I/O)** to improve transfer efficiency.

Interrupt and DMA Support

- The OSPI peripheral can notify transfer completion via **interrupts**.
- Supports **DMA mode** to reduce CPU load.
- Can be integrated with RT-Thread as a **block device (blkdev)**, facilitating file system mounting or data caching.

RT-Thread Driver Integration

- The RA8 OSPI driver can be registered as an RT-Thread **block device**.
- LittleFS or FatFS can be mounted on this block device for file system operations.
- Supports **thread-safe APIs**, allowing multi-threaded access to Flash.
- Configurable **page cache size** and **block erase/write strategy** to optimize LittleFS performance.

High Reliability Features

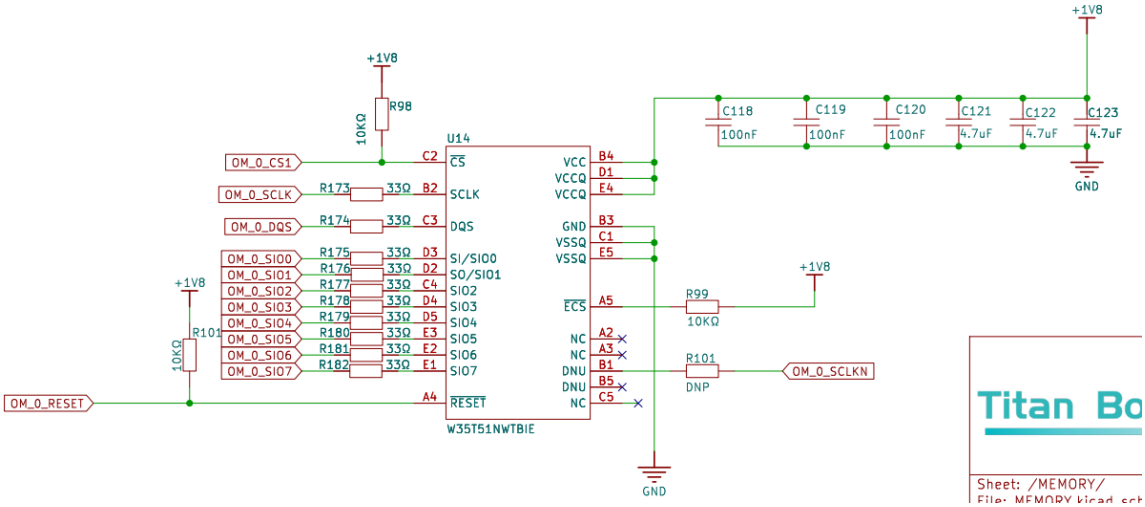
- Supports **write protection** and **software lock**.
- Supports **error detection**, such as command errors or data CRC verification.
- Ensures safe file system mounting after power loss or system reset.

1.3 Abstraction Layer

- **FAL** (Flash Abstraction Layer) is an abstraction layer for managing and operating Flash devices and Flash-based partitions. It provides a unified API for upper-layer Flash and partition operations (as shown in the framework diagram below), with the following features:
  - Supports a statically configurable partition table, which can be associated with multiple Flash devices.
  - The partition table supports **automatic loading**, avoiding the issue of the partition table being repeatedly defined in multi-firmware projects.
  - Lightweight code with **no dependency on an operating system**, allowing it to run on bare-metal platforms, such as bootloaders with limited resources.
  - A unified operation interface ensures the reusability of underlying Flash drivers for components that rely on Flash, such as file systems, OTA, and NVM (e.g., EasyFlash).
  - Built-in Finsh/MSH-based test commands, enabling developers to read, write, and erase Flash or partitions via shell commands with byte-level addressing, which facilitates debugging and testing.

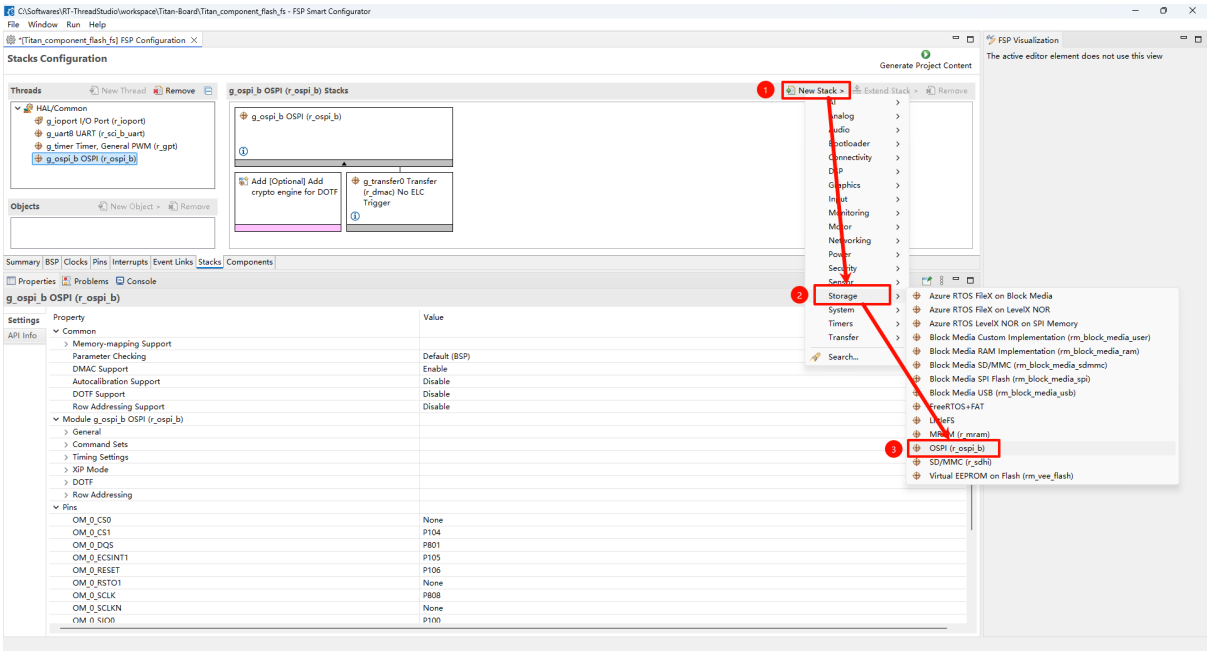
1.4 Hardware Description

FLASH

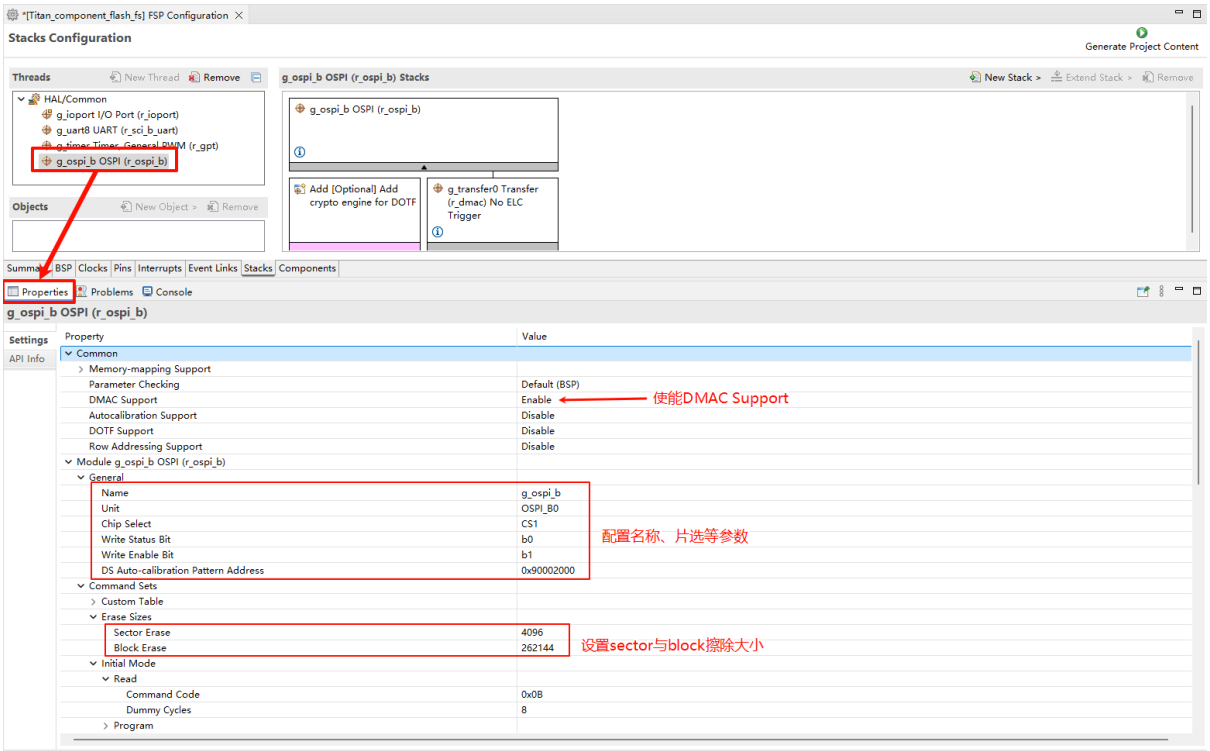


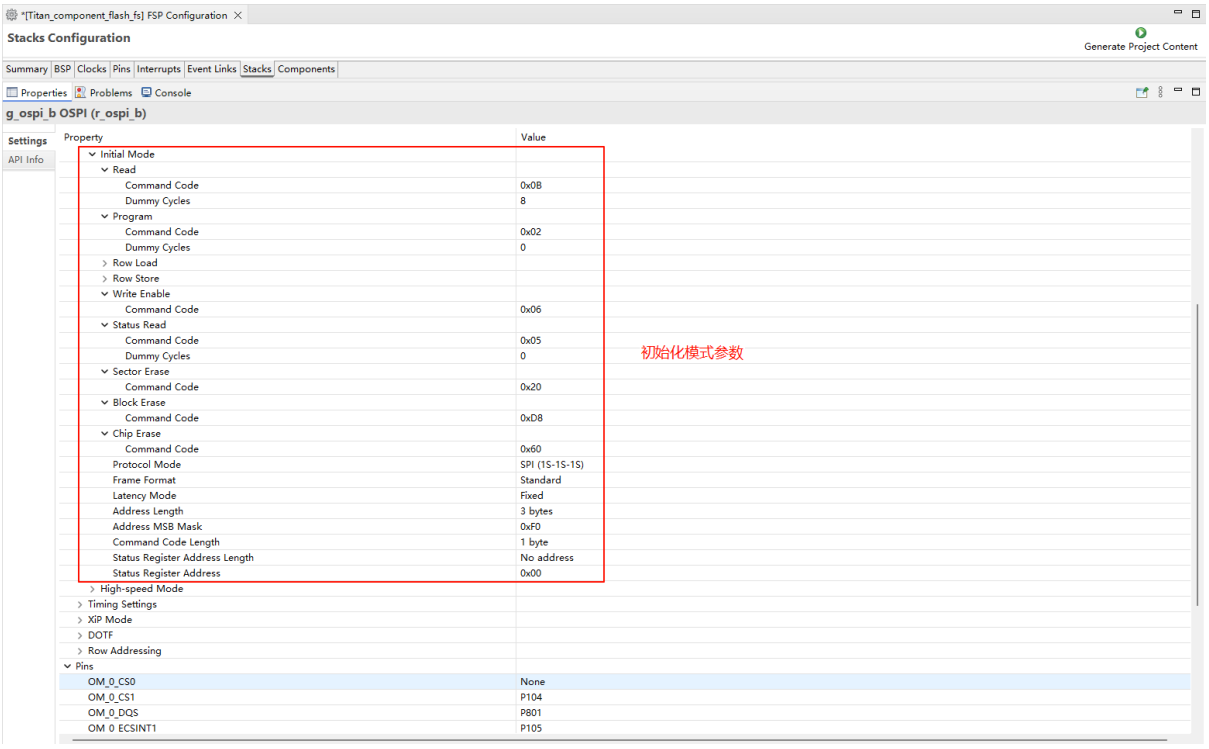
1.5 FSP Configuration

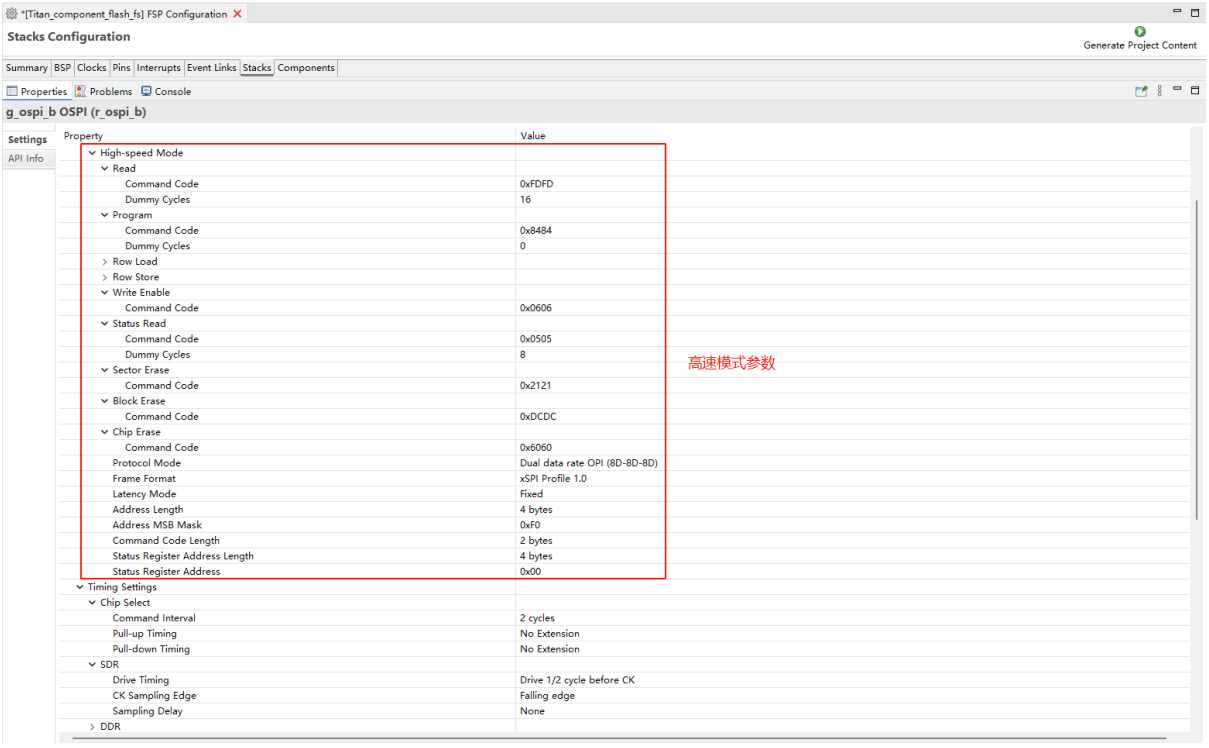
- Create a r_ospi_b stack:

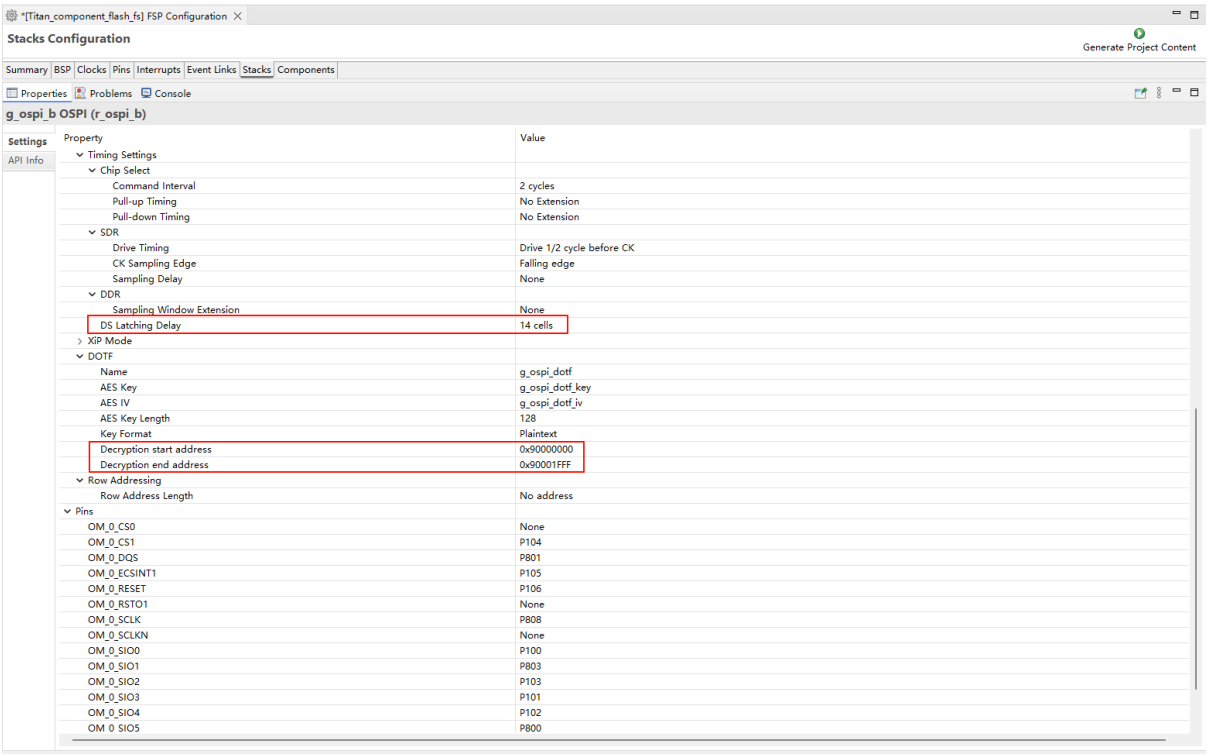


- Configure the r_ospi_b stack:

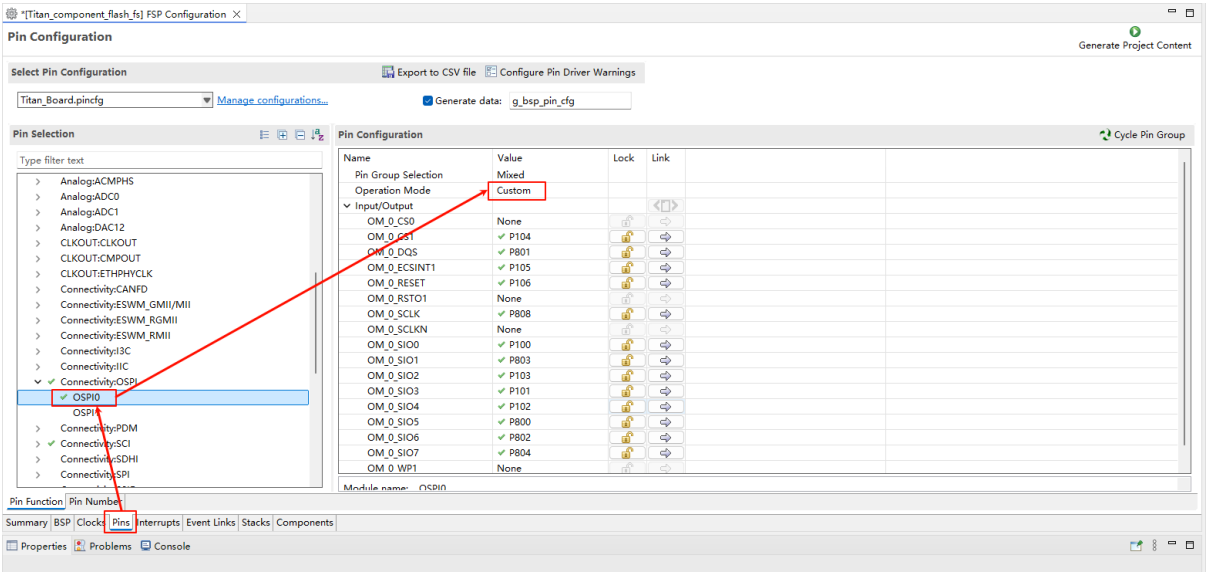



*[Titan_component_flash_fs] FSP Configuration
Stacks Configuration
Generate Project Content
Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems Console
g_ospi_b OSPI (r_ospi_b)
Settings
API Info
Property
Value
Initial Mode
Read
Command Code 0x0B
Dummy Cycles 8
Program
Command Code 0x02
Dummy Cycles 0
Row Load
Row Store
Write Enable
Command Code 0x06
Status Read
Command Code 0x05
Dummy Cycles 0
Sector Erase
Command Code 0x20
Block Erase
Command Code 0xD8
Chip Erase
Command Code 0x60
Protocol Mode SPI (1S-1S-1S)
Frame Format Standard
Latency Mode Fixed
Address Length 3 bytes
Address MSB Mask 0xF0
Command Code Length 1 byte
Status Register Address Length No address
Status Register Address 0x00
High-speed Mode
Timing Settings
XiP Mode
DOTF
Row Addressing
Pins
OM_0_CS0 None
OM_0_CS1 P104
OM_0_DQS P801
OM_0_ECSINT1 P105
初始化模式参数


*[Titan_component_flash_fs] FSP Configuration
Stacks Configuration
Generate Project Content
Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems Console
g_ospi_b OSPI (r_ospi_b)
Settings
API Info
Property
Value
High-speed Mode
Read
Command Code 0xFDFD
Dummy Cycles 16
Program
Command Code 0x8484
Dummy Cycles 0
Row Load
Row Store
Write Enable
Command Code 0x0606
Status Read
Command Code 0x0505
Dummy Cycles 8
Sector Erase
Command Code 0x2121
Block Erase
Command Code 0xDCDC
Chip Erase
Command Code 0x6060
Protocol Mode Dual data rate OPI (8D-8D-8D)
Frame Format xSPI Profile 1.0
Latency Mode Fixed
Address Length 4 bytes
Address MSB Mask 0xF0
Command Code Length 2 bytes
Status Register Address Length 4 bytes
Status Register Address 0x00
Timing Settings
Chip Select
Command Interval 2 cycles
Pull-up Timing No Extension
Pull-down Timing No Extension
SDR
Drive Timing Drive 1/2 cycle before CK
CK Sampling Edge Falling edge
Sampling Delay None
DDR
高速模式参数

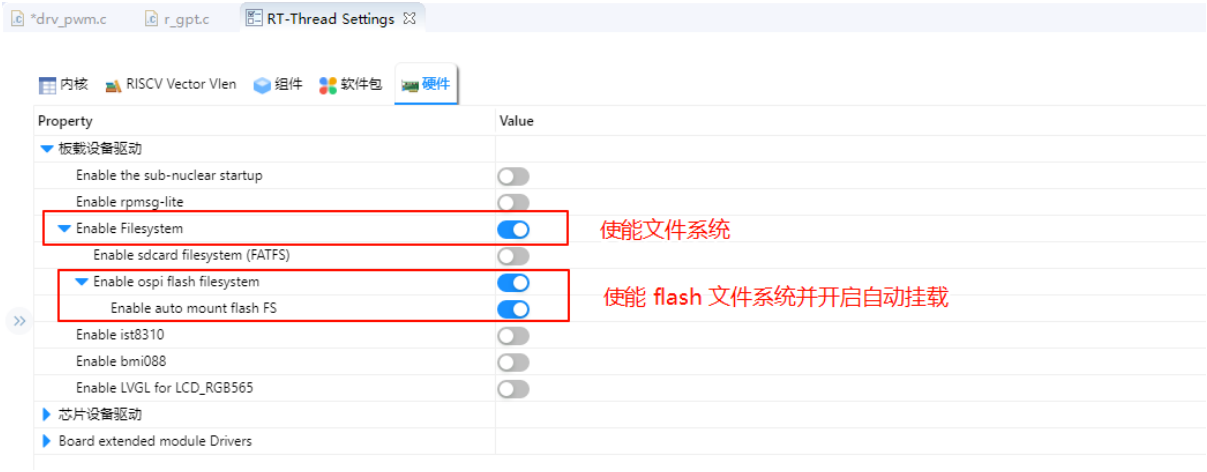


- Flash pin configuration:



1.6 RT-Thread Settings Configuration

- Enable Flash filesystem in RT-Thread Settings.



1.7 Software Overview

The source code for file system initialization in this example is located at: ./board/ports/drv_filesystem.c 中：

```
#include <rtthread.h>

#if defined(BSP_USING_FILESYSTEM)
#include <dfs_romfs.h>
#include <dfs_fs.h>
#include <dfs_file.h>

#if DFS_FILESYSTEMS_MAX < 4
#error "Please define DFS_FILESYSTEMS_MAX more than 4"
#endif
#if DFS_FILESYSTEM_TYPES_MAX < 4
#error "Please define DFS_FILESYSTEM_TYPES_MAX more than 4"
#endif

#define DBG_TAG "app.filesystem"
#define DBG_LVL DBG_INFO
#include <rtdbg.h>

#ifdef BSP_USING_FS_AUTO_MOUNT
#ifdef BSP_USING_SDCARD_FATFS
static int onboard_sdcard_mount(void)
{
    if (dfs_mount("sd", "/sdcard", "elm", 0, 0) == RT_EOK)
    {
        LOG_I("SD card mount to '/sdcard'");
```

(continues on next page)

(continued from previous page)

```
    }
    else
    {
        LOG_E("SD card mount to '/sdcard' failed!");
        rt_pin_write(0x000D, PIN_LOW);
    }

    return RT_EOK;
}
#endif /* BSP_USING_SDCARD_FATFS */
#endif /* BSP_USING_FS_AUTO_MOUNT */

#ifdef BSP_USING_FLASH_FS_AUTO_MOUNT
#ifdef BSP_USING_FLASH_FATFS
#define FS_PARTITION_NAME "filesystem"

static int onboard_fal_mount(void)
{
    /* init fal */
    extern int fal_init(void);
    extern struct rt_device* fal_mtd_nor_device_create(const char *parition_
  name);
    fal_init ();
    /* Create a block device on the ospi flash partition named "filesystem" */
    struct rt_device *mtd_dev = fal_mtd_nor_device_create (FS_PARTITION_NAME);
    if (mtd_dev == NULL)
    {
        LOG_E("Can't create a mtd device on '%s' partition.", FS_PARTITION_NAME);
        return -RT_ERROR;
    }
    else
    {
        LOG_D("Create a mtd device on the %s partition of flash successful.", FS_
  PARTITION_NAME);
    }

    /* Mount the filesystem on the ospi flash partition named "filesystem" */
    if (dfs_mount (FS_PARTITION_NAME, "/fal", "lfs", 0, 0) == 0)
    {
        LOG_I("Filesystem initialized!");
    }
    else
    {
        dfs_mkfs ("lfs", FS_PARTITION_NAME);
        if (dfs_mount ("filesystem", "/fal", "lfs", 0, 0) == 0)
        {
```

(continues on next page)

(continued from previous page)

```
            LOG_I("Filesystem initialized!");
        }
        else
        {
            LOG_E("Failed to initialize filesystem!");
            rt_pin_write(0x000D, PIN_LOW);
        }
    }

    return RT_EOK;
}
#endif /*BSP_USING_FLASH_FATFS*/
#endif /*BSP_USING_FLASH_FS_AUTO_MOUNT*/

const struct romfs_dirent _romfs_root[] =
{
#ifdef BSP_USING_SDCARD_FATFS
    {ROMFS_DIRENT_DIR, "sdcard", RT_NULL, 0},
#endif

#ifdef BSP_USING_FLASH_FATFS
  { ROMFS_DIRENT_DIR, "fal", RT_NULL, 0 },
#endif
        };

const struct romfs_dirent romfs_root =
{
ROMFS_DIRENT_DIR, "/", (rt_uint8_t*) _romfs_root, sizeof(_romfs_root) / sizeof(_
  romfs_root[0])
};

static int filesystem_mount(void)
{

#ifdef RT_USING_DFS_ROMFS
    if (dfs_mount(RT_NULL, "/", "rom", 0, &(romfs_root)) != 0)
    {
        LOG_E("rom mount to '/' failed!");
    }

    /* Ensure that the block device is successfully registered before mounting
  the filesystem */
    rt_thread_delay(500);
#endif
#ifdef BSP_USING_FS_AUTO_MOUNT
    onboard_sdcard_mount();
```

(continues on next page)

(continued from previous page)

```
#endif /* BSP_USING_FS_AUTO_MOUNT */

#ifdef BSP_USING_FLASH_FS_AUTO_MOUNT
    onboard_fal_mount ();
#endif

    return RT_EOK;
}
INIT_COMPONENT_EXPORT(filesystem_mount);
#endif /* defined(BSP_USING_FILESYSTEM)*/
```

1.8 Build & Download

- **RT-Thread Studio:** Download the Titan Board resource pack from the RT-Thread Studio package manager, then create a new project and compile it.

Once compiled, connect the development board’ s USB-DBG interface to the PC, and download the firmware to the development board.

1.9 Running Result

Press the reset button to restart the development board and observe the terminal logs from the board.

```
initialize rti_board_start
:0 done

 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Jul 25 2025 15:14:51
 2006 - 2024 Copyright by RT-Thread team
do components initialization.
initialize rti_board_end
:0 done
initialize rt_work_sys_workqueue_init
:0 done
initialize dfs_init
:0 done
initialize dfs_lfs_init
:0 done
initialize finsh_system_init
:0 done

Hello RT-Thread!
================================================
This example project is an ospi flash routine!
================================================
[I/drv.ospi_flash] Get flash id: 0x021a5bef
[I/drv.ospi_flash] OSPI flash initialized successfully
[D/FAL] Flash device |             ospi_flash | addr: 0x90000000 | len: 0x04000000 | blk_size: 0x00001000 |initialized finish.
[I/FAL] ==================== FAL partition table ====================
[I/FAL] | name       | flash_dev  |   offset   |    length  |
[I/FAL] -------------------------------------------------------------
[I/FAL] | wifi_image | ospi_flash | 0x00000000 | 0x00080000 |
[I/FAL] | bt_image   | ospi_flash | 0x00080000 | 0x00080000 |
[I/FAL] | download   | ospi_flash | 0x00100000 | 0x00200000 |
[I/FAL] | easyflash  | ospi_flash | 0x00300000 | 0x00100000 |
[I/FAL] | filesystem | ospi_flash | 0x00400000 | 0x00c00000 |
[I/FAL] =============================================================
[I/FAL] RT-Thread Flash Abstraction Layer initialize success.
[I/FAL] The FAL MTD NOR device (filesystem) created successfully
[I/main] Filesystem initialized!
msh />
```

2. Netutils Usage Instructions

English | Chinese

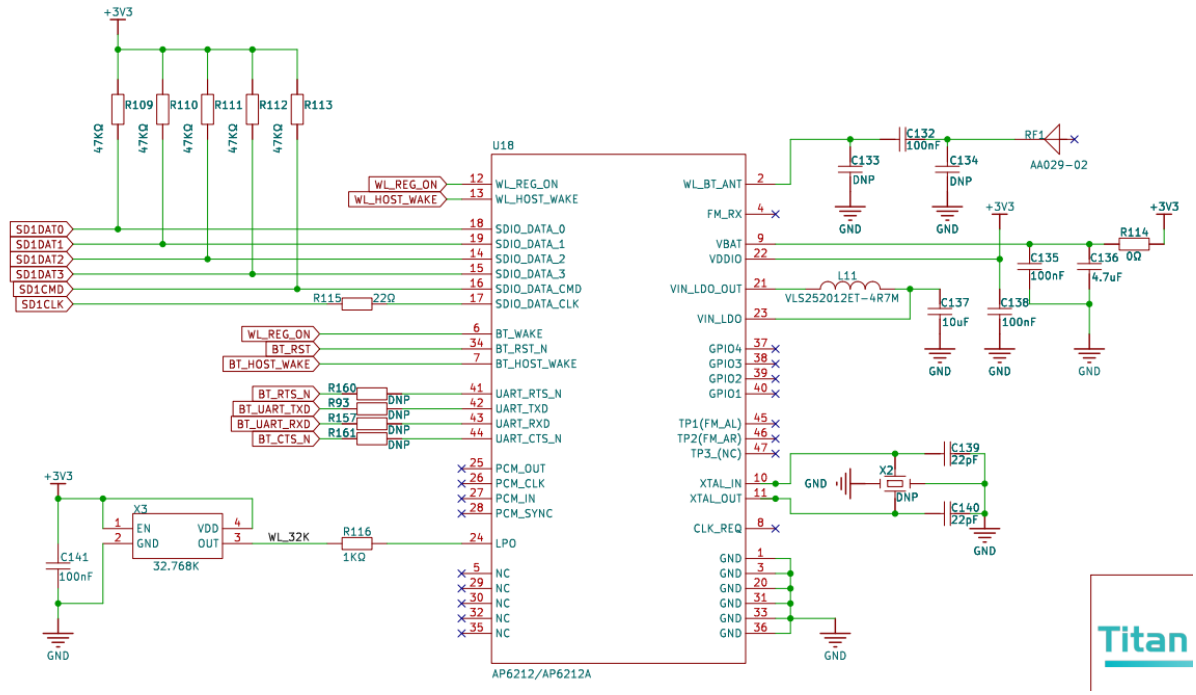
2.1 Instruction

netutils is a package that contains many simple and easy to use network tools. Using this package, developers can bring a lot of convenience when debugging network functions. When you need to use some network debugging gadgets, all you need is the netutils package, the Swiss Army knife of network functionality debugging.

This routine shows how to use the various features of the netutils package on the Titan Board development board.

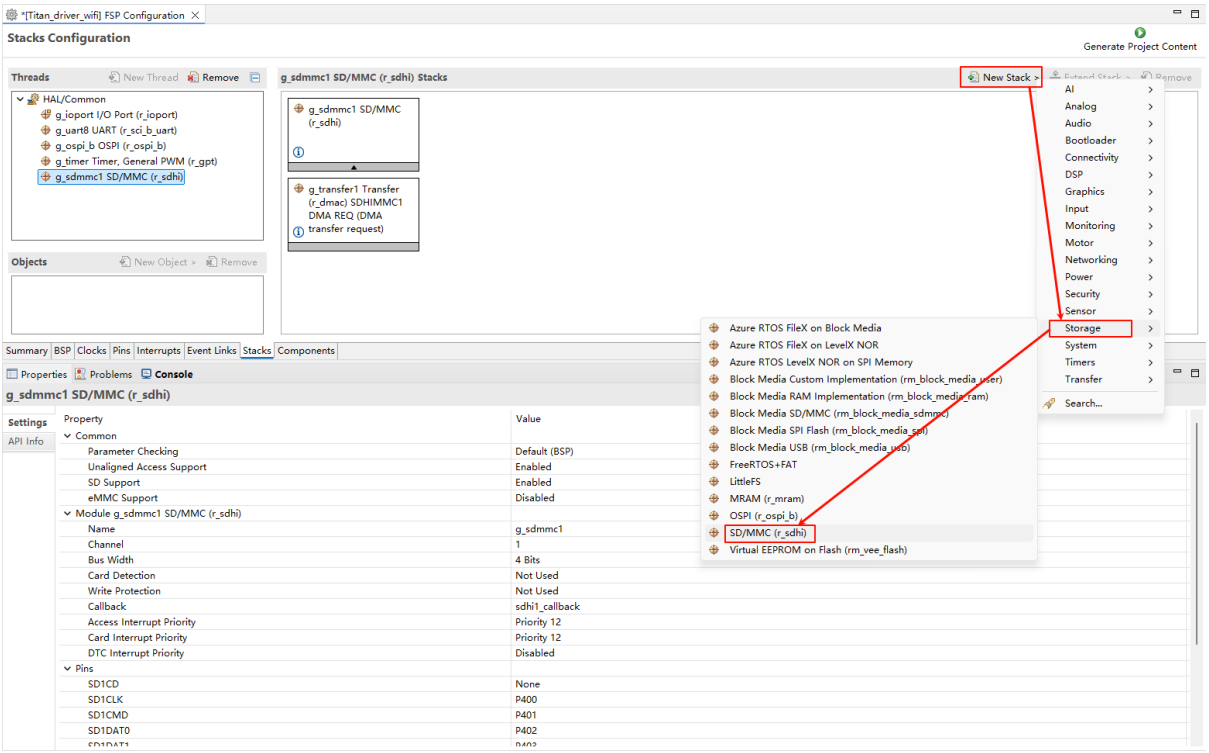
2.2 Hardware Description

This routine needs to rely on the WiFi module on the Titan Board to complete the network communication, so please ensure that the WiFi module on the hardware platform can work properly.

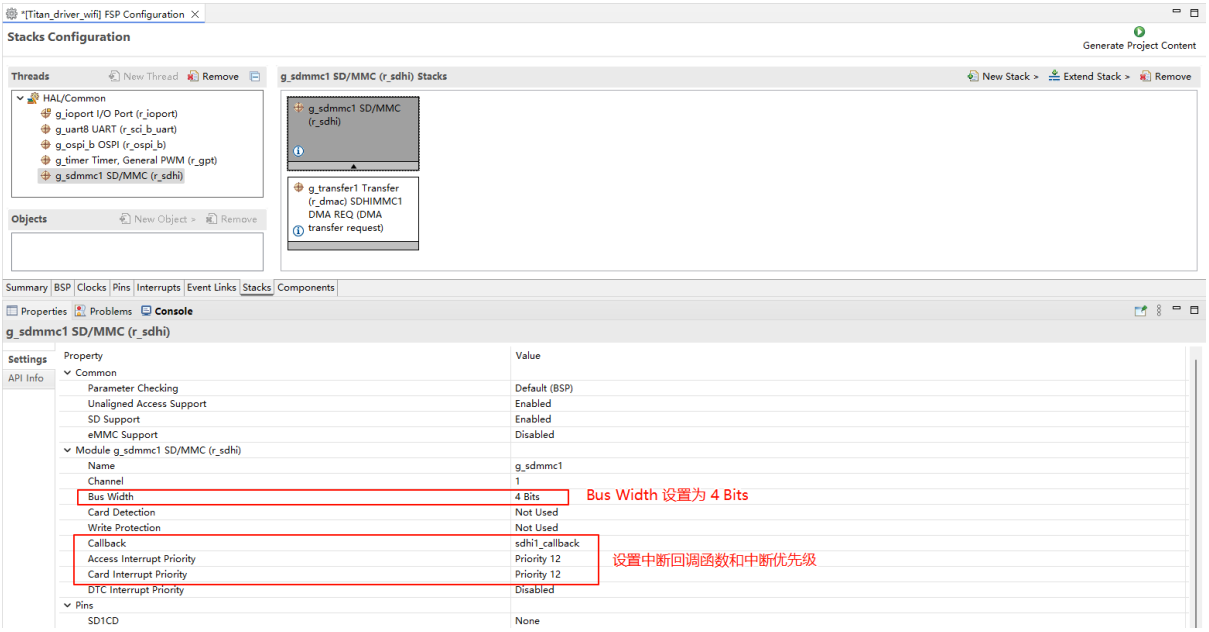


2.3 FSP Configuration

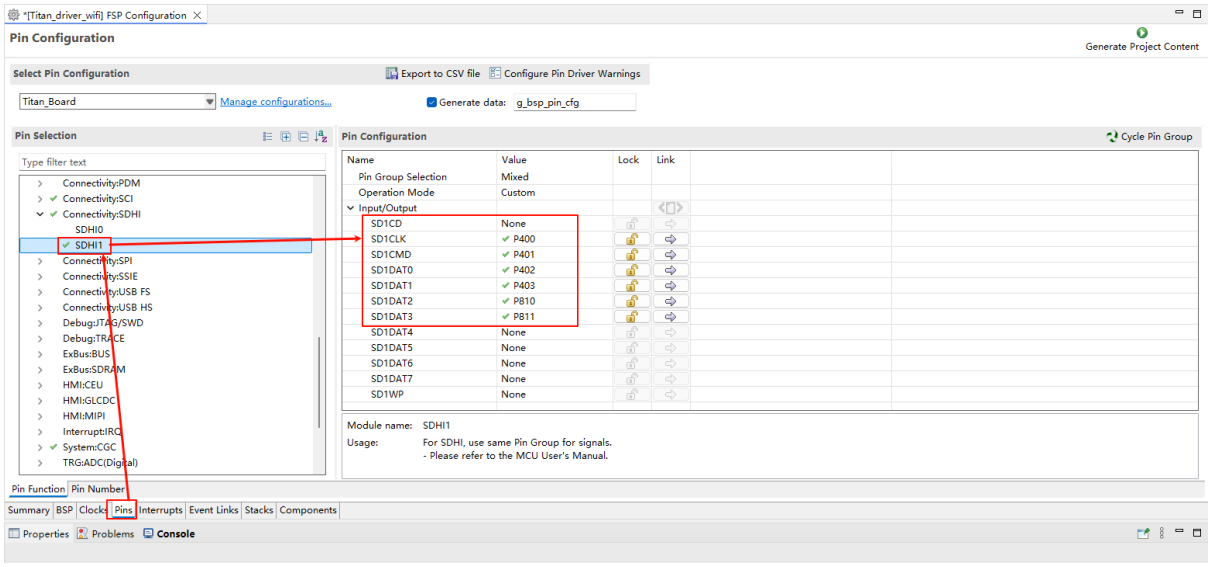
- First you need to configure Flash. See the *README.md* in the Titan_component_flash_fs project for the Flash configuration.
- Next, configure SDHI1 and create a r_sdhi stack:



- Configure SDHI1 stack:

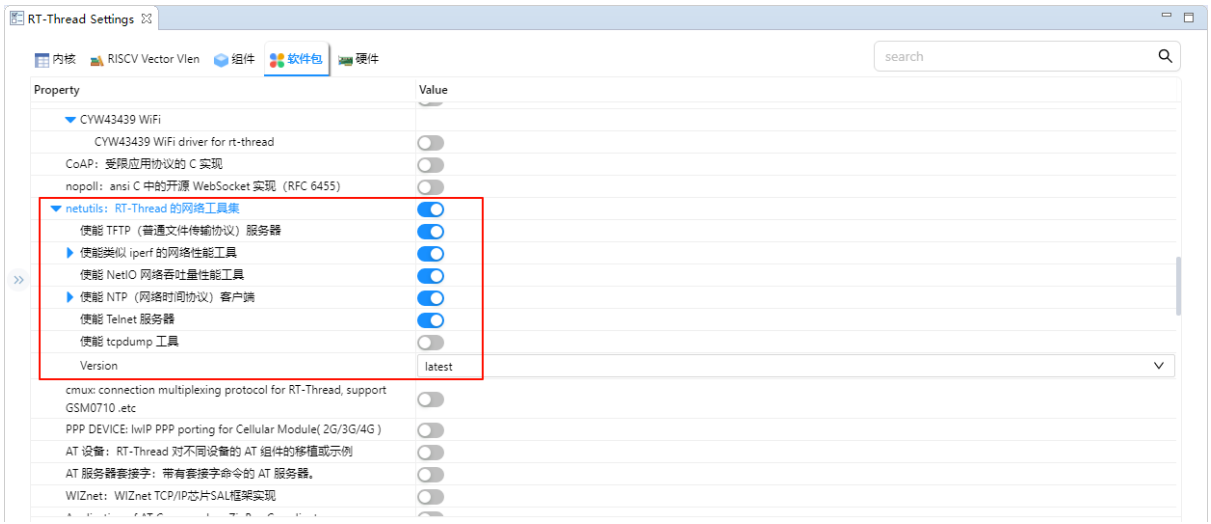


- Configure SDHI1 pins:



2.4 RT-Thread Settings Configuration

- Add the netutils package to enable the web tools you want to use.



2.5 Software Description

The source code for this routine is located at /project/Titan_component_netutils.

Description of netutils package file structure

Here is a breakdown and summary of the RT-Thread netutils package’ s functionality:

Name	Classification	Function Introduction
Ping	Debug tests	Using the “ping” command can check whether the network is connected, can be a good help us to analyze and determine the network failure
NTP	Time synchronization	Network time protocol
TFTP	File transfer	TFTP is a simple protocol for transferring files that is even lighter than FTP
Iperf	Performance tests	Maximum TCP and UDP bandwidth performance is tested and bandwidth, delay jitter and packet loss can be reported
NetIO	Performance tests	A tool for testing the throughput of a network
Telnet	Remote access	You can remotely log into the Finsh/MSH Shell of RT-Thread
tcpdump	Network debugging	Tcpdump is a network packet capture tool for RT-Thread based on lwIP

The file structure of the netutils software package is as follows:

```
netutils                 // netutils folder
├── iperf                 // iperf Network Performance Test
├── netio                 // Netio Network Traffic Measurement Test
├── ntp                  // NTP Time Synchronization Function
├── ping                  // Ping Function
├── tcpdump          // Network Capture Tools
├── telnet                  // Telnet server
├── tftp                 // TFTP Function
├── tools                 // Network Testing Tools
```

2.6 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio’s package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board’sUSB-DBG interface to the PC and download the firmware to the development board.

2.7 Run Effect

Mount the file system

Since the TFTP function requires the support of the file system when transferring files within the system, the system will initialize the file system-related functions during its initialization process. The normal initialization prompt message for the file system is as follows:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Sep  1 2025 14:02:13
 2006 - 2024 Copyright by RT-Thread team
lwIP-2.0.3 initialized!
[I/sal.skt] Socket Abstraction Layer initialize success.
Hello RT-Thread!
============================================================
This example project is an driver wifi routine(hyperflash)!
============================================================
[I/drv.ospi_flash] Get flash id: 0x021a5bef
[I/drv.ospi_flash] OSPI flash initialized successfully
[D/FAL] Flash device |               ospi_flash | addr: 0x90000000 | len:
  0x04000000 | blk_size: 0x00001000 |initialized finish.
[I/FAL] ==================== FAL partition table ====================
[I/FAL] | name         | flash_dev  |   offset   |    length  |
[I/FAL] -------------------------------------------------------------
[I/FAL] | whd_firmware | ospi_flash | 0x00000000 | 0x00080000 |
[I/FAL] | whd_clm      | ospi_flash | 0x00080000 | 0x00080000 |
[I/FAL] | download     | ospi_flash | 0x00100000 | 0x00200000 |
[I/FAL] | easyflash    | ospi_flash | 0x00300000 | 0x00100000 |
[I/FAL] | filesystem   | ospi_flash | 0x00400000 | 0x00c00000 |
[I/FAL] =============================================================
[I/FAL] RT-Thread Flash Abstraction Layer initialize success.
[I/FAL] The FAL MTD NOR device (filesystem) created successfully
[I/wifi] Filesystem initialized!
msh />WLAN MAC Address : 2C:B0:FD:A2:2D:60
WLAN Firmware    : wl0: Mar 28 2021 22:55:55 version 7.45.98.117 (dc5d9c4 CY)
  FWID 01-d36e8386
WLAN CLM         : API: 12.2 Data: 9.10.39 Compiler: 1.29.4 ClmImport: 1.36.3
  Creation: 2021-03-28 22:47:33
WHD VERSION      : 3.1.0.23284 : v3.1.0 : GCC 13.2 : 2024-03-21 22:57:11 +0800
```

(continues on next page)

(continued from previous page)

```
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w0
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w1
```

Connect to Wi-Fi

After the program runs, it will enter the MSH command line and wait for the user to configure the device to connect to the network. Use the MSH command "wifi join ssid password" to configure the network, as shown below:

```
msh />wifi join ssid_test password
join ssid:ssid_test
[I/WLAN.mgnt] wifi connect success ssid:ssid_test
msh />[I/WLAN.lwip] Got IP address : 192.168.173.8
```

Ping tool

Ping is a network diagnostic tool used to test whether data packets can reach a specific host via the IP protocol. It estimates the packet loss rate (packet loss ratio) between the host and the device and the round-trip time of the packets.

- Ping domain

```
msh />ping www.rt-thread.org
ping: not found specified netif, using default netdev w0.
60 bytes from 120.222.223.251 icmp_seq=1 ttl=48 time=76 ms
60 bytes from 120.222.223.251 icmp_seq=2 ttl=48 time=54 ms
60 bytes from 120.222.223.251 icmp_seq=3 ttl=48 time=74 ms
60 bytes from 120.222.223.251 icmp_seq=4 ttl=48 time=74 ms

--- 120.222.223.251 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
minimum = 54ms, maximum = 76ms, average = 69ms
msh />
```

- Ping IP

```
msh />ping 192.168.173.247
ping: not found specified netif, using default netdev w0.
60 bytes from 192.168.173.247 icmp_seq=1 ttl=64 time=42 ms
60 bytes from 192.168.173.247 icmp_seq=2 ttl=64 time=44 ms
60 bytes from 192.168.173.247 icmp_seq=3 ttl=64 time=24 ms
60 bytes from 192.168.173.247 icmp_seq=4 ttl=64 time=24 ms
```

(continues on next page)

(continued from previous page)

```
--- 192.168.173.247 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
minimum = 24ms, maximum = 44ms, average = 33ms
msh />
```

NTP tool

NTP is the Network Time Protocol, which is a protocol used to synchronize the times of various computers in a network. The netutils software package implements the NTP client. After connecting to the network, one can obtain the current UTC time and update it to the RTC.

The effect of the MSH command is as follows:

```
msh />ntp_sync
[I/ntp] Get local time from NTP server: Mon Sep  1 15:15:35 2025
msh />
```

TFTP tool

TFTP (**Trivial File Transfer Protocol**, Simple File Transfer Protocol) is one of the protocols within the TCP/IP suite, used for transferring files between a client and a server. A simple protocol for file transfer between the server and the client, providing a straightforward and low-cost file transfer service. The port number is 69, which is different from the traditional one. The FTP protocol is much more lightweight and is suitable for use in small embedded products. The preparation for the TFTP tool requires the following two steps:

- The installation file for the TFTP client is located at **packages/ netutils/ tools/ Tftpd64-4.60-setup.exe**. Before using TFTP, please install this software first.
- To start the TFTP server before transferring files, you need to use the Finsh/MSH commands on RT-Thread to initiate the TFTP server. The general effect is as follows:

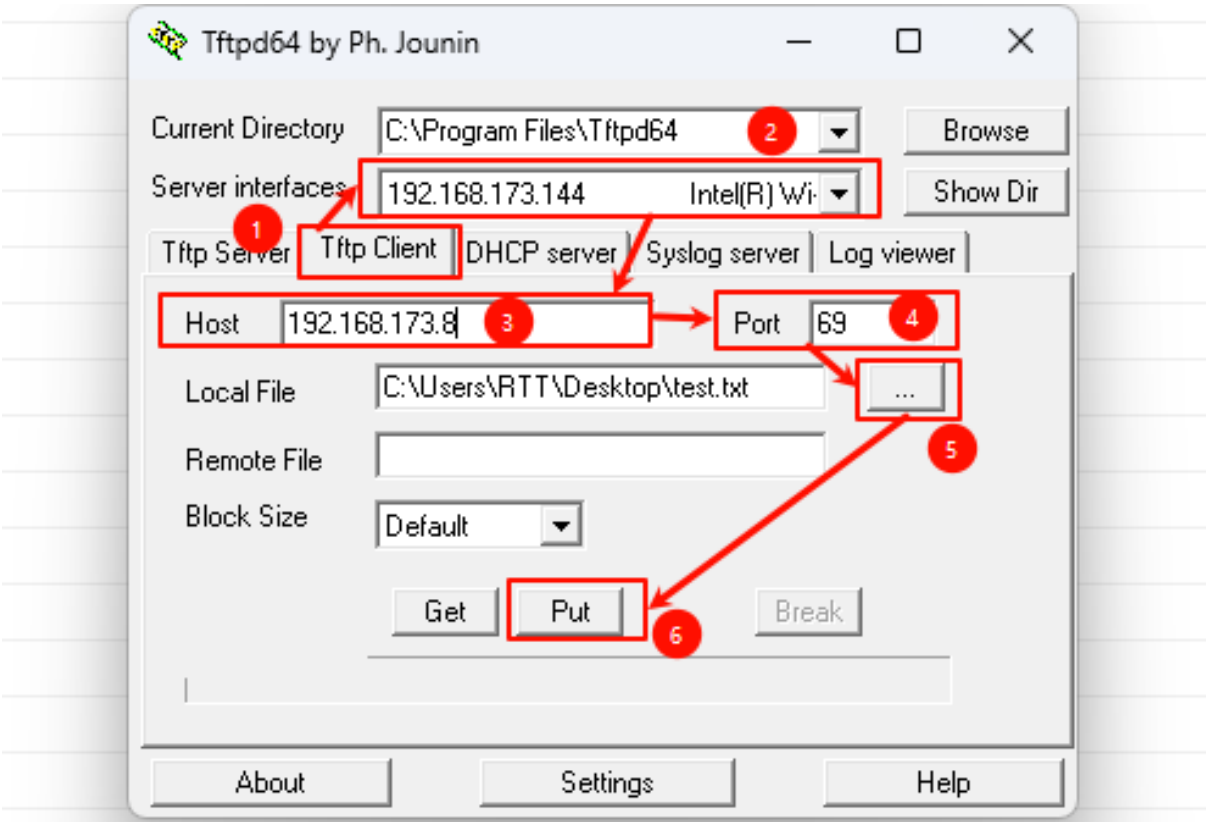
```
msh />tftp -s
tftp server start!
msh />
```

- Connect to the RT-Thread operating system and send files

1. Open the newly installed Tftpd64 software and follow the steps below to configure it: 选择 Tftp Client ;
2. In the “Server interfaces” dropdown box, make sure to select the network card that is in the same network segment as RT-Thread.

3. Enter the IP address of the TFTP server. You can view it using the ifconfig command in the MSH of RT-Thread.
4. Enter the port number of the TFTP server. The default is 69.
5. Select the file to be sent. "Remote File" is the path (including the file name) where the file is saved on the server side. The options support both relative and absolute paths. Since RT-Thread by default enables the DFS_USING_WORKDIR option, at this time the relative path is based on the current directory entered by MSH. Therefore, when using a relative path, be sure to switch the directory in advance.
6. Just click the "Put" button.

Note: If DFS_USING_WORKDIR is not enabled and the Remote File is empty, the file will be saved in the root directory.



View the files sent by PC in MSH:

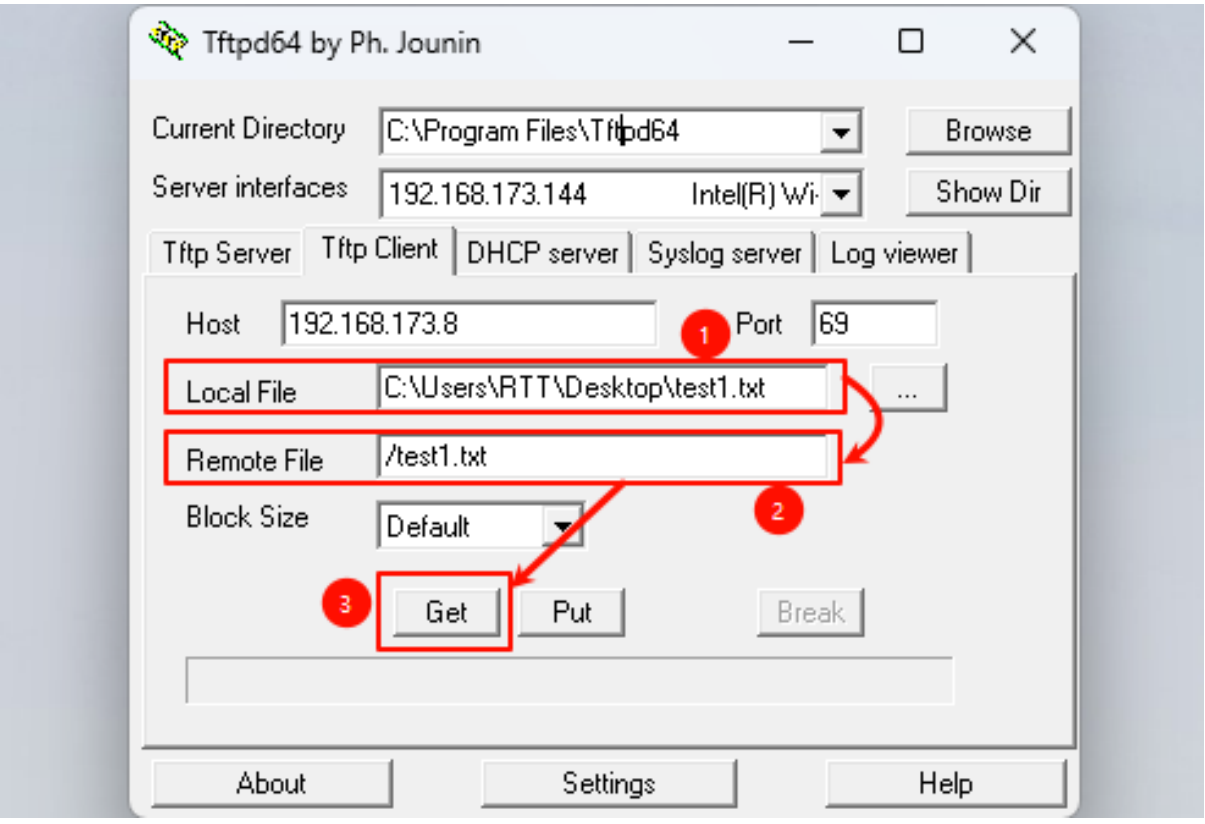
```
msh />ls
Directory /:
test.txt            16
msh />cat test.txt
Hello RT-Thread!
msh />
```

- Retrieve the file from RT-Thread

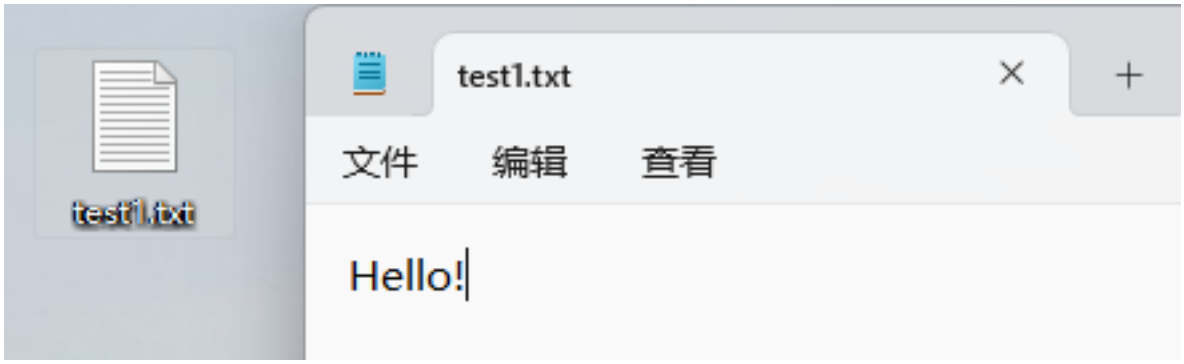
First, create a text file in the root directory and enter the content “Hello!”。

```
msh />echo "Hello!" test1.txt
msh />cat test1.txt
Hello!
msh />
```

1. In the Tftpd64 software, enter the path (including the file name) of the file to be received and saved.
2. Remote File is the file path (including the file name) that the server is waiting to receive back. The option supports both relative and absolute paths. Since RT-Thread by default enables the DFS_USING_WORKDIR option, at this time, the relative path is based on the current directory entered by Finsh/MSH. Therefore, when using a relative path, be sure to switch the directory in advance.
3. Just click the “Get” button. As shown below, save “/test1.txt” to the local disk. Here, an absolute path is used.



The following picture is the file obtained from RT-Thread:

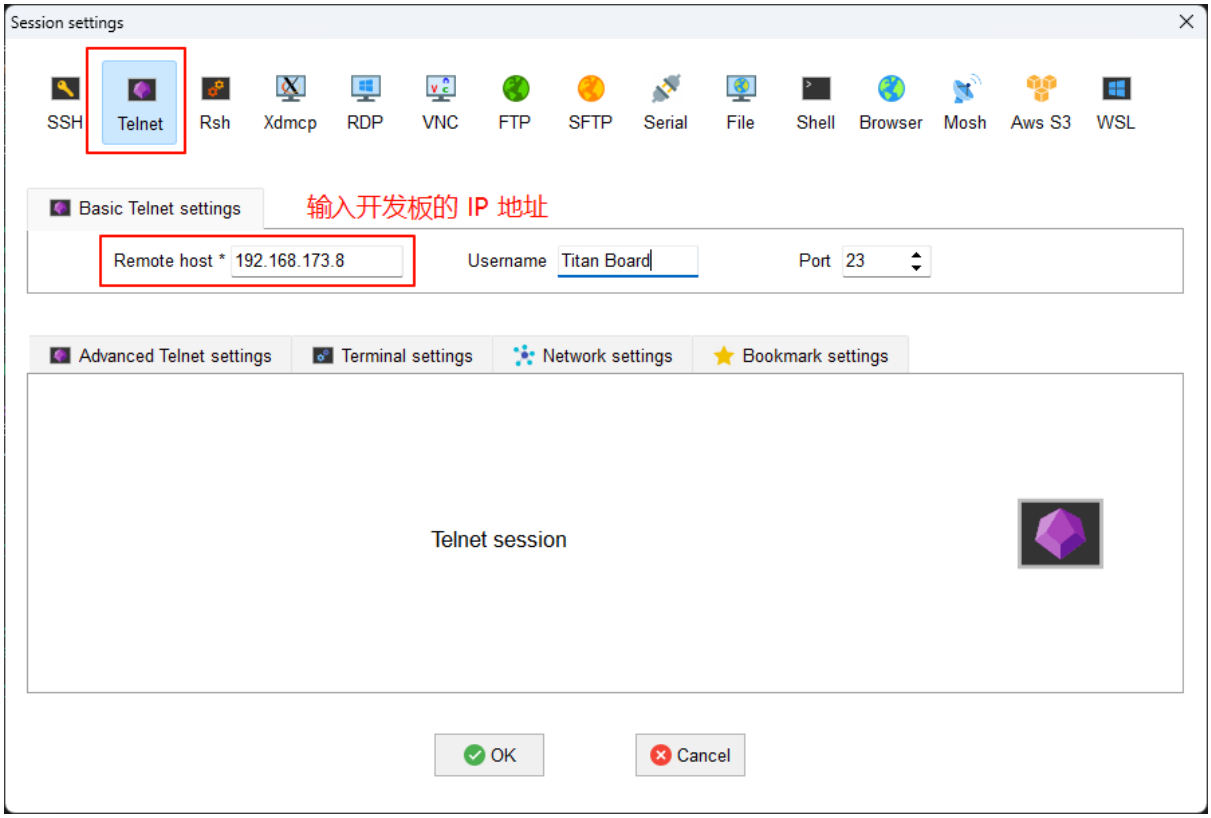


Telnet tool

Telnet is a TCP-based remote terminal protocol used for remotely logging into the command line interface of devices over a network. Through Telnet, one can remotely access the command line of the development board from a PC, enabling debugging, monitoring, and control. To use the Finsh/MSH commands to start the telnet service on RT-Thread, the general effect is as follows:

```
msh />telnet_server
Telnet server start successfully
msh />telnet: waiting for connection
```

Next, use the remote connection tool to connect to the development board.



After the connection is successful, you can remotely control the development board.

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Sep  1 2025 14:02:13
 2006 - 2024 Copyright by RT-Thread team
msh />
msh />
msh />list device
device               type         ref count
------------ -------------------- ----------
telnet       Character Device     2
w1           Network Interface    1
w0           Network Interface    1
wlan0        Network Interface    1
wlan1        Network Interface    1
filesystem   MTD Device           1
rtc          RTC                  0
uart8        Character Device     0
pin          Pin Device           0
msh />
```

More online tools

In addition to the commonly used network tools mentioned above, the netutils software package also provides some useful network tools for development and debugging, such as the NetIO tool, Iperf tool and tcpdump tool. The usage methods of these tools can be referred to the instruction files in the software package’s function directory.

Multimedia Display

1. CEU Camera Usage Instructions

English|Chinese

1.1 Introduction

This example demonstrates how to use the **CEU (Camera Engine Unit)** interface on the **Titan Board** to connect an **OV5640 camera**, and display the captured images on an **RGB565 LCD screen** via the **RT-Thread LCD framework**.

Key functionalities include:

- Initialize the CEU camera interface to capture real-time video streams
- Configure OV5640 camera parameters (resolution, frame rate, output format)
- Display captured images using the RT-Thread LCD driver
- Support image format conversion (YUV422 → RGB565)

1.2 RA8 Series CEU (Camera Engine Unit) Features

The RA8 series MCU integrates a **CEU hardware module** for efficient capture of camera image data, supporting multiple image formats and resolutions, and enabling direct transfer to memory or display interfaces.

CEU Hardware Interface Features

1. **Interface Types**
    - **DVP (Digital Video Port)** interface for connecting CMOS cameras
    - Supports 8/10/12-bit data bus
    - Synchronization signals:
        - **PCLK**: Pixel clock
        - **HSYNC**: Line synchronization
        - **VSYNC**: Frame synchronization
2. **Input Resolution and Frame Rate**
    - Supports VGA, QVGA, SXGA, UXGA, and other common resolutions
    - Frame rates from 1–60 fps, configurable for different applications
3. **Camera Compatibility**
    - Compatible with common CMOS cameras such as OV5640, OV7670
    - Supports auto-initialization and register configuration

Image Formats and Processing Capabilities

1. **Supported Image Formats**
    - **YUV422** (commonly used for video transfer)
    - **RGB565** (suitable for LCD display)
    - **RAW10/RAW12** (for image processing and algorithm development)
2. **Image Processing Features**
    - **Color space conversion**: YUV RGB
    - **Image cropping**: capture only a specific ROI (Region of Interest)
    - **Image scaling**: support proportional scaling up or down
    - **Mirror and flip**: horizontal or vertical mirroring
3. **Hardware Acceleration**
    - CEU contains a hardware processing unit to reduce CPU load
    - Provides fast image format conversion and scaling

DMA Support and Buffer Mechanism

1. **High-Speed DMA Transfer**
    - Works with MCU DMAC for fast memory writes
    - Supports direct writing to frame buffer or LCD buffer
2. **Multi-Buffer Mechanism**
    - Supports double buffering or ring buffers for continuous video capture
    - Reduces frame loss and display latency
3. **Flexible DMA Configuration**
    - Configurable buffer start address and size
    - Supports interrupt triggering and callbacks

Interrupt Mechanism

1. **Interrupt Types**
    - **Frame End Interrupt**: triggered when each frame capture completes
    - **Line End Interrupt (optional)**: triggered at the end of each line capture
    - **Error Interrupt**: including buffer overflow or sync signal anomalies
2. **Interrupt Features**
    - Supports RT-Thread ISR callback registration
    - Can work with DMA for real-time processing and display

Timing and Synchronization Features

1. **Line/Frame Synchronization**
    - HSYNC aligns each line of data
    - VSYNC aligns each frame
2. **Pixel Clock**
    - CEU supports external PCLK or internal clock division
    - Ensures synchronization with camera output to avoid sampling errors
3. **Data Alignment**
    - Supports byte or pixel alignment
    - Automatically adjusts according to image format

Performance and Optimization

1. **High Throughput**
    - DMA + double buffering enables continuous video capture
    - Low CPU usage suitable for real-time applications
2. **Reliability**
    - Sync signal anomalies trigger interrupts
    - Buffer overflow detection
    - Automatic recovery from frame loss
3. **Flexibility**
    - Supports multiple resolutions and format switching
    - Configurable cropping and scaling regions to improve display efficiency

Application Scenarios

- Real-time video display on LCD
- Video capture and image processing algorithm testing
- Embedded vision applications, e.g., surveillance, gesture recognition, robotics vision

1.3 RA8 Series MCU GLCDC (Graphics LCD Controller) Features

The RA8 series MCU (e.g., RA8P1) integrates a **GLCDC hardware module** for driving TFT/LCD displays, enabling high-speed graphics rendering and video display, supporting multiple resolutions, color formats, and display modes.

Hardware Features

1. **Resolution Support**
    - Can drive common resolutions from QVGA (320×240) to WQVGA/XGA
    - Limited by on-chip RAM and display interface bandwidth
2. **Color Support**
    - Supports 1/4/8/16/24/32-bit color depth
    - Common formats: RGB565, RGB888
    - Supports palette mode (CLUT)
    - Hardware color conversion available
3. **Interface Types**
    - Parallel RGB (TFT LCD interface)
    - Supports 16/18/24-bit data bus
    - Direct connection to external LCD panels
    - Programmable timing: HSYNC, VSYNC, DE, PCLK, RGB output

Layers and Display Modes

1. **Layer Support**
    - Single-layer mode (single screen display)
    - Multi-layer mode (palette or hardware alpha blending)
    - Supports transparent/semi-transparent overlay
2. **Display Modes**
    - RGB mode (direct color output)
    - CLUT/Palette mode (indexed color via lookup table)
    - Configurable scan direction (horizontal/vertical)

DMA and Frame Buffer

1. **Frame Buffer Access**
    - GLCDC can directly access on-chip or external SRAM frame buffer
    - Supports single or double buffering
    - Supports ring buffer for continuous refresh
2. **DMA Support**
    - Works with MCU DMAC to reduce CPU usage
    - Can transfer images directly from memory to LCD
    - Supports line, block, or full-frame transfer

Hardware Graphics Functions

1. **Window Cropping and Scaling**
    - Can specify display window area
    - Supports simple horizontal/vertical scaling
2. **Hardware Graphics Acceleration**
    - Rectangle fill, color replacement
    - Image transparency processing
    - Can be combined with CEU or DMA for video display
3. **Color Format Conversion**
    - YCbCr $\rightarrow$ RGB
    - RGB888 $\rightarrow$ RGB565
    - Hardware accelerated to reduce CPU load

Interrupt Mechanism

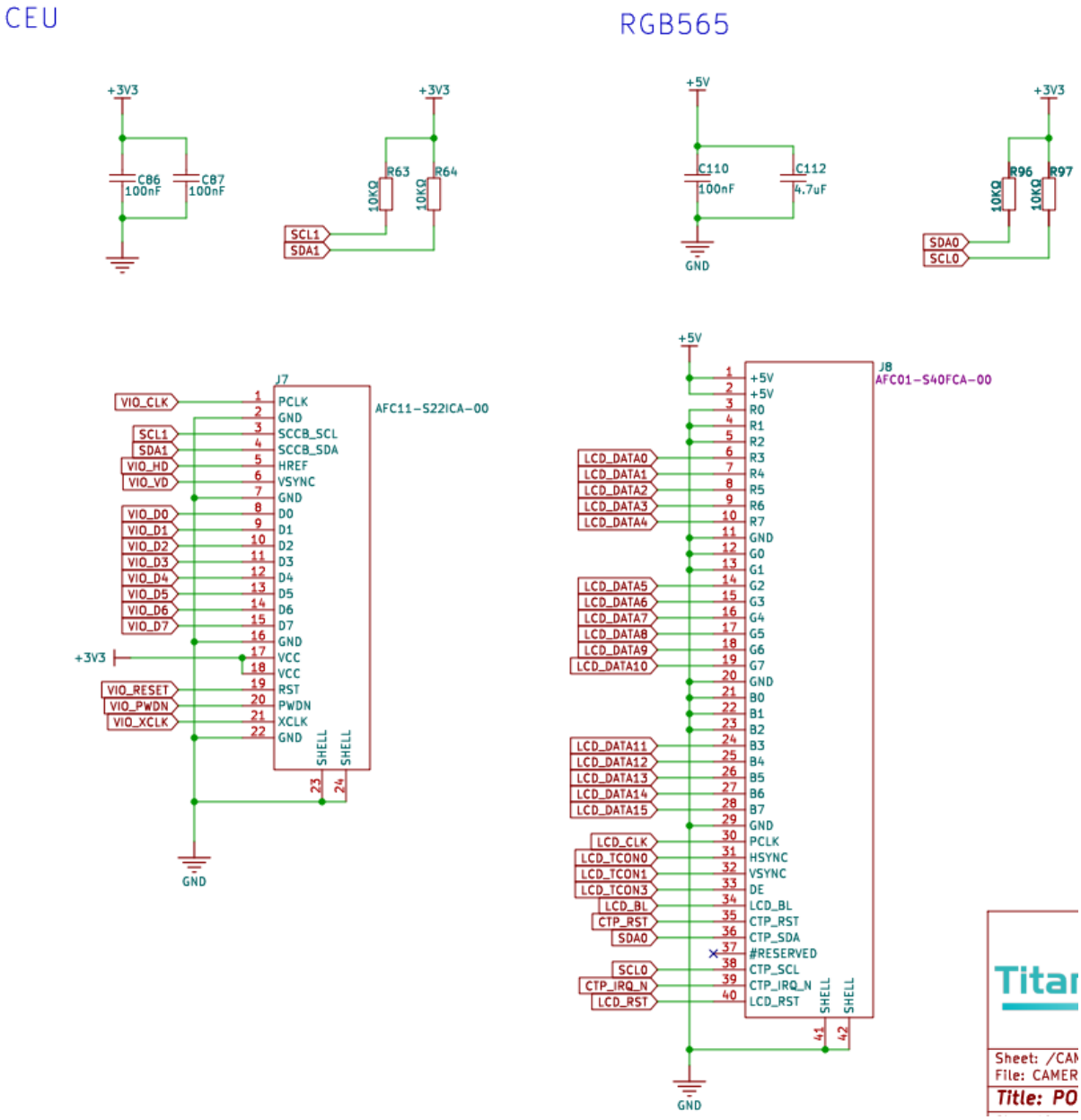
1. **Interrupt Types**
    - Frame End
    - Line End (optional)
    - Access error/overflow
2. **Interrupt Application**
    - Integrates with RT-Thread ISR
    - Can trigger buffer update or double-buffer swap on frame completion
    - Enables smooth animations and video display

Performance Optimization

1. **Double Buffer Mechanism**
    - Reduces flicker
    - CPU can render next frame in background
    - GLCDC hardware automatically switches display buffer
2. **Frame Rate Control**
    - Programmable clock and line/frame synchronization
    - Supports common refresh rates (30fps, 60fps)
3. **CPU Offload**
    - Many graphics operations performed in hardware
    - DMA + GLCDC combination enables efficient image display

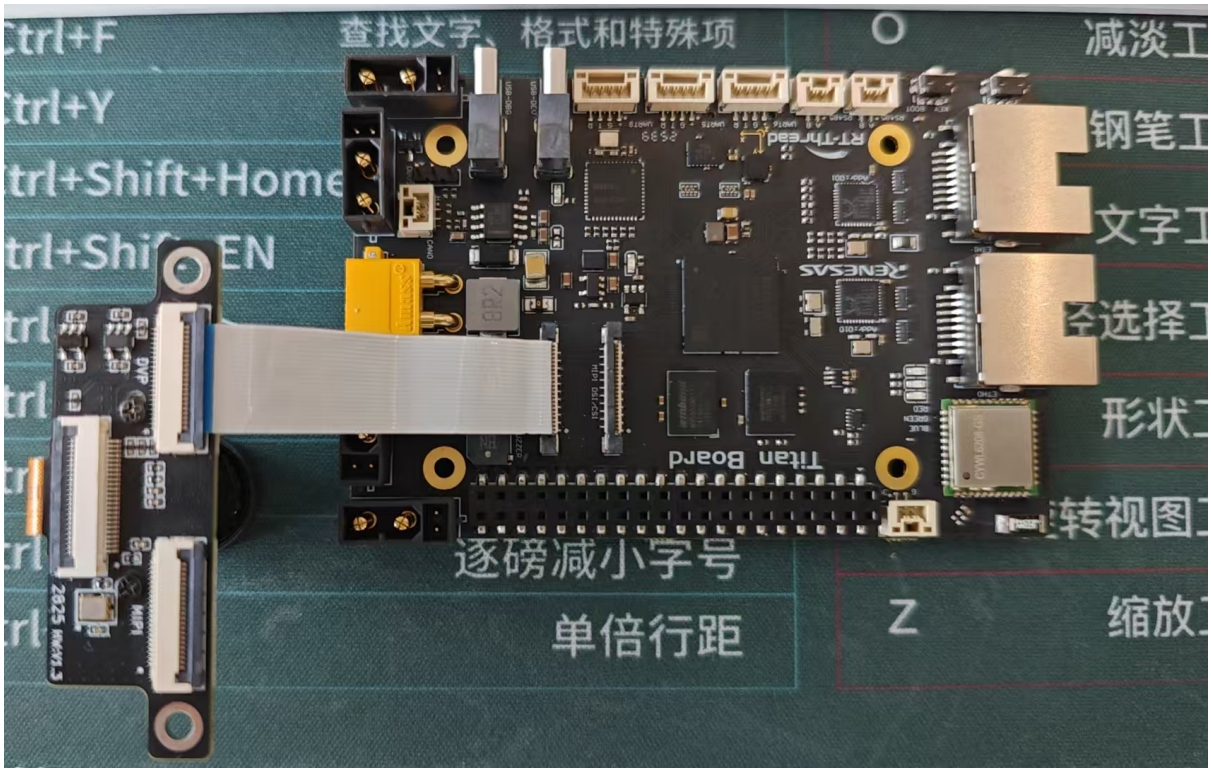
1.4 Hardware Description

- The CEU camera interface and RGB LCD interface are shown in the following figure:



- The connection method of the CEU camera is as follows:

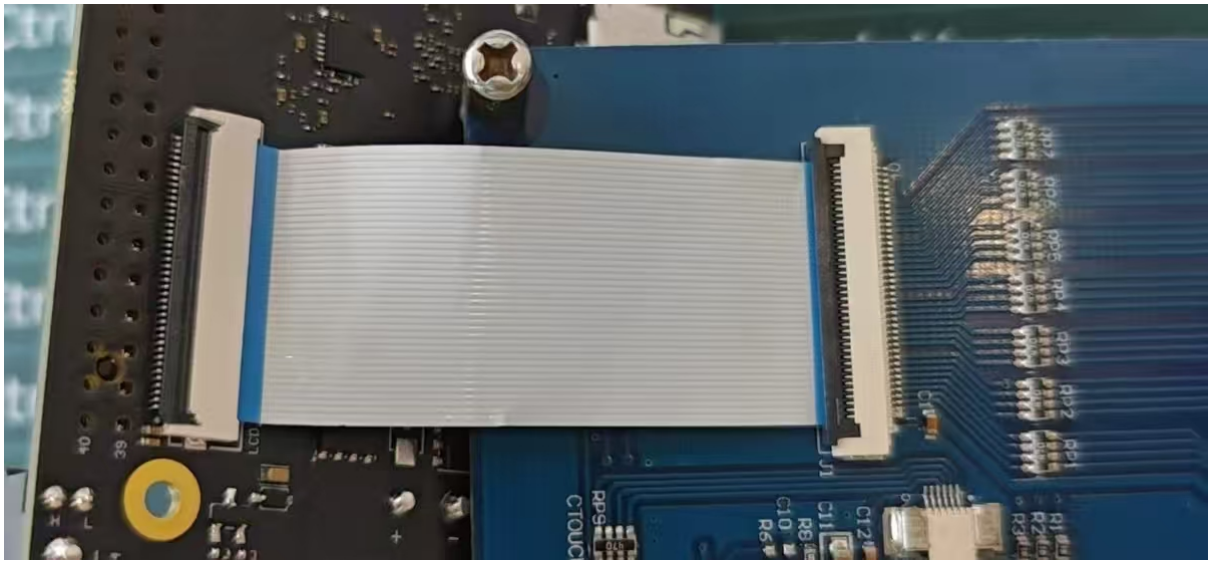
Use a **22-pin reverse FFC cable** to connect the development board’ s **CEU_CAM** connector to the camera adapter board’ s **DVP** connector.



- The connection method of RGB LCD is as follows:

Display model: **ALIENTEK 4.3″ RGB LCD 800×480**

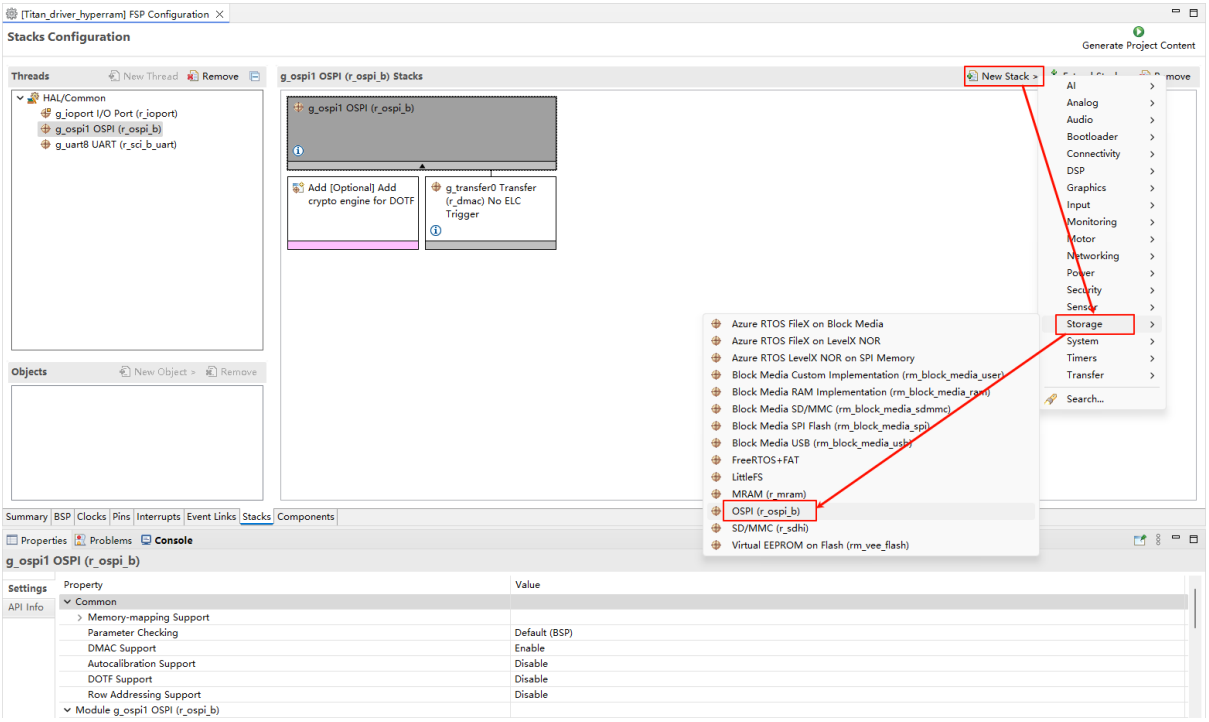
Use a **40-pin same-direction FFC cable** to connect the development board ' s **LCD_RGB** connector to the display.



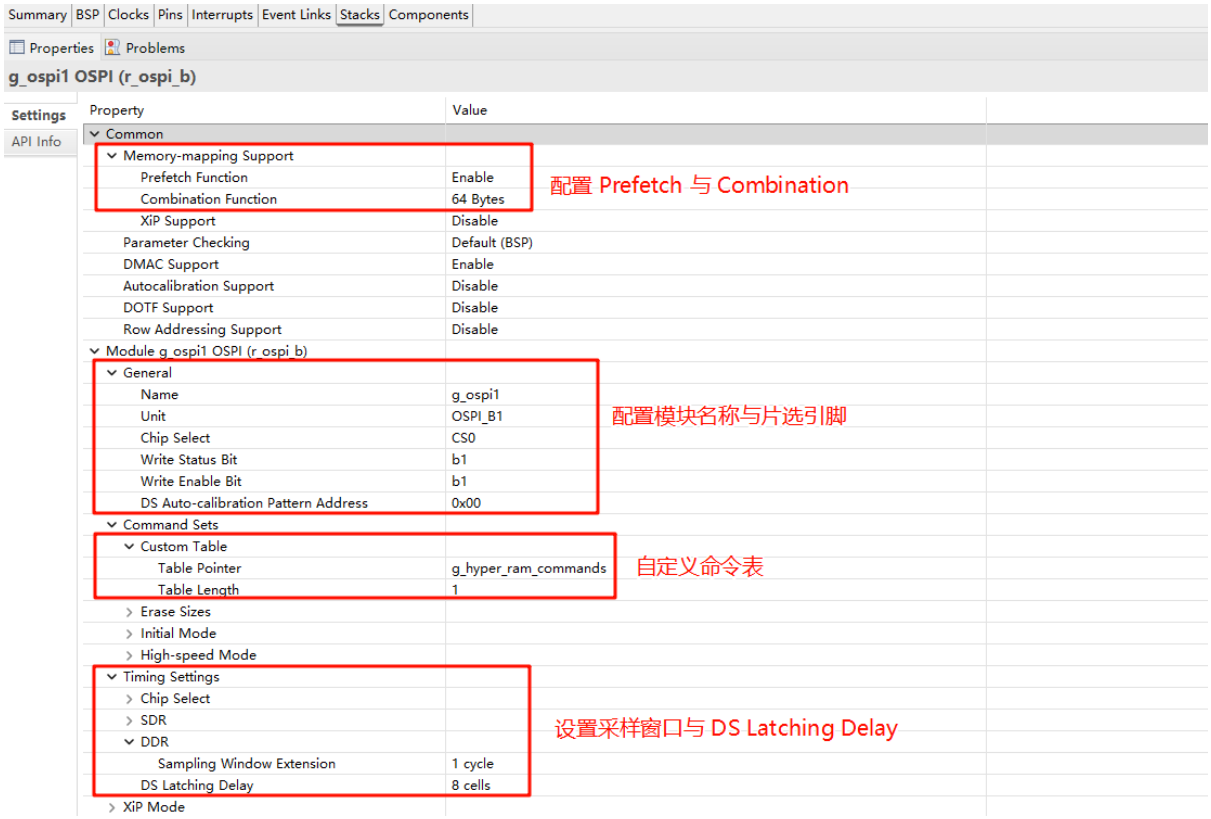
1.5 FSP Configuration

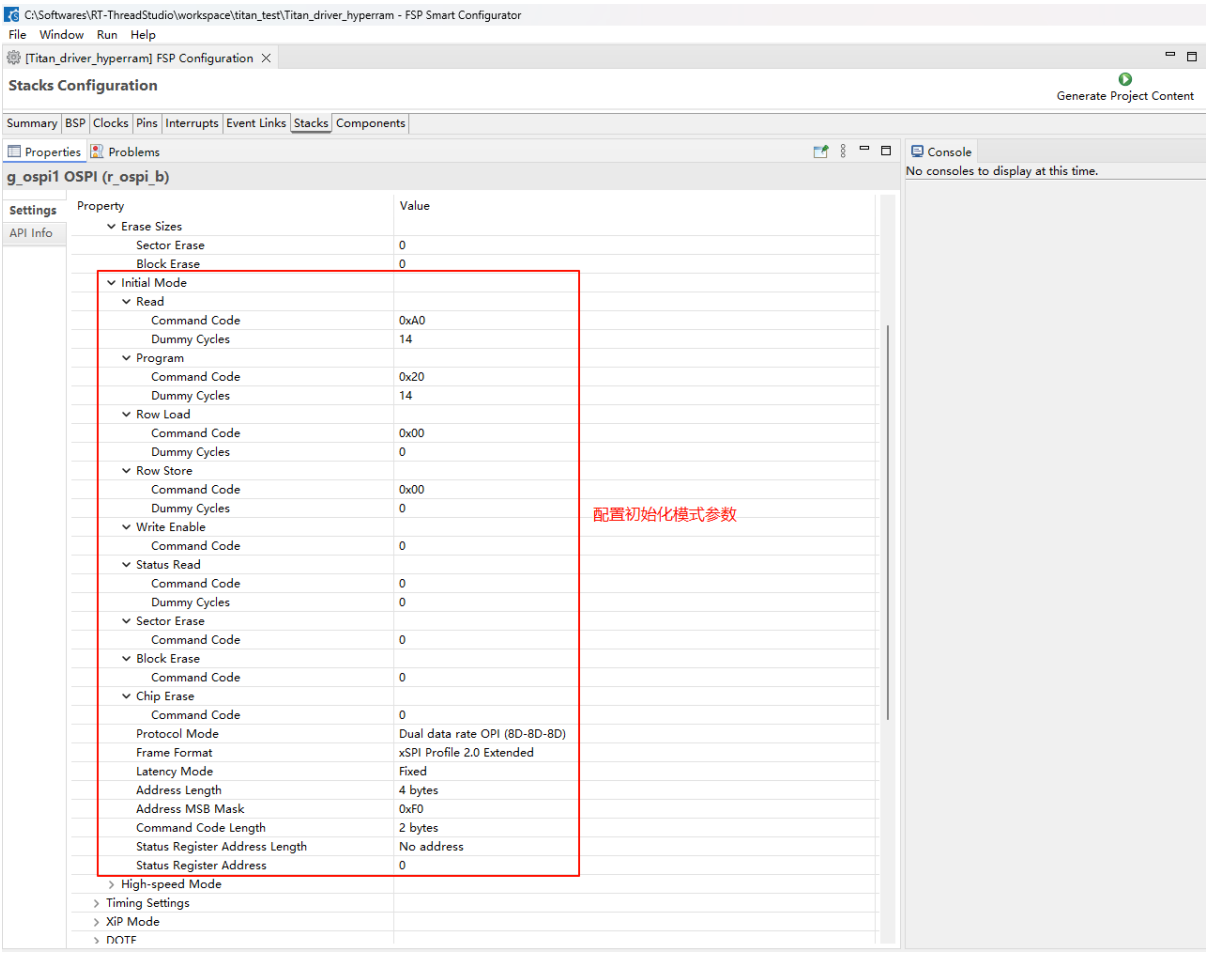
HyperRAM Configuration

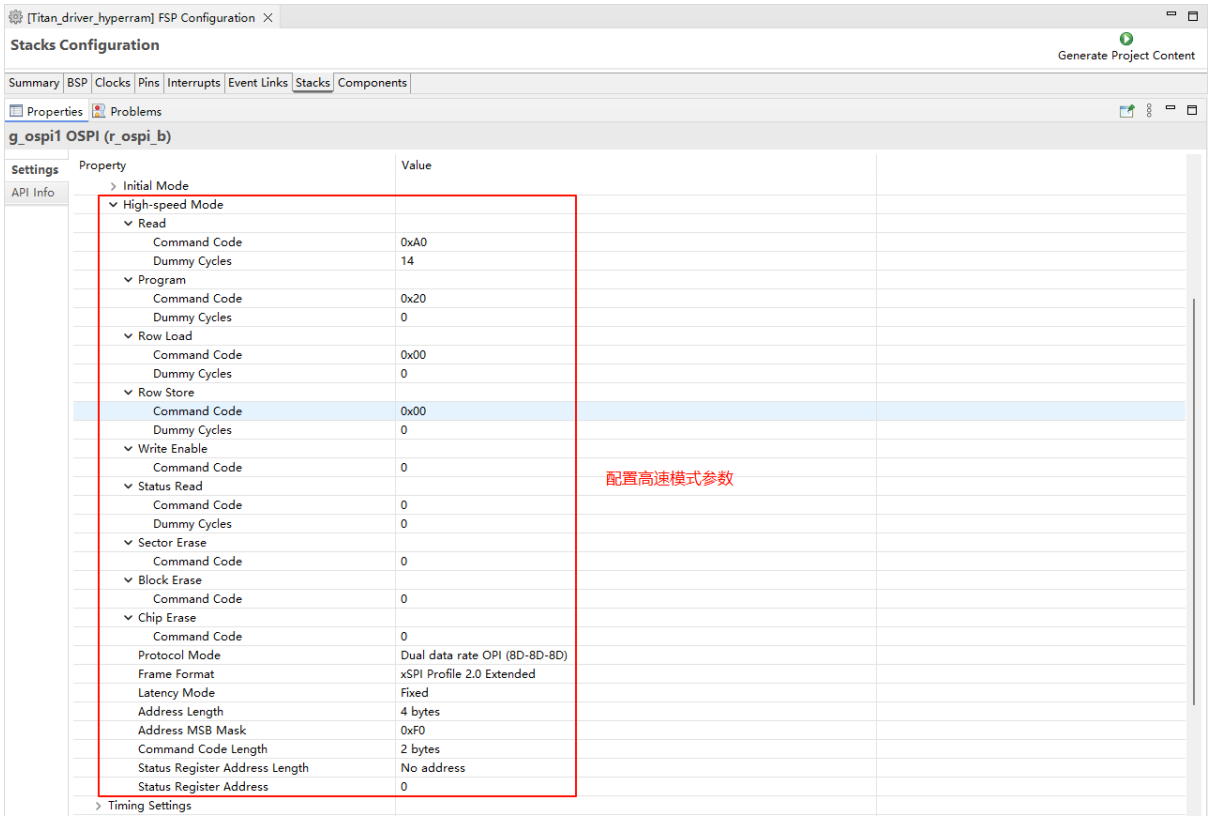
- Create a r_ospi_b stack:



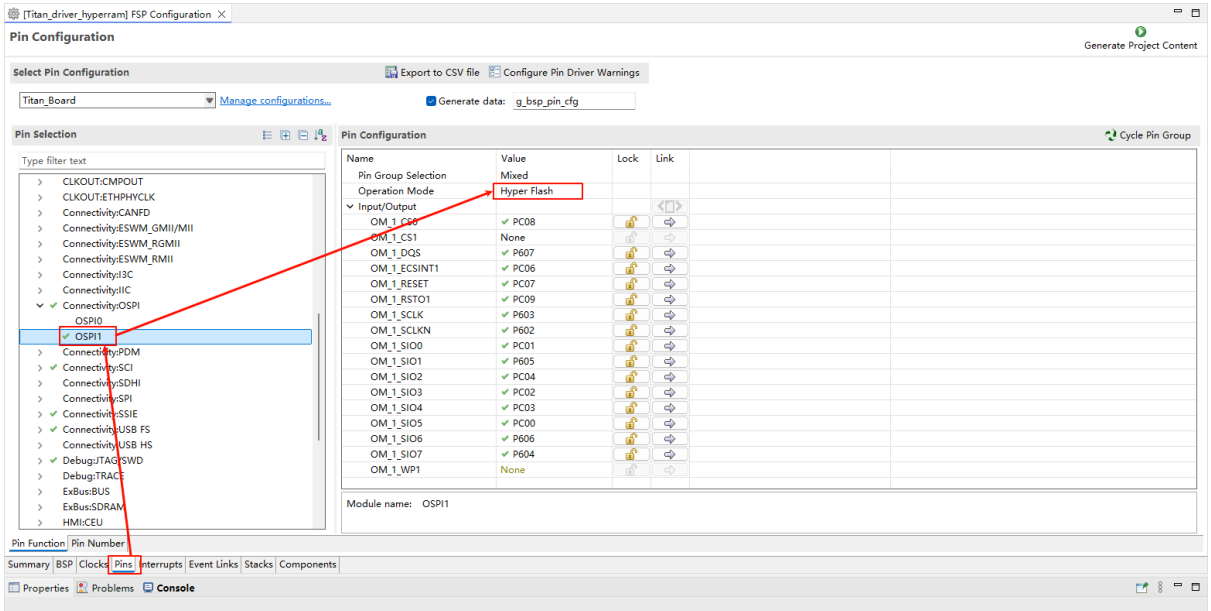
- Configure the r_ospi_b stack:


Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems
g_ospi1 OSPI (r_ospi_b)
Settings
API Info
Property
Value
Common
Memory-mapping Support
Prefetch Function
Enable
Combination Function
64 Bytes
XiP Support
Disable
Parameter Checking
Default (BSP)
DMAC Support
Enable
Autocalibration Support
Disable
DOTF Support
Disable
Row Addressing Support
Disable
Module g_ospi1 OSPI (r_ospi_b)
General
Name
g_ospi1
Unit
OSPI_B1
Chip Select
CS0
Write Status Bit
b1
Write Enable Bit
b1
DS Auto-calibration Pattern Address
0x00
Command Sets
Custom Table
Table Pointer
g_hyper_ram_commands
Table Length
1
Erase Sizes
Initial Mode
High-speed Mode
Timing Settings
Chip Select
SDR
DDR
Sampling Window Extension
1 cycle
DS Latching Delay
8 cells
XiP Mode
配置 Prefetch 与 Combination
配置模块名称与片选引脚
自定义命令表
设置采样窗口与 DS Latching Delay

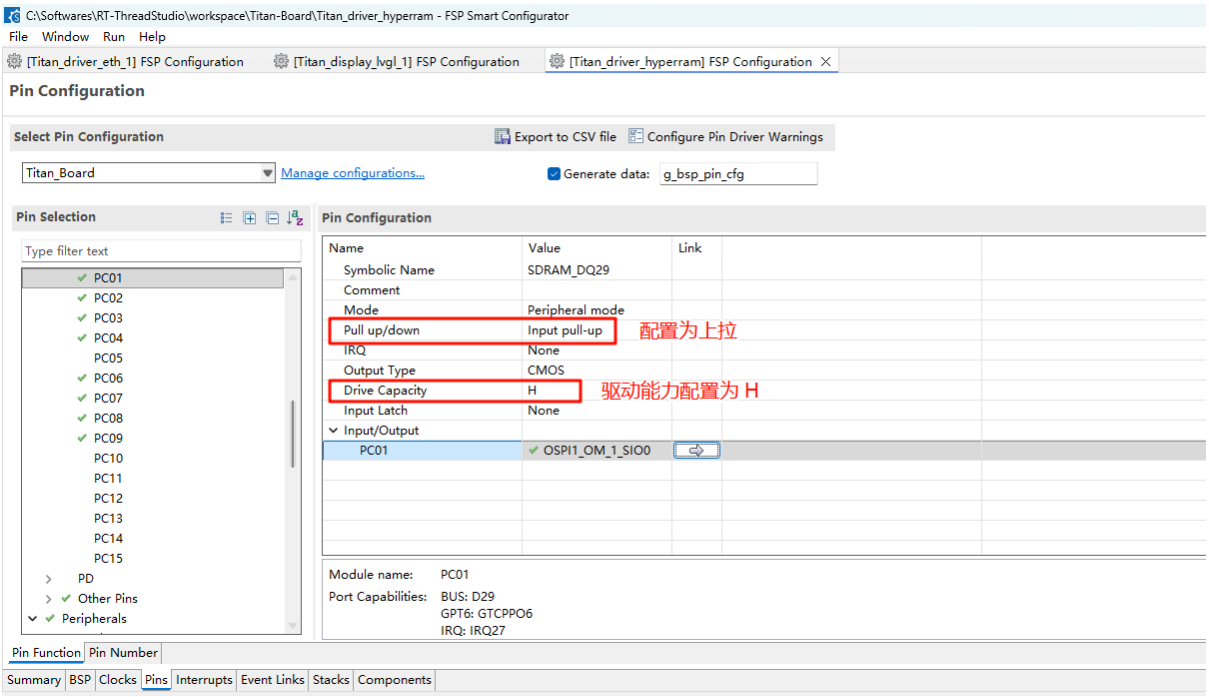

C:\Softwares\RT-ThreadStudio\workspace\titan_test\Titan_driver_hyperram - FSP Smart Configurator
File Window Run Help
[Titan_driver_hyperram] FSP Configuration
Stacks Configuration
Generate Project Content
Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems
g_ospi1 OSPI (r_ospi_b)
Settings
API Info
Property
Value
Erase Sizes
Sector Erase 0
Block Erase 0
Initial Mode
Read
Command Code 0xA0
Dummy Cycles 14
Program
Command Code 0x20
Dummy Cycles 14
Row Load
Command Code 0x00
Dummy Cycles 0
Row Store
Command Code 0x00
Dummy Cycles 0
Write Enable
Command Code 0
Status Read
Command Code 0
Dummy Cycles 0
Sector Erase
Command Code 0
Block Erase
Command Code 0
Chip Erase
Command Code 0
Protocol Mode Dual data rate OPI (8D-8D-8D)
Frame Format xSPI Profile 2.0 Extended
Latency Mode Fixed
Address Length 4 bytes
Address MSB Mask 0xF0
Command Code Length 2 bytes
Status Register Address Length No address
Status Register Address 0
High-speed Mode
Timing Settings
XiP Mode
DOTF
Console
No consoles to display at this time.
配置初始化模式参数



- HyperRAM pin configuration:

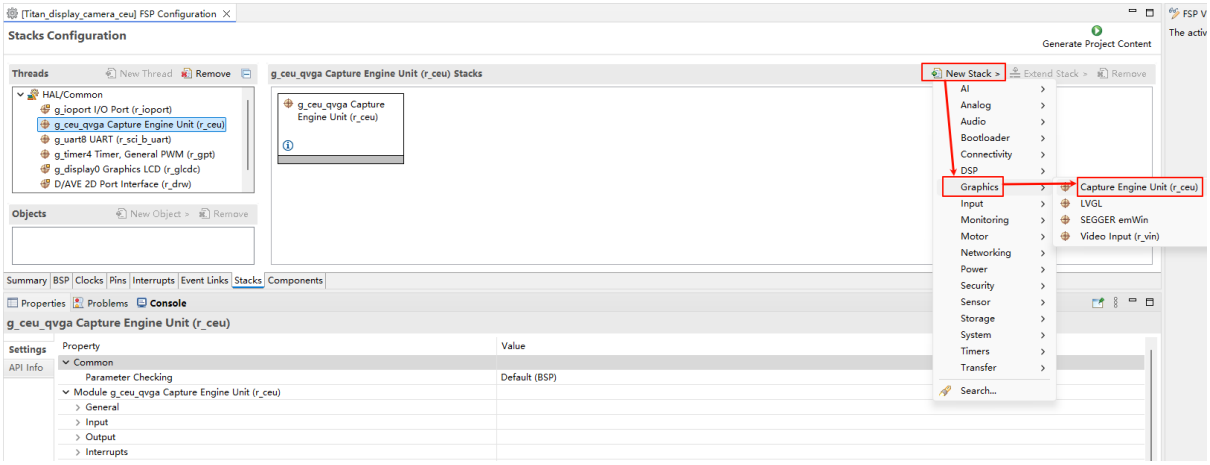


- The drive capability of all pins related to HyperRAM should be configured as H, and OM_1_SIO0 to OM_1_SIO7 need to be configured as Input pull-up.

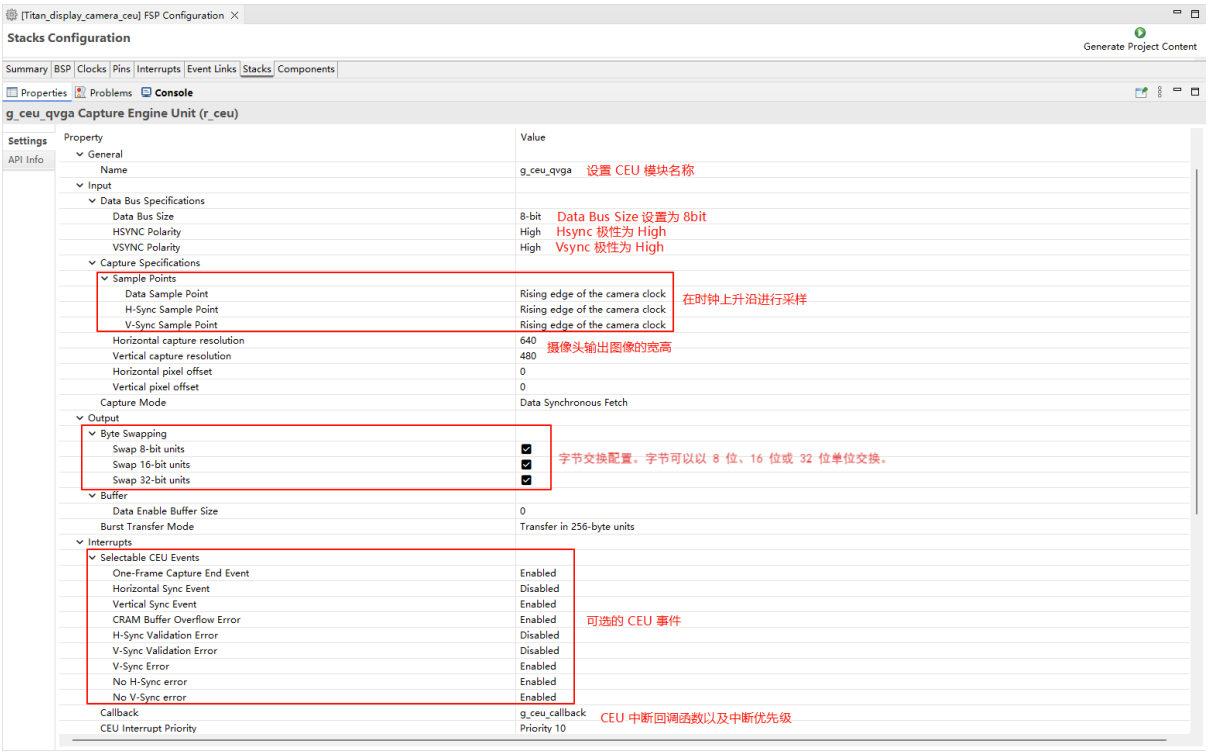


CEU Configuration

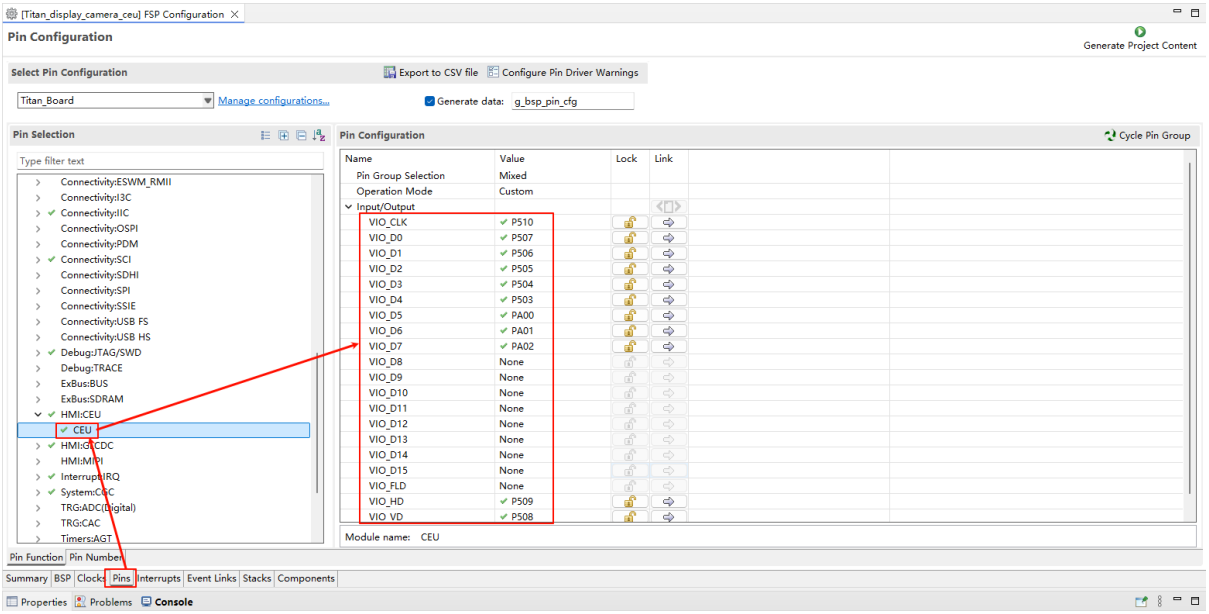
- Create a r_ceu stack：



- Configure CEU：

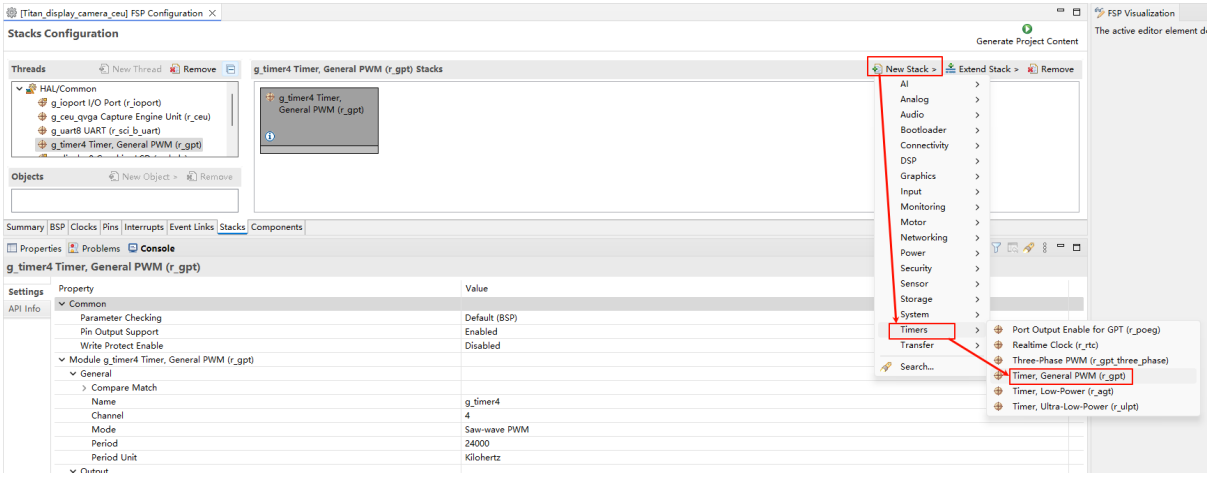


- Configure CEU pins：

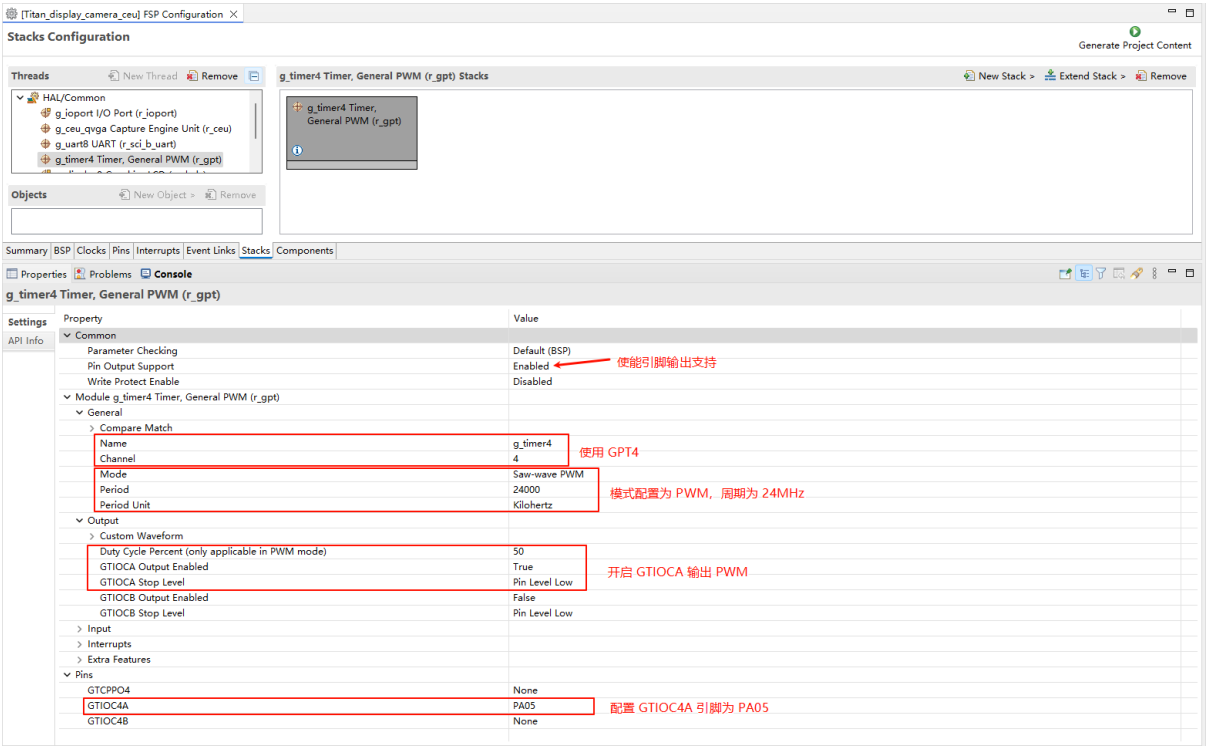


CEU Clock Configuration

- Create a r_gpt stack：



- Configure the camera clock PWM output:

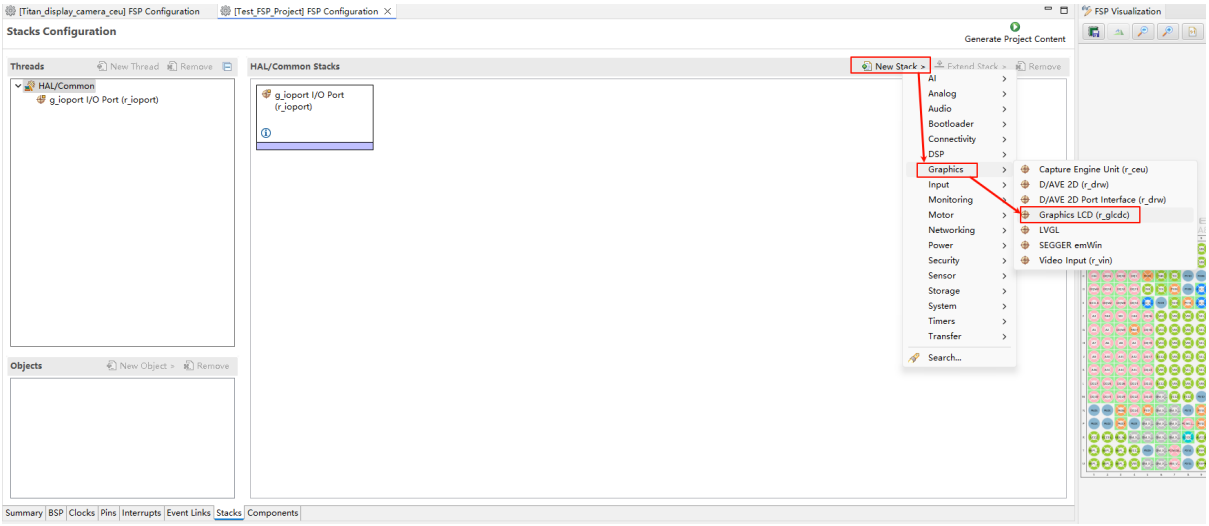


D/AVE 2D Configuration

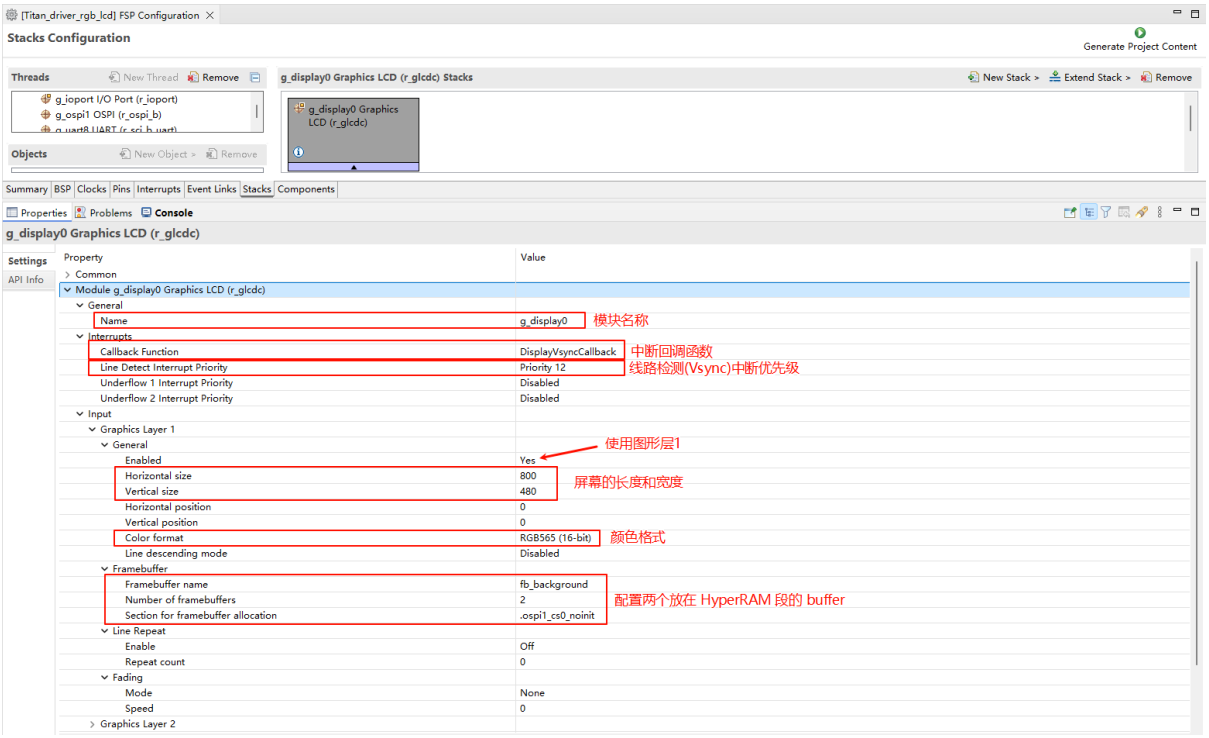
- Create a r_drw stack:

RGB LCD Configuration

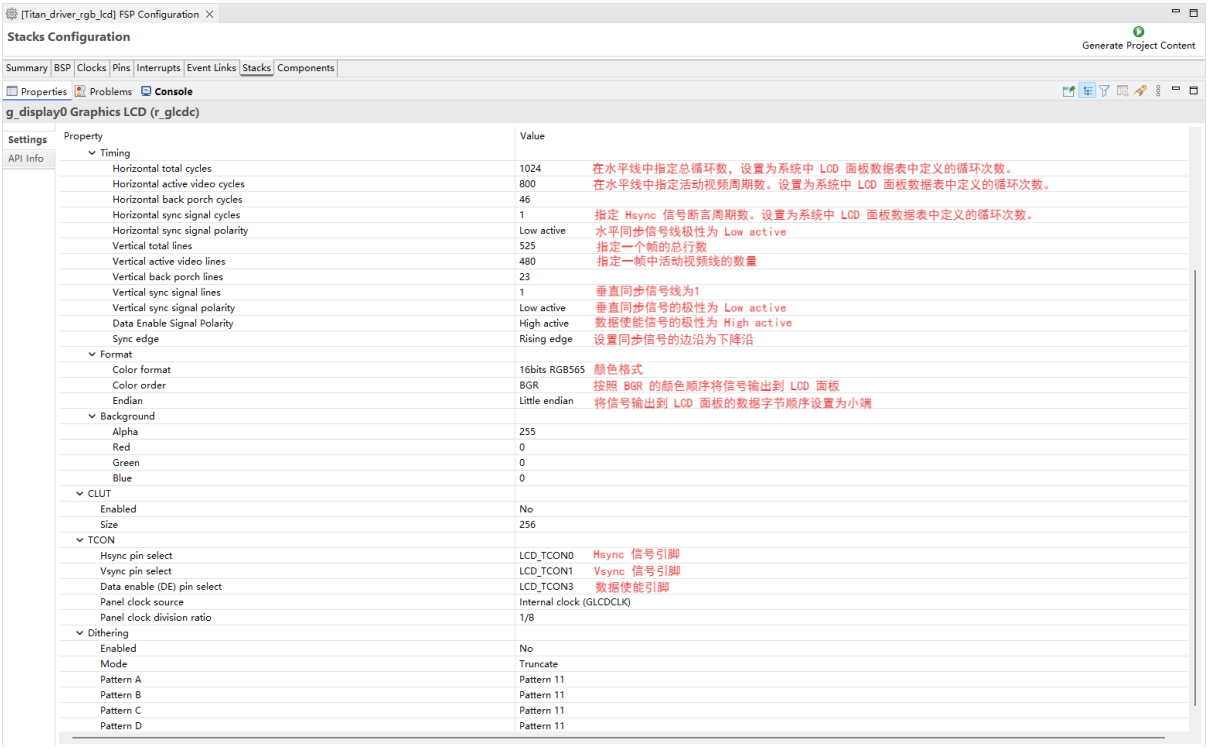
- Create a r_glcdc stack：



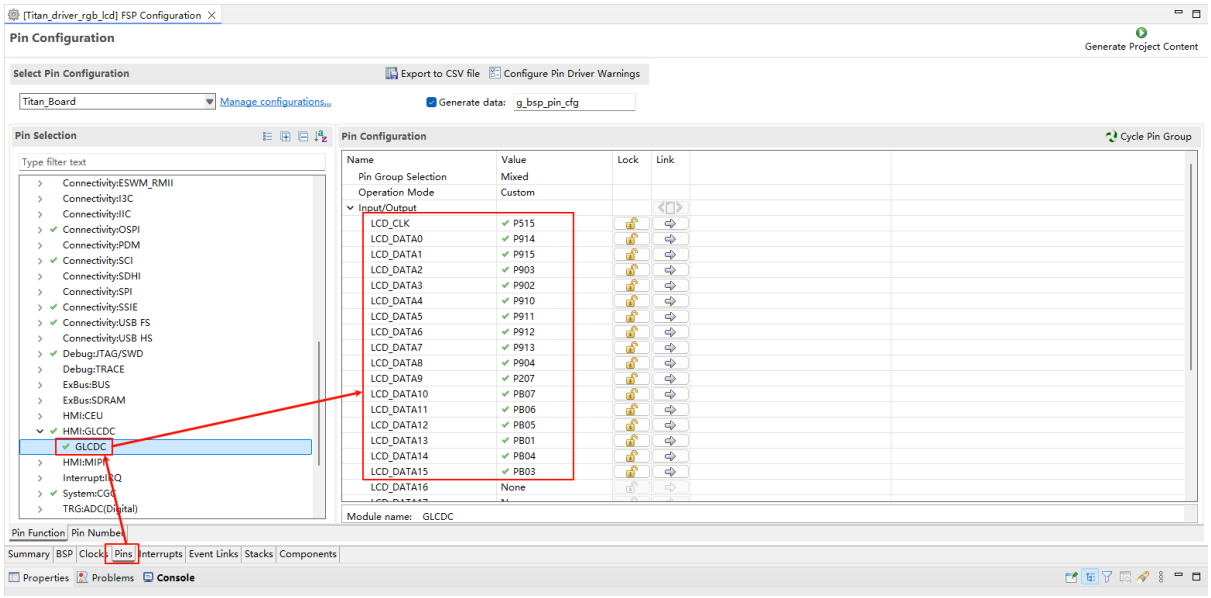
- Configure interrupt callback and graph layer 1:

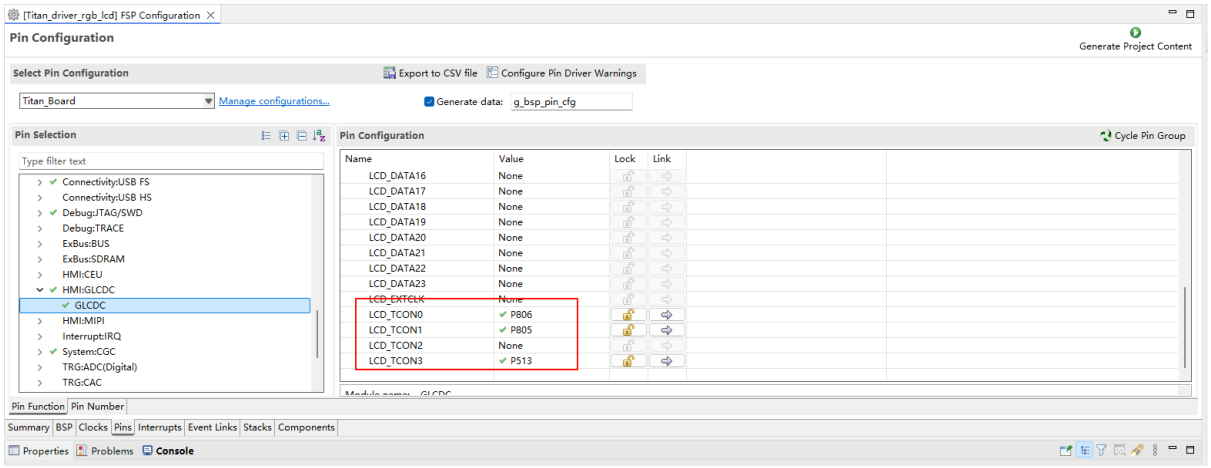


- Configure the output parameters, CLUT, TCON, and Dithering:



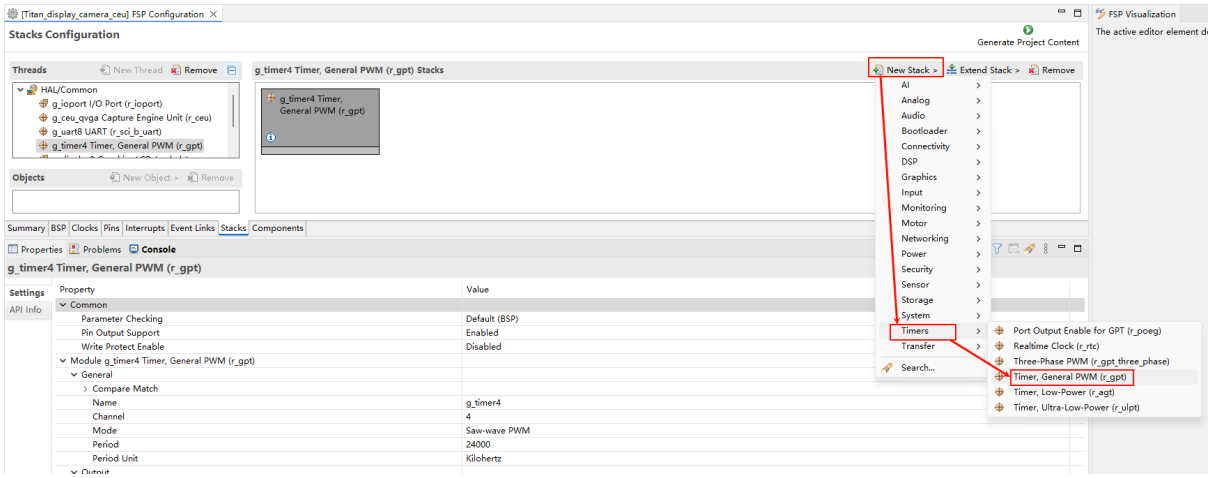
- Configure GLCDC pins:



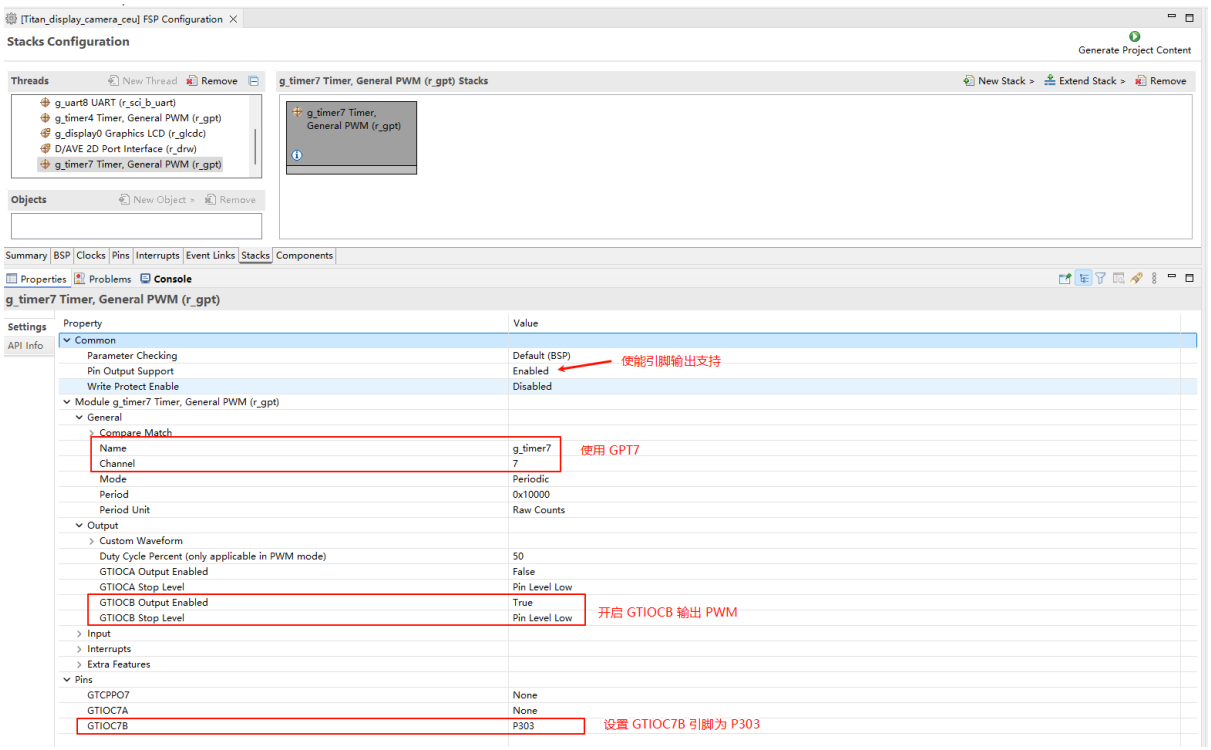


LCD Backlight Configuration

- Create a r_gpt stack:

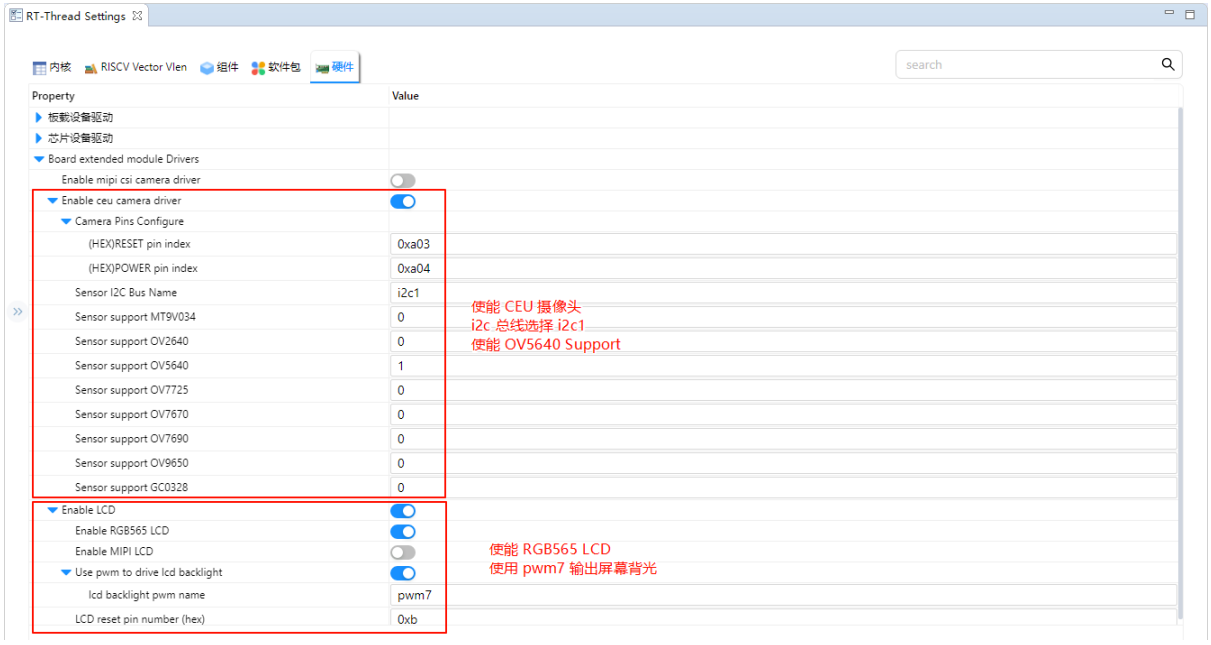


- Configure back light PWM output:



1.6 RT-Thread Settings Configuration

- Enable CUE camera, using i2c1 and ov5640 camera. Enable RGB565 LCD, using pwm7 output backlight.



1.7 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio's package manager, download the Titan Board resource package, create a new project, and compile it.

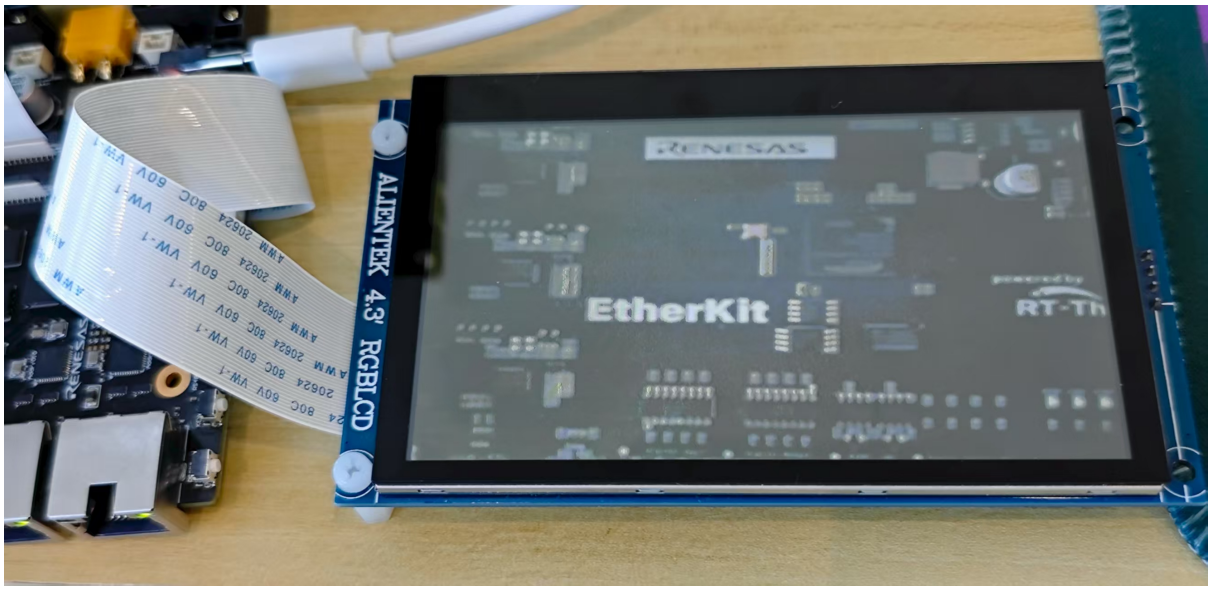
After compilation, connect the development board's USB-DBG interface to the PC and download the firmware to the development board.

1.8 Run Effect

After resetting the Titan Board, the terminal will output the following message:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Aug 15 2025 14:55:33
 2006 - 2024 Copyright by RT-Thread team
[D/drv_lcd]
lcd framebuffer address:0x22097200

Hello RT-Thread!
============================================================
This example project is an display with ceu camera routine!
============================================================

msh >i2c probe camera id:0x78
[I/sensor] set framesize:640X480
[I/sensor] Open CEU device
```

Here is the image displayed on the LCD screen:



2. MIPI CSI Camera Usage Instructions

English|Chinese

2.1 Introduction

This example demonstrates how to use the **MIPI CSI (Camera Serial Interface)** on the **Titan Board** to connect an **OV5640 camera**, and display the captured images on an **RGB565 LCD screen** via the **RT-Thread LCD framework**.

Key functionalities include:

- Initialize the MIPI CSI camera interface to capture real-time video streams
- Configure OV5640 camera parameters (resolution, frame rate, output format)
- Display captured images using the RT-Thread LCD driver
- Support image format conversion (YUV422 $\rightarrow$ RGB565)

2.2 RA8 Series MIPI CSI Features

The RA8 series MCU integrates a **MIPI CSI hardware module** for high-speed, low-power camera data acquisition, suitable for HD video and real-time image processing.

Hardware Interface Features

1. **Interface Type**
   - **MIPI CSI-2** D-PHY interface for high-bandwidth serial data transmission
   - Supports 1–4 data lanes
   - Synchronization is handled by MIPI D-PHY; no additional HSYNC/VSYNC required
2. **Data Rate and Resolution**
   - Supports up to 1.5–2.5 Gbps per lane (depending on MCU series and PHY configuration)
   - Can drive common camera resolutions: VGA, QVGA, SXGA, UXGA, 1080p, etc.
3. **Camera Compatibility**
   - Compatible with OV5640, IMX219, and other common CMOS cameras
   - Supports camera register configuration and auto-initialization

Image Formats and Processing Capabilities

1. **Supported Image Formats**
   - **RAW10 / RAW12** (for algorithm development and image processing)
   - **YUV422** (for video display)
   - **RGB565** (suitable for LCD display)
2. **Image Processing Features**

- **Color space conversion** (YUV → RGB)
- **Image cropping** (ROI capture)
- **Mirror and flip**
- **Hardware accelerated scaling**

3. **Hardware Acceleration**
   - MIPI CSI includes built-in DMA to reduce CPU load
   - Supports high-speed format conversion and scaling

DMA Support and Buffering

1. **High-Speed DMA Transfer**
   - Works with MCU DMAC to write images directly to memory or LCD buffer
   - Reduces CPU intervention and increases frame rate
2. **Multi-Buffer Mechanism**
   - Supports double buffering or ring buffers for continuous video capture
   - Prevents frame loss and display latency
3. **Flexible DMA Configuration**
   - Configurable buffer start address and size
   - Supports interrupt callback handling

Interrupt Mechanism

1. **Interrupt Types**
   - **Frame End Interrupt**: triggered at the end of each frame capture
   - **Line End Interrupt (optional)**
   - **Error Interrupt**: buffer overflow, PHY errors, etc.
2. **Interrupt Features**
   - Supports RT-Thread ISR callbacks
   - Can work with DMA to enable real-time display

Timing and Synchronization Features

1. **Synchronization Method**
   - CSI relies on MIPI D-PHY for clock and data synchronization
   - No additional HSYNC/VSYNC required
2. **Pixel Clock and Data Alignment**
   - Supports pixel-aligned or byte-aligned data
   - Automatically handles RAW/YUV/RGB data alignment

Performance Optimization

1. **High Throughput**
    - DMA + double buffering enables continuous video capture
    - Low CPU usage suitable for real-time applications
2. **Reliability**
    - PHY errors or data loss interrupts can trigger exception handling
    - Supports buffer overflow detection
3. **Flexibility**
    - Supports multiple resolutions and formats
    - ROI capture and hardware scaling improve display efficiency

Application Scenarios

- Real-time video display on LCD
- Video capture and processing algorithm verification
- Embedded vision applications such as surveillance, gesture recognition, and robotics vision

2.3 RA8 Series MCU GLCDC (Graphics LCD Controller) Features

The RA8 series MCU (e.g., RA8P1) integrates a **GLCDC hardware module** for driving TFT/LCD displays, enabling high-speed graphics rendering and video display, supporting multiple resolutions, color formats, and display modes.

Hardware Features

1. **Resolution Support**
    - Can drive common resolutions from QVGA (320×240) to WQVGA/XGA
    - Limited by on-chip RAM and display interface bandwidth
2. **Color Support**
    - Supports 1/4/8/16/24/32-bit color depth
    - Common formats: RGB565, RGB888
    - Supports palette mode (CLUT)
    - Hardware color conversion available
3. **Interface Types**
    - Parallel RGB (TFT LCD interface)
    - Supports 16/18/24-bit data bus
    - Direct connection to external LCD panels
    - Programmable timing: HSYNC, VSYNC, DE, PCLK, RGB output

Layers and Display Modes

1. **Layer Support**
    - Single-layer mode (single screen display)
    - Multi-layer mode (palette or hardware alpha blending)
    - Supports transparent/semi-transparent overlay
2. **Display Modes**
    - RGB mode (direct color output)
    - CLUT/Palette mode (indexed color via lookup table)
    - Configurable scan direction (horizontal/vertical)

DMA and Frame Buffer

1. **Frame Buffer Access**
    - GLCDC can directly access on-chip or external SRAM frame buffer
    - Supports single or double buffering
    - Supports ring buffer for continuous refresh
2. **DMA Support**
    - Works with MCU DMAC to reduce CPU usage
    - Can transfer images directly from memory to LCD
    - Supports line, block, or full-frame transfer

Hardware Graphics Functions

1. **Window Cropping and Scaling**
    - Can specify display window area
    - Supports simple horizontal/vertical scaling
2. **Hardware Graphics Acceleration**
    - Rectangle fill, color replacement
    - Image transparency processing
    - Can be combined with CEU or DMA for video display
3. **Color Format Conversion**
    - YCbCr $\rightarrow$ RGB
    - RGB888 $\rightarrow$ RGB565
    - Hardware accelerated to reduce CPU load

Interrupt Mechanism

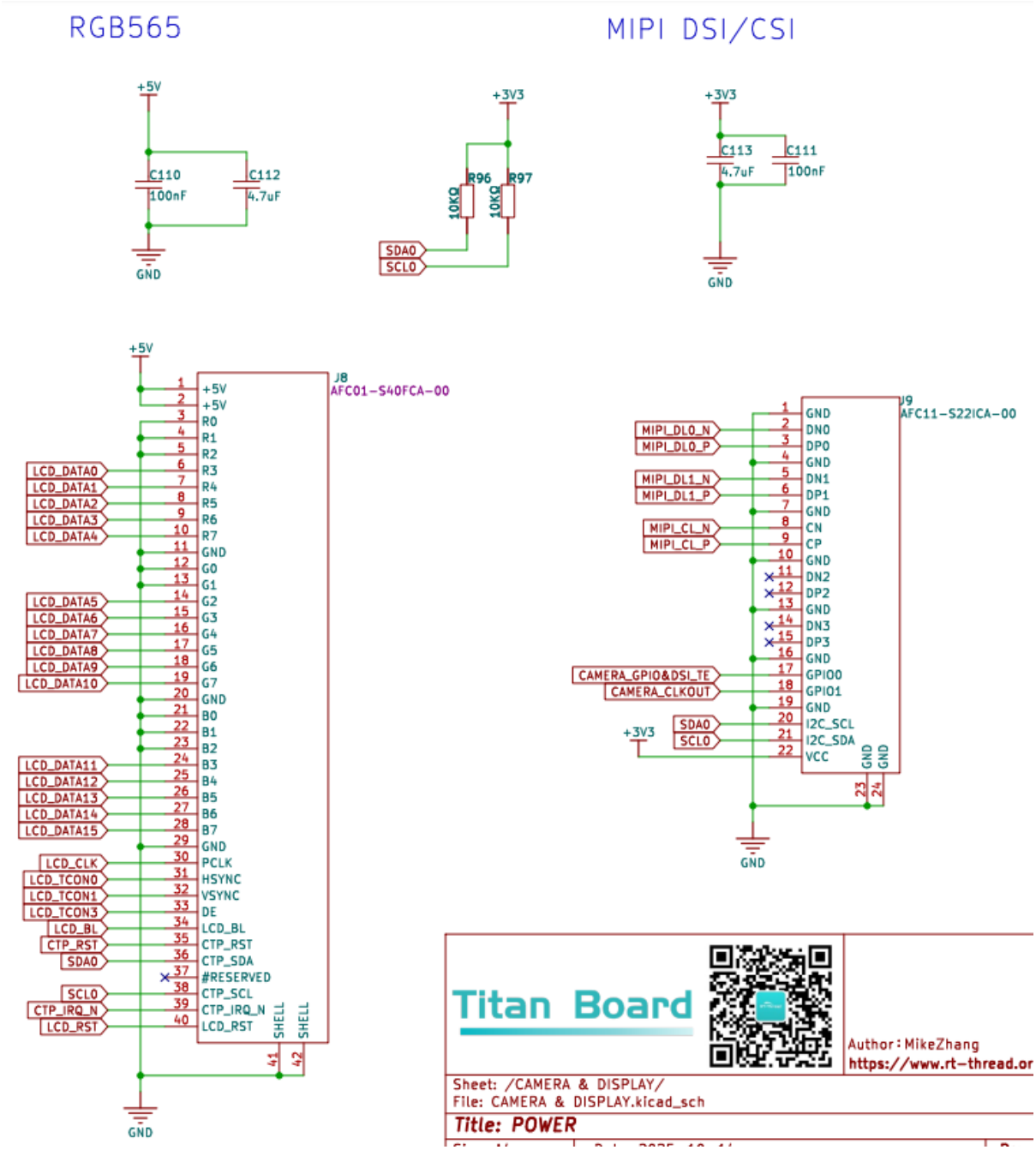
1. **Interrupt Types**
    - Frame End
    - Line End (optional)
    - Access error/overflow
2. **Interrupt Application**
    - Integrates with RT-Thread ISR
    - Can trigger buffer update or double-buffer swap on frame completion
    - Enables smooth animations and video display

Performance Optimization

1. **Double Buffer Mechanism**
    - Reduces flicker
    - CPU can render next frame in background
    - GLCDC hardware automatically switches display buffer
2. **Frame Rate Control**
    - Programmable clock and line/frame synchronization
    - Supports common refresh rates (30fps, 60fps)
3. **CPU Offload**
    - Many graphics operations performed in hardware
    - DMA + GLCDC combination enables efficient image display

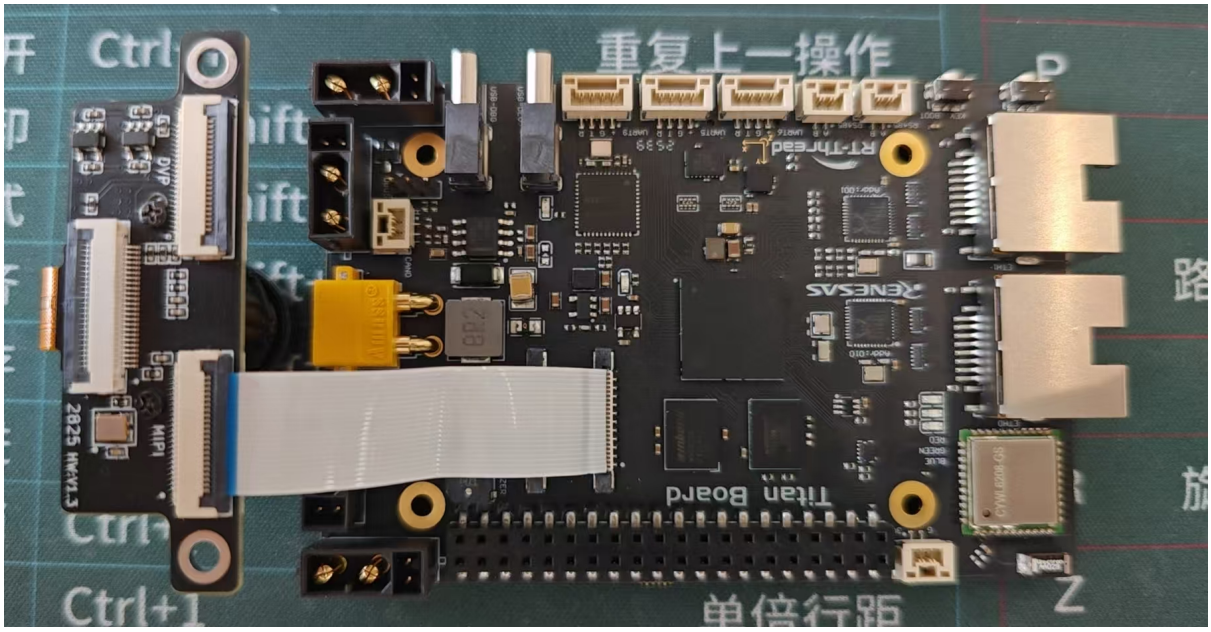
2.4 Hardware Description

- The MIPI DSI/CSI interface and the RGB LCD interface are shown in the following figure:



- The connection method of the MIPI CSI camera is as follows:

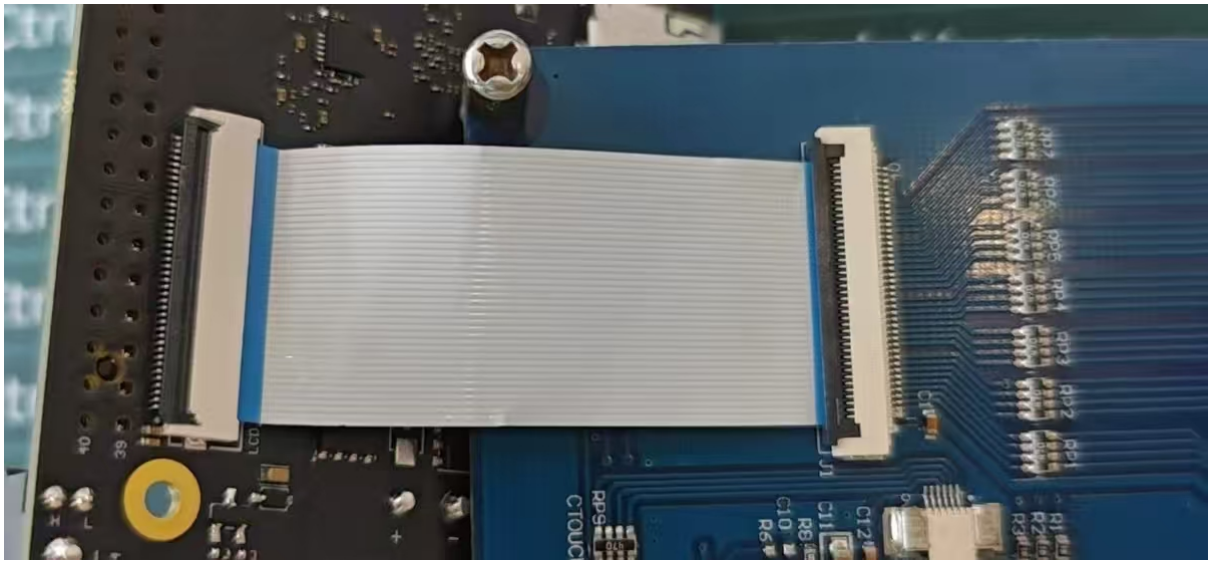
Use a **22-pin reverse FFC cable** to connect the development board’ s **MIPI DSI/CSI** connector to the camera adapter board’ s **MIPI** connector.



- The connection method of RGB LCD is as follows:

Display model: **ALIENTEK 4.3'' RGB LCD 800×480**

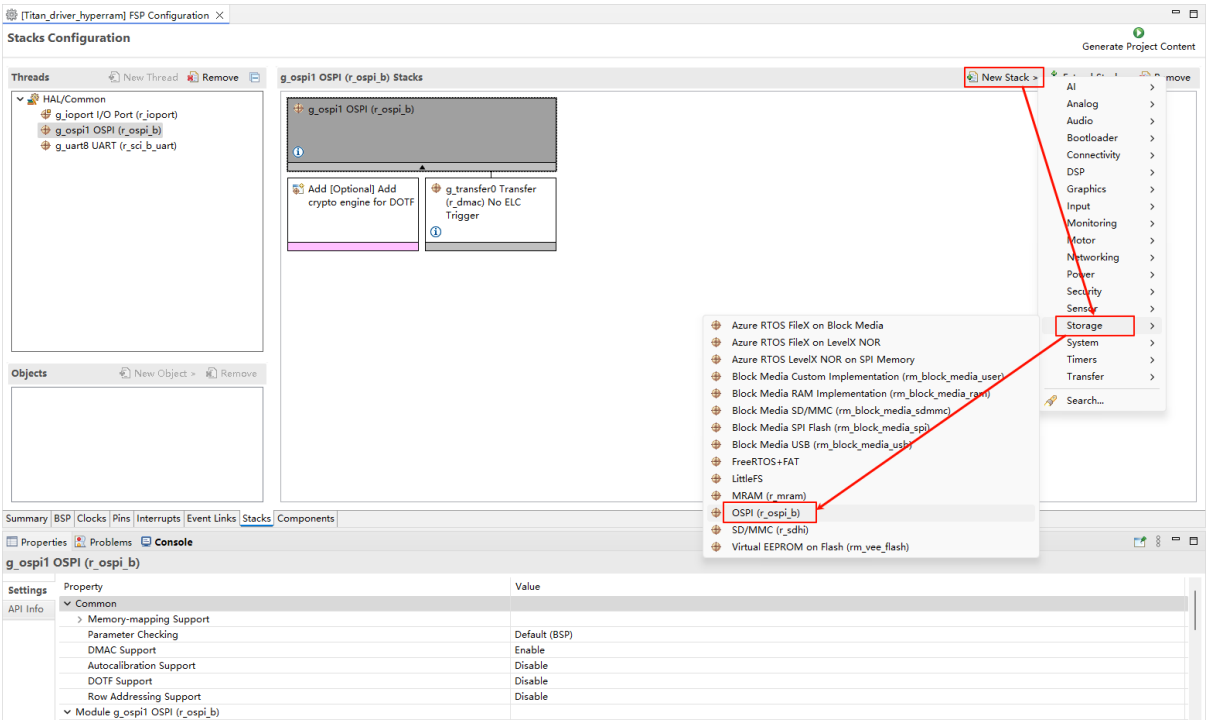
Use a **40-pin same-direction FFC cable** to connect the development board's **LCD_RGB** connector to the display.



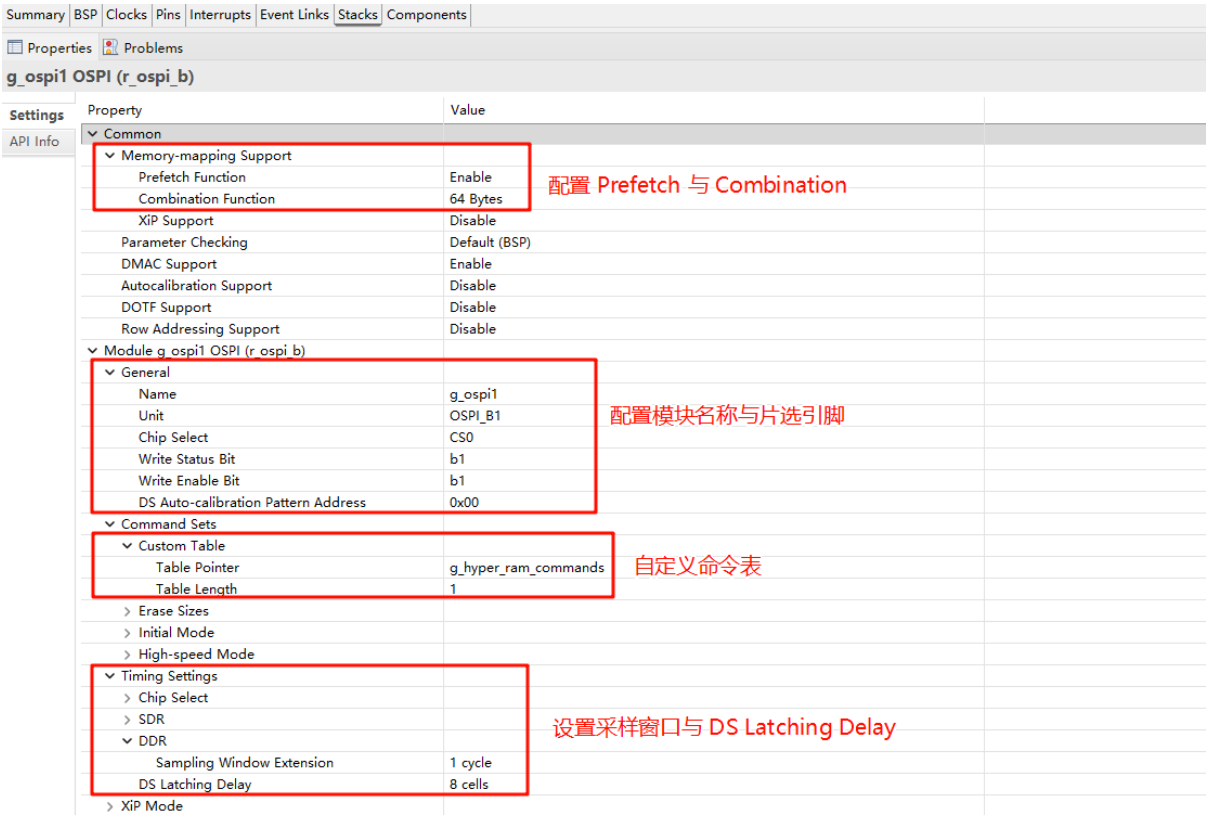
2.5 FSP Configuration

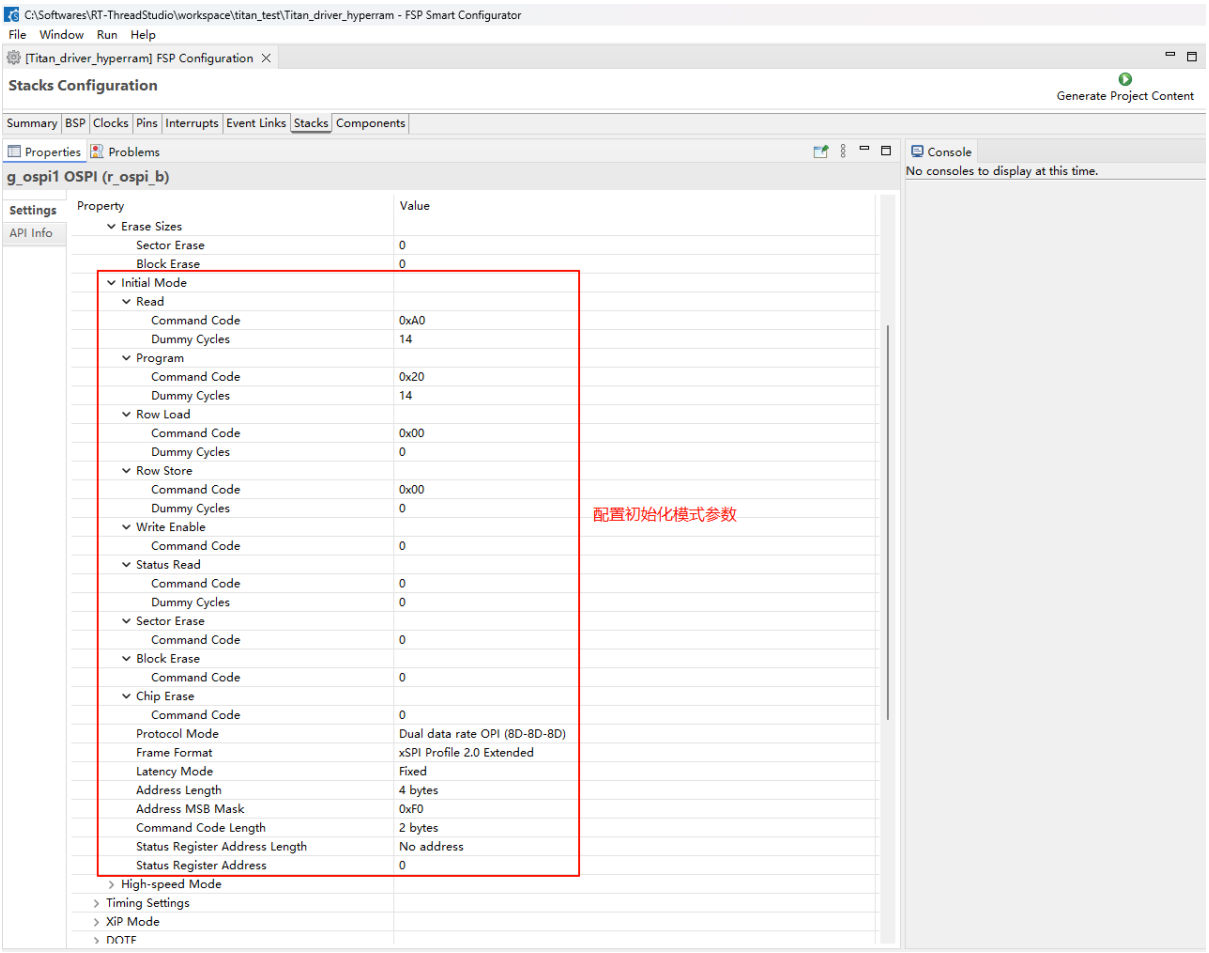
HyperRAM 配置

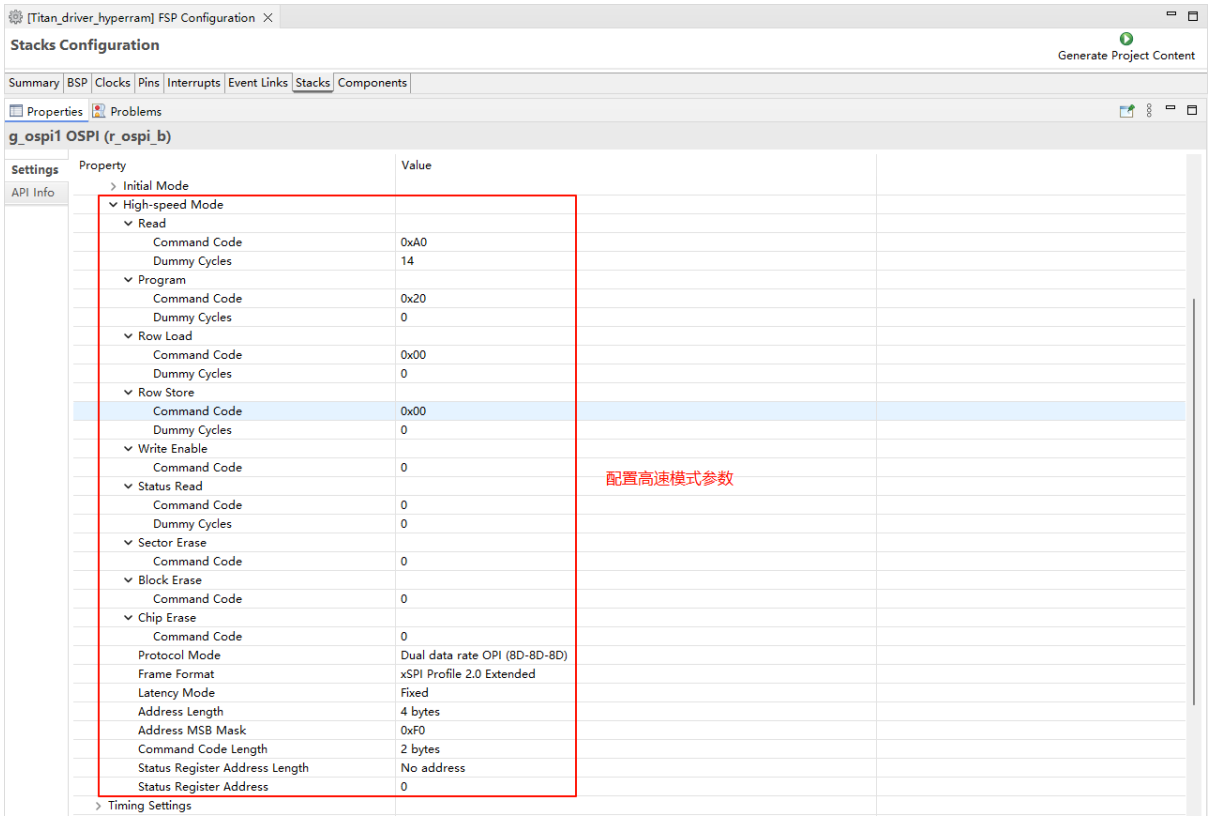
- Create a r_ospi_b stack:



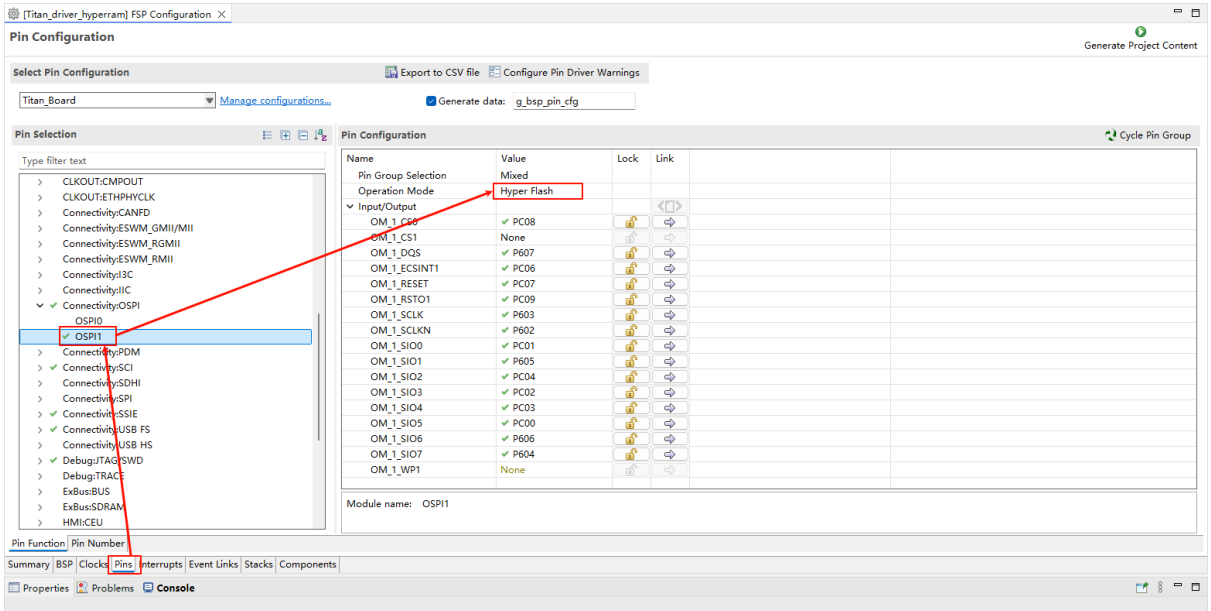
- Configure the r_ospi_b stack:



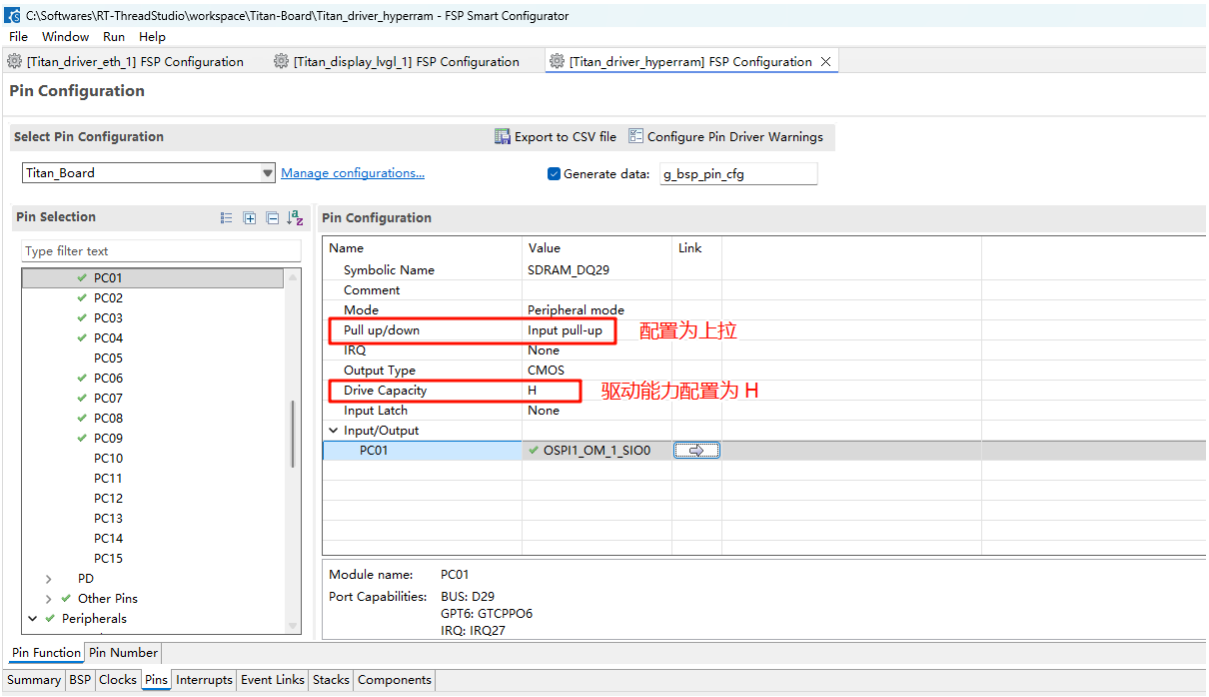

C:\Softwares\RT-ThreadStudio\workspace\titan_test\Titan_driver_hyperram - FSP Smart Configurator
File Window Run Help
[Titan_driver_hyperram] FSP Configuration
Stacks Configuration
Generate Project Content
Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems
g_ospi1 OSPI (r_ospi_b)
Settings
API Info
Property
Value
Erase Sizes
Sector Erase 0
Block Erase 0
Initial Mode
Read
Command Code 0xA0
Dummy Cycles 14
Program
Command Code 0x20
Dummy Cycles 14
Row Load
Command Code 0x00
Dummy Cycles 0
Row Store
Command Code 0x00
Dummy Cycles 0
Write Enable
Command Code 0
Status Read
Command Code 0
Dummy Cycles 0
Sector Erase
Command Code 0
Block Erase
Command Code 0
Chip Erase
Command Code 0
Protocol Mode Dual data rate OPI (8D-8D-8D)
Frame Format xSPI Profile 2.0 Extended
Latency Mode Fixed
Address Length 4 bytes
Address MSB Mask 0xF0
Command Code Length 2 bytes
Status Register Address Length No address
Status Register Address 0
High-speed Mode
Timing Settings
XiP Mode
DOTF
Console
No consoles to display at this time.
配置初始化模式参数



- HyperRAM pin configuration:

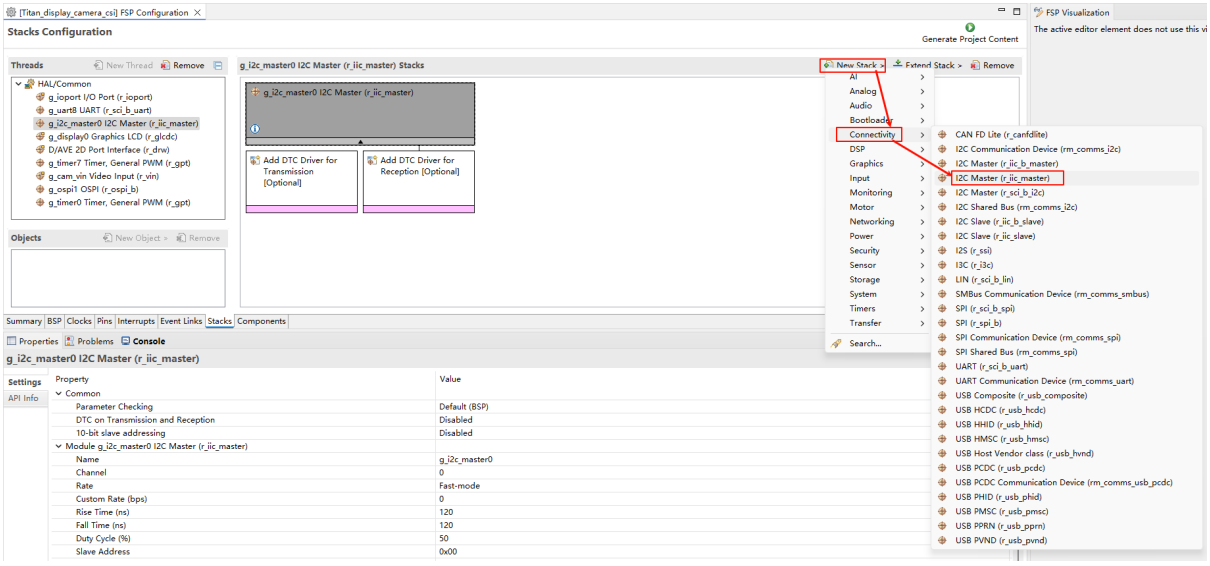


- The drive capability of all pins related to HyperRAM should be configured as H, and OM_1_SIO0 to OM_1_SIO7 need to be configured as Input pull-up.

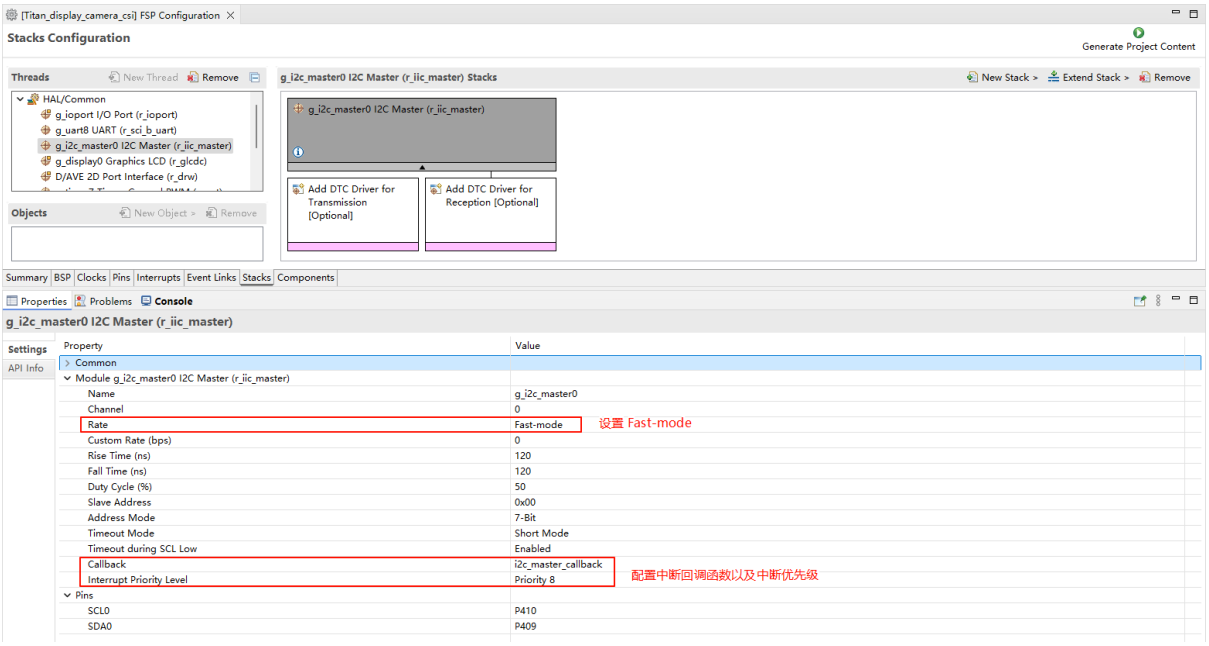


I2C0 Configuration

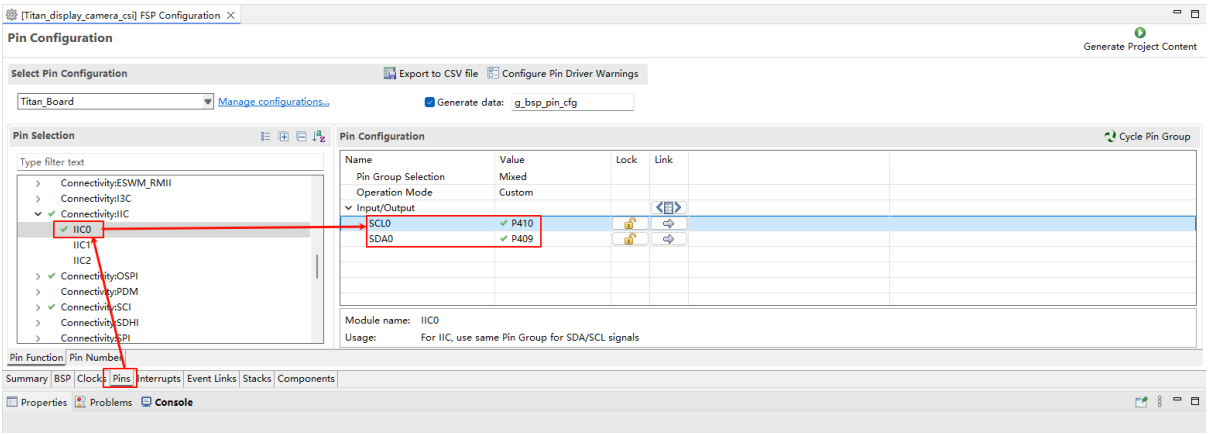
- Create a r_iic_master stack:



- Configure I2C0:



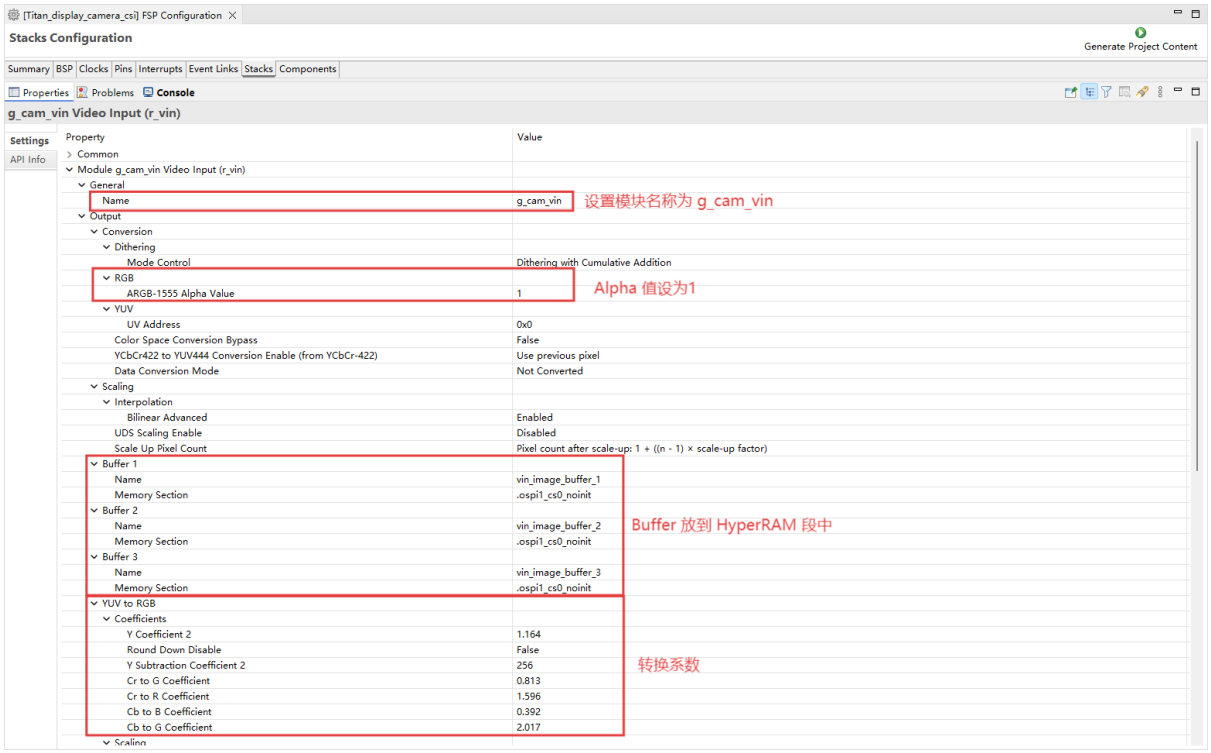
- Configure I2C0 pins:

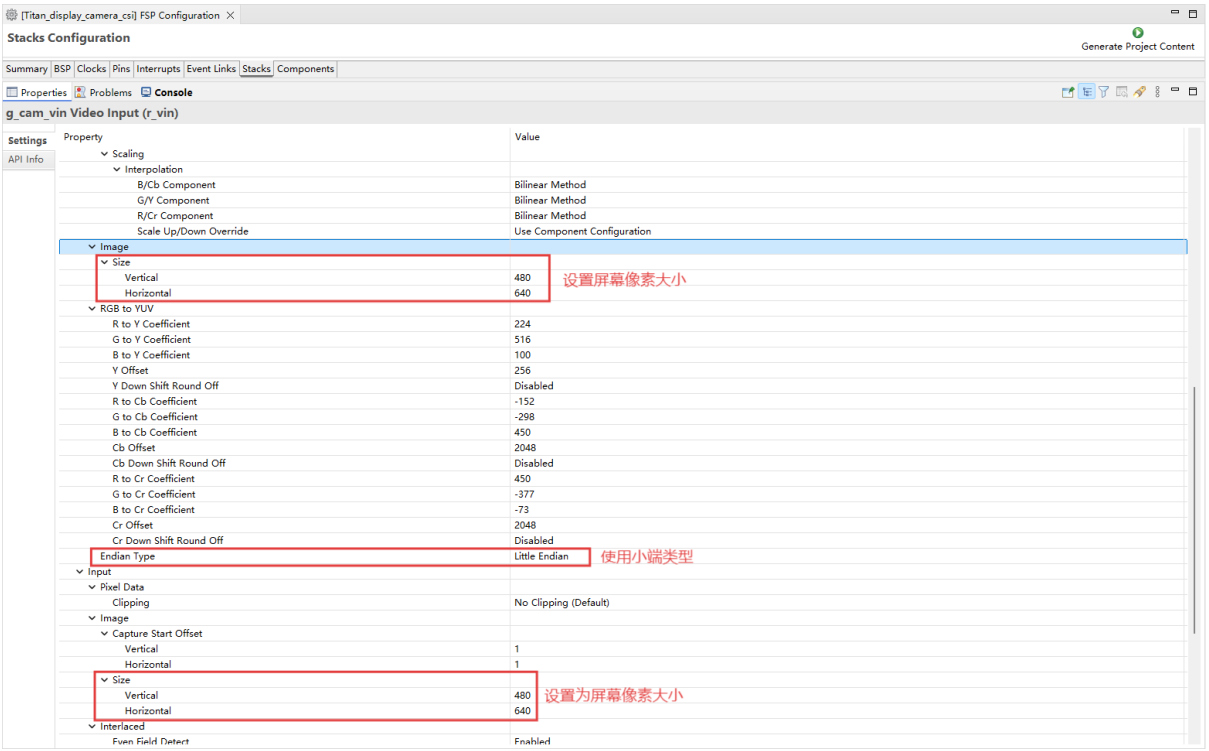


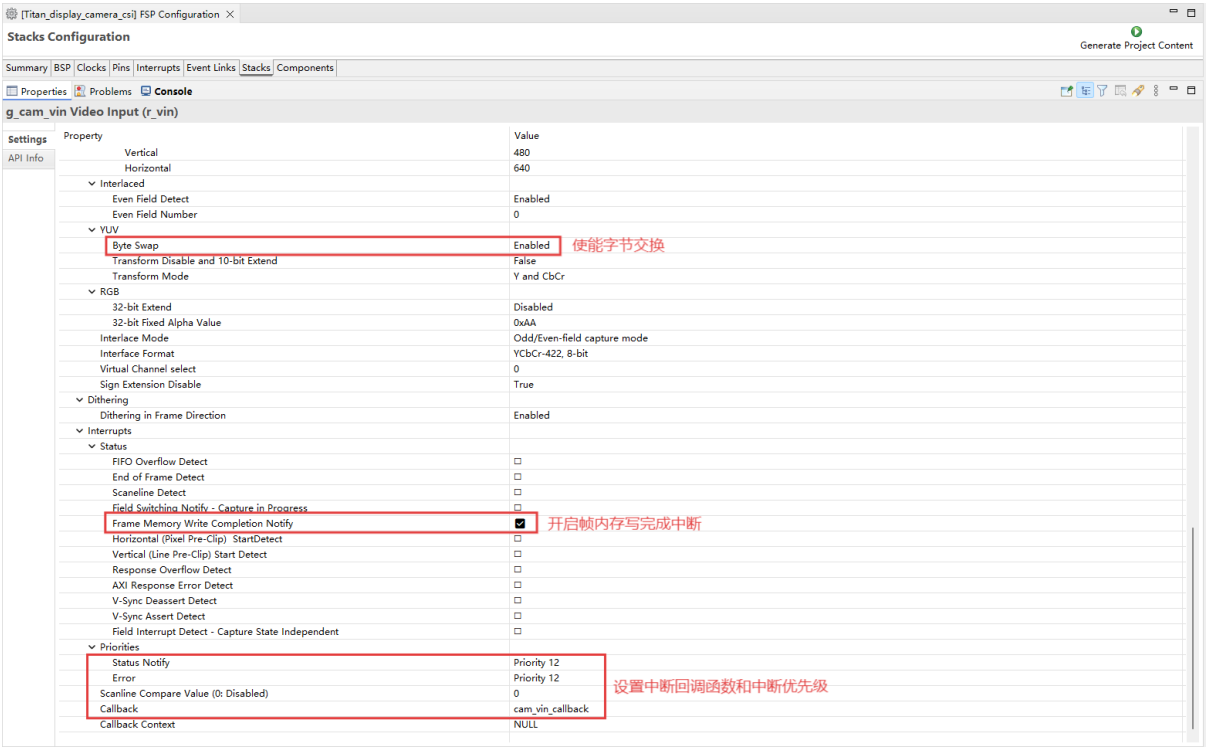
VIN Configuration

- Create a r_vin stack:

- Configure VIN:

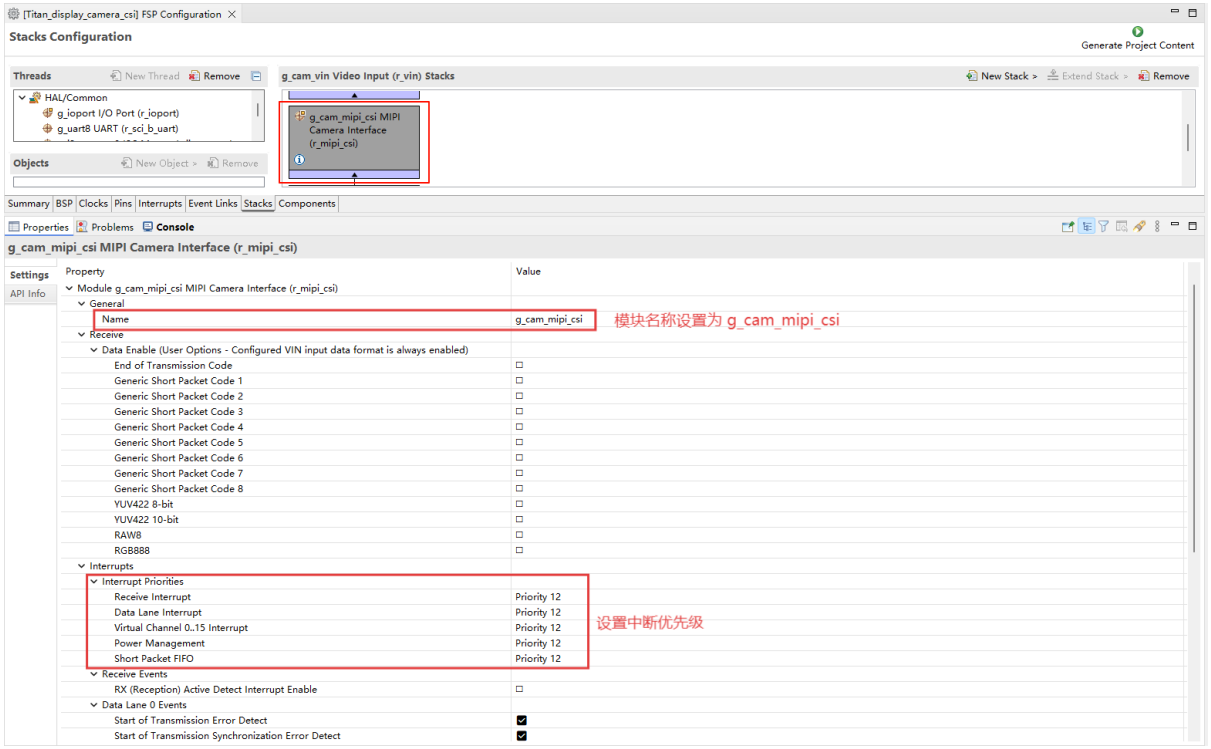


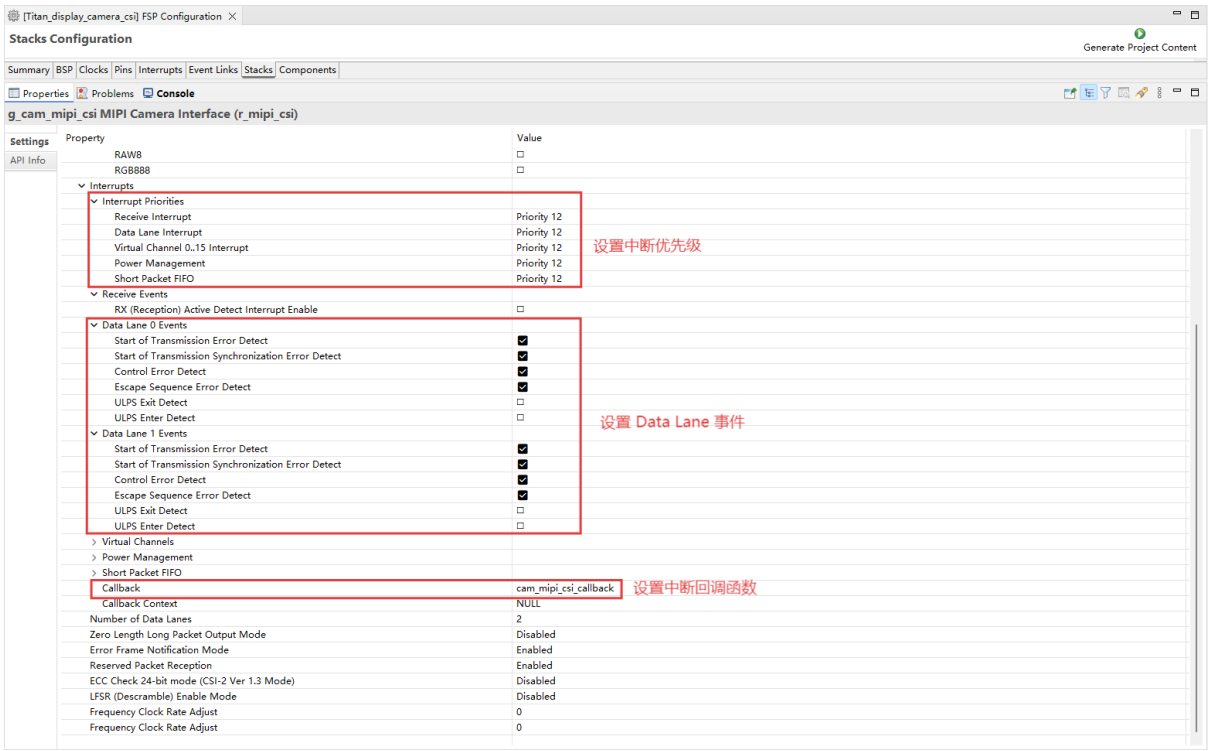

[Titan_display_camera_csi] FSP Configuration
Stacks Configuration
Generate Project Content
Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems Console
g_cam_vin Video Input (r_vin)
Settings
API Info
Property
Value
Scaling
Interpolation
B/Cb Component
Bilinear Method
G/Y Component
Bilinear Method
R/Cr Component
Bilinear Method
Scale Up/Down Override
Use Component Configuration
Image
Size
Vertical
480
Horizontal
640
设置屏幕像素大小
RGB to YUV
R to Y Coefficient
224
G to Y Coefficient
516
B to Y Coefficient
100
Y Offset
256
Y Down Shift Round Off
Disabled
R to Cb Coefficient
-152
G to Cb Coefficient
-298
B to Cb Coefficient
450
Cb Offset
2048
Cb Down Shift Round Off
Disabled
R to Cr Coefficient
450
G to Cr Coefficient
-377
B to Cr Coefficient
-73
Cr Offset
2048
Cr Down Shift Round Off
Disabled
Endian Type
Little Endian
使用小端类型
Input
Pixel Data
Clipping
No Clipping (Default)
Image
Capture Start Offset
Vertical
1
Horizontal
1
Size
Vertical
480
Horizontal
640
设置为屏幕像素大小
Interlaced
Even Field Detect
Enabled


[Titan_display_camera_csi] FSP Configuration
Stacks Configuration
Generate Project Content
Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems Console
g_cam_vin Video Input (r_vin)
Settings
API Info
Property
Value
Vertical
480
Horizontal
640
Interlaced
Even Field Detect
Enabled
Even Field Number
0
YUV
Byte Swap
Enabled
使能字节交换
Transform Disable and 10-bit Extend
False
Transform Mode
Y and CbCr
RGB
32-bit Extend
Disabled
32-bit Fixed Alpha Value
0xAA
Interlace Mode
Odd/Even-field capture mode
Interface Format
YCbCr-422, 8-bit
Virtual Channel select
0
Sign Extension Disable
True
Dithering
Dithering in Frame Direction
Enabled
Interrupts
Status
FIFO Overflow Detect
End of Frame Detect
Scaneline Detect
Field Switching Notify - Capture in Progress
Frame Memory Write Completion Notify
开启帧内存写完成中断
Horizontal (Pixel Pre-Clip) StartDetect
Vertical (Line Pre-Clip) Start Detect
Response Overflow Detect
AXI Response Error Detect
V-Sync Deassert Detect
V-Sync Assert Detect
Field Interrupt Detect - Capture State Independent
Priorities
Status Notify
Priority 12
Error
Priority 12
Scanline Compare Value (0: Disabled)
0
Callback
cam_vin_callback
设置中断回调函数和中断优先级
Callback Context
NULL

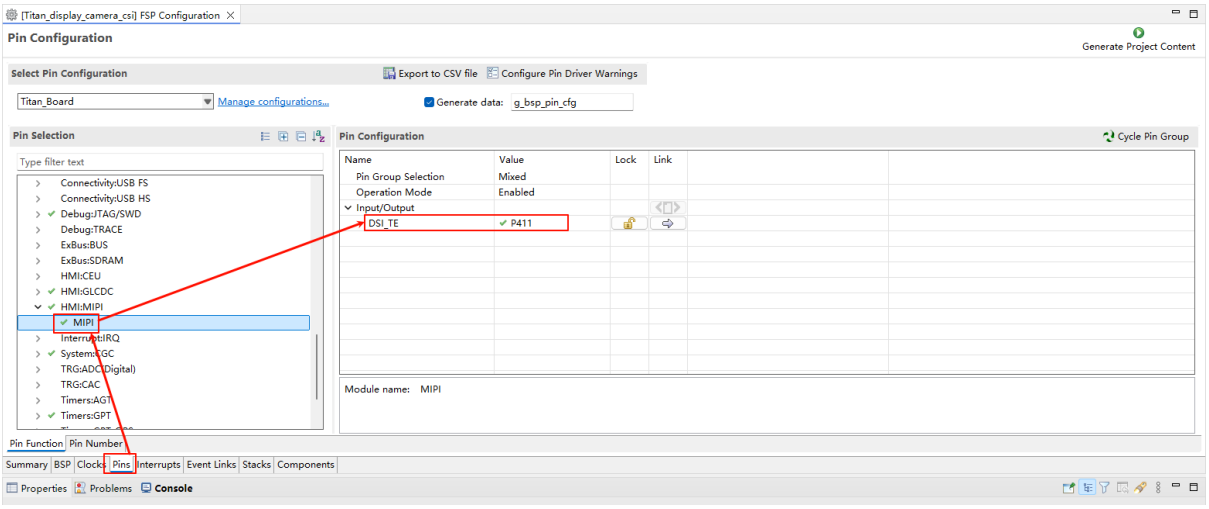
MIPI CSI Configuration

- Configure MIPI CSI:



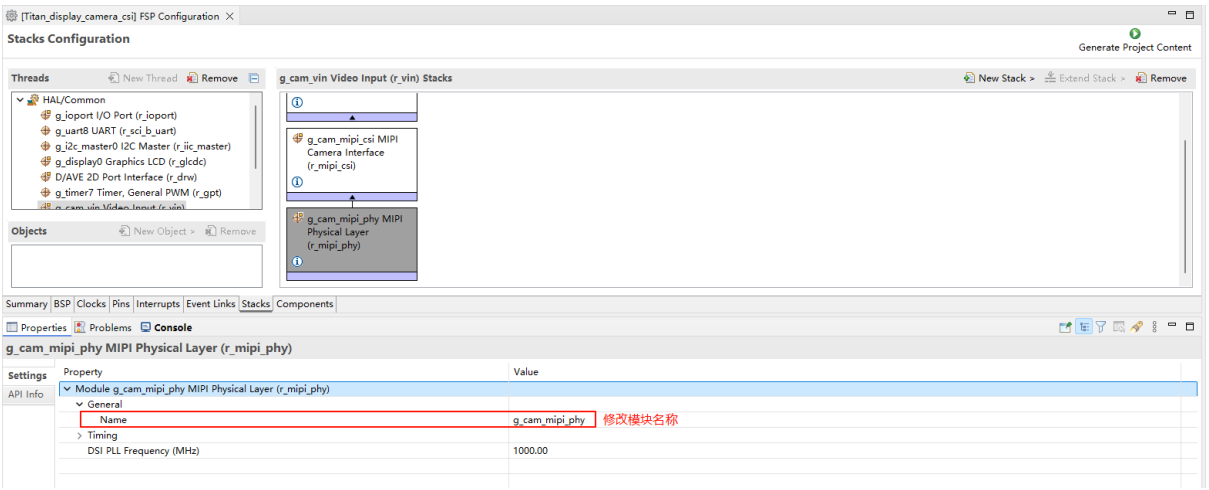


- Configure MIPI pins:



MIPI PHY Configuration

- Set the MIPI PHY module name:

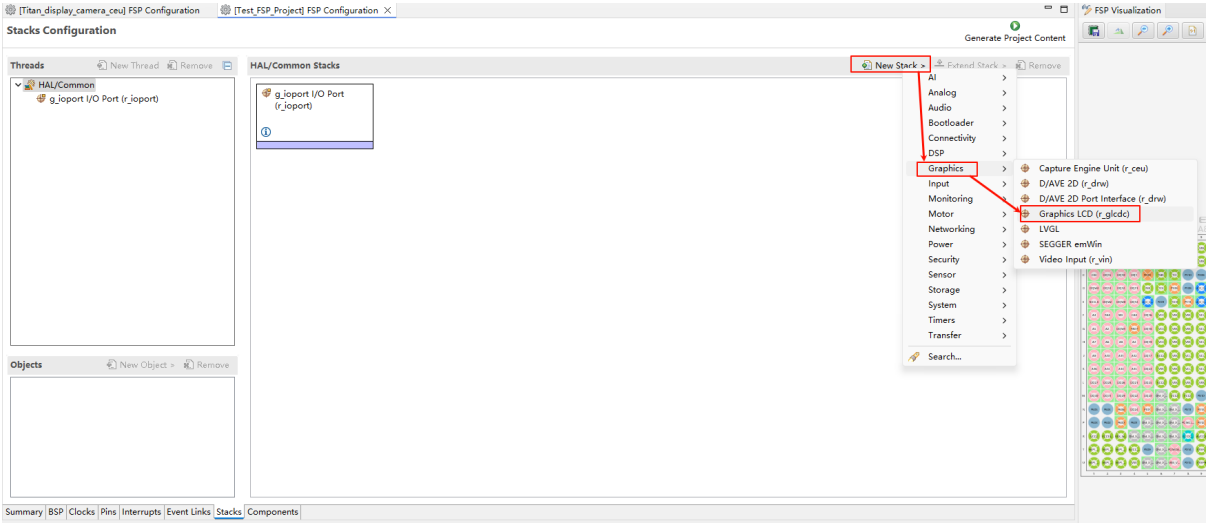


D/AVE 2D Configuration

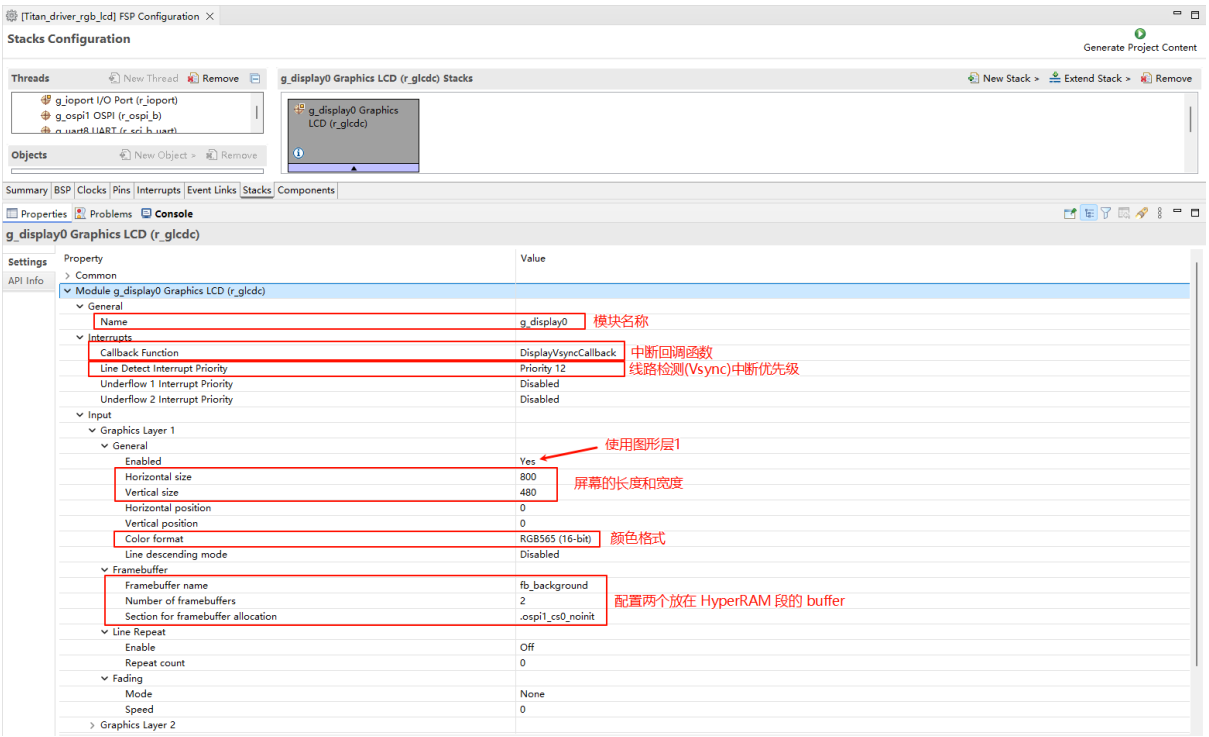
- Create a r_drw stack:

RGB LCD Configuration

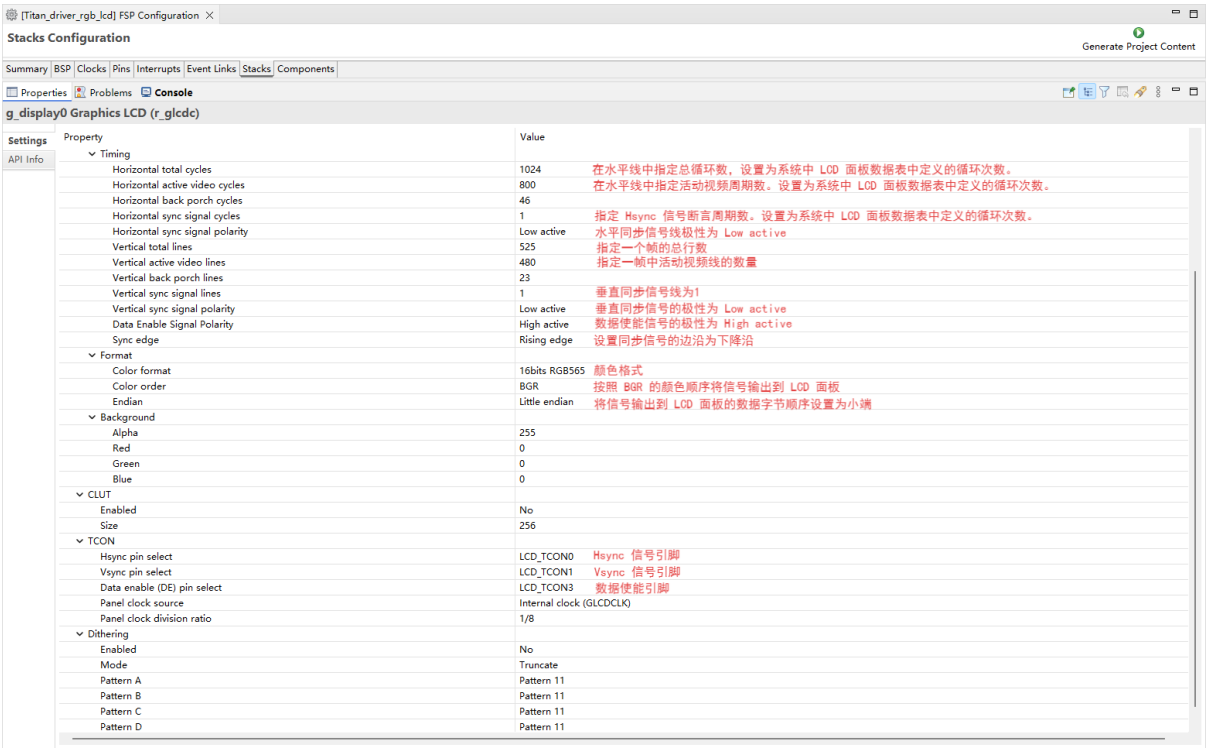
- Create a r_glcdc stack:



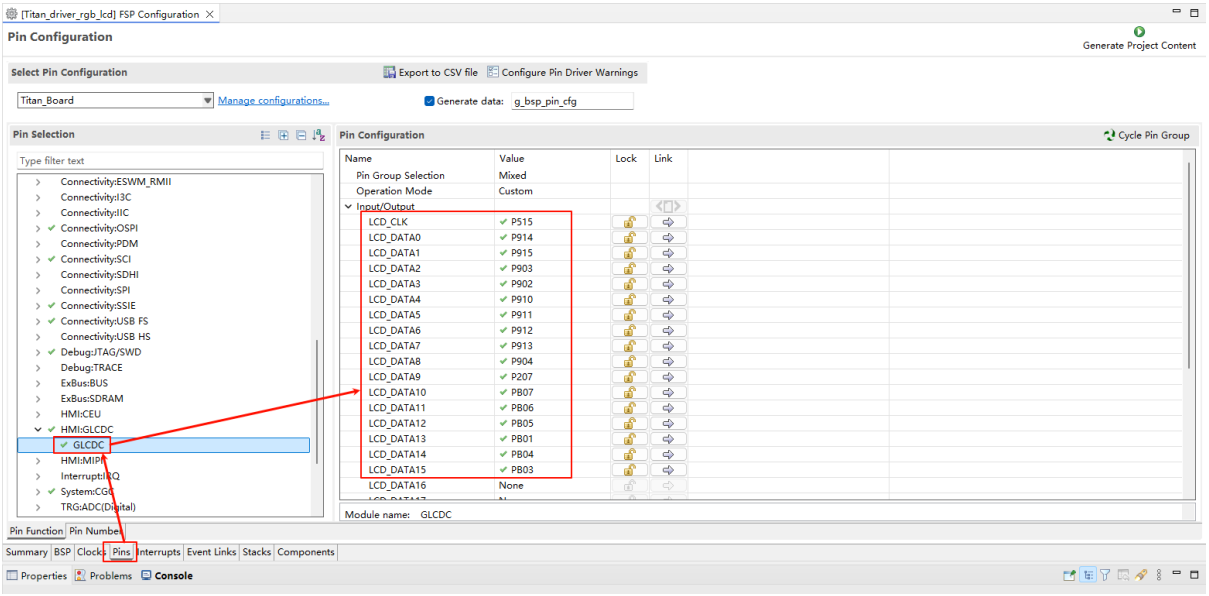
- Configure interrupt callback and graph layer 1:

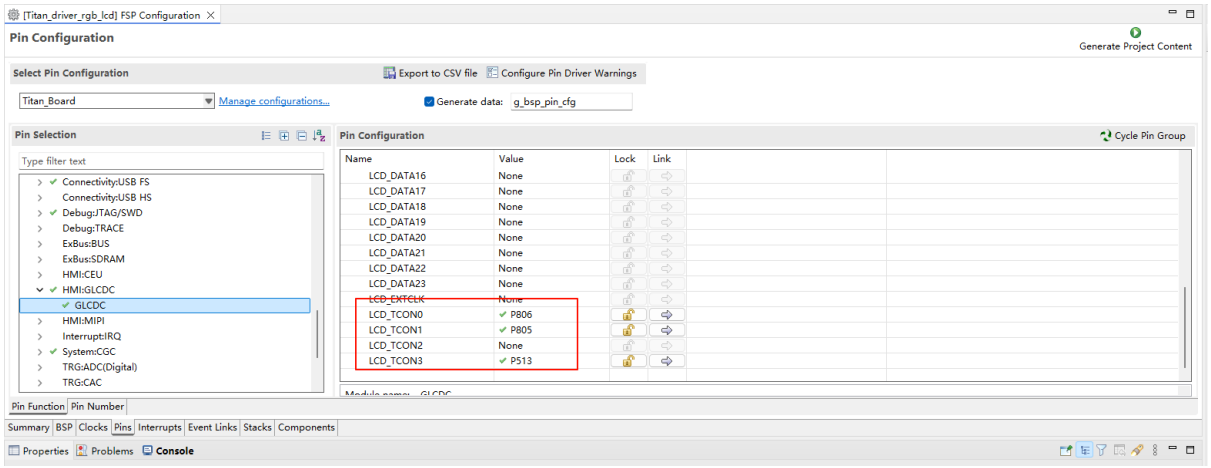


- Configure the output parameters, CLUT, TCON, and Dithering:



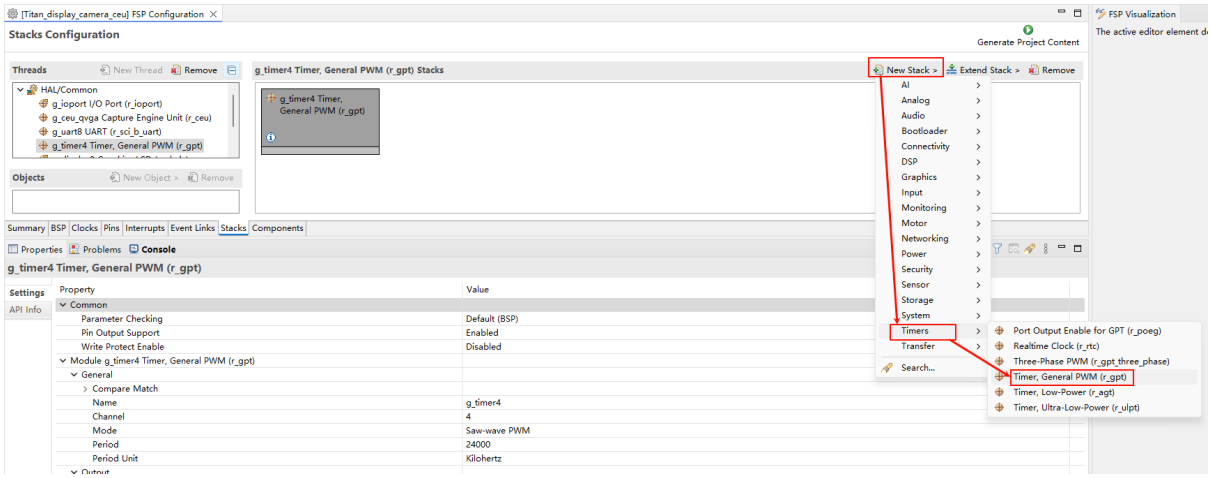
- Configure GLCDC pins:



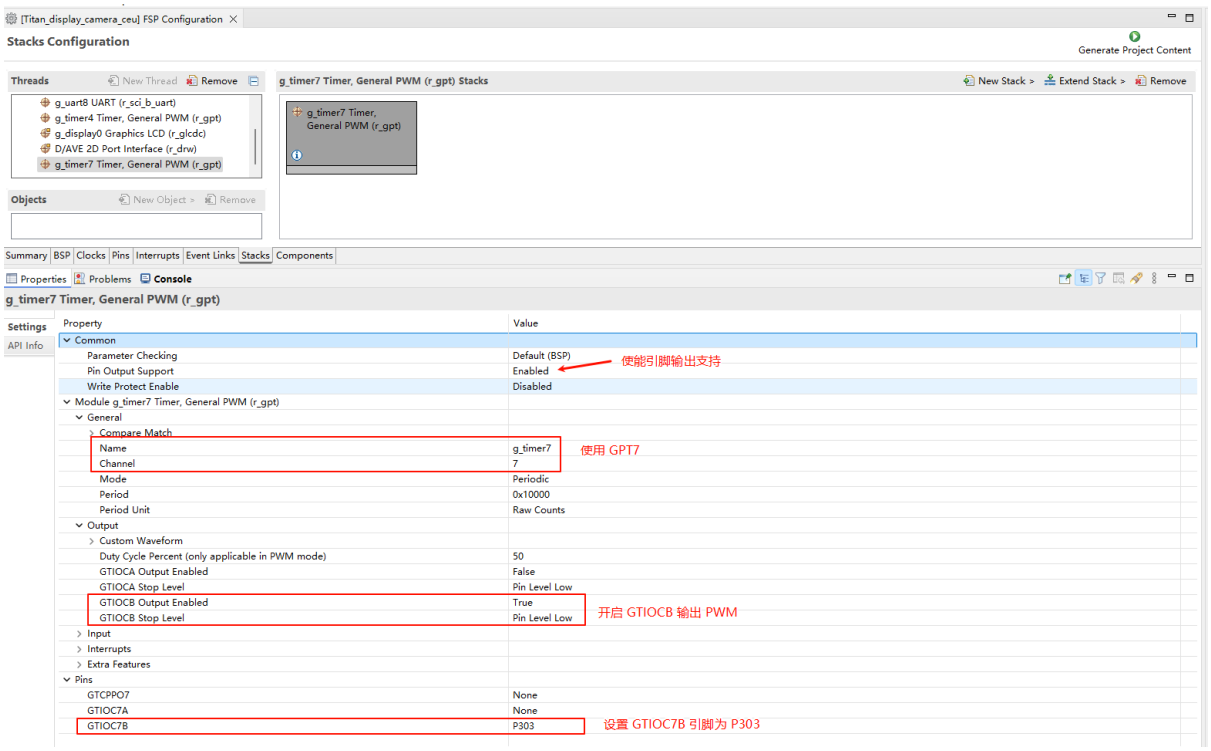


LCD Backlight Configuration

- Create a r_gpt stack:



- Configure backlight PWM output:



2.6 RT-Thread Settings Configuration

- Enable MIPI CSI camera, using i2c0; Enable RGB565 LCD, using pwm7 output backlight.

2.7 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio’s package manager, download the Titan Board resource package, create a new project, and compile it.

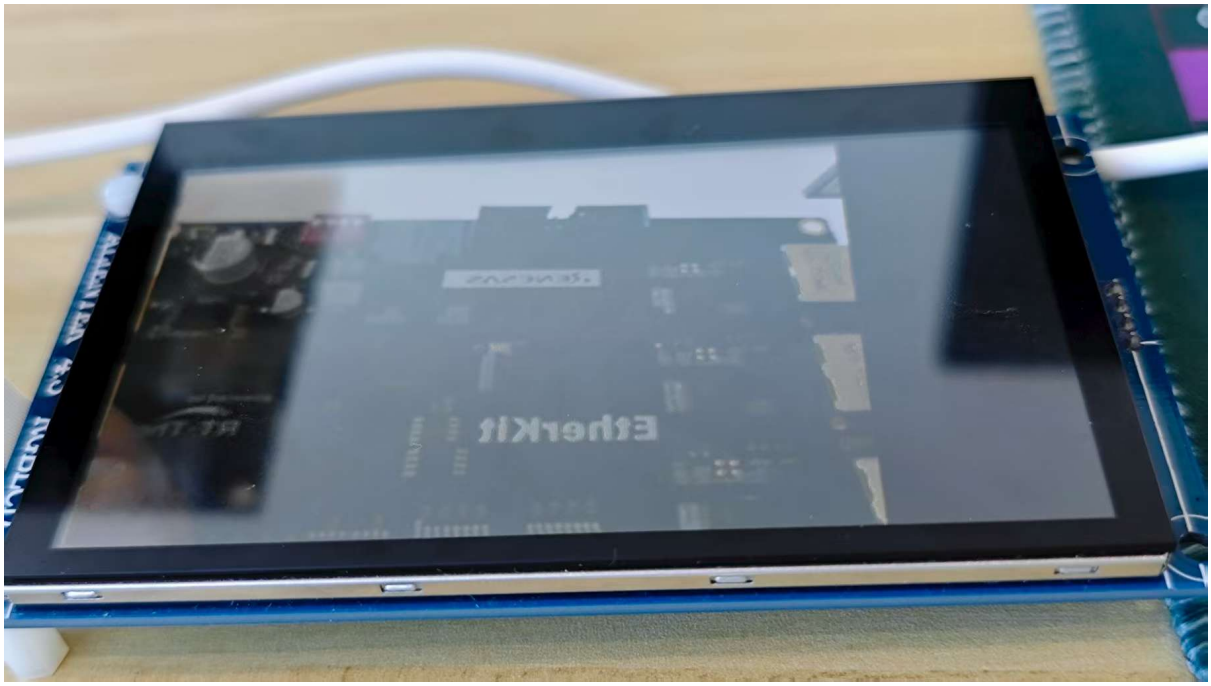
After compilation, connect the development board’s USB-DBG interface to the PC and download the firmware to the development board.

2.8 Run Effect

After resetting the Titan Board, the terminal will output the following message:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Aug 15 2025 15:02:50
 2006 - 2024 Copyright by RT-Thread team
[D/drv_lcd]
lcd framebuffer address:0x22020d40
msh >
Hello Titan Board!
================================================
This example project is an mipi-csi camera display routine!
================================================
[I/camera.i2c] I2C bus initialized: i2c0
The vin driver uses hardware mailboxes for the buffer.
```

Here is the image displayed on the LCD screen:



3. CEU Camera Usage Instructions(MIPI LCD)

English|Chinese

3.1 Introduction

This example demonstrates how to use the **CEU (Camera Engine Unit)** interface on the **Titan Board** to connect an **OV5640 camera**, and display the captured images on an **MIPI LCD screen** via the **RT-Thread LCD framework**.

Key functionalities include:

- Initialize the CEU camera interface to capture real-time video streams
- Configure OV5640 camera parameters (resolution, frame rate, output format)
- Display captured images using the RT-Thread LCD driver
- Support image format conversion (YUV422 $\rightarrow$ RGB565)

3.2 RA8 Series CEU (Camera Engine Unit) Features

The RA8 series MCU integrates a **CEU hardware module** for efficient capture of camera image data, supporting multiple image formats and resolutions, and enabling direct transfer to memory or display interfaces.

CEU Hardware Interface Features

1. **Interface Types**
   - **DVP (Digital Video Port)** interface for connecting CMOS cameras
   - Supports 8/10/12-bit data bus
   - Synchronization signals:
     - **PCLK**: Pixel clock
     - **HSYNC**: Line synchronization
     - **VSYNC**: Frame synchronization
2. **Input Resolution and Frame Rate**
   - Supports VGA, QVGA, SXGA, UXGA, and other common resolutions
   - Frame rates from 1–60 fps, configurable for different applications
3. **Camera Compatibility**
   - Compatible with common CMOS cameras such as OV5640, OV7670
   - Supports auto-initialization and register configuration

Image Formats and Processing Capabilities

1. **Supported Image Formats**
    - **YUV422** (commonly used for video transfer)
    - **RGB565** (suitable for LCD display)
    - **RAW10/RAW12** (for image processing and algorithm development)
2. **Image Processing Features**
    - **Color space conversion**: YUV RGB
    - **Image cropping**: capture only a specific ROI (Region of Interest)
    - **Image scaling**: support proportional scaling up or down
    - **Mirror and flip**: horizontal or vertical mirroring
3. **Hardware Acceleration**
    - CEU contains a hardware processing unit to reduce CPU load
    - Provides fast image format conversion and scaling

DMA Support and Buffer Mechanism

1. **High-Speed DMA Transfer**
    - Works with MCU DMAC for fast memory writes
    - Supports direct writing to frame buffer or LCD buffer
2. **Multi-Buffer Mechanism**
    - Supports double buffering or ring buffers for continuous video capture
    - Reduces frame loss and display latency
3. **Flexible DMA Configuration**
    - Configurable buffer start address and size
    - Supports interrupt triggering and callbacks

Interrupt Mechanism

1. **Interrupt Types**
    - **Frame End Interrupt**: triggered when each frame capture completes
    - **Line End Interrupt (optional)**: triggered at the end of each line capture
    - **Error Interrupt**: including buffer overflow or sync signal anomalies
2. **Interrupt Features**
    - Supports RT-Thread ISR callback registration
    - Can work with DMA for real-time processing and display

Timing and Synchronization Features

1. **Line/Frame Synchronization**
    - HSYNC aligns each line of data
    - VSYNC aligns each frame
2. **Pixel Clock**
    - CEU supports external PCLK or internal clock division
    - Ensures synchronization with camera output to avoid sampling errors
3. **Data Alignment**
    - Supports byte or pixel alignment
    - Automatically adjusts according to image format

Performance and Optimization

1. **High Throughput**
    - DMA + double buffering enables continuous video capture
    - Low CPU usage suitable for real-time applications
2. **Reliability**
    - Sync signal anomalies trigger interrupts
    - Buffer overflow detection
    - Automatic recovery from frame loss
3. **Flexibility**
    - Supports multiple resolutions and format switching
    - Configurable cropping and scaling regions to improve display efficiency

Application Scenarios

- Real-time video display on LCD
- Video capture and image processing algorithm testing
- Embedded vision applications, e.g., surveillance, gesture recognition, robotics vision

3.3 RA8 Series GLCDC Module

Overview

The **GLCDC (Graphics LCD Controller)** is a high-performance graphics controller integrated in RA8 series MCUs, specifically designed to drive TFT/RGB LCD screens. It supports various resolutions, color formats, and graphics processing functions. Combined with RT-Thread ' s **LCD driver framework**, it provides a unified interface for screen initialization, refresh, graphics rendering, and DMA-accelerated operations.

The RA8 **GLCDC** enables image output from either internal MCU memory or external frame buffers to RGB/LCD displays. Key features include:

- **Frame buffer control**: Supports multiple frame buffers for page switching or double-buffered display
- **Color format support**: RGB565, RGB888, ARGB8888, etc.
- **Graphics processing**: Background layers, text/graphics composition, alpha blending, palette mapping
- **Synchronization signal generation**: HSYNC, VSYNC, DE (Data Enable)
- **DMA support**: High-speed data transfer, reducing CPU load
- **Interrupt functionality**: Frame-end and line-end interrupts

Module Architecture

The RA8 GLCDC module mainly consists of the following submodules:

1. **Layer Composition Unit**
   - Supports multiple layer stacking
   - Provides alpha blending, transparency control, and color keying
   - Allows rotation and flipping of layers
2. **Frame Buffer Interface**
   - Supports access to MCU internal SRAM or external memory
   - Provides single or double buffering to ensure continuous display
   - Works with DMA to automatically read image data
3. **DMA Controller**
   - Automatically transfers pixel data to the RGB output port
   - Configurable burst length to improve bandwidth utilization
   - Supports circular transfer, suitable for video or animation scenarios
4. **Timing Generator**
   - Automatically generates HSYNC, VSYNC, and DE signals
   - Supports TTL interface RGB timing
   - Polarity, sync width, and front/back porch timings are configurable
5. **Interrupt and Event Controller**
   - Provides frame-end and line-end interrupts
   - Can be used for page switching, dynamic drawing, or scrolling display
   - Supports interrupts triggered by DMA transfer completion

GLCDC Working Principle

1. **Frame buffer read**
    - GLCDC uses DMA to fetch image data from memory, supporting single or double buffering for continuous display.
2. **Layer composition**
    - Supports multiple layers such as background + foreground + icons/text
    - Provides alpha blending and palette mapping
3. **Pixel timing output**
    - Generates HSYNC/VSYNC/DE signals according to LCD interface requirements
    - Supports RGB parallel interface, TTL interface, or LVDS (depending on board implementation)
4. **Interrupts and events**
    - **Frame-end interrupt (VBlank)**: Can be used to update the next frame
    - **Line-end interrupt**: Useful for scrolling displays or dynamic rendering

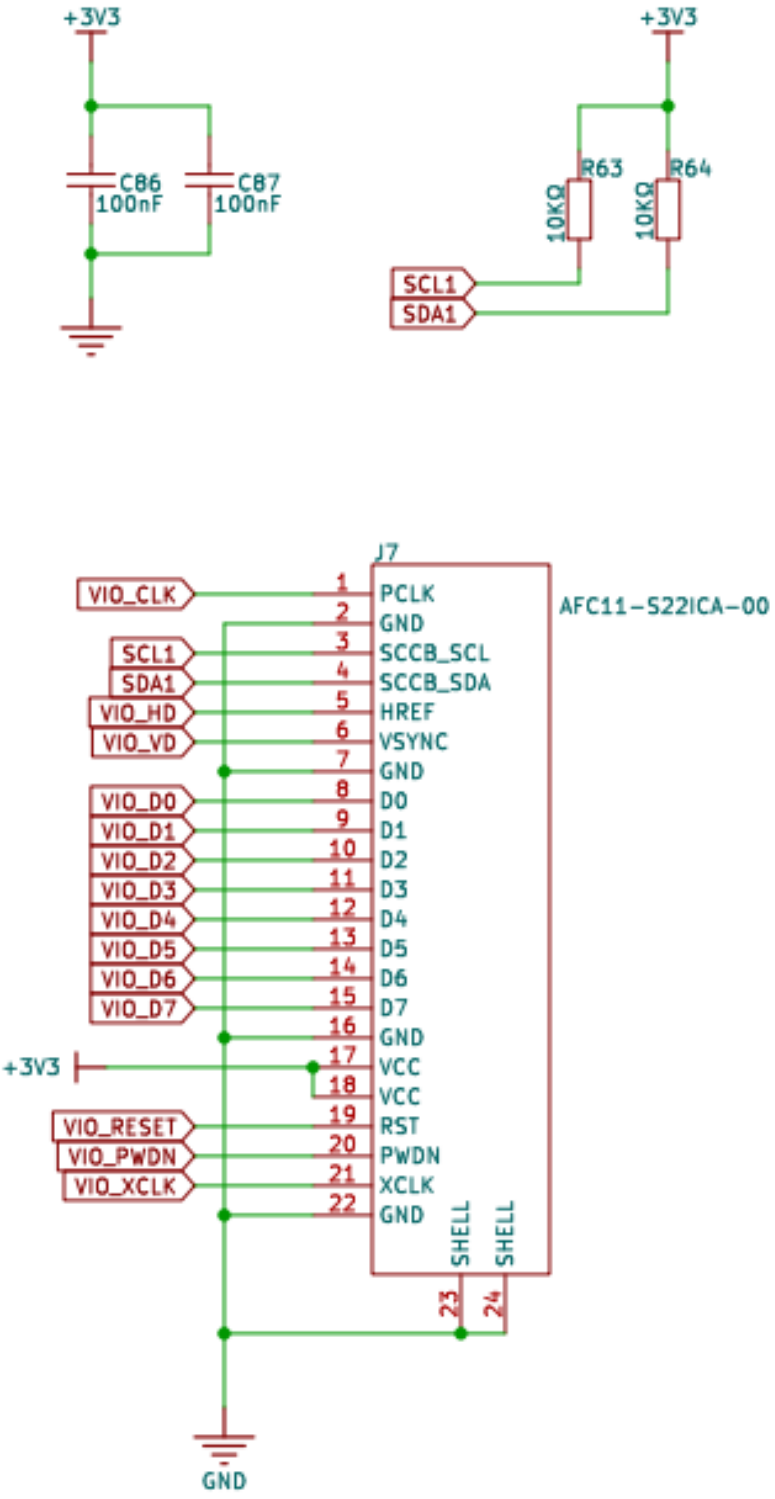
GLCDC Supported Features

Feature Category	Description
Resolution	Up to 1280x800
Color formats	RGB565, RGB888, ARGB8888, etc.
Multi-layer	Background + foreground + icon layers, supports blending
Frame buffer	Single/double buffer mode, DMA improves performance
Palette	8/16-bit palette mapping for color conversion
Sync signals	HSYNC, VSYNC, DE, configurable polarity and timing
DMA support	Automatic memory transfer, CPU-free
Interrupts	Frame-end, line-end interrupts for synchronized refresh
Rotation/Flip	Supports 90°/180°/270° rotation and X/Y flipping

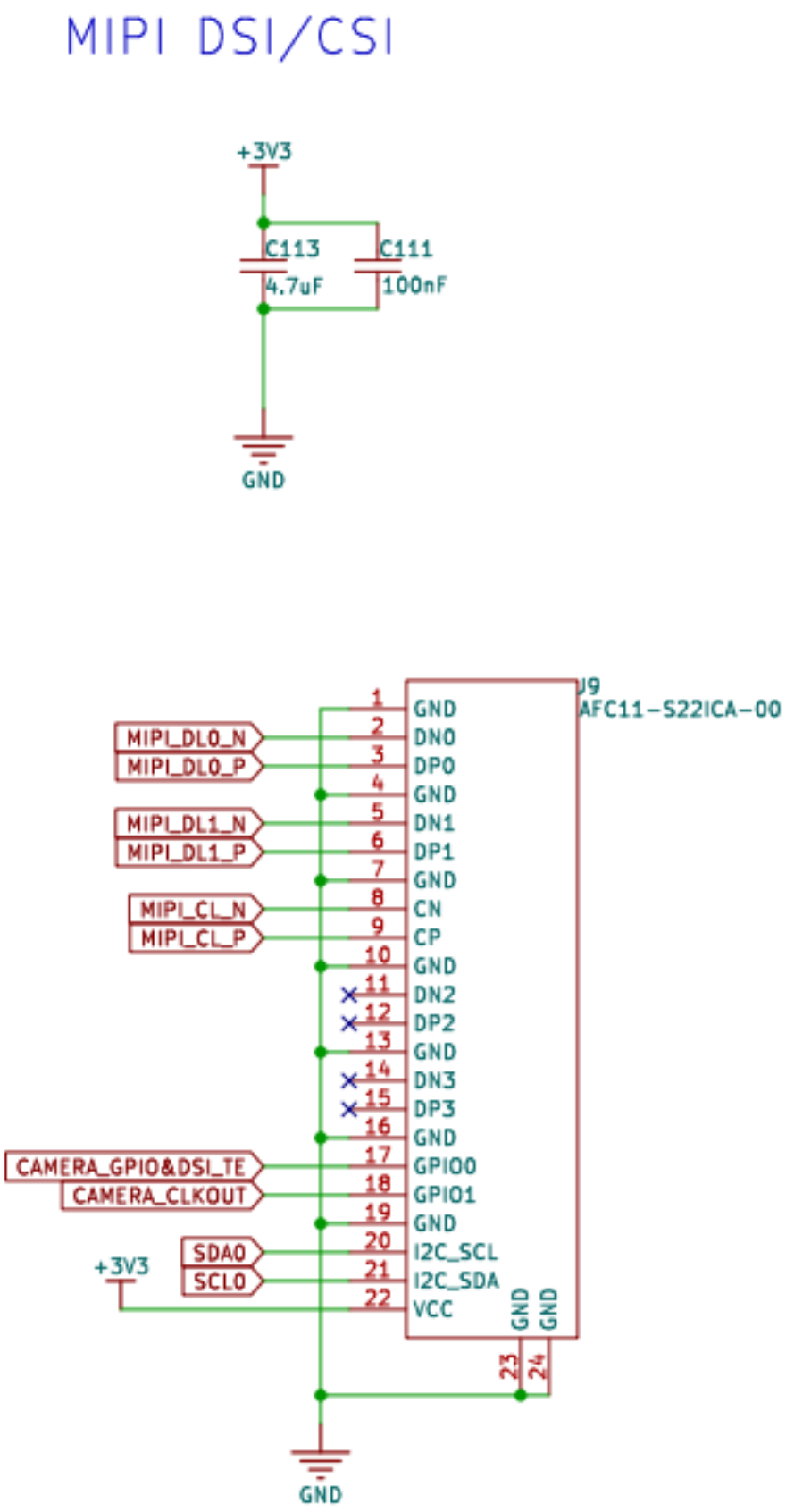
3.4 Hardware Description

- The CEU camera interface is shown in the following figure:

CEU

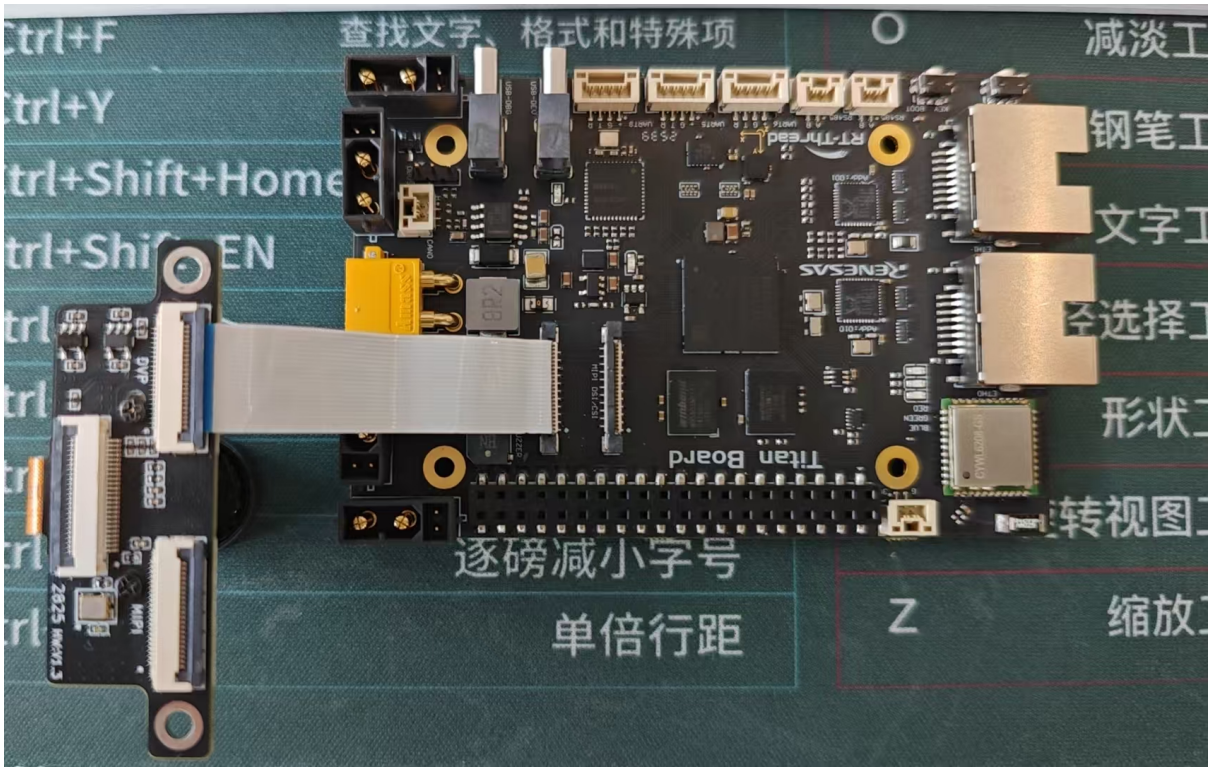

+3V3
C86
100nF
C87
100nF
+3V3
R63
10KΩ
R64
10KΩ
SCL1
SDA1
J7
VIO_CLK
1
PCLK
AFC11-S22ICA-00
2
GND
SCL1
3
SCCB_SCL
SDA1
4
SCCB_SDA
VIO_HD
5
HREF
VIO_VD
6
VSYNC
7
GND
VIO_D0
8
D0
VIO_D1
9
D1
VIO_D2
10
D2
VIO_D3
11
D3
VIO_D4
12
D4
VIO_D5
13
D5
VIO_D6
14
D6
VIO_D7
15
D7
16
GND
+3V3
17
VCC
18
VCC
VIO_RESET
19
RST
VIO_PWDN
20
PWDN
VIO_XCLK
21
XCLK
22
GND
SHELL
SHELL
23
24
GND

- The MIPI LCD interface is shown in the following figure:


MIPI DSI/CSI
+3V3
C113
4.7uF
C111
100nF
GND
U9
AFC11-S22ICA-00
1 GND
2 DN0
3 DP0
4 GND
5 DN1
6 DP1
7 GND
8 CN
9 CP
10 GND
11 DN2
12 DP2
13 GND
14 DN3
15 DP3
16 GND
17 GPIO0
18 GPIO1
19 GND
20 I2C_SCL
21 I2C_SDA
22 VCC
23 GND
24 GND
MIPI_DL0_N
MIPI_DL0_P
MIPI_DL1_N
MIPI_DL1_P
MIPI_CL_N
MIPI_CL_P
CAMERA_GPIO&DSI_TE
CAMERA_CLKOUT
SDA0
SCL0
+3V3
GND

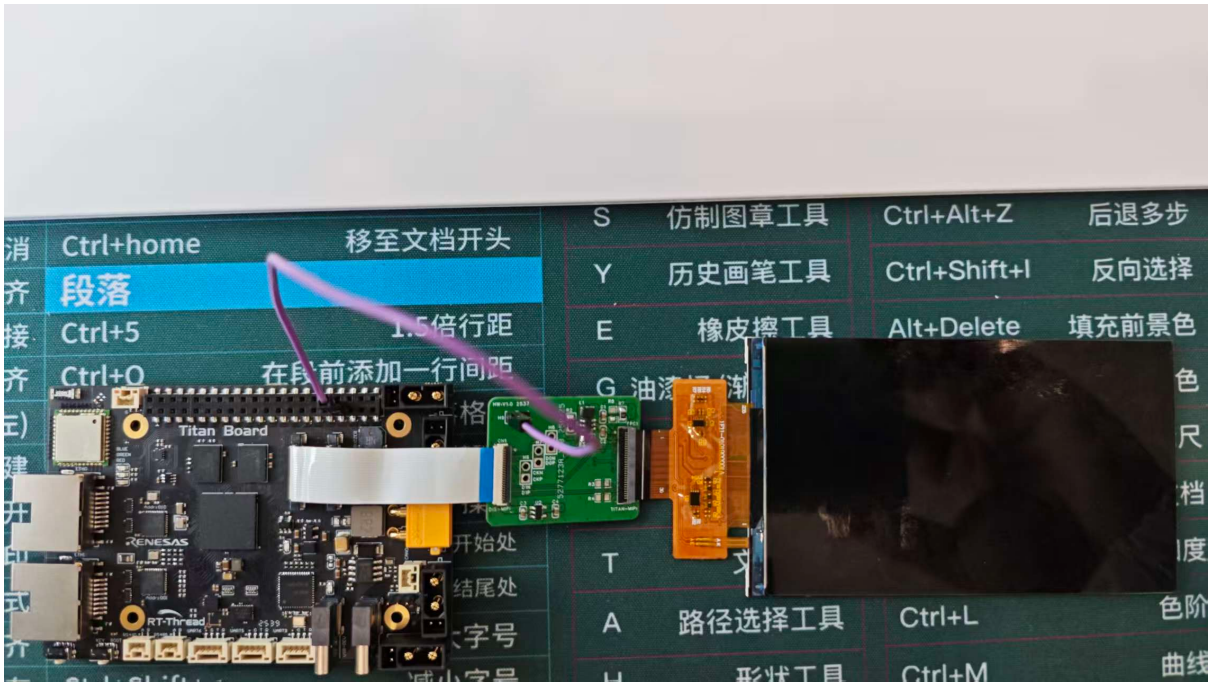
- The connection method of the CEU camera is as shown in the following figure:

Use a **22-pin reverse FFC cable** to connect the development board’s **CEU_CAM** connector to the camera adapter board’s **DVP** connector.



- The MIPI LCD connection method is as shown in the following diagram:

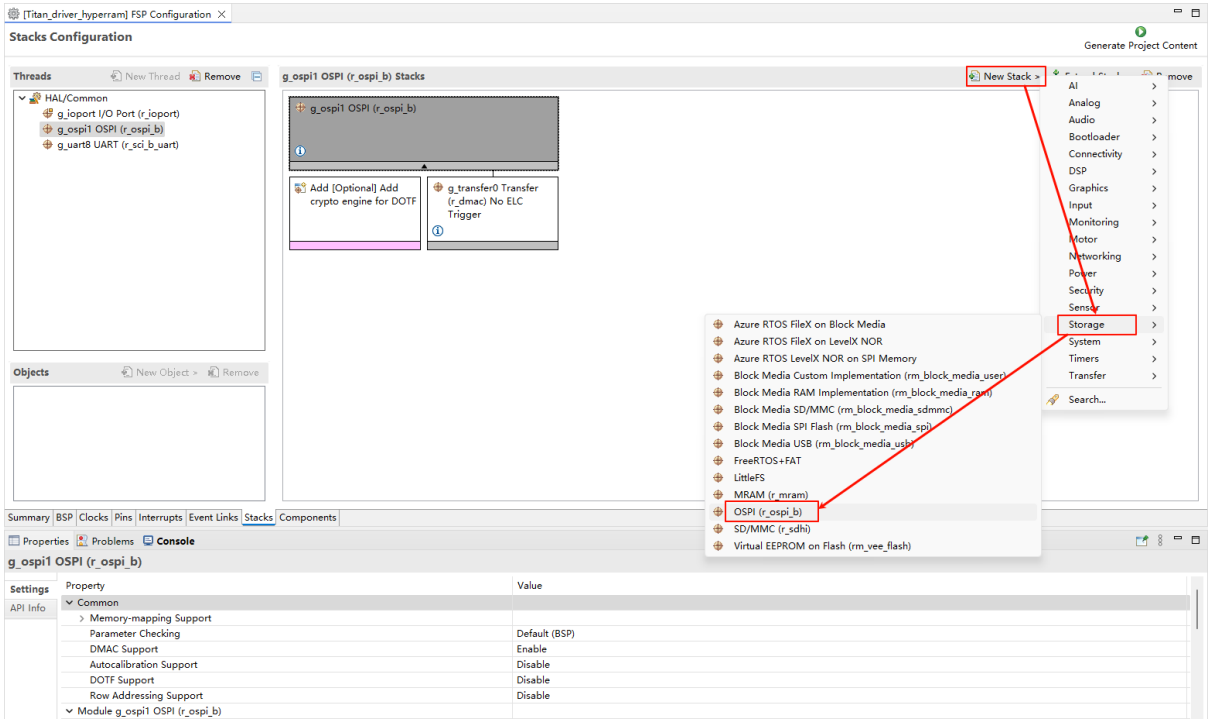
Use a **22-pin reverse FFC cable** to connect the development board’s **MIPI DSI/CSI** connector to the display adapter board’s **DIS-MIPI** connector. Then connect the **MIPI display** to the adapter board’s **TITAN-MIPI** connector.



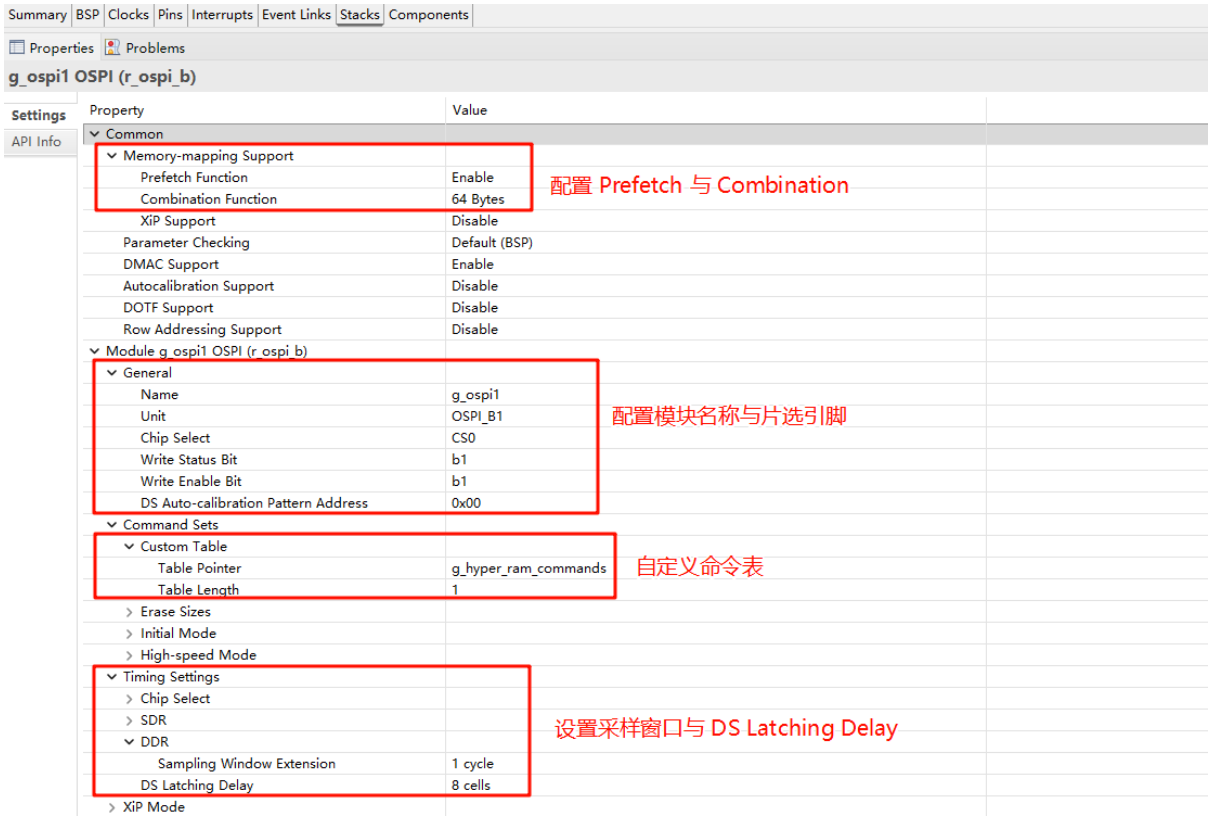
3.5 FSP Configuration

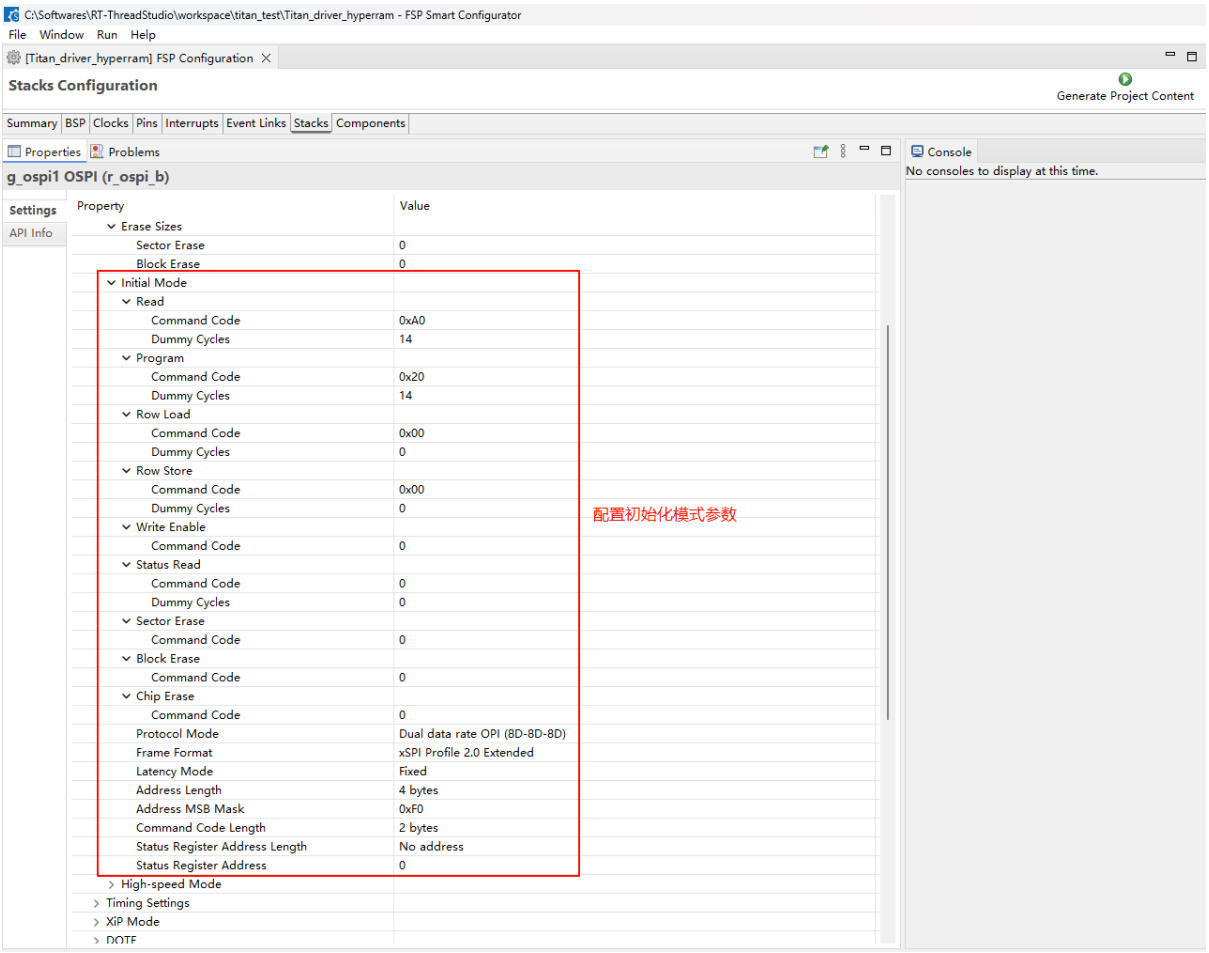
HyperRAM Configuration

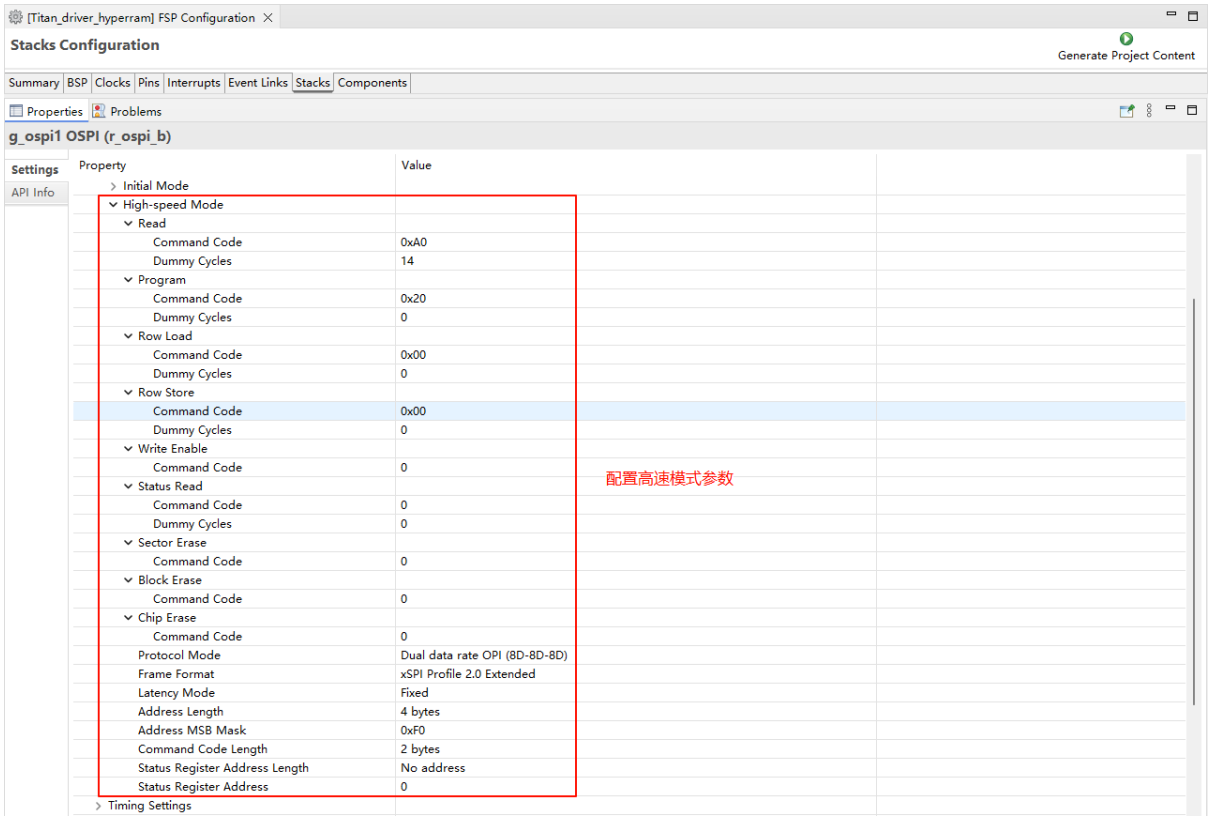
- Create a r_ospi_b stack:



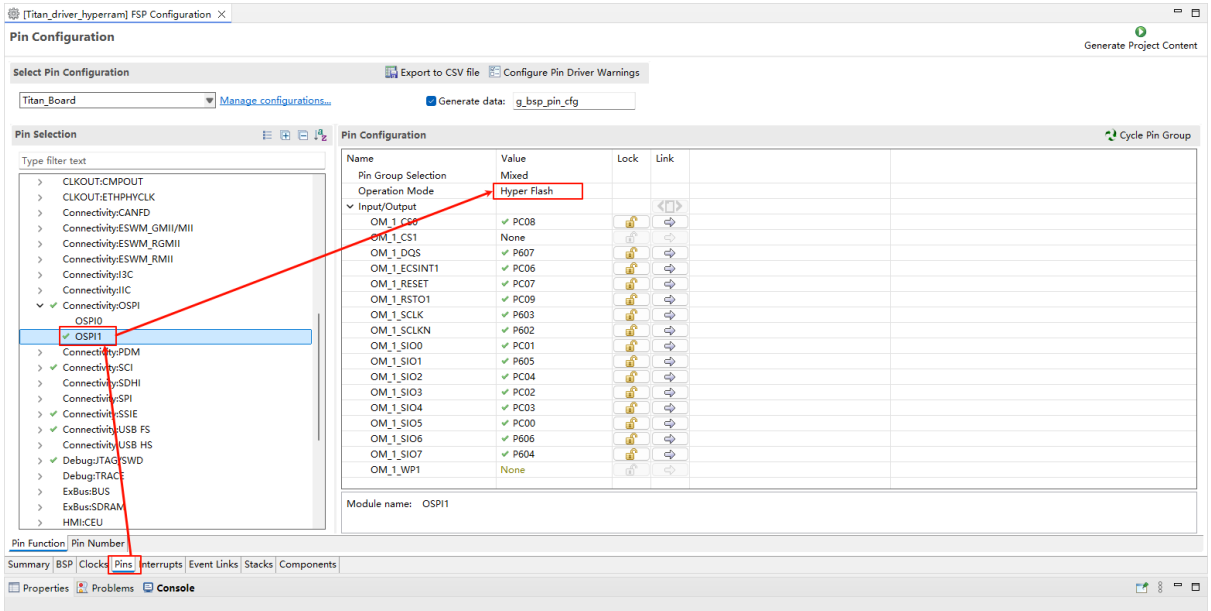
- Configure the r_ospi_b stack:


Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems
g_ospi1 OSPI (r_ospi_b)
Settings
API Info
Property
Value
Common
Memory-mapping Support
Prefetch Function
Enable
Combination Function
64 Bytes
XiP Support
Disable
Parameter Checking
Default (BSP)
DMAC Support
Enable
Autocalibration Support
Disable
DOTF Support
Disable
Row Addressing Support
Disable
Module g_ospi1 OSPI (r_ospi_b)
General
Name
g_ospi1
Unit
OSPI_B1
Chip Select
CS0
Write Status Bit
b1
Write Enable Bit
b1
DS Auto-calibration Pattern Address
0x00
Command Sets
Custom Table
Table Pointer
g_hyper_ram_commands
Table Length
1
Erase Sizes
Initial Mode
High-speed Mode
Timing Settings
Chip Select
SDR
DDR
Sampling Window Extension
1 cycle
DS Latching Delay
8 cells
XiP Mode
配置 Prefetch 与 Combination
配置模块名称与片选引脚
自定义命令表
设置采样窗口与 DS Latching Delay

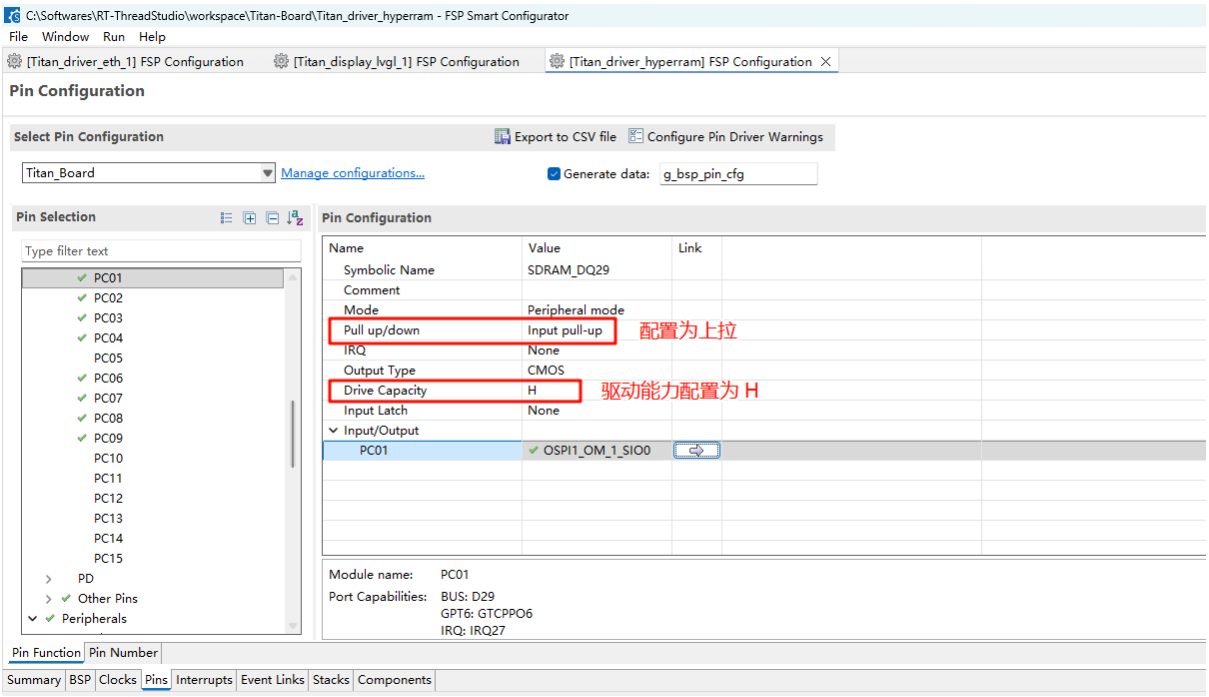

C:\Softwares\RT-ThreadStudio\workspace\titan_test\Titan_driver_hyperram - FSP Smart Configurator
File Window Run Help
[Titan_driver_hyperram] FSP Configuration
Stacks Configuration
Generate Project Content
Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems
g_ospi1 OSPI (r_ospi_b)
Settings
API Info
Property
Value
Erase Sizes
Sector Erase 0
Block Erase 0
Initial Mode
Read
Command Code 0xA0
Dummy Cycles 14
Program
Command Code 0x20
Dummy Cycles 14
Row Load
Command Code 0x00
Dummy Cycles 0
Row Store
Command Code 0x00
Dummy Cycles 0
Write Enable
Command Code 0
Status Read
Command Code 0
Dummy Cycles 0
Sector Erase
Command Code 0
Block Erase
Command Code 0
Chip Erase
Command Code 0
Protocol Mode Dual data rate OPI (8D-8D-8D)
Frame Format xSPI Profile 2.0 Extended
Latency Mode Fixed
Address Length 4 bytes
Address MSB Mask 0xF0
Command Code Length 2 bytes
Status Register Address Length No address
Status Register Address 0
High-speed Mode
Timing Settings
XiP Mode
DOTF
Console
No consoles to display at this time.
配置初始化模式参数



- HyperRAM pin configuration:

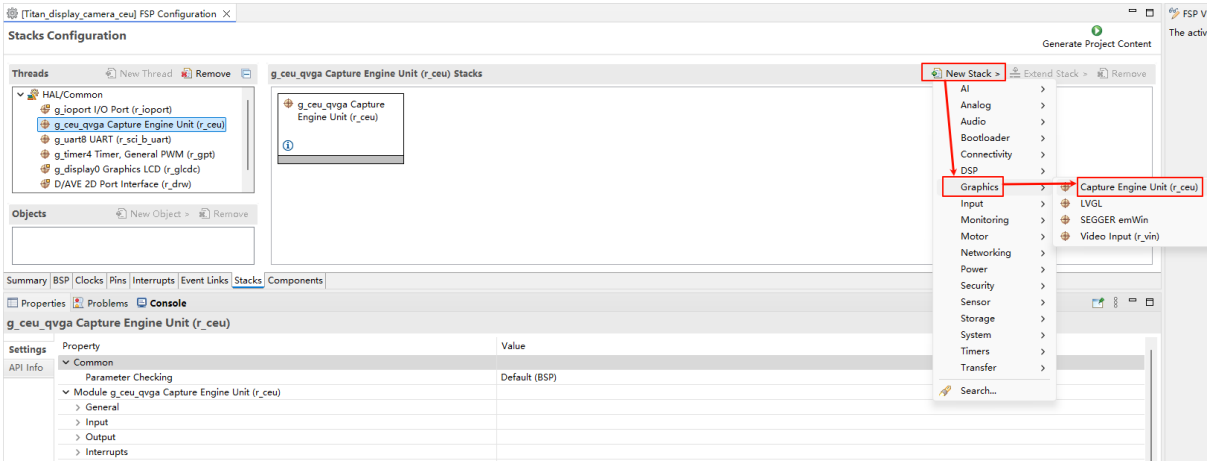


- The drive capability of all pins related to HyperRAM should be configured as H, and OM_1_SIO0 to OM_1_SIO7 need to be configured as Input pull-up.

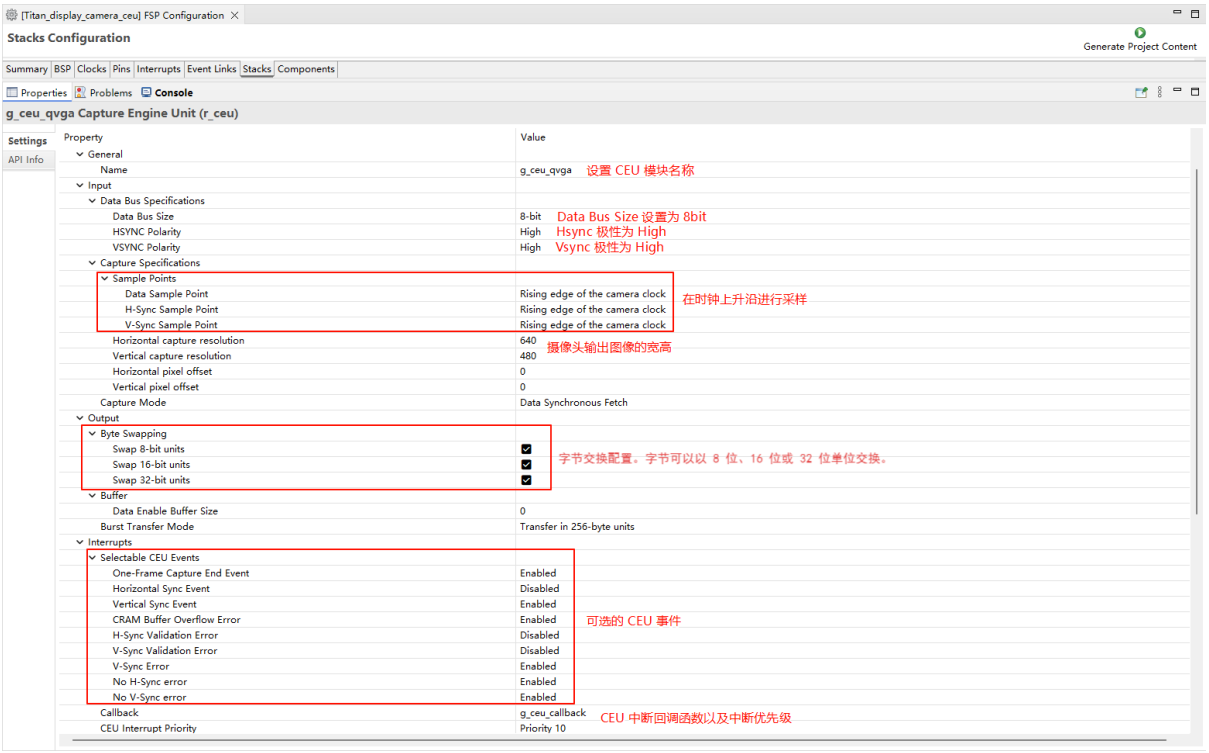


CEU Configuration

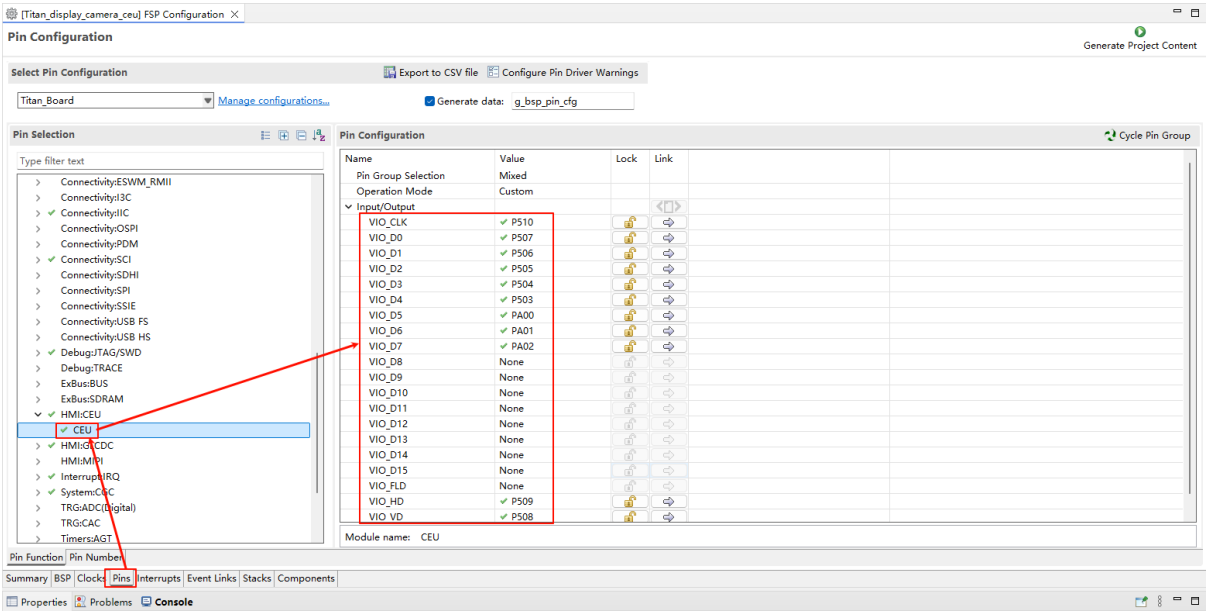
- Create a r_ceu stack：



- Configure CEU：



- Configure CEU pins：

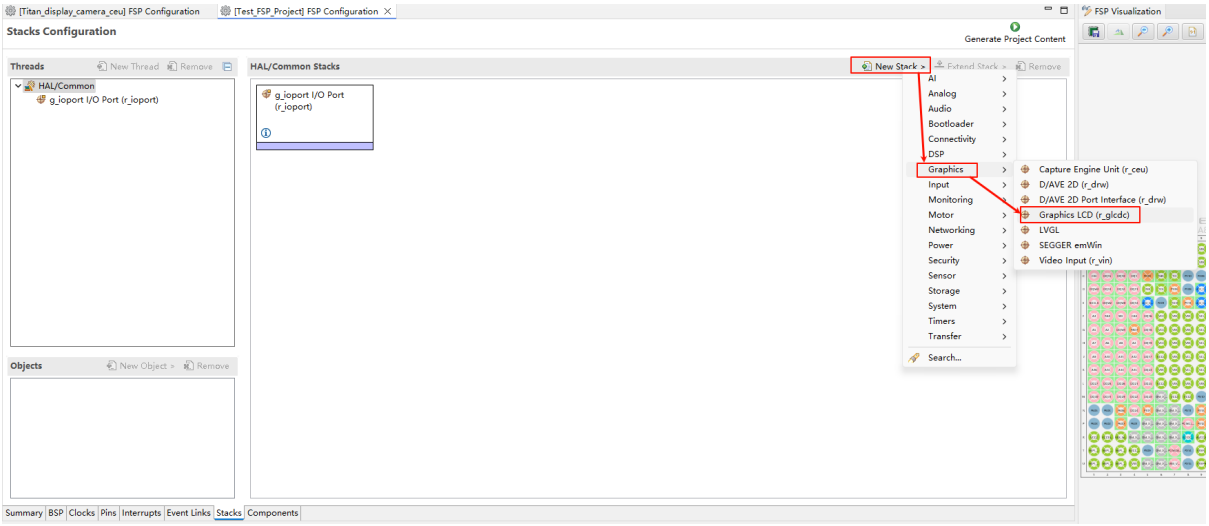


D/AVE 2D Configuration

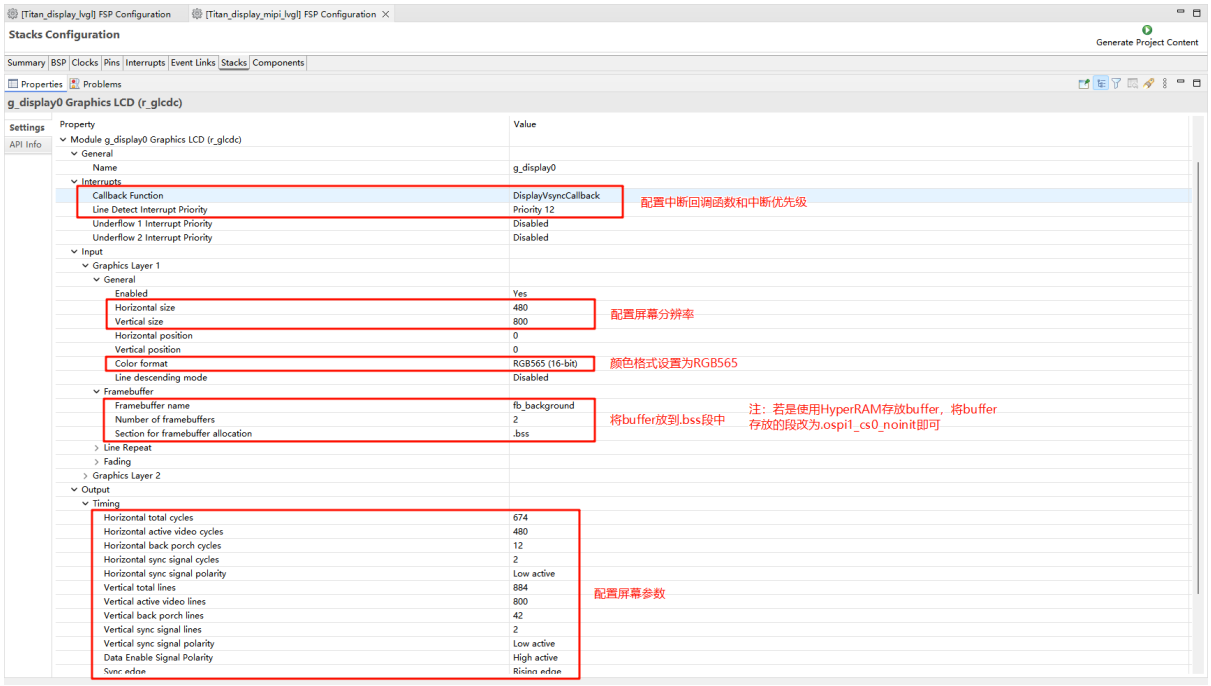
- Create a r_drw stack:

GLCDC Configuration

- Create a r_glcdc stack:



- Configure interrupt callback and graph layer 1:

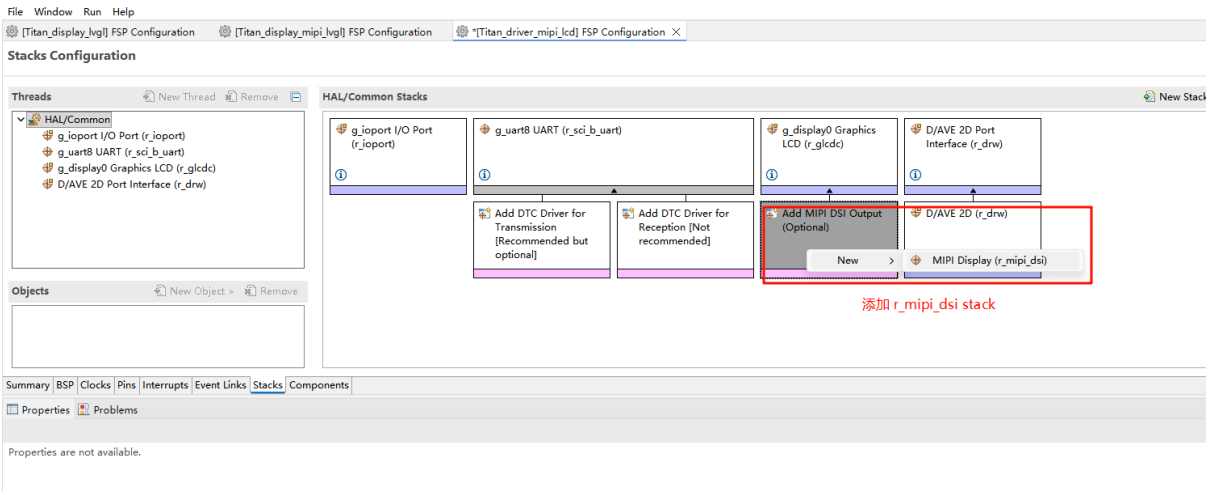


- Configure Output、CLUT、TCON and Dithering:

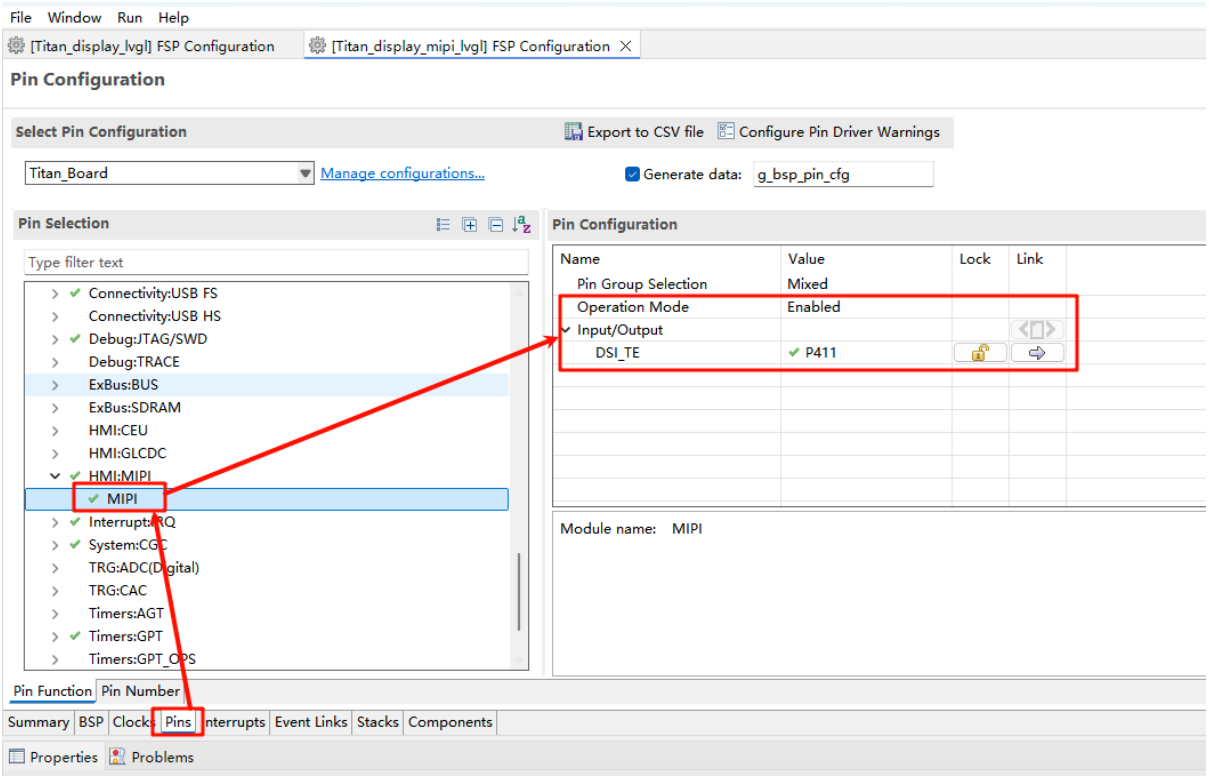
Background	
CLUT	
Enabled	No
Size	256
TCON	
Hsync pin select	LCD_TCON1
Vsync pin select	LCD_TCON0
Data enable (DE) pin select	LCD_TCON2
Panel clock source	Internal clock (GLCDCLK)
Panel clock division ratio	1/8
Dithering	
Enabled	No
Mode	Truncate
Pattern A	Pattern 11
Pattern B	Pattern 11
Pattern C	Pattern 11
Pattern D	Pattern 11

MIPI DSI Configuration

- Create a r_mipi_dsi stack.

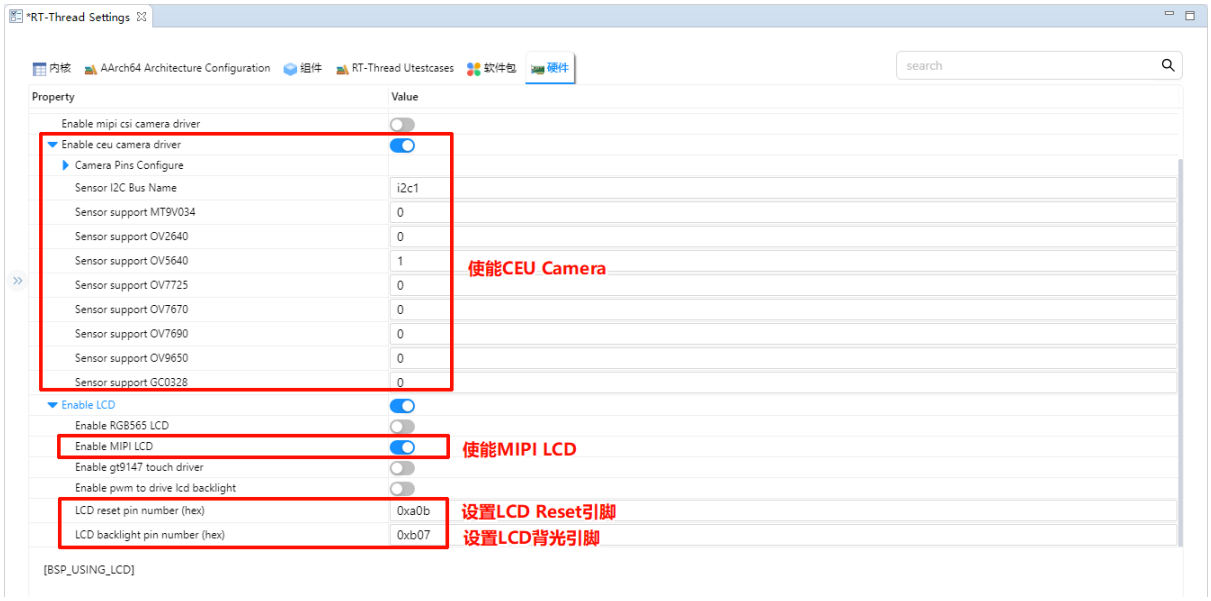


- Configure MIPI DSI_TE pins.



3.6 RT-Thread Settings Configuration

- Enable CUE camera, using i2c1 and ov5640 camera. Enable MIPI LCD, configure reset and backlight pins.



3.7 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio’s package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board’s USB-DBG interface to the PC and download the firmware to the development board.

3.8 Run Effect

After resetting the Titan Board, the terminal will output the following message:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Feb 26 2026 16:50:25
 2006 - 2024 Copyright by RT-Thread team
[I/I2C] I2C bus [i2c1] registered
[D/drv_lcd]
lcd framebuffer address:0x70096000
[D/mipi_cfg] initialize mipi dsi configs


Hello RT-Thread!
============================================================
This example project is an display with ceu camera routine!
============================================================

msh >i2c probe camera id:0x78
[I/sensor] set framesize:640X480
[I/sensor] Open CEU device
```

Here is the image displayed on the LCD screen:

4. MIPI DSI LVGL Usage Instructions

English | Chinese

4.1 Instruction

This example is based on the **LVGL** graphics library and demonstrates a sample graphical user interface (GUI) project designed for embedded devices. It integrates both display and input drivers, showcasing the complete workflow from low-level driver adaptation to high-level interface construction. Through this example, developers can not only learn how to port and configure LVGL in an embedded environment but also gain practical knowledge of key techniques such as interface layout, event handling, and theme customization, thereby laying a solid foundation for building real product interfaces.

4.2 LVGL Instruction

LVGL (Light and Versatile Graphics Library) is an open-source embedded GUI (Graphical User Interface) development framework. It was designed with the goal of providing smooth and modern graphical interfaces for resource-constrained embedded devices. To achieve this, it has been highly optimized in terms of runtime efficiency, memory usage, and portability. Whether running on simple low-end MCUs or more powerful MPU platforms, LVGL can operate efficiently while offering a wide range of graphical widgets and interactive features.

Key Features

1. **Lightweight** The greatest advantage of LVGL is its lightweight nature, requiring very few resources. It can still run smoothly on microcontrollers with only a few dozen KB of memory. The framework itself has minimal memory overhead, and its rendering algorithms are carefully optimized to ensure low CPU usage while delivering good visual performance on limited hardware. This makes LVGL particularly suitable for resource-sensitive scenarios such as low-power devices, wearables, and home appliance control panels.
2. **Cross-Platform** LVGL stands out for its cross-platform capability. It can run on multiple operating systems including FreeRTOS, RT-Thread, Zephyr, and Linux, or even directly on bare-metal systems. Developers only need to provide display and input driver interfaces for LVGL, and porting can be completed quickly, enabling the same UI code to be reused across different hardware platforms. This flexibility significantly reduces development costs and makes LVGL a universal solution for embedded GUIs.
3. **Rich Widgets** LVGL comes with a wide variety of built-in GUI widgets such as buttons, labels, sliders, progress bars, checkboxes, lists, tables, and charts. These components cover nearly all common HMI (Human-Machine Interface) requirements, sparing developers from designing and implementing them from scratch and thus shortening the development cycle. LVGL also allows extending existing widgets to create custom components, making it adaptable for both simple numeric displays and complex graphical control interfaces.

4. **Advanced Rendering Capabilities** In terms of visual presentation, LVGL offers a rich set of rendering features. It supports anti-aliasing, transparency, gradients, shadows, borders, and rounded corners, enabling modern and visually appealing interfaces. Additionally, LVGL includes a built-in animation system with multiple easing functions, making smooth movements, transitions, and dynamic effects easy to implement. These capabilities enhance not only functionality but also the overall user experience.
5. **Input Device Support** LVGL supports multiple input device types, including touchscreens, capacitive panels, mice, keyboards, and rotary encoders, with even multi-touch interaction available. A unified input interface layer is provided, so developers only need to implement low-level driver adaptation to pass input events into LVGL' s event system. This allows them to focus on interface logic instead of spending excessive time on input handling.
6. **Internationalization and Multilanguage Support** LVGL has excellent internationalization support. It uses UTF-8 encoding, which can handle nearly all character sets. It also supports bidirectional text rendering, ensuring correct display of languages such as Arabic and Hebrew that are written right-to-left. This makes LVGL suitable for globally targeted products, allowing users in different countries and regions to enjoy a localized and seamless experience.
7. **Extensibility** LVGL provides a flexible theming and styling system, enabling developers to easily customize the appearance of widgets to achieve various UI styles. By switching themes, the overall look and feel of the interface can be changed quickly. Furthermore, LVGL can integrate with third-party graphics libraries, file systems, and image decoders to expand its functionality. For instance, developers can use a file system to load external fonts and images, or integrate JPEG/PNG decoders to display complex images.

Application Scenarios

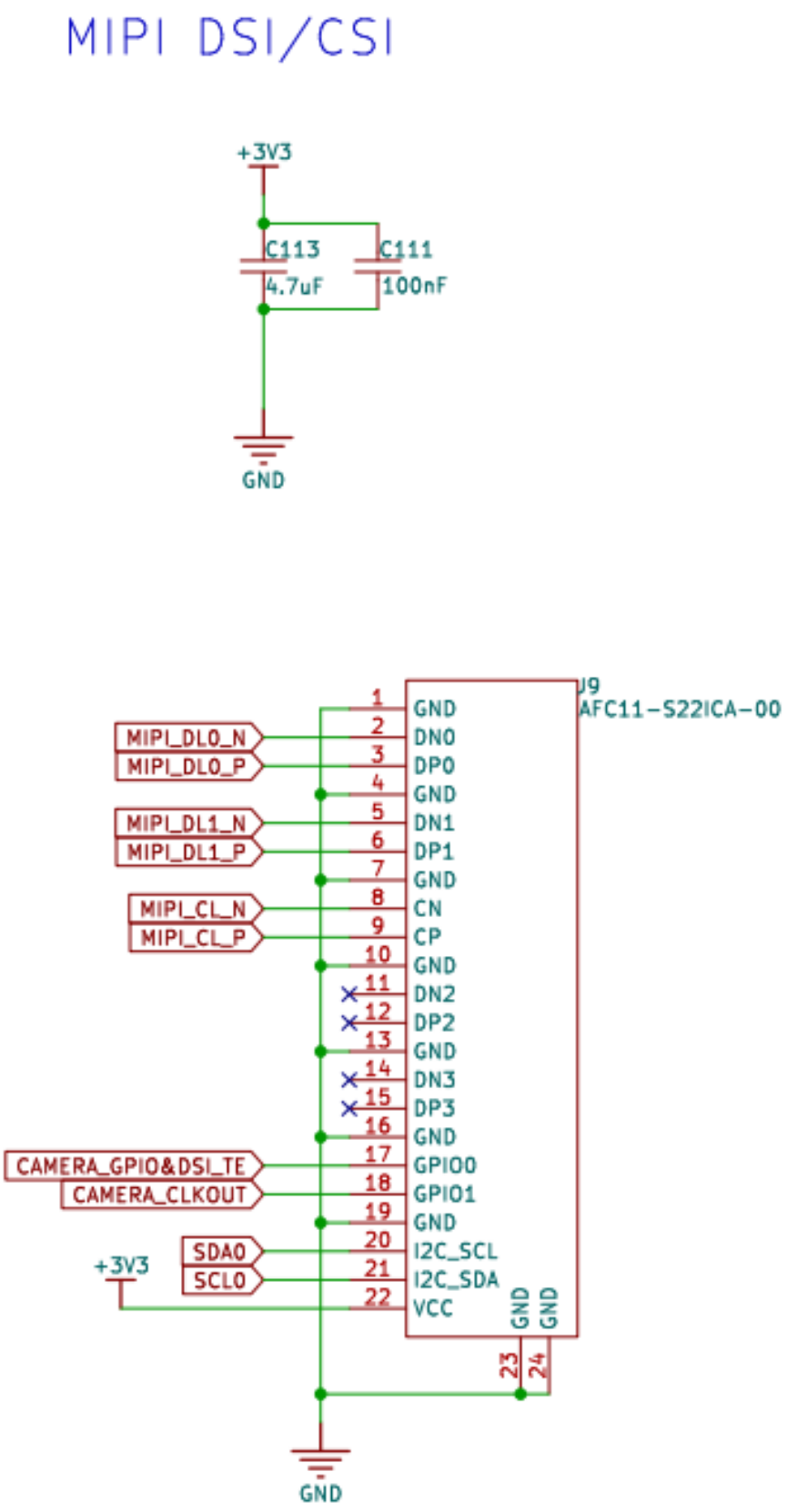
LVGL has a very broad range of real-world applications. In consumer electronics, it is commonly used in smart home control panels, appliance displays, smartwatches, and fitness devices, all of which require user-friendly interfaces on constrained hardware. In industrial control, LVGL is applied to HMIs (Human-Machine Interfaces) and various instruments, helping users intuitively monitor and control equipment. In automotive electronics, LVGL can power infotainment systems, passenger entertainment displays, and even instrument clusters. In the medical field, it is well-suited for small-scale displays, such as handheld diagnostic tools and portable monitoring devices.

Ecosystem and Community

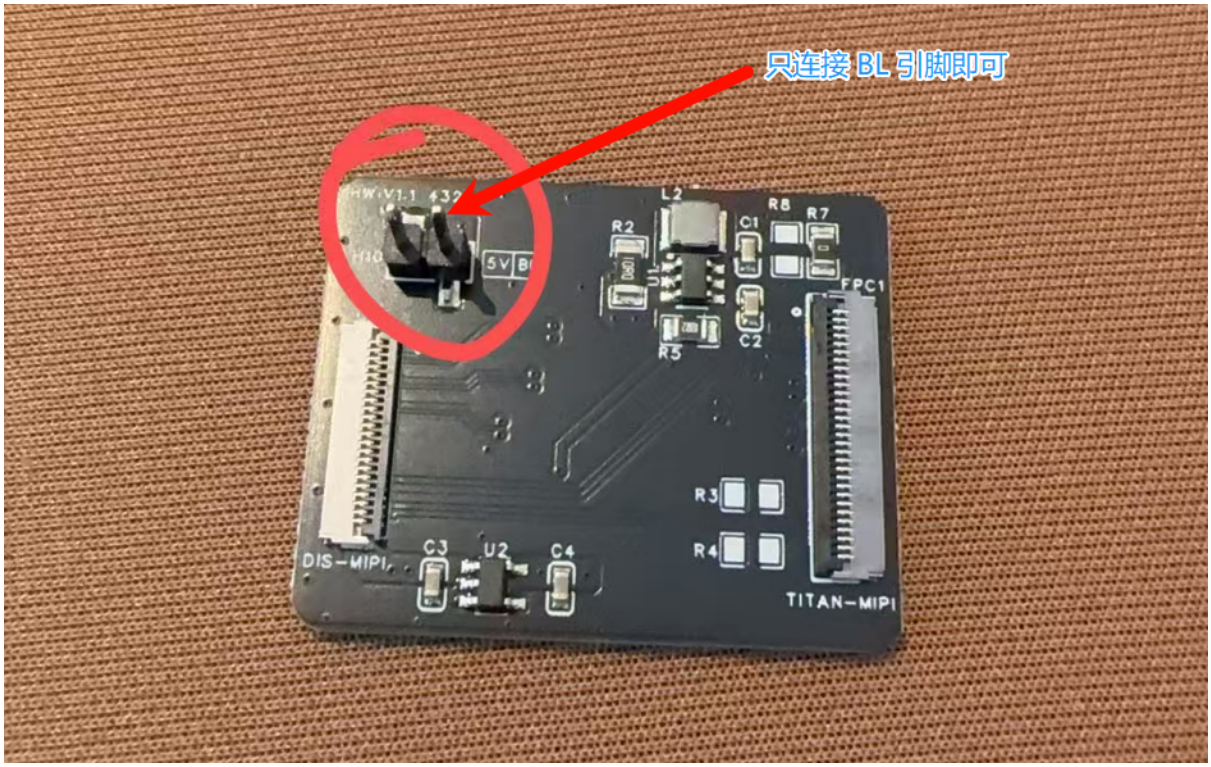
As an open-source project under the **MIT License**, LVGL is equally suitable for personal learning and free use in commercial products. Around LVGL, an active ecosystem has formed. Official tools such as **SquareLine Studio** provide visual drag-and-drop design with exportable LVGL code, greatly improving development efficiency. The **LVGL Simulator** enables developers to debug interfaces directly on a PC without repeatedly flashing target hardware. A vibrant global developer community contributes numerous open-source widgets, themes, and porting examples, offering valuable resources and support to both beginners and enterprise developers.

4.3 Hardware Description

- The following figure shows the MIPI DSI/CSI interface on the Titan Board. To connect the MIPI DSI screen, another adapter board is needed.


MIPI DSI/CSI
+3V3
C113
4.7uF
C111
100nF
GND
U9
AFC11-S22ICA-00
MIPI_DL0_N
MIPI_DL0_P
MIPI_DL1_N
MIPI_DL1_P
MIPI_CL_N
MIPI_CL_P
CAMERA_GPIO&DSI_TE
CAMERA_CLKOUT
SDA0
SCL0
+3V3
1 GND
2 DN0
3 DP0
4 GND
5 DN1
6 DP1
7 GND
8 CN
9 CP
10 GND
11 DN2
12 DP2
13 GND
14 DN3
15 DP3
16 GND
17 GPIO0
18 GPIO1
19 GND
20 I2C_SCL
21 I2C_SDA
22 VCC
GND
GND
23
24
GND

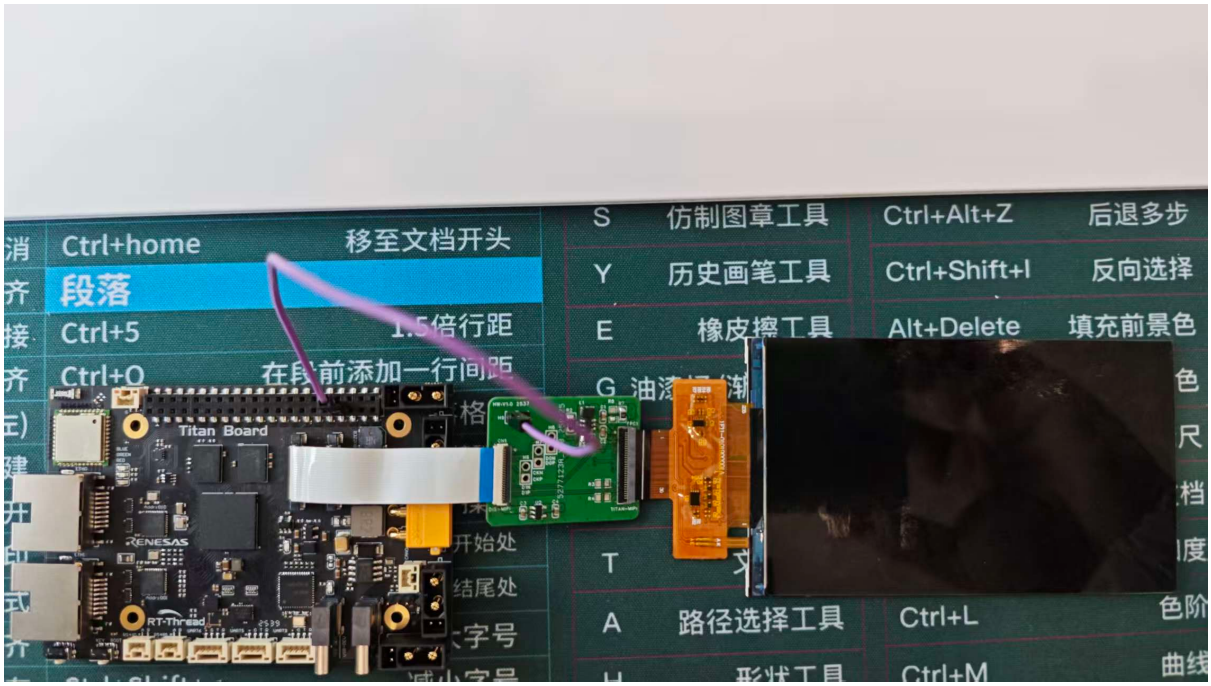
- In this example, a Dupont wire is required to connect the **BL** pin on the display adapter board to the **PB07** pin (the GPIO pin used to provide backlight can be changed as needed).



- The MIPI LCD connection method is as follows:

Display model: **Guanxian TL043WVV02CT**

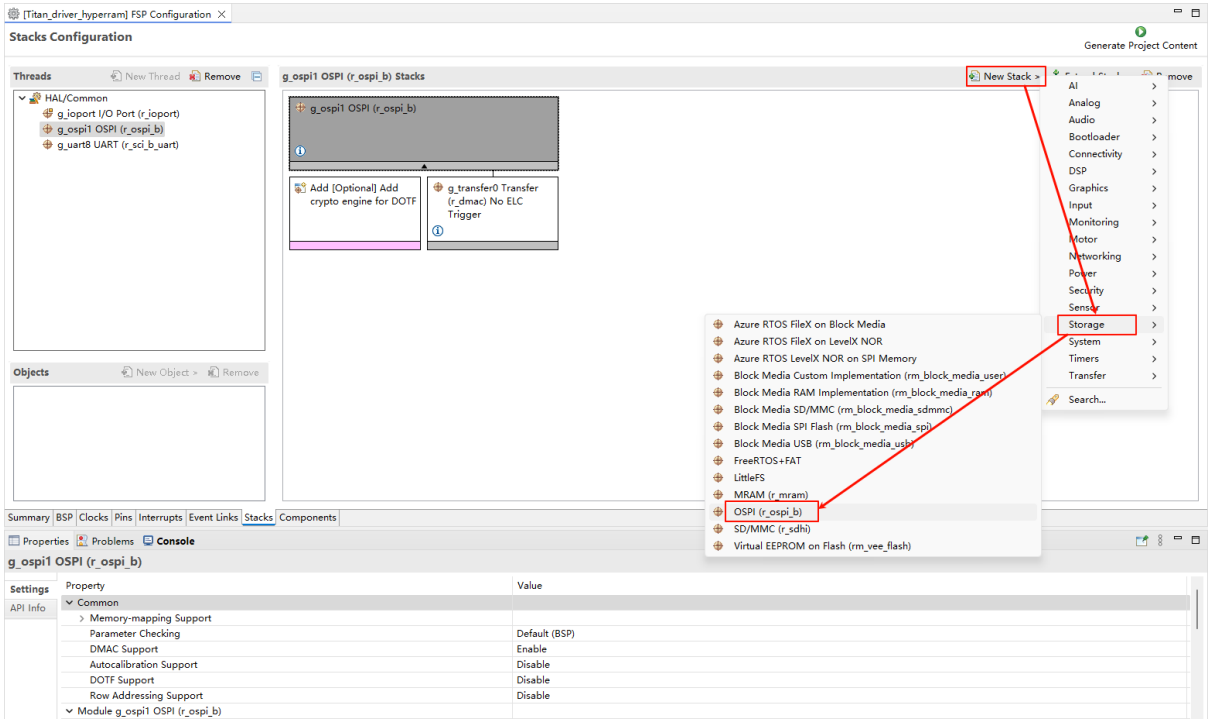
Use a **22-pin reverse FFC cable** to connect the development board's **MIPI DSI/CSI** connector to the display adapter board's **DIS-MIPI** connector. Then connect the **MIPI display** to the adapter board's **TITAN-MIPI** connector.



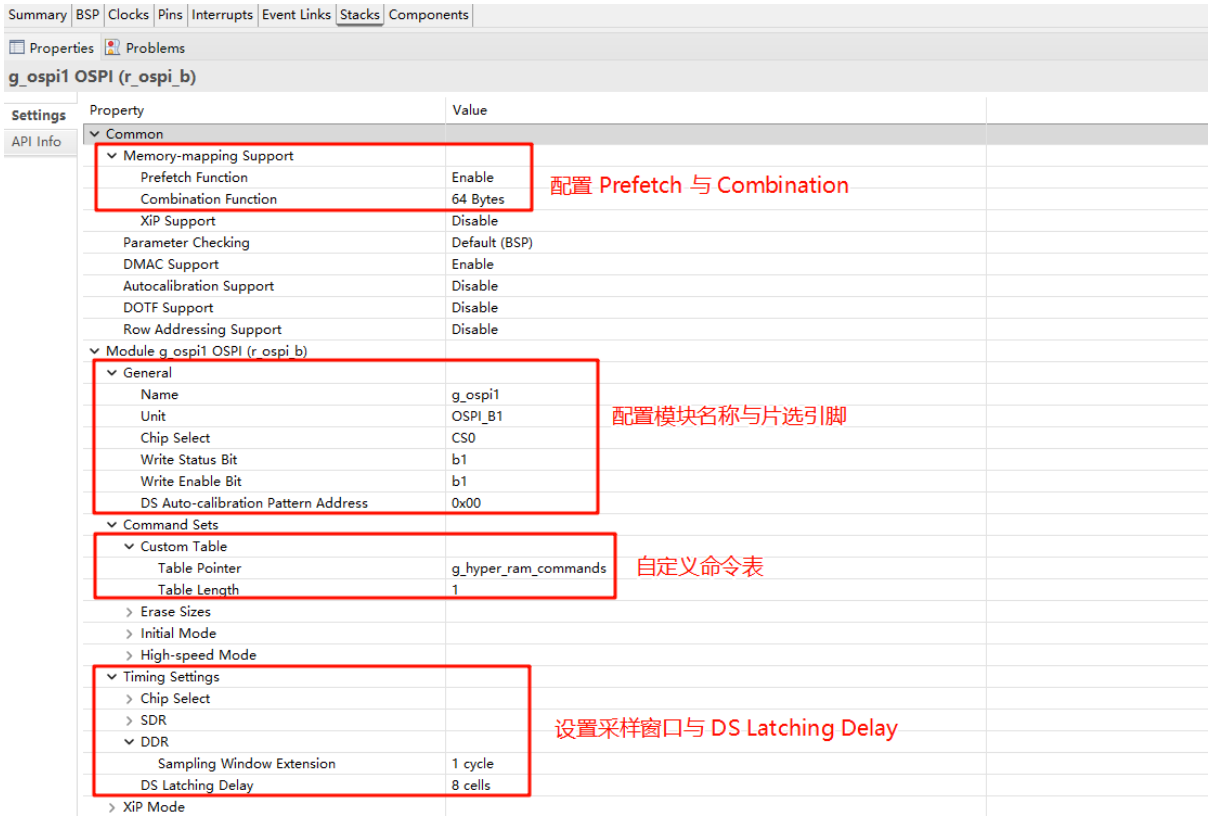
4.4 FSP Configuration

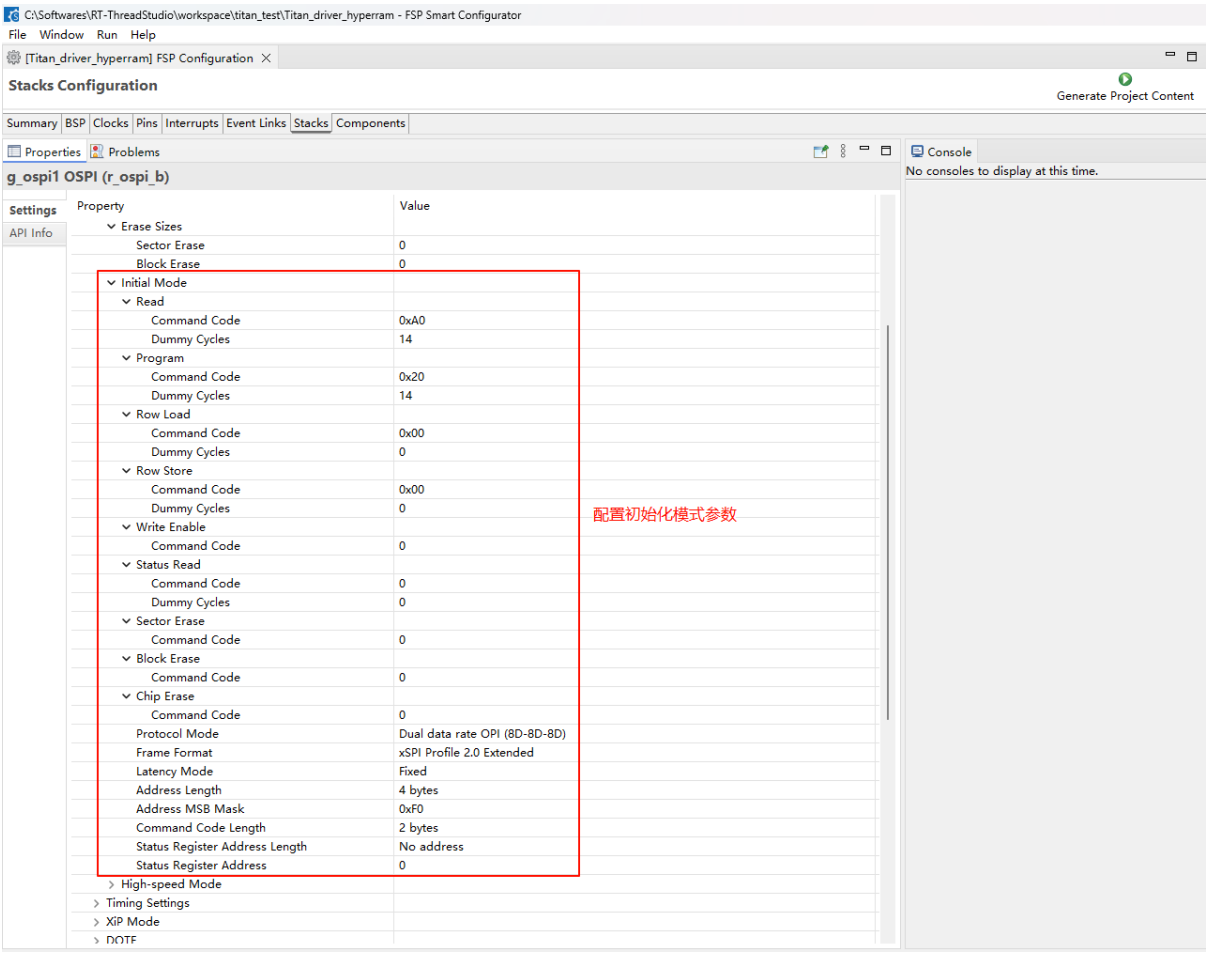
HyperRAM 配置

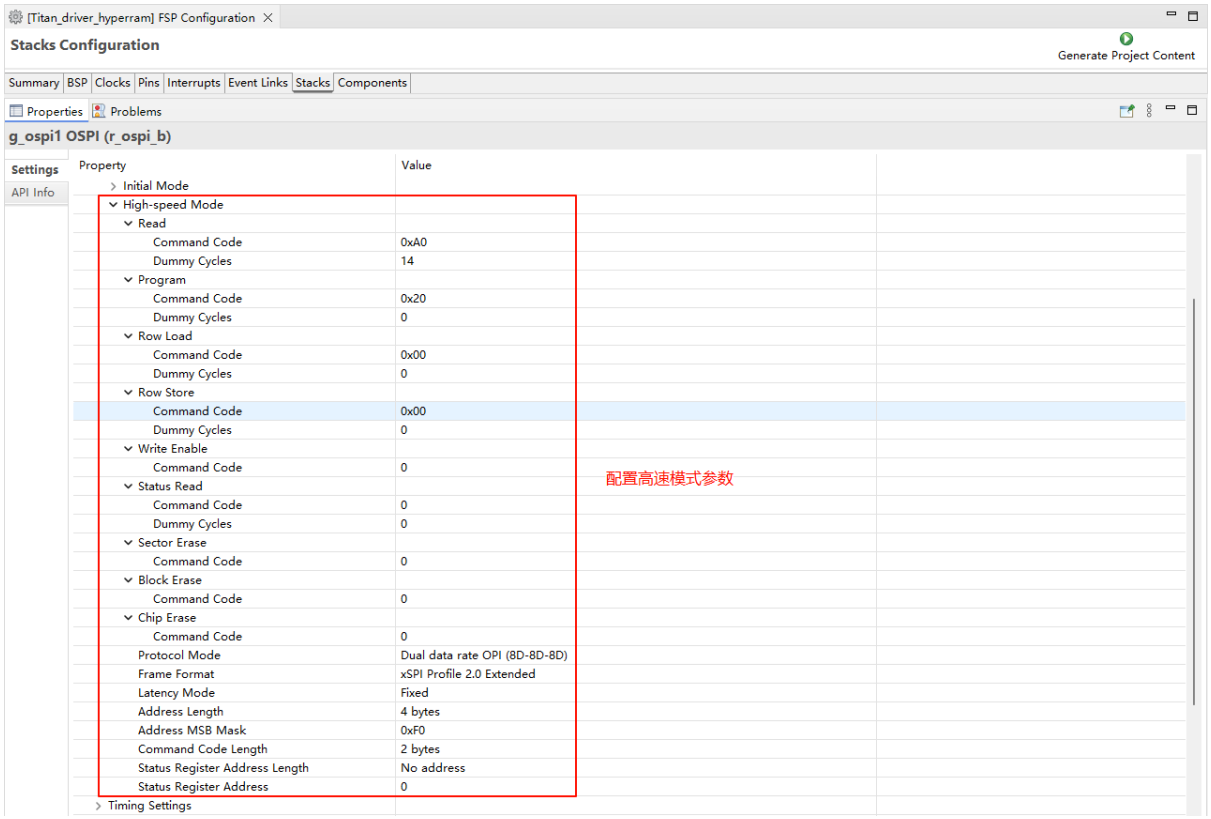
- Create a r_ospi_b stack：



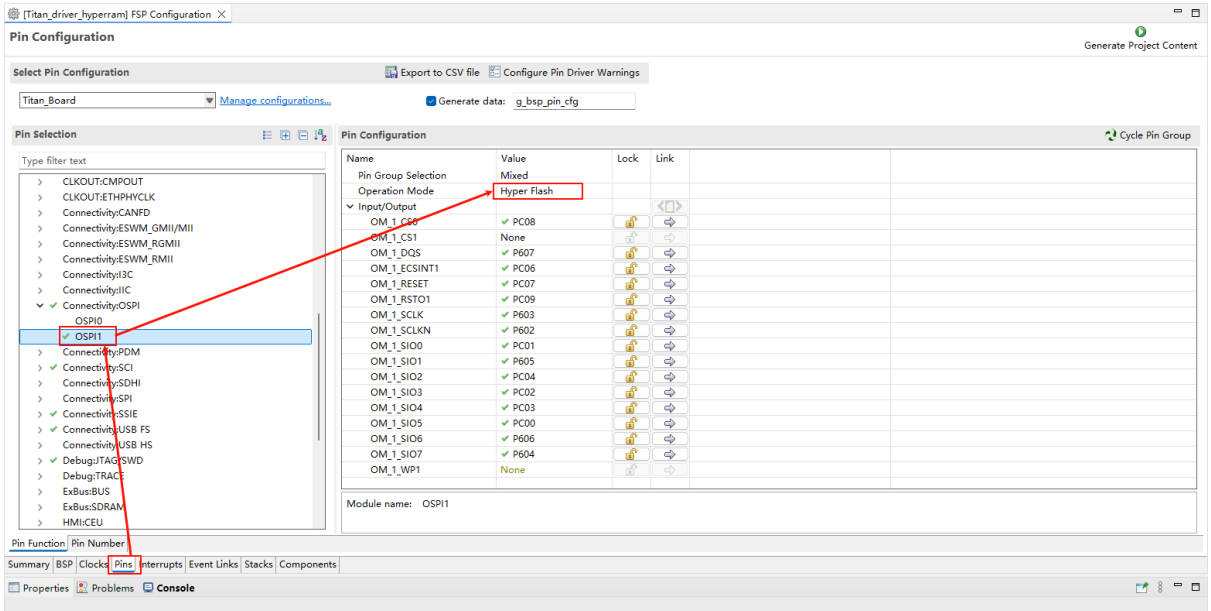
- Configure r_ospi_b stack：


Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems
g_ospi1 OSPI (r_ospi_b)
Settings
API Info
Property
Value
Common
Memory-mapping Support
Prefetch Function
Enable
Combination Function
64 Bytes
XiP Support
Disable
Parameter Checking
Default (BSP)
DMAC Support
Enable
Autocalibration Support
Disable
DOTF Support
Disable
Row Addressing Support
Disable
Module g_ospi1 OSPI (r_ospi_b)
General
Name
g_ospi1
Unit
OSPI_B1
Chip Select
CS0
Write Status Bit
b1
Write Enable Bit
b1
DS Auto-calibration Pattern Address
0x00
Command Sets
Custom Table
Table Pointer
g_hyper_ram_commands
Table Length
1
Erase Sizes
Initial Mode
High-speed Mode
Timing Settings
Chip Select
SDR
DDR
Sampling Window Extension
1 cycle
DS Latching Delay
8 cells
XiP Mode
配置 Prefetch 与 Combination
配置模块名称与片选引脚
自定义命令表
设置采样窗口与 DS Latching Delay

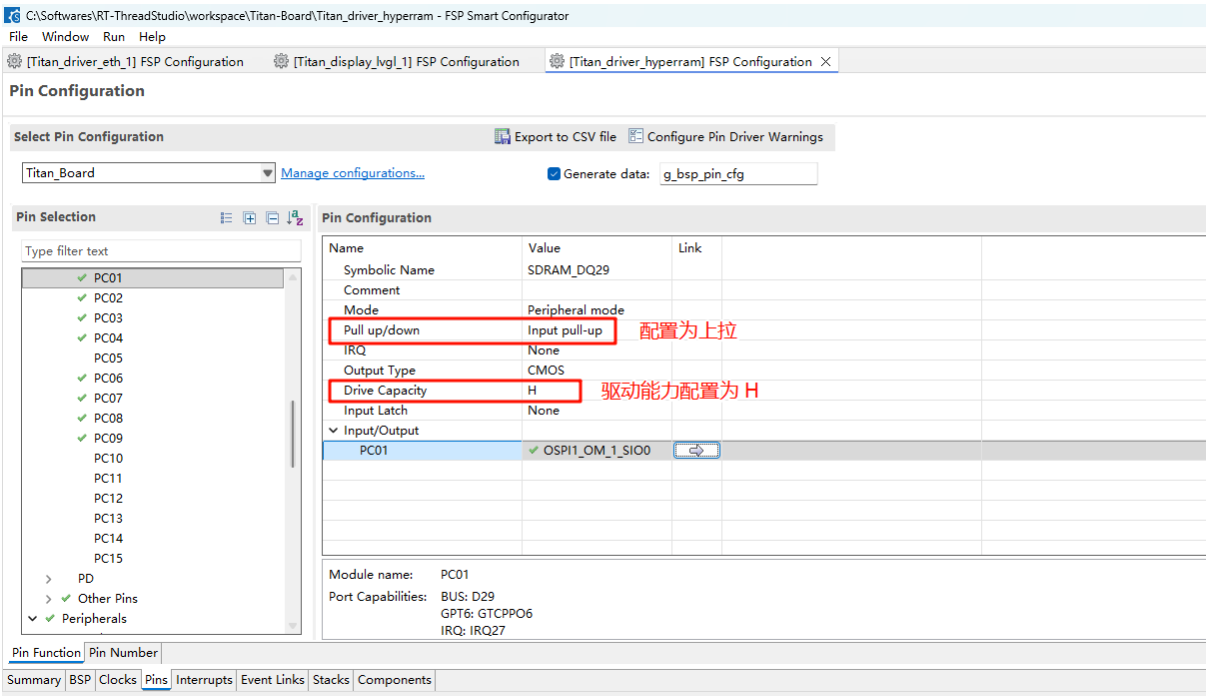

C:\Softwares\RT-ThreadStudio\workspace\titan_test\Titan_driver_hyperram - FSP Smart Configurator
File Window Run Help
[Titan_driver_hyperram] FSP Configuration
Stacks Configuration
Generate Project Content
Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems
g_ospi1 OSPI (r_ospi_b)
Settings
API Info
Property
Value
Erase Sizes
Sector Erase 0
Block Erase 0
Initial Mode
Read
Command Code 0xA0
Dummy Cycles 14
Program
Command Code 0x20
Dummy Cycles 14
Row Load
Command Code 0x00
Dummy Cycles 0
Row Store
Command Code 0x00
Dummy Cycles 0
Write Enable
Command Code 0
Status Read
Command Code 0
Dummy Cycles 0
Sector Erase
Command Code 0
Block Erase
Command Code 0
Chip Erase
Command Code 0
Protocol Mode Dual data rate OPI (8D-8D-8D)
Frame Format xSPI Profile 2.0 Extended
Latency Mode Fixed
Address Length 4 bytes
Address MSB Mask 0xF0
Command Code Length 2 bytes
Status Register Address Length No address
Status Register Address 0
High-speed Mode
Timing Settings
XiP Mode
DOTF
Console
No consoles to display at this time.
配置初始化模式参数



- HyperRAM pin configuration:

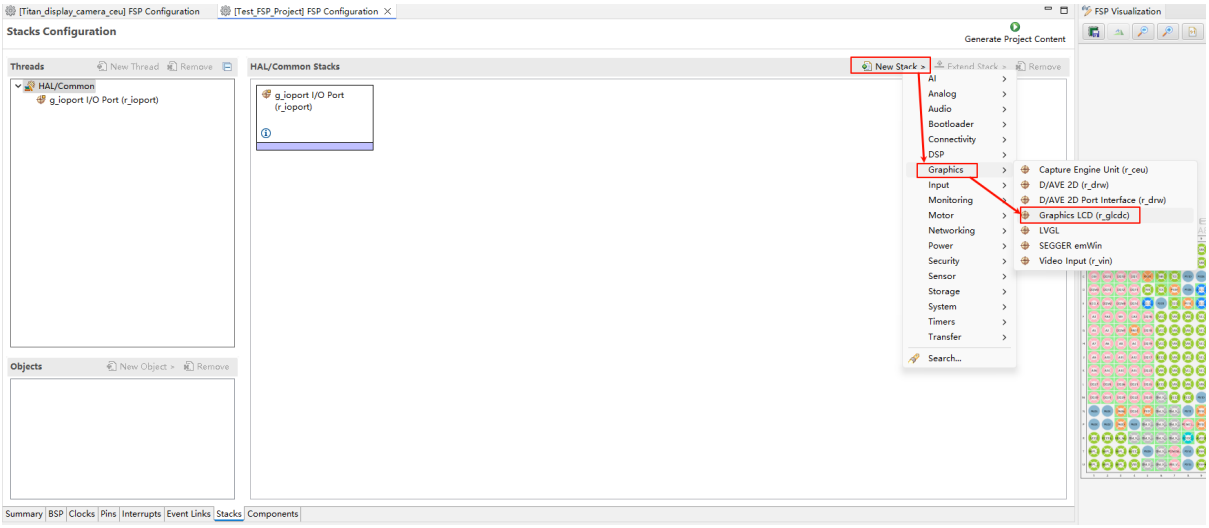


- The drive capability of all pins related to HyperRAM should be configured as H, and OM_1_SIO0 to OM_1_SIO7 need to be configured as Input pull-up.

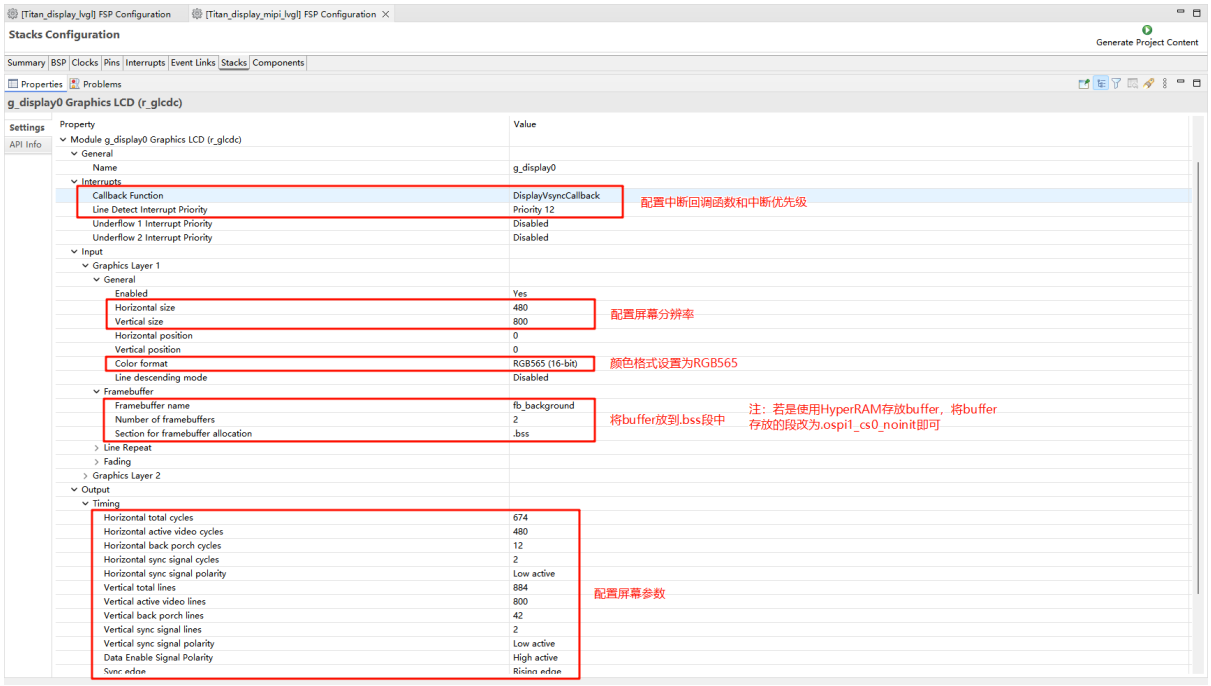


LCD Configuration

- Create a r_glcdc stack:



- Configure interrupt callback and Graphics Layer 1:

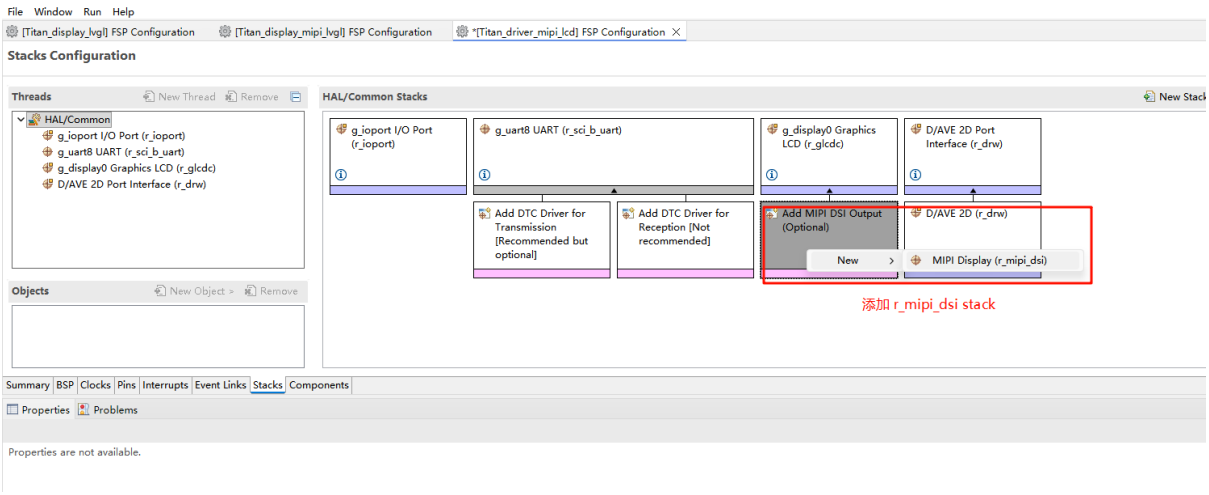


- Configure output parameters, CLUT, TCON, and Dsithering：

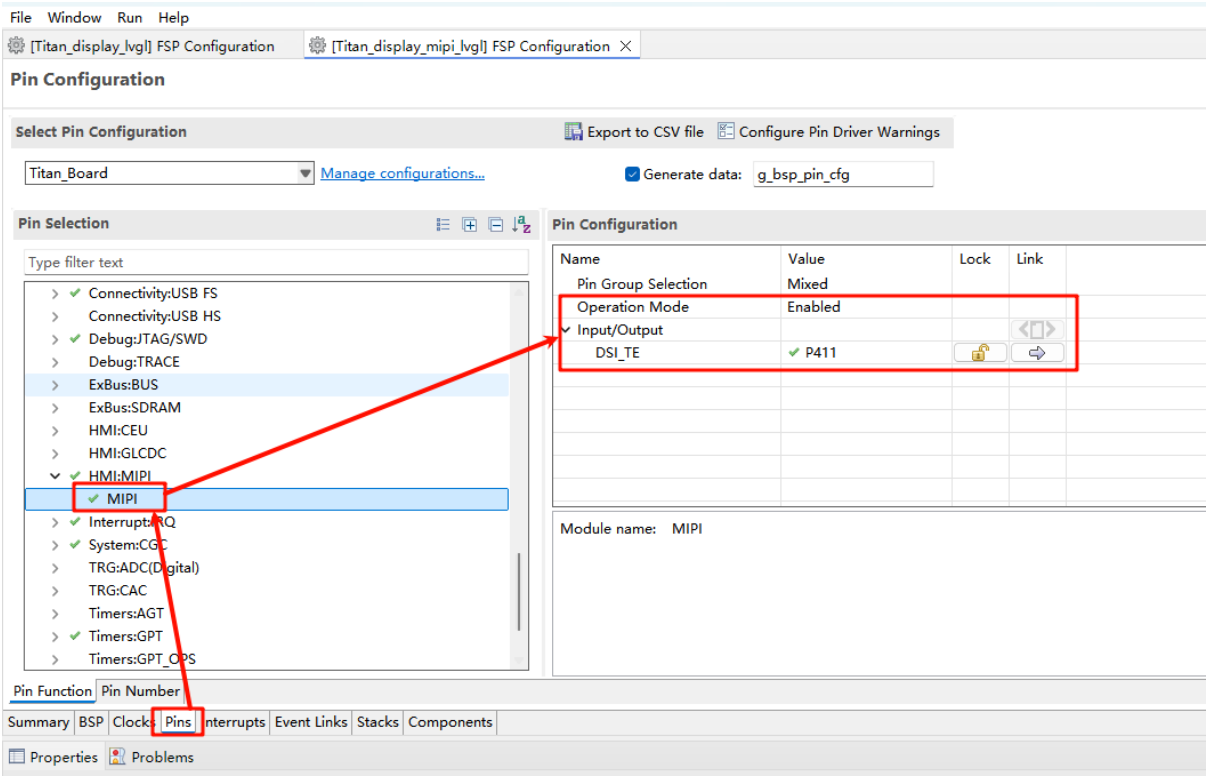
Background	
CLUT	
Enabled	No
Size	256
TCON	
Hsync pin select	LCD_TCON1
Vsync pin select	LCD_TCON0
Data enable (DE) pin select	LCD_TCON2
Panel clock source	Internal clock (GLCDCLK)
Panel clock division ratio	1/8
Dithering	
Enabled	No
Mode	Truncate
Pattern A	Pattern 11
Pattern B	Pattern 11
Pattern C	Pattern 11
Pattern D	Pattern 11

MIPI DSI Configuration

- Create a r_mipi_dsi stack.



- Configure MIPI DSI_TE pins.

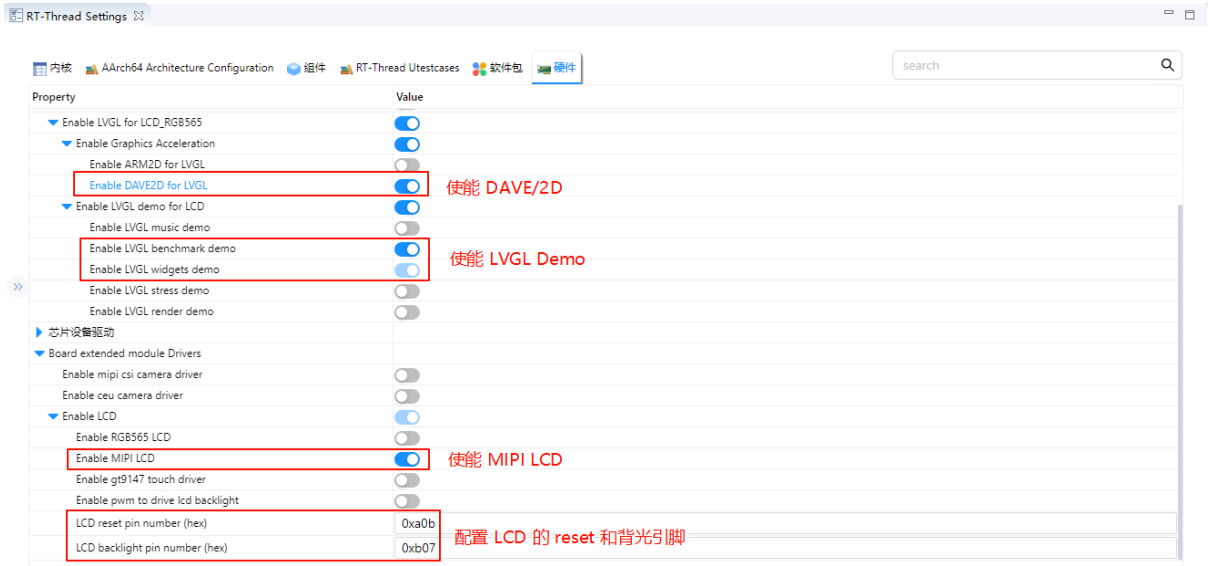


D/AVE 2D Configuration

- Create a r_drw stack:

4.5 RT-Thread Settings Configuration

- Enable the MIPI LCD and LVGL dmeo.



4.6 Software Description

The LVGL-related code is located in ./board/lvgl.

- lv_port_disp.c

```
#include <lvgl.h>
#include <rtthread.h>
#include "ra8/lcd_config.h"
#include "hal_data.h"

static rt_sem_t _SemaphoreVsync = RT_NULL;
static uint8_t lvgl_init_flag = 0;

void DisplayVsyncCallback(display_callback_args_t *p_args)
```

(continues on next page)

(continued from previous page)

```
{
    rt_interrupt_enter();
    if (DISPLAY_EVENT_LINE_DETECTION == p_args->event)
    {
        if (lvgl_init_flag != 0)
            rt_sem_release(_SemaphoreVsync);
    }
    rt_interrupt_leave();
}

static void vsync_wait_cb(lv_display_t *display)
{
    if (!lv_display_flush_is_last(display)) return;

    // If Vsync semaphore has already been set, clear it then wait to avoid
  tearing
    rt_sem_take(_SemaphoreVsync, RT_WAITING_FOREVER);
}

static void disp_flush(lv_display_t *display, const lv_area_t *area, uint8_t *px_
  map)
{
    if (!lv_display_flush_is_last(display)) return;

#if (BSP_CFG_DCACHE_ENABLED)
    int32_t size;
    /* Invalidate cache - so the HW can access any data written by the CPU */
    size = sizeof(fb_background[0]);

    SCB_CleanInvalidateDCache_by_Addr(px_map, size);
#endif

    R_GLCDC_BufferChange(&g_display0_ctrl,
                         (uint8_t *) px_map,
                         (display_frame_layer_t) 0);
}

void lv_port_disp_init(void)
{
    static rt_device_t device;
    /* LCD Device Init */
    device = rt_device_find("lcd");
    RT_ASSERT(device != RT_NULL);

    _SemaphoreVsync = rt_sem_create("lvgl_sem", 1, RT_IPC_FLAG_PRIO);
```

(continues on next page)

(continued from previous page)

```
    if (RT_NULL == _SemaphoreVsync)
    {
        rt_kprintf("lvgl semaphore create failed\r\n");
        RT_ASSERT(0);
    }

    /*-----------------------------------
     * Create a display and set a flush_cb
     * ----------------------------------*/
    lv_display_t *disp = lv_display_create(LV_HOR_RES_MAX, LV_VER_RES_MAX);
    lv_display_set_flush_cb(disp, disp_flush);
    lv_display_set_flush_wait_cb(disp, vsync_wait_cb);
    lv_display_set_buffers(disp, &fb_background[0][0], &fb_background[1][0],
  sizeof(fb_background[0]), LV_DISPLAY_RENDER_MODE_DIRECT);

    lvgl_init_flag = 1;
}
```

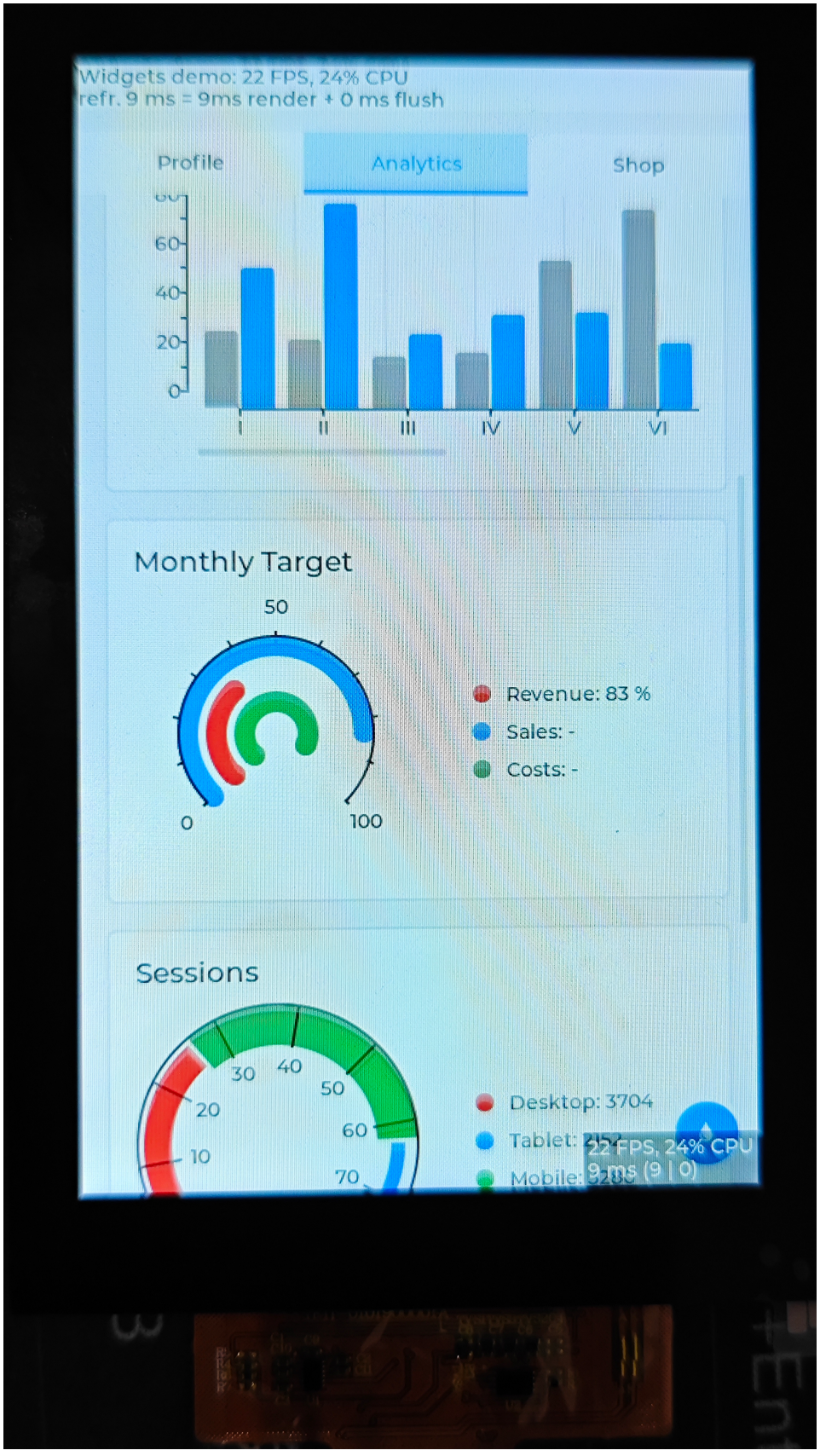
4.7 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio’s package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board’s USB-DBG interface to the PC and download the firmware to the development board.

4.8 Run Effect

After resetting the development board, you can see the LVGL demo running on the LCD screen.


Widgets demo: 22 FPS, 24% CPU
refr. 9 ms = 9ms render + 0 ms flush
Profile
Analytics
Shop
60
40
20
0
I
II
III
IV
V
VI
Monthly Target
50
0
100
Revenue: 83 %
Sales: -
Costs: -
Sessions
10
20
30
40
50
60
70
Desktop: 3704
Tablet:
Mobile:
22 FPS, 24% CPU
9 ms (9 | 0)

5. RGB565 LVGL Usage Instructions

English | Chinese

5.1 Instruction

This example is based on the **LVGL** graphics library and demonstrates a sample graphical user interface (GUI) project designed for embedded devices. It integrates both display and input drivers, showcasing the complete workflow from low-level driver adaptation to high-level interface construction. Through this example, developers can not only learn how to port and configure LVGL in an embedded environment but also gain practical knowledge of key techniques such as interface layout, event handling, and theme customization, thereby laying a solid foundation for building real product interfaces.

5.2 LVGL Instruction

LVGL (Light and Versatile Graphics Library) is an open-source embedded GUI (Graphical User Interface) development framework. It was designed with the goal of providing smooth and modern graphical interfaces for resource-constrained embedded devices. To achieve this, it has been highly optimized in terms of runtime efficiency, memory usage, and portability. Whether running on simple low-end MCUs or more powerful MPU platforms, LVGL can operate efficiently while offering a wide range of graphical widgets and interactive features.

Key Features

1. **Lightweight** The greatest advantage of LVGL is its lightweight nature, requiring very few resources. It can still run smoothly on microcontrollers with only a few dozen KB of memory. The framework itself has minimal memory overhead, and its rendering algorithms are carefully optimized to ensure low CPU usage while delivering good visual performance on limited hardware. This makes LVGL particularly suitable for resource-sensitive scenarios such as low-power devices, wearables, and home appliance control panels.
2. **Cross-Platform** LVGL stands out for its cross-platform capability. It can run on multiple operating systems including FreeRTOS, RT-Thread, Zephyr, and Linux, or even directly on bare-metal systems. Developers only need to provide display and input driver interfaces for LVGL, and porting can be completed quickly, enabling the same UI code to be reused across different hardware platforms. This flexibility significantly reduces development costs and makes LVGL a universal solution for embedded GUIs.
3. **Rich Widgets** LVGL comes with a wide variety of built-in GUI widgets such as buttons, labels, sliders, progress bars, checkboxes, lists, tables, and charts. These components cover nearly all common HMI (Human-Machine Interface) requirements, sparing developers from designing and implementing them from scratch and thus shortening the development cycle. LVGL also allows extending existing widgets to create custom components, making it adaptable for both simple numeric displays and complex graphical control interfaces.

4. **Advanced Rendering Capabilities** In terms of visual presentation, LVGL offers a rich set of rendering features. It supports anti-aliasing, transparency, gradients, shadows, borders, and rounded corners, enabling modern and visually appealing interfaces. Additionally, LVGL includes a built-in animation system with multiple easing functions, making smooth movements, transitions, and dynamic effects easy to implement. These capabilities enhance not only functionality but also the overall user experience.
5. **Input Device Support** LVGL supports multiple input device types, including touchscreens, capacitive panels, mice, keyboards, and rotary encoders, with even multi-touch interaction available. A unified input interface layer is provided, so developers only need to implement low-level driver adaptation to pass input events into LVGL' s event system. This allows them to focus on interface logic instead of spending excessive time on input handling.
6. **Internationalization and Multilanguage Support** LVGL has excellent internationalization support. It uses UTF-8 encoding, which can handle nearly all character sets. It also supports bidirectional text rendering, ensuring correct display of languages such as Arabic and Hebrew that are written right-to-left. This makes LVGL suitable for globally targeted products, allowing users in different countries and regions to enjoy a localized and seamless experience.
7. **Extensibility** LVGL provides a flexible theming and styling system, enabling developers to easily customize the appearance of widgets to achieve various UI styles. By switching themes, the overall look and feel of the interface can be changed quickly. Furthermore, LVGL can integrate with third-party graphics libraries, file systems, and image decoders to expand its functionality. For instance, developers can use a file system to load external fonts and images, or integrate JPEG/PNG decoders to display complex images.

Application Scenarios

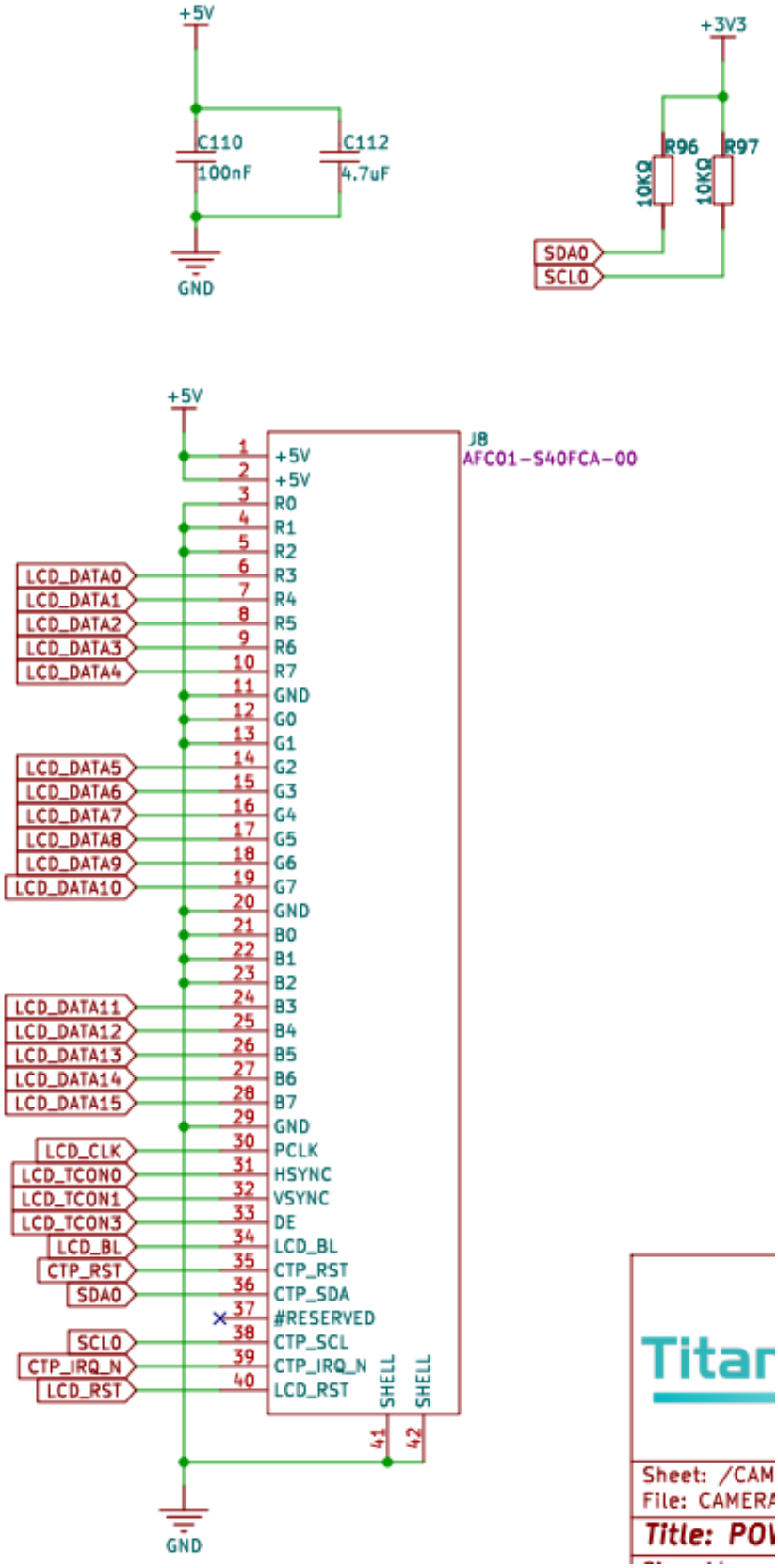
LVGL has a very broad range of real-world applications. In consumer electronics, it is commonly used in smart home control panels, appliance displays, smartwatches, and fitness devices, all of which require user-friendly interfaces on constrained hardware. In industrial control, LVGL is applied to HMIs (Human-Machine Interfaces) and various instruments, helping users intuitively monitor and control equipment. In automotive electronics, LVGL can power infotainment systems, passenger entertainment displays, and even instrument clusters. In the medical field, it is well-suited for small-scale displays, such as handheld diagnostic tools and portable monitoring devices.

Ecosystem and Community

As an open-source project under the **MIT License**, LVGL is equally suitable for personal learning and free use in commercial products. Around LVGL, an active ecosystem has formed. Official tools such as **SquareLine Studio** provide visual drag-and-drop design with exportable LVGL code, greatly improving development efficiency. The **LVGL Simulator** enables developers to debug interfaces directly on a PC without repeatedly flashing target hardware. A vibrant global developer community contributes numerous open-source widgets, themes, and porting examples, offering valuable resources and support to both beginners and enterprise developers.

5.3 Hardware Description

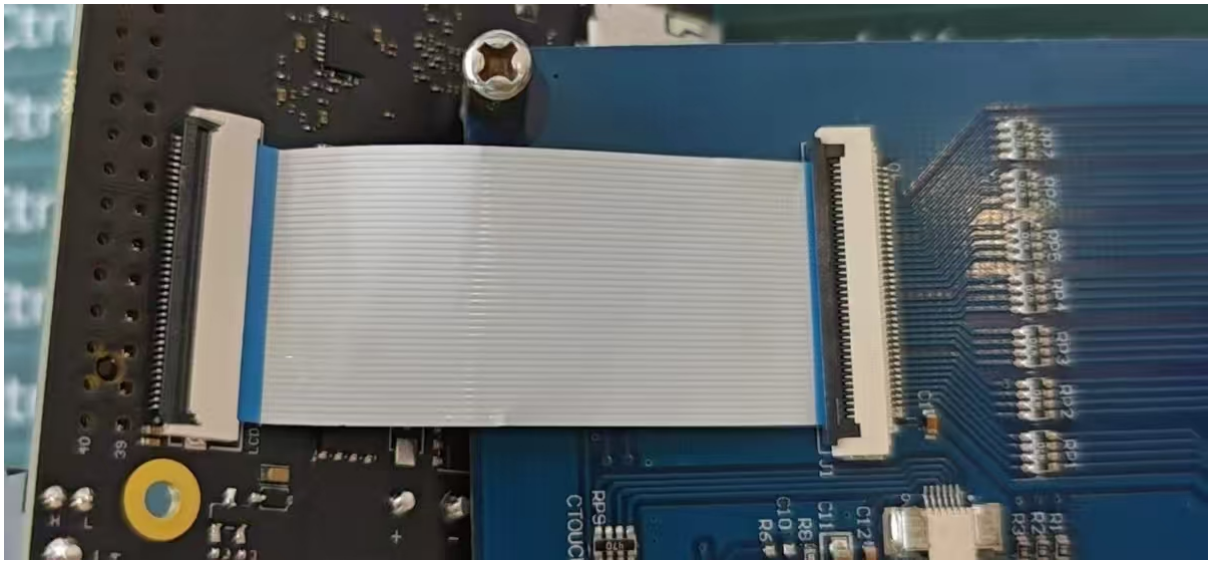
- To run this example, an external RGB565 LCD screen is required. The 40-pin RGB565 LCD interface on the development board is as follows:


+5V
C110
100nF
C112
4.7uF
GND
+3V3
R96
R97
10KΩ
10KΩ
SDA0
SCL0
+5V
J8
AFC01-S40FCA-00
1 +5V
2 +5V
3 R0
4 R1
5 R2
6 R3
7 R4
8 R5
9 R6
10 R7
11 GND
12 G0
13 G1
14 G2
15 G3
16 G4
17 G5
18 G6
19 G7
20 GND
21 B0
22 B1
23 B2
24 B3
25 B4
26 B5
27 B6
28 B7
29 GND
30 PCLK
31 HSYNC
32 VSYNC
33 DE
34 LCD_BL
35 CTP_RST
36 CTP_SDA
37 #RESERVED
38 CTP_SCL
39 CTP_IRQ_N
40 LCD_RST
41 SHELL
42 SHELL
LCD_DATA0
LCD_DATA1
LCD_DATA2
LCD_DATA3
LCD_DATA4
LCD_DATA5
LCD_DATA6
LCD_DATA7
LCD_DATA8
LCD_DATA9
LCD_DATA10
LCD_DATA11
LCD_DATA12
LCD_DATA13
LCD_DATA14
LCD_DATA15
LCD_CLK
LCD_TCON0
LCD_TCON1
LCD_TCON3
LCD_BL
CTP_RST
SDA0
SCL0
CTP_IRQ_N
LCD_RST
GND
Titan
Sheet: /CAM
File: CAMERA
Title: POV

- The connection method of RGB LCD is as follows:

Display model: **ALIENTEK 4.3" RGB LCD 800×480**

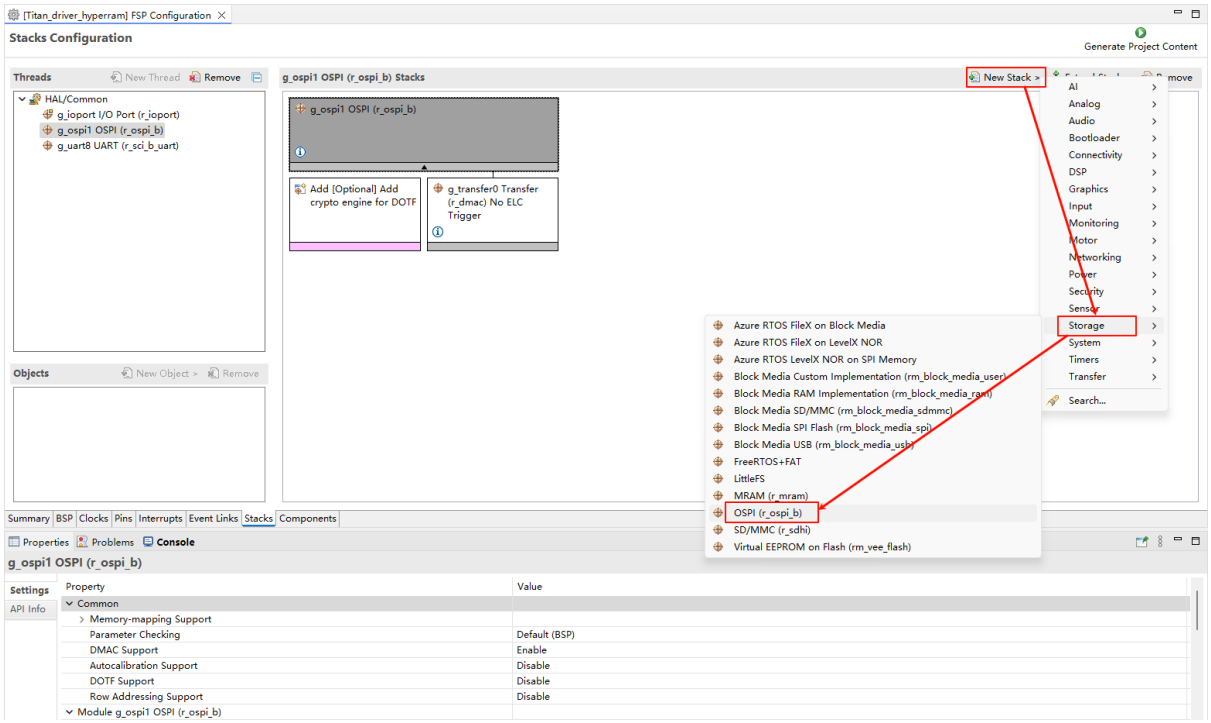
Use a **40-pin same-direction FFC cable** to connect the development board's **LCD_RGB** connector to the display.



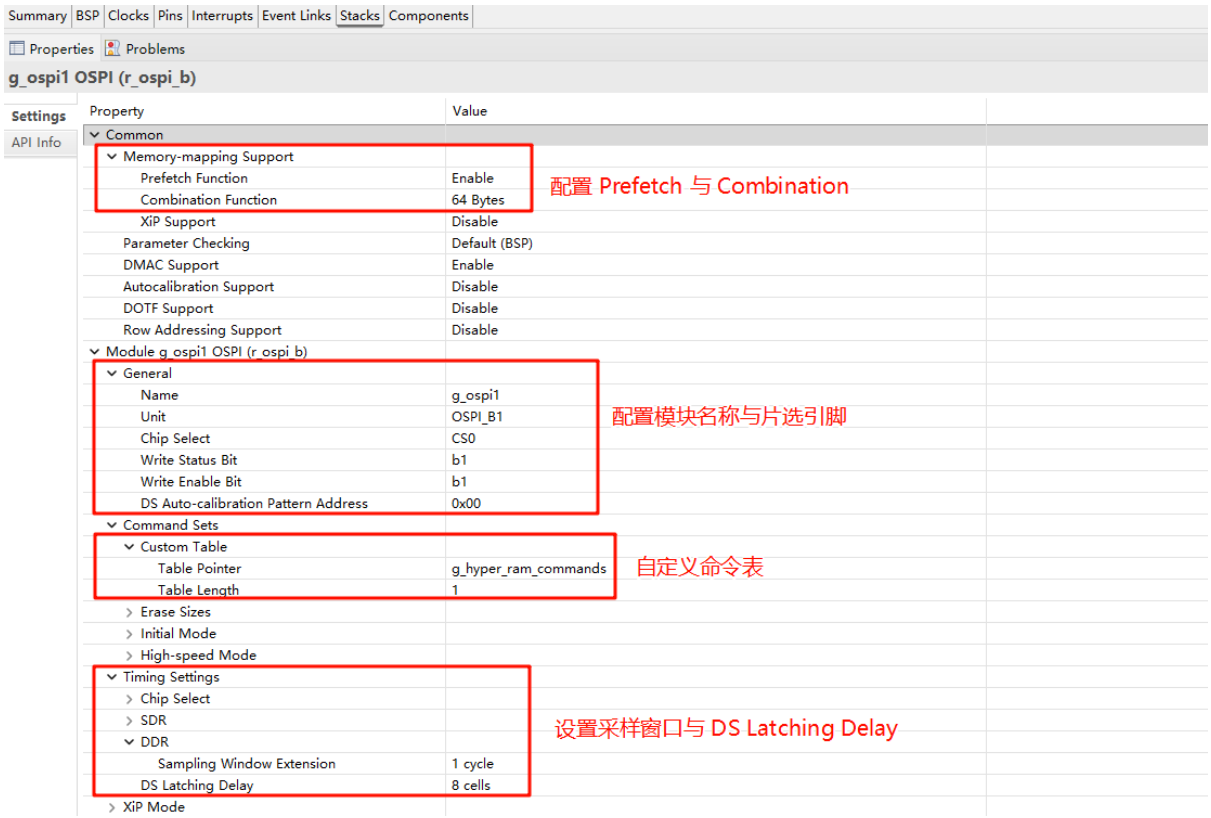
5.4 FSP Configuration

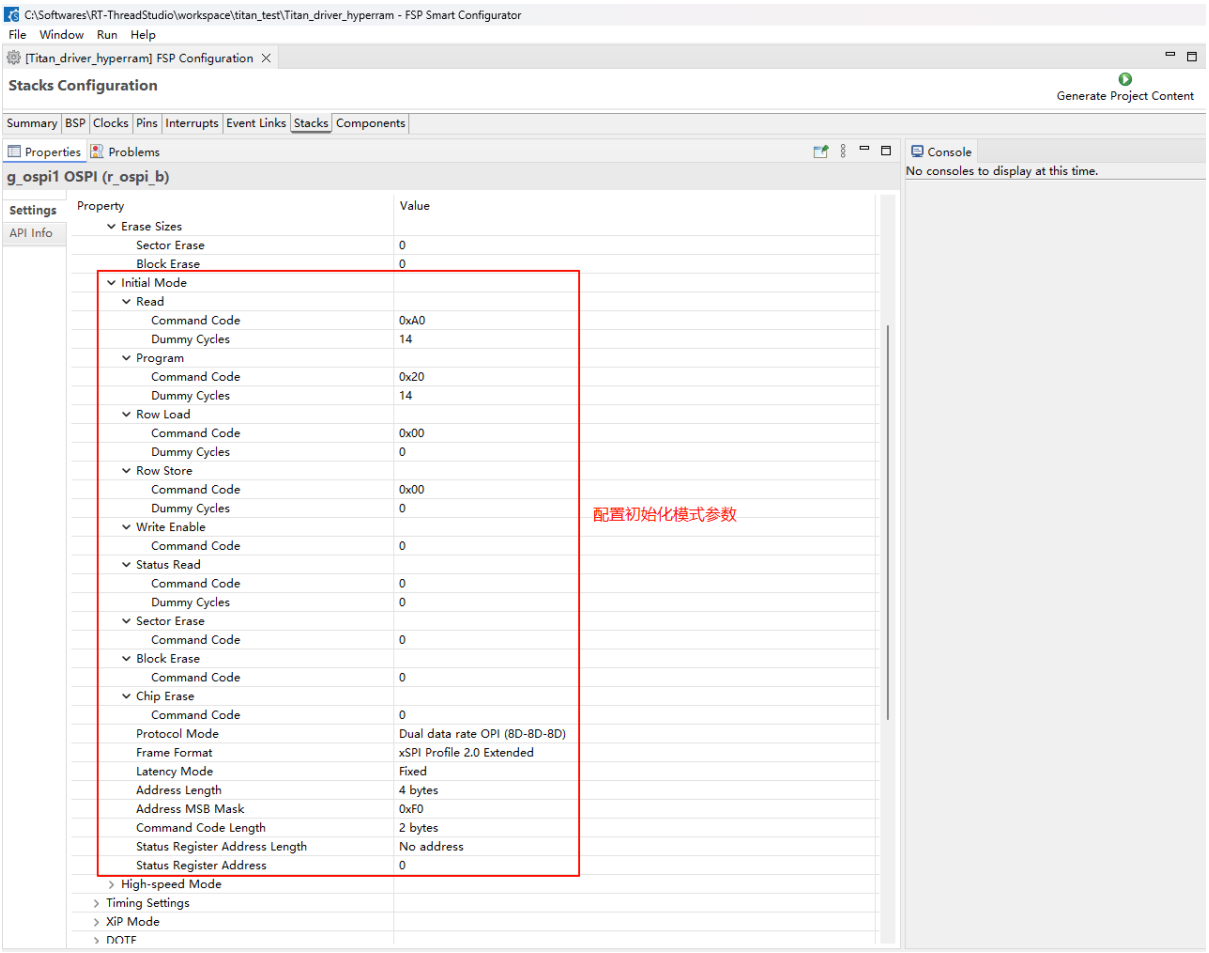
HyperRAM Configuration

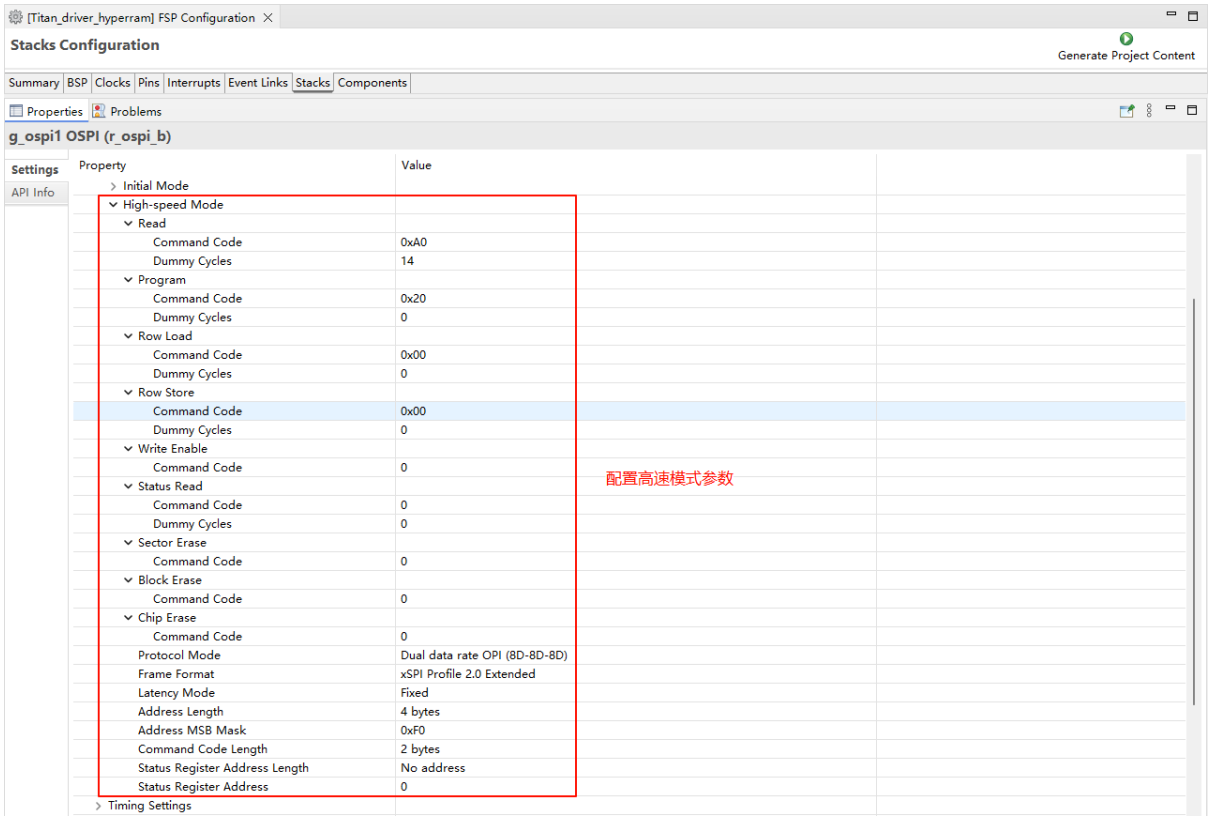
- Create a r_ospi_b stack:



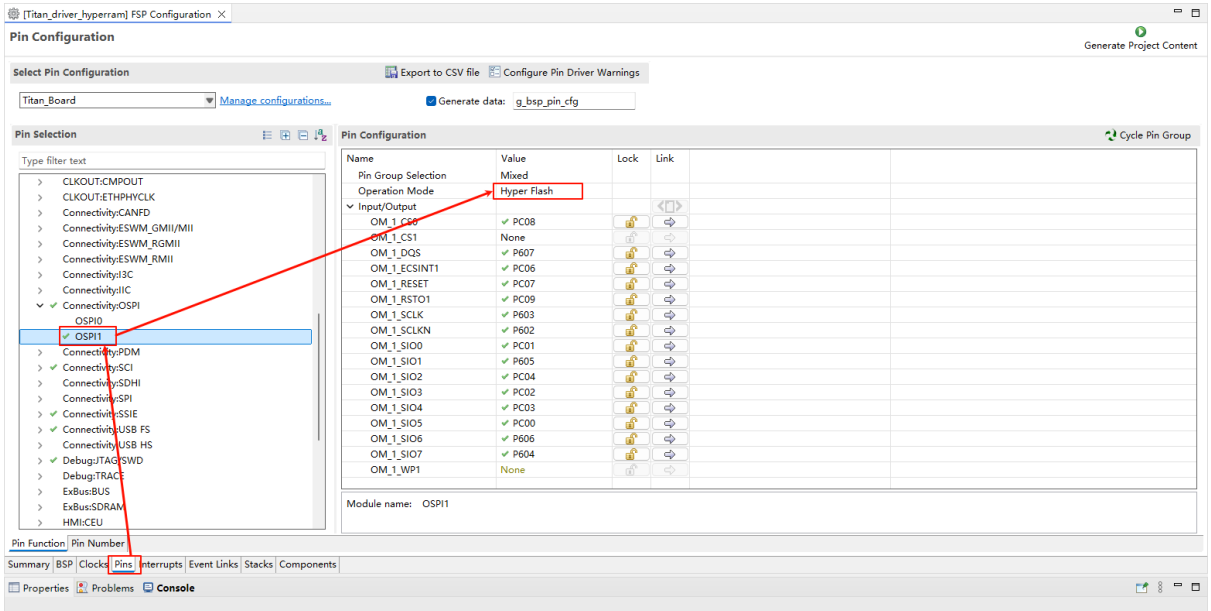
- Configuration r_ospi_b stack：



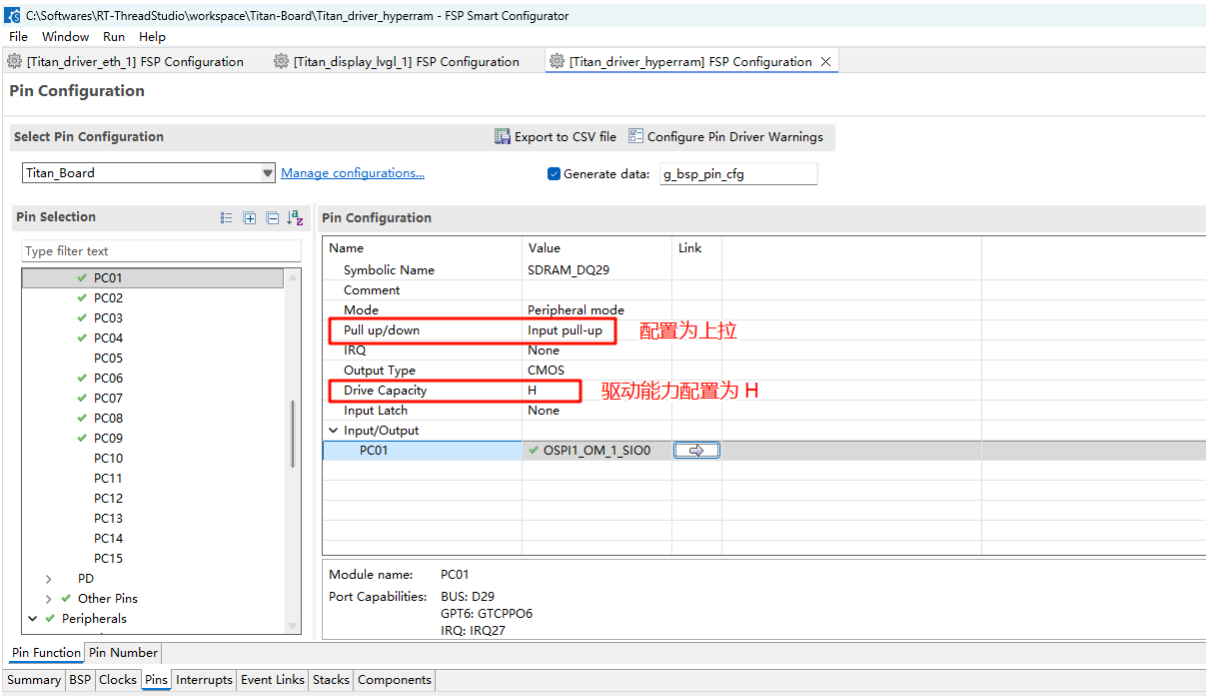

C:\Softwares\RT-ThreadStudio\workspace\titan_test\Titan_driver_hyperram - FSP Smart Configurator
File Window Run Help
[Titan_driver_hyperram] FSP Configuration
Stacks Configuration
Generate Project Content
Summary BSP Clocks Pins Interrupts Event Links Stacks Components
Properties Problems
g_ospi1 OSPI (r_ospi_b)
Settings
API Info
Property
Value
Erase Sizes
Sector Erase 0
Block Erase 0
Initial Mode
Read
Command Code 0xA0
Dummy Cycles 14
Program
Command Code 0x20
Dummy Cycles 14
Row Load
Command Code 0x00
Dummy Cycles 0
Row Store
Command Code 0x00
Dummy Cycles 0
Write Enable
Command Code 0
Status Read
Command Code 0
Dummy Cycles 0
Sector Erase
Command Code 0
Block Erase
Command Code 0
Chip Erase
Command Code 0
Protocol Mode Dual data rate OPI (8D-8D-8D)
Frame Format xSPI Profile 2.0 Extended
Latency Mode Fixed
Address Length 4 bytes
Address MSB Mask 0xF0
Command Code Length 2 bytes
Status Register Address Length No address
Status Register Address 0
High-speed Mode
Timing Settings
XiP Mode
DOTF
Console
No consoles to display at this time.
配置初始化模式参数



- Configure HyperRAM pins：

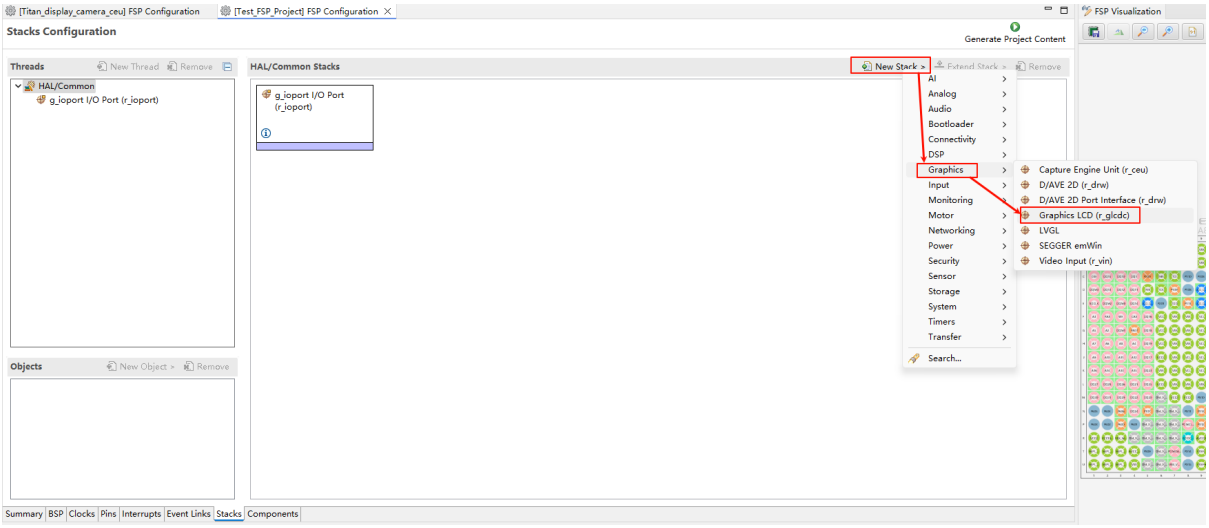


- The drive capability of all pins related to HyperRAM should be configured as H, and OM_1_SIO0 to OM_1_SIO7 need to be configured as Input pull-up

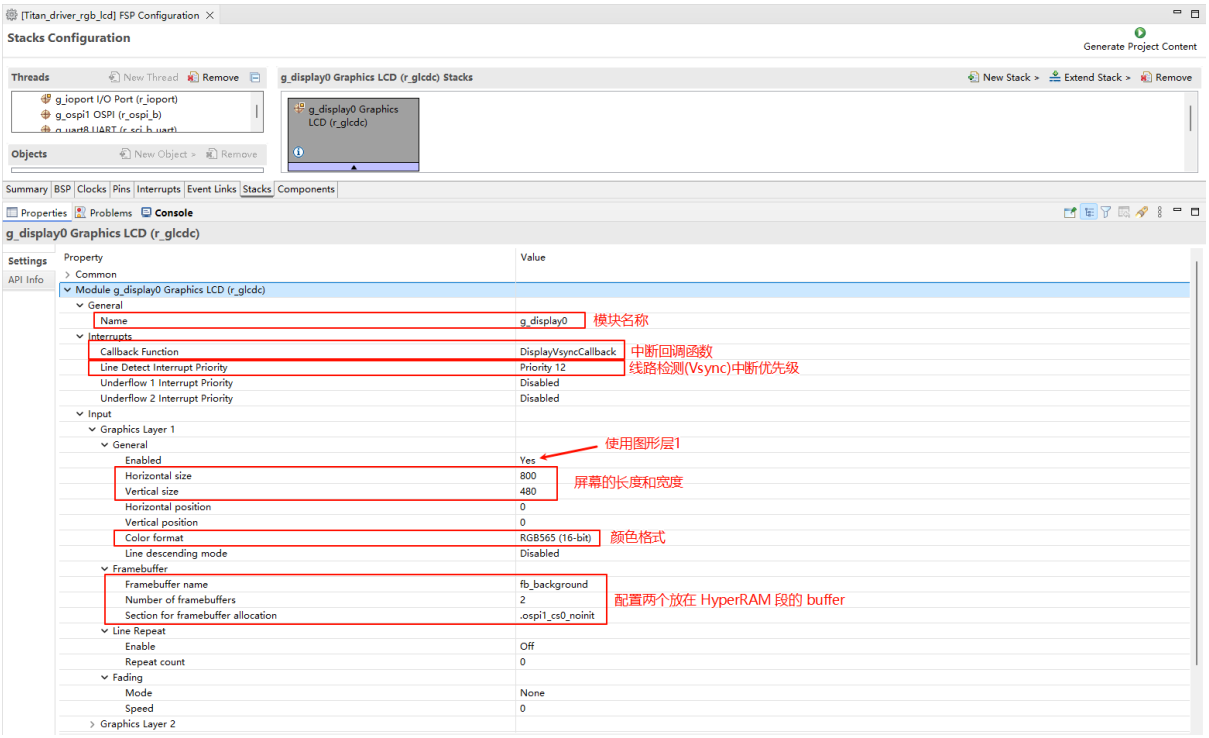


RGB LCD Configuration

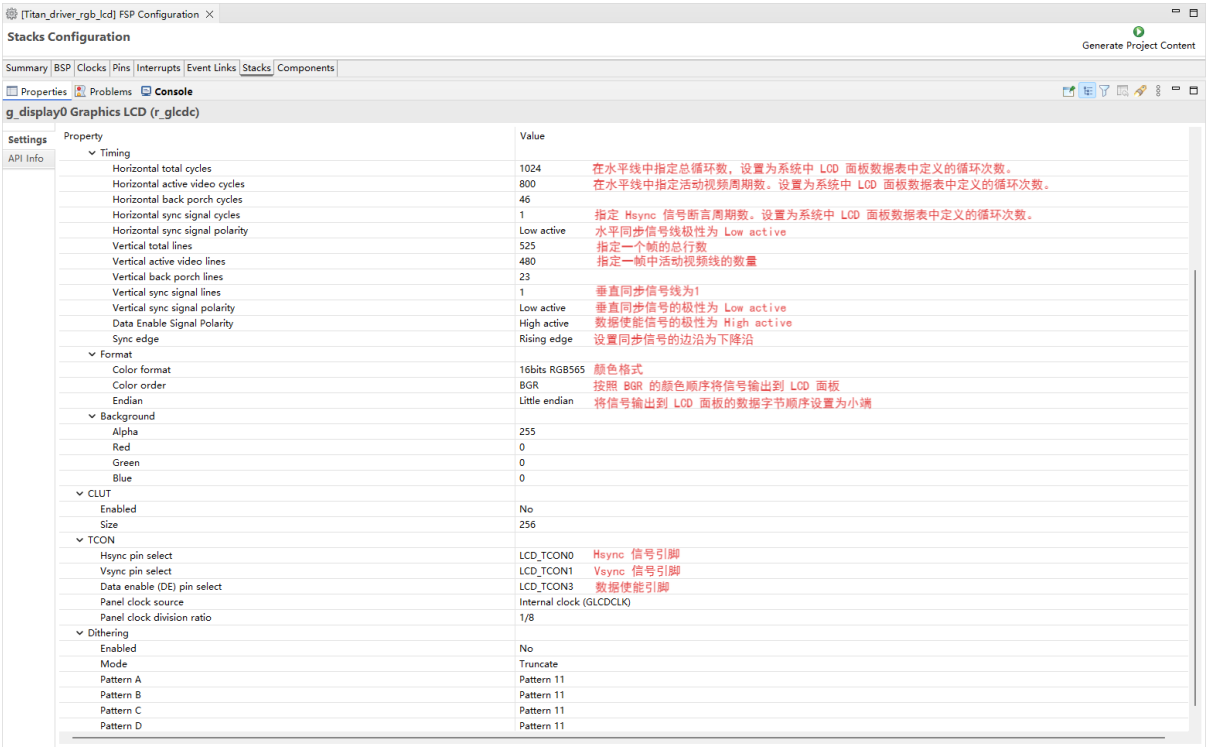
- Create a r_glcdc stack:



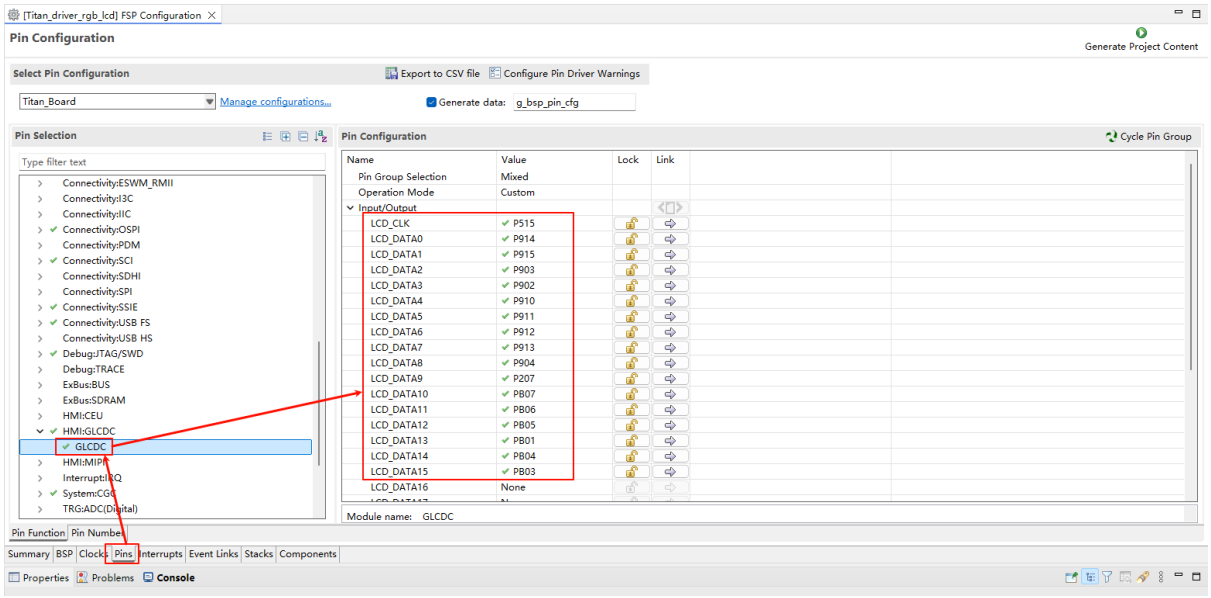
- Configure interrupt callback and Graphics Layer 1:

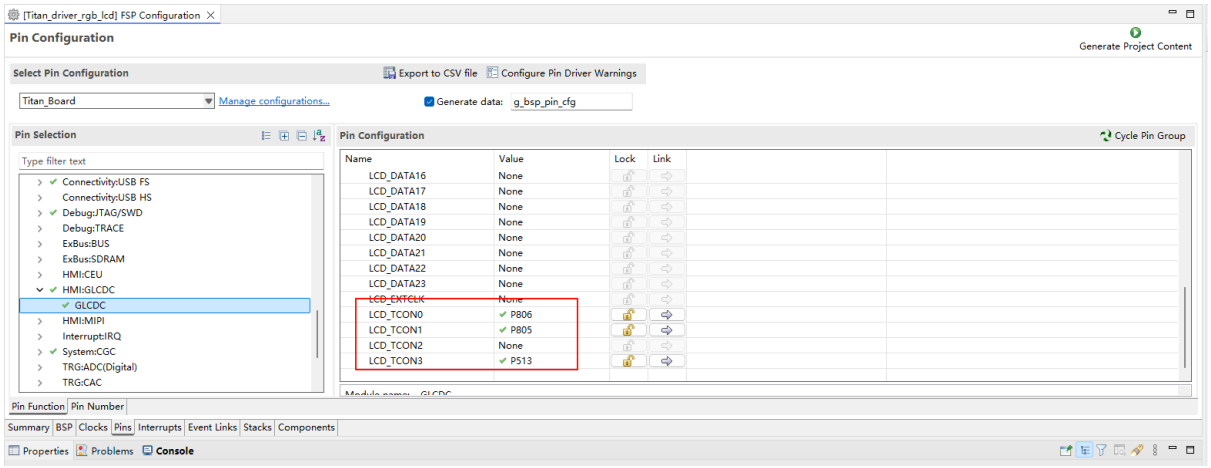


- Configure output parameters, CLUT, TCON, and Dsithering：



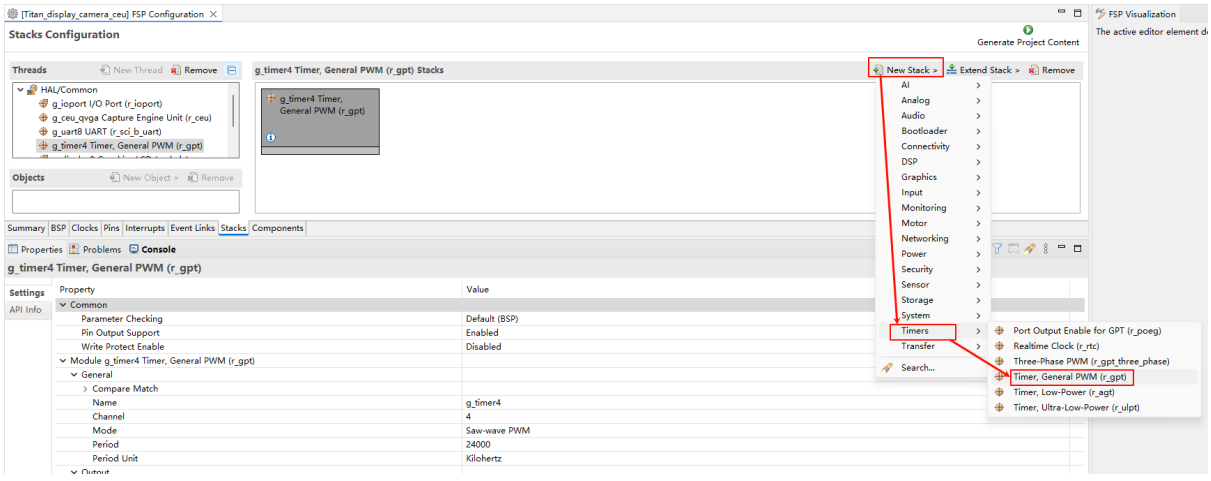
- Configure GLCDC pins:



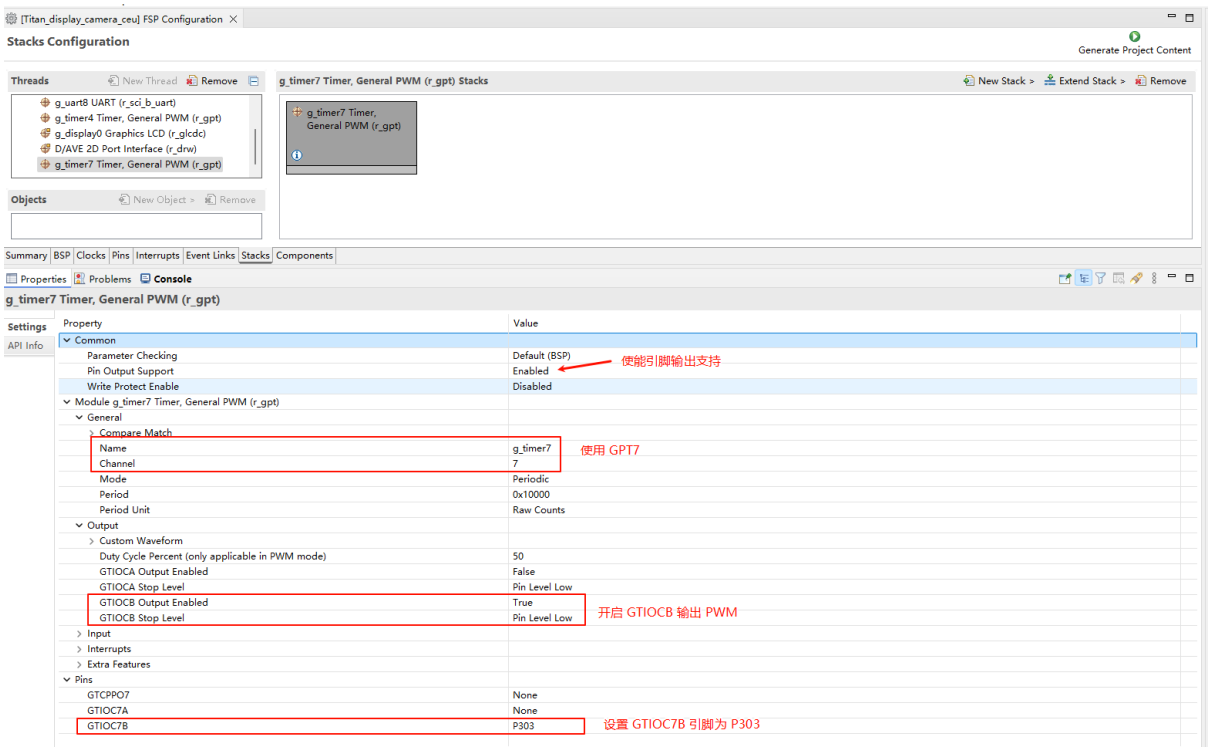


LCD Backlight Configuration

- Create a r_gpt stack:



- Configure backlight PWM output:

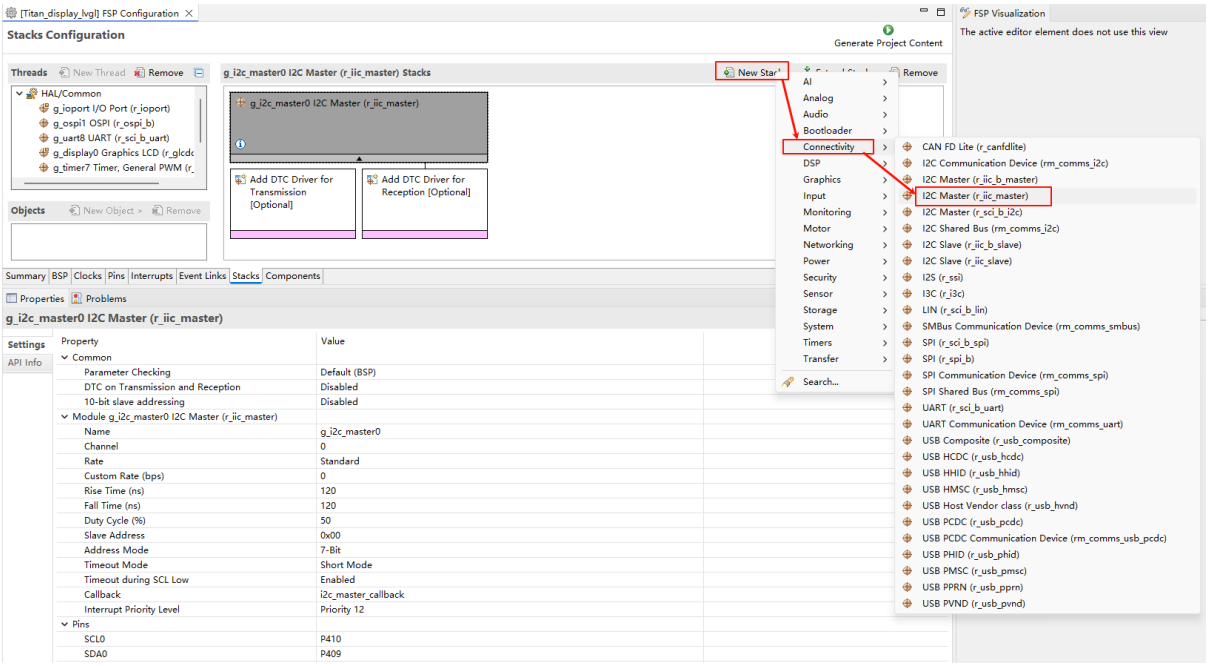


D/AVE 2D Configuration

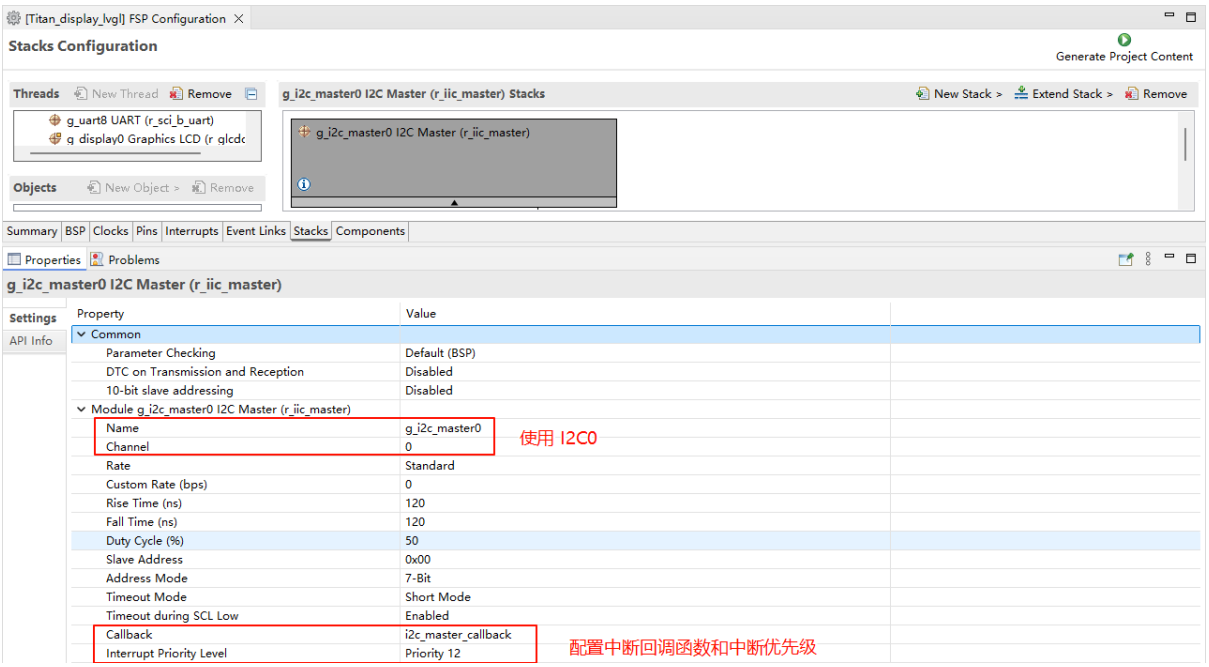
- Create a r_drw stack:

I2C0 Configuration

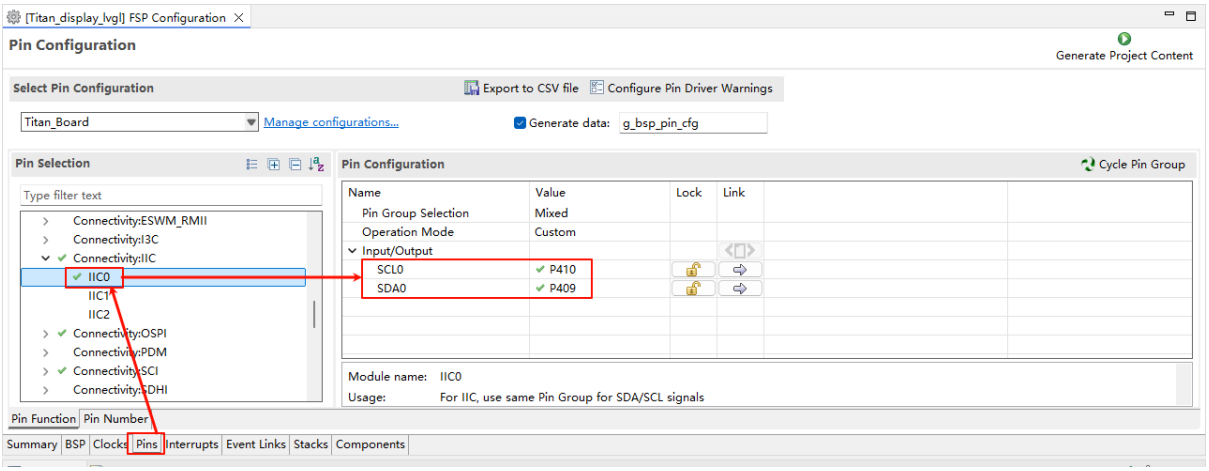
- Create a r_iic_master stack:



- Configure I2C0:

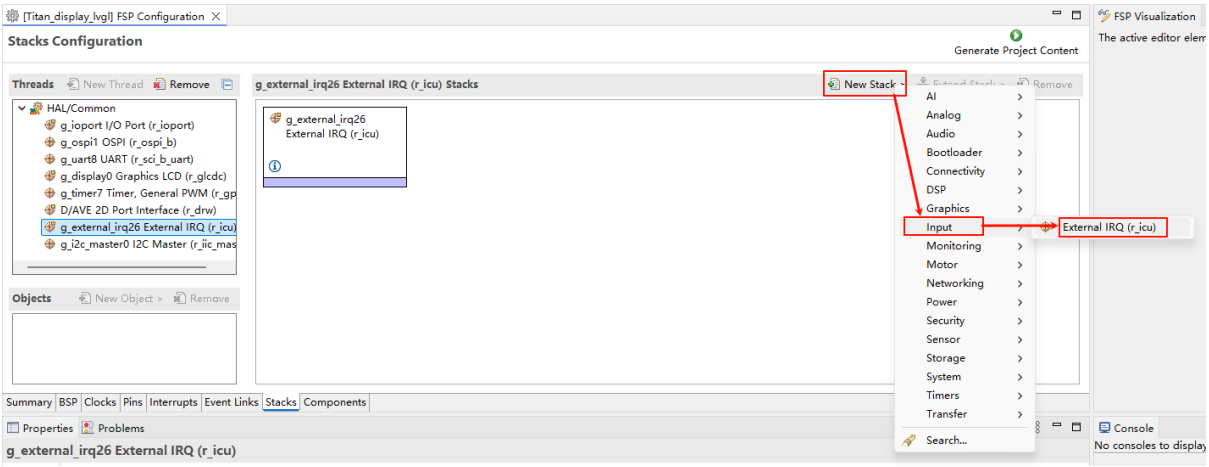


- Configure I2C0 pins:

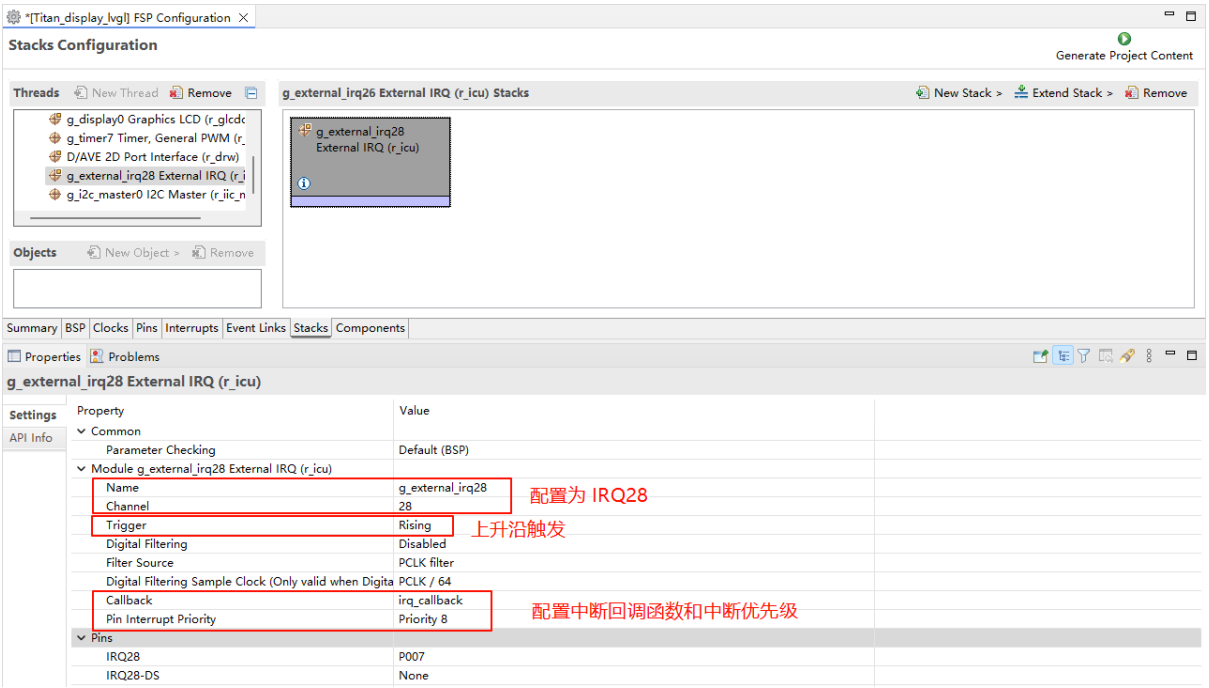


Touch Interrupt Configuration

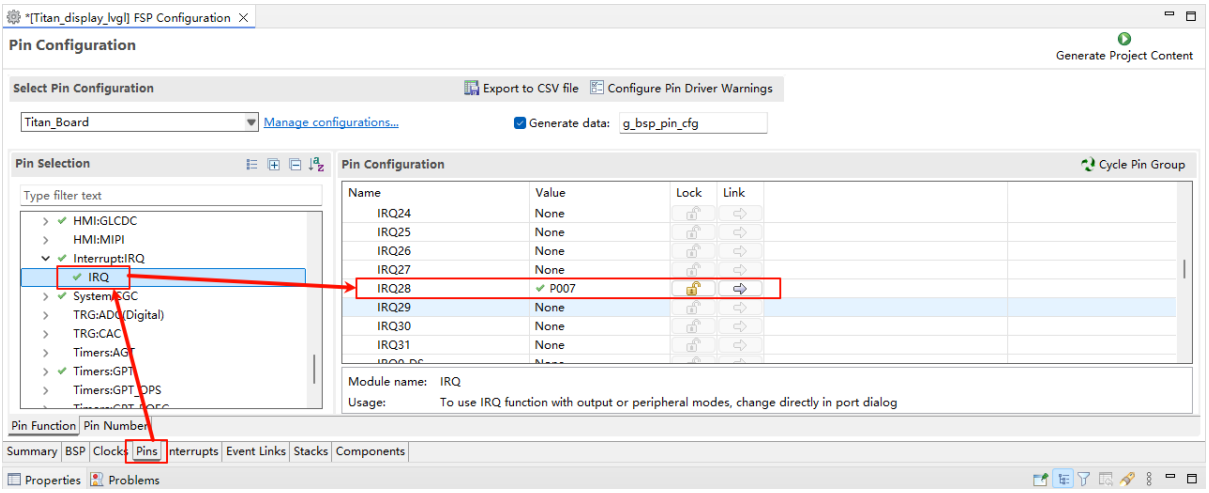
- Create a r_icu stack:



- Configure IRQ28:



- Configure IRQ28 pins:



5.5 RT-Thread Settings Configuration

- Enable the RGB565 LCD, the GT9147 touch driver, and LVGL.

5.6 Software Description

The LVGL-related code is located in ./board/lvgl.

- lv_port_disp.c

```
#include <lvgl.h>
#include <rtthread.h>
#include "ra8/lcd_config.h"
#include "hal_data.h"

static rt_sem_t _SemaphoreVsync = RT_NULL;
static uint8_t lvgl_init_flag = 0;

void DisplayVsyncCallback(display_callback_args_t *p_args)
{
    rt_interrupt_enter();
    if (DISPLAY_EVENT_LINE_DETECTION == p_args->event)
    {
        if (lvgl_init_flag != 0)
            rt_sem_release(_SemaphoreVsync);
    }
    rt_interrupt_leave();
}

static void vsync_wait_cb(lv_display_t *display)
{
    if (!lv_display_flush_is_last(display)) return;

    // If Vsync semaphore has already been set, clear it then wait to avoid
  tearing
```

(continues on next page)

(continued from previous page)

```
    rt_sem_take(_SemaphoreVsync, RT_WAITING_FOREVER);
}

static void disp_flush(lv_display_t *display, const lv_area_t *area, uint8_t *px_
  map)
{
    if (!lv_display_flush_is_last(display)) return;

#if (BSP_CFG_DCACHE_ENABLED)
    int32_t size;
    /* Invalidate cache - so the HW can access any data written by the CPU */
    size = sizeof(fb_background[0]);

    SCB_CleanInvalidateDCache_by_Addr(px_map, size);
#endif

    R_GLCDC_BufferChange(&g_display0_ctrl,
                         (uint8_t *) px_map,
                         (display_frame_layer_t) 0);
}

void lv_port_disp_init(void)
{
    static rt_device_t device;
    /* LCD Device Init */
    device = rt_device_find("lcd");
    RT_ASSERT(device != RT_NULL);

    _SemaphoreVsync = rt_sem_create("lvgl_sem", 1, RT_IPC_FLAG_PRIO);

    if (RT_NULL == _SemaphoreVsync)
    {
        rt_kprintf("lvgl semaphore create failed\r\n");
        RT_ASSERT(0);
    }

    /*-----------------------------------
     * Create a display and set a flush_cb
     * ----------------------------------*/
    lv_display_t *disp = lv_display_create(LV_HOR_RES_MAX, LV_VER_RES_MAX);
    lv_display_set_flush_cb(disp, disp_flush);
    lv_display_set_flush_wait_cb(disp, vsync_wait_cb);
    lv_display_set_buffers(disp, &fb_background[0][0], &fb_background[1][0],
  sizeof(fb_background[0]), LV_DISPLAY_RENDER_MODE_DIRECT);

    lvgl_init_flag = 1;
}
```

- lv_port_indev.c

```
#include <lvgl.h>
#include <rtdevice.h>

#ifdef RT_USING_TOUCH

#include "gt9147.h"
#define GT9147_IRQ_PIN   BSP_IO_PORT_00_PIN_07
#define GT9147_RST_PIN   BSP_IO_PORT_04_PIN_12

#define DBG_TAG "lv_port_indev"
#define DBG_LVL DBG_LOG
#include <rtdbg.h>

#include "hal_data.h"

static rt_device_t touch_dev;
struct rt_touch_data *read_data;
static lv_indev_t *touch_indev;

#if LVGL_VERSION_MAJOR < 9
static void touchpad_read(lv_indev_drv_t *indev, lv_indev_data_t *data)
#else
static void touchpad_read(lv_indev_t *indev, lv_indev_data_t *data)
#endif
{
    if (RT_NULL != rt_device_read(touch_dev, 0, read_data, 1))
    {
        if (read_data->event == RT_TOUCH_EVENT_NONE)
            return;

        data->point.x = LV_HOR_RES_MAX - read_data->x_coordinate;
        data->point.y = LV_VER_RES_MAX - read_data->y_coordinate;

        if (read_data->event == RT_TOUCH_EVENT_DOWN)
            data->state = LV_INDEV_STATE_PR;
        if (read_data->event == RT_TOUCH_EVENT_MOVE)
            data->state = LV_INDEV_STATE_PR;
        if (read_data->event == RT_TOUCH_EVENT_UP)
            data->state = LV_INDEV_STATE_REL;
    }
}

static rt_err_t gt9147_probe(rt_uint16_t x, rt_uint16_t y)
{
    void *id;
```

(continues on next page)

(continued from previous page)

```
    touch_dev = rt_device_find("gt9147");
    if (touch_dev == RT_NULL)
    {
        rt_kprintf("can't find device gt9147\n");
        return -1;
    }

    if (rt_device_open(touch_dev, RT_DEVICE_FLAG_INT_RX) != RT_EOK)
    {
        rt_kprintf("open device failed!");
        return -1;
    }

    id = rt_malloc(sizeof(struct rt_touch_info));
    rt_device_control(touch_dev, RT_TOUCH_CTRL_GET_ID, id);
    rt_uint8_t *read_id = (rt_uint8_t *)id;
    rt_kprintf("id = GT%d%d%d \n", read_id[0] - '0', read_id[1] - '0', read_id[2]
  - '0');

    rt_device_control(touch_dev, RT_TOUCH_CTRL_SET_X_RANGE, &x);  /* if possible
  you can set your x y coordinate*/
    rt_device_control(touch_dev, RT_TOUCH_CTRL_SET_Y_RANGE, &y);
    rt_device_control(touch_dev, RT_TOUCH_CTRL_GET_INFO, id);
    rt_kprintf("range_x = %d \n", (*(struct rt_touch_info *)id).range_x);
    rt_kprintf("range_y = %d \n", (*(struct rt_touch_info *)id).range_y);
    rt_kprintf("point_num = %d \n", (*(struct rt_touch_info *)id).point_num);
    rt_free(id);

    read_data = (struct rt_touch_data *)rt_calloc(1, sizeof(struct rt_touch_
  data));
    if (!read_data)
    {
        return -RT_ENOMEM;
    }

    return RT_EOK;
}

rt_err_t rt_hw_gt9147_register(void)
{
    struct rt_touch_config config;
    rt_uint8_t rst;
    rst = GT9147_RST_PIN;
    config.dev_name = "i2c0";
    config.irq_pin.pin = GT9147_IRQ_PIN;
```

(continues on next page)

(continued from previous page)

```
    config.irq_pin.mode = PIN_MODE_INPUT_PULLDOWN;
    config.user_data = &rst;

    if (rt_hw_gt9147_init("gt9147", &config) != RT_EOK)
    {
        rt_kprintf("touch device gt9147 init failed.\n");
        return -RT_ERROR;
    }

    if (gt9147_probe(LV_HOR_RES_MAX, LV_VER_RES_MAX) != RT_EOK)
    {
        rt_kprintf("probe gt9147 failed.\n");
        return -RT_ERROR;
    }
    return RT_EOK;
}
#endif

void lv_port_indev_init(void)
{
#ifdef RT_USING_TOUCH

#if LVGL_VERSION_MAJOR < 9
    static lv_indev_drv_t indev_drv;         /* Descriptor of a input device
  driver */
    lv_indev_drv_init(&indev_drv);           /* Basic initialization */
    indev_drv.type = LV_INDEV_TYPE_POINTER;  /* Touch pad is a pointer-like
  device */
    indev_drv.read_cb = touchpad_read;       /* Set your driver function */

    /* Register the driver in LVGL and save the created input device object */
    touch_indev = lv_indev_drv_register(&indev_drv);
#else
    /*Register a touchpad input device*/
    touch_indev = lv_indev_create();
    lv_indev_set_type(touch_indev, LV_INDEV_TYPE_POINTER);
    lv_indev_set_read_cb(touch_indev, touchpad_read);
#endif
    /* Register touch device */
    rt_err_t res = rt_hw_gt9147_register();
    RT_ASSERT(res == RT_EOK);

#endif
}
```

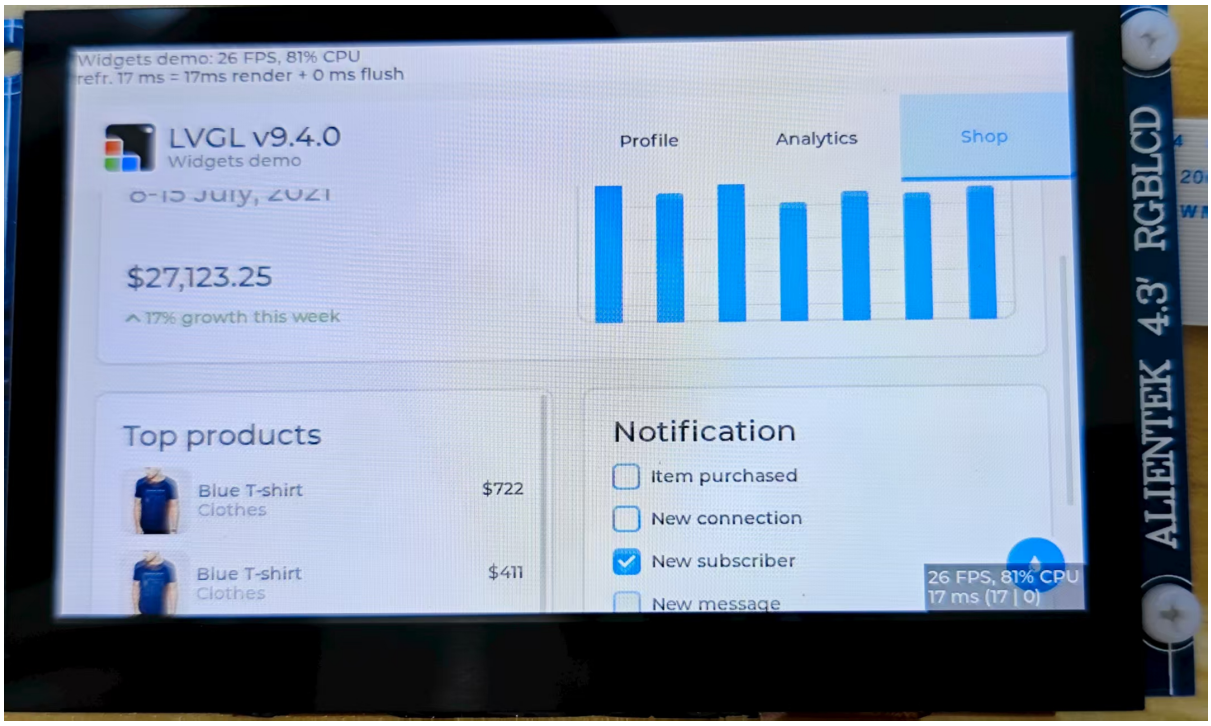
5.7 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio's package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board's USB-DBG interface to the PC and download the firmware to the development board.

5.8 Run Effect

After resetting the development board, you can see the LVGL demo running on the LCD screen.



Multicore Communication

1. Dual-core Project Instructions (Core0)

English | Chinese

1.1 Introduction

This project is based on the **Titan Board** development board and runs on the **Cortex-M85 (Core0)**, serving as the primary control template project in a dual-core system.

Core0 (Cortex-M85 @ 1GHz): The high-performance main core, responsible for major system logic, complex computations, and peripheral management.

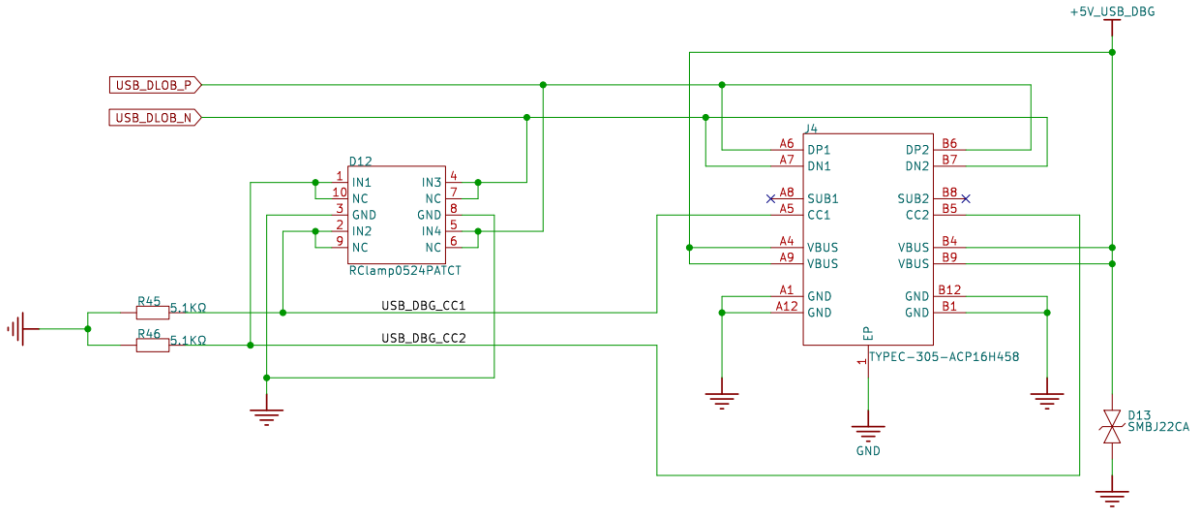
Core1 (Cortex-M33 @ 250MHz): The low-power auxiliary core, suitable for handling real-time tasks, security services, or low-power scenarios.

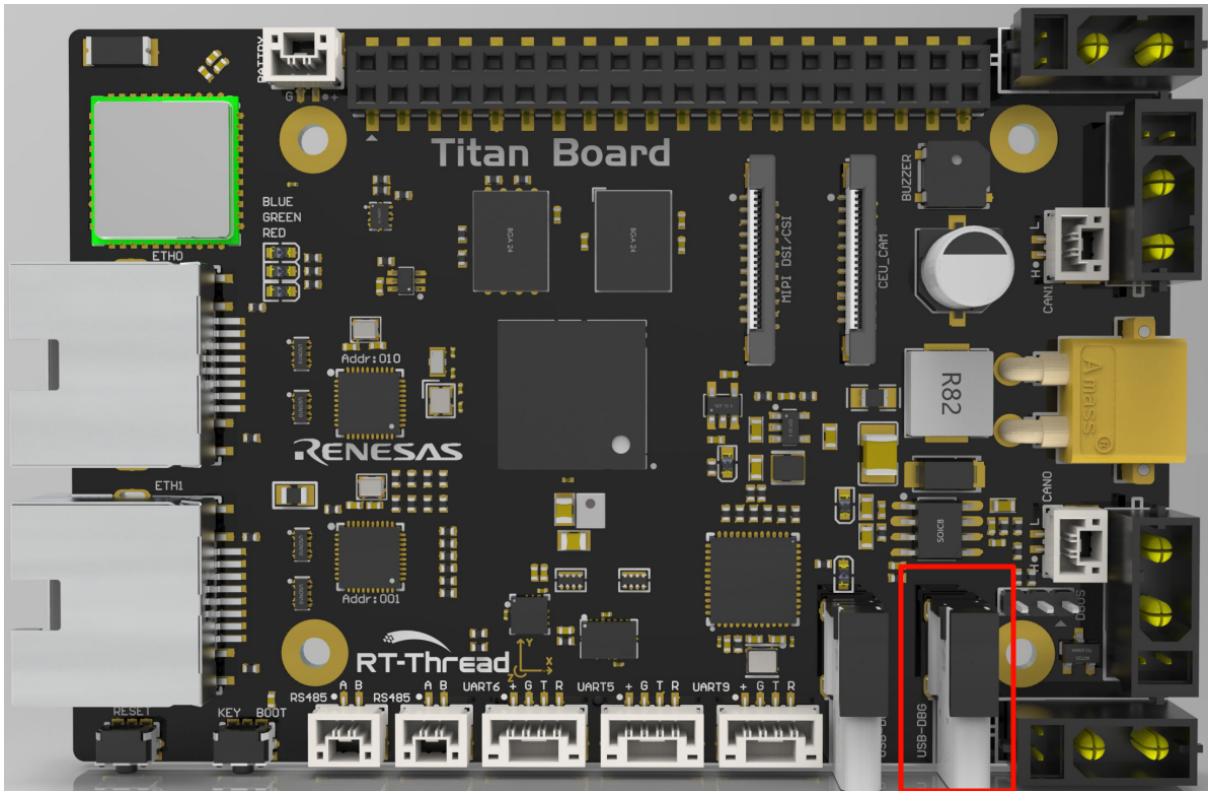
This template project provides the basic startup code, RT-Thread support, and peripheral initialization framework for subsequent application development.

Note: The **Titan_dual_core0** and **Titan_dual_core1** projects are designed to work together.

1.2 Hardware Description

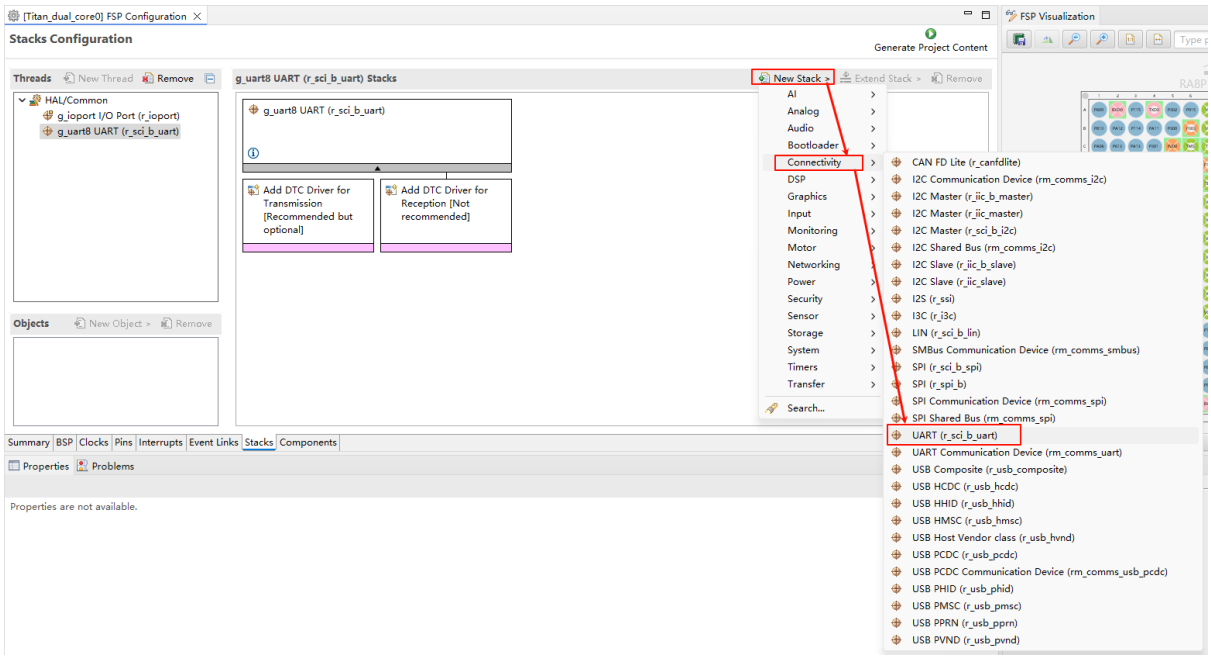
Core0 uses UART8 (USB-DBG) as the debug serial port.



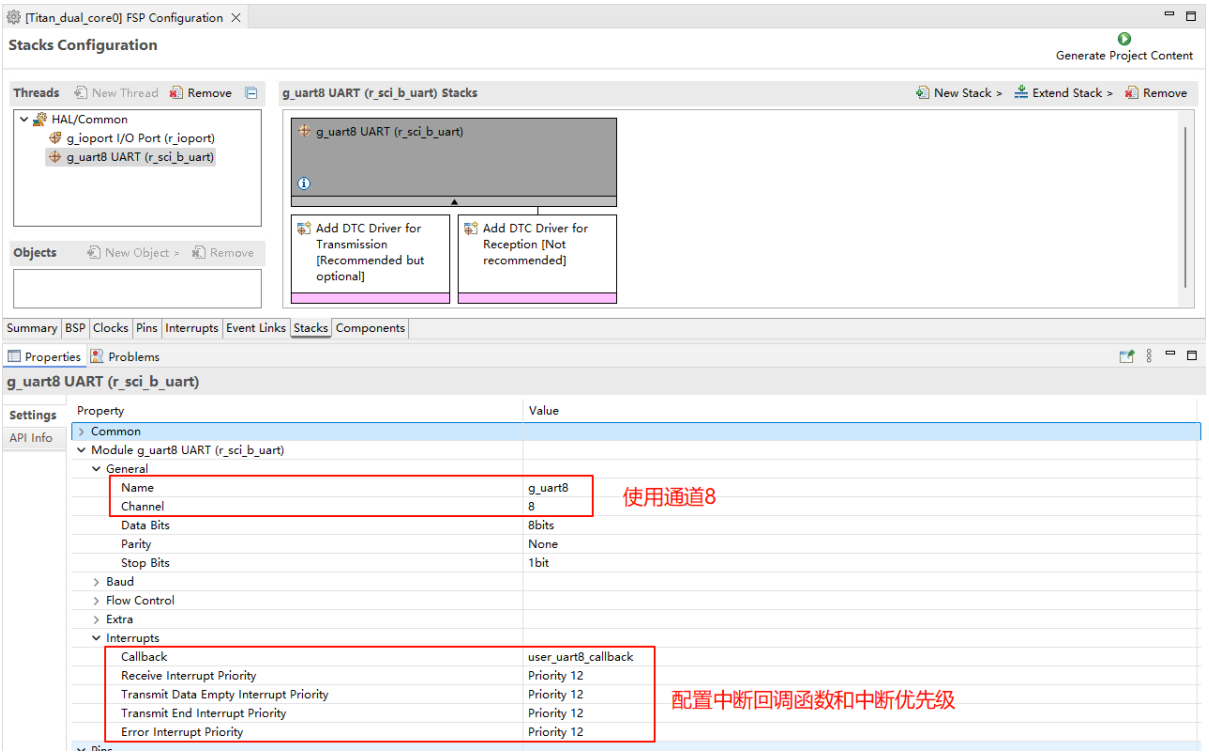


1.3 FSP Configuration

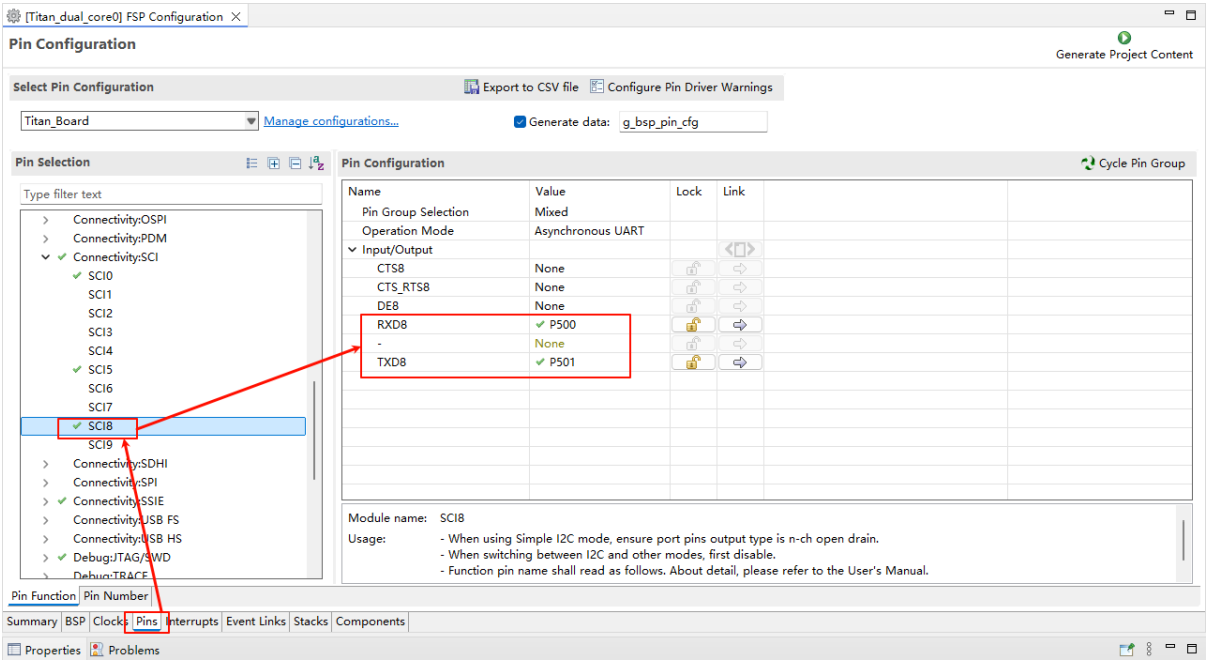
- Create a r_sci_b_uart stack:



- Configure r_sci_b_uart:

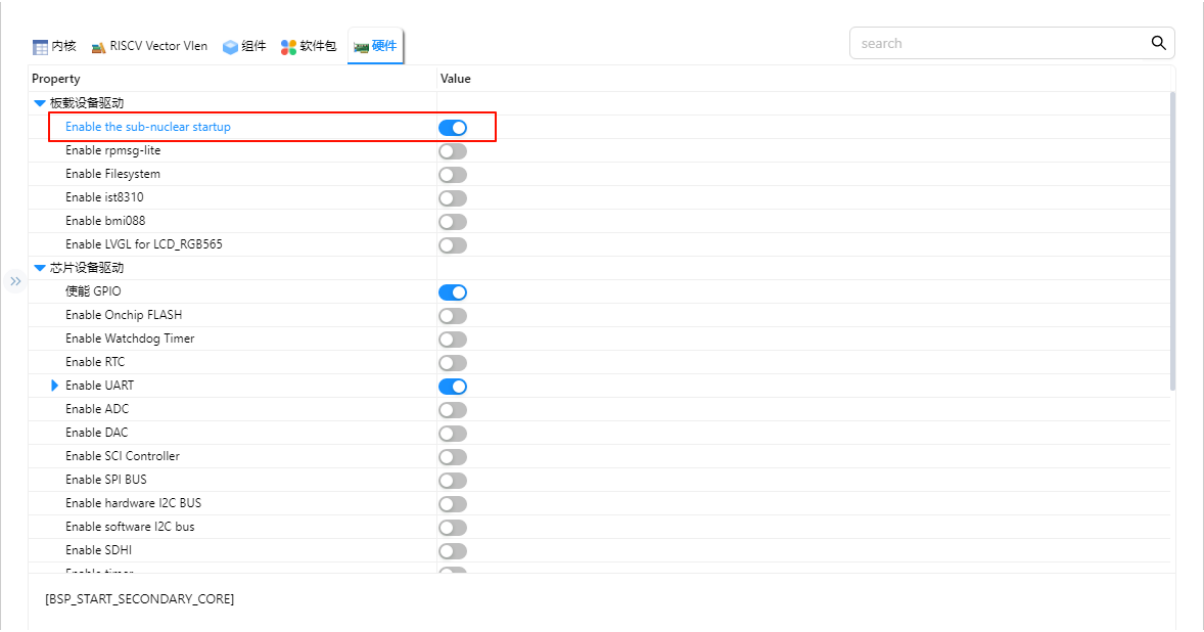


- Configure UART8 pins:



1.4 RT-Thread Settings Configuration

- Enable the sub-nuclear startup.



1.5 Example Code Description

The sample program is located in /src/hal_entry.c and uses the blue LED (LED_B) to indicate the running status of Core0.

```
#include <rtthread.h>
#include "hal_data.h"
#include <rtdevice.h>
#include <board.h>

#define LED_PIN     BSP_IO_PORT_00_PIN_12 /* Onboard LED pins */

void hal_entry(void)
{
    rt_kprintf("\n==================================================\n");
    rt_kprintf("This is core0!\n");
    rt_kprintf("Hello, Titan Board!\n");
    rt_kprintf("==================================================\n");

    rt_kprintf("\nThe dual-core project should collaborate with the SDK project
  for development: << Titan_dual_core1 >>\n");

    while (1)
    {
        rt_pin_write(LED_PIN, PIN_HIGH);
```

(continues on next page)

(continued from previous page)

```
        rt_thread_mdelay(500);
        rt_pin_write(LED_PIN, PIN_LOW);
        rt_thread_mdelay(500);
    }
}
```

1.6 Compilation & Download

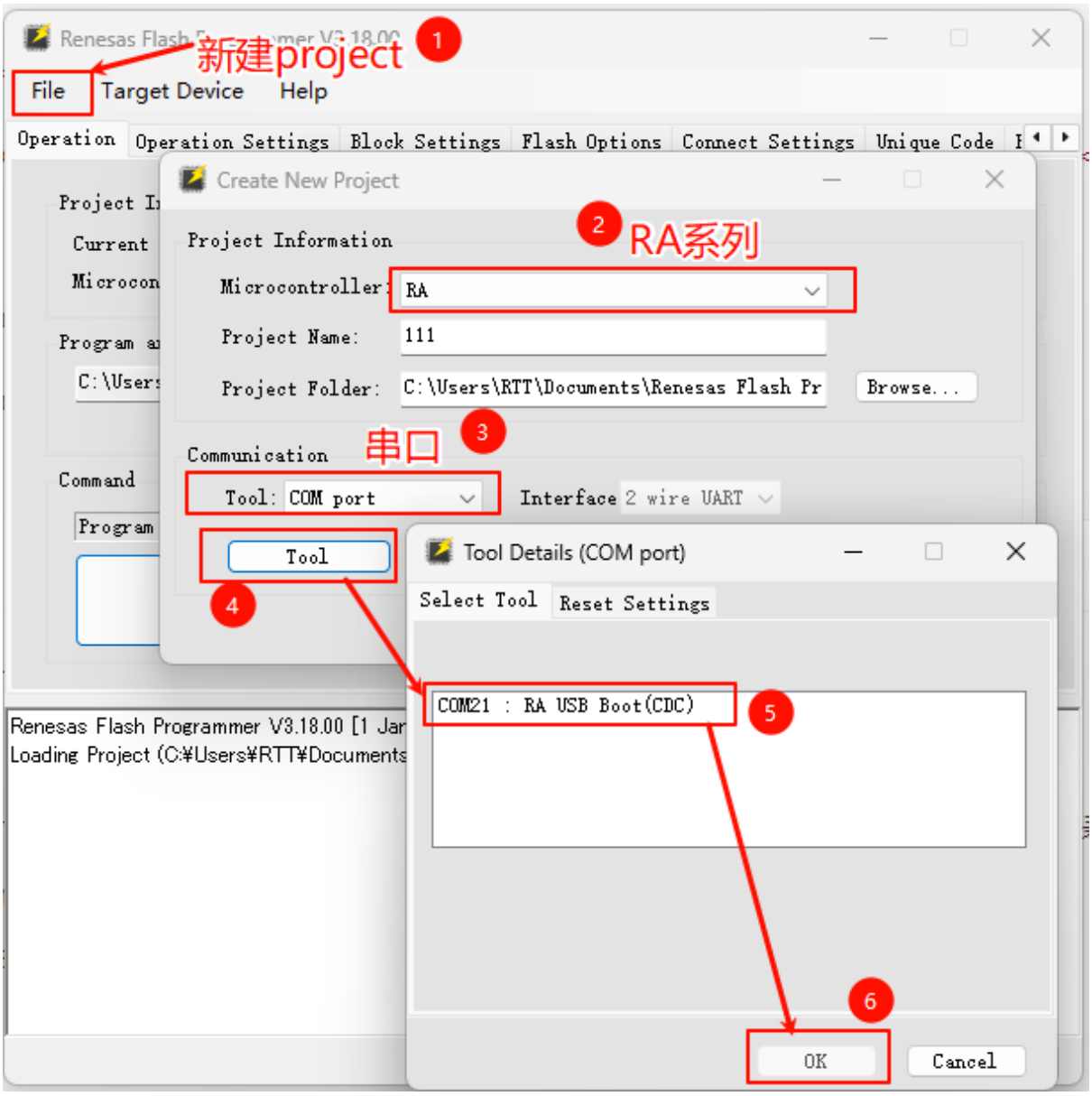
- **RT-Thread Studio**: In RT-Thread Studio's package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board's USB-DBG interface to the PC and download the firmware to the development board.

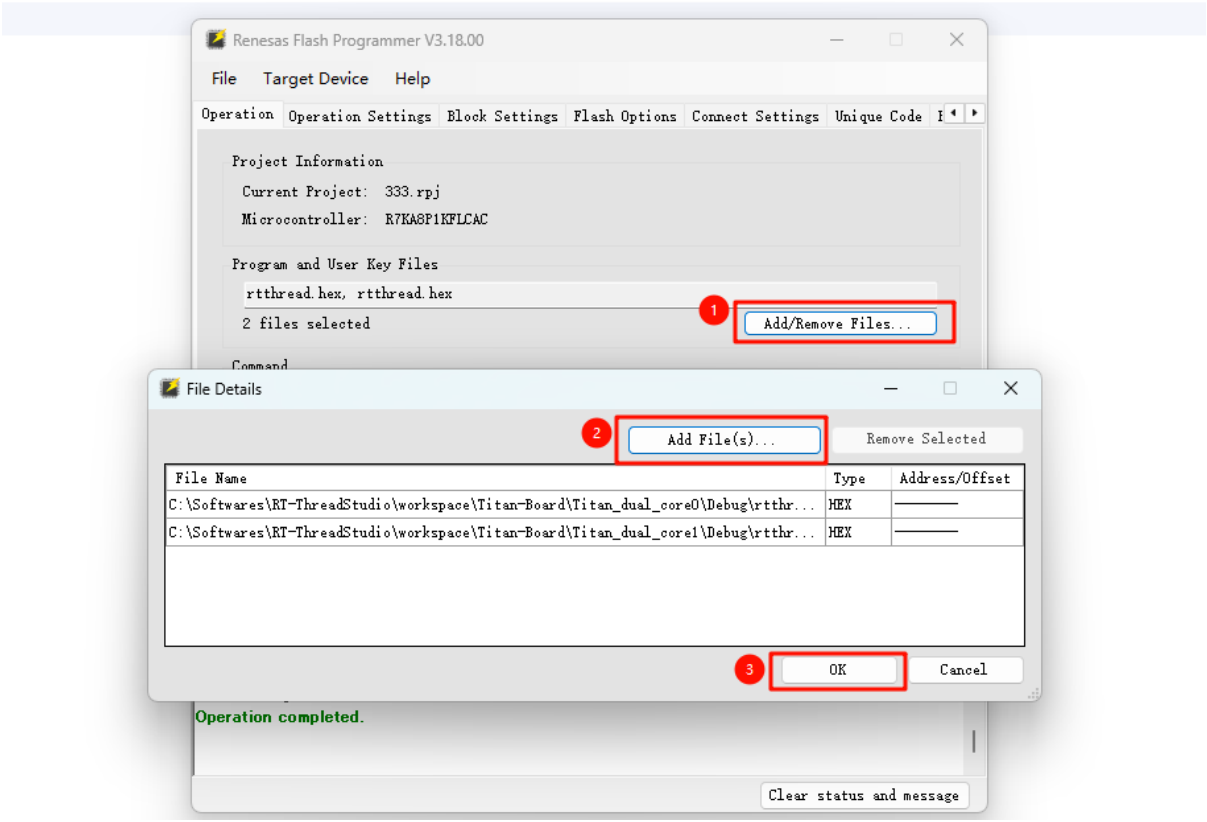
Download Methods

Currently, DAP-Link does not support programming the Core1 firmware, so you need to **use the Renesas Flash Programmer tool to program the firmware**. The detailed programming steps are as follows:

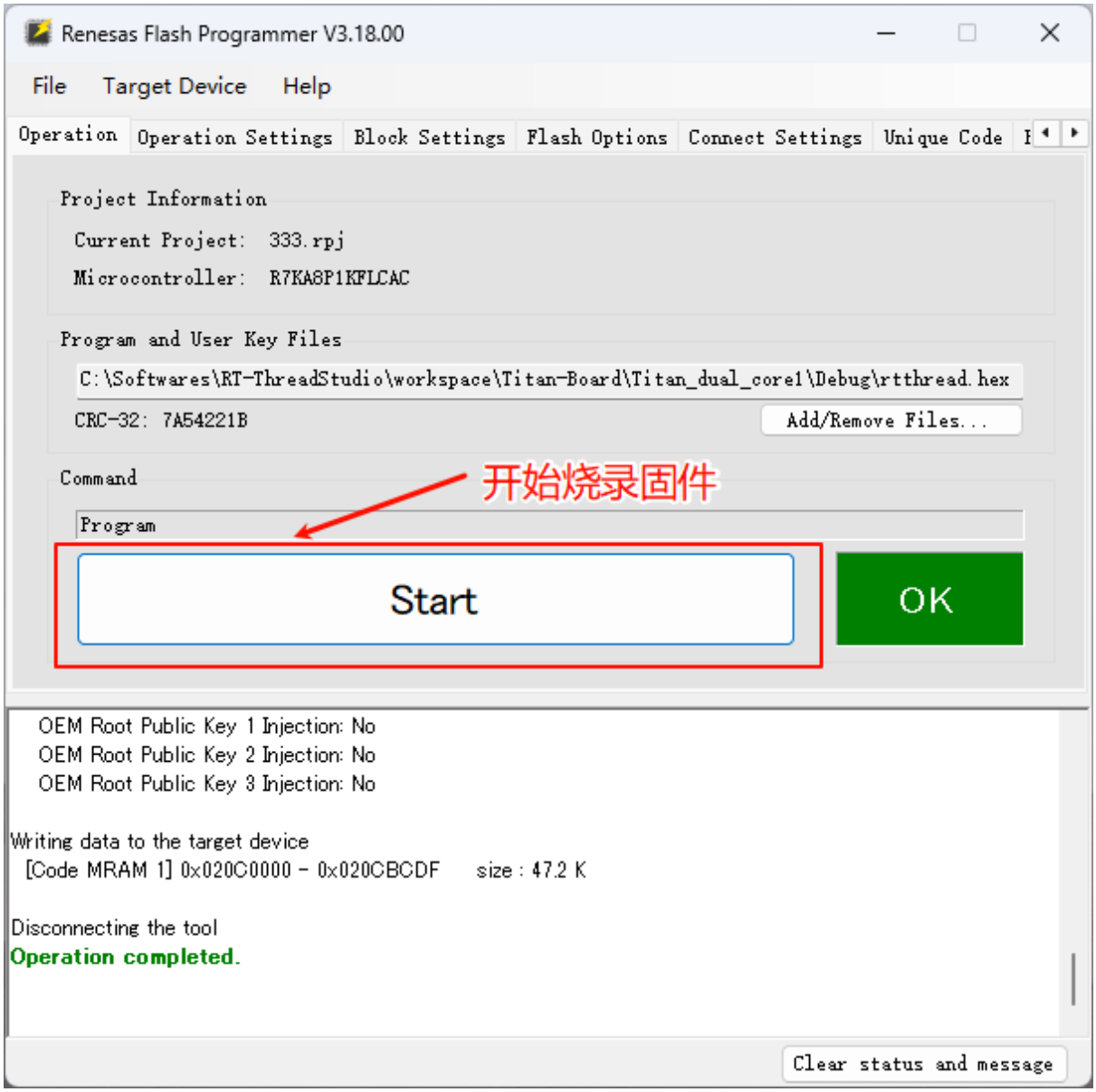
1. Download and install the Renesas Flash Programmer tool: Renesas Flash Programmer (Programming GUI) | Renesas.
2. Connect the USB-DEV port of the development board to the PC.
3. Create a project: The development board needs to enter BOOT mode (**hold down the BOOT button and press the RESET button once**).



4. Select the firmware to be programmed (Core0 and Core1 firmware):



5. Flashing firmware (at this time, the development board must be in **BOOT** mode):



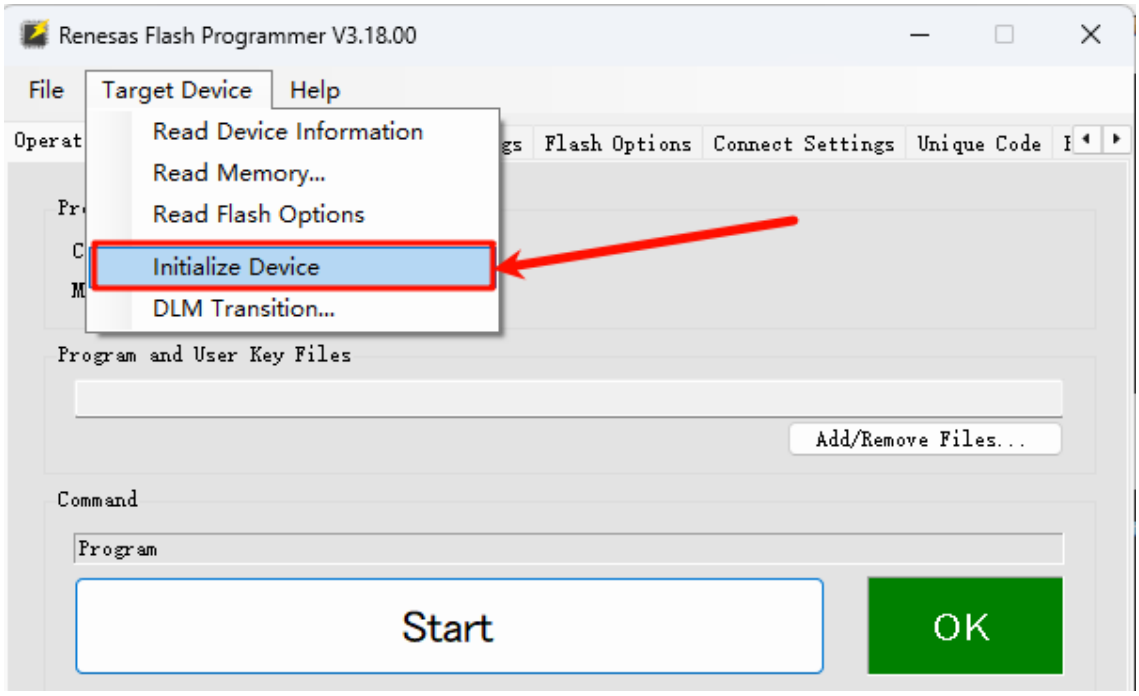
1.7 Run Effect

After resetting the development board, the blue LED will keep flashing.

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Aug 29 2025 13:34:27
 2006 - 2024 Copyright by RT-Thread team

================================================
This is core0!
Hello, Titan Board!
================================================

The dual-core project should collaborate with the SDK project for development: << Titan_dual_core1 >>
msh >
```

1.8 Notes on Using Dual-Core Projects

1. **DAP-Link does not currently support dual-core projects.** Please use **Renesas Flash Programmer (RFP)** for downloading, or replace the DAP-Link firmware with another debugging firmware (a tutorial is available on the RT-Thread community forum).
2. **Before downloading a dual-core project**, it is recommended to use **Renesas Flash Programmer** to **erase the Flash memory**.



3. **When configuring FSP for dual-core projects**, Core0 and Core1 must not configure the **same peripheral or pin** simultaneously. For example: if **UART8** is configured in Core0’ s FSP, then **UART8** must not be configured again in Core1’ s FSP.

2. Dual-core Project Instructions (Core1)

English | Chinese

2.1 Introduction

This project is based on the **Titan Board** development board and runs on the **Cortex-M33 (Core1)**, serving as the primary control template project in a dual-core system.

Core0 (Cortex-M85 @ 1GHz): The high-performance main core, responsible for major system logic, complex computations, and peripheral management.

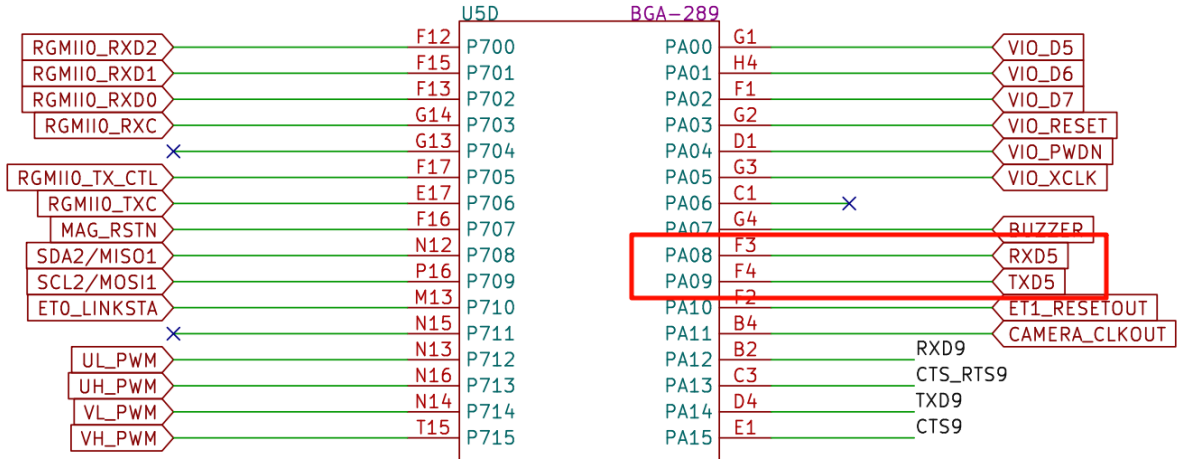
Core1 (Cortex-M33 @ 250MHz): The low-power auxiliary core, suitable for handling real-time tasks, security services, or low-power scenarios.

This template project provides the basic startup code, RT-Thread support, and peripheral initialization framework for subsequent application development.

Note: The **Titan_dual_core0** and **Titan_dual_core1** projects are used together **(Core1 is started by Core0)**.

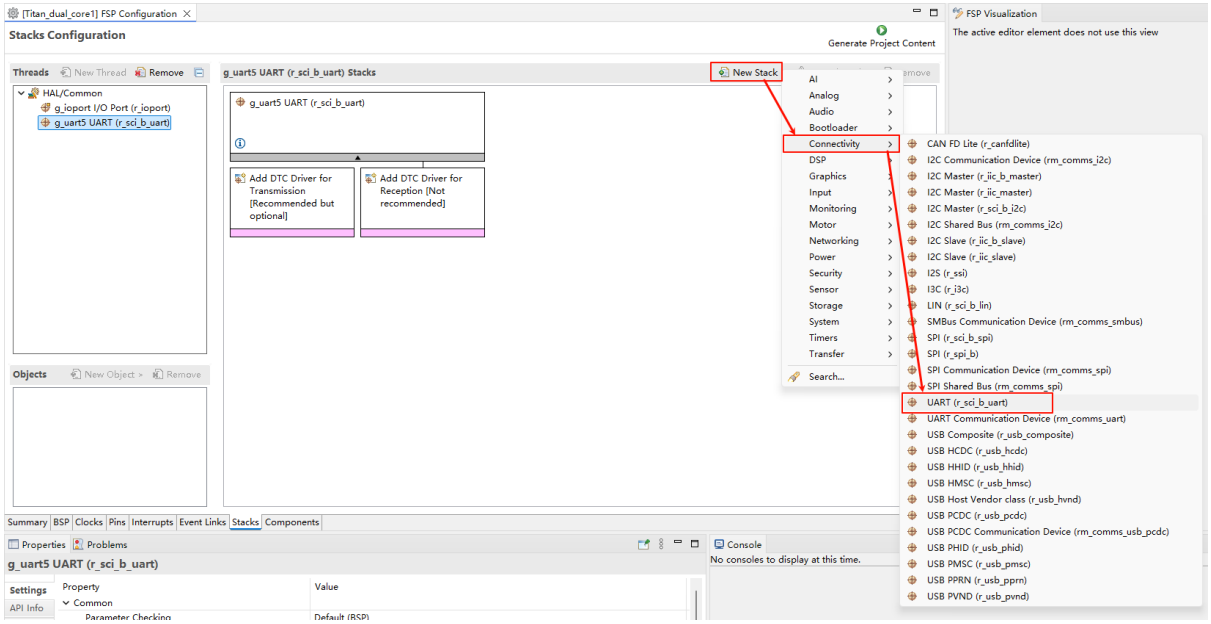
2.2 Hardware Description

Core1 uses UART5 as the debug serial port.

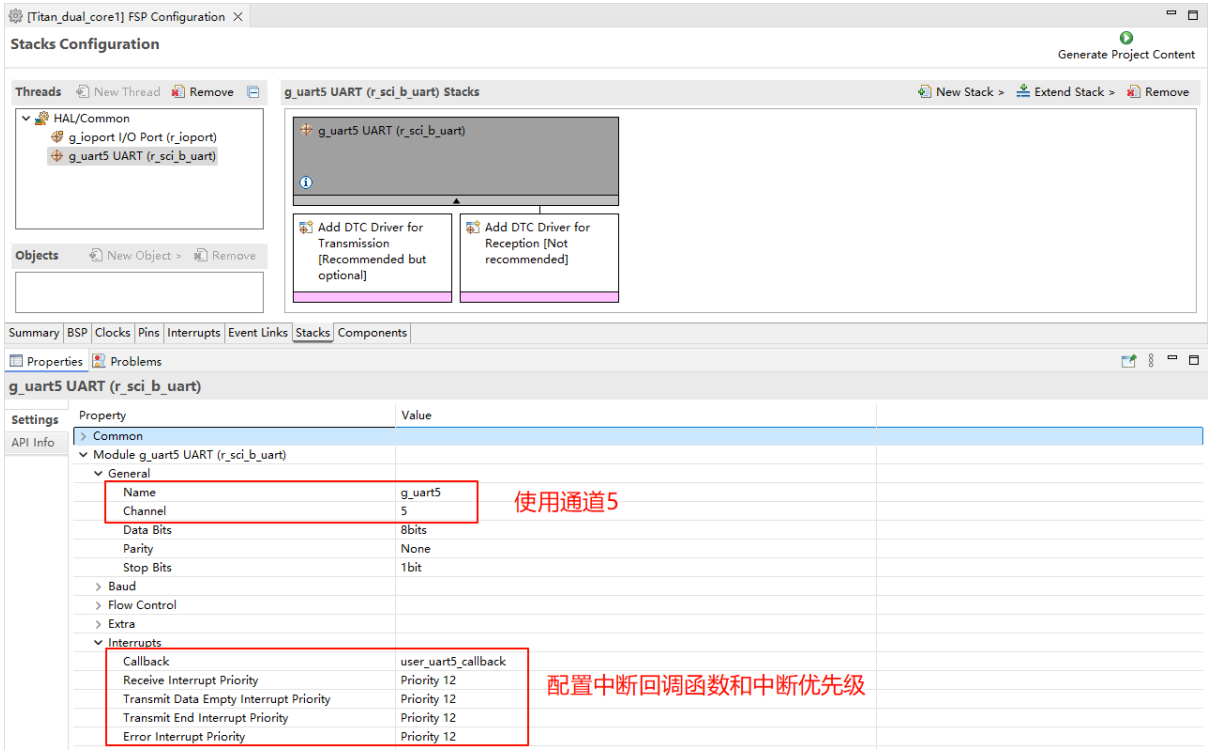


2.3 FSP Configuration

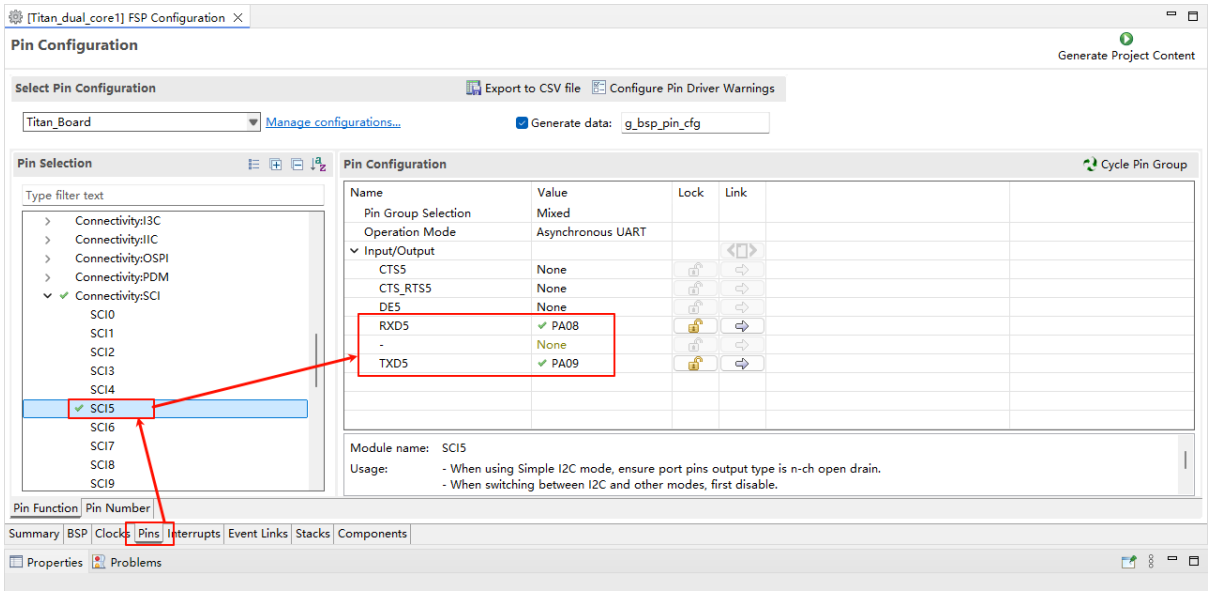
- Create a r_sci_b_uart stack:



- Configure r_sci_b_uart:

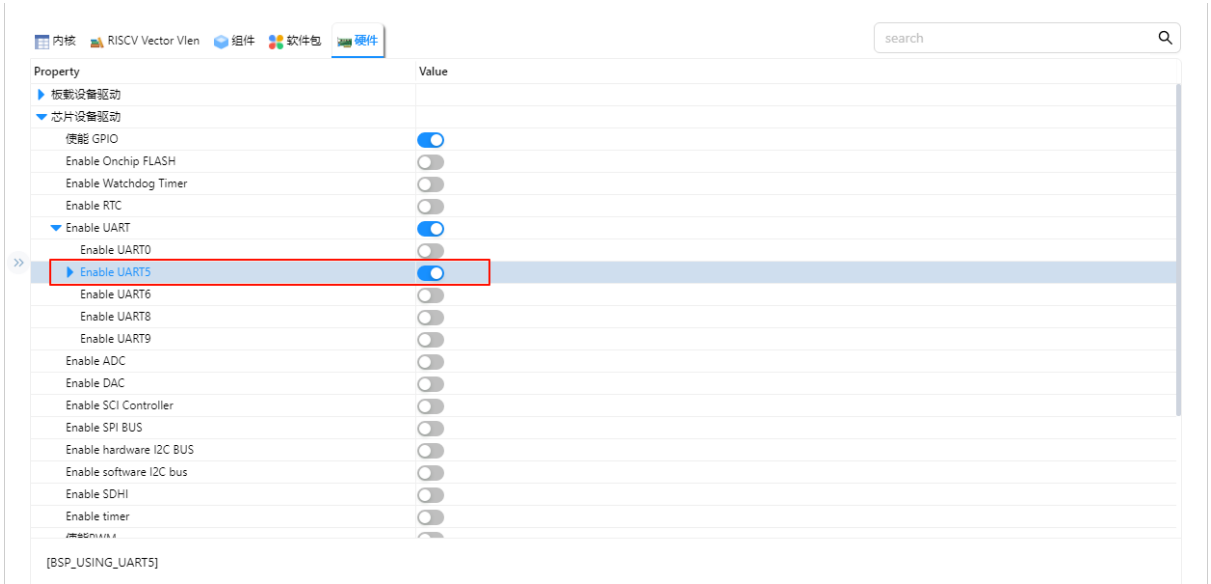


- Configure UART5 pin:



2.4 RT-Thread Settings Configuration

- Enable UART5.



2.5 Example Code Description

The sample program is located in /src/hal_entry.c and uses the green LED (LED_G) to indicate the running status of Core1.

```
#include <rtthread.h>
#include "hal_data.h"
#include <rtdevice.h>
```

(continues on next page)

(continued from previous page)

```
#include <board.h>

#define LED_PIN    BSP_IO_PORT_06_PIN_13 /* Onboard LED pins */

void hal_entry(void)
{
    rt_kprintf("\n=====================================================\n");
    rt_kprintf("This is core1!\n");
    rt_kprintf("Hello, Titan Board!\n");
    rt_kprintf("=====================================================\n");

    rt_kprintf("\nThe dual-core project should collaborate with the SDK project
  for development: << Titan_dual_core0 >>\n");
    while (1)
    {
        rt_pin_write(LED_PIN, PIN_HIGH);
        rt_thread_mdelay(1000);
        rt_pin_write(LED_PIN, PIN_LOW);
        rt_thread_mdelay(1000);
    }
}
```

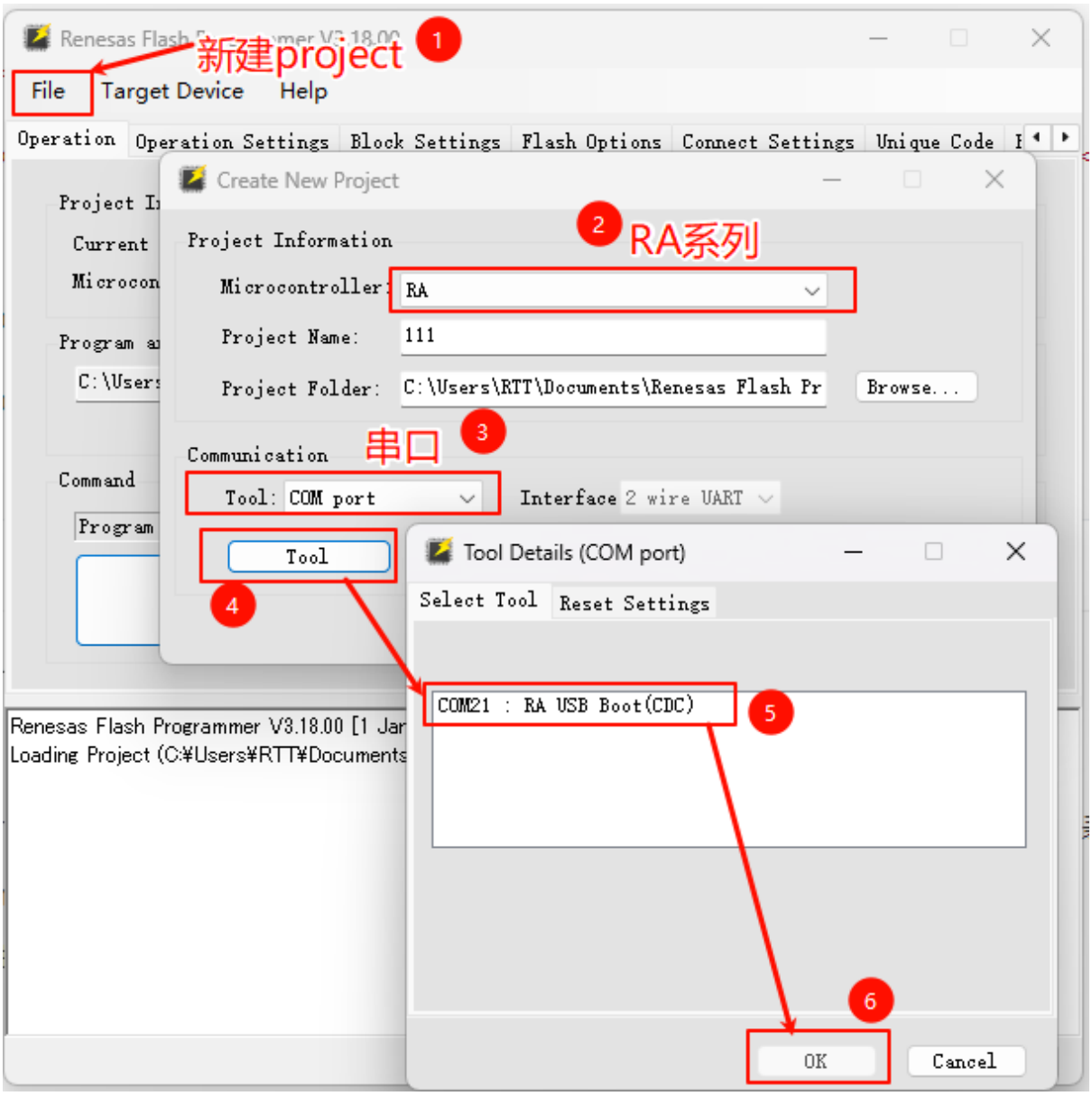
2.6 Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio ' s package manager, download the Titan Board resource package, create a new project, and compile it.

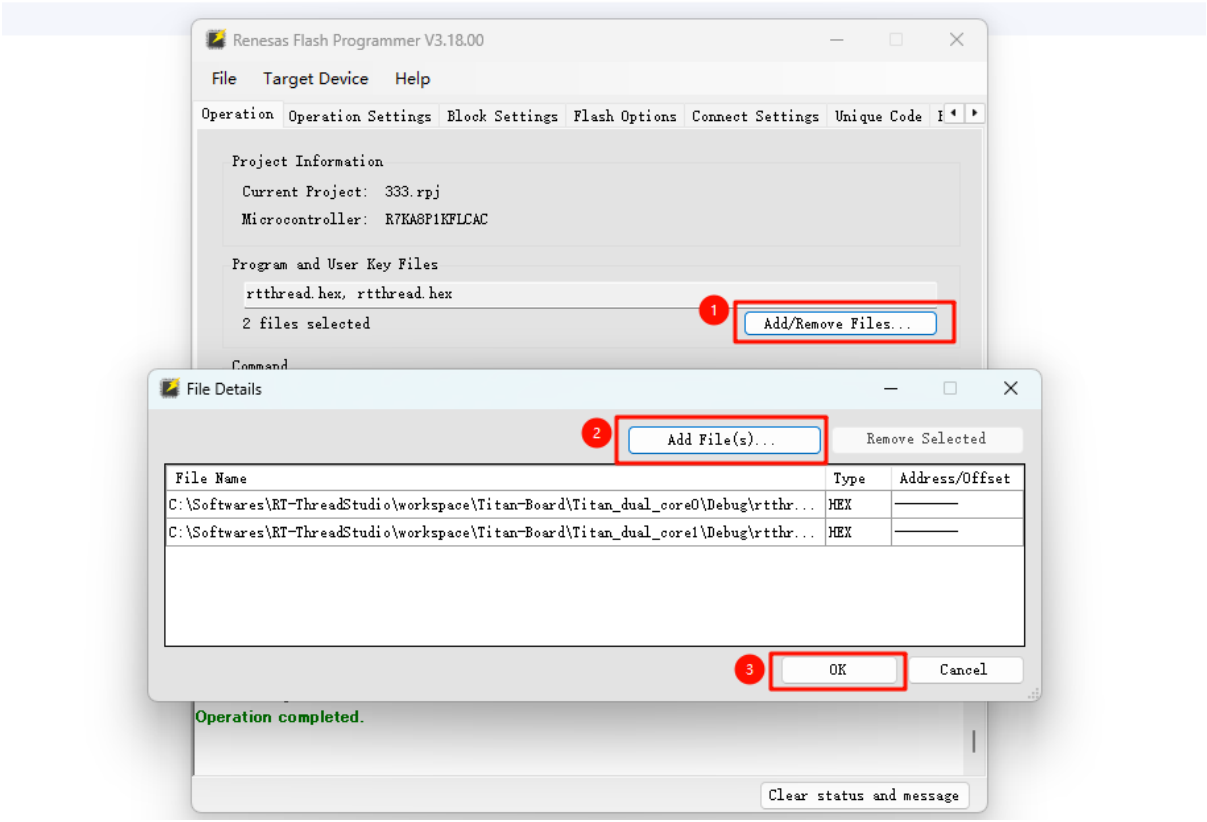
Download Methods

Currently, DAP-Link does not support programming the Core1 firmware, so you need to **use the Renesas Flash Programmer tool to program the firmware**. The detailed programming steps are as follows:

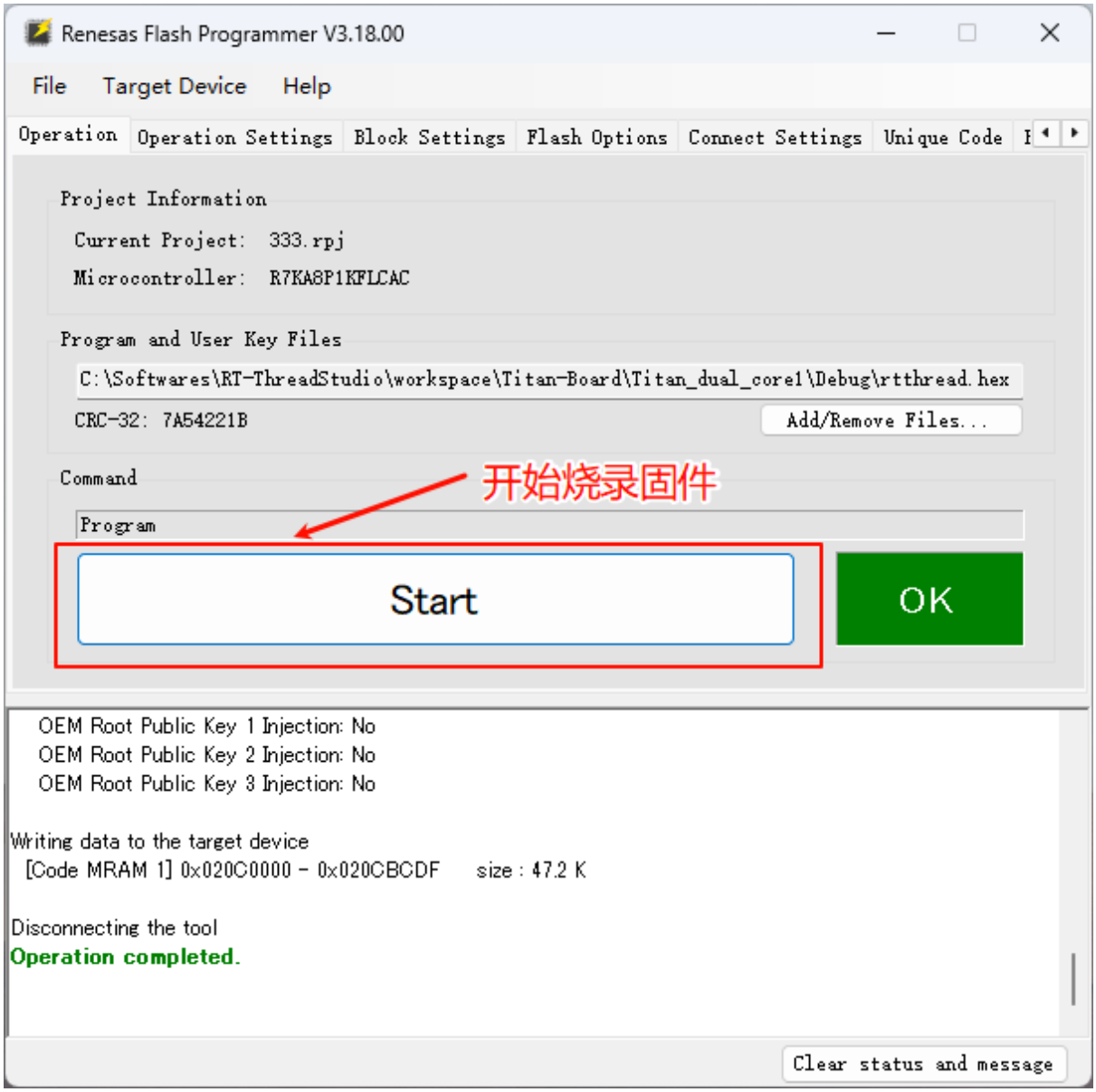
1. Download and install the Renesas Flash Programmer tool: Renesas Flash Programmer (Programming GUI) | Renesas.
2. Connect the USB-DEV port of the development board to the PC.
3. Create a project: The development board needs to enter BOOT mode (**hold down the BOOT button and press the RESET button once**).



4. Select the firmware to be programmed (Core0 and Core1 firmware):



5. Flashing firmware (at this time, the development board must be in **BOOT** mode):



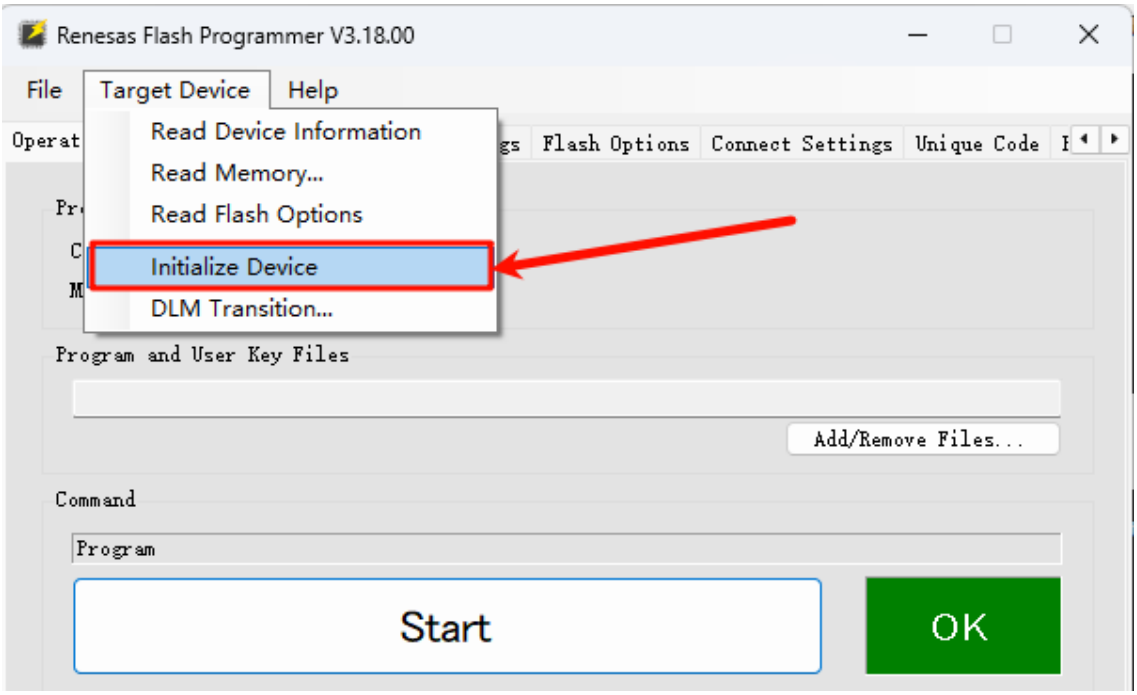
2.7 Run Effect

After resetting the development board, the blue (Core0) and green (Core1) LEDs will keep flashing continuously.

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Aug 29 2025 13:45:36
 2006 - 2024 Copyright by RT-Thread team

================================================
This is core1!
Hello, Titan Board!
================================================

The dual-core project should collaborate with the SDK project for development: << Titan_dual_core0 >>
msh >
```

2.8 Notes on Using Dual-Core Projects

1. **DAP-Link does not currently support dual-core projects.** Please use **Renesas Flash Programmer (RFP)** for downloading, or replace the DAP-Link firmware with another debugging firmware (a tutorial is available on the RT-Thread community forum).
2. **Before downloading a dual-core project**, it is recommended to use **Renesas Flash Programmer** to **erase the Flash memory**.



3. **When configuring FSP for dual-core projects**, Core0 and Core1 must not configure the **same peripheral or pin** simultaneously. For example: if **UART8** is configured in Core0’ s FSP, then **UART8** must not be configured again in Core1’ s FSP.

3. RPMsg-Lite Usage Instructions（Core0 Master）

English | Chinese

3.1 Introduction

This project runs on **RA8P1 Core0 (Cortex-M85, nominal 1 GHz)** and serves as the main control application in a dual-core system:

- Responsible for **system initialization, peripheral resource management**, and **Core1 startup/ management**.
- Establishes a **message channel** with Core1 over shared memory using **RPMsg-Lite** (virtqueue/ vring), supporting command delivery, data exchange, event notification, etc.

Key Features

- **Multicore Communication**: RPMsg-Lite master-side initialization, endpoint creation, message send/receive, blocking/non-blocking queues.
- **RT-Thread Support**: Dedicated communication thread, safe queues, event synchronization.
- **Shared Memory & Cache Safety**: Reserved **non-cacheable / strongly consistent** shared memory regions; provides memory barriers and cache operation guidelines.
- **Clear Startup Sequence**: Core0 handles loading/launching Core1 and performs **link-up/hand-shake**.
- **Trimmed Protocol Layer**: Includes simple **Ping/Pong** and **frame format** examples for easy expansion of application command sets.

3.2 RPMsg-Lite Instructions

RPMsg-Lite is a lightweight implementation of Remote Processor Messaging (RPMsg), primarily used in multi-core processor systems to enable data exchange and message passing between different cores. It is based on the **OpenAMP** framework and leverages shared memory and virtual channels (Virtio Channels) to achieve efficient and reliable inter-core communication in resource-constrained embedded systems.

Design Background

In multi-core MCUs or SoC systems, different cores may execute different types of tasks, for example:

- Cortex-M cores run real-time control tasks
- Cortex-A cores run operating systems or high-level applications

In such cases, an efficient communication mechanism is needed to exchange data or trigger events between cores. RPMsg-Lite provides a lightweight, easy-to-use, and reliable message-passing solution.

Key Features

1. **Lightweight**
   - Compared with full RPMsg implementations, RPMsg-Lite consumes less memory and CPU resources, making it ideal for resource-constrained embedded systems.
   - Core data structures and queue mechanisms are optimized to minimize memory copies.
2. **Inter-core Communication**
   - Supports communication between different cores.
   - Uses shared memory as a data buffer, with messages transmitted via virtual channels.
3. **Reliability**
   - Provides reliable message delivery mechanisms to ensure messages are neither lost nor duplicated.
   - Configurable message queue lengths allow for high-reliability data transmission.
4. **Flexibility**
   - Supports static or dynamic creation of communication channels to adapt to different system architectures.
5. **Simple Interface**
   - Offers easy-to-use APIs for sending, receiving, and managing channels.
   - Developers can integrate it quickly without needing to understand the underlying shared memory or interrupt mechanisms.

Working Principle

RPMsg-Lite is based on the **virtio** virtual device mechanism:

1. Shared memory stores message data and queue information.
2. The sending core writes messages to the shared memory queue and notifies the receiving core via an interrupt.
3. The receiving core reads the messages and processes them according to application requirements.
4. After processing, the receiving core can send acknowledgment messages to ensure communication reliability.

Application Scenarios

- **Dual-core or Multi-core MCU systems**: For example, in the RA8P1 dual-core MCU, M85 handles high-performance tasks while M33 handles real-time tasks, with communication between the cores via RPMsg-Lite.
- **Real-time task separation**: Time-sensitive tasks and high-level application tasks can run separately to improve system performance.

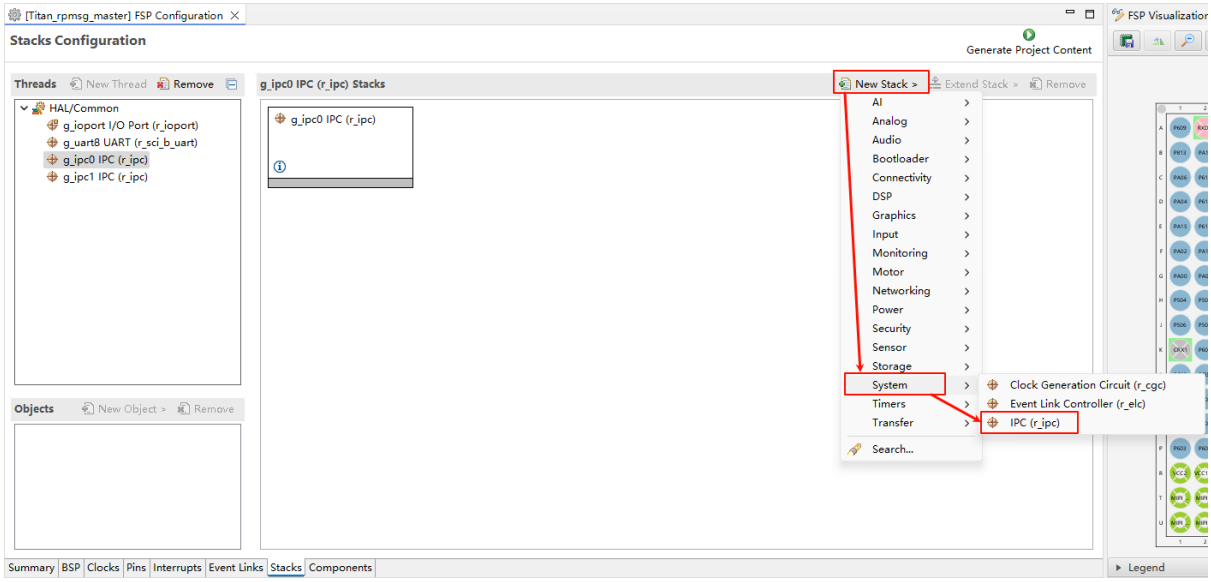
- **Resource-constrained embedded systems**: Low memory footprint and communication overhead make it suitable for industrial control, robotics, automotive electronics, and other fields.

3.3 Hardware Description

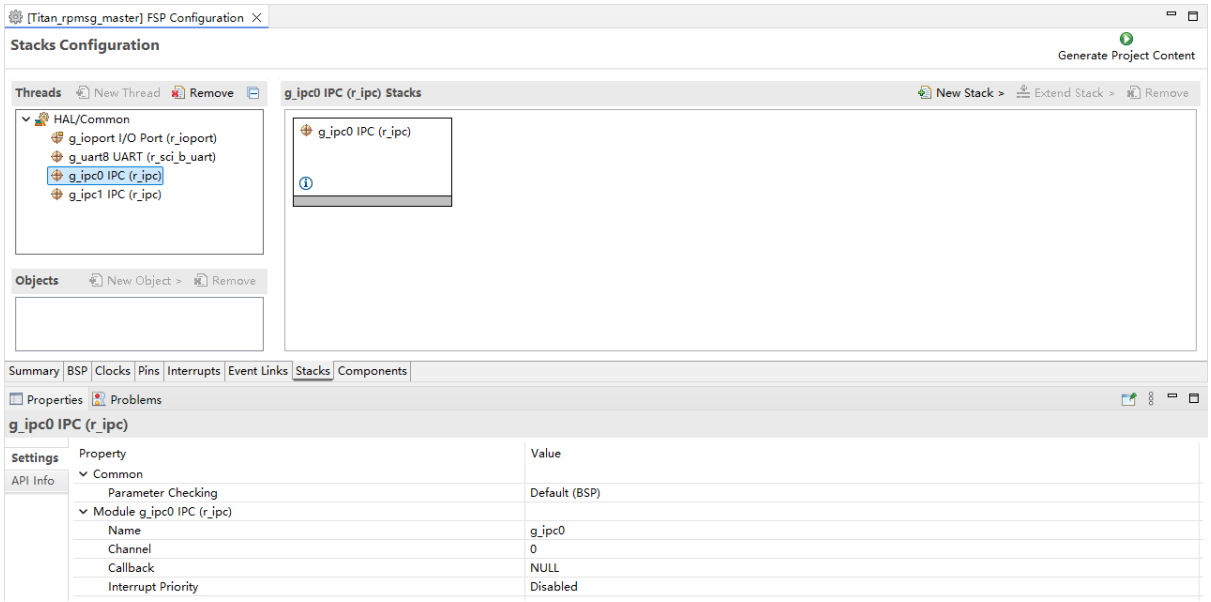
None

3.4 FSP Configuration

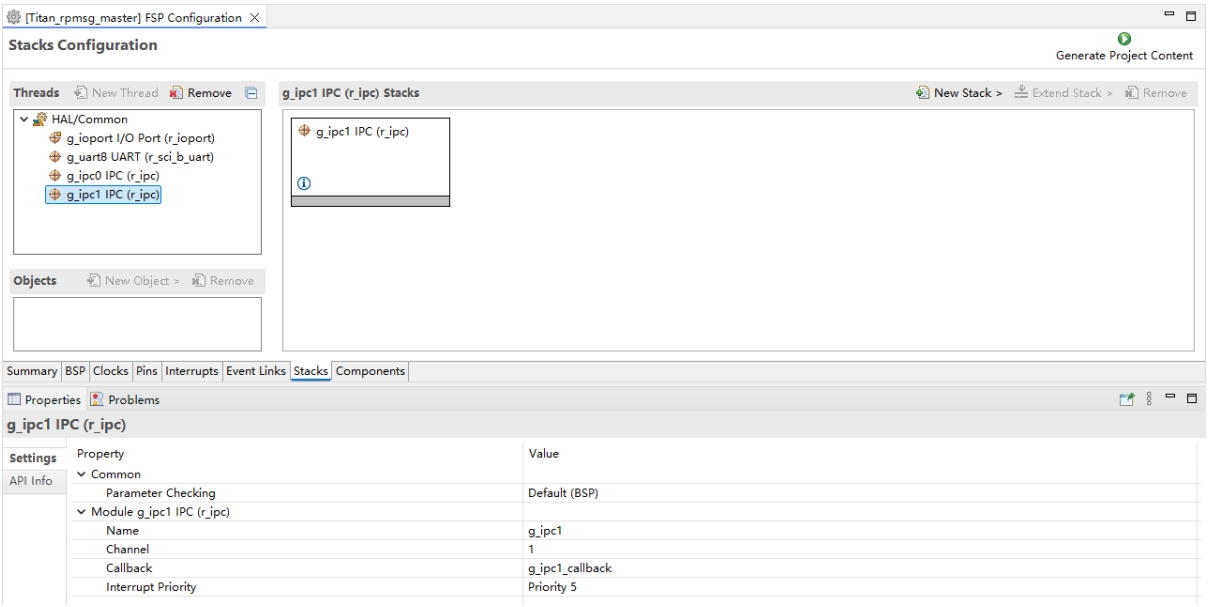
- Create two r_ipc stacks:



- Configure IPC0:

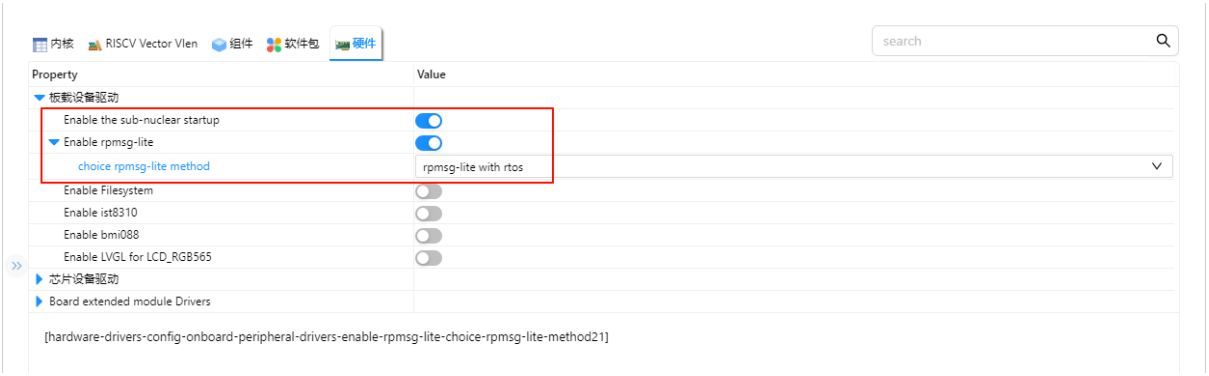


- Configure IPC1:



3.5 RT-Thread Settings Configuration

- Enable sub-nuclear startup and rpmsg-lite。



3.6 Example Code Description

The sample program is located at /board/ports/rpmsg-adapt.

3.7 Compilation & Download

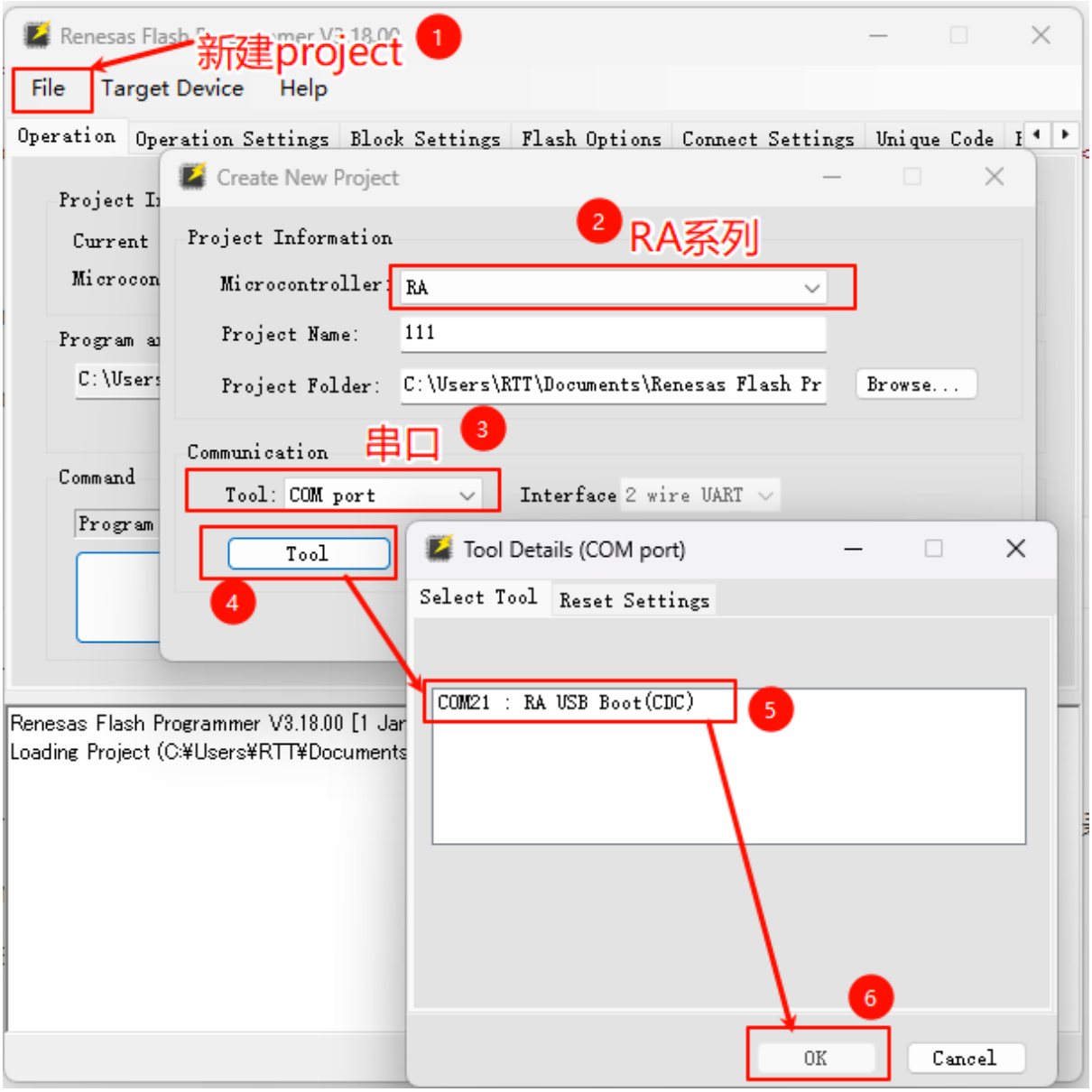
- **RT-Thread Studio**: In RT-Thread Studio’s package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board’s USB-DBG interface to the PC and download the firmware to the development board.

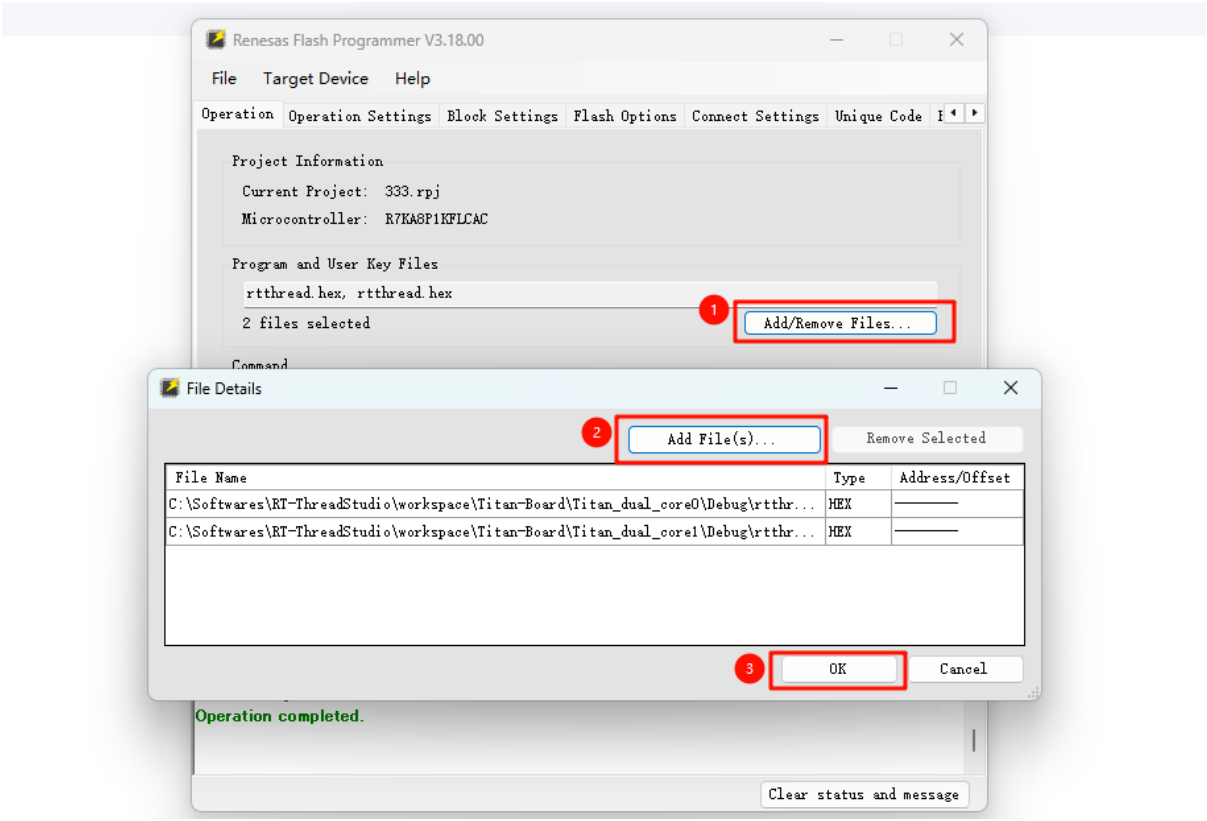
Download Methods

Currently, DAP-Link does not support programming the Core1 firmware, so you need to **use the Renesas Flash Programmer tool to program the firmware**. The detailed programming steps are as follows:

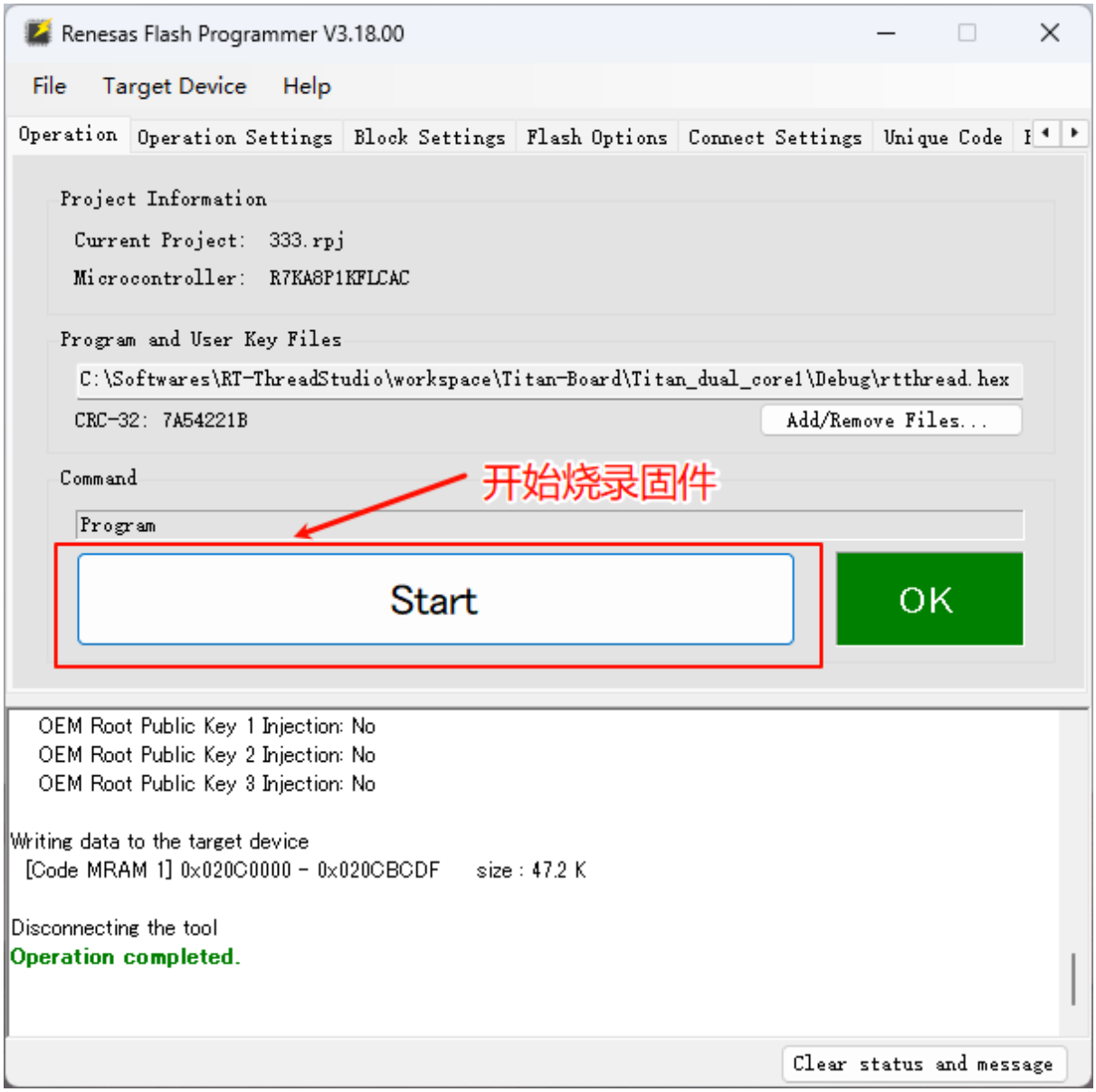
1. Download and install the Renesas Flash Programmer tool: Renesas Flash Programmer (Programming GUI) | Renesas.
2. Connect the USB-DEV port of the development board to the PC.
3. Create a project: The development board needs to enter BOOT mode (**hold down the BOOT button and press the RESET button once**).



4. Select the firmware to be programmed (Core0 and Core1 firmware):

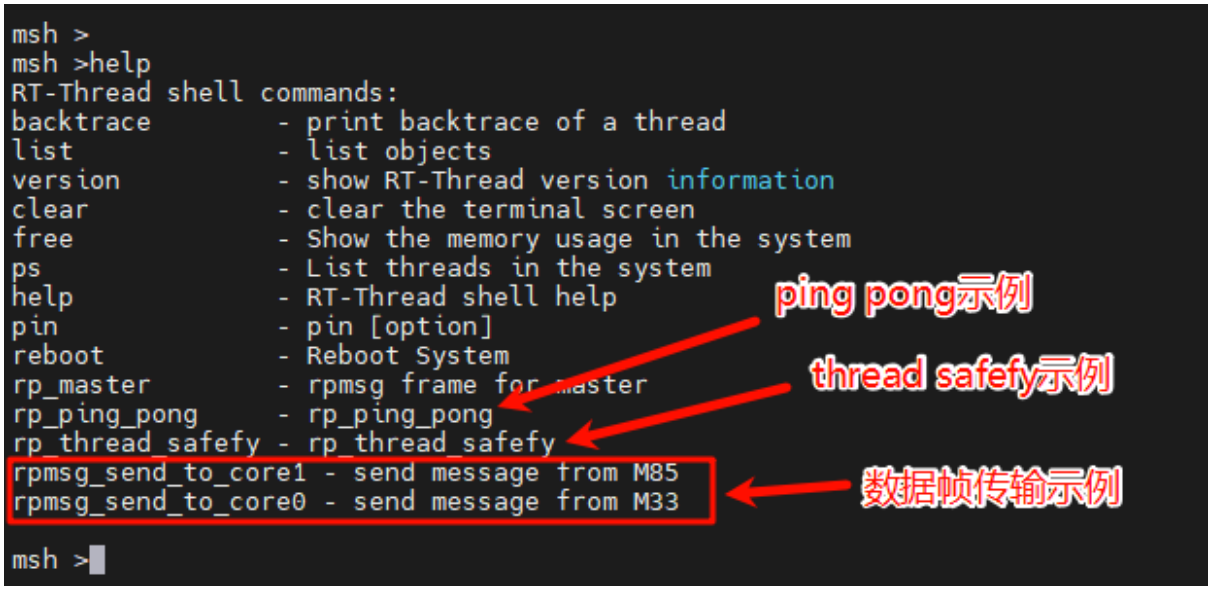


5. Flashing firmware (at this time, the development board must be in **BOOT** mode):



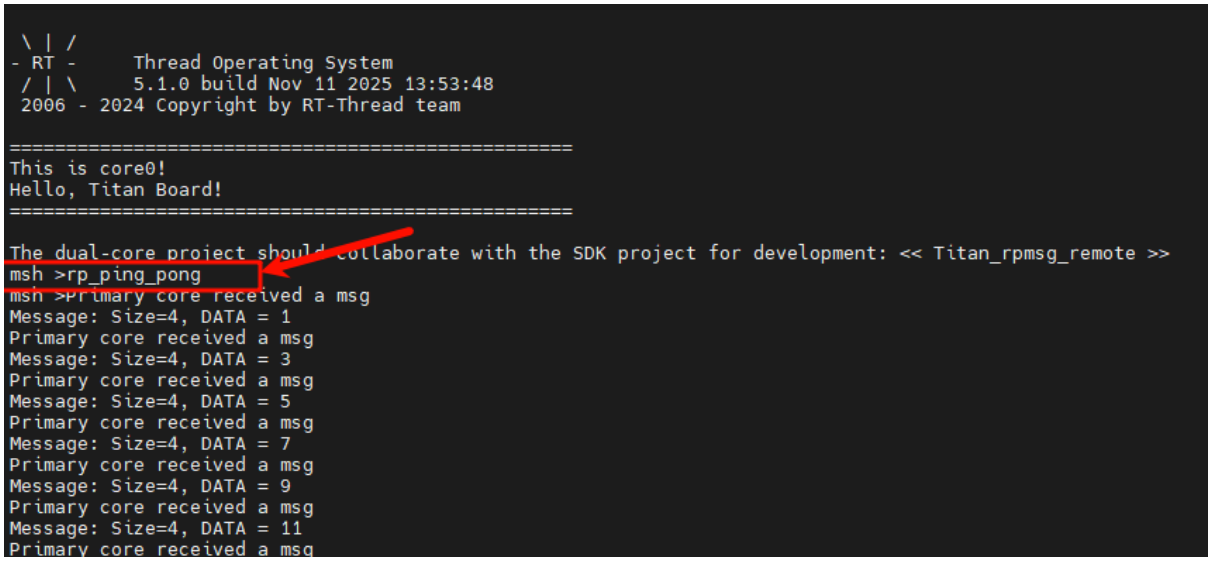
3.8 Run Effect

Note: In the RPMsg-Lite example project, before running the following examples —*ping pong*, *thread safety*, and *data frame transmission* —you must first start the **remote** endpoint on **Core1**.



Ping Pong Example

First, enter the rp_ping_pong command on the **Core1** terminal, then enter the same rp_ping_pong command on the **Core0** terminal to start the RPMsg master.



Thread Safety Example

First, enter the rp_thread_safefy command on the **Core1** terminal, then enter the same rp_thread_safefy command on the **Core0** terminal to start the RPMsg master.

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Nov 11 2025 13:53:48
 2006 - 2024 Copyright by RT-Thread team

================================================
This is core0!
Hello, Titan Board!
================================================

The dual-core project should collaborate with the SDK project for development: << Titan_rpmsg_remote >>
msh >rp_thread_safefy
              << Official thread-safety test of the rpmsg-lite component (RTOS) >>

Verification method: If the operation is successful, no information will be output; if it fails, an assert log will be prompted.
                     if it fails, an assert log will be prompted, as follows:
                     Assertion failed: xxx...
You can enable the reception of logs by defining the macro RPMSG_APP_LOG.
However, this action may result in the delay of data reception. It is recommended to disable it!

msh >All test procedures have been completed.
```

Data Frame Transmission Example

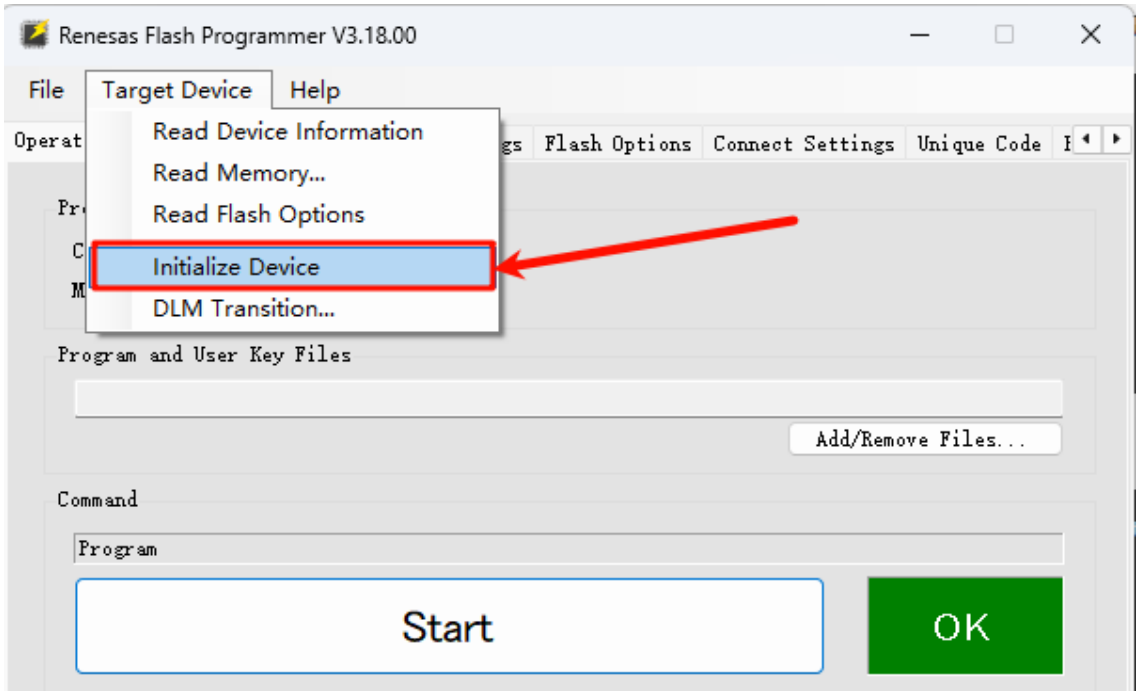
First, enter the rp_remote command on the **Core1** terminal to start the remote endpoint, then enter the rp_master command on the **Core0** terminal to start the RPMsg master.

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Nov 11 2025 13:53:48
 2006 - 2024 Copyright by RT-Thread team

================================================
This is core0!
Hello, Titan Board!
================================================

The dual-core project should collaborate with the SDK project for development: << Titan_rpmsg_remote >>
msh >rp_master
msh >[I/rpmsg.comm] Master rpmsg communication initialized

[I/rpmsg.master] ================================================
[I/rpmsg.master] Start listening for data from core1...
[I/rpmsg.master] ================================================
```

After that, you can use the rpmsg_send_to_core1 command to send data to **Core1**.

3.9 Notes on Using Dual-Core Projects

1. **DAP-Link does not currently support dual-core projects.** Please use **Renesas Flash Programmer (RFP)** for downloading, or replace the DAP-Link firmware with another debugging firmware (a tutorial is available on the RT-Thread community forum).
2. **Before downloading a dual-core project**, it is recommended to use **Renesas Flash Programmer** to **erase the Flash memory**.



3. **When configuring FSP for dual-core projects**, Core0 and Core1 must not configure the **same peripheral or pin** simultaneously. For example: if **UART8** is configured in Core0’ s FSP, then **UART8** must not be configured again in Core1’ s FSP.

4. RPMsg-Lite Usage Instructions（Core0 Master）

English | Chinese

4.1 Introduction

This project runs on **Core1 of RA8P1 (Cortex-M33, 250 MHz)** as the remote core in a dual-core system:

- Receives commands and data from Core0 (Master) via **RPMsg-Lite**;
- Handles specific tasks (such as communication processing, data acquisition, safety control, and low-power management) and sends results back to Core0;
- Supports independent application logic and a message-driven mechanism.

Key Features

- **Multicore Communication**: RPMsg-Lite Remote initialization, endpoint creation, and message transmission/reception.
- **RT-Thread Support**: Independent threads handle message sending and receiving, supporting event/queue synchronization.
- **Shared Memory & Cache Safety**: Uses shared memory reserved by Core0, ensuring cache coherency.
- **Handshake Mechanism**: HELLO/READY/ACK communication with the Core0 Master.
- **Extensible Protocol Layer**: Lightweight protocol examples (Ping/Pong, command frames, data streams).

4.2 RPMsg-Lite Instructions

RPMsg-Lite is a lightweight implementation of Remote Processor Messaging (RPMsg), primarily used in multi-core processor systems to enable data exchange and message passing between different cores. It is based on the **OpenAMP** framework and leverages shared memory and virtual channels (Virtio Channels) to achieve efficient and reliable inter-core communication in resource-constrained embedded systems.

Design Background

In multi-core MCUs or SoC systems, different cores may execute different types of tasks, for example:

- Cortex-M cores run real-time control tasks
- Cortex-A cores run operating systems or high-level applications

In such cases, an efficient communication mechanism is needed to exchange data or trigger events between cores. RPMsg-Lite provides a lightweight, easy-to-use, and reliable message-passing solution.

Key Features

1. **Lightweight**
   - Compared with full RPMsg implementations, RPMsg-Lite consumes less memory and CPU resources, making it ideal for resource-constrained embedded systems.
   - Core data structures and queue mechanisms are optimized to minimize memory copies.
2. **Inter-core Communication**
   - Supports communication between different cores.
   - Uses shared memory as a data buffer, with messages transmitted via virtual channels.
3. **Reliability**
   - Provides reliable message delivery mechanisms to ensure messages are neither lost nor duplicated.
   - Configurable message queue lengths allow for high-reliability data transmission.
4. **Flexibility**
   - Supports static or dynamic creation of communication channels to adapt to different system architectures.
5. **Simple Interface**
   - Offers easy-to-use APIs for sending, receiving, and managing channels.
   - Developers can integrate it quickly without needing to understand the underlying shared memory or interrupt mechanisms.

Working Principle

RPMsg-Lite is based on the **virtio** virtual device mechanism:

1. Shared memory stores message data and queue information.
2. The sending core writes messages to the shared memory queue and notifies the receiving core via an interrupt.
3. The receiving core reads the messages and processes them according to application requirements.
4. After processing, the receiving core can send acknowledgment messages to ensure communication reliability.

Application Scenarios

- **Dual-core or Multi-core MCU systems**: For example, in the RA8P1 dual-core MCU, M85 handles high-performance tasks while M33 handles real-time tasks, with communication between the cores via RPMsg-Lite.
- **Real-time task separation**: Time-sensitive tasks and high-level application tasks can run separately to improve system performance.

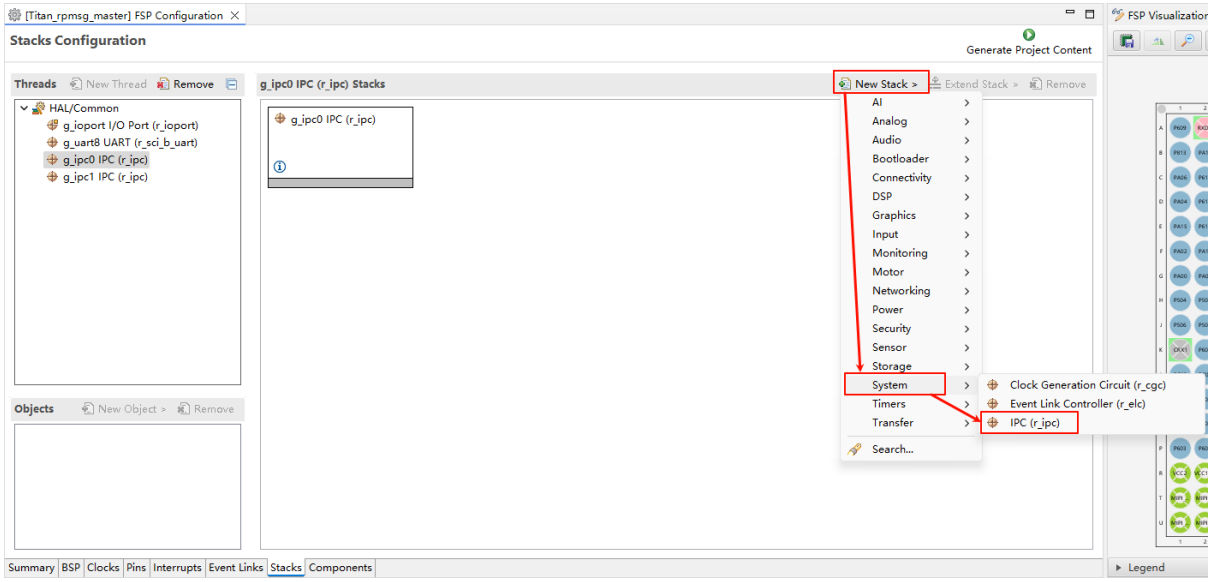
- **Resource-constrained embedded systems**: Low memory footprint and communication overhead make it suitable for industrial control, robotics, automotive electronics, and other fields.

4.3 Hardware Description

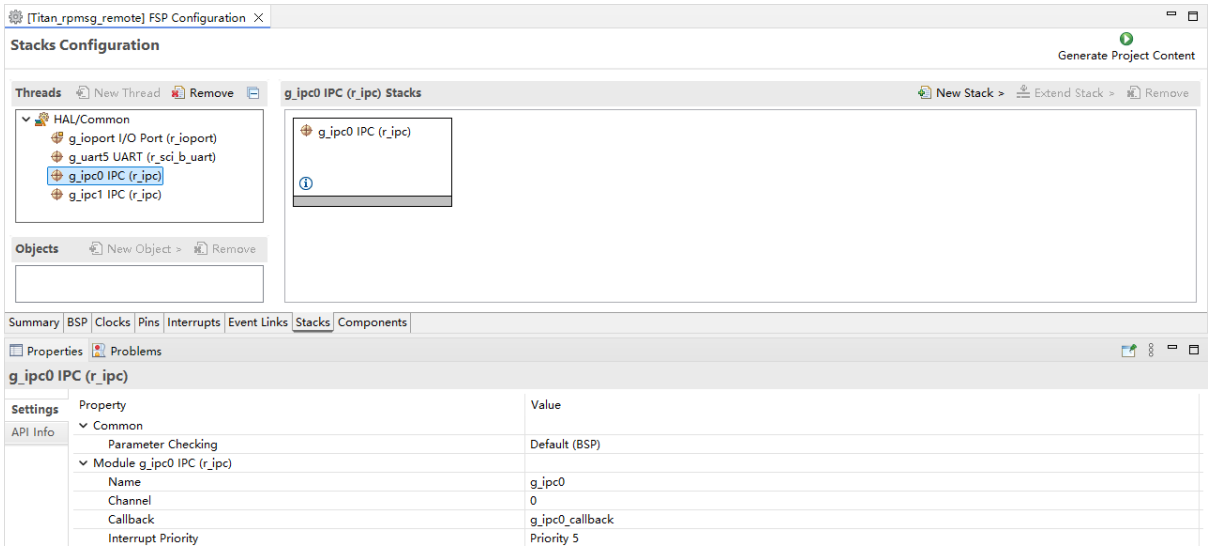
None

4.4 FSP Configuration

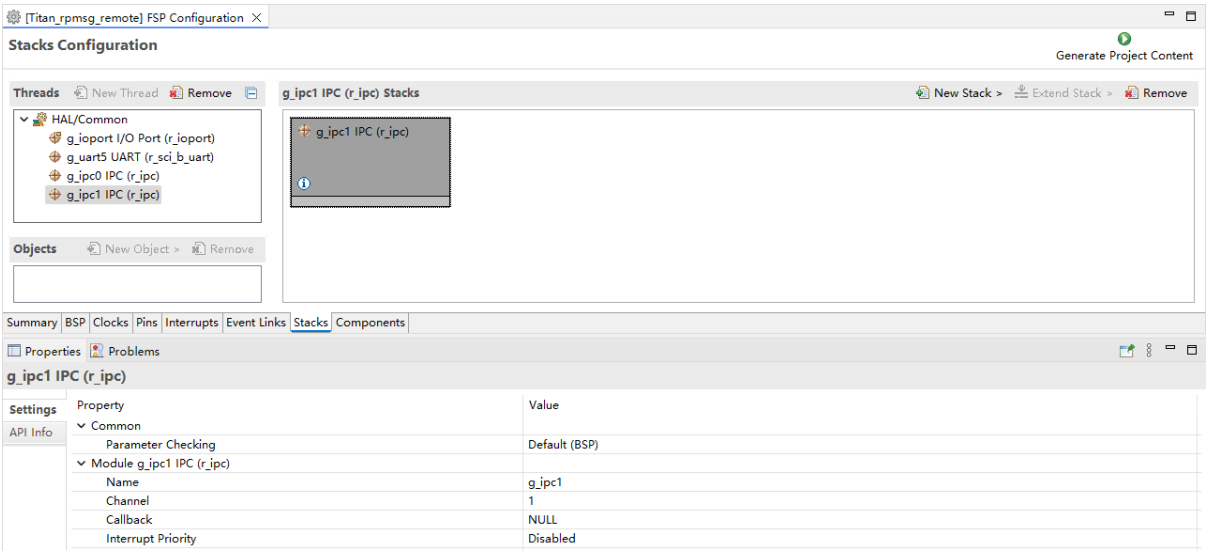
- Create two r_ipc stacks:



- Configure IPC0:

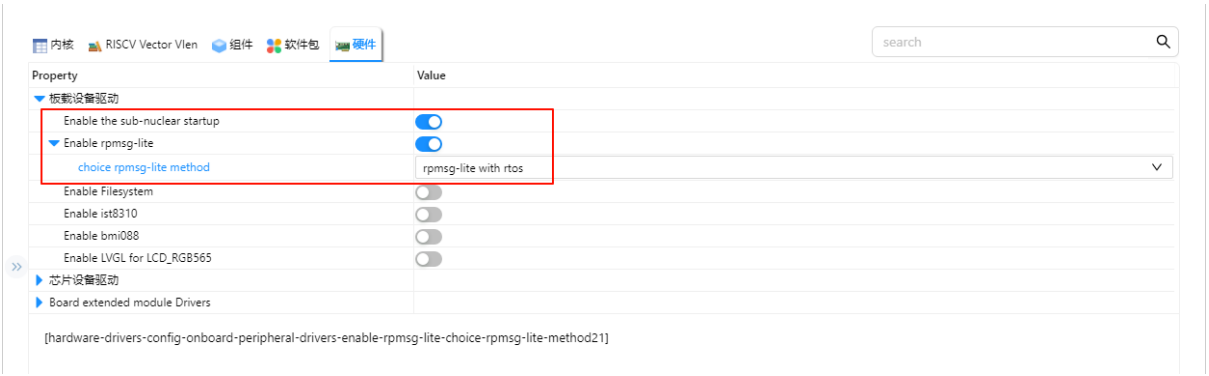


- Configure IPC1:



4.5 RT-Thread Settings Configuration

- Enable sub-nuclear startup and rpmsg-lite。



4.6 Example Code Description

The sample program is located at /board/ports/rpmsg-adapt.

4.7 Compilation & Download

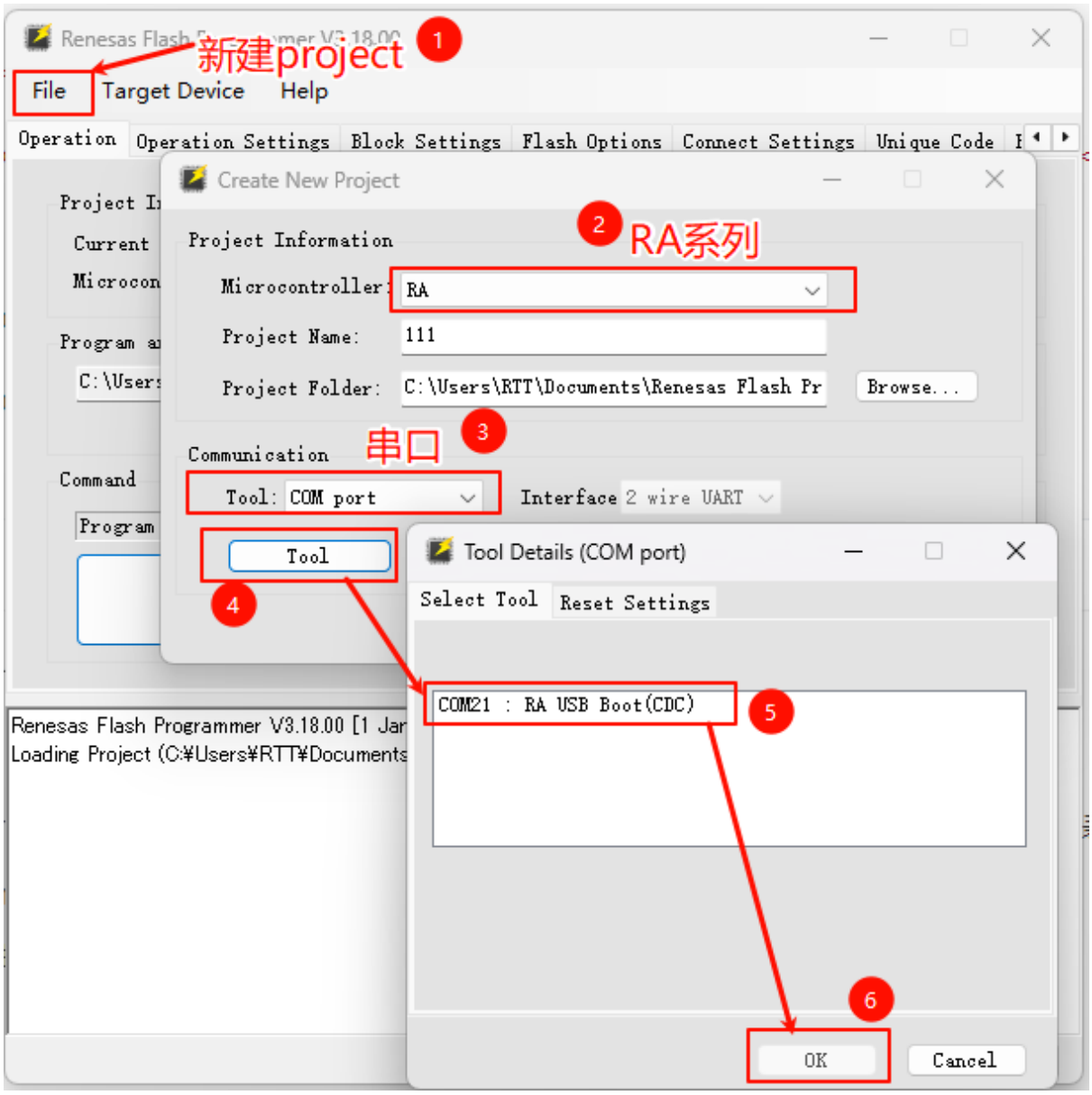
- **RT-Thread Studio**: In RT-Thread Studio ' s package manager, download the Titan Board resource package, create a new project, and compile it.

Download Methods

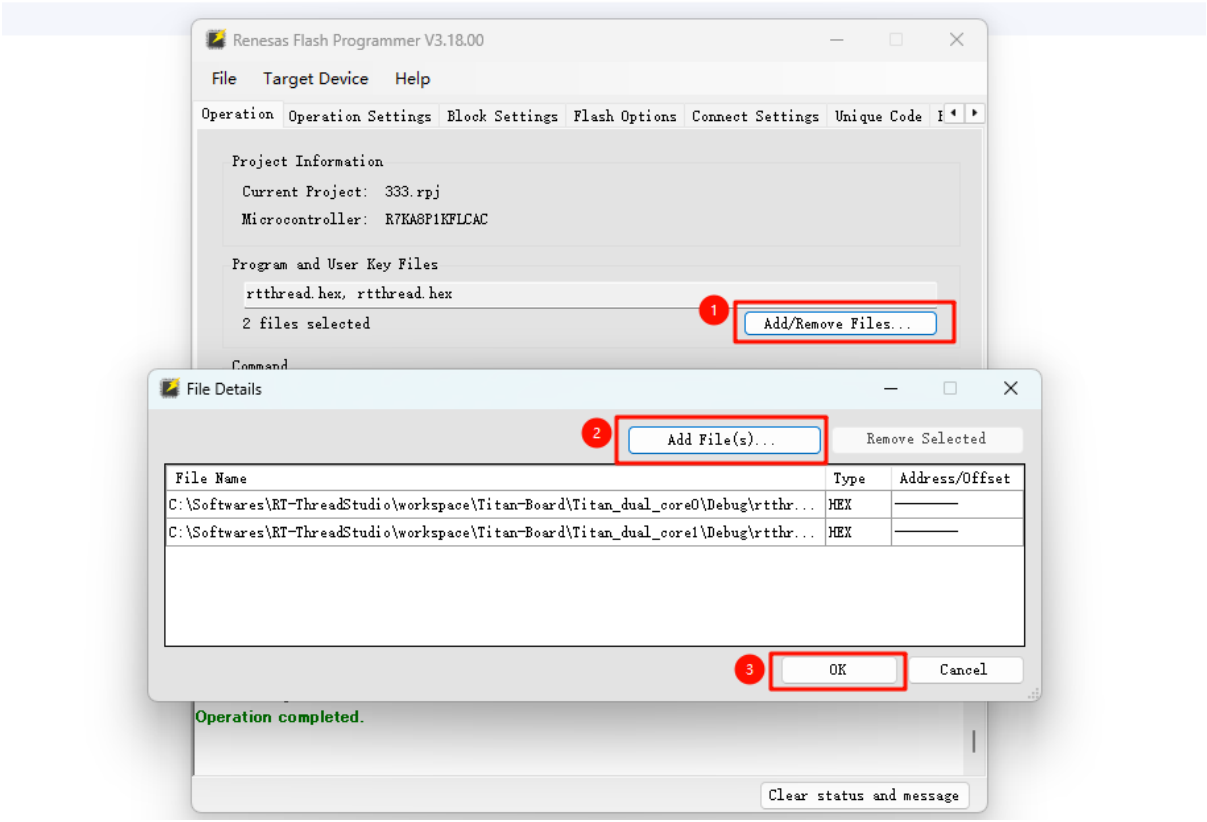
Currently, DAP-Link does not support programming the Core1 firmware, so you need to **use the Renesas Flash Programmer tool to program the firmware**. The detailed programming steps are as follows:

1. Download and install the Renesas Flash Programmer tool: Renesas Flash Programmer (Programming GUI) | Renesas.

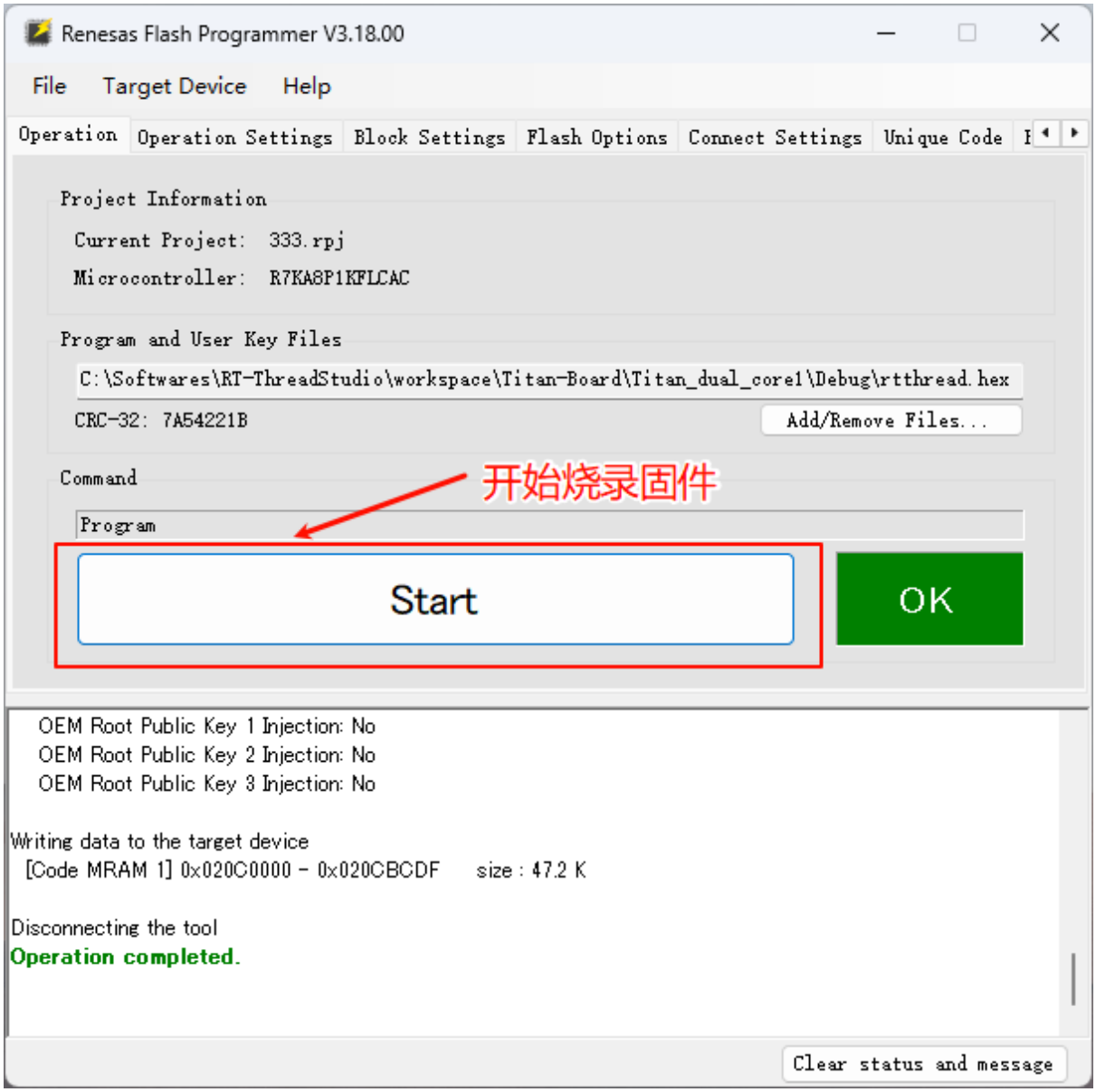
2. Connect the USB-DEV port of the development board to the PC.
3. Create a project: The development board needs to enter BOOT mode (**hold down the BOOT button and press the RESET button once**).



4. Select the firmware to be programmed:

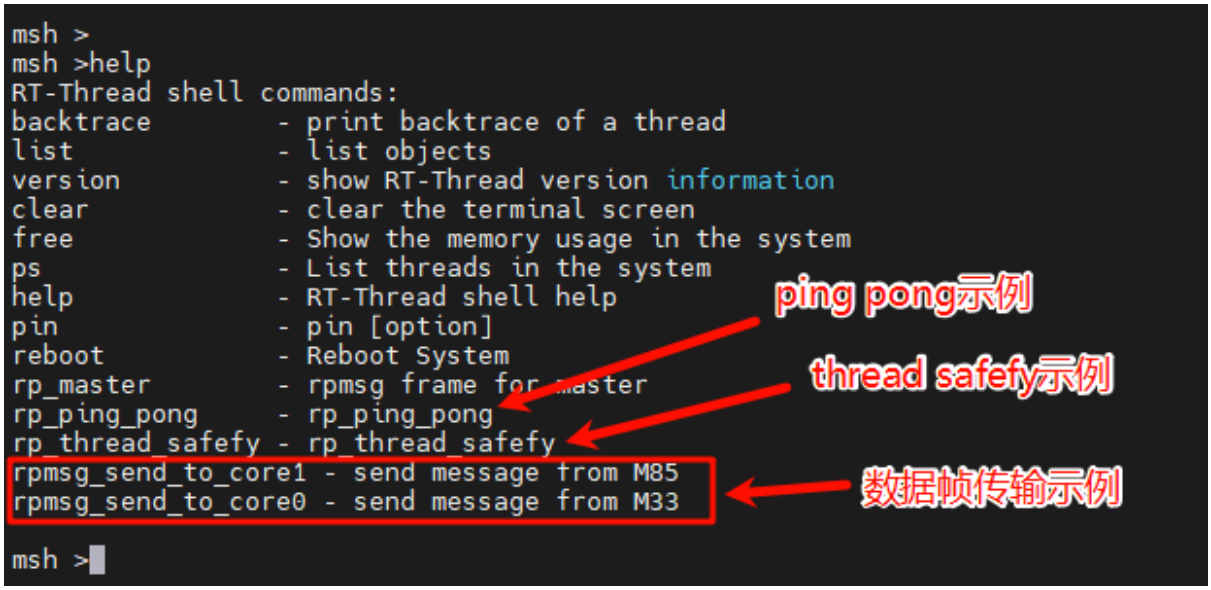


5. Flashing firmware (at this time, the development board must be in **BOOT** mode):



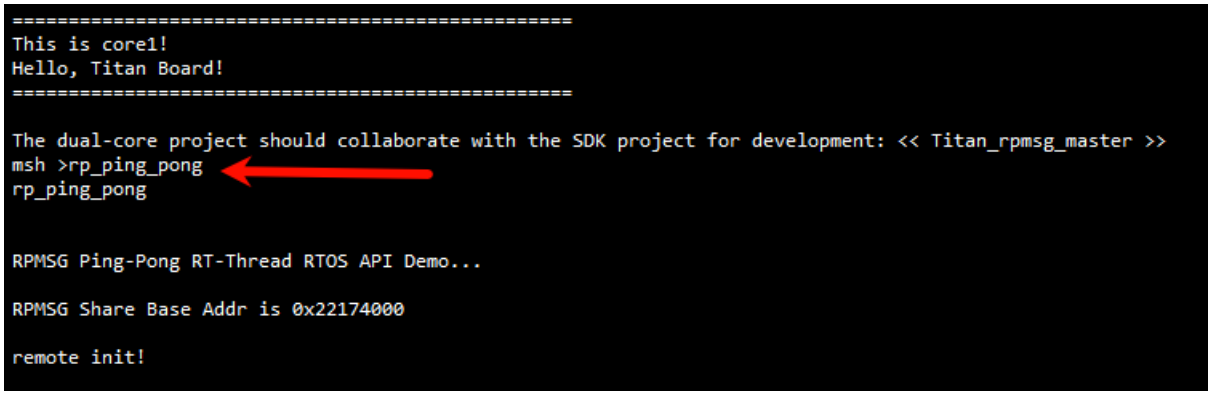
4.8 Run Effect

Note: In the RPMsg-Lite example project, before running the *ping pong*, *thread safety*, and *data frame transmission* examples shown below, you must first start the **remote** endpoint on **Core1**.



Ping Pong Example

On **Core1**, enter the command rp_ping_pong to initialize the remote endpoint. Then, on **Core0**, enter the same rp_ping_pong command to start the master.

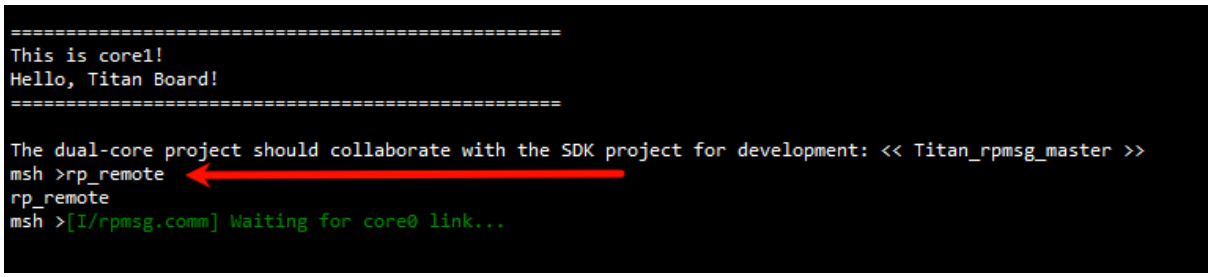


Thread Safety Example

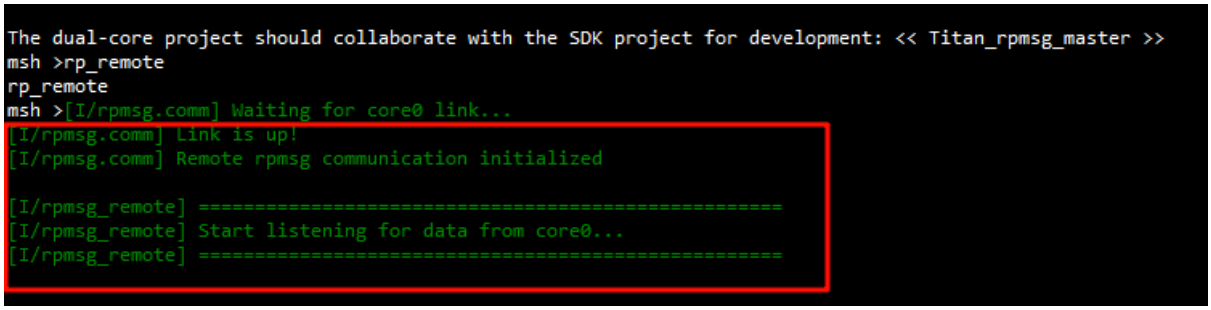
On **Core1**, enter the command rp_thread_safefy to initialize the remote endpoint. Then, on **Core0**, enter the same rp_thread_safefy command to start the master.

Data Frame Transmission Example

On **Core1**, enter the command rp_remote to start the remote endpoint. Then, on **Core0**, enter the command rp_master to start the RPMsg master.



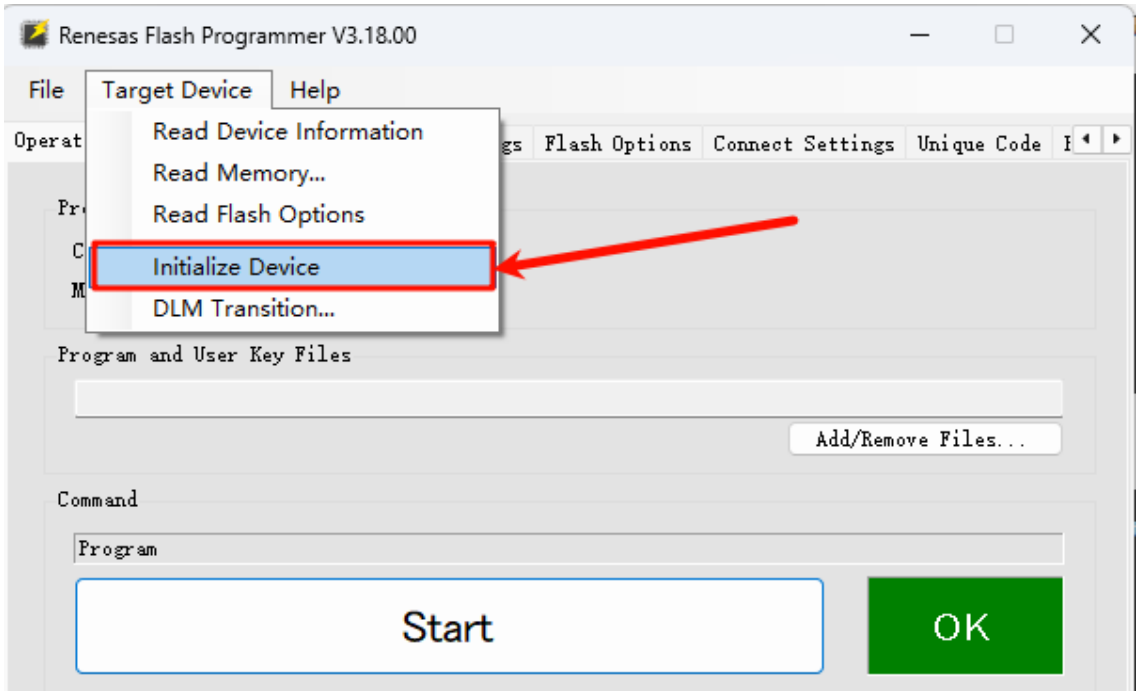
After the master starts on **Core0**, the terminal will display the following message, indicating that the connection has been successfully established.



You can then use the rpmsg_send_to_core0 command to transmit data to **Core0**.

4.9 Notes on Using Dual-Core Projects

1. **DAP-Link does not currently support dual-core projects.** Please use **Renesas Flash Programmer (RFP)** for downloading, or replace the DAP-Link firmware with another debugging firmware (a tutorial is available on the RT-Thread community forum).
2. **Before downloading a dual-core project**, it is recommended to use **Renesas Flash Programmer** to **erase the Flash memory**.



3. **When configuring FSP for dual-core projects**, Core0 and Core1 must not configure the **same peripheral or pin** simultaneously. For example: if **UART8** is configured in Core0’ s FSP, then **UART8** must not be configured again in Core1’ s FSP.

NPU Intelligent Vision

1. Titan Board OpenMV

English | 中文

OpenMV 5.0.1 and MicroPython for the RA8P1 Titan Board, running on RT-Thread. The project provides OV5640 MIPI camera capture, OpenMV IDE preview, USB CDC/MSC, SD storage and TFLM/Ethos-U55 integration. Dependencies are included in ThirdParty/; the project uses this SDK ' s rt-thread/ and libraries/.

1.1 Hardware

Resource	Configuration
CPU	Cortex-M85; Cortex-M33 is not started
External RAM	32 MiB HyperRAM at 0x70000000
Camera	OV5640 on J9, two MIPI lanes
Application USB	J3, USB HS CDC + MSC
Debug connection	J4, J-Link/SWD
Console	SCI8, P500 RX / P501 TX, 115200 baud, 8N1
Storage	FAT/FAT32 SD card on SDHI0
LEDs	Red P013, green P613, blue P012; active low
User key	P110; active low

Python supports board.LED, GPIO/IRQ, I2C, SoftI2C, SoftSPI and software timers. Hardware machine.UART, PWM, SPI, ADC and LCD/display are disabled. See *pin assignments and supported interfaces*.

1.2 Build and download

Use Arm GNU Toolchain **13.3.Rel1**, Python **3.10+**, SCons, and native GCC, GNU make and a POSIX shell for mpy-cross. Full setup and Studio import steps are in *build instructions*.

In RT-Thread Studio, build with make -j16 openmv-all, then run the **openmv-download** Make Target. From the Studio build console in Debug/:

```
make -j16 openmv-download
```

Alternatively, from the project directory:

```
python -m SCons -j8
python tools/download.py --jlink "C:/tools/JLink_v848/JLink.exe"
```

Replace the J-Link path with the local installation. The project selects J-Link **v8.48**. Use the supplied download target/script for programming.

Both build methods produce:

File	Destination
firmware/rtthread.hex	MCU, programmed through J-Link
firmware/openmv-vision.bin	SD card root directory

Always update both files from the same build. Camera and image processing require the matching vision file on the SD card, including live IDE preview. Without it, basic USB/REPL access remains available.

1.3 Run an example

1. With power off, connect the OV5640 to J9 and insert a FAT/FAT32 SD card.
2. Program the HEX and copy openmv-vision.bin to the SD card root. Connect J3 to the computer to access the USB storage drive, or use a card reader.
3. Safely eject the drive, reset the board and connect OpenMV IDE through J3.
4. Run *board_smoke_test.py*, then *helloworld.py*.

The startup indicator is blue while initializing, green when the runtime and vision image are ready, and red on an initialization/load error. Calling board.LED gives the script control of the LEDs until the next soft reset.

Save an application as main.py in the SD root for automatic startup; optional boot.py and lib/ also reside there. /rom is a read-only alias of the SD root. Finish and safely eject computer-side file access before board-side writes.

OpenMV IDE may identify the board as **AE3** for protocol compatibility. Update with this project's files and download tool; Alif AE3 firmware is incompatible.

1.4 Examples and dependencies

- *Python examples*
- *Model deployment and licenses*
- *Dependency versions and licenses*

The local ra/, ra_cfg/ and ra_gen/ contain the FSP 6.4 configuration. Maintain these source files together; a RASC configuration.xml is not included.

2. NPU Accelerated Face Detection Instructions

English | Chinese

2.1 Instructions

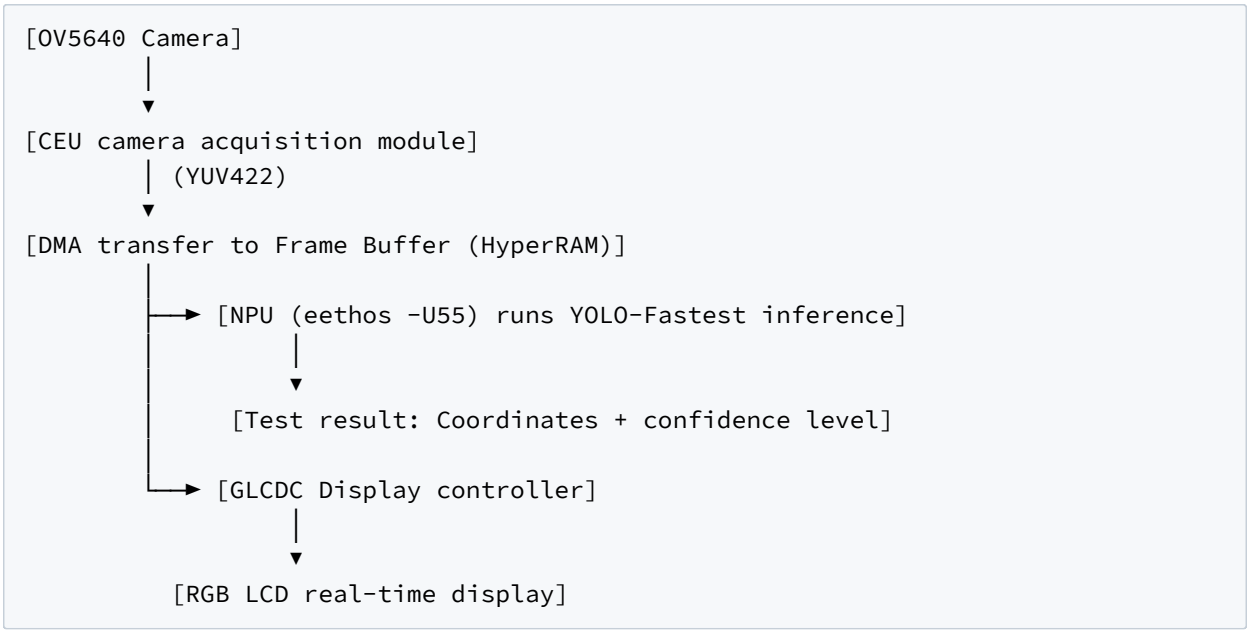
This example shows how to accelerate the operation of the YOLO-Fastest face detection model on the **Titan Board** using the **Arm® Ethos™ -u55 NPU**, And combine the CEU (Camera Engine Unit) camera interface and the RGB LCD display screen to achieve real-time face detection and display.

The main functions include:

- Collect real-time video streams via CEU (OV5640 camera)
- Perform YOLO-Fastest model inference on video frames using NPU Display the detection results (with face box) on the LCD screen
- Supports hardware-accelerated YUV to RGB conversion and graphic rendering

2.2 Overall system architecture

The system data flow of this example is shown in the following figure:



2.3 Arm® Ethos™-U55 NPU Features

The RA8P1 MCU used by the Titan Board integrates the **Arm® Ethos™-U55 neural processing unit (NPU) **, which can work in coordination with the Cortex-M85 CPU to significantly enhance the inference performance of neural networks.

Hardware Features

1. **Computing Power and Acceleration**
   - Supports INT8 quantized models
   - Delivers performance up to several hundred GOPS (depending on configuration)
   - Supports common operators such as convolution, pooling, ReLU, and Softmax
2. **Collaboration with CPU**
   - Works with the Cortex-M85 through the CMSIS-NN and Ethos-U drivers
   - Supports asynchronous execution between NPU and CPU
   - Model pre-processing and post-processing are handled by the CPU
3. **Memory and Bandwidth**
   - Supports direct feature map access from on-chip SRAM or external HyperRAM
   - DMA accelerates model input/output data transfer
   - Multi-level caching mechanisms reduce latency
4. **Compatibility**
   - Fully compatible with TensorFlow Lite for Microcontrollers (TFLM)
   - Supports model formats converted by the Arm NN SDK (.tflite)

2.4 YOLO-Fastest Model Introduction

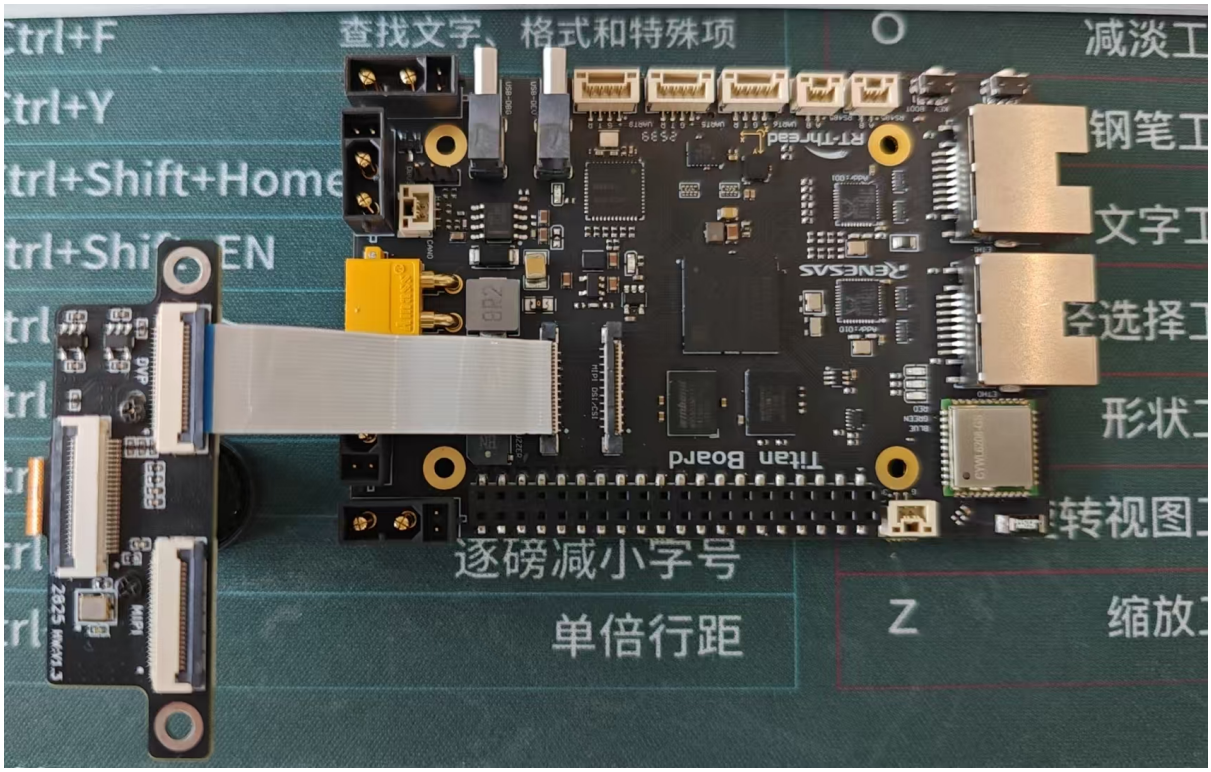
YOLO-Fastest is a lightweight object detection network designed for real-time operation on embedded devices.

Item	Specification
Model Type	YOLO-Fastest (Face Detection)
Model Framework	TensorFlow Lite (INT8)
Input Size	192 x 192
Output	Face bounding box coordinates + confidence
Inference Time (Ethos-U55)	Approx. 25 ms/frame
Application Scenarios	Face detection / Real-time visual recognition

2.5 Hardware Description

- The connection method of the CEU camera is as follows:

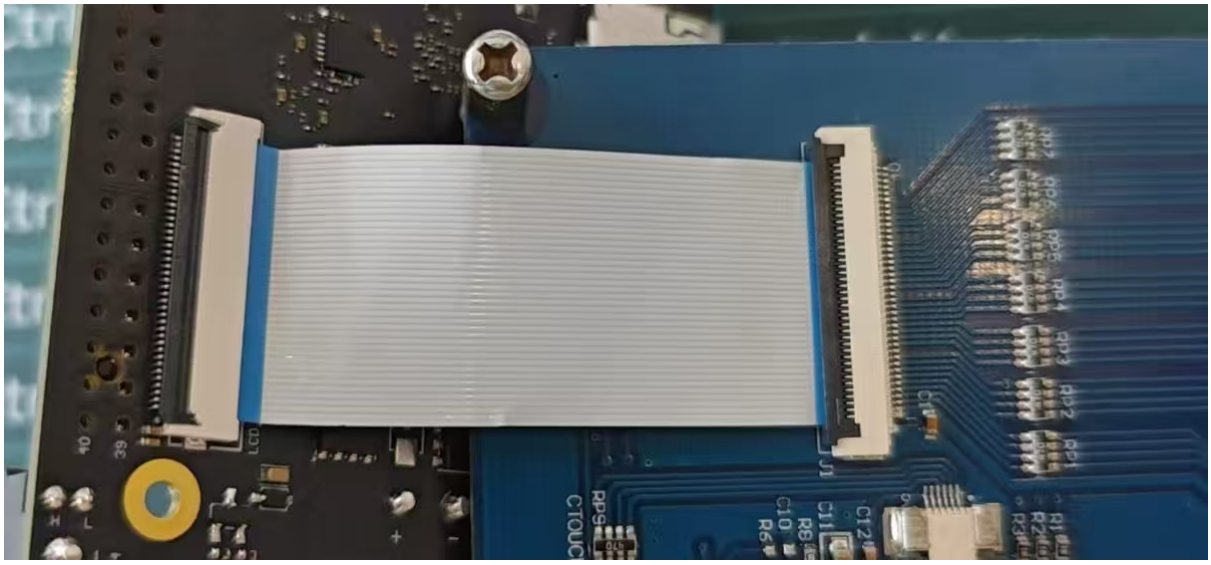
Use a **22-pin reverse FFC cable** to connect the development board’ s **CEU_CAM** connector to the camera adapter board’ s **DVP** connector.



- The connection method of RGB LCD is as follows:

Display model: **ALIENTEK 4.3″ RGB LCD 800×480**

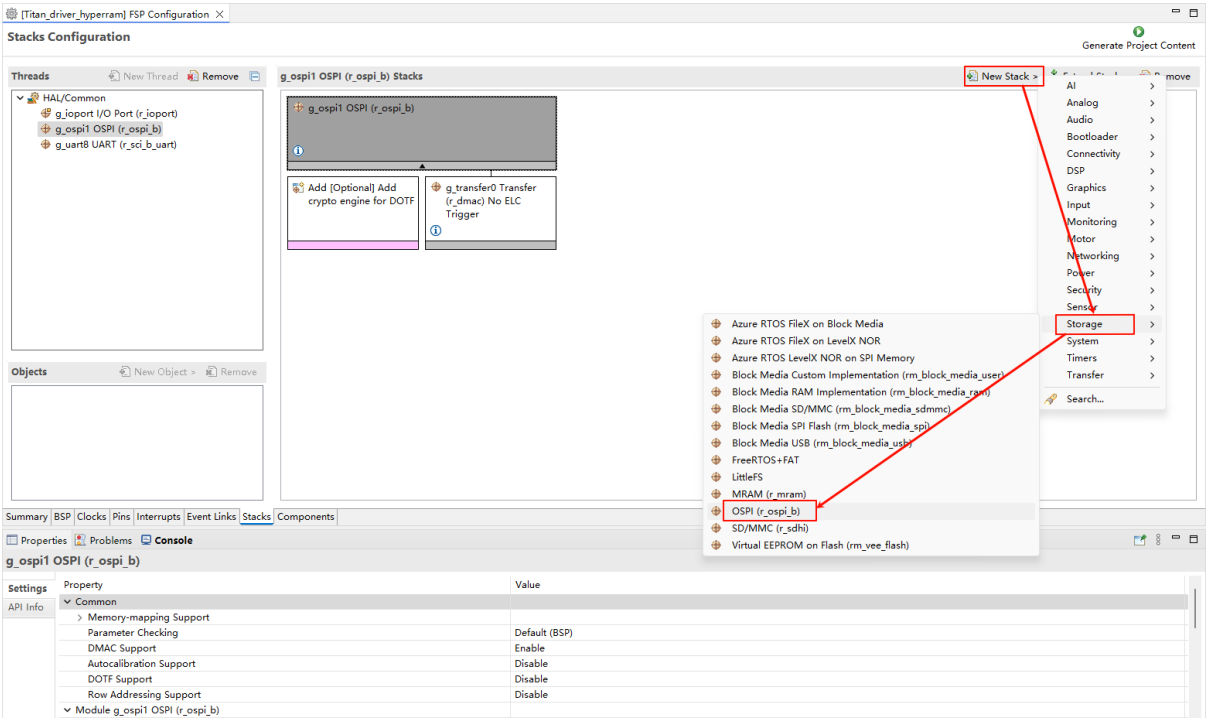
Use a **40-pin same-direction FFC cable** to connect the development board ' s **LCD_RGB** connector to the display.



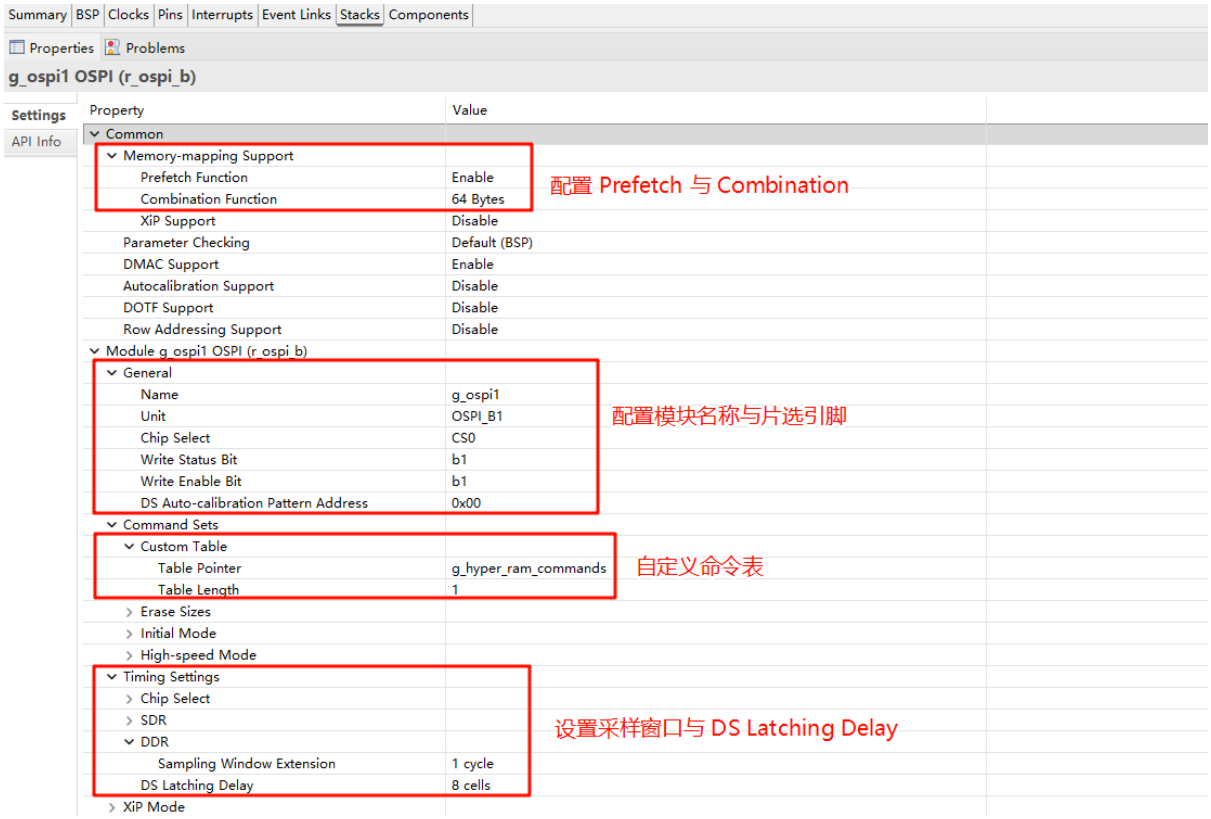
2.6 FSP Configuration

HyperRAM Configure

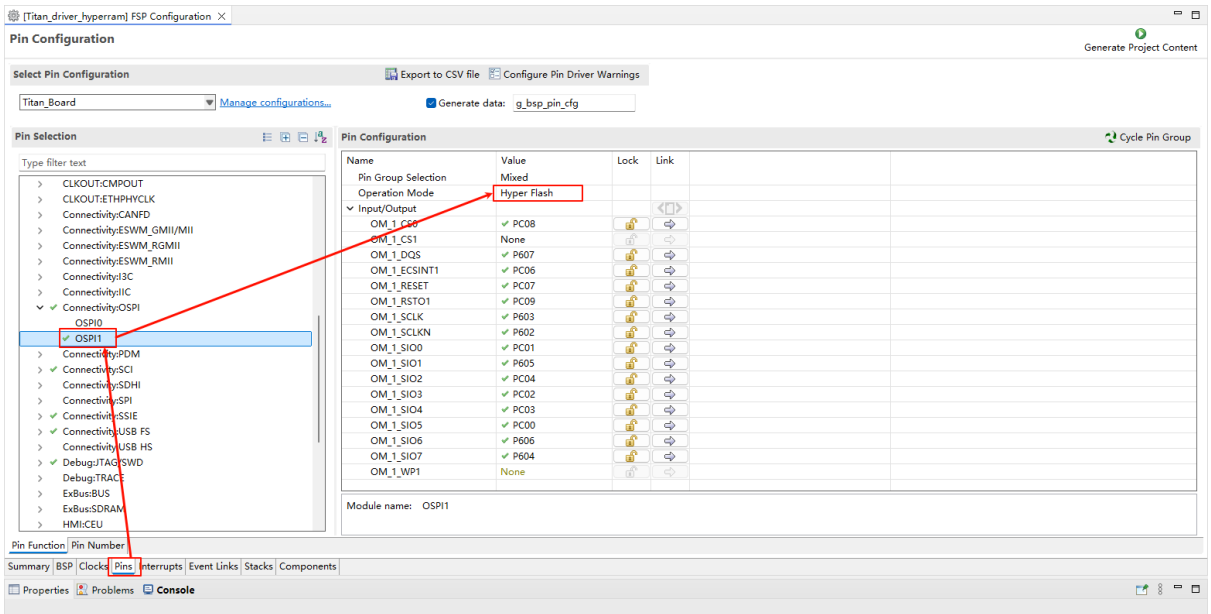
- Create a r_ospi_b stack:



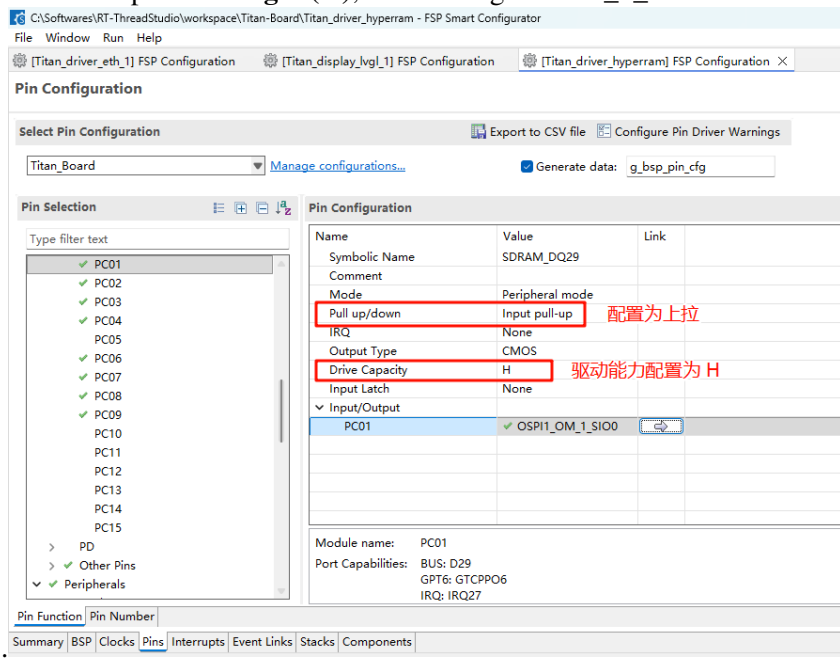
- Configure r_ospi_b stack:



- HyperRAM pins configure:

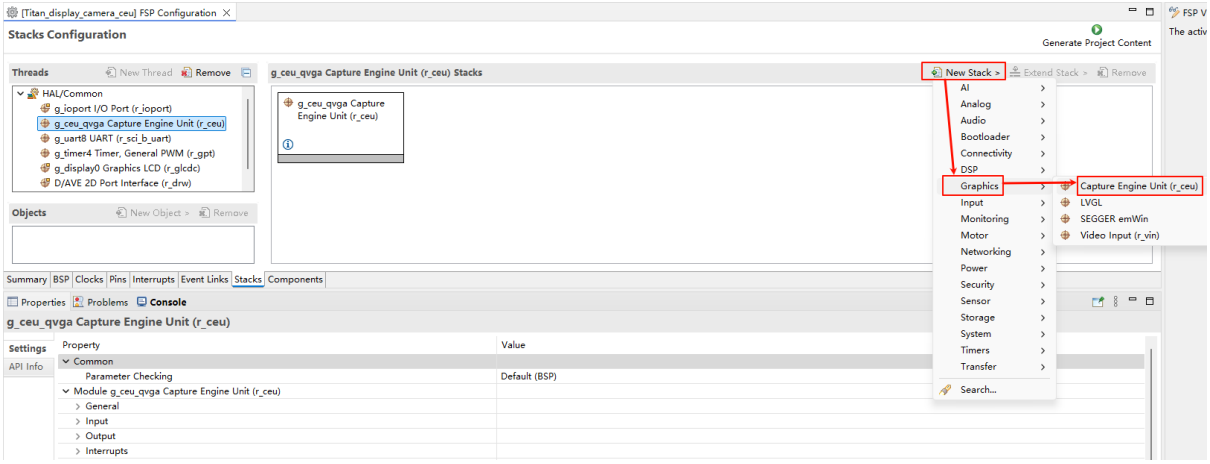


- Set the drive strength of all HyperRAM-related pins to **High (H)**, and configure OM_1_SIO0~OM_1_SIO7 as **input with pull-up**.

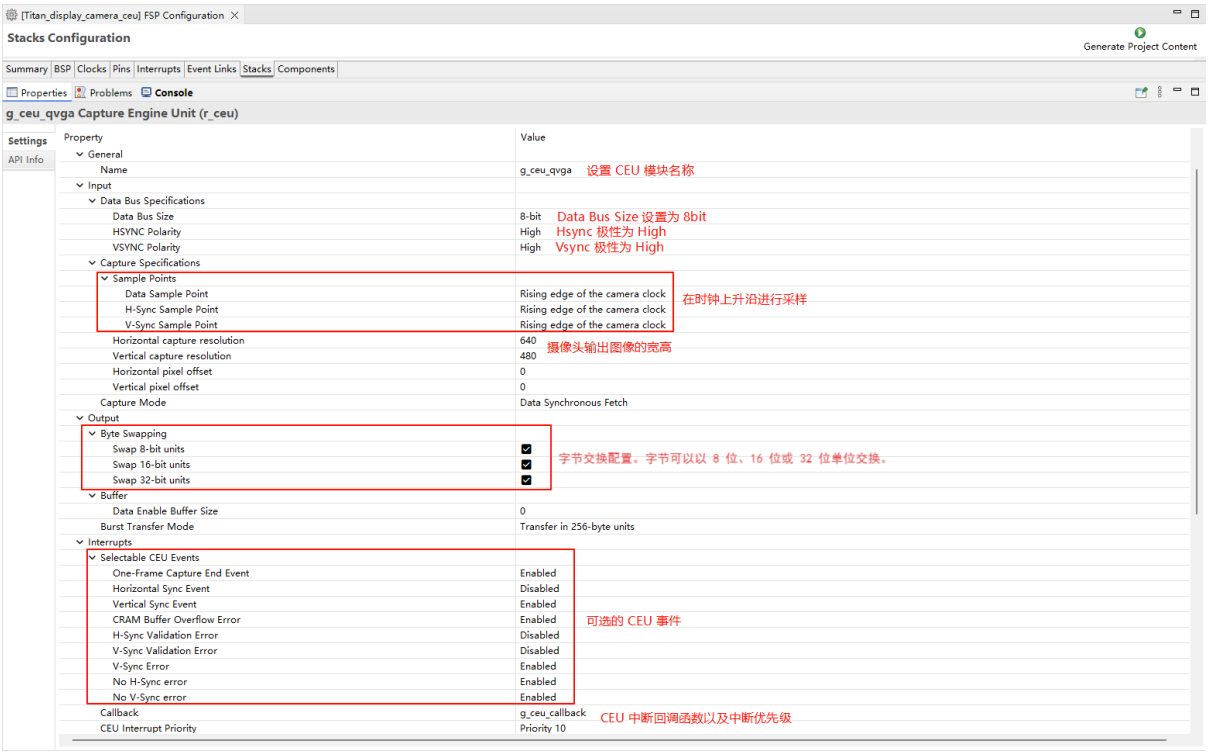


CEU Configuration

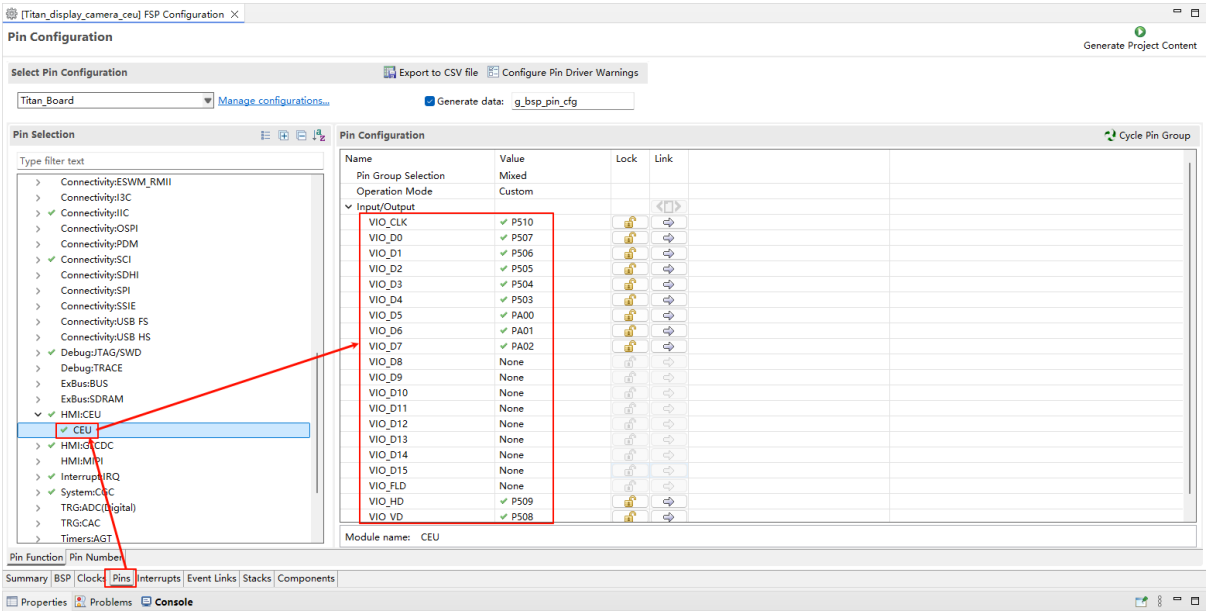
- Create a r_ceu stack:



- Configure CEU:

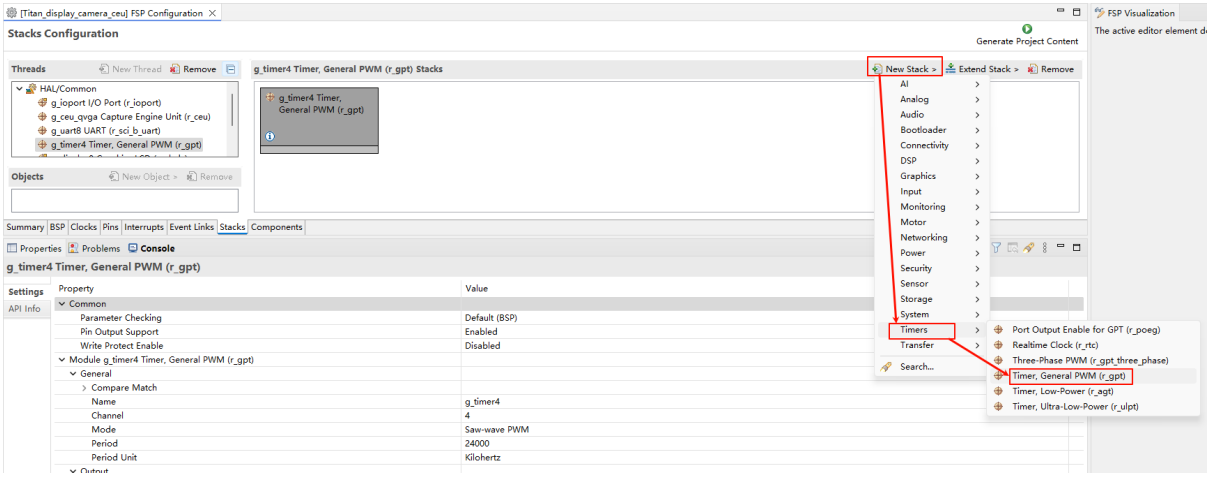


- Configure CEU pins:

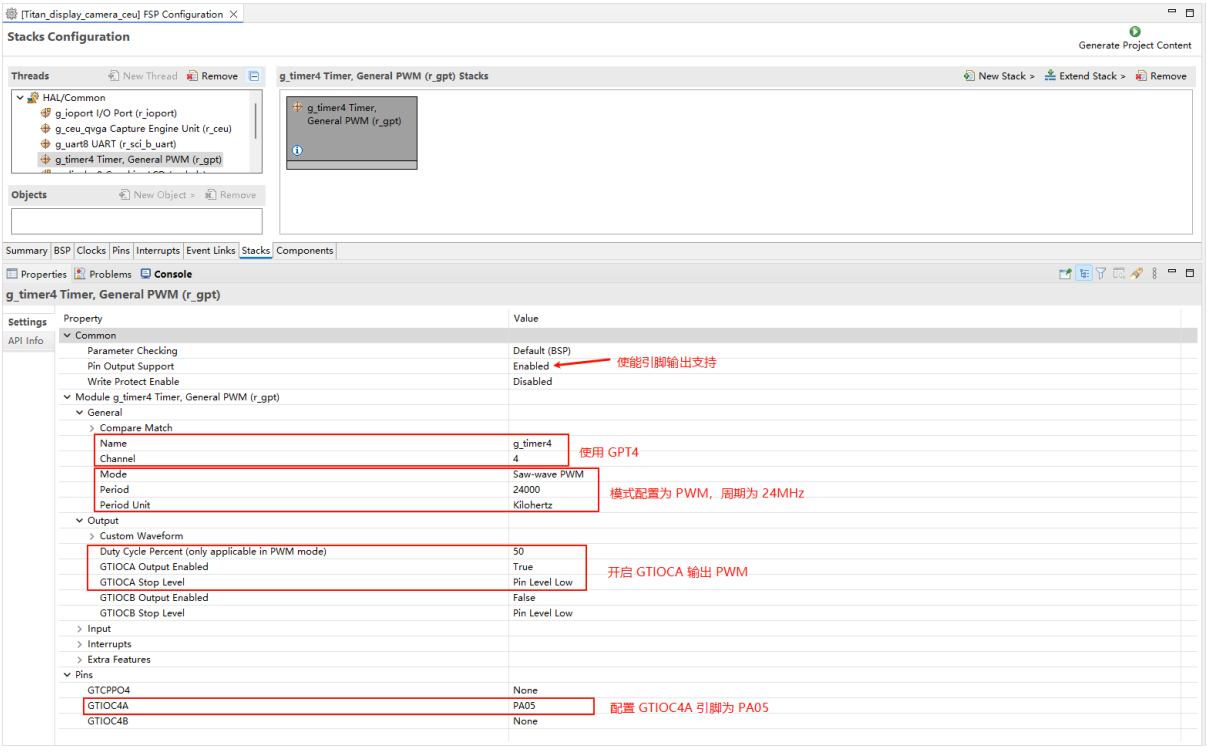


CEU Clock Configuration

- Create a r_gpt stack:



- Configure the PWM output of the camera clock

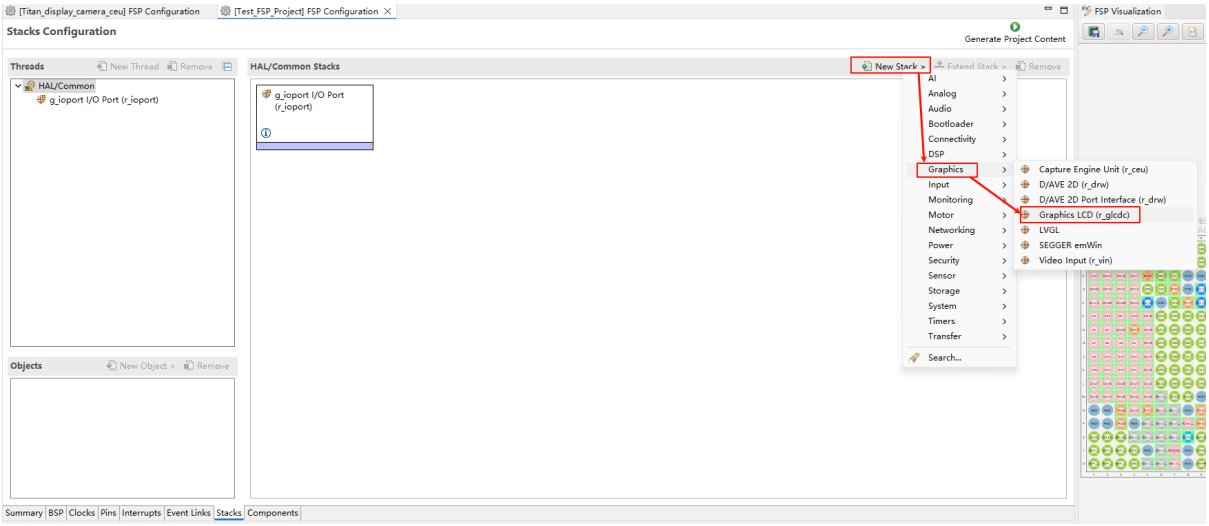


D/AVE 2D Configuration

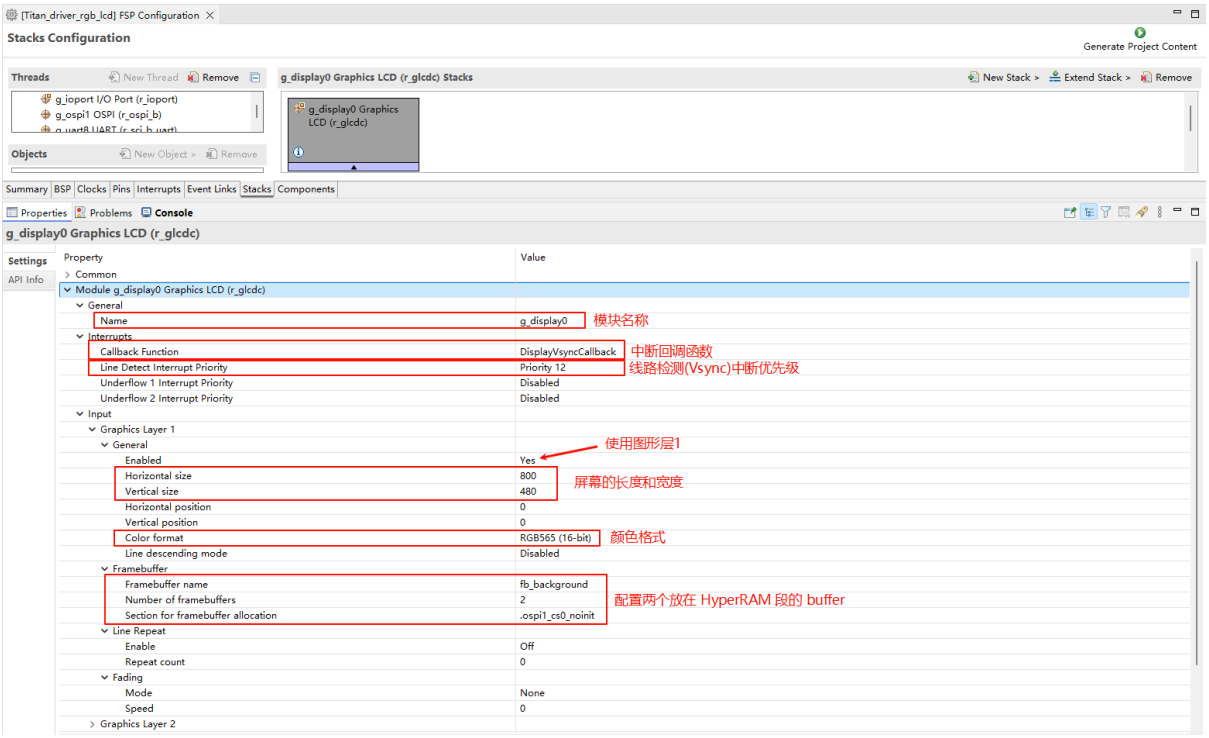
- Create a r_drw stack:

RGB LCD Configuration

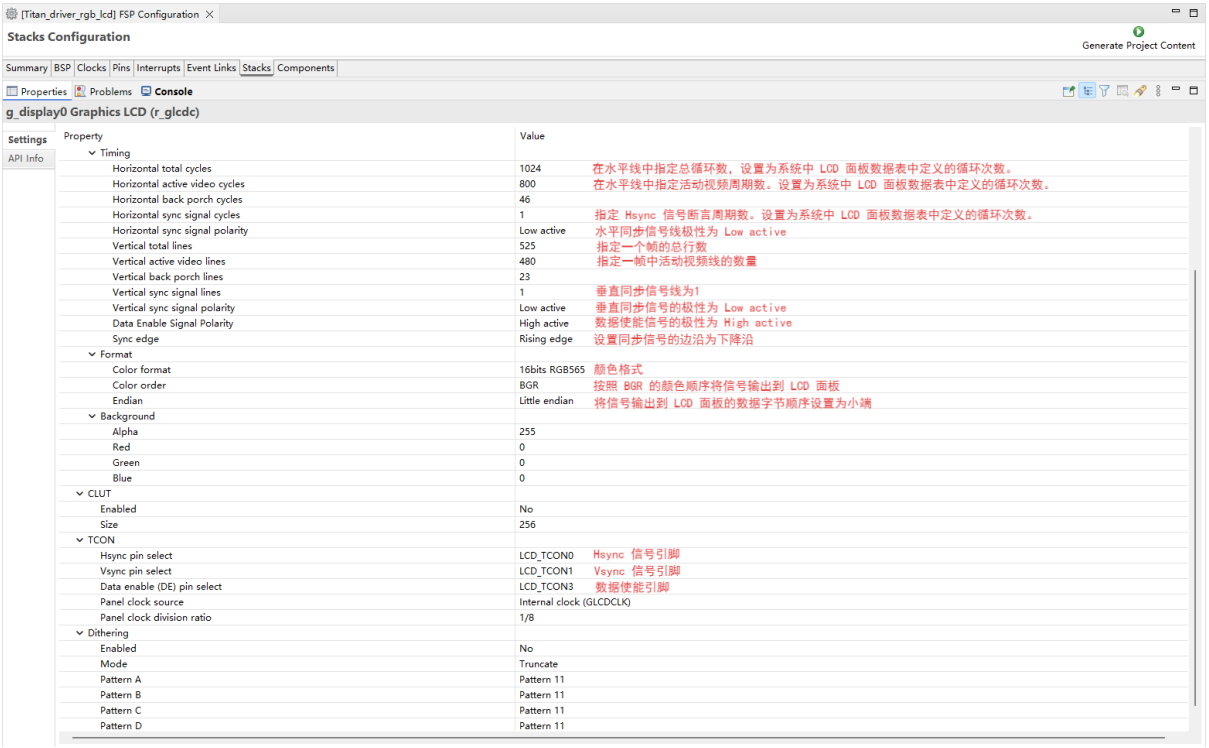
- Create a r_glcdc stack:



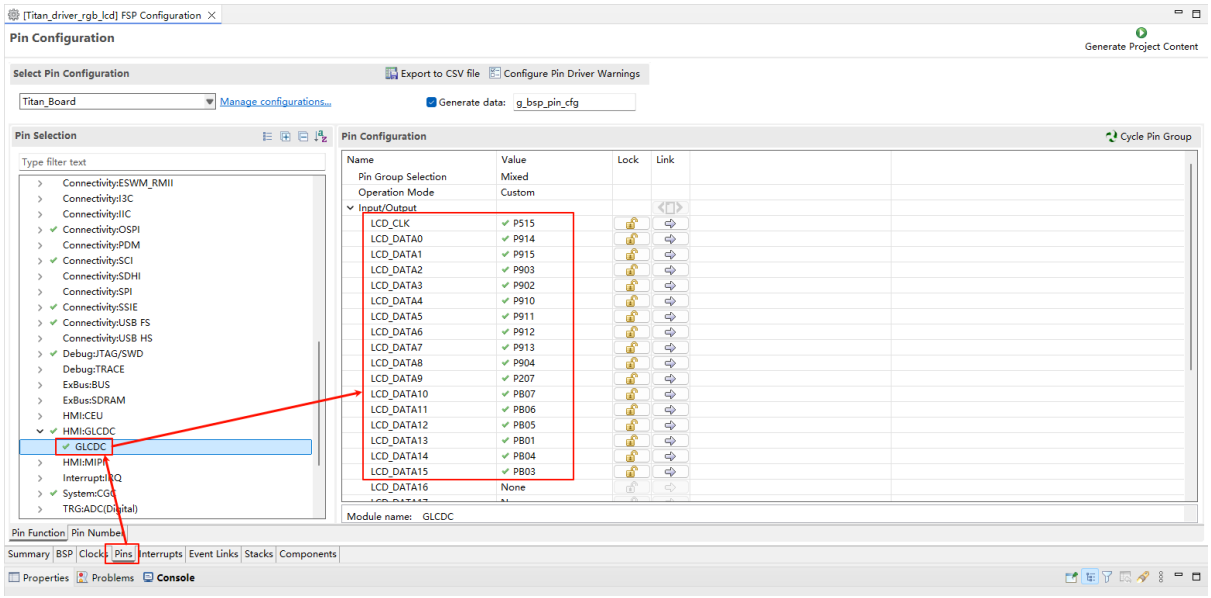
- Configure interrupt callback and graphics Layer 1:

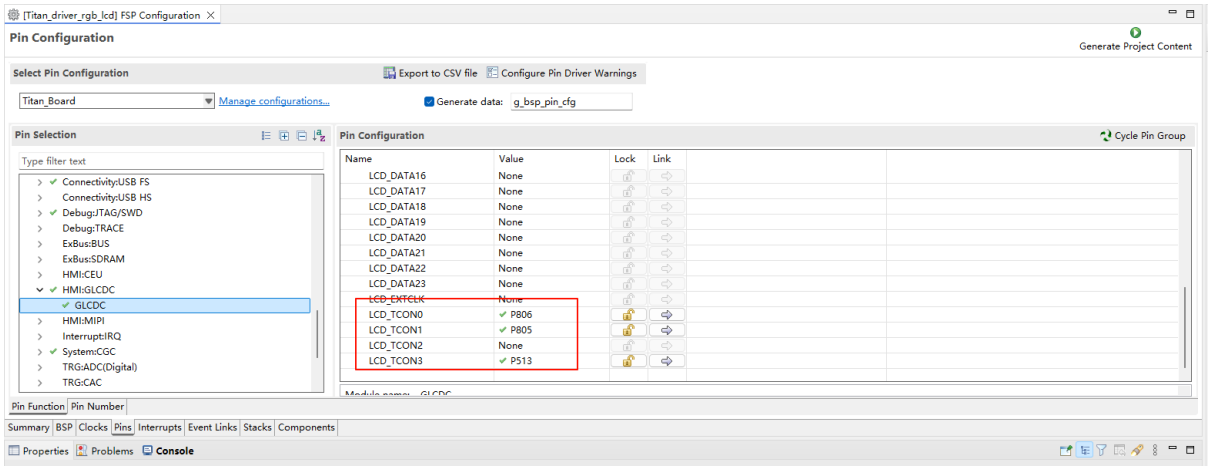


- Configure output parameters, CLUT, TCON.



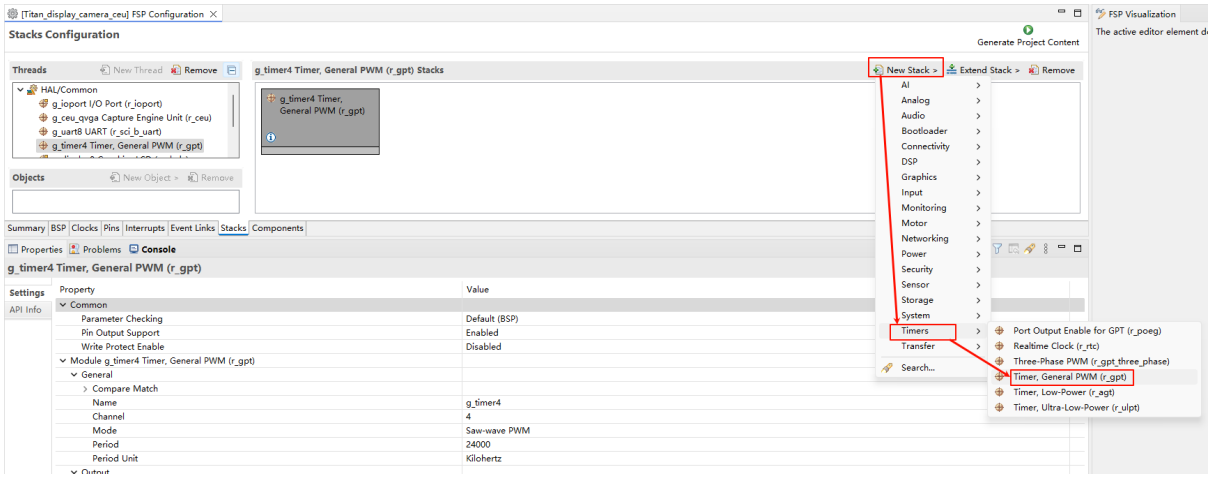
- Configure GLCDC pins:



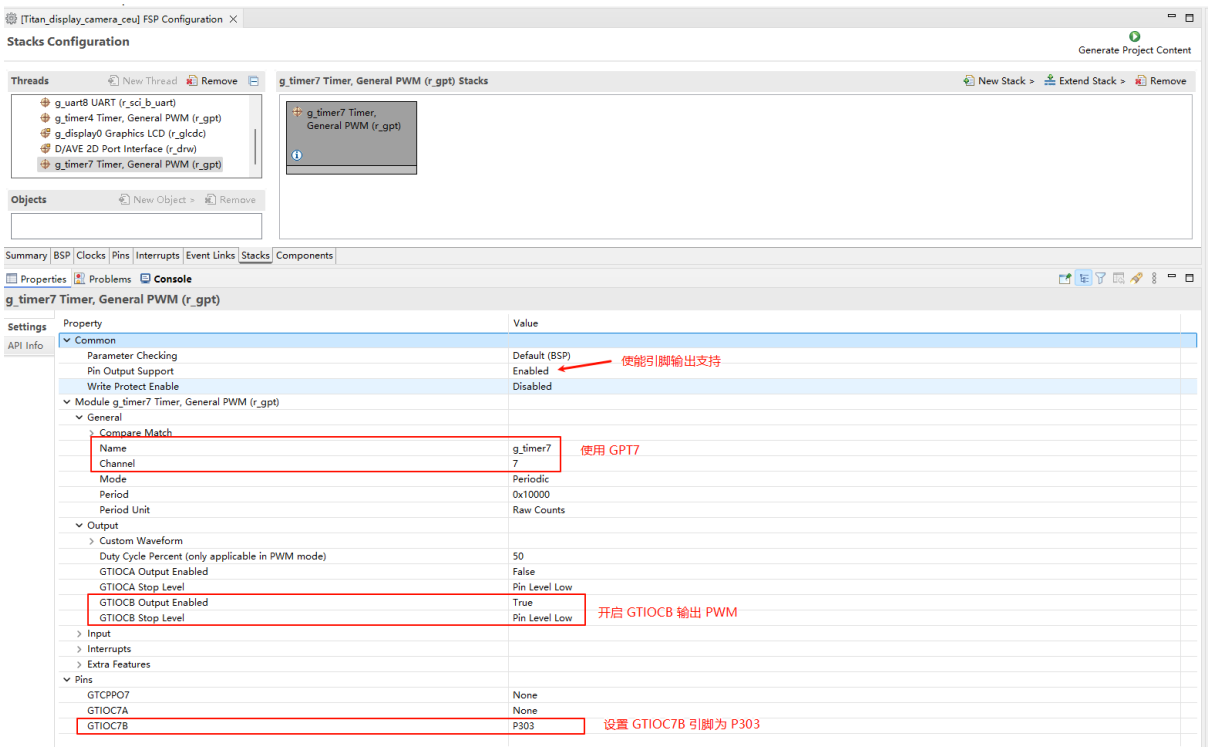


LCD Backlight Configuration

- Create a r_gpt stack:

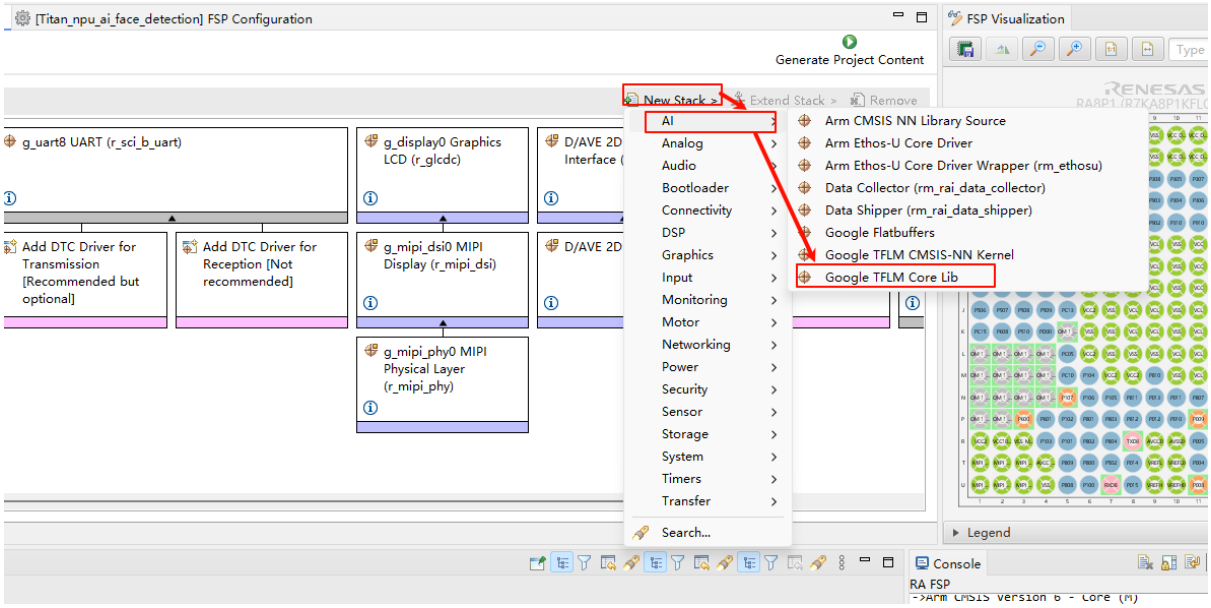


- Configure backlight PWM output:



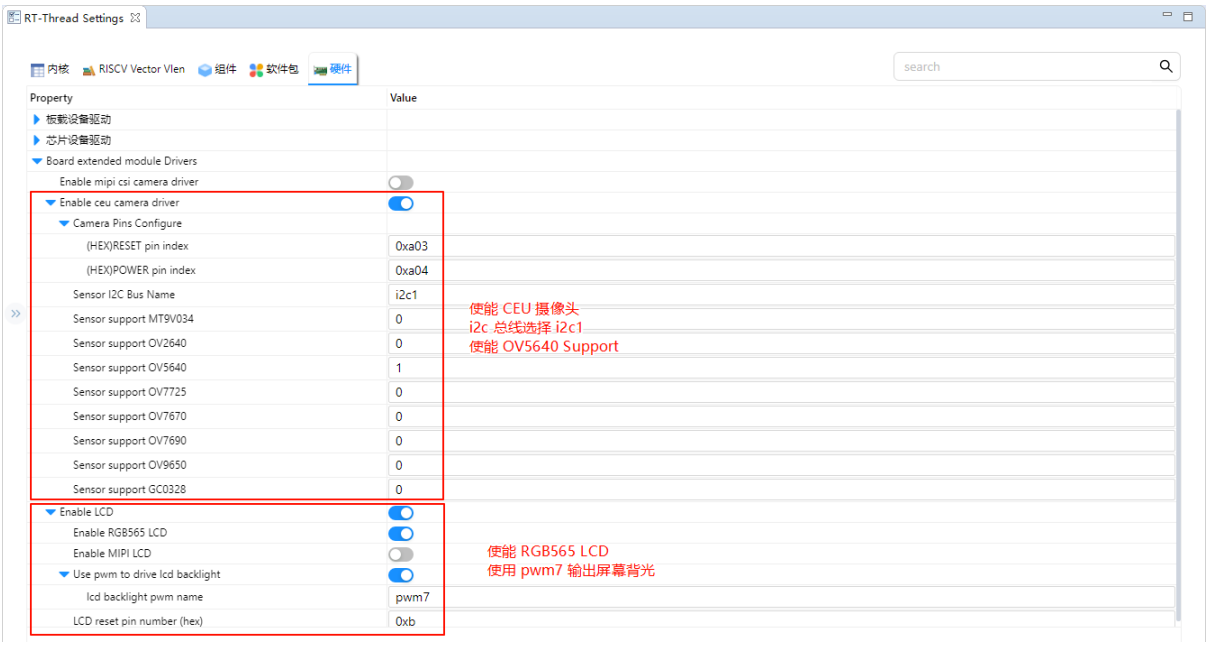
NPU Configuration

Create Google TFLM Core Lib stack:



2.7 RT-Thread Settings Configuration

- Enable the CUE camera, using the i2c1 and ov5640 cameras; Enable RGB565 LCD and use pwm7 output backlight.



2.8 Build & Download

- **RT-Thread Studio**: In RT-Thread Studio’s package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board’s USB-DBG interface to the PC and download the firmware to the development board.

2.9 Run Effect

After resetting the Titan Board, prepare a face picture and place it in front of the camera. At this time, observe the LCD screen and you can see that the face is framed by a green rectangle.

Factory Tests

1. Production Test Program User Guide

English | Chinese

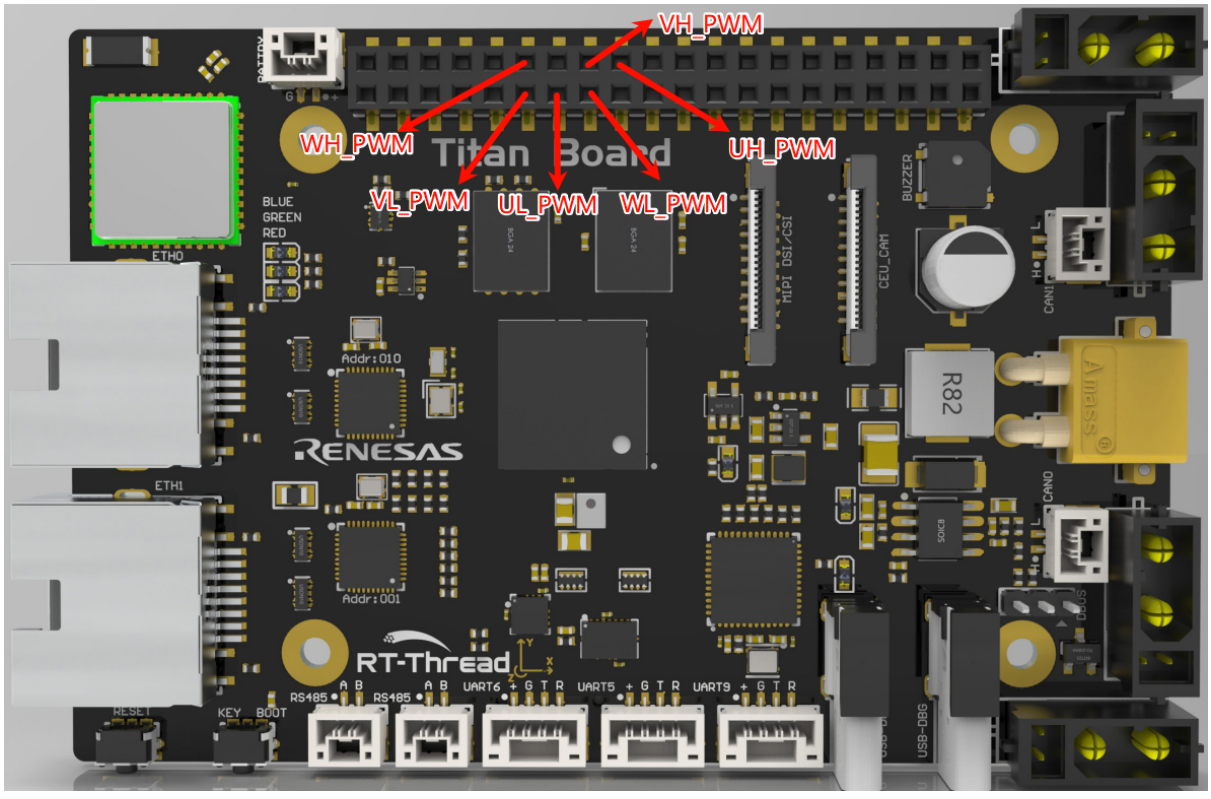
1.1 Test Preparation

1. Development board test firmware: **rtthread.hex**
2. Wi-Fi firmware: **43438A1.bin**, **43438A1.clm_blob**
3. SD card: Place Wi-Fi firmware in the root directory of the SD card
4. MIPI CSI interface OV5640 camera
5. 800 × 480 RGB LCD screen
6. 3–6 LED lights
7. WiFi AP (ssid: TitanBoard, password: 12345678)

1.2 Hardware Connections

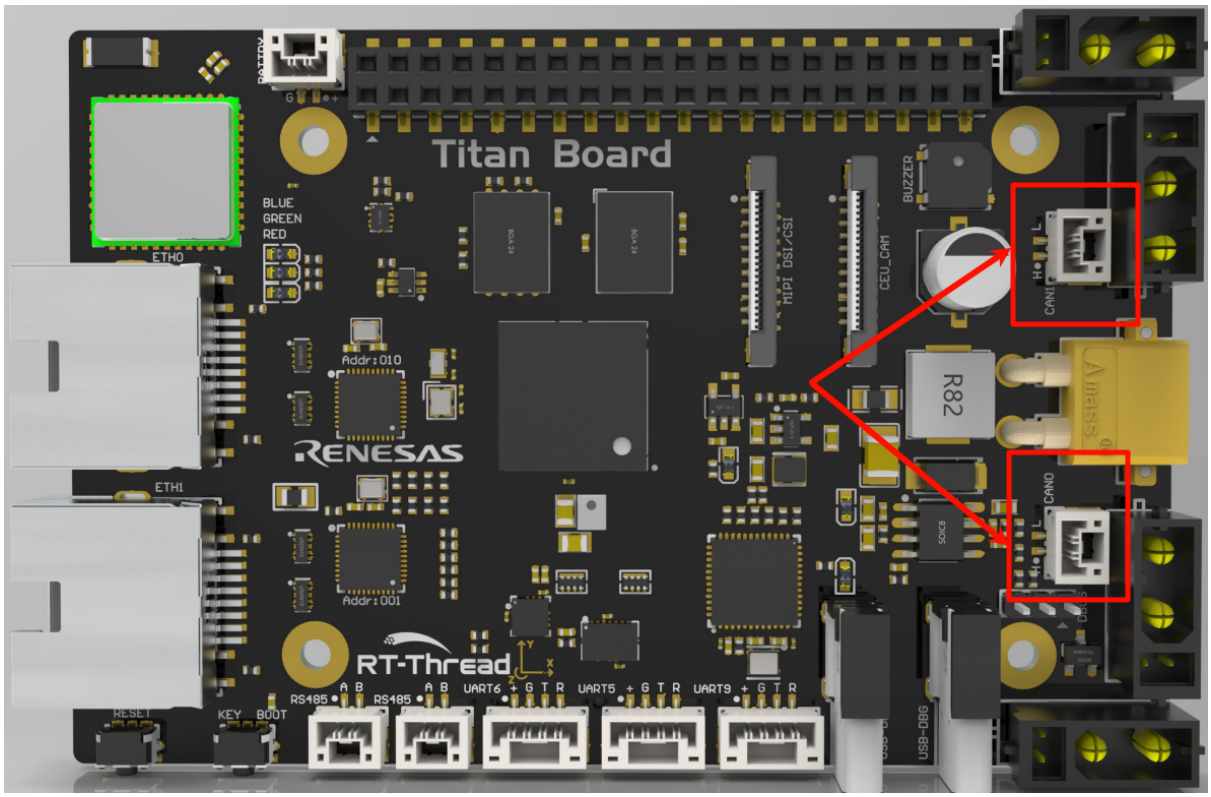
PWM

Connect LEDs to the six PWM pins on the Raspberry Pi interface: **VL_PWM (P714), UL_PWM (P712), WL_PWM (P108), VH_PWM (P715), UH_PWM (P713), and WH_PWM (P109).**



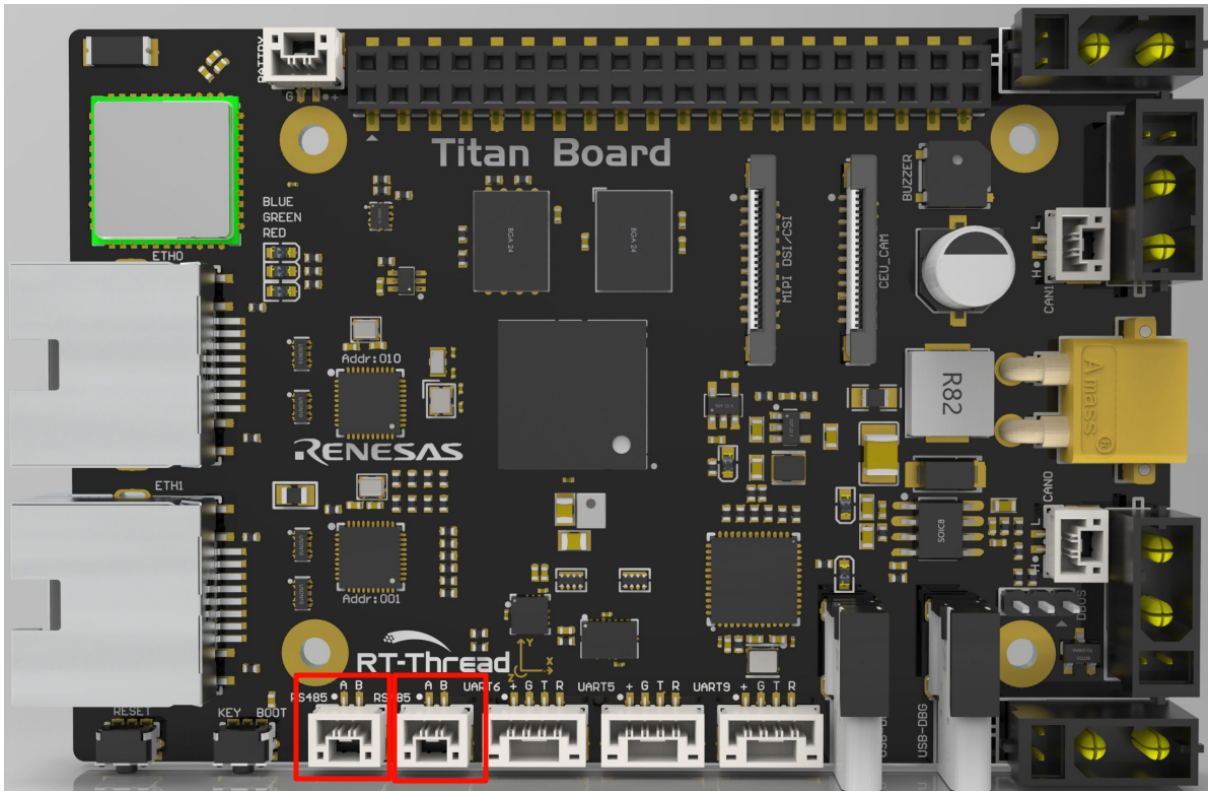
CANFD

Connect CAN0 and CAN1 (CAN0_H to CAN1_H; CAN0_L to CAN1_L).



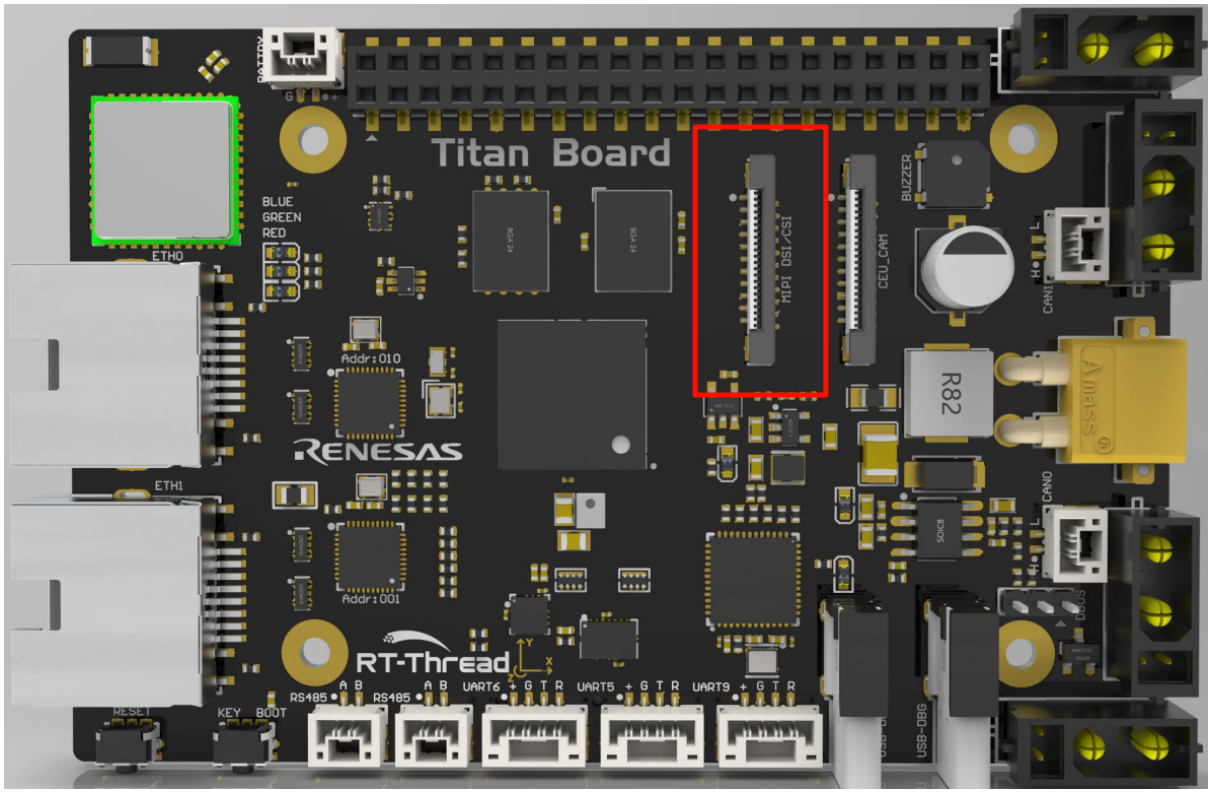
RS485

Connect either of the two RS485 ports on the development board to the RS485 port of another development board (A to A; B to B).



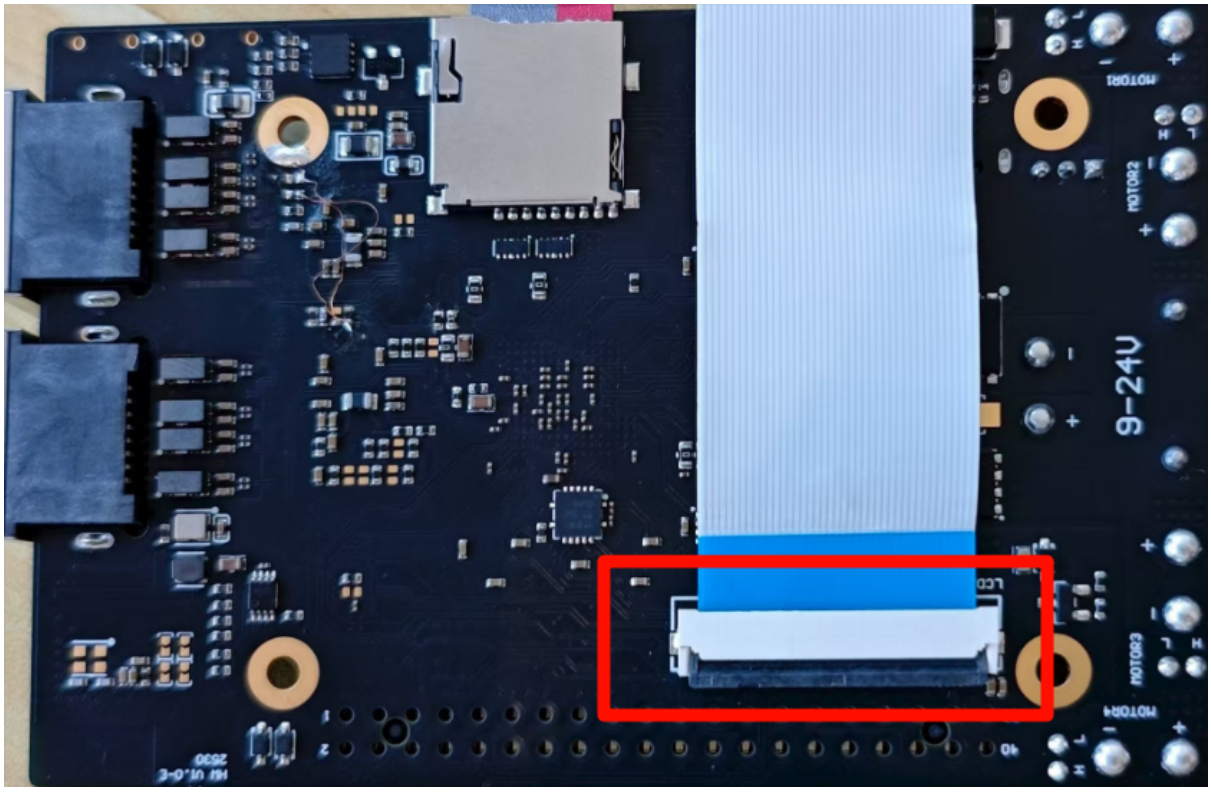
MIPI CSI Camera

Connect an OV5640 camera module to the MIPI CSI interface.



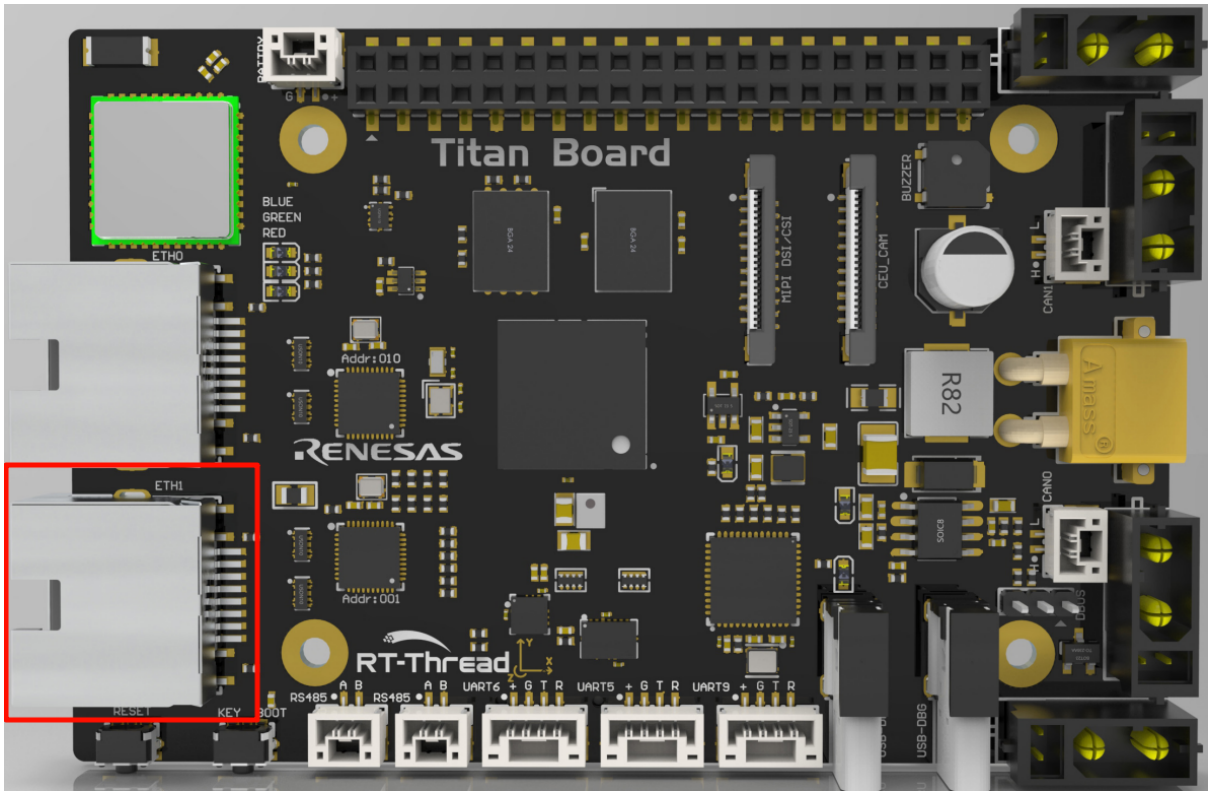
RGB LCD Screen

Connect an 800 × 480 RGB screen to the RGB LCD interface on the back of the development board.



Ethernet

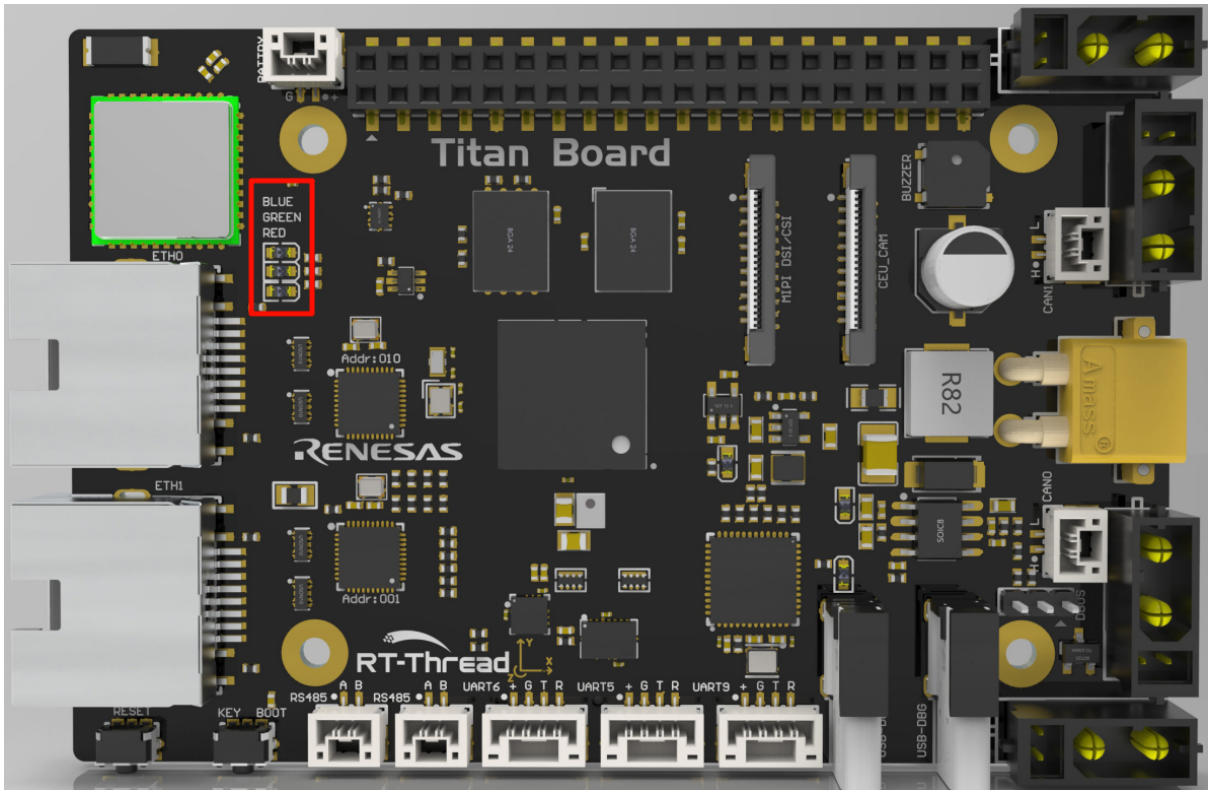
Insert a network cable into the ETH1 port.



1.3 LED Indicators

The development board has three LED indicators: blue, green, and red.

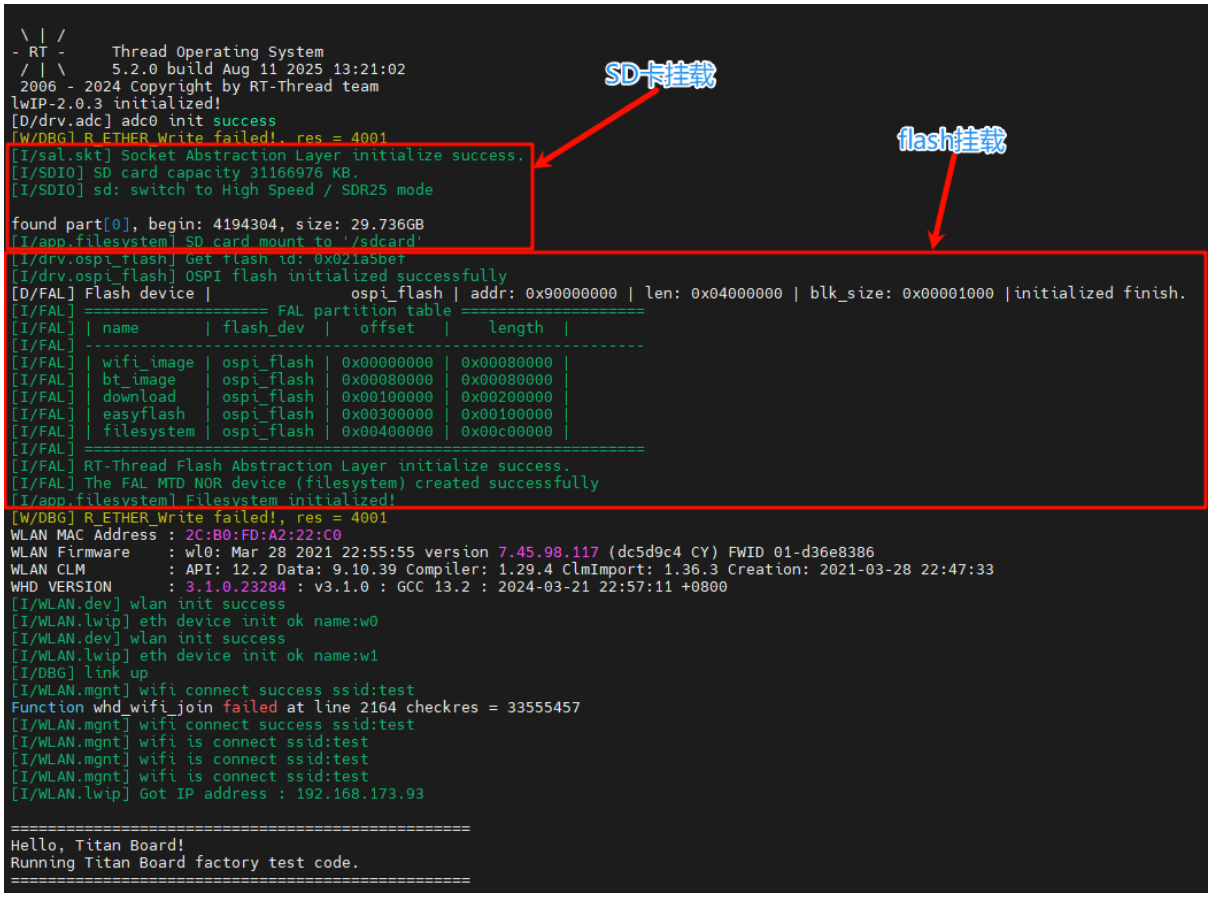
- **Blue LED:** Flashes continuously at 500 ms intervals, indicating program running status. If the blue LED stops flashing, the program has crashed.
- **Green LED:** Off by default. Lights up when all functional tests pass.
- **Red LED:** Off by default. Lights up when a functional test fails.



1.4 Program Execution Flow

Mounting HyperFlash and SD Card File Systems

After power-on, the board automatically mounts the SD card and HyperFlash file systems.




```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Aug 11 2025 13:21:02
 2006 - 2024 Copyright by RT-Thread team
lwIP-2.0.3 initialized!
[D/drv.adc] adc0 init success
[W/DBG] R_ETHER_Write failed!, res = 4001
[I/sal.skt] Socket Abstraction Layer initialize success.
[I/SDIO] SD card capacity 31166976 KB.
[I/SDIO] sd: switch to High Speed / SDR25 mode

found part[0], begin: 4194304, size: 29.736GB
[I/app.filesystem] SD card mount to '/sdcard'
[I/drv.ospi_flash] Get flash id: 0x021a5bef
[I/drv.ospi_flash] OSPI flash initialized successfully
[D/FAL] Flash device |              ospi_flash | addr: 0x90000000 | len: 0x04000000 | blk_size: 0x00001000 |initialized finish.
[I/FAL] ==================== FAL partition table ====================
[I/FAL] | name       | flash_dev  |   offset   |    length  |
[I/FAL] -------------------------------------------------------------
[I/FAL] | wifi_image | ospi_flash | 0x00000000 | 0x00080000 |
[I/FAL] | bt_image   | ospi_flash | 0x00080000 | 0x00080000 |
[I/FAL] | download   | ospi_flash | 0x00100000 | 0x00200000 |
[I/FAL] | easyflash  | ospi_flash | 0x00300000 | 0x00100000 |
[I/FAL] | filesystem | ospi_flash | 0x00400000 | 0x00c00000 |
[I/FAL] =============================================================
[I/FAL] RT-Thread Flash Abstraction Layer initialize success.
[I/FAL] The FAL MTD NOR device (filesystem) created successfully
[I/app.filesystem] Filesystem initialized!
[W/DBG] R_ETHER_Write failed!, res = 4001
WLAN MAC Address : 2C:B0:FD:A2:22:C0
WLAN Firmware    : wl0: Mar 28 2021 22:55:55 version 7.45.98.117 (dc5d9c4 CY) FWID 01-d36e8386
WLAN CLM         : API: 12.2 Data: 9.10.39 Compiler: 1.29.4 ClmImport: 1.36.3 Creation: 2021-03-28 22:47:33
WHD VERSION      : 3.1.0.23284 : v3.1.0 : GCC 13.2 : 2024-03-21 22:57:11 +0800
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w0
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w1
[I/DBG] link up
[I/WLAN.mgnt] wifi connect success ssid:test
Function whd_wifi_join failed at line 2164 checkres = 33555457
[I/WLAN.mgnt] wifi connect success ssid:test
[I/WLAN.mgnt] wifi is connect ssid:test
[I/WLAN.mgnt] wifi is connect ssid:test
[I/WLAN.mgnt] wifi is connect ssid:test
[I/WLAN.lwip] Got IP address : 192.168.173.93

=================================================
Hello, Titan Board!
Running Titan Board factory test code.
=================================================
```

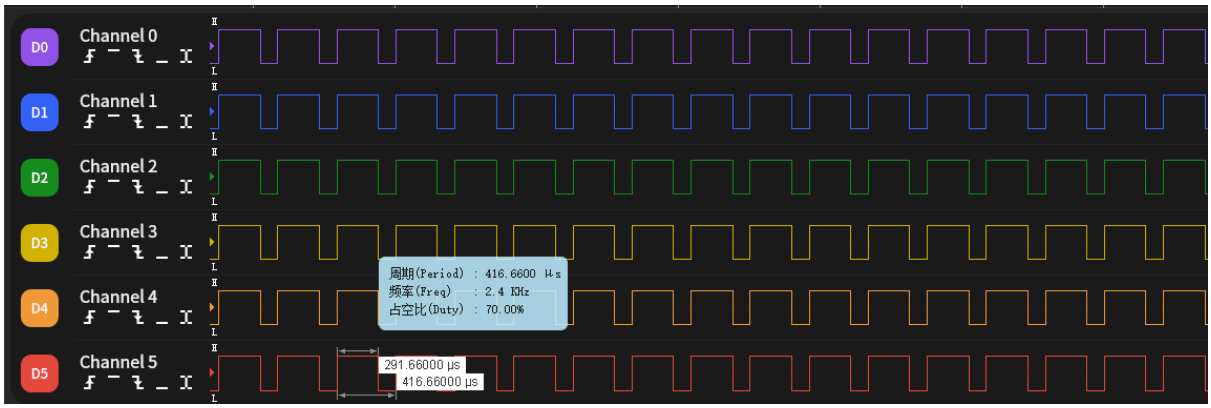
Automatic Wi-Fi Connection

After mounting the SD card, the board loads Wi-Fi firmware from it, initializes Wi-Fi, and automatically connects to the Wi-Fi network (SSID: **Titan Board**, Password: **12345678**).

```
=================================================
Hello, Titan Board!
Running Titan Board factory test code.
=================================================
msh />[W/DBG] R_ETHER_Write failed!, res = 4001
WLAN MAC Address : 2C:B0:FD:A2:22:C0
WLAN Firmware    : wl0: Mar 28 2021 22:55:55 version 7.45.98.117 (dc5d9c4 CY) FWID 01-d36e8386
WLAN CLM         : API: 12.2 Data: 9.10.39 Compiler: 1.29.4 ClmImport: 1.36.3 Creation: 2021-03-28 22:47:33
WHD VERSION      : 3.1.0.23284 : v3.1.0 : GCC 13.2 : 2024-03-21 22:57:11 +0800
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w0
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w1
[I/DBG] link up
[I/WLAN.mgnt] wifi connect success ssid:Titan Board
[I/WLAN.mgnt] wifi is connect ssid:Titan Board
[I/WLAN.mgnt] wifi is connect ssid:Titan Board
[I/WLAN.lwip] Got IP address : 192.168.173.93
```

PWM Output

After power-on, the six PWM pins on the Raspberry Pi interface output PWM waves. LEDs connected to these pins should show brightness changes (breathing light effect).



Start production test procedure

Start production testing by typing **factory_test** in Serial assistant.

Buzzer Test

The buzzer plays the melody of “Two Tigers” as a functional test.

HyperRAM Test

The HyperRAM test program measures HyperRAM read/write speed.

```
======================RAM TEST=============================
Write speed: 88000 KB/s (369 ms)
Read speed: 23000 KB/s (1411 ms)
Hyper RAM test passed successfully!
======================RAM TEST END=========================
```

Wi-Fi Test

The Wi-Fi test program initializes the Wi-Fi module, loads firmware from the SD card, and uses the ping command to test connectivity.

```
======================WIFI TEST============================
[Wi-Fi]Start ping!
msh />60 bytes from 192.168.173.247 icmp_seq=1 ttl=64 time=36 ms
60 bytes from 192.168.173.247 icmp_seq=2 ttl=64 time=25 ms
60 bytes from 192.168.173.247 icmp_seq=3 ttl=64 time=15 ms
60 bytes from 192.168.173.247 icmp_seq=4 ttl=64 time=15 ms

--- 192.168.173.247 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
minimum = 15ms, maximum = 36ms, average = 22ms
======================WIFI TEST END========================
```

Ethernet Test

The Ethernet test program uses the ping command to verify network connectivity.

```
=====================ETH TEST============================
[ETH]Start ping!
60 bytes from 10.23.8.254 icmp_seq=1 ttl=254 time=0 ms
60 bytes from 10.23.8.254 icmp_seq=2 ttl=254 time=0 ms
60 bytes from 10.23.8.254 icmp_seq=3 ttl=254 time=0 ms
60 bytes from 10.23.8.254 icmp_seq=4 ttl=254 time=0 ms

--- 10.23.8.254 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
minimum = 0ms, maximum = 0ms, average = 0ms
=====================ETH TEST END============================
```

RTC Test

The RTC test outputs the configured date and time in the terminal.

```
=====================RTC TEST============================
set RTC date to 2025-8-1
now: Wed Aug  9 08:18:36 3386
=====================RTC TEST END============================
```

ADC Test

The ADC test program sequentially tests channels 0, 1, 2, and 3 of ADC0.

```
=====================ADC TEST============================
=====================ADC0-CHANNEL0 Test Start============================
vol: 0
the adc voltage is: 0.00
vol: 80
the adc voltage is: 0.00
vol: 88
the adc voltage is: 0.00
vol: 84
the adc voltage is: 0.00
vol: 76
the adc voltage is: 0.00
vol: 52
the adc voltage is: 0.00
vol: 84
the adc voltage is: 0.00
vol: 80
the adc voltage is: 0.00
=====================ADC0-CHANNEL0 END Test============================
```

CANFD Test

In the CANFD test program, CANFD0 transmits data, and CANFD1 receives it.

```
=====================CANFD TEST============================
ID:78   messege:01234567
ID:78   messege:01234567
=====================CANFD TEST END============================
```

IMU Test

The IMU test program verifies correct data reading from the BMI088 module.

```
====================BMI088 TEST============================
[I/sensor] rt_sensor[acce_bmi] init success
[I/sensor] rt_sensor[gyro_bmi] init success
acce value:[X]:-52 [Y]:967 [Z]:9603
gyro value:[X]:-6994 [Y]:10491 [Z]:-6994

acce value:[X]:-64 [Y]:1037 [Z]:9575
gyro value:[X]:3497 [Y]:41965 [Z]:3497

====================BMI088 TEST END========================
```

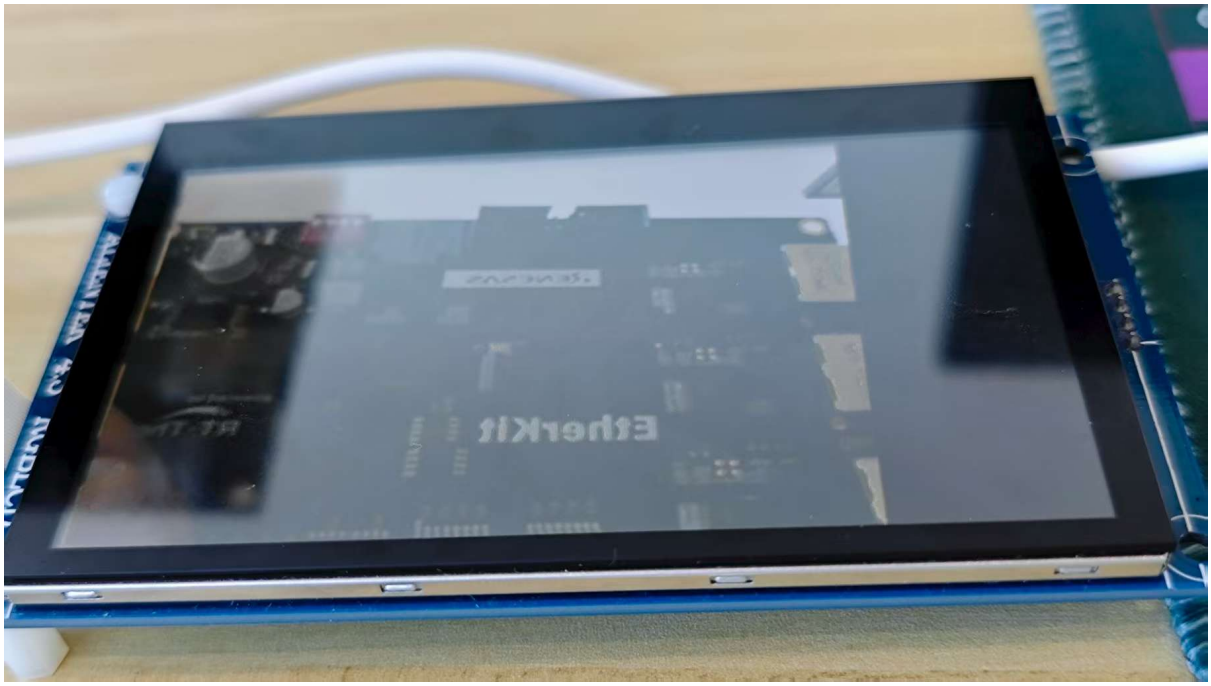
Magnetometer Test

The magnetometer test program verifies correct data reading from the IST8310 module.

```
====================IST8310 TEST============================
Magnetic: X=0.00 μT, Y=0.00 μT, Z=0.00 μT
Heading: 225.69°
Magnetic: X=15872.00 μT, Y=-21503.00 μT, Z=-4866.00 μT
Heading: 225.35°
====================IST8310 TEST END========================
```

Camera and Screen Test

At the end of the test program, the camera is activated. Wave your hand in front of the camera to trigger capture; the LCD screen will display the captured image.



1.5 Test Result Description

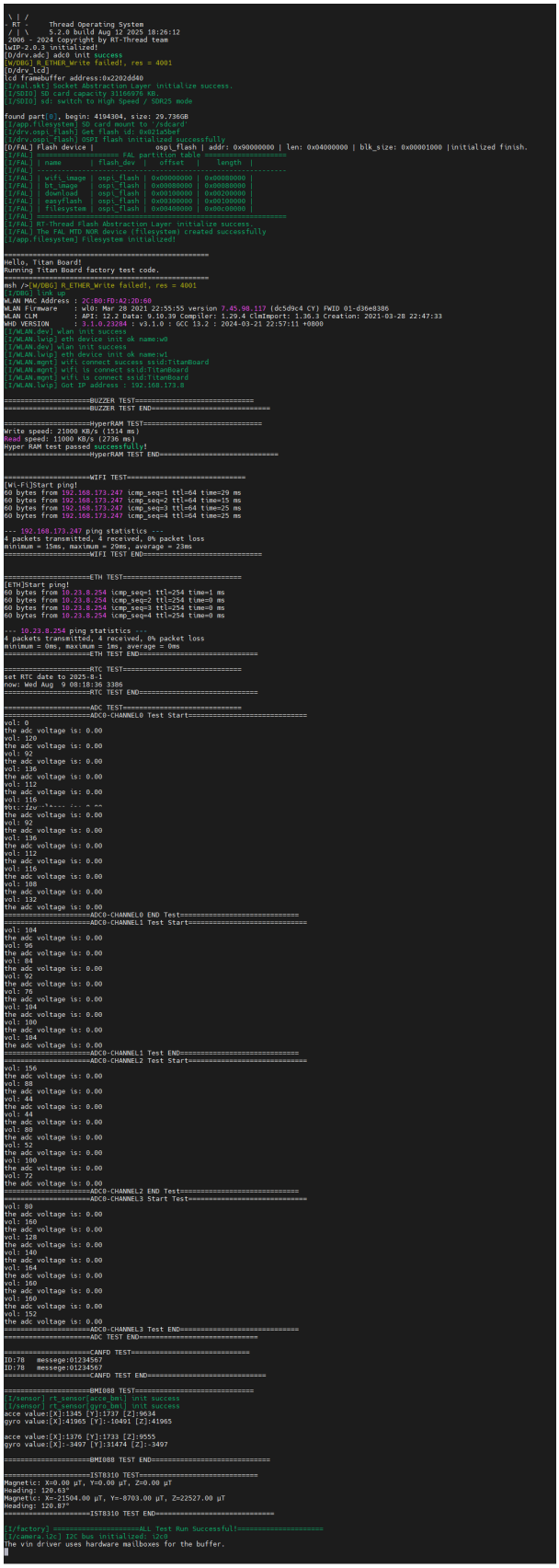
- **All tests passed:** At the end of testing, if all functions pass, the green LED lights up and the terminal outputs ALL Test Run Successful!.

```
====================BMI088 TEST============================
[I/sensor] rt_sensor[acce_bmi] init success
[I/sensor] rt_sensor[gyro_bmi] init success
acce value:[X]:-52 [Y]:967 [Z]:9603
gyro value:[X]:-6994 [Y]:10491 [Z]:-6994

acce value:[X]:-64 [Y]:1037 [Z]:9575
gyro value:[X]:3497 [Y]:41965 [Z]:3497

====================BMI088 TEST END============================

====================IST8310 TEST============================
Magnetic: X=0.00 µT, Y=0.00 µT, Z=0.00 µT
Heading: 225.69°
Magnetic: X=15872.00 µT, Y=-21503.00 µT, Z=-4866.00 µT
Heading: 225.35°
====================IST8310 TEST END============================

[I/factory] ====================ALL Test Run Successful!====================
```

- **Test failures detected:** If some tests fail, the red LED lights up and the terminal outputs which functions failed.

```
[E/factory] RS485 Test Failed!

[E/factory] ERROR Flag is 57596
[E/factory] Detected errors:
 - ADC0_CH0_ERROR
 - ADC0_CH1_ERROR
 - ADC0_CH2_ERROR
 - ADC0_CH3_ERROR
 - RS485_ERROR
 - CANFD_ERROR
 - HYPERRAM_ERROR
 - ETH_ERROR
 - WIFI_ERROR
```

Example terminal output of test program:


. \ | /
- RT -   Thread Operating System
 / | \   5.2.0 build Aug 12 2025 18:26:12
2006 - 2024 Copyright by RT-Thread team
lwIP-2.0.3 initialized!
[D/drv.adc] adc0 init success
[W/DBG] R_ETHER_Write failed!, res = 4001
[D/drv_lcd]
lcd framebuffer address:0x2202dd40
[I/sal.skt] Socket Abstraction Layer initialize success.
[I/SDIO] SD card capacity 31166976 KB.
[I/SDIO] sd: switch to High Speed / SDR25 mode
found part[0], begin: 4194304, size: 29.736GB
[I/app.filesystem] SD card mount to '/sdcard'
[I/drv.ospi_flash] Get flash id: 0x021a5bef
[I/drv.ospi_flash] OSPI flash initialized successfully
[D/FAL] Flash device | ospi_flash | addr: 0x90000000 | len: 0x04000000 | blk_size: 0x00001000 |initialized finish.
[I/FAL] ================== FAL partition table ==================
[I/FAL] | name | flash_dev | offset | length |
[I/FAL] ---
[I/FAL] | wifi_image | ospi_flash | 0x00000000 | 0x00880000 |
[I/FAL] | bt_image | ospi_flash | 0x00880000 | 0x00880000 |
[I/FAL] | download | ospi_flash | 0x01100000 | 0x00200000 |
[I/FAL] | easyflash | ospi_flash | 0x01300000 | 0x00100000 |
[I/FAL] | filesystem | ospi_flash | 0x01400000 | 0x0bc00000 |
[I/FAL] ==
[I/FAL] RT-Thread Flash Abstraction Layer initialize success.
[I/FAL] The FAL MTD NOR device (filesystem) created successfully
[I/app.filesystem] Filesystem initialized!
==
Hello, Titan Board!
Running Titan Board factory test code.
==
msh />[W/DBG] R_ETHER_Write failed!, res = 4001
[I/DBG] link up
WLAN MAC Address : 2C:80:FD:A2:2D:60
WLAN Firmware   : wl0: Mar 28 2021 22:55:55 version 7.45.98.117 (dc5d9c4 CY) FWID 01-d36e0386
WLAN CLM   : API: 12.2 Data: 9.10.39 Compiler: 1.29.4 ClmImport: 1.36.3 Creation: 2021-03-28 22:47:33
WHD VERSION   : 3.1.0.23284 : v3.1.0 : GCC 13.2 : 2024-03-21 22:57:11 +0800
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w0
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w1
[I/WLAN.mgnt] wifi connect success ssid:TitanBoard
[I/WLAN.mgnt] wifi is connect ssid:TitanBoard
[I/WLAN.mgnt] wifi is connect ssid:TitanBoard
[I/WLAN.lwip] Got IP address : 192.168.173.8
====================BUZZER TEST============================
====================BUZZER TEST END============================
====================HyperRAM TEST============================
Write speed: 21000 KB/s (1514 ms)
Read speed: 11000 KB/s (2736 ms)
Hyper RAM test passed successfully!
====================HyperRAM TEST END============================
====================WIFI TEST============================
[Wi-Fi]Start ping!
60 bytes from 192.168.173.247 icmp_seq=1 ttl=64 time=29 ms
60 bytes from 192.168.173.247 icmp_seq=2 ttl=64 time=15 ms
60 bytes from 192.168.173.247 icmp_seq=3 ttl=64 time=25 ms
60 bytes from 192.168.173.247 icmp_seq=4 ttl=64 time=25 ms
--- 192.168.173.247 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
minimum = 15ms, maximum = 29ms, average = 23ms
====================WIFI TEST END============================
====================ETH TEST============================
[ETH]Start ping!
60 bytes from 10.23.8.254 icmp_seq=1 ttl=254 time=1 ms
60 bytes from 10.23.8.254 icmp_seq=2 ttl=254 time=0 ms
60 bytes from 10.23.8.254 icmp_seq=3 ttl=254 time=0 ms
60 bytes from 10.23.8.254 icmp_seq=4 ttl=254 time=0 ms
--- 10.23.8.254 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
minimum = 0ms, maximum = 1ms, average = 0ms
====================ETH TEST END============================
====================RTC TEST============================
set RTC date to 2025-8-1
now: Wed Aug 9 00:18:36 3386
====================RTC TEST END============================
====================ADC TEST============================
====================ADC0-CHANNEL0 Test Start============================
vol: 0
the adc voltage is: 0.00
vol: 120
the adc voltage is: 0.00
vol: 92
the adc voltage is: 0.00
vol: 136
the adc voltage is: 0.00
vol: 112
the adc voltage is: 0.00
vol: 116
the adc voltage is: 0.00
vol: 92
the adc voltage is: 0.00
vol: 136
the adc voltage is: 0.00
vol: 112
the adc voltage is: 0.00
vol: 116
the adc voltage is: 0.00
vol: 108
the adc voltage is: 0.00
vol: 132
the adc voltage is: 0.00
====================ADC0-CHANNEL0 END Test============================
====================ADC0-CHANNEL1 Test Start============================
vol: 104
the adc voltage is: 0.00
vol: 96
the adc voltage is: 0.00
vol: 84
the adc voltage is: 0.00
vol: 92
the adc voltage is: 0.00
vol: 76
the adc voltage is: 0.00
vol: 104
the adc voltage is: 0.00
vol: 100
the adc voltage is: 0.00
vol: 104
the adc voltage is: 0.00
====================ADC0-CHANNEL1 Test END============================
====================ADC0-CHANNEL2 Test Start============================
vol: 156
the adc voltage is: 0.00
vol: 88
the adc voltage is: 0.00
vol: 44
the adc voltage is: 0.00
vol: 44
the adc voltage is: 0.00
vol: 80
the adc voltage is: 0.00
vol: 52
the adc voltage is: 0.00
vol: 100
the adc voltage is: 0.00
vol: 72
the adc voltage is: 0.00
====================ADC0-CHANNEL2 END Test============================
====================ADC0-CHANNEL3 Start Test============================
vol: 80
the adc voltage is: 0.00
vol: 160
the adc voltage is: 0.00
vol: 128
the adc voltage is: 0.00
vol: 140
the adc voltage is: 0.00
vol: 164
the adc voltage is: 0.00
vol: 160
the adc voltage is: 0.00
vol: 160
the adc voltage is: 0.00
vol: 152
the adc voltage is: 0.00
====================ADC0-CHANNEL3 Test END============================
====================ADC TEST END============================
====================CANFD TEST============================
ID:78   messege:01234567
ID:78   messege:01234567
====================CANFD TEST END============================
====================BMI088 TEST============================
[I/sensor] rt_sensor[acce_bmi] init success
[I/sensor] rt_sensor[gyro_bmi] init success
acce value:[X]:1345 [Y]:1737 [Z]:9634
gyro value:[X]:41965 [Y]:-10491 [Z]:41965
acce value:[X]:1376 [Y]:1733 [Z]:9555
gyro value:[X]:-3497 [Y]:31474 [Z]:-3497
====================BMI088 TEST END============================
====================IST8310 TEST============================
Magnetic: X=0.00 μT, Y=0.00 μT, Z=0.00 μT
Heading: 120.63°
Magnetic: X=-21504.00 μT, Y=-8703.00 μT, Z=22527.00 μT
Heading: 120.87°
====================IST8310 TEST END============================
[I/factory] ====================ALL Test Run Successful!====================
[I/camera.i2c] I2C bus initialized. i2c2
The vin driver uses hardware mailboxes for the buffer.

FAQ

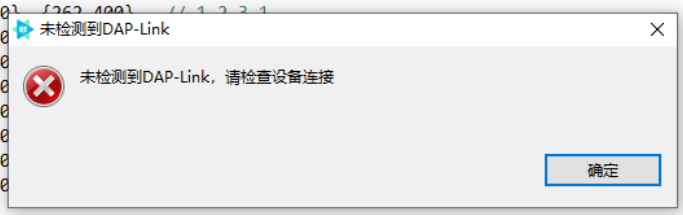
1. Frequently Asked Questions (FAQ)

This document collects common issues encountered by users during usage and provides corresponding solutions.

1.1 Firmware Programming

Q1. DAP-Link not detected during firmware flashing

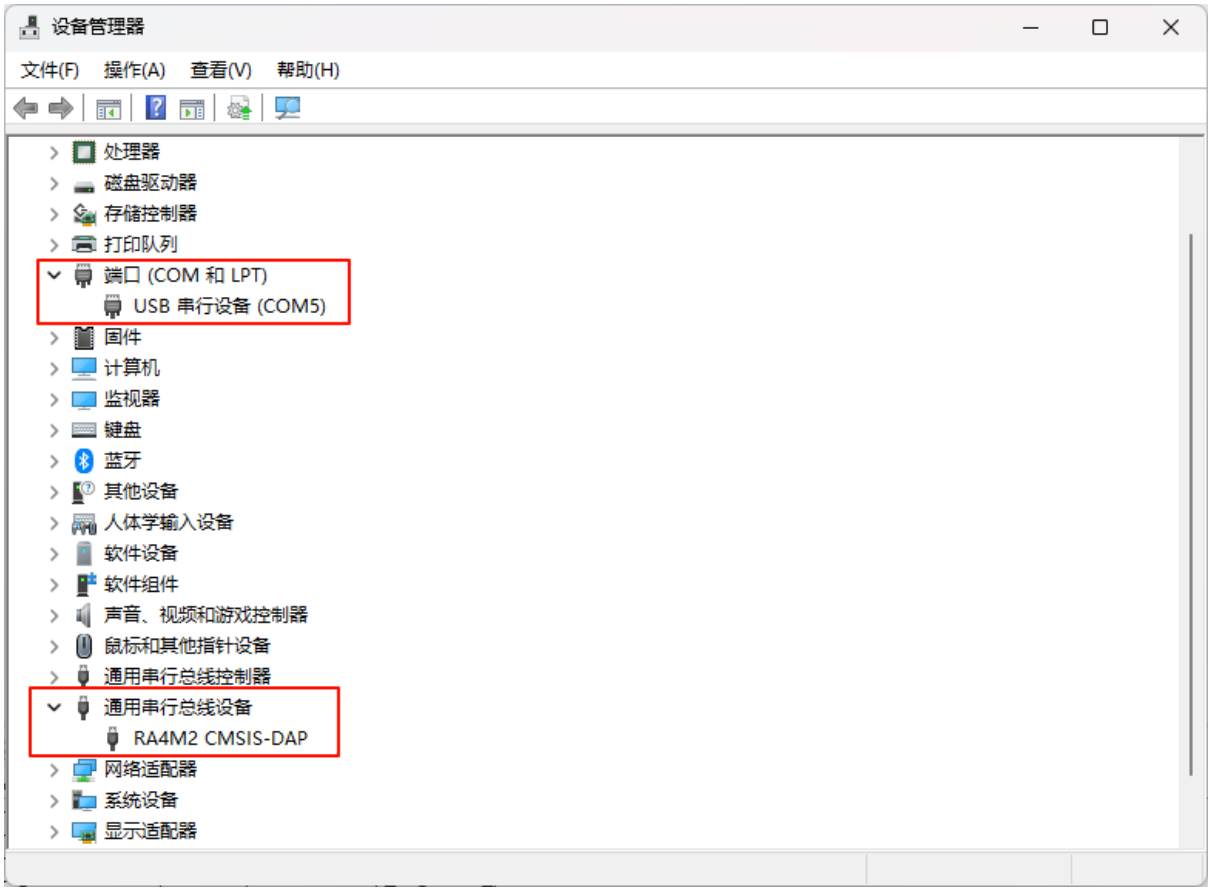
```
 uint16_t freq;    // 频率Hz
 uint16_t duration; // 持续时间ms
ote_t;

e_t song[] =

 {262,400}, {294,400}, {330,400},
 {262,400}, {294,400}, {330,400},
 {330,400}, {349,400}, {392,800},
 {330,400}, {349,400}, {392,800},
 {392,200}, {440,200}, {392,200},
 {392,200}, {440,200}, {392,200},
 {262,400}, {196,400}, {262,400},
 {262,400}, {196,400}, {262,400},

tic int buzzer_test(void)
```



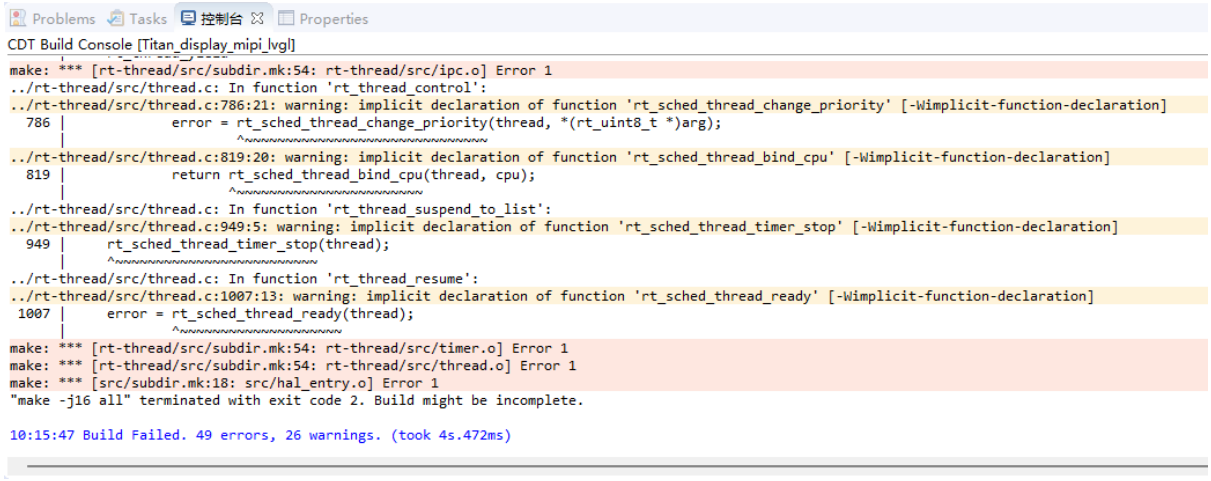
Solution:

Uninstall the **WinUSB** device in *Device Manager*, then reconnect the USB cable.

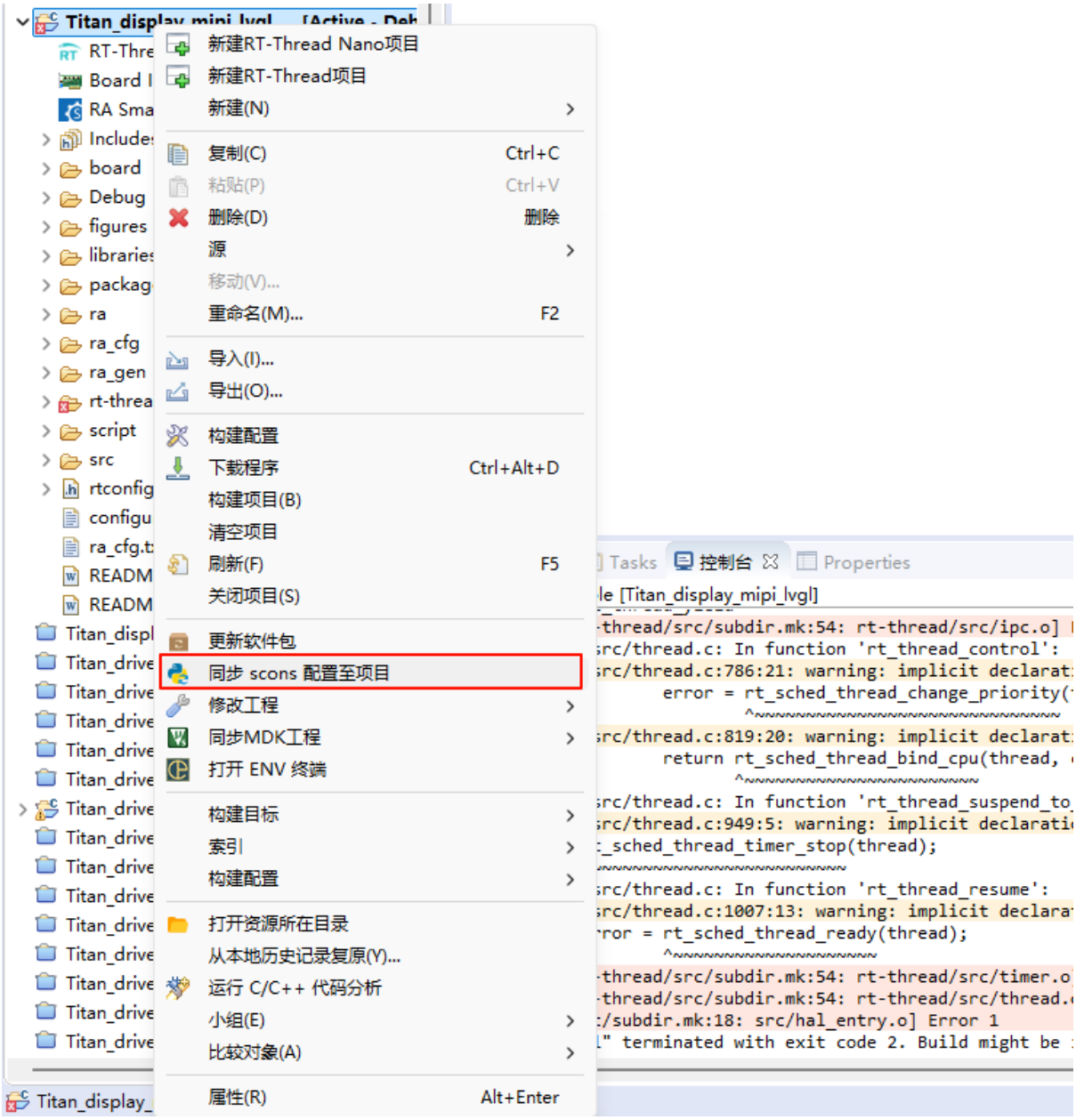
After doing so, both the serial port and **CMSIS-DAP** devices should appear in *Device Manager*.



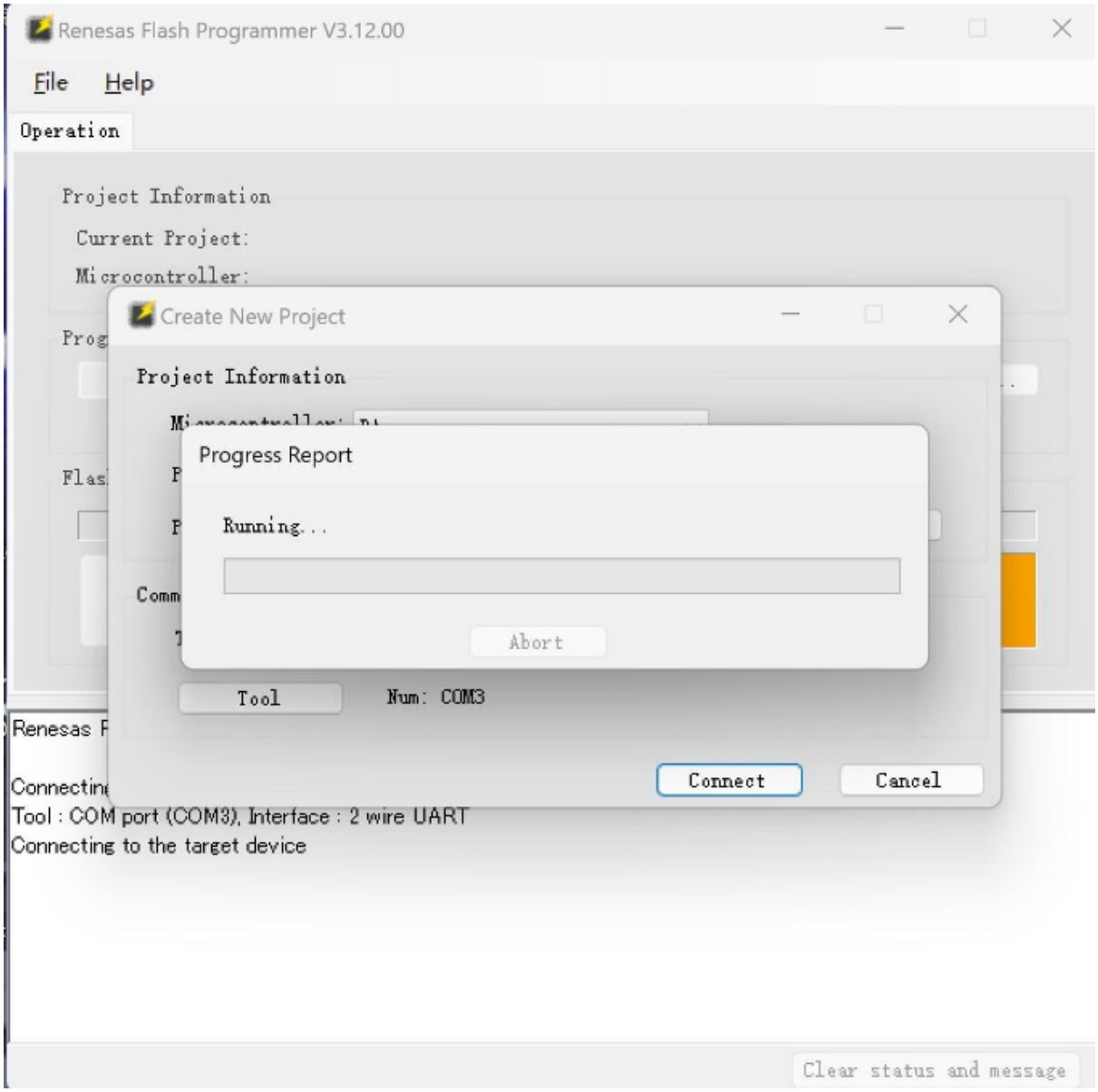
Q2. First compilation error after creating an example project



Solution: Right-click the project and select **Synchronize scons configuration to project**.

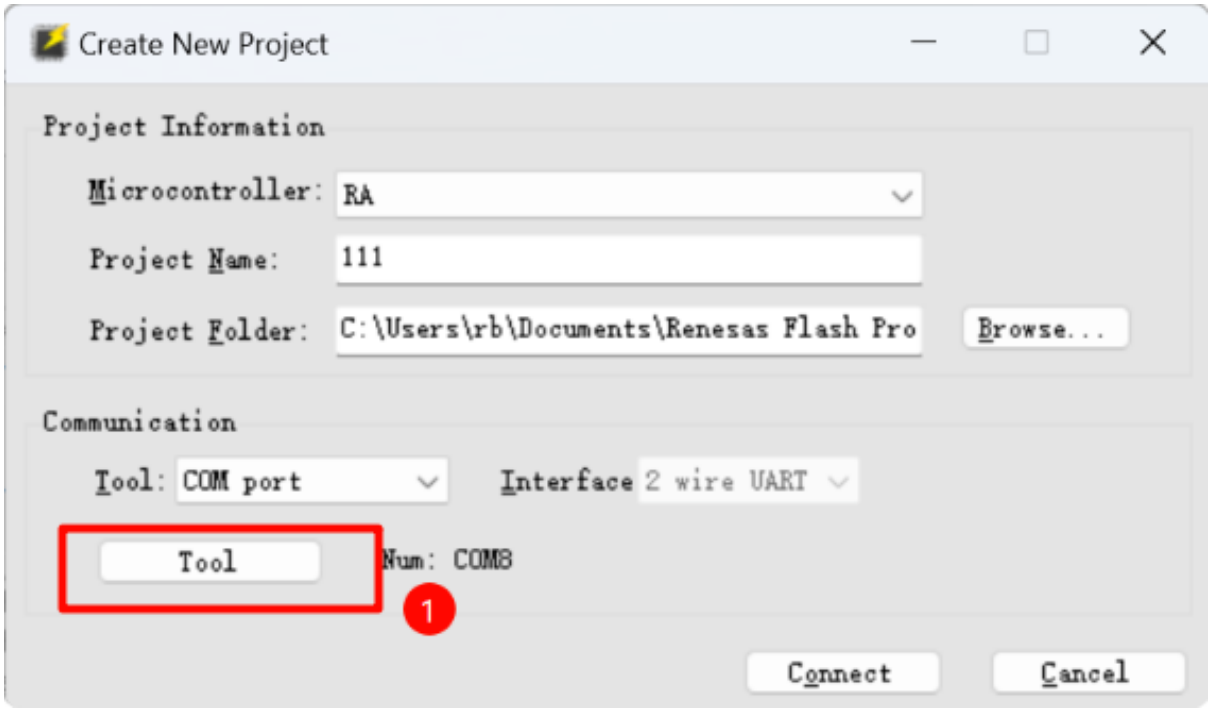

新建RT-Thread Nano项目
新建RT-Thread项目
新建(N)
复制(C) Ctrl+C
粘贴(P) Ctrl+V
删除(D) 删除
源
移动(V)...
重命名(M)... F2
导入(I)...
导出(O)...
构建配置
下载程序 Ctrl+Alt+D
构建项目(B)
清空项目
刷新(F) F5
关闭项目(S)
更新软件包
同步 scons 配置至项目
修改工程
同步MDK工程
打开 ENV 终端
构建目标
索引
构建配置
打开资源所在目录
从本地历史记录复原(Y)...
运行 C/C++ 代码分析
小组(E)
比较对象(A)
属性(R) Alt+Enter
Tasks
控制台
Properties
le [Titan_display_mipi_lvgl]
-thread/src/subdir.mk:54: rt-thread/src/ipc.o]
src/thread.c: In function 'rt_thread_control':
src/thread.c:786:21: warning: implicit declarat
error = rt_sched_thread_change_priority(
src/thread.c:819:20: warning: implicit declarat
return rt_sched_thread_bind_cpu(thread,
src/thread.c: In function 'rt_thread_suspend_to
src/thread.c:949:5: warning: implicit declarati
_sched_thread_timer_stop(thread);
src/thread.c: In function 'rt_thread_resume':
src/thread.c:1007:13: warning: implicit declara
ror = rt_sched_thread_ready(thread);
-thread/src/subdir.mk:54: rt-thread/src/timer.o
-thread/src/subdir.mk:54: rt-thread/src/thread.
:/subdir.mk:18: src/hal_entry.o] Error 1
l" terminated with exit code 2. Build might be

Q3. RFP stuck while creating a new project

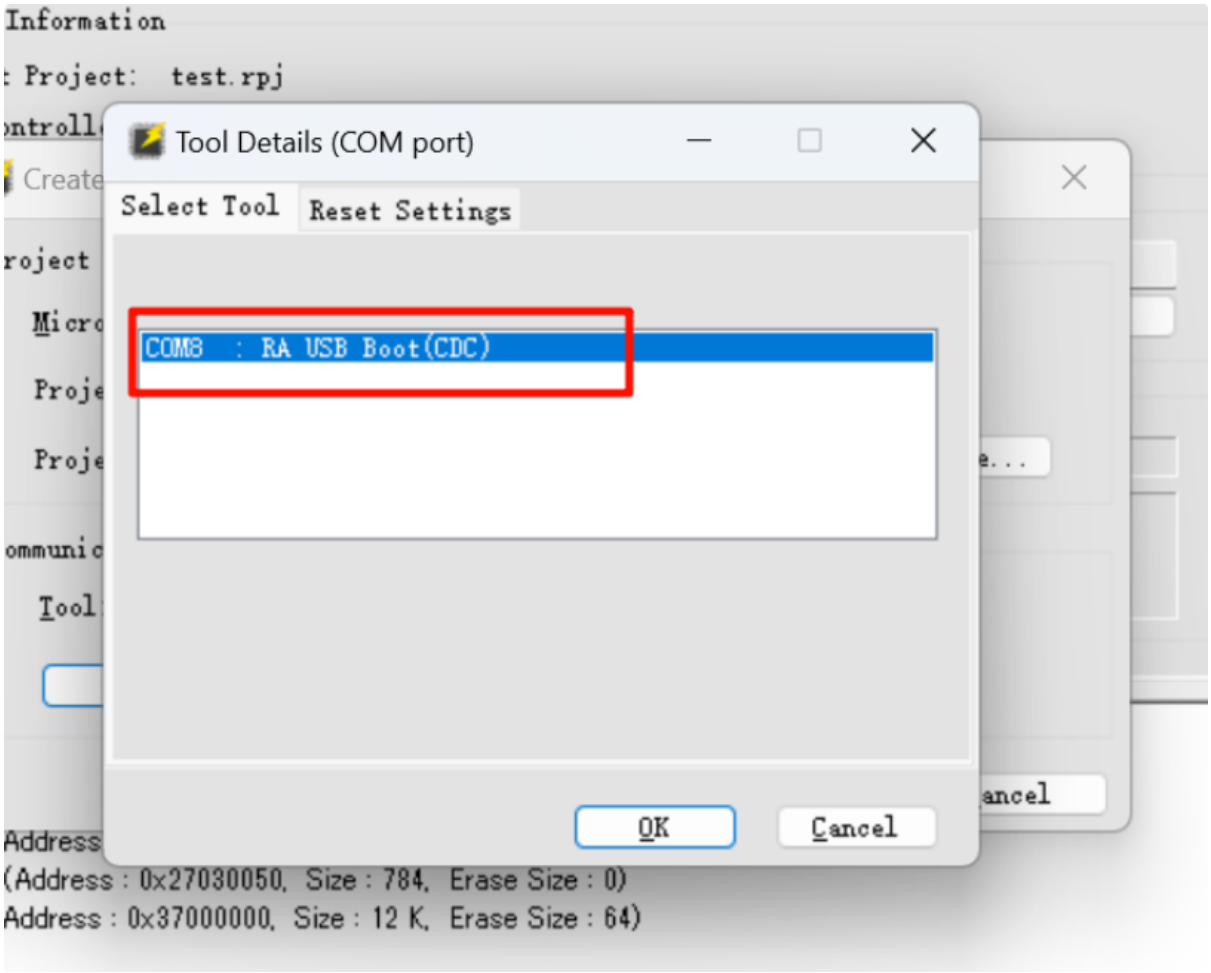


Solution:

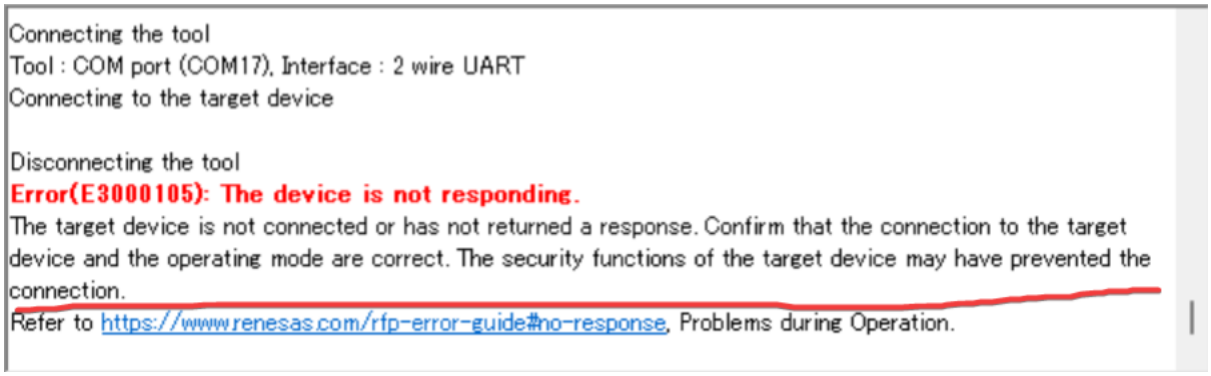
1. Select **Tool**



2. Put the development board into **BOOT mode** (hold the **BOOT** button while pressing **RST**). When the following window appears, click **OK** to connect.(Do not release the BOOT button until the project is created successfully.)



Q4. Unable to flash HEX file using RFP

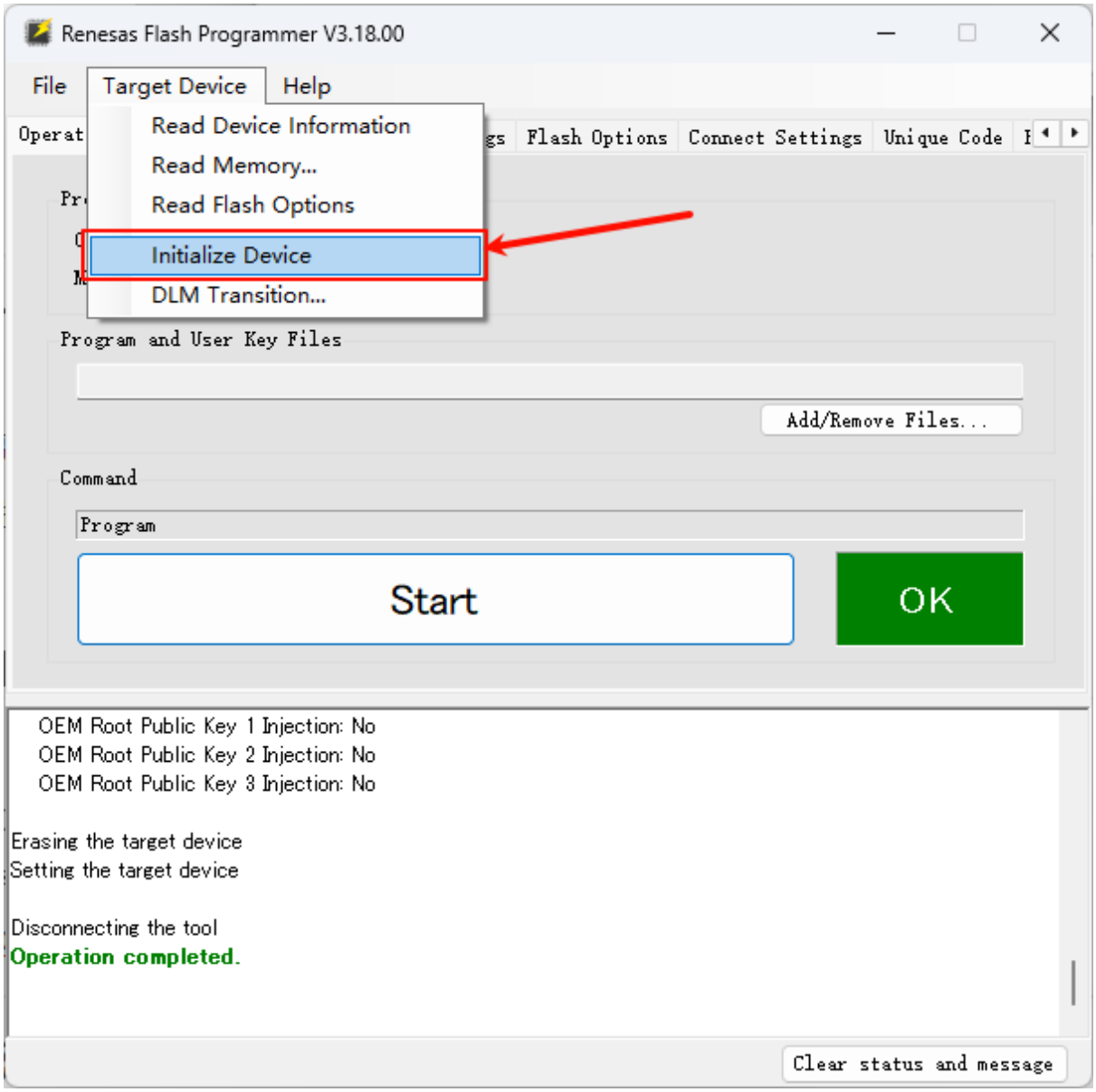


1. **Solution:**
    1. Connect via the USB port labeled **USB-DEV**.
    2. Make sure the development board is in **BOOT mode**.

Q5. Chip cannot be programmed

Solution: If the chip cannot be programmed, you can reinitialize it as follows:

Enter **BOOT mode**, then click **Initialize Chip** as shown below:

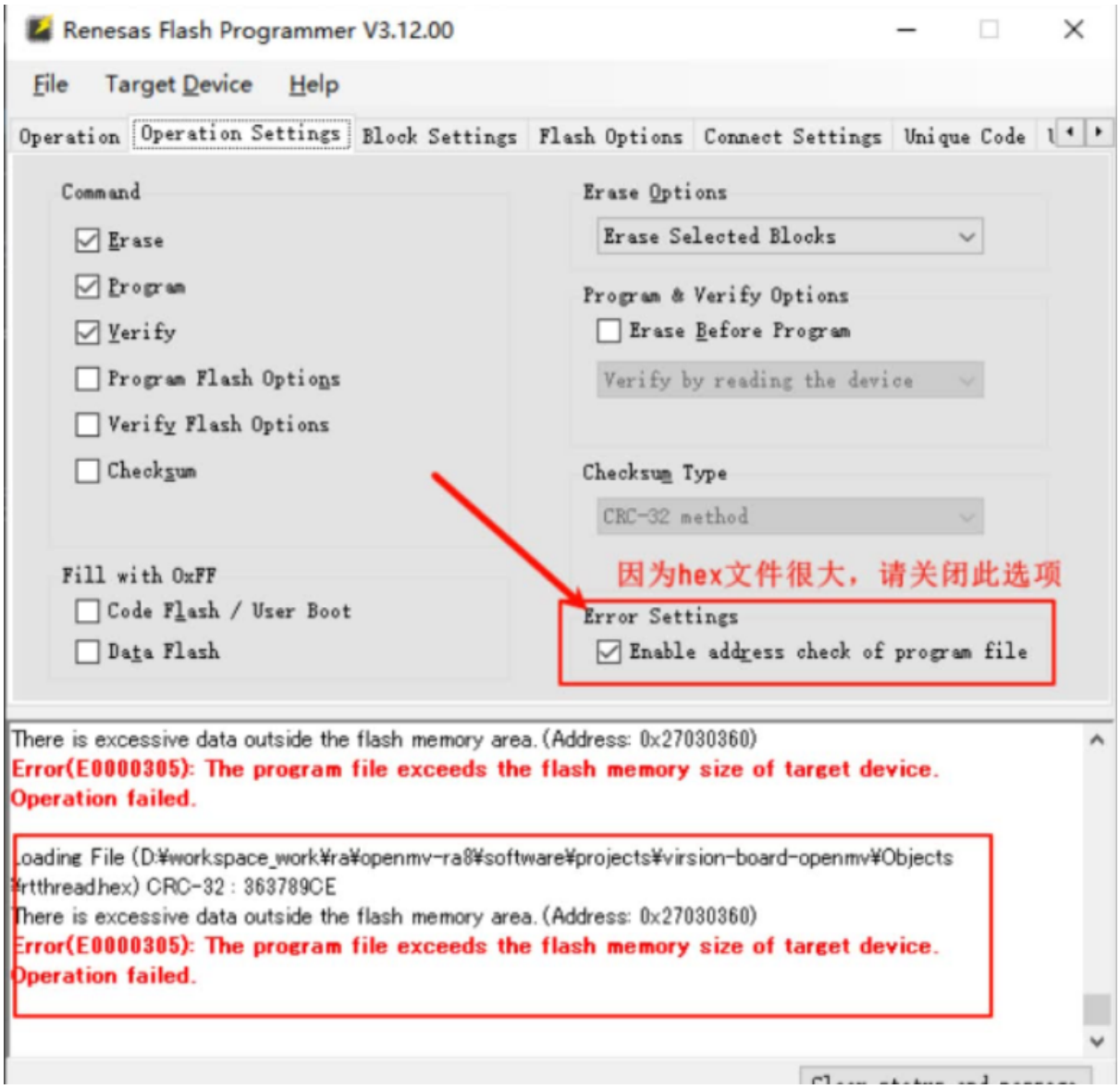


Q6. Error after clicking download:

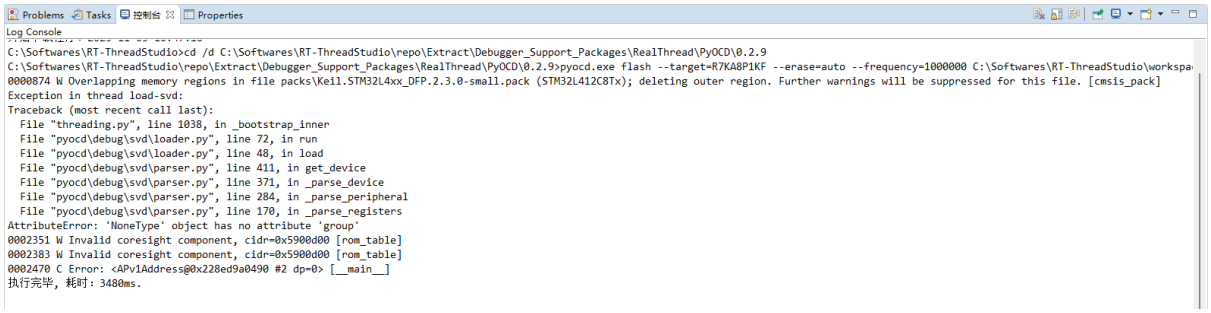
```
Error(E0000305): The program file exceeds the flash memory size of target device.
  Operation failed.
```

Solution:

Refer to the image below:



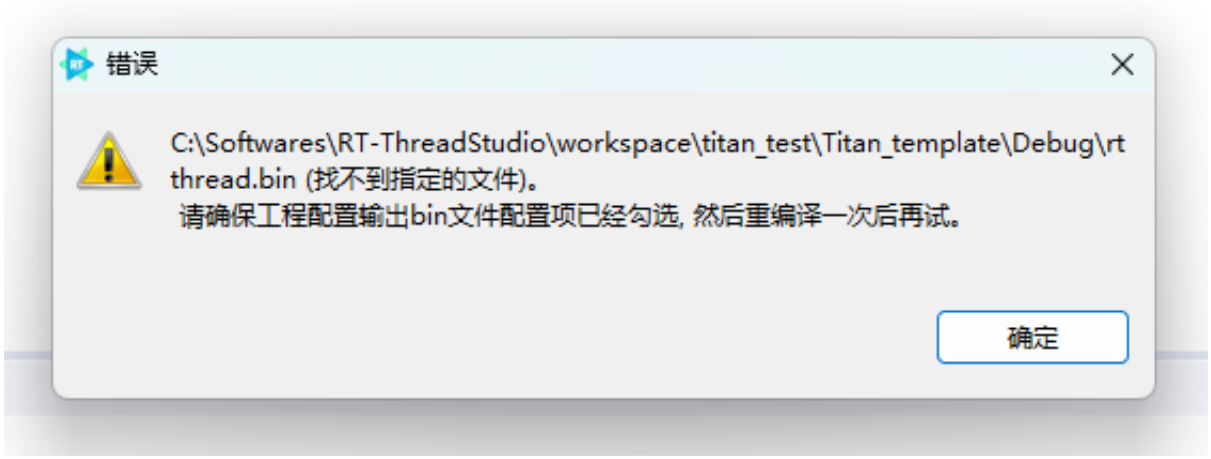
Q7. DAP-Link malfunction after flashing dual-core program



1. **Solution:**
    1. **Method 1:** Use **RFP** to initialize the device and flash a **single-core project**.
    2. **Method 2:** A community developer shared an article about **updating the debugger firmware**, which supports flashing **dual-core projects**: https://club.rt-thread.org/ask/article/3387ad4472d12ead.html

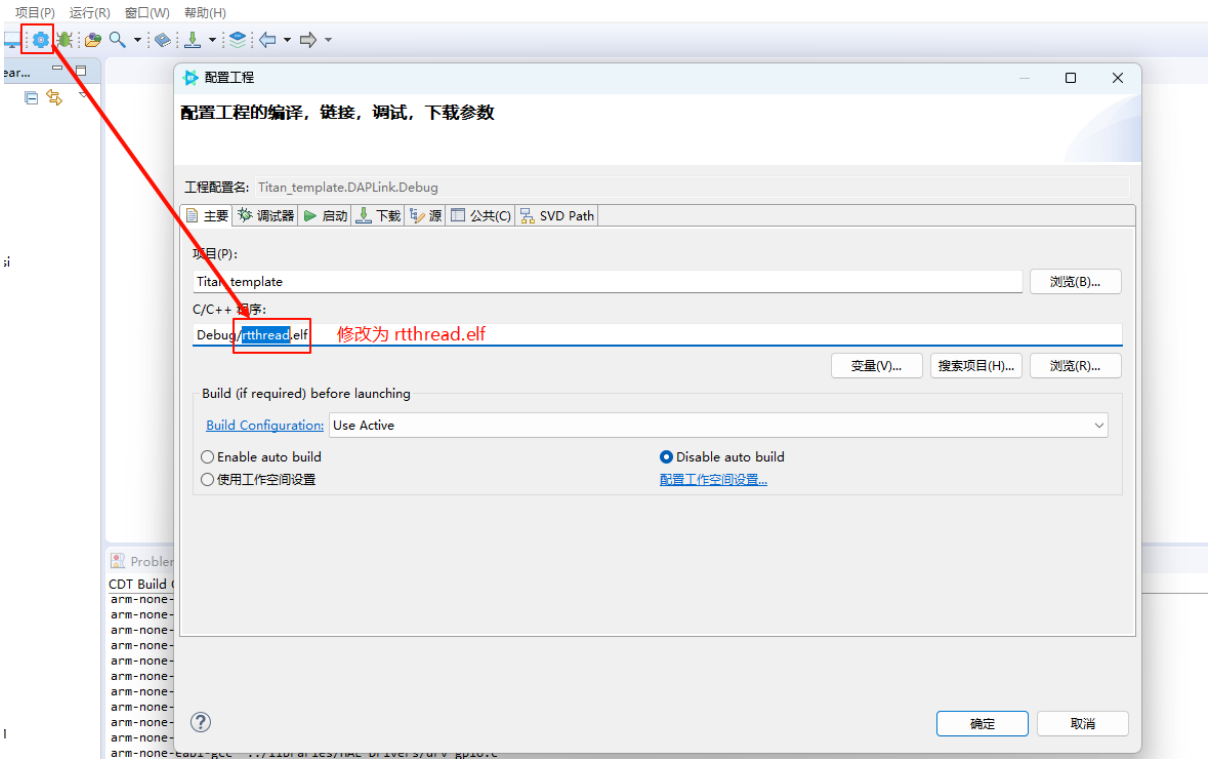
---

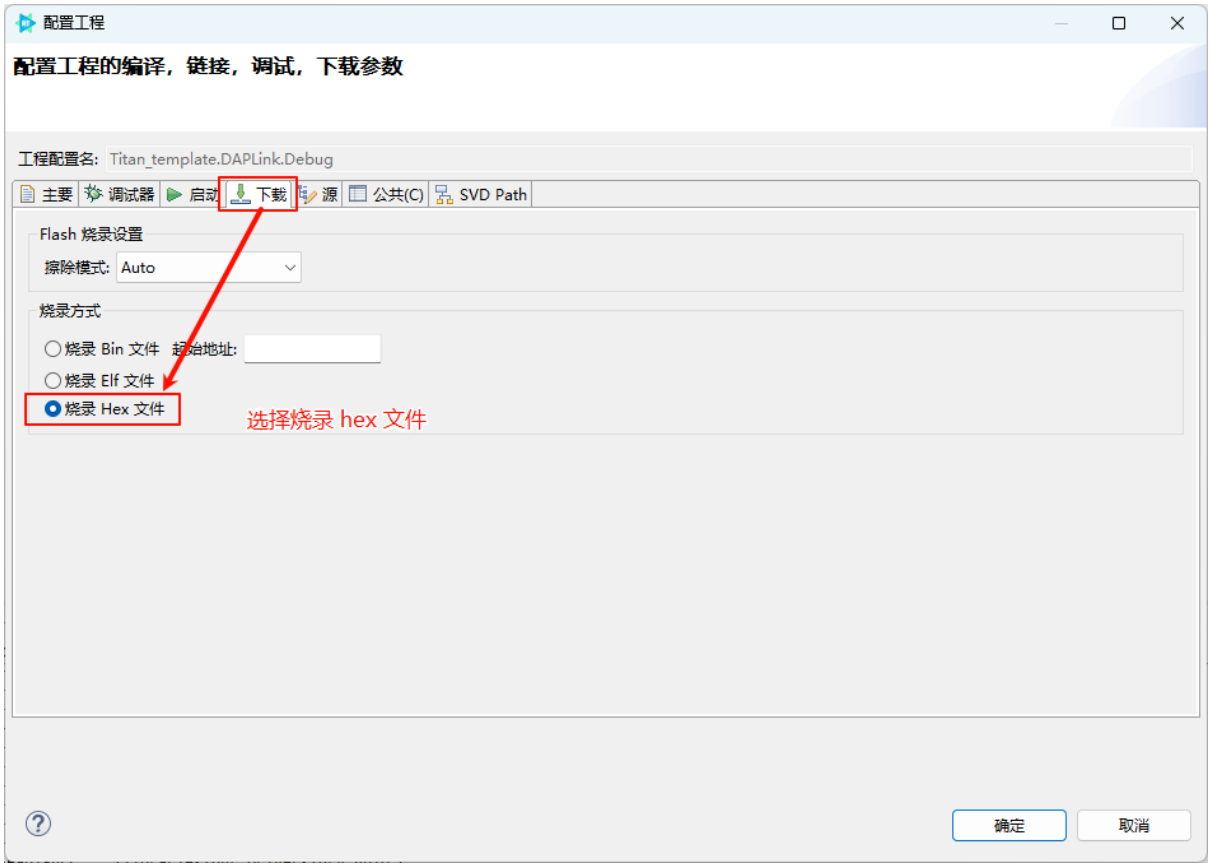
Q8. Error "rtthread.bin not found (file not found)" when flashing



Solution:

Configure the project's **debug and download parameters** as shown below:


项目(P) 运行(R) 窗口(W) 帮助(H)
配置工程
配置工程的编译，链接，调试，下载参数
工程配置名: Titan_template.DAPLink.Debug
主要 调试器 启动 下载 源 公共(C) SVD Path
项目(P):
Titan_template
浏览(B)...
C/C++ 程序:
Debug/rtthread.elf
修改为 rtthread.elf
变量(V)...
搜索项目(H)...
浏览(R)...
Build (if required) before launching
Build Configuration: Use Active
Enable auto build
Disable auto build
使用工作空间设置
配置工作空间设置...
确定
取消


配置工程
配置工程的编译，链接，调试，下载参数
工程配置名: Titan_template.DAPLink.Debug
主要 调试器 启动 下载 源 公共(C) SVD Path
Flash 烧录设置
擦除模式: Auto
烧录方式
烧录 Bin 文件 起始地址:
烧录 Elf 文件
烧录 Hex 文件
选择烧录 hex 文件
确定
取消

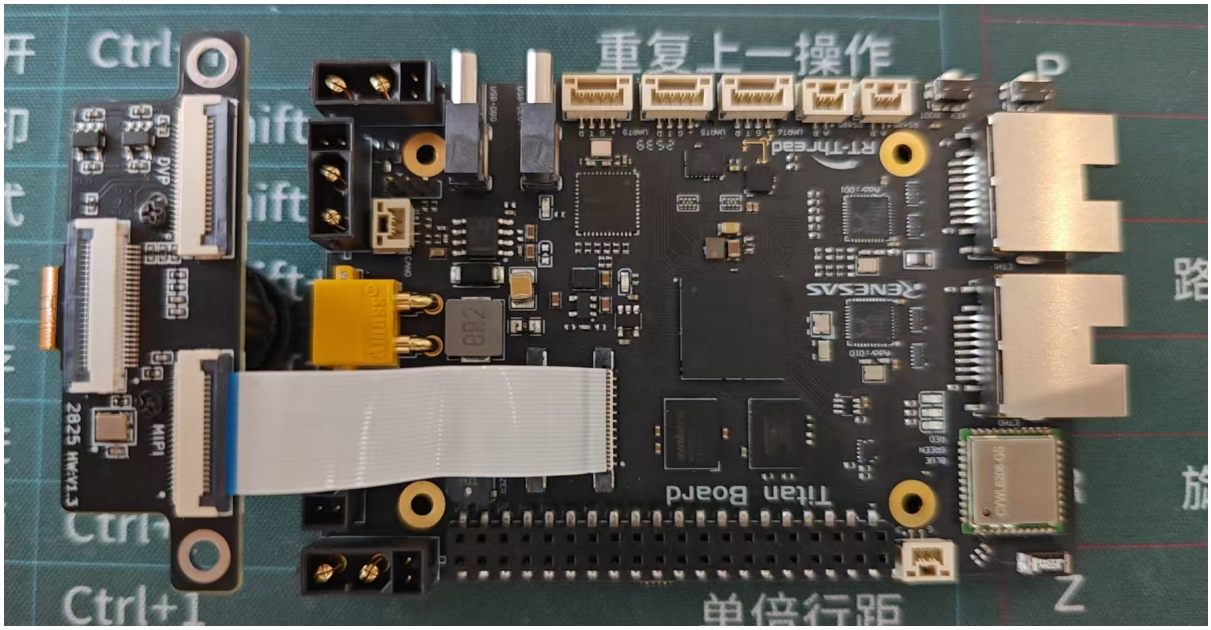
1.2 Camera and Display Connection

Q9. Camera Connection

Solution:

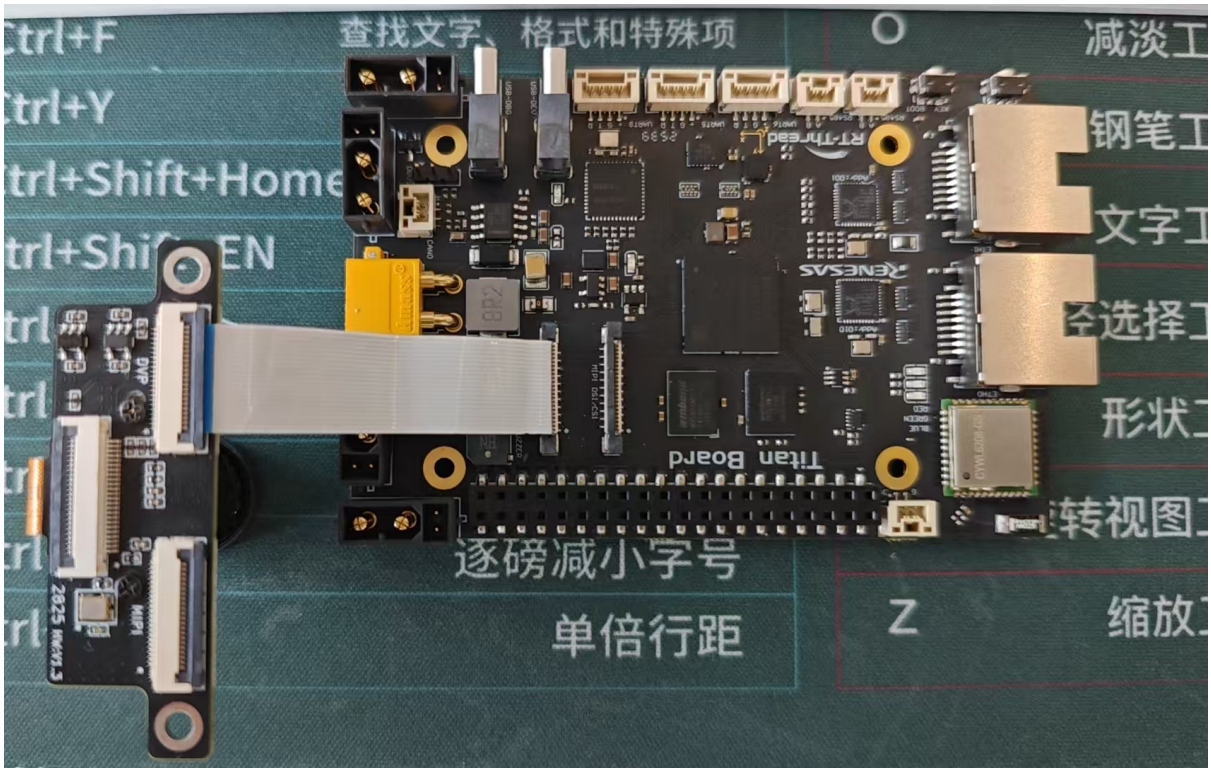
- **MIPI CSI:**

  Use a **22-pin reverse FFC cable** to connect the development board’ s **MIPI DSI/CSI** connector to the camera adapter board’ s **MIPI** connector.



- CEU：

Use a **22-pin reverse FFC cable** to connect the development board’ s **CEU_CAM** connector to the camera adapter board’ s **DVP** connector.

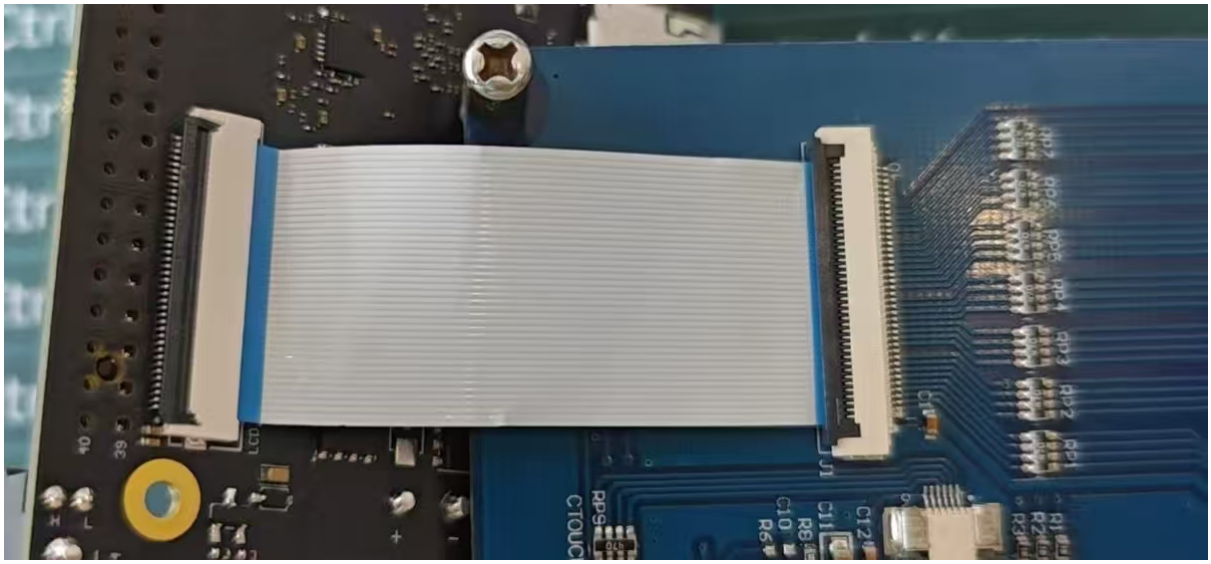


Q10. RGB LCD Connection

Solution:

Display model: **ALIENTEK 4.3" RGB LCD 800×480**

Use a **40-pin same-direction FFC cable** to connect the development board's **LCD_RGB** connector to the display.



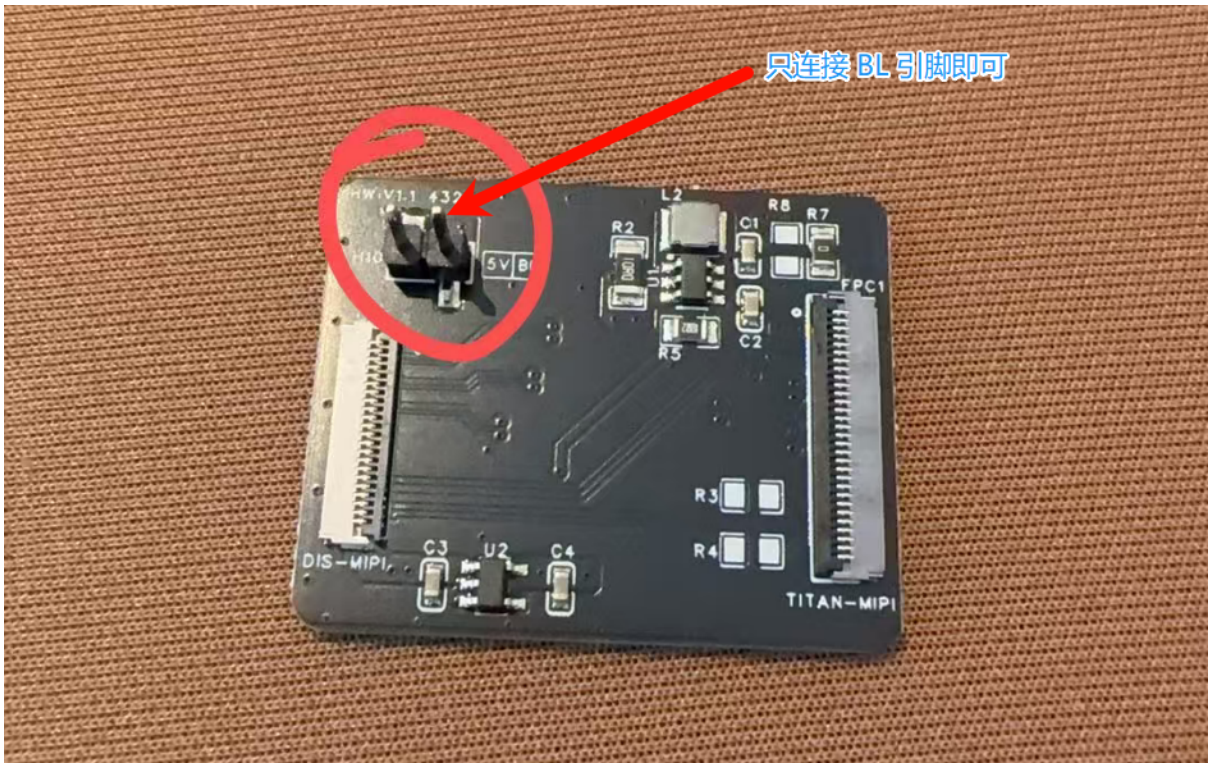
Q11. MIPI LCD Connection

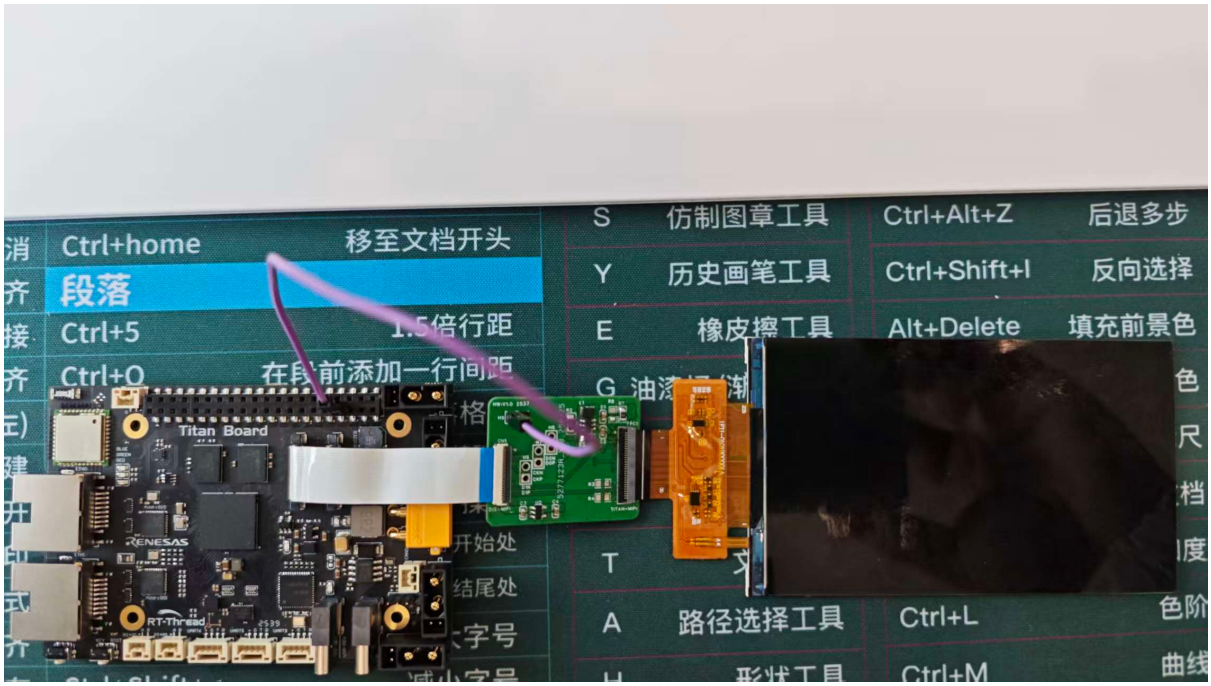
Solution:

Display model: **Guanxian TL043WVV02CT**

Use a **22-pin reverse FFC cable** to connect the development board’s **MIPI DSI/CSI** connector to the display adapter board’s **DIS-MIPI** connector. Then connect the **MIPI display** to the adapter board’s **TITAN-MIPI** connector.

A Dupont wire is required to connect the **BL** pin on the display adapter board to the **PB07** pin (the GPIO pin used to provide backlight can be changed as needed).





1.3 Runtime Issues

Q12. System freezes after running for a while

Solution:

Possible causes include:

- **Insufficient task stack space** → Increase stack size when creating threads.
- **Memory leaks** → Check heap usage.
- **Deadlock or priority inversion** → Review mutex and semaphore usage.

##Feedback & Support

If you encounter other issues, you can get help through the following channels:

- View project documentation: User Manual
- Submit an issue: GitHub Issues
- Contact technical support: RT-Thread Community Forum